SECTION 1

Introduction

The SAP-C02 Exam

The SAP-C02 Exam

Level: Professional

Length: 180 minutes

Format: 75 questions

Cost: $300 USD

Delivery Method: Testing center or online

Scoring:

- Scaled score between 100 – 1000
- Minimum passing score of 750

The SAP-C02 Exam

Question format:

- Multiple-choice: Has one correct response and three incorrect responses
- Multiple-response: Has two or more correct responses out of five or more options

The SAP-C02 Exam

Domain 1: Design Solutions for Organizational Complexity

- Task Statement 1.1: Architect network connectivity strategies
- Task Statement 1.2: Prescribe security controls
- Task Statement 1.3: Design reliable and resilient architectures
- Task Statement 1.4: Design a multi-account AWS environment
- Task Statement 1.5: Determine cost optimization and visibility strategies

The SAP-C02 Exam

Domain 2: Design for New Solutions

- Task Statement 2.1: Design a deployment strategy to meet business requirements
- Task Statement 2.2: Design a solution to ensure business continuity
- Task Statement 2.3: Determine security controls based on requirements
- Task Statement 2.4: Design a strategy to meet reliability requirements
- Task Statement 2.5: Design a solution to meet performance objectives
- Task Statement 2.6: Determine a cost optimization strategy to meet solution goals and objectives

The SAP-C02 Exam

Domain 3: Continuous Improvement for Existing Solutions

- Task Statement 3.1: Determine a strategy to improve overall operational excellence
- Task Statement 3.2: Determine a strategy to improve security
- Task Statement 3.3: Determine a strategy to improve performance
- Task Statement 3.4: Determine a strategy to improve reliability
- Task Statement 3.5: Identify opportunities for cost optimizations

The SAP-C02 Exam

Domain 4: Accelerate Workload Migration and Modernization

- Task Statement 4.1: Select existing workloads and processes for potential migration
- Task Statement 4.2: Determine the optimal migration approach for existing workloads
- Task Statement 4.3: Determine a new architecture for existing workloads
- Task Statement 4.4: Determine opportunities for modernization and enhancements

SECTION 2

AWS Accounts and Organizations

Create your AWS Free Tier Account

What you need...

Credit card for setting up the account and paying any bills

Unique email address for this account

Check if you can use a **dynamic alias** with an existing email address

john@gmail.com

john+ACCOUNT-ALIAS-1@gmail.com

john+ACCOUNT-ALIAS-2@gmail.com

AWS account name / alias

Phone to receive an **SMS** verification code

Configure Account and Create a Budget and Alarm

Account Configuration

- Configure **Account Alias**
- Enable access to billing for **IAM users**
- Update **billing preferences**
- Create a **budget and alarm**

Install Tools and Configure AWS CLI

Install Tools and Configure AWS CLI

- ✓ Download the code (from the *next* lesson)
- ✓ Install Visual Studio Code
- ✓ Install and Configure the AWS CLI
- ✓ Access AWS CloudShell

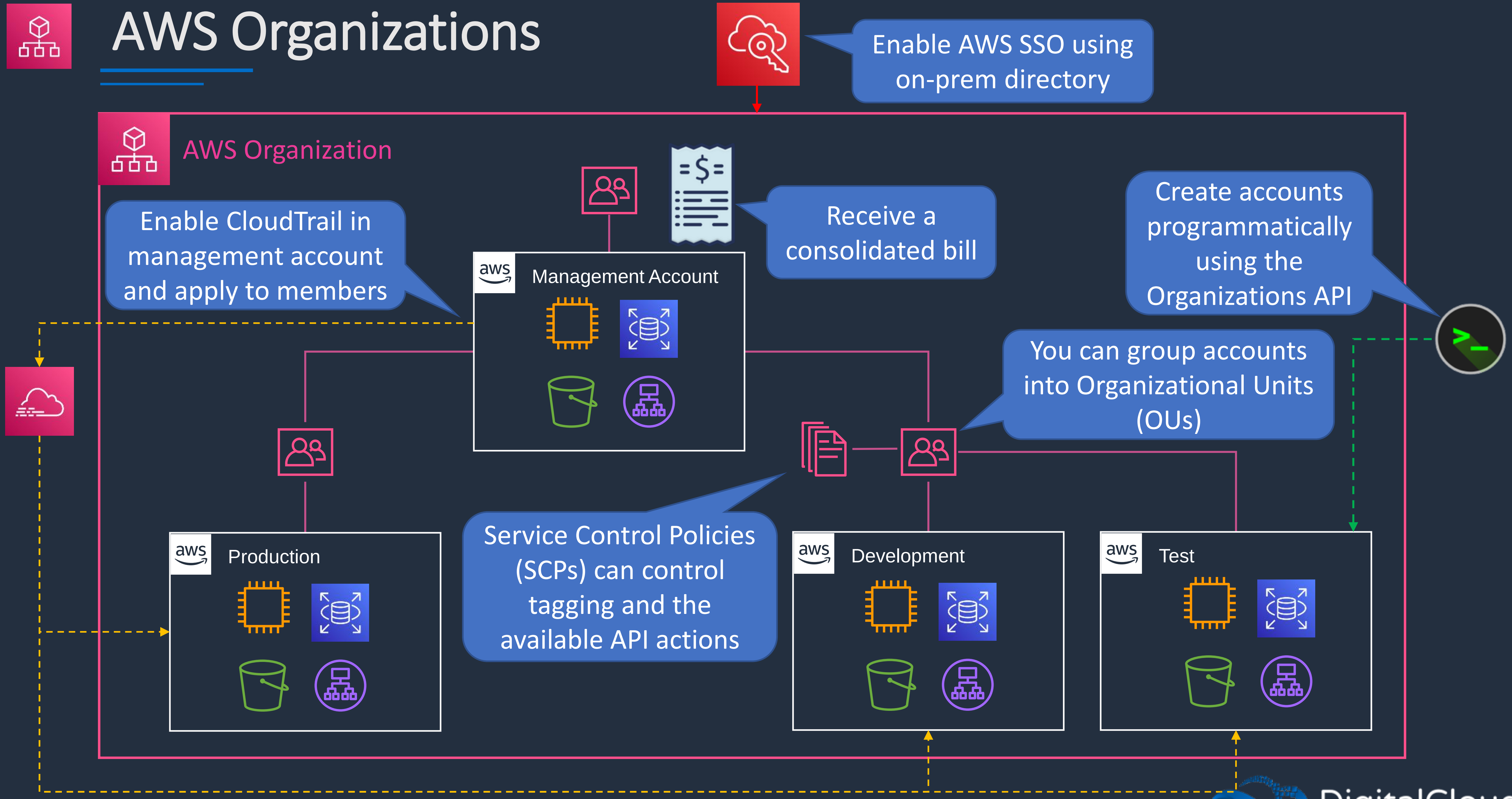
Setup Individual User Account

Root User vs IAM User

User	Login Details	Permissions
Root User	Email address	Full - Unrestricted
IAM User	Friendly name: **John** + AWS account ID or Alias	IAM Permissions Policy

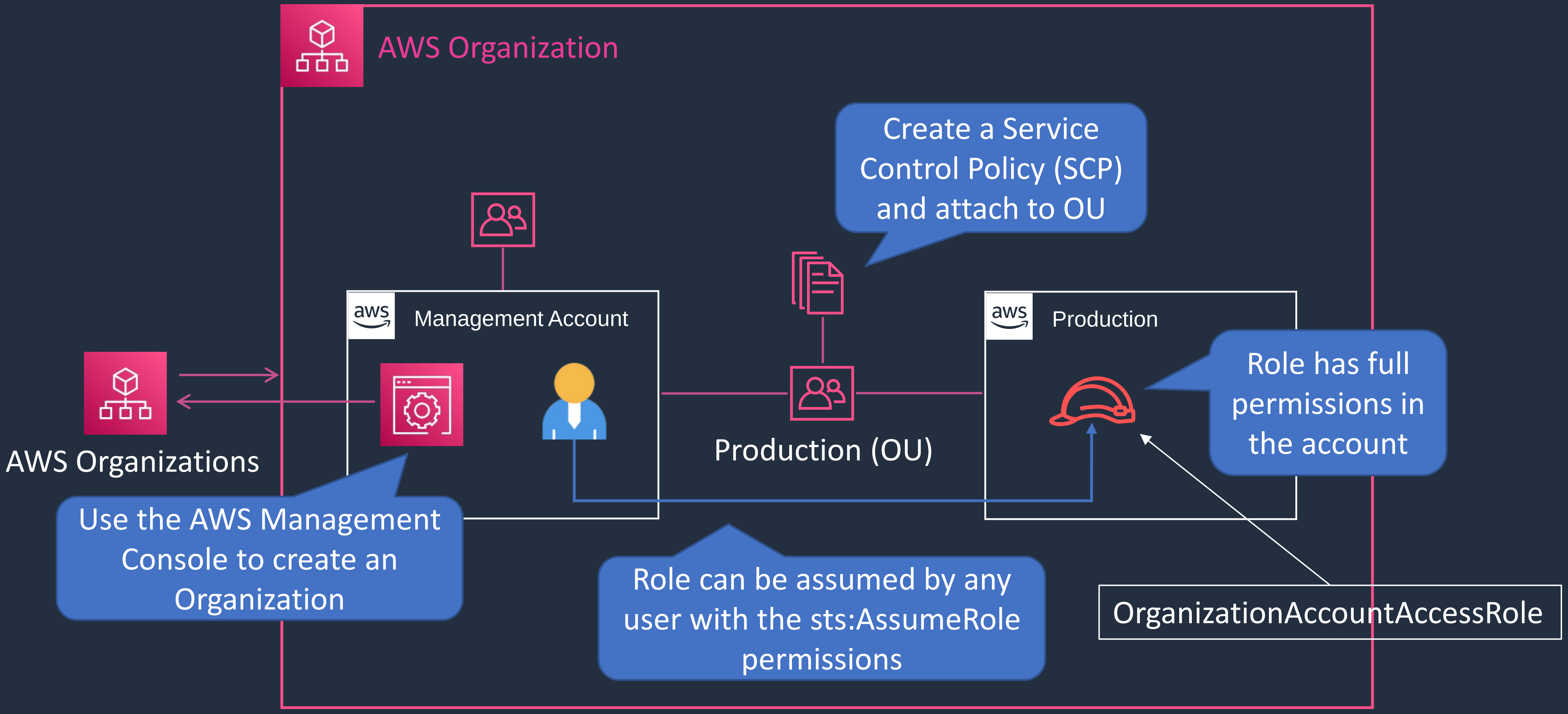
AWS Organizations

AWS Organizations



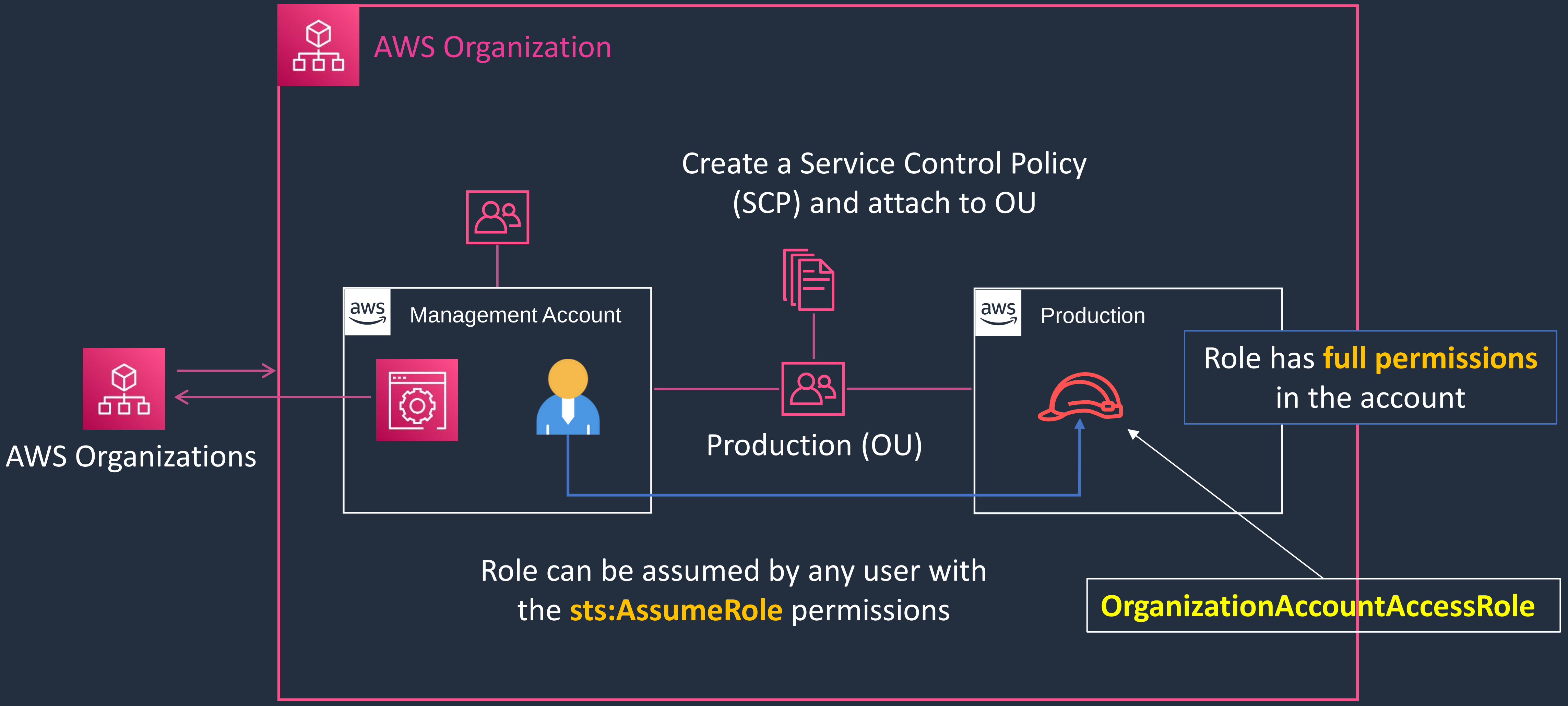
Account Configuration

Account Configuration



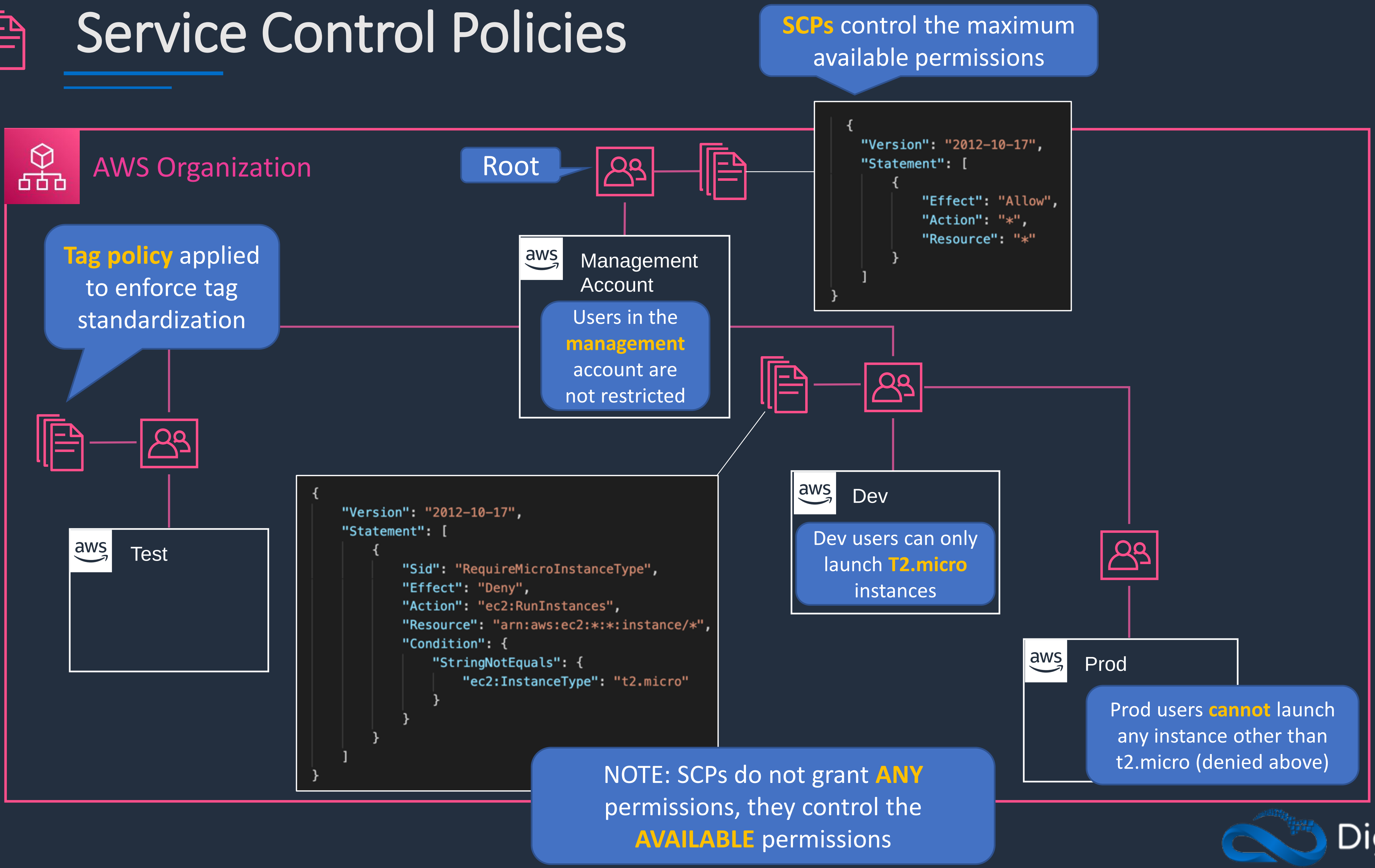
Create AWS Organization and Add Account

Account Configuration



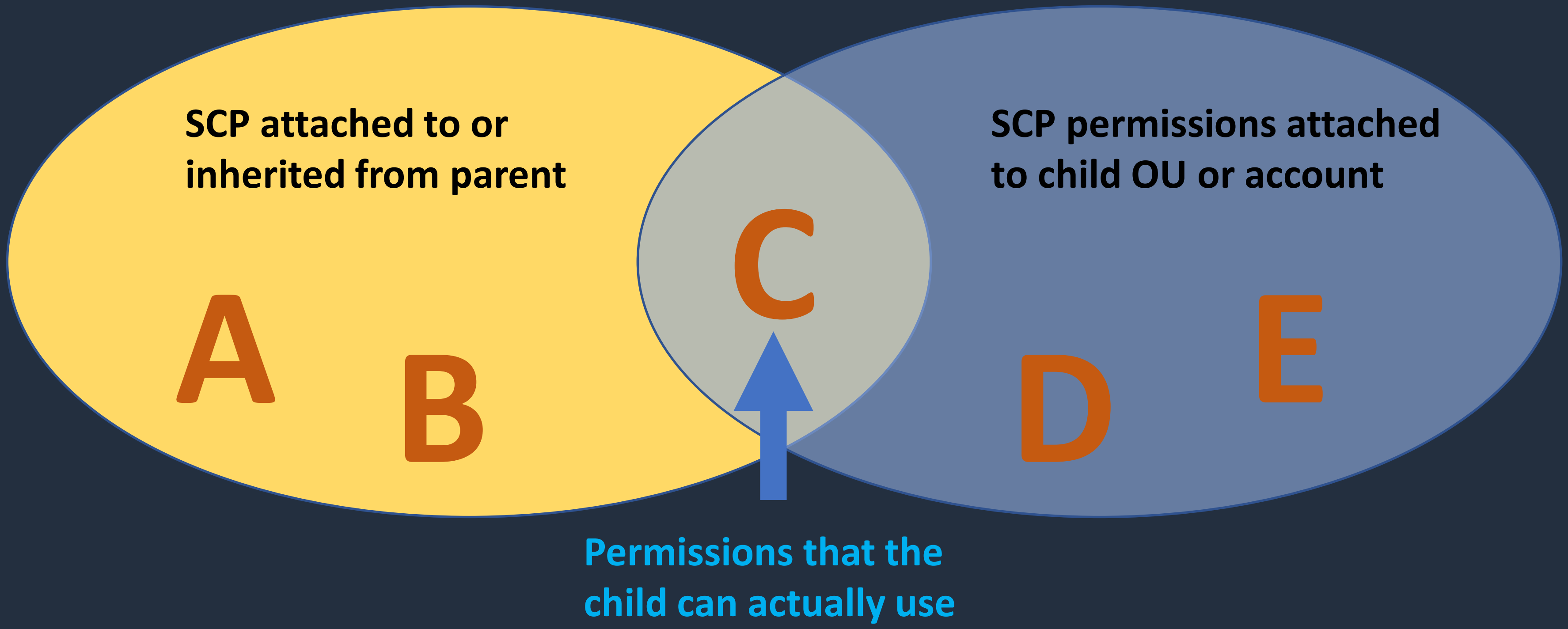
Service Control Policies (SCPs)

Service Control Policies



SCP Strategies and Inheritance

SCP Strategies and Inheritance



SCP Strategies and Inheritance

Deny List Strategy

- The FullAWSAccess SCP is attached to every OU and account
- Explicitly allows all permissions to flow down from the root
- Can explicitly override with a deny in an SCP
- This is the default setup

Note: An **explicit deny** overrides any kind of **allow**

Allow List Strategy

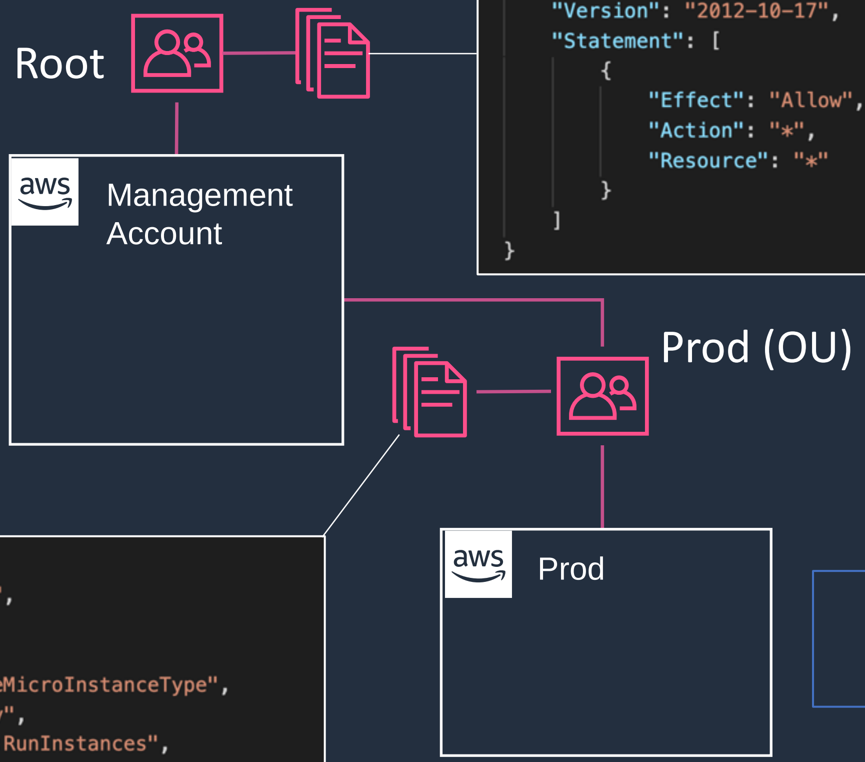
- The FullAWSAccess SCP is **removed** from every OU and account
- To allow a permission, SCPs with allow statements must be added to the account and every OU above it including root
- Every SCP in the hierarchy must explicitly allow the APIs you want to use

Note: An **explicit allow** overrides an **implicit deny**

DigitalCloud TRAINING

Test SCP Inheritance

Service Control Policies




```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

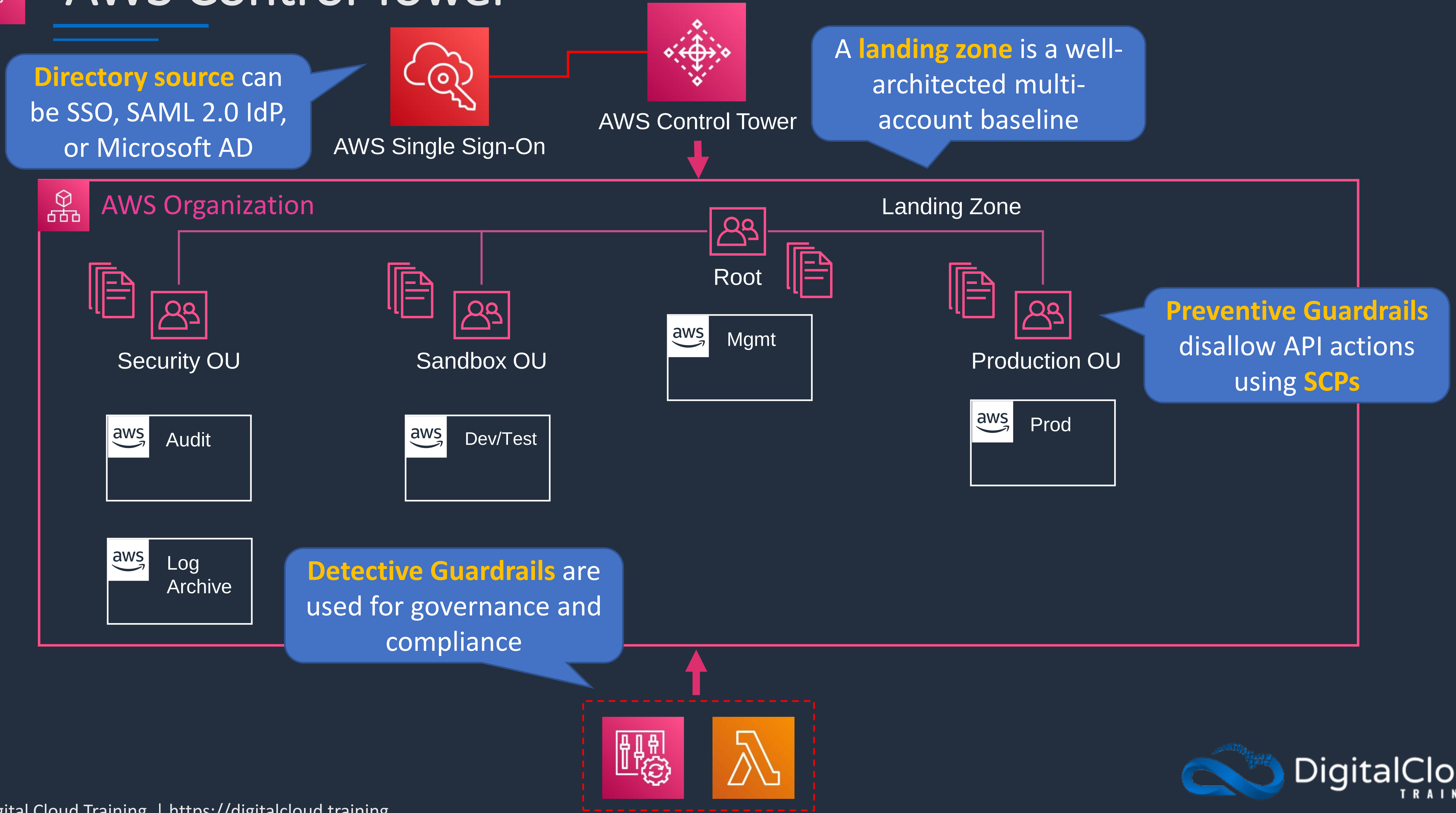
Users in the management account can launch **any instance type**

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequireMicroInstanceType",
      "Effect": "Deny",
      "Action": "ec2:RunInstances",
      "Resource": "arn:aws:ec2:*:*:instance/*",
      "Condition": {
        "StringNotEquals": {
          "ec2:InstanceType": "t2.micro"
        }
      }
    }
  ]
}
```

Prod users can only launch **t2.micro** instances

AWS Control Tower

AWS Control Tower



AWS Control Tower

- Control Tower creates a well-architected multi-account baseline based on best practices
- This is known as a landing zone
- Guardrails are used for governance and compliance:
  - **Preventive guardrails** are based on SCPs and disallow API actions
  - **Detective guardrails** are implemented using AWS Config rules and Lambda functions and monitor and govern compliance
- The root user in the management account can perform actions that guardrails would disallow

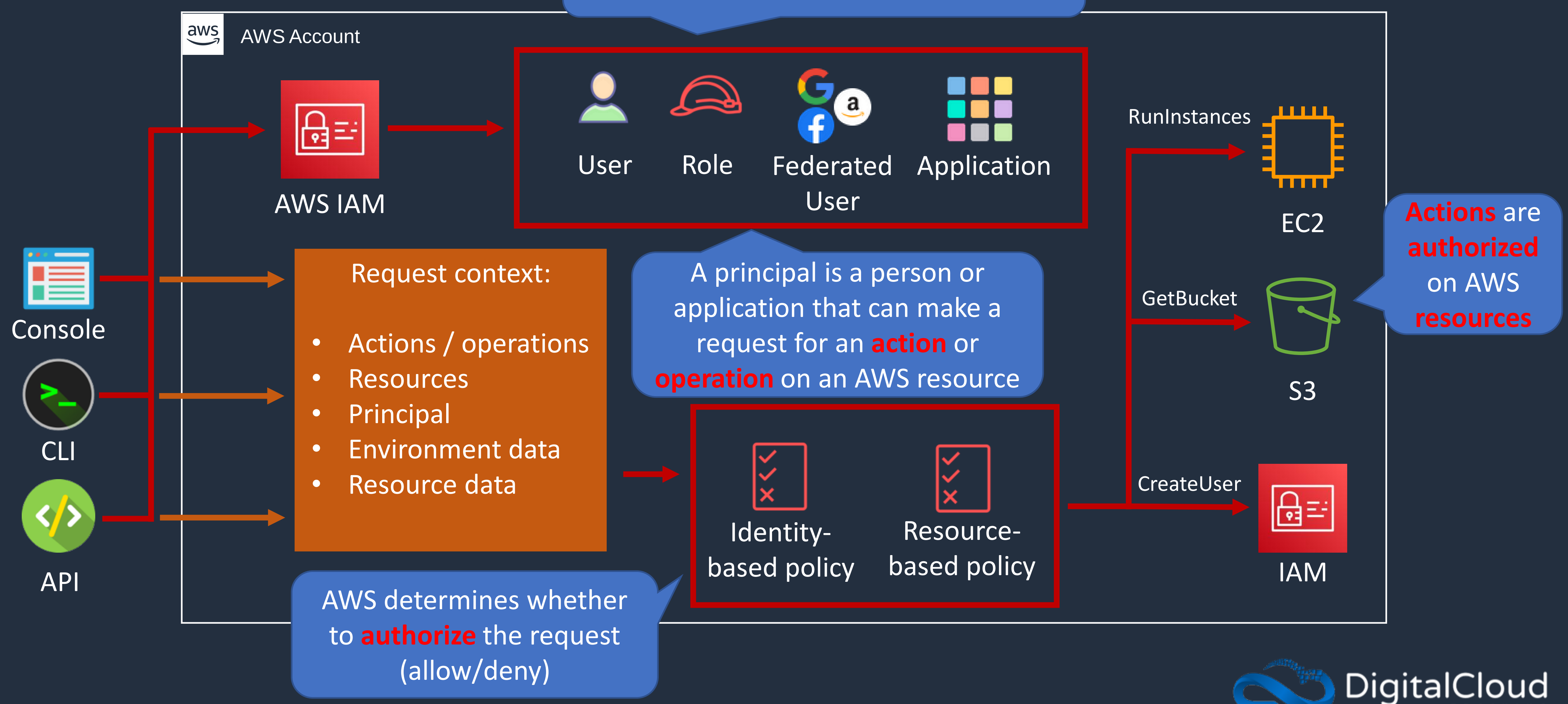
Create a Landing Zone

SECTION 3

Identity Management and Permissions

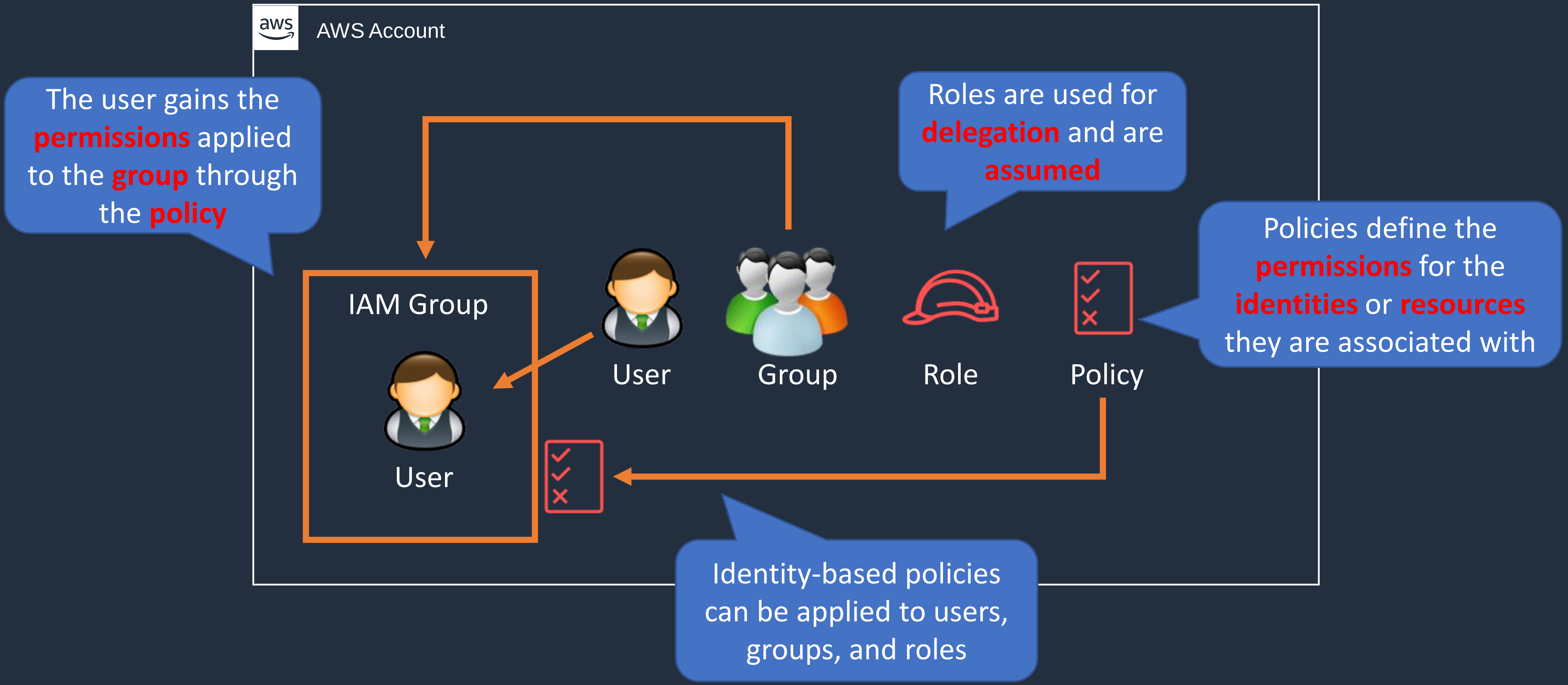
How IAM Works

How IAM Works



Overview of Users, Groups, Roles and Policies

Users, Groups, Roles and Policies



IAM Users

Account Root User

The root user has full permissions. It's a best practice to avoid using the root user account + enable **MFA**

Email used for signup

AWS IAM

Eric

Ethan

Andrea

Friendly name:
Andrea

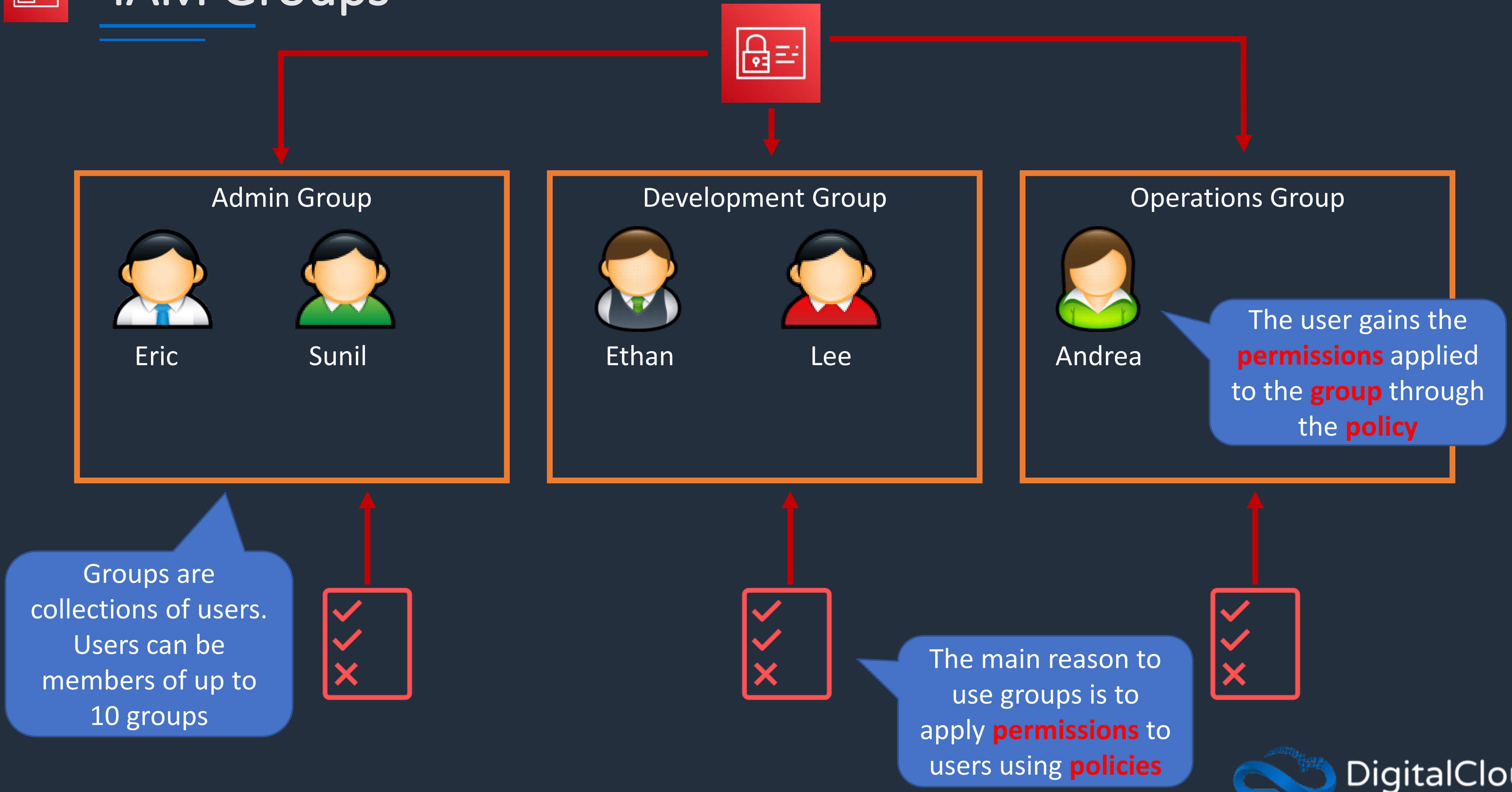
Amazon Resource Name:
arn:aws:iam::625148252389:user/Andrea

Up to 5000 individual user accounts can be created. Users have **no permissions** by default.

Authentication via **username/password** for console or **access keys** for API/CLI

DigitalCloud TRAINING

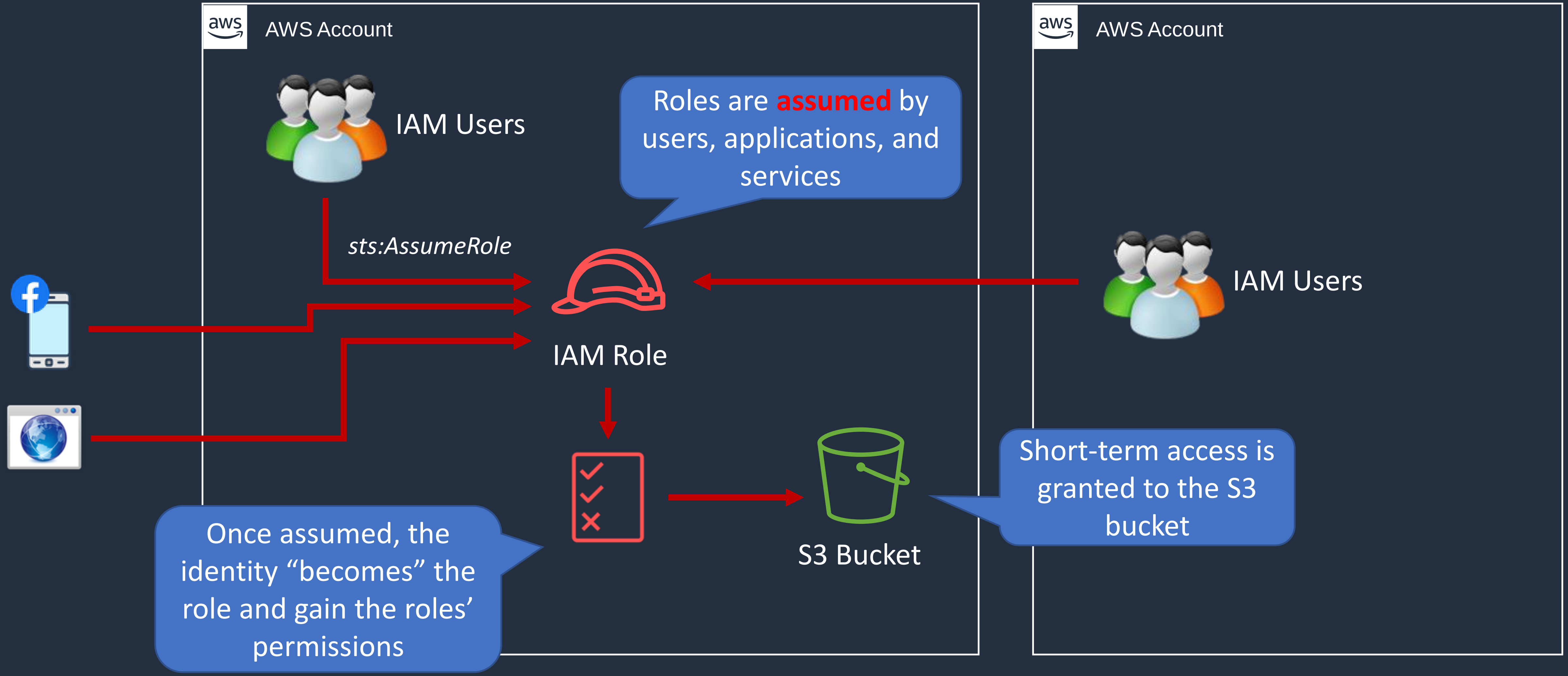
IAM Groups



Admin Group
Eric
Sunil
Development Group
Ethan
Lee
Operations Group
Andrea
The user gains the permissions applied to the group through the policy
Groups are collections of users. Users can be members of up to 10 groups
The main reason to use groups is to apply permissions to users using policies

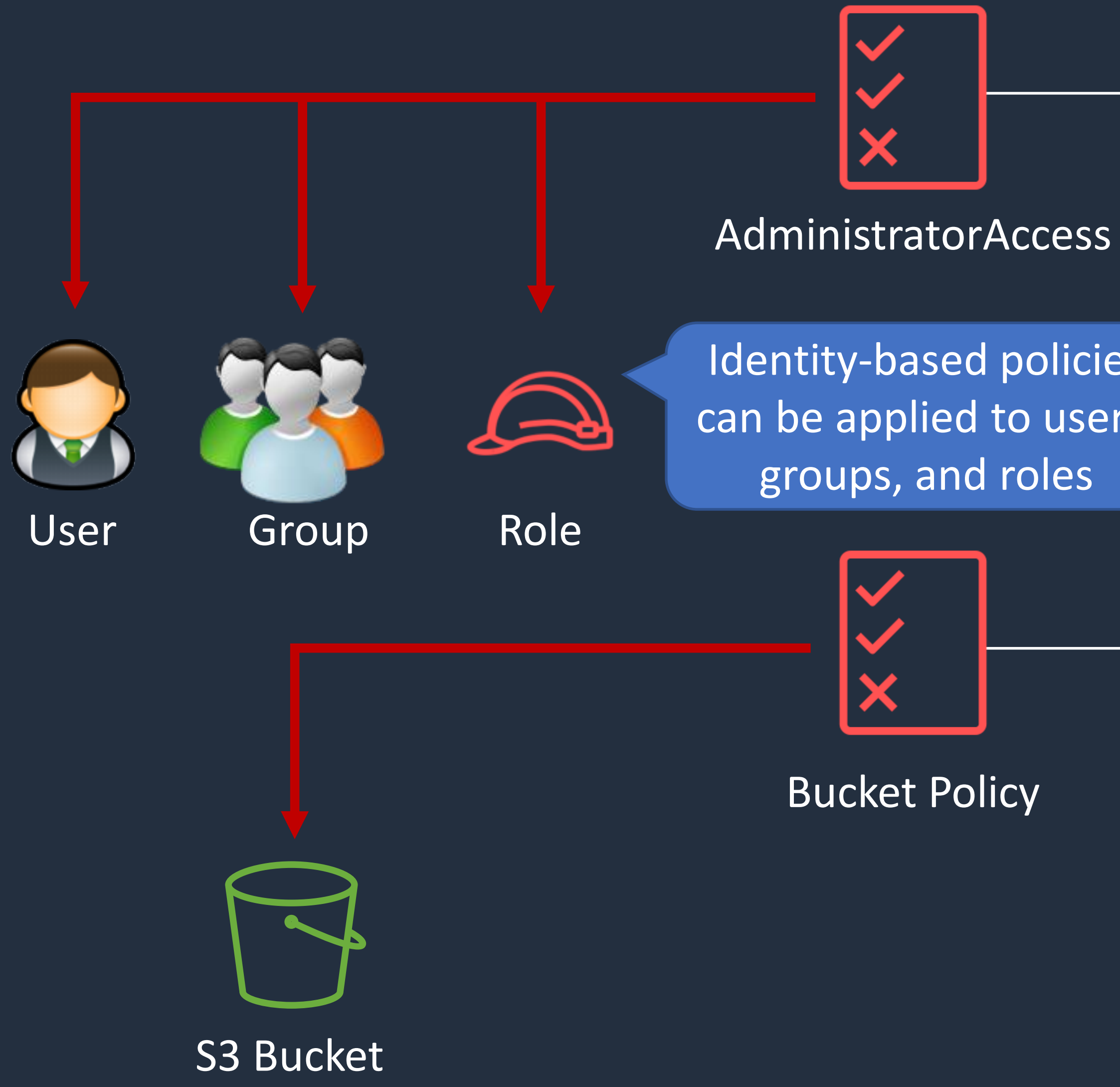
IAM Roles

An IAM role is an IAM identity that that has specific permissions



IAM Policies

Policies are documents that define permissions and are written in JSON




```json
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "*",
            "Resource": "*"
        }
    ]
}
```

All permissions are implicitly denied by default

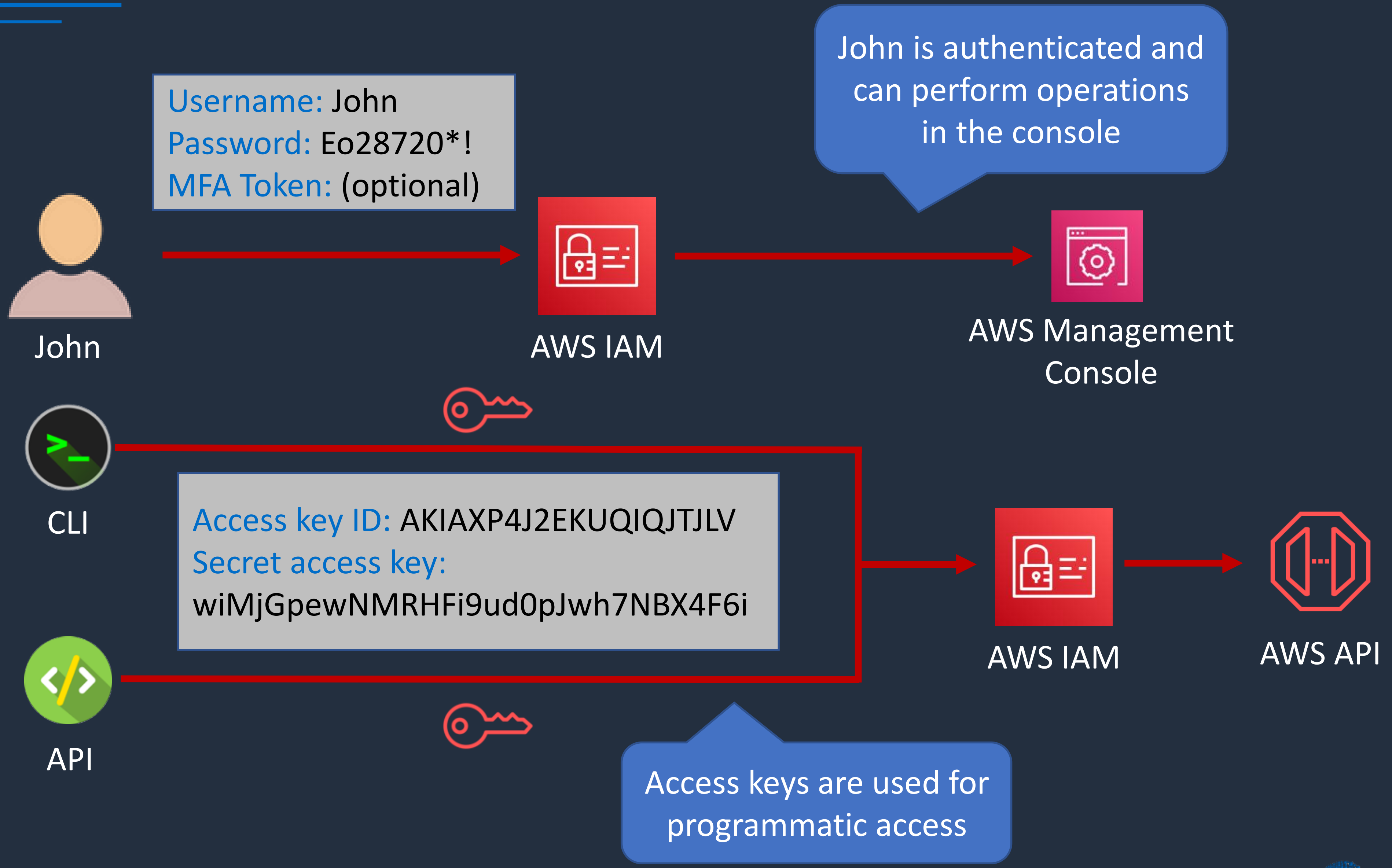
Identity-based policies can be applied to users, groups, and roles

```json
{
    "Version": "2012-10-17",
    "Id": "Policy1561964929358",
    "Statement":[
    {
        "Sid": "Stmt1561964454052",
        "Effect": "Allow",
        "Principal": {
            "AWS": "arn:aws:iam::515148227241:user/Paul"
        },
        "Action": "s3:*",
        "Resource": "arn:aws:s3:::dctcompany",
        "Condition": {
            "StringLike": {
                "s3:prefix": "Confidential/*"
            }
        }
    }
    ]
}
```

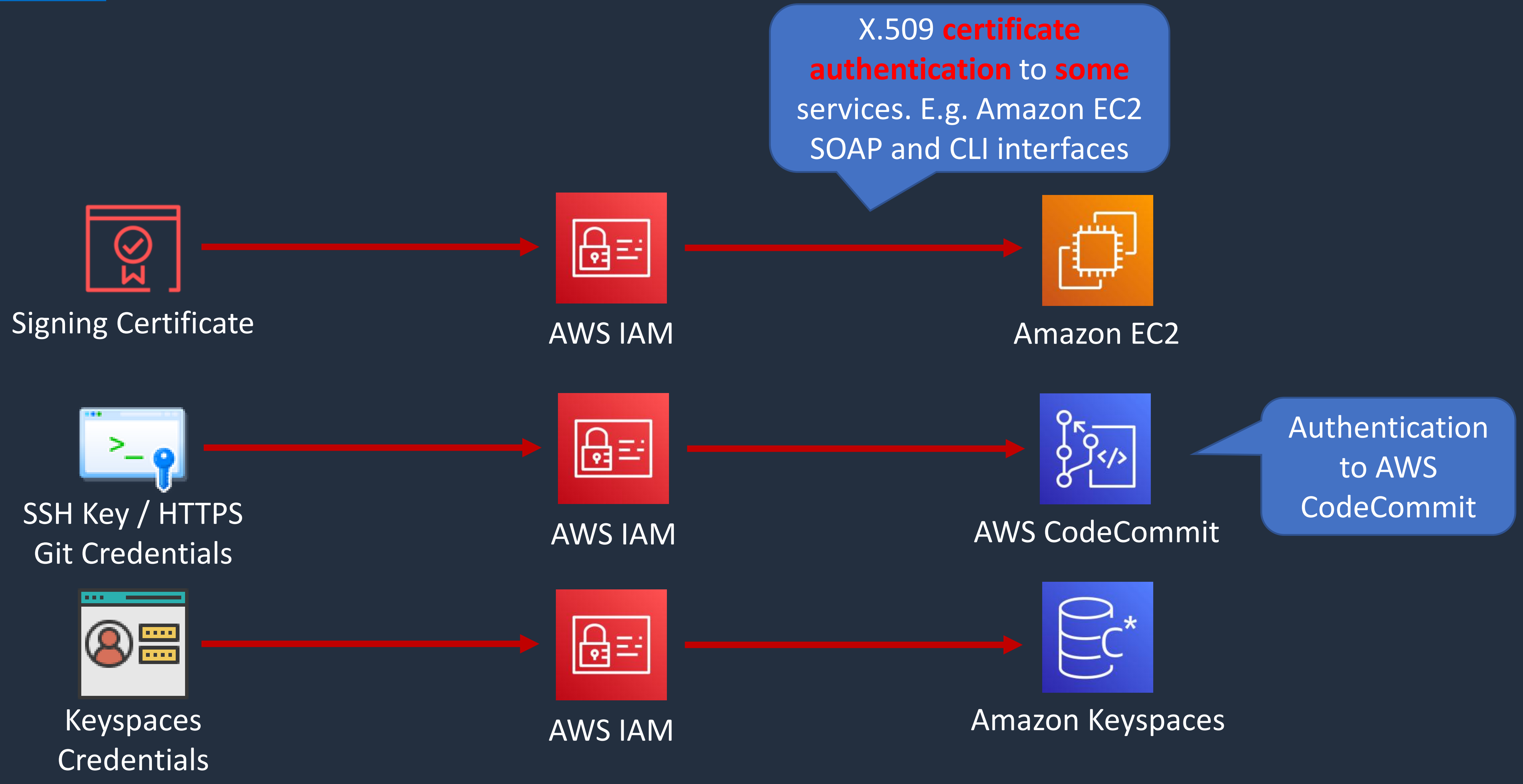
Resource-based policies apply to resources such as S3 buckets or DynamoDB tables

IAM Authentication Methods

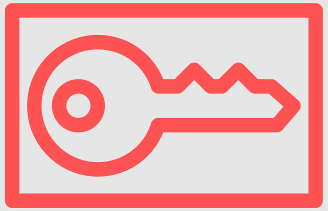
IAM Authentication Methods



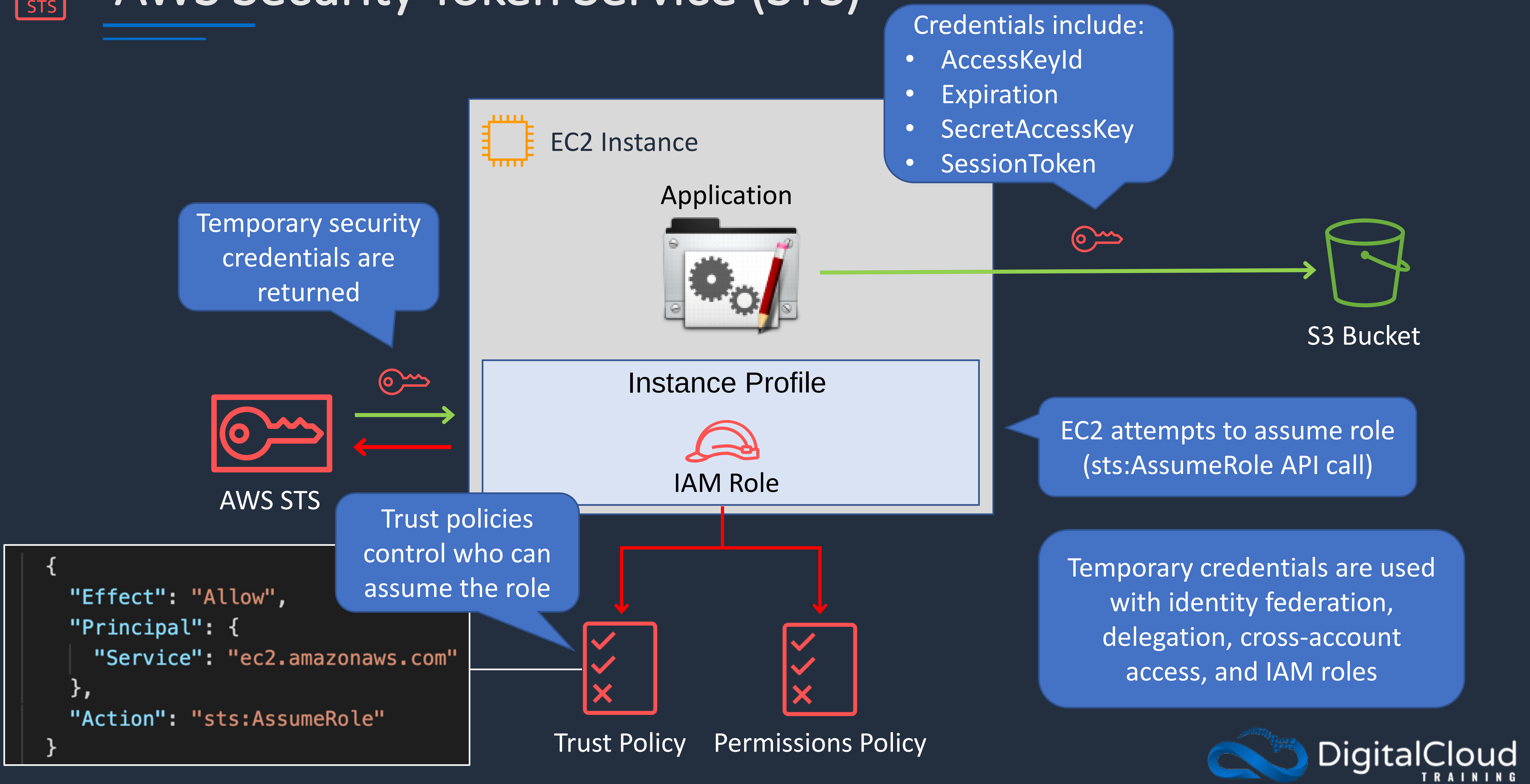
IAM Authentication Methods



AWS Security Token Service (STS)



AWS Security Token Service (STS)




```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "ec2.amazonaws.com"
  },
  "Action": "sts:AssumeRole"
}
```

Multi-Factor Authentication (MFA)

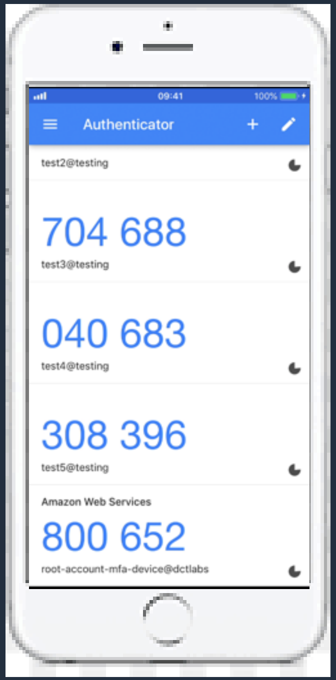
Multi-Factor Authentication

Something you know:

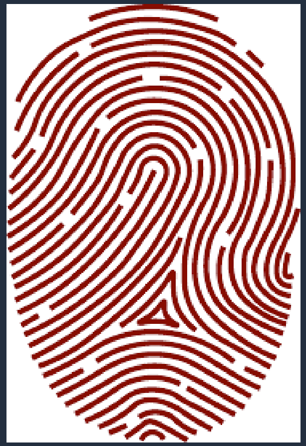
EJPx!*21p9%

Password

Something you have:



Something you are:



DigitalCloud TRAINING

Multi-Factor Authentication

Something you know:

IAM User

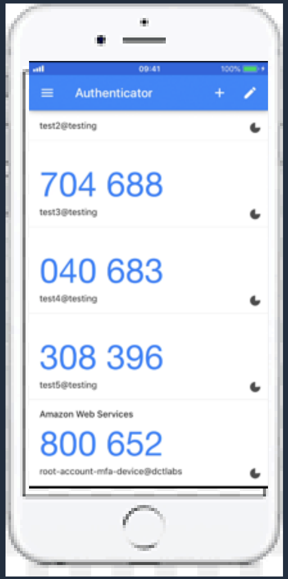
EJPx!*21p9%

Password

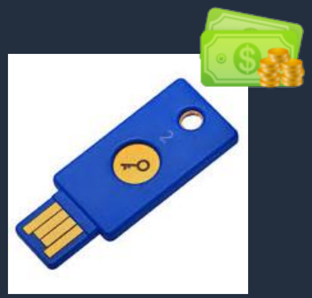
Something you have:

MFA Virtual MFA

e.g. Google Authenticator on your smart phone



MFA Physical MFA



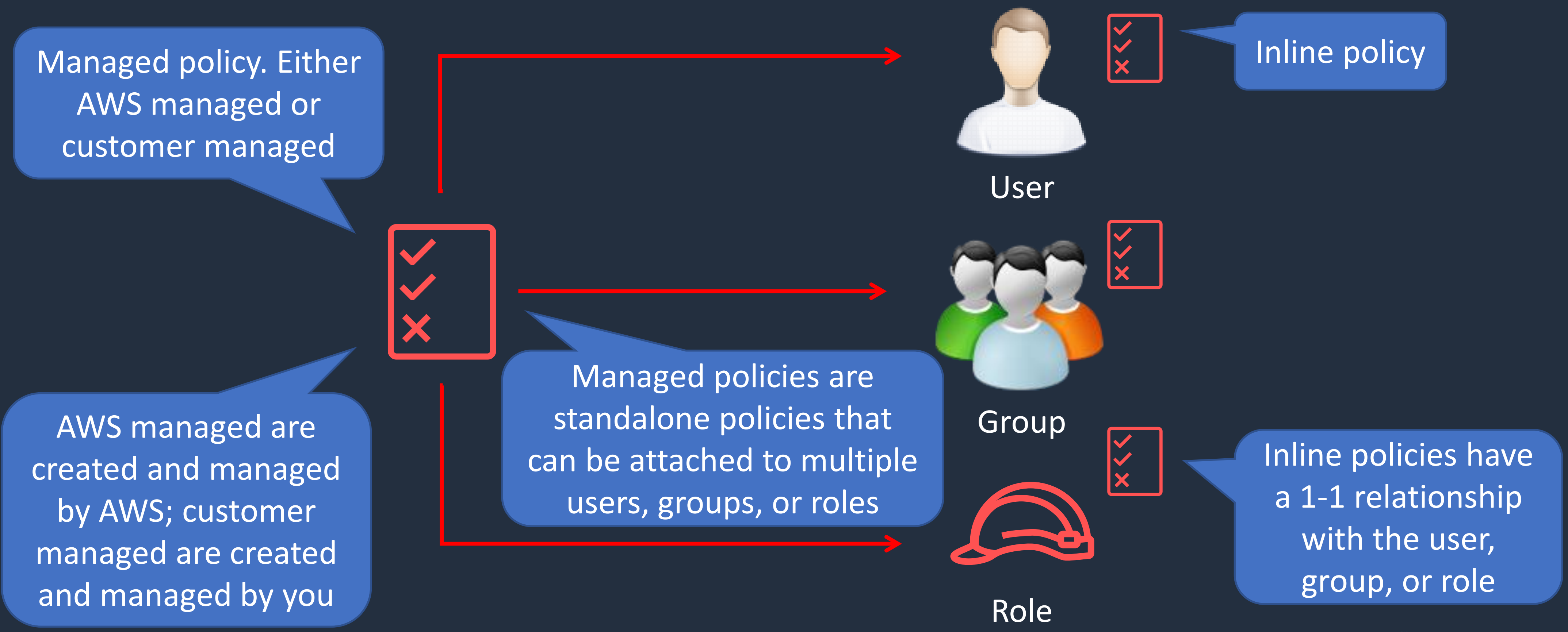
Setup Multi-Factor Authentication (MFA)

DigitalCloud TRAINING

Identity-Based Policies and Resource-Based Policies

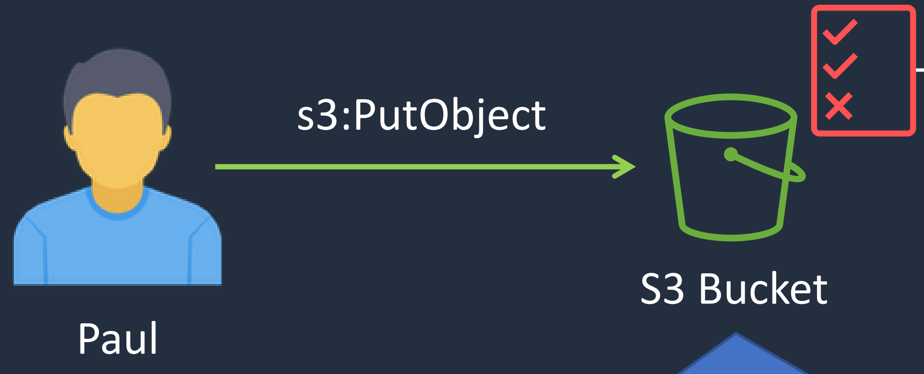
Identity-Based IAM Policies

Identity-based policies are JSON permissions policy documents that control what actions an identity can perform, on which resources, and under what conditions



Resource-Based Policies

Resource-based policies are JSON policy documents that you attach to a resource such as an Amazon S3 bucket

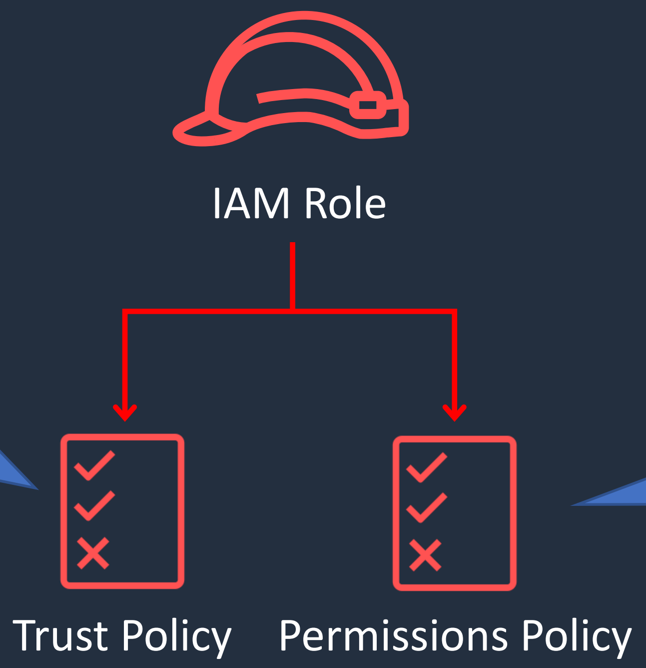


Resource-based policies grant the specified **principal** (Paul) **permission** to perform specific **actions** on the **resource**

```
{
    "Version": "2012-10-17",
    "Id": "Policy1561964929358",
    "Statement":[
    {
        "Sid": "Stmt1561964454052",
        "Effect": "Allow",
        "Principal": {
            "AWS": "arn:aws:iam::515148227241:user/Paul"
        },
        "Action": "s3:*",
        "Resource": "arn:aws:s3:::dctcompany"
    }
    ]
}
```

Resource-Based Policies

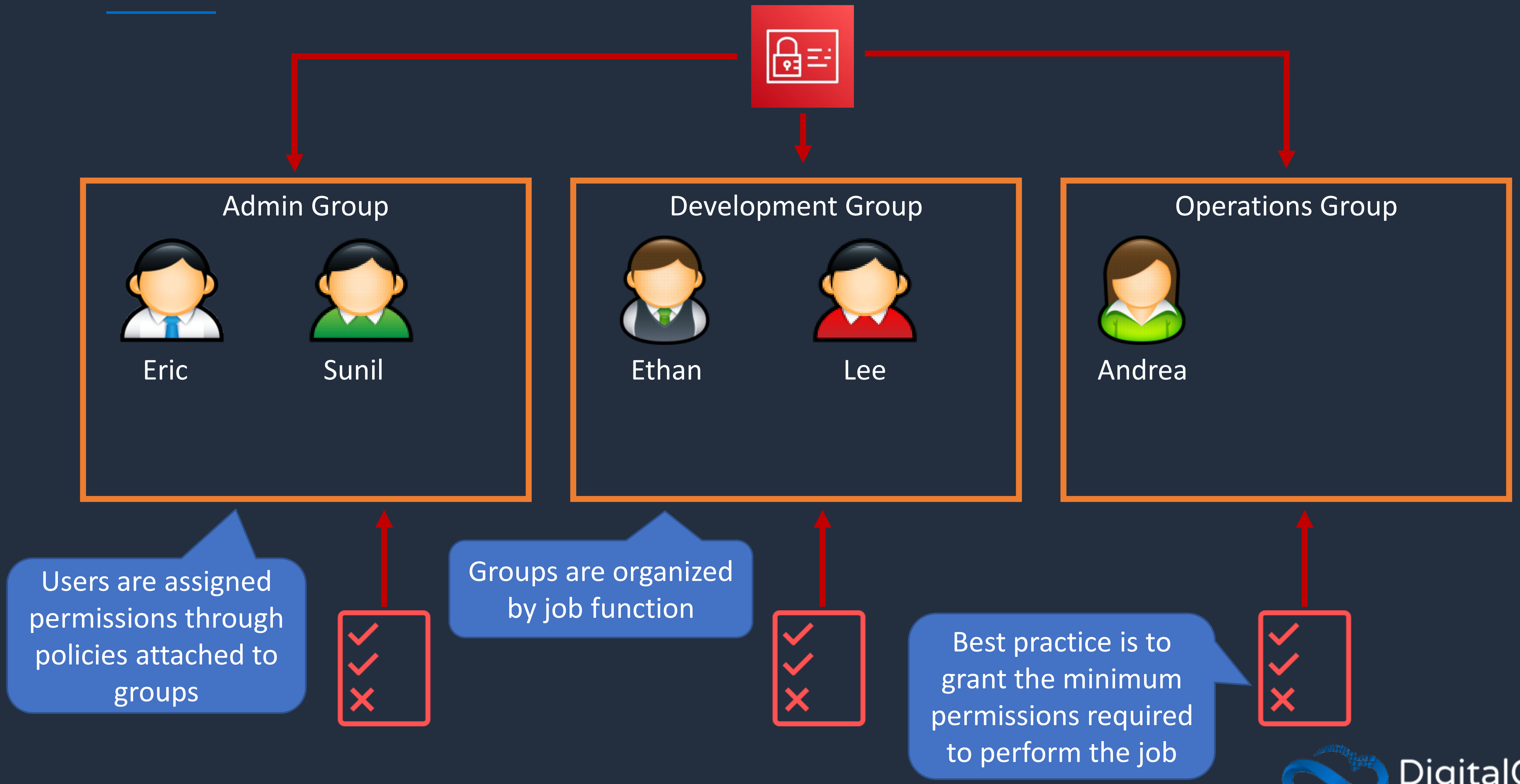
A trust policy is also an example of a **resource-based** policy



Whereas a permissions policy is an **identity-based** policy

Access Control Methods - RBAC & ABAC

Role-Based Access Control (RBAC)

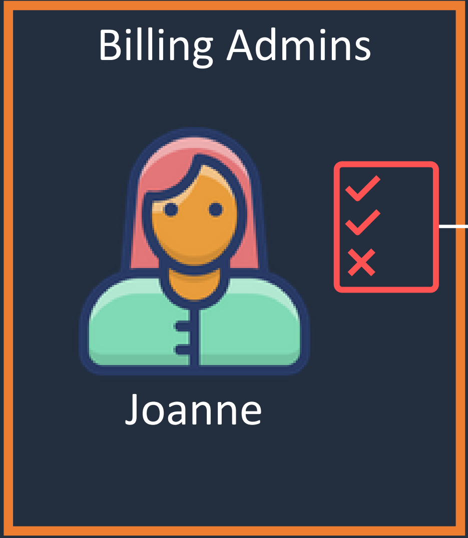


Role-Based Access Control (RBAC)

Job function policies:

- Administrator
- Billing
- Database administrator
- Data scientist
- Developer power user
- Network administrator
- Security auditor
- Support user
- System administrator
- View-only user

The Billing managed policy is attached to the group

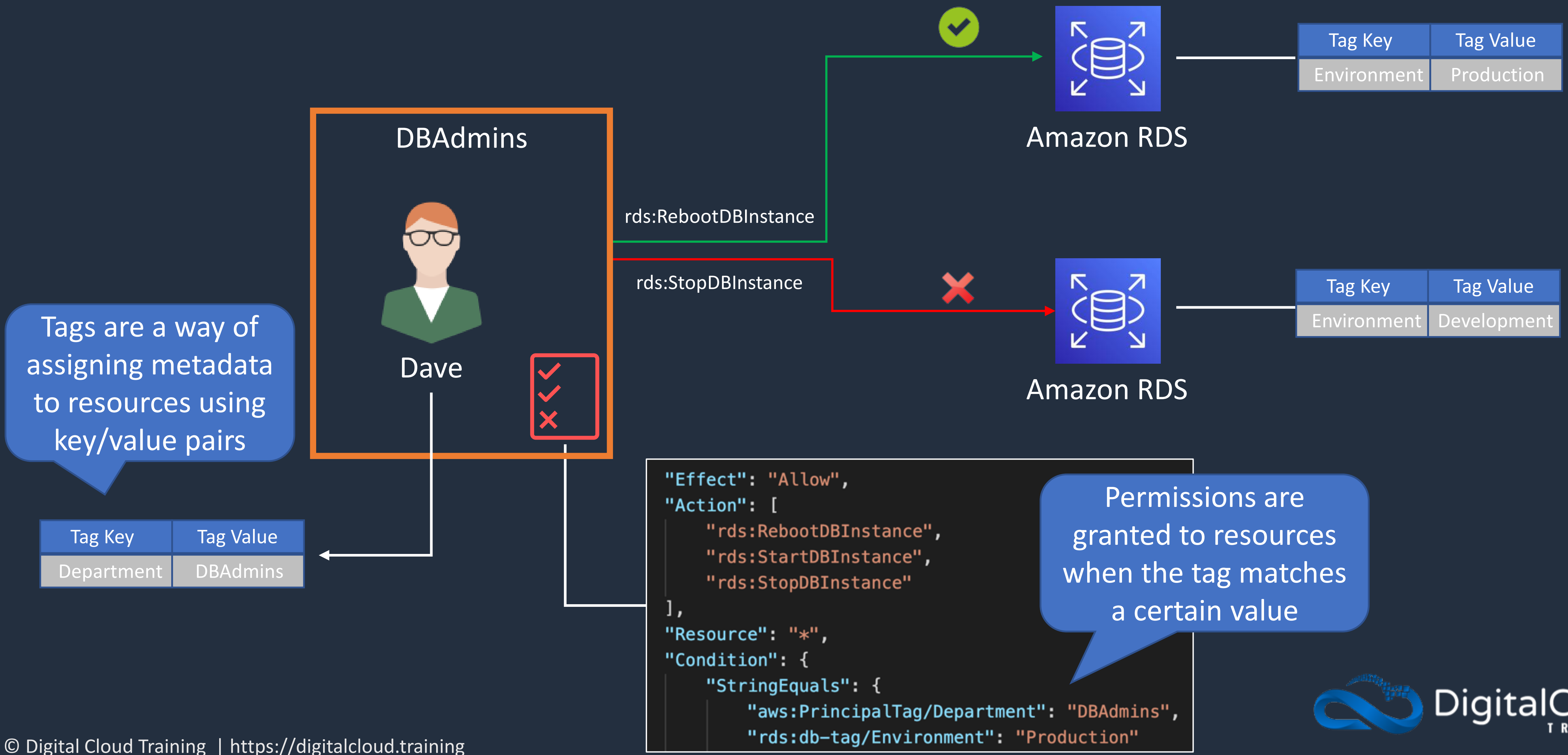



```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "aws-portal:*Billing",
                "aws-portal:*Usage",
                "aws-portal:*PaymentMethods",
                "budgets:ViewBudget",
                "budgets:ModifyBudget",
                "ce:UpdatePreferences",
                "ce:CreateReport",
                "ce:UpdateReport",
                "ce:DeleteReport",
                "ce:CreateNotificationSubscription",
                "ce:UpdateNotificationSubscription",
                "ce:DeleteNotificationSubscription",
                "cur:DescribeReportDefinitions",
                "cur:PutReportDefinition",
                "cur:ModifyReportDefinition",
                "cur:DeleteReportDefinition",
                "purchase-orders:*PurchaseOrders"
            ],
            "Resource": "*"
        }
    ]
}
```

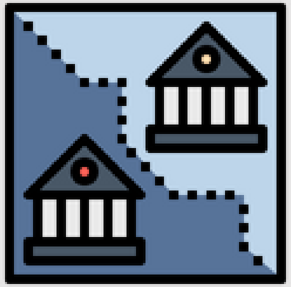
AWS managed policies for job functions are designed to closely align to common job functions in the IT industry

DigitalCloud TRAINING

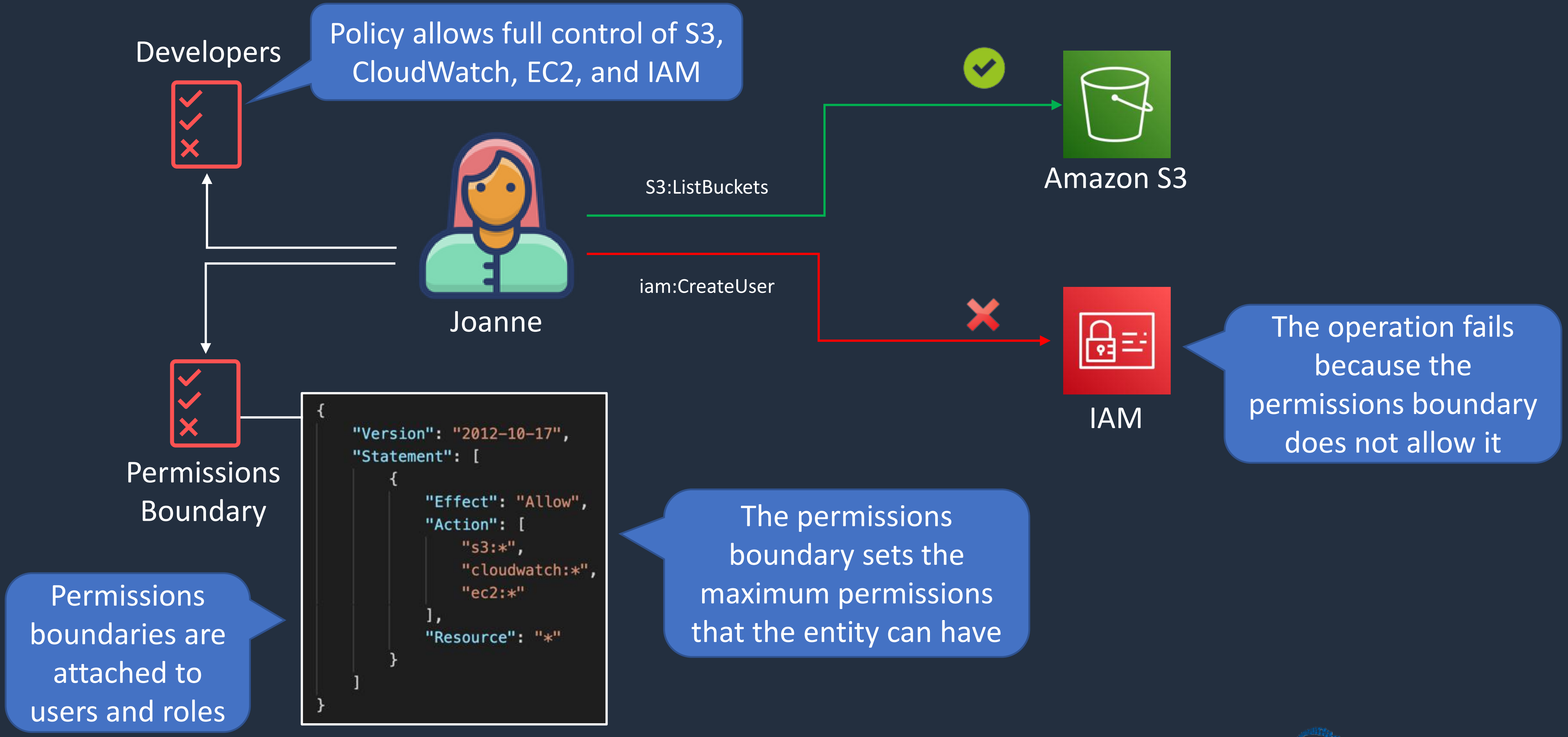
Attribute-Based Access Control (ABAC)



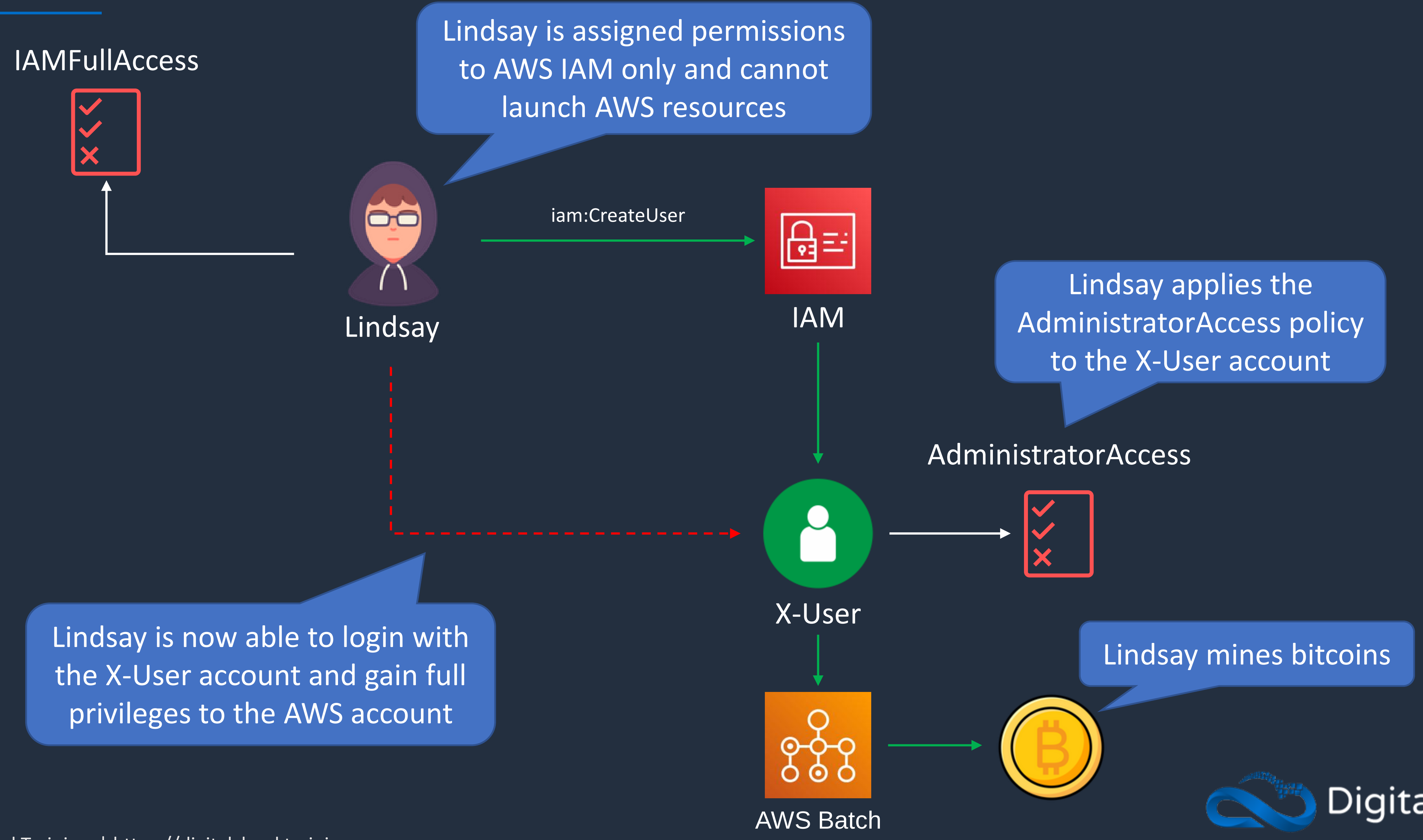
Permissions Boundaries



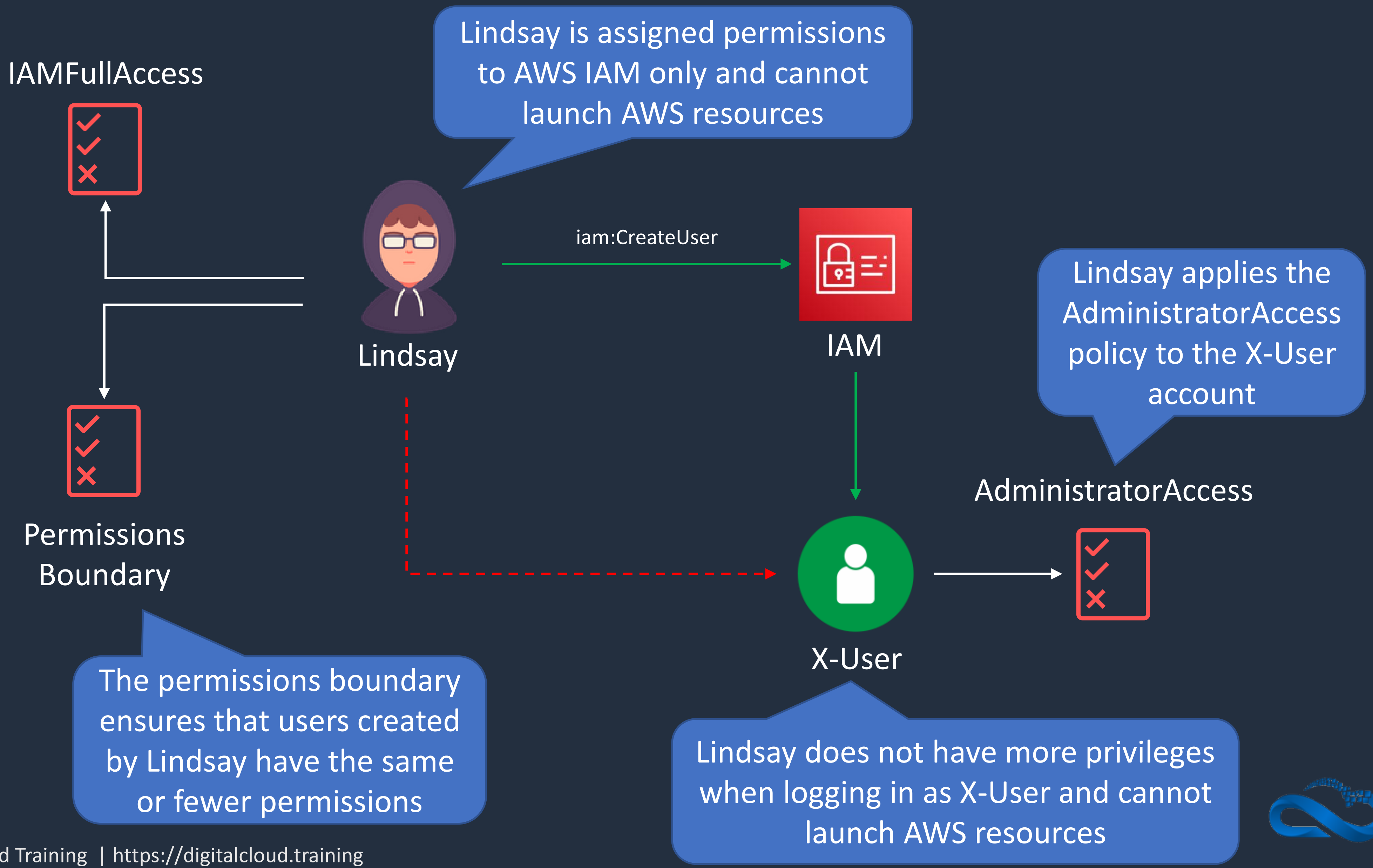
Permissions Boundaries



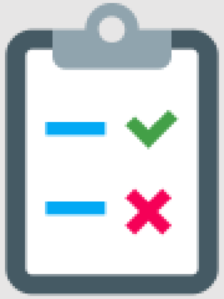
Privilege Escalation



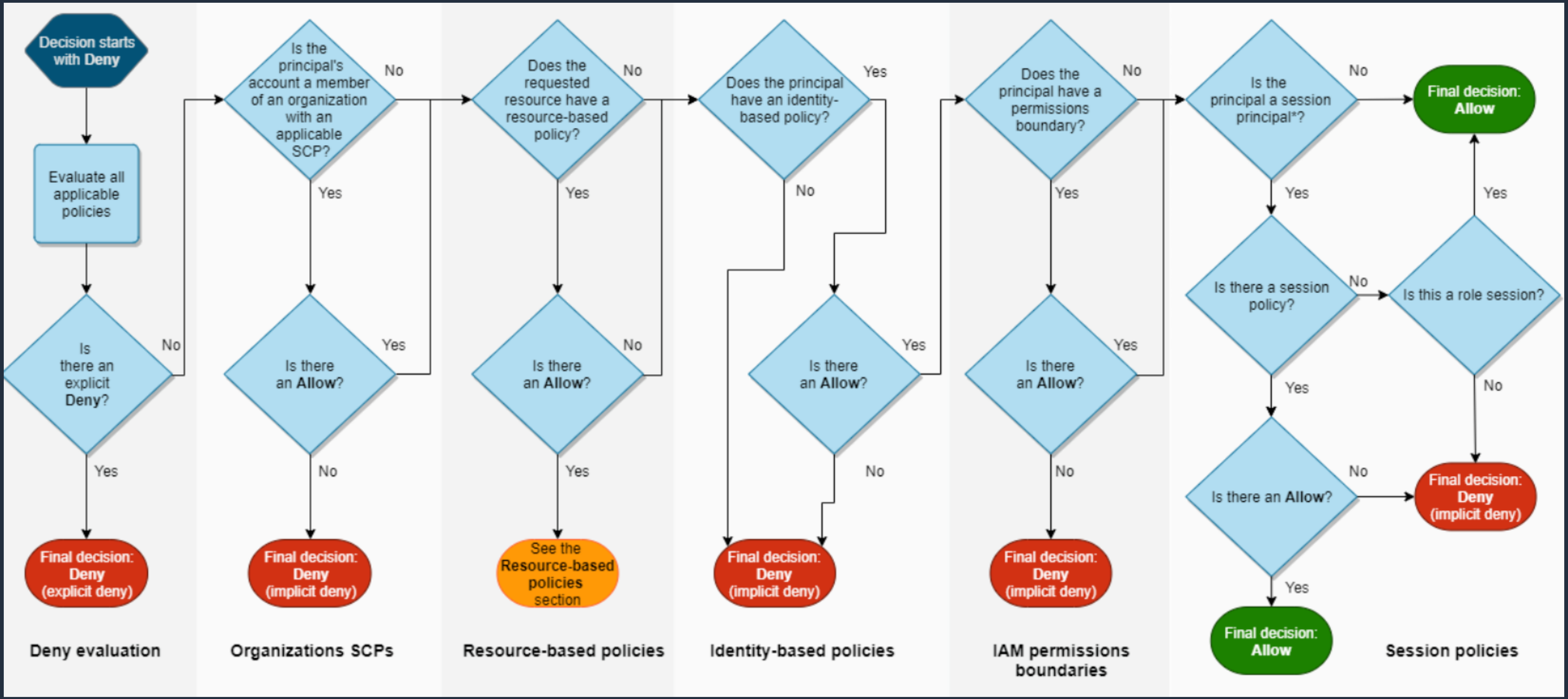
Preventing Privilege Escalation



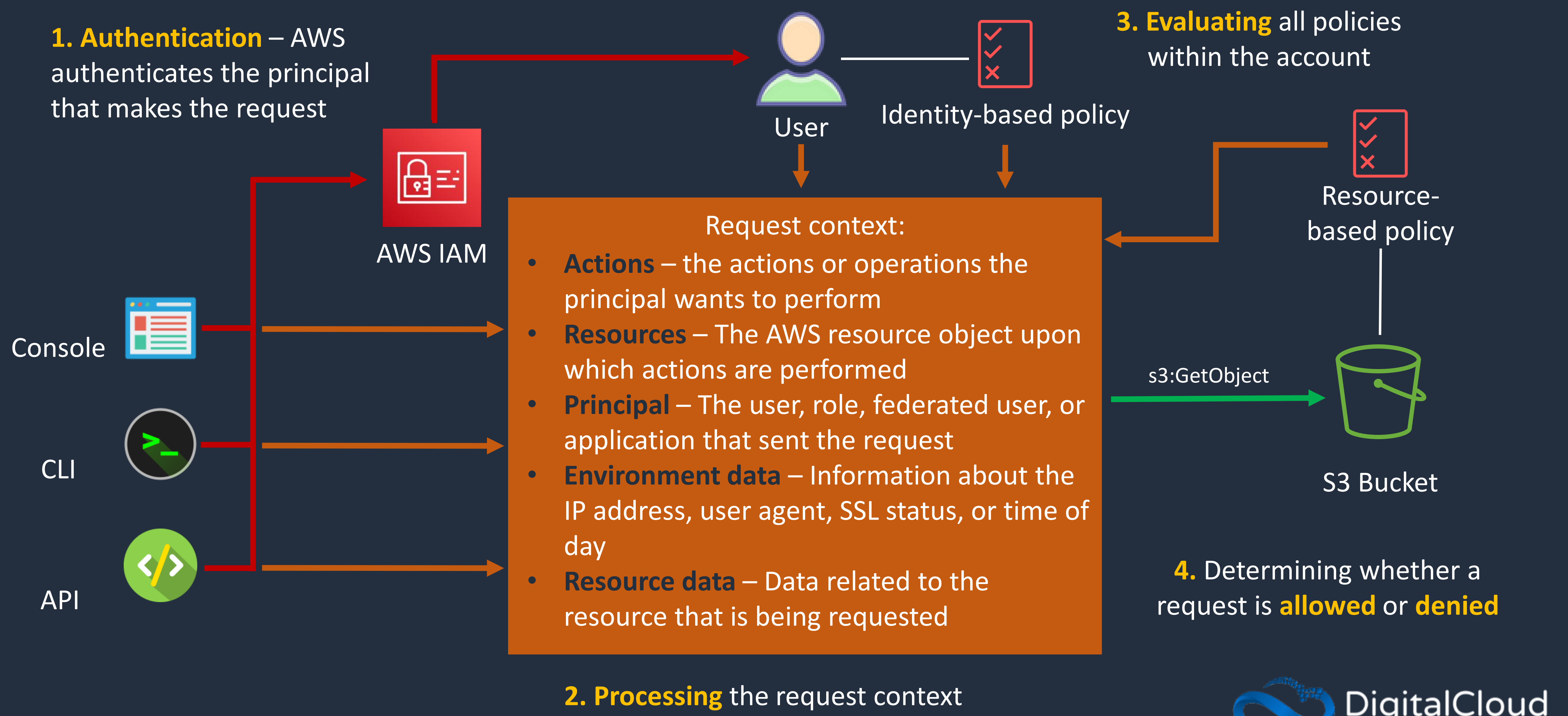
IAM Policy Evaluation



Evaluation Logic



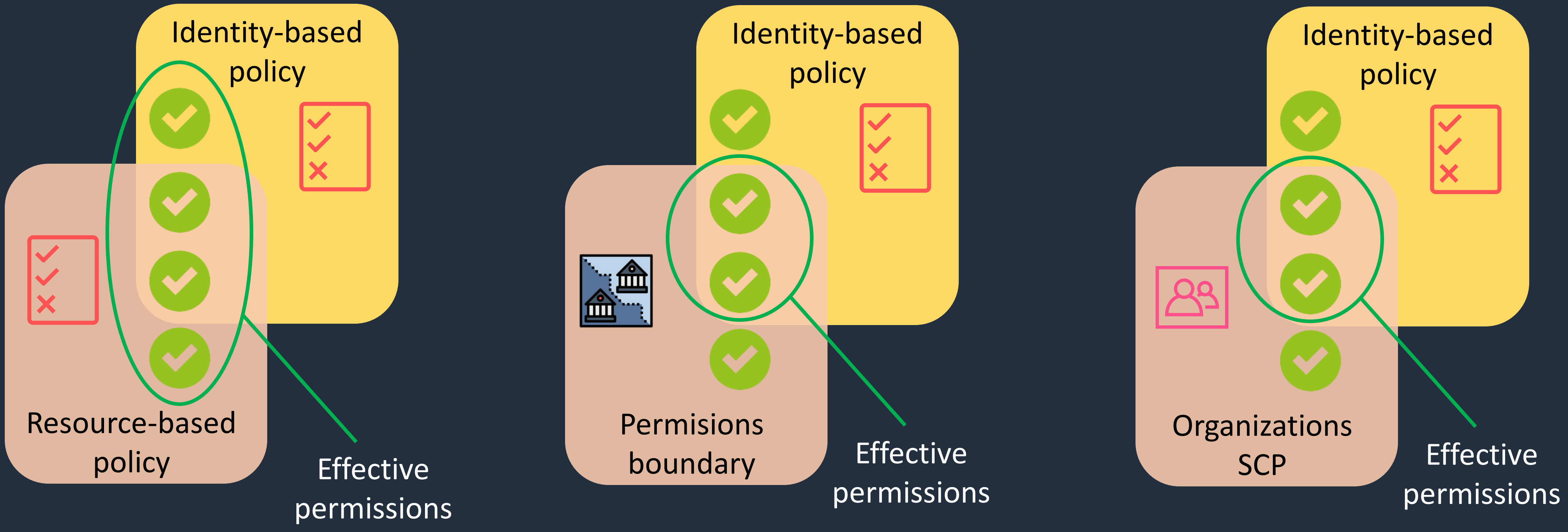
Steps for Authorizing Requests to AWS



Types of Policy

- **Identity-based policies** – attached to users, groups, or roles
- **Resource-based policies** – attached to a resource; define permissions for a principal accessing the resource
- **IAM permissions boundaries** – set the maximum permissions an identity-based policy can grant an IAM entity
- **AWS Organizations service control policies (SCP)** – specify the maximum permissions for an organization or OU
- **Session policies** – used with AssumeRole* API actions

Evaluating Policies within an AWS Account



Determination Rules

1. By default, all requests are implicitly denied (though the root user has full access)
2. An explicit allow in an identity-based or resource-based policy overrides this default
3. If a permissions boundary, Organizations SCP, or session policy is present, it might override the allow with an implicit deny
4. An explicit deny in any policy overrides any allows

IAM Policy Structure

IAM Policy Structure

An IAM policy is a JSON document that consists of one or more statements

```
{
    "Statement":[{
      "Effect":"effect",
      "Action":"action",
      "Resource":"arn",
      "Condition":{
        "condition":{
          "key":"value"
          }
        }
      }
    ]
}
```

The **Effect** element can be Allow or Deny

The **Action** element is the specific API action for which you are granting or denying permission

The **Resource** element specifies the resource that's affected by the action

The **Condition** element is optional and can be used to control when your policy is in effect

DigitalCloud TRAINING

IAM Policy Example 1

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "*",
            "Resource": "*"
        }
    ]
}
```

The AdministratorAccess policy uses wildcards (*) to allow all actions on all resources

IAM Policy Example 2

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["ec2:TerminateInstances"],
            "Resource": ["*"]
        },
        {
            "Effect": "Deny",
            "Action": ["ec2:TerminateInstances"],
            "Condition": {
                "NotIpAddress": {
                    "aws:SourceIp": [
                        "192.0.2.0/24",
                        "203.0.113.0/24"
                    ]
                }
            },
            "Resource": ["*"]
        }
    ]
}
```

The specific API action is defined

The effect is to deny the API action if the IP address is not in the specified range

IAM Policy Example 3

```
{
    "Version": "2012-10-17",
    "Id": "ExamplePolicy01",
    "Statement": [
        {
            "Sid": "ExampleSatement01",
            "Effect": "Allow",
            "Principal": {
                "AWS": "*"
            },
            "Action": [
                "elasticfilesystem:ClientRootAccess",
                "elasticfilesystem:ClientMount",
                "elasticfilesystem:ClientWrite"
            ],
            "Condition": {
                "Bool": {
                    "aws:SecureTransport": "true"
                }
            }
        }
    ]
}
```

You can tell this is a resource-based policy as it has a principal element defined

The policy grants read and write access to an EFS file systems to all IAM principals ("AWS ": "*")

Additionally, the policy condition element requires that SSL/TLS encryption is used

IAM Policy Example 4

```
{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": ["s3:ListBucket"],
        "Effect": "Allow",
        "Resource": ["arn:aws:s3:::mybucket"],
        "Condition": {"StringLike": {"s3:prefix": ["${aws:username}/*"]}}
      },
      {
        "Action": [
          "s3:GetObject",
          "s3:PutObject"
        ],
        "Effect": "Allow",
        "Resource": ["arn:aws:s3:::mybucket/${aws:username}/*"]
      }
    ]
}
```

A variable is used for the s3:prefix that is replaced with the user's friendly name

The actions are allowed only within the user's folder within the bucket

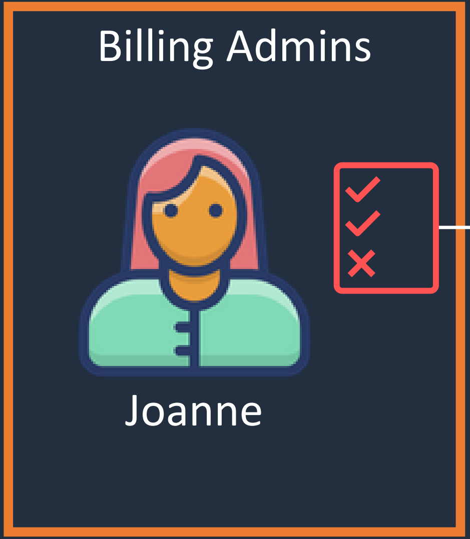
DigitalCloud TRAINING

Using Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC)

Job function policies:

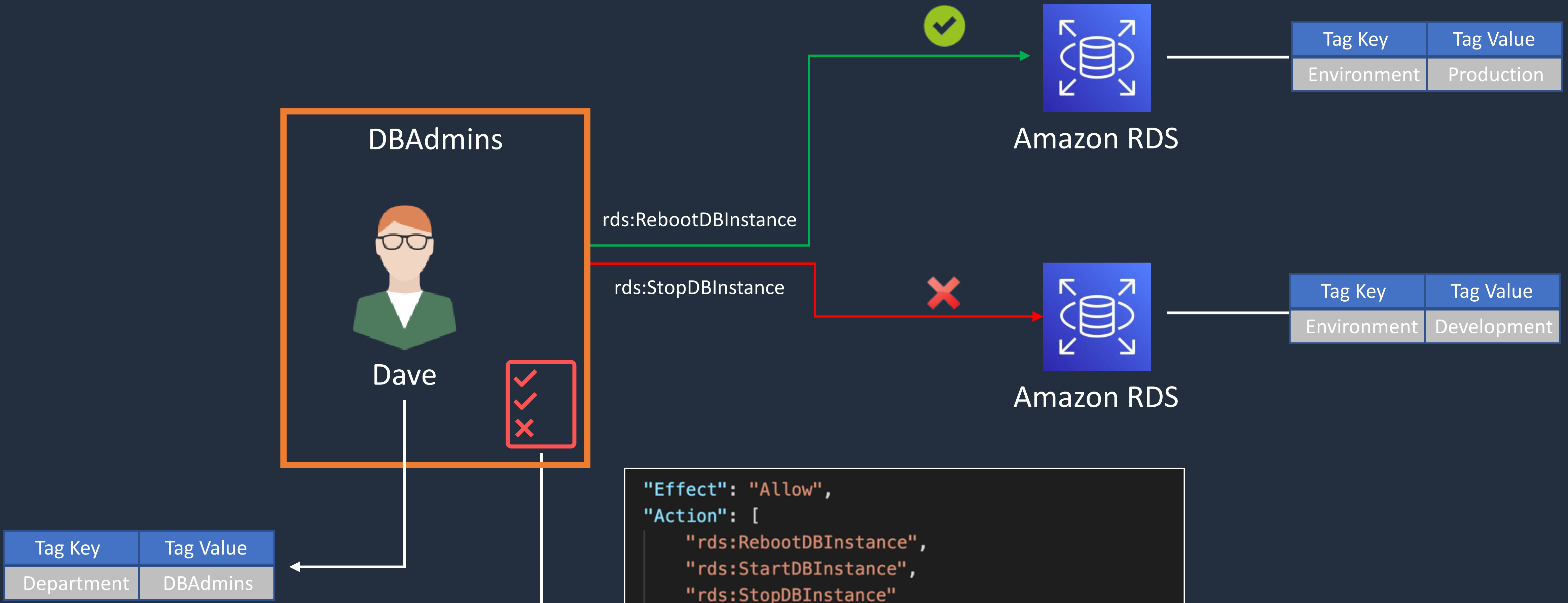
- Administrator
- Billing
- Database administrator
- Data scientist
- Developer power user
- Network administrator
- Security auditor
- Support user
- System administrator
- View-only user




```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "aws-portal:*Billing",
                "aws-portal:*Usage",
                "aws-portal:*PaymentMethods",
                "budgets:ViewBudget",
                "budgets:ModifyBudget",
                "ce:UpdatePreferences",
                "ce:CreateReport",
                "ce:UpdateReport",
                "ce:DeleteReport",
                "ce:CreateNotificationSubscription",
                "ce:UpdateNotificationSubscription",
                "ce:DeleteNotificationSubscription",
                "cur:DescribeReportDefinitions",
                "cur:PutReportDefinition",
                "cur:ModifyReportDefinition",
                "cur:DeleteReportDefinition",
                "purchase-orders:*PurchaseOrders"
            ],
            "Resource": "*"
        }
    ]
}
```

Using Attribute-Based Access Control (ABAC)

Attribute-Based Access Control (ABAC)




```
"Effect": "Allow",
"Action": [
    "rds:RebootDBInstance",
    "rds:StartDBInstance",
    "rds:StopDBInstance"
],
"Resource": "*",
"Condition": {
    "StringEquals": {
        "aws:PrincipalTag/Department": "DBAdmins",
        "rds:db-tag/Environment": "Production"
```

Apply Permissions Boundary

Permisions Boundary Hands-On Practice

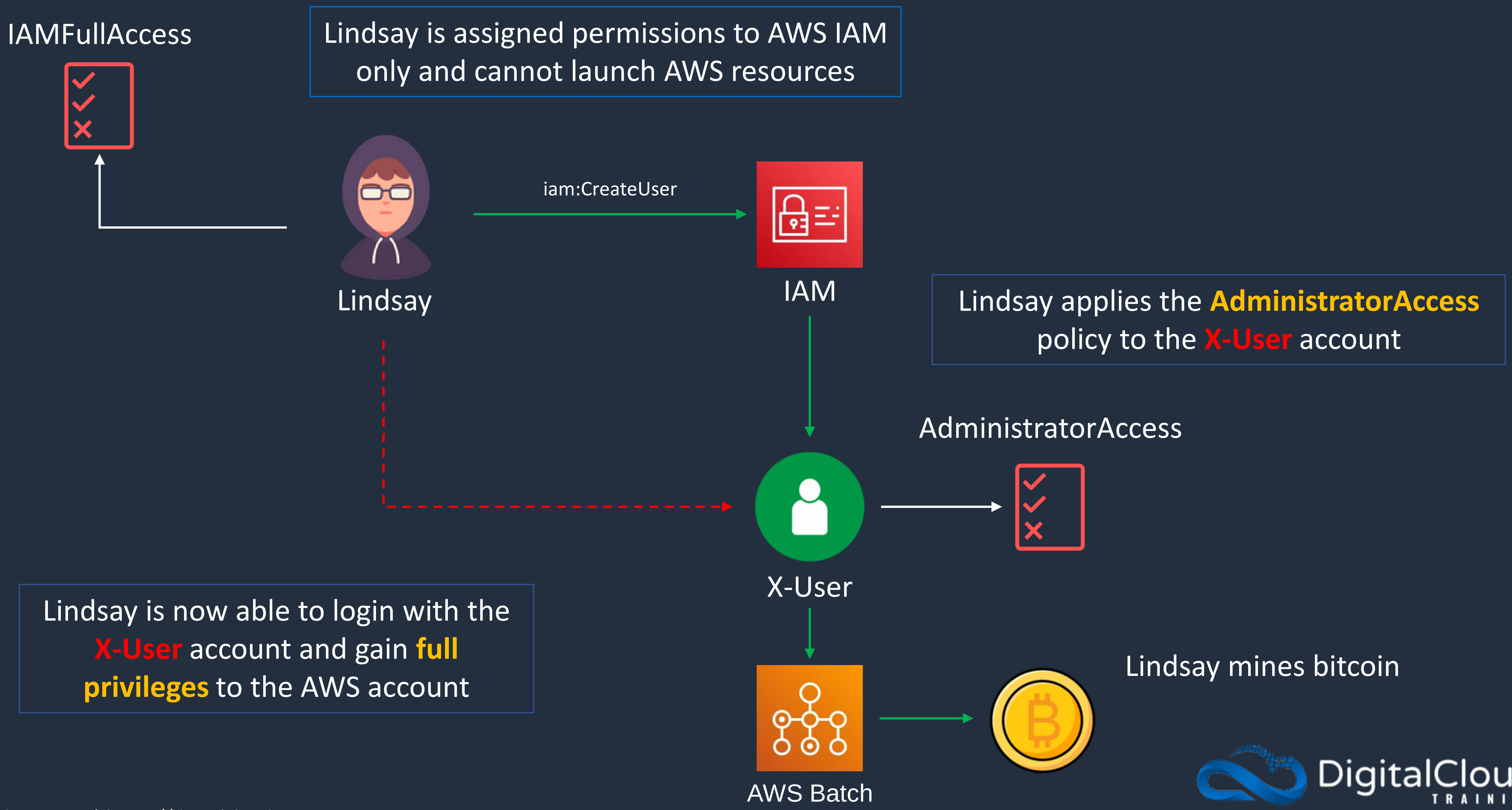
*** Use the PermissionsBoundary.json file

from the course download ***

The policy will enforce the following:

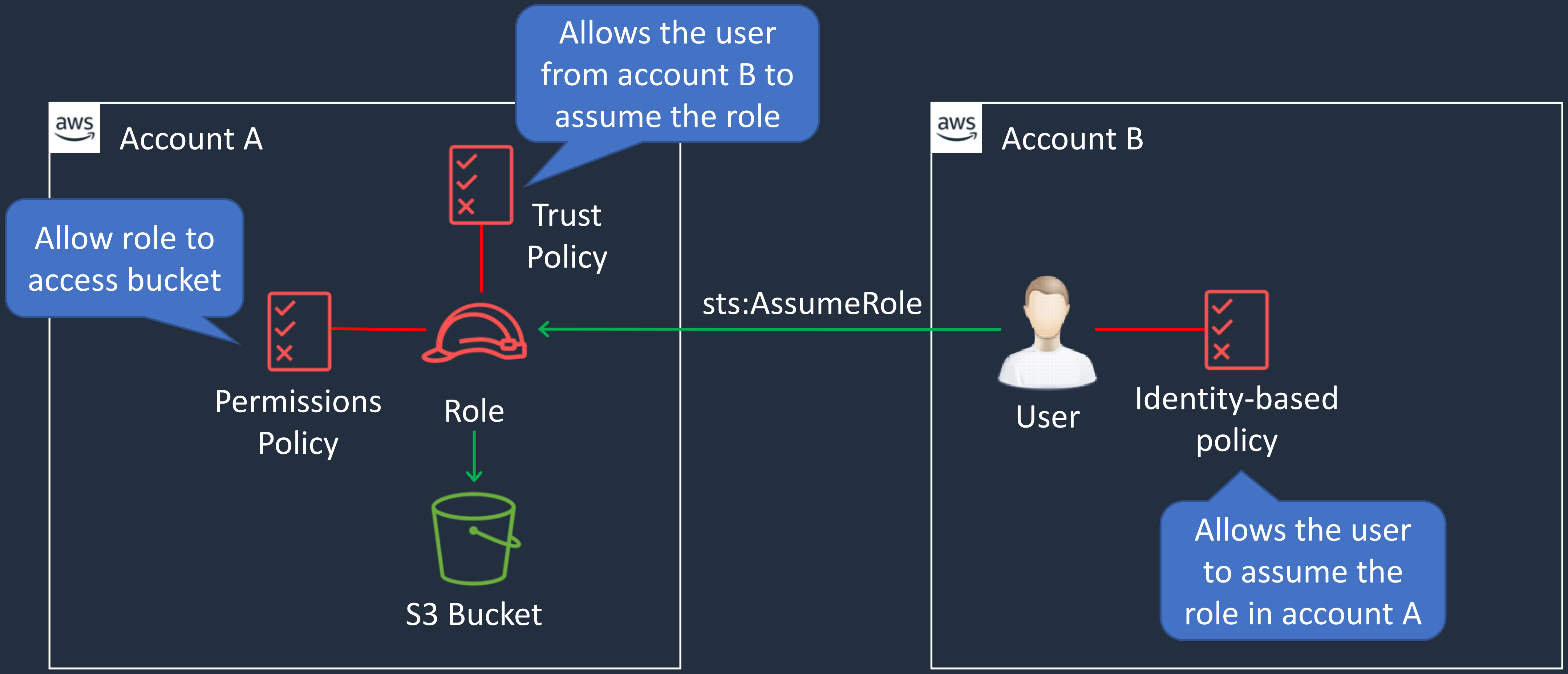
- IAM principals can't alter the permissions boundary to allow their own permissions to access restricted services
- IAM principals must attach the permissions boundary to any IAM principals they create
- IAM admins can't create IAM principals with more privileges than they already have
- The IAM principals created by IAM admins can't create IAM principals with more permissions than IAM admins

Privilege Escalation

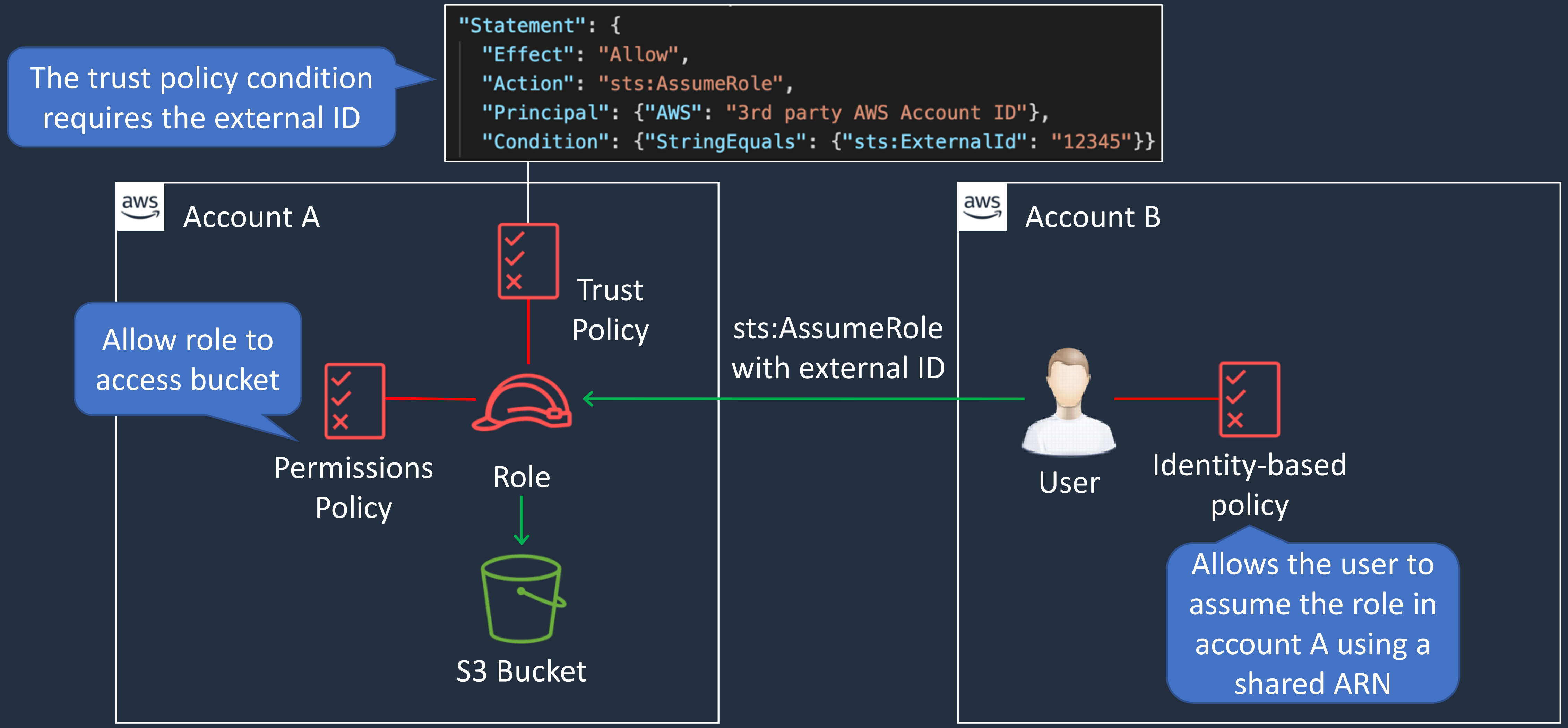


Use Cases for IAM Roles

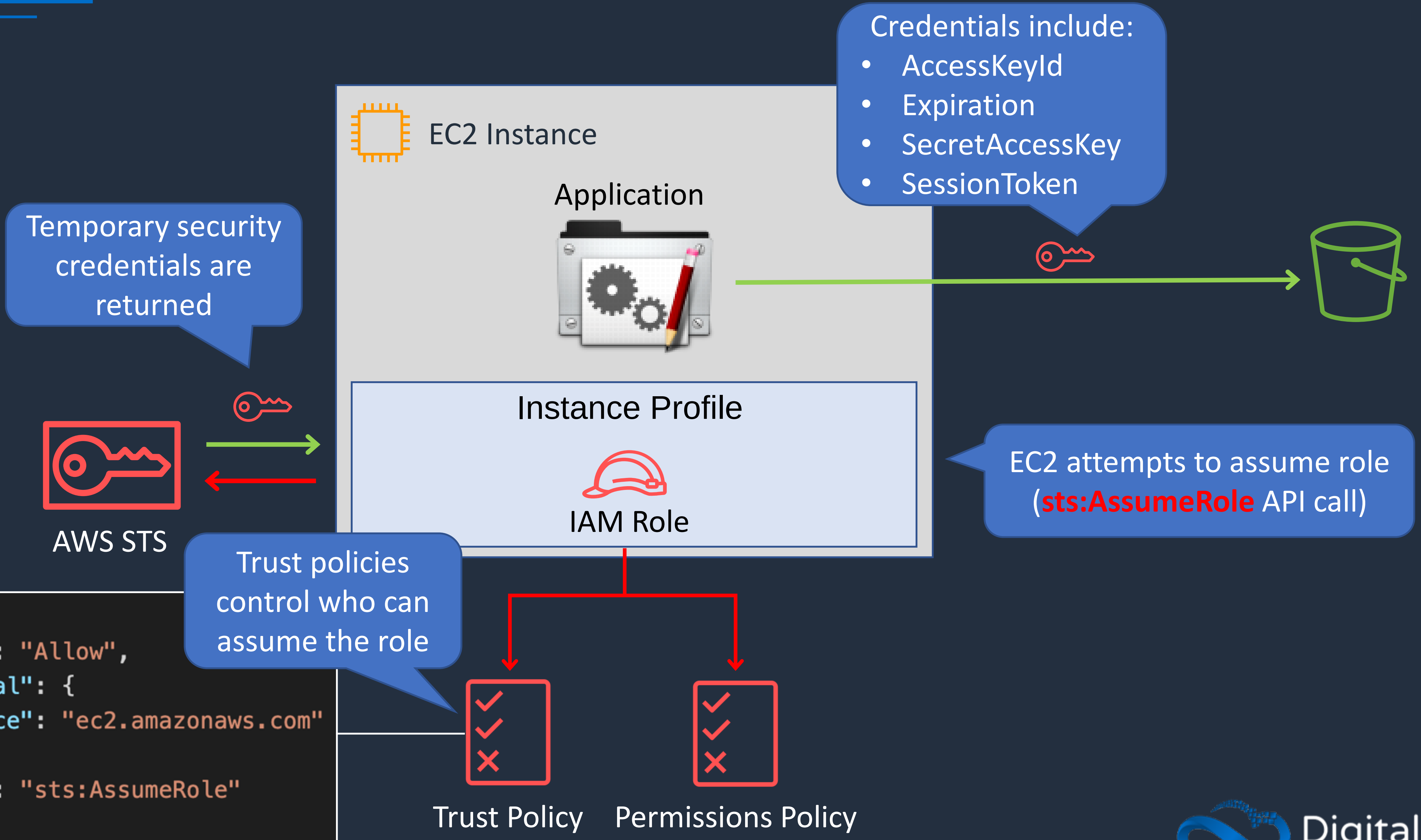
Use Case: Cross Account Access



Use Case: Cross Account Access (3rd Party)



Use Case: Delegation to AWS Services



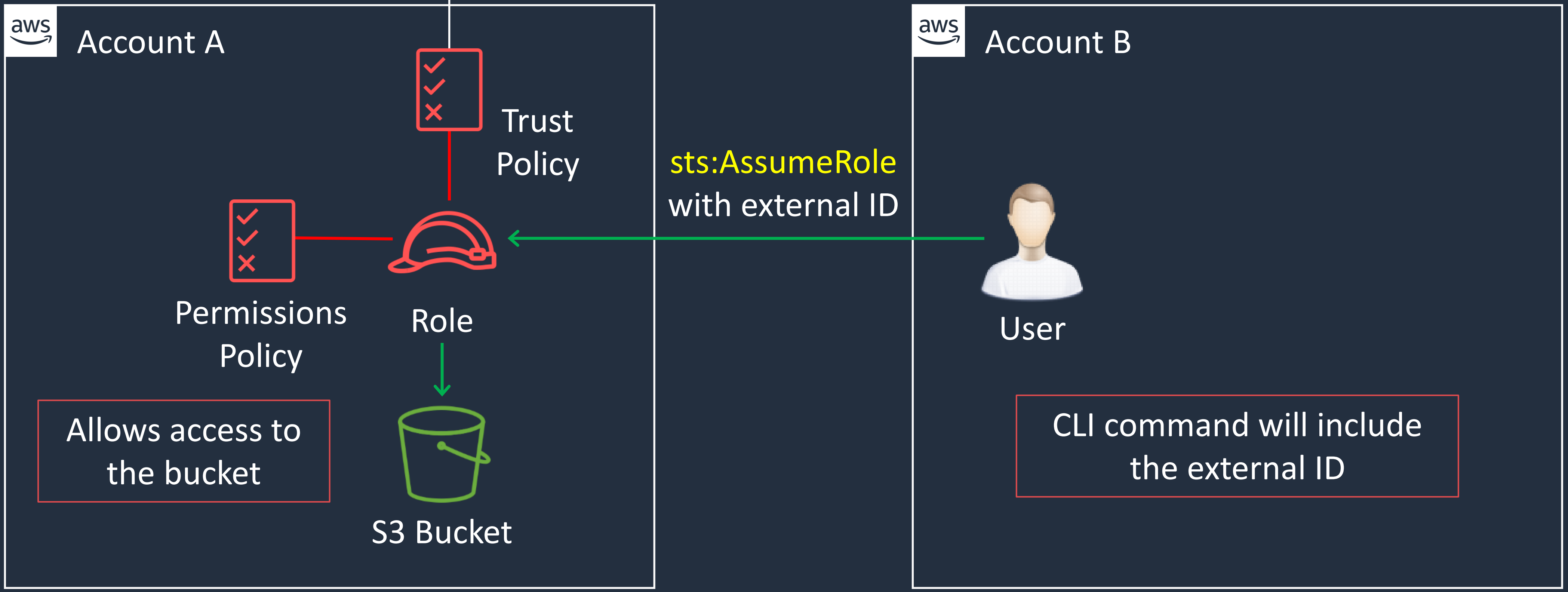

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "ec2.amazonaws.com"
  },
  "Action": "sts:AssumeRole"
}
```

Cross-Account Access

Cross Account Access (IAM Role)

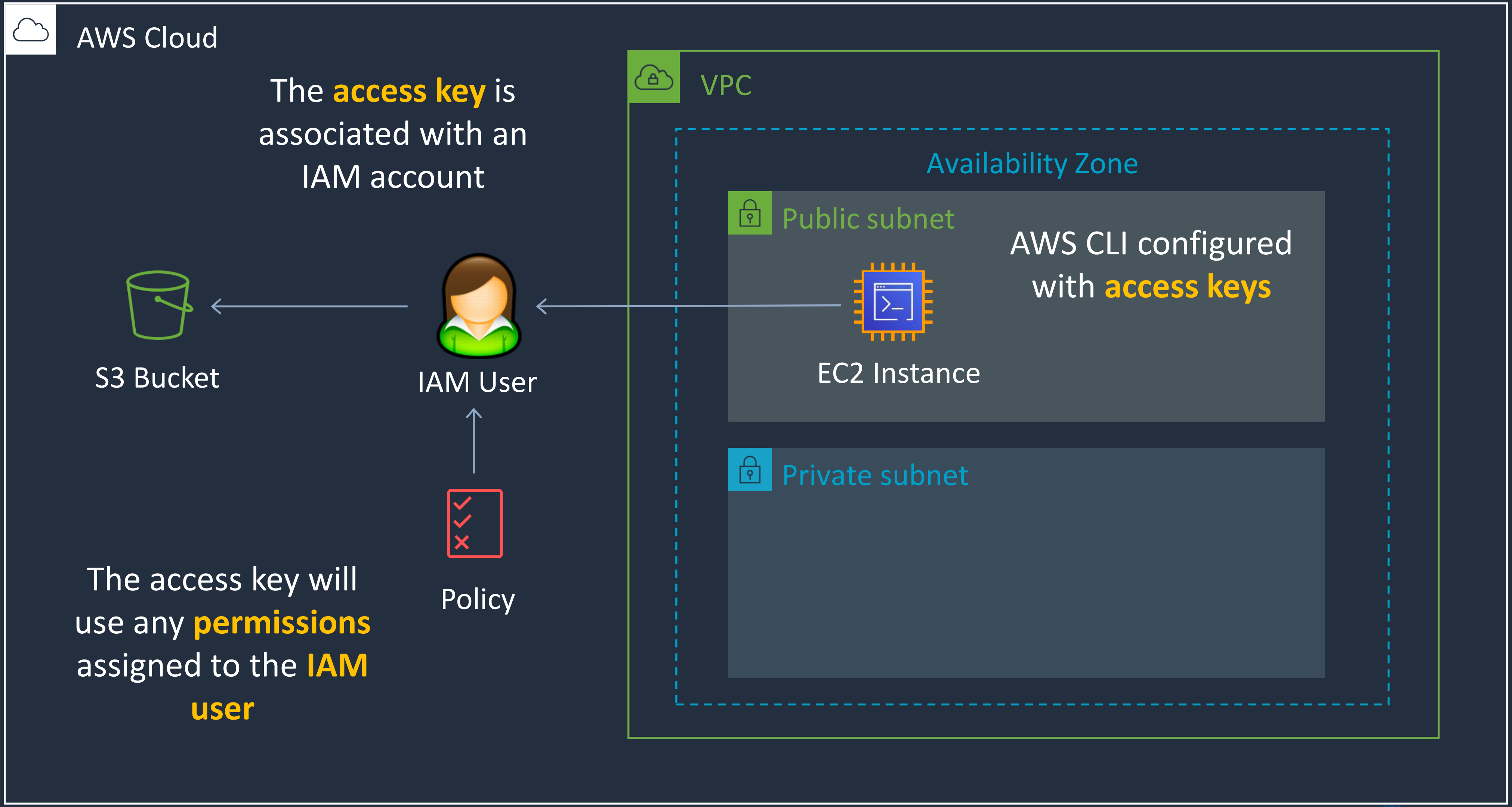
The trust policy condition requires the external ID

```
"Statement": {
  "Effect": "Allow",
  "Action": "sts:AssumeRole",
  "Principal": {"AWS": "3rd party AWS Account ID"},
  "Condition": {"StringEquals": {"sts:ExternalId": "12345"}}
```

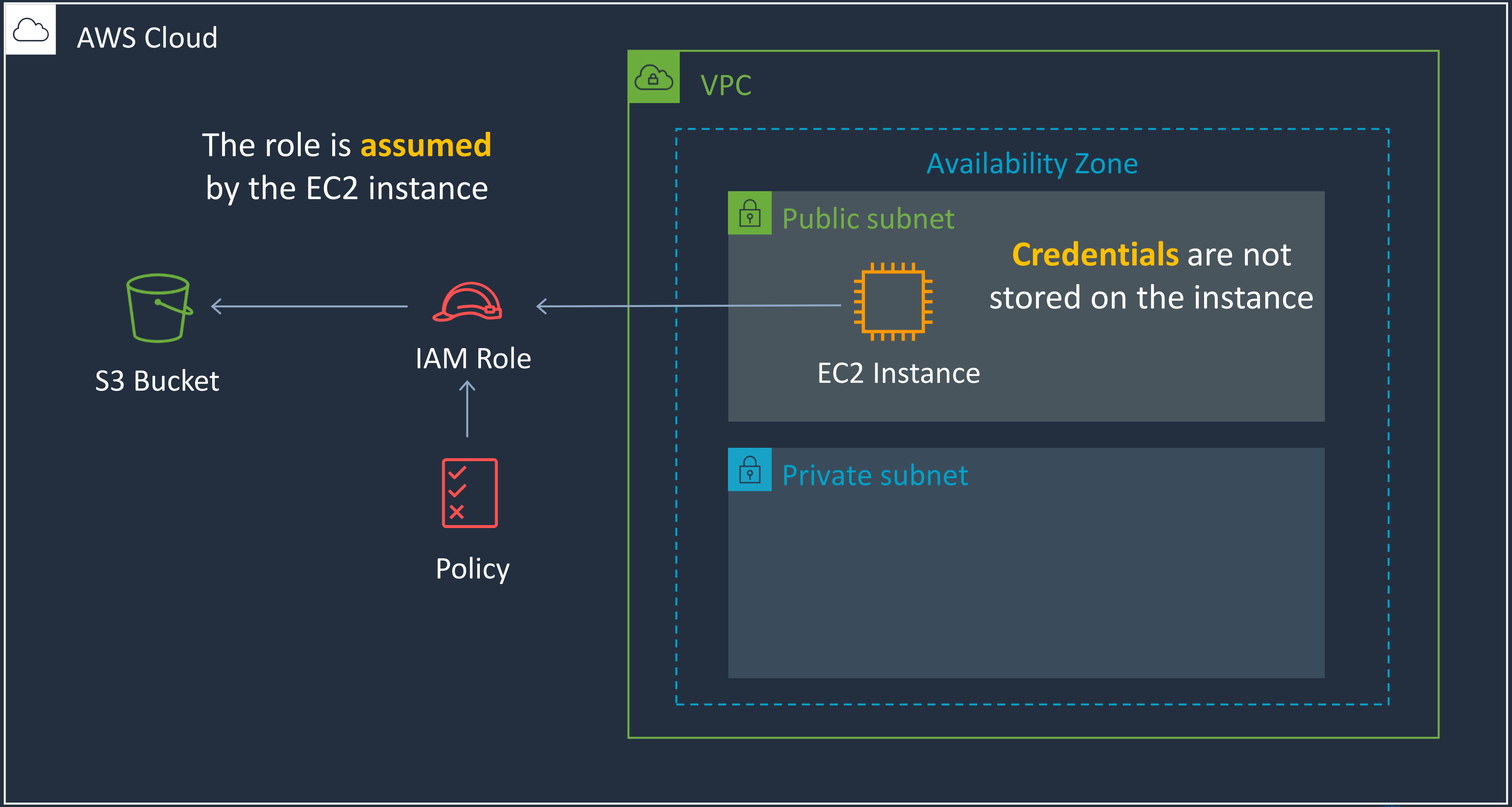


Access Keys and IAM Roles with EC2

Using Access Keys with Amazon EC2

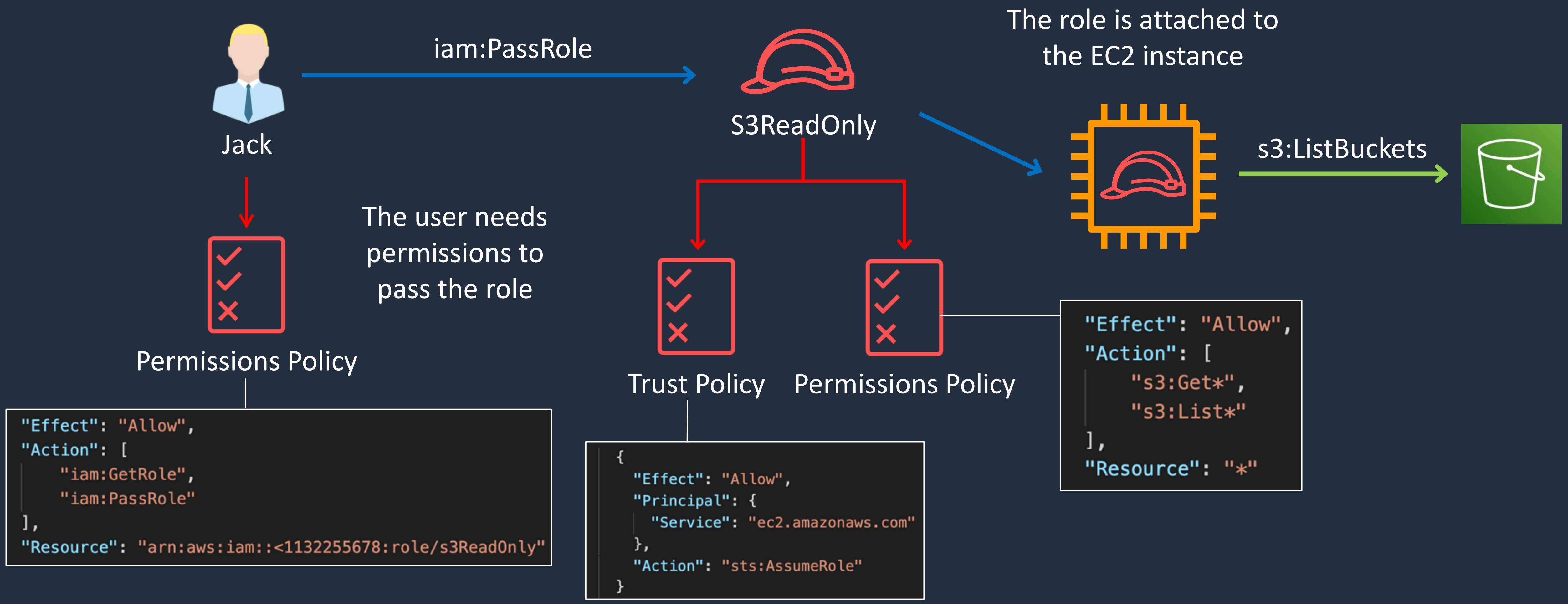


Using Roles with Amazon EC2



Amazon EC2 Instance Profile

Attach Role to EC2 Instance



IAM Best Practices

AWS IAM Best Practices

- Require human users to use federation with an identity provider to access AWS using temporary credentials
- Require workloads to use temporary credentials with IAM roles to access AWS
- Require multi-factor authentication (MFA)
- Rotate access keys regularly for use cases that require long-term credentials
- Safeguard your root user credentials and don't use them for everyday tasks
- Apply least-privilege permissions
- Get started with AWS managed policies and move toward least-privilege permissions

AWS IAM Best Practices

- Use IAM Access Analyzer to generate least-privilege policies based on access activity
- Regularly review and remove unused users, roles, permissions, policies, and credentials
- Use conditions in IAM policies to further restrict access
- Verify public and cross-account access to resources with IAM Access Analyzer
- Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions
- Establish permissions guardrails across multiple accounts
- Use permissions boundaries to delegate permissions management within an account

SECTION 4

AWS Directory Services and Federation

AWS Directory Services

AWS Directory Services

Types of Directory Services:

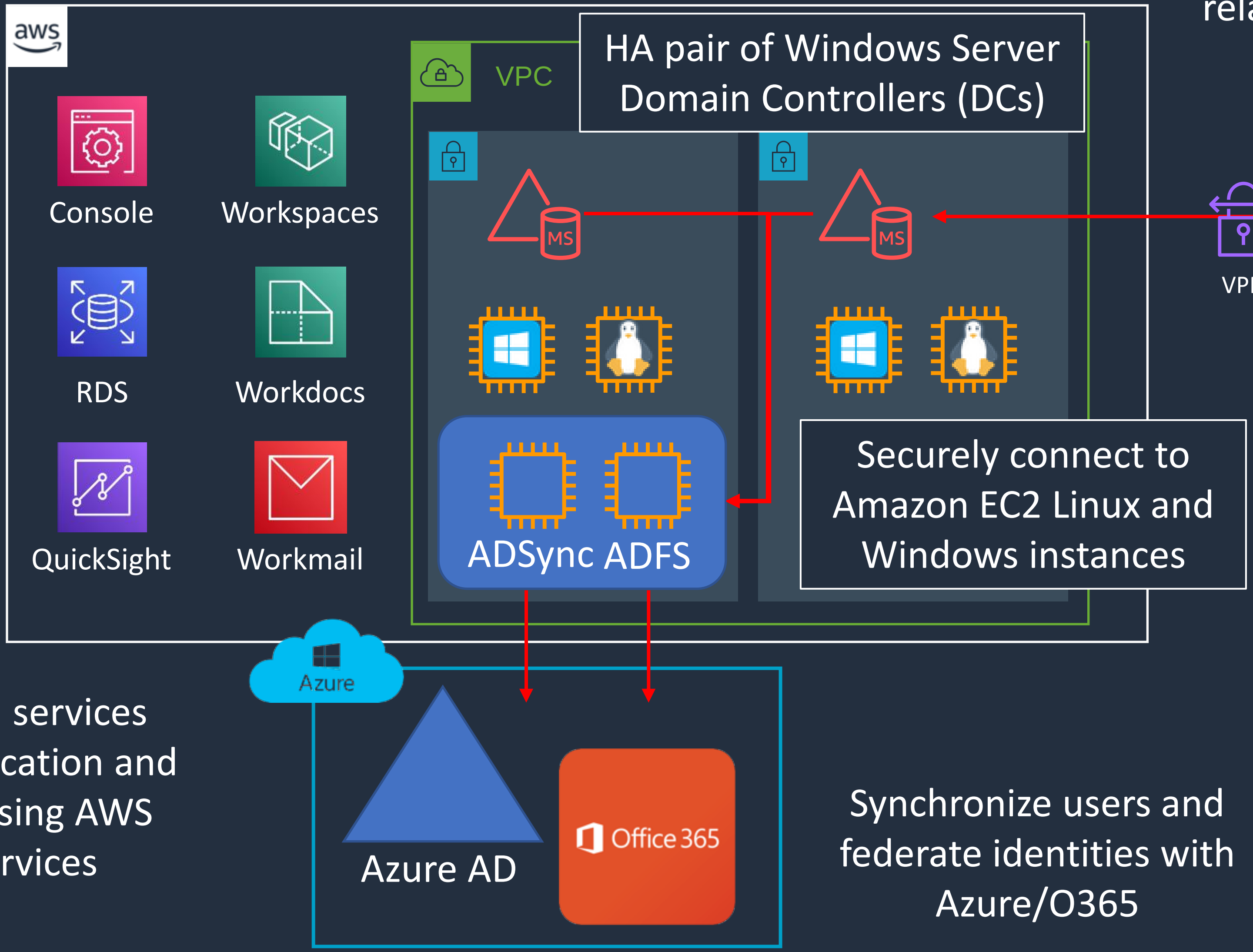
- **AWS Managed Microsoft AD** – Full AD in AWS, supports Group Policy and trust relationships
- **AD Connector** – Proxy to on-prem AD, no replication
- **Simple AD** – Lightweight, Samba-based, limited features, lower cost

DigitalCloud TRAINING

AWS Managed Microsoft AD

Managed implementation of Microsoft Active Directory running on Windows Server

A one or two-way trust relationship is established



Also Allows you to:

- Apply group policy
- Use single sign-on to apps and services
- Enable MFA with RADIUS

Many apps and services support authentication and authorization using AWS Directory Services

Synchronize users and federate identities with Azure/O365

AWS Managed Microsoft AD

Key Features:

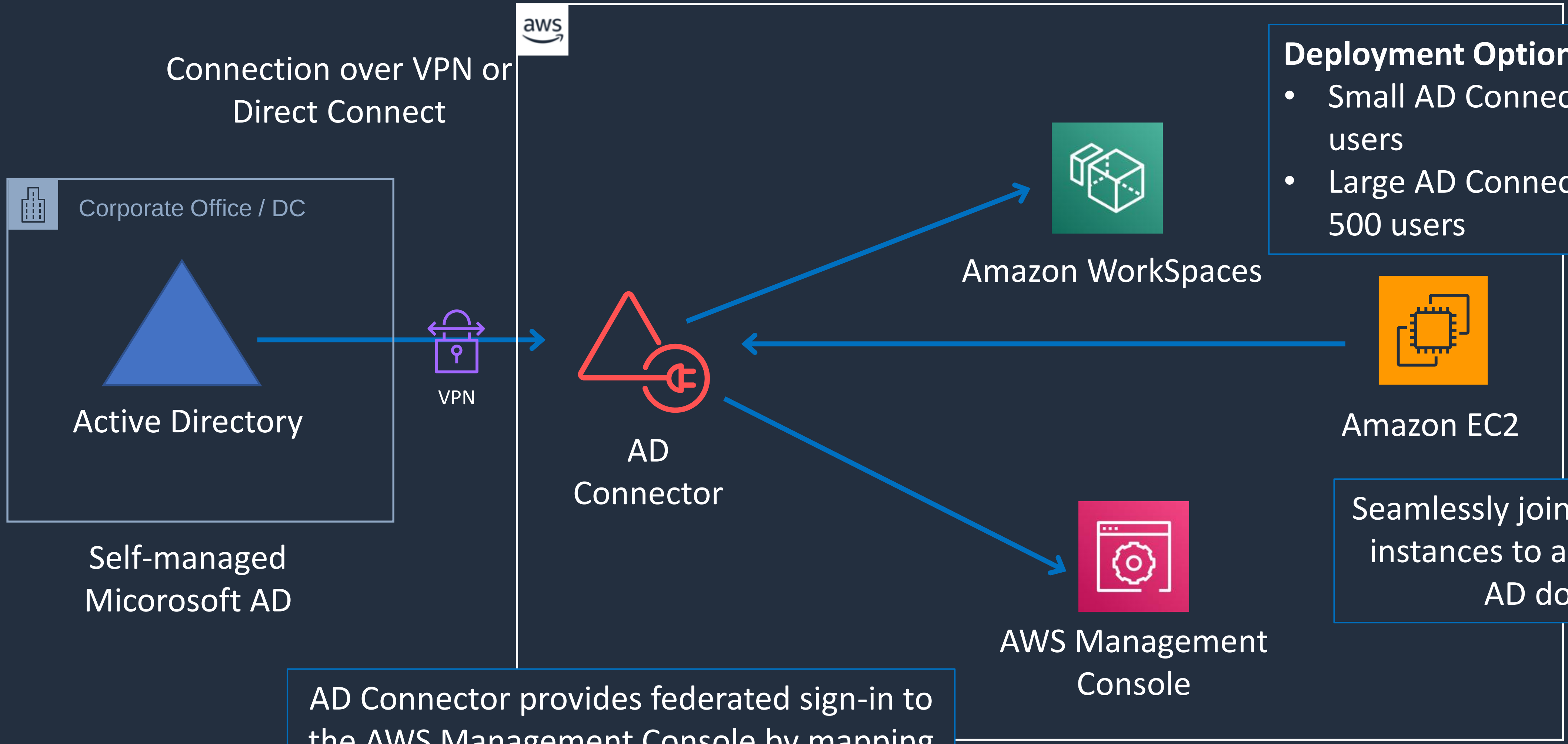
- Supports Group Policy, Kerberos, LDAP, and NTLM authentication
- Trust relationships with on-prem AD (one-way or two-way)
- Multi-AZ replication for high availability
- Custom OU (Organizational Unit) support

Use Cases:

- Extend on-prem Active Directory to AWS (hybrid cloud)
- Manage Windows-based workloads in AWS (EC2, FSx, RDS for SQL Server)
- Centralized identity management for AWS services like WorkSpaces

AD Connector

Sign-in to AWS applications such as Amazon WorkSpaces, Amazon WorkDocs, and Amazon WorkMail by using your Active Directory credentials



Deployment Options:

- Small AD Connector: Up to 500 users
- Large AD Connector: More than 500 users

Seamlessly join Windows EC2 instances to an on-premise AD domain

AD Connector provides federated sign-in to the AWS Management Console by mapping Active Directory identities to IAM Roles

AD Connector

Key Features:

- No directory data replication to AWS (queries forwarded to on-prem AD)
- Supports Windows workloads, IAM authentication, AWS IAM Identity Center
- Redundant, deployed across two subnets in a VPC

Use Cases:

- Extend on-prem AD authentication to AWS without syncing users
- Authenticate services for AWS WorkSpaces, FSx, RDS SQL, and EC2
- Avoid AD replication for compliance/security reasons

Simple AD

What it is: A lightweight, Samba 4-based directory in AWS

Key Features:

- Supports LDAP, Kerberos, and NTLM authentication.
- No trust relationships with on-prem AD
- Basic Group Policy support but lacks advanced AD features

Use Cases:

- Small-scale applications that don't require full Microsoft AD
- Standalone Linux/Unix authentication
- Basic AWS WorkSpaces or EC2 authentication

Simple AD

Deployment options:

- Small Directory: Up to 500 users
- Large Directory: Up to 5,000 users

AWS Directory Services

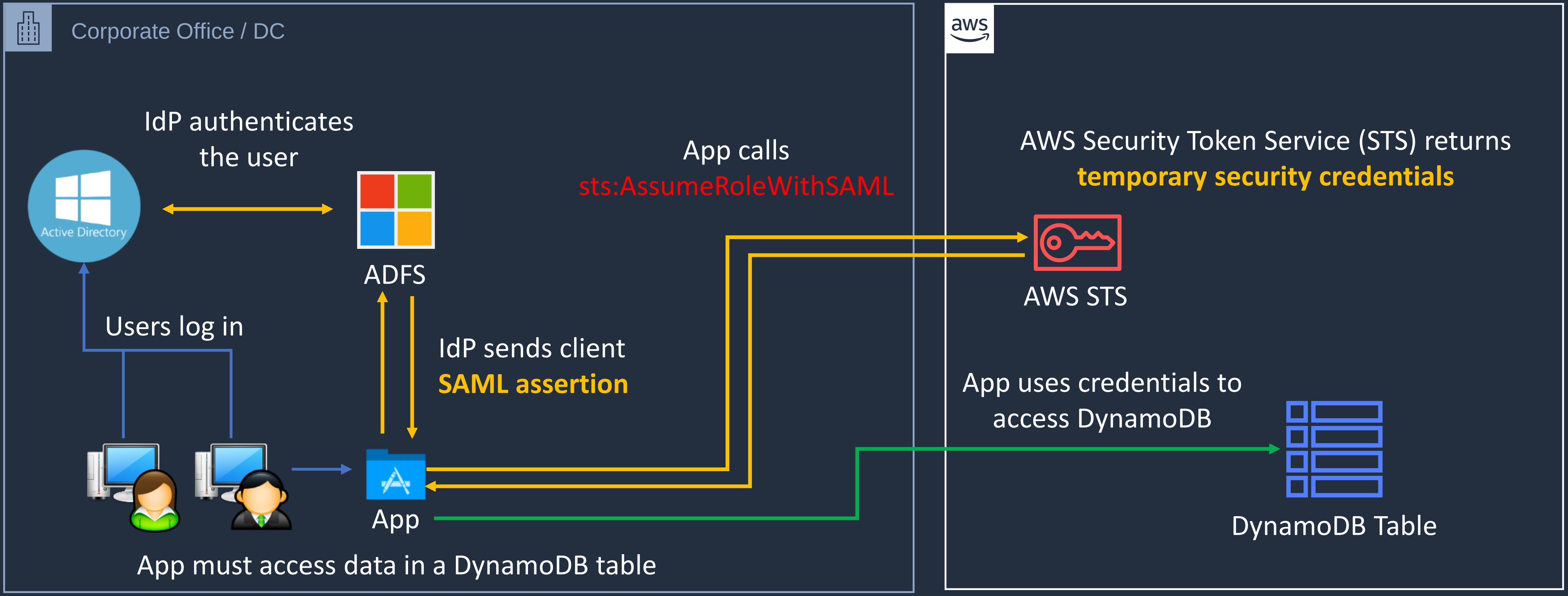
Feature	Managed AD	AD Connector	Simple AD
Type	Full AD in AWS	Proxy to on-prem AD	Lightweight AD
Replication	Yes	No	No
Trusts with on-prem AD	Yes	Uses existing on-prem AD	No
User authentication	AWS + on-prem users	On-prem users only	Users within the directory
Group Policy support	Yes	Yes (via on-prem AD)	Limited
AWS Services Integration	Yes	Yes	Limited
Use Case	Windows workloads, hybrid AD	Use on-prem AD for AWS login	Small, standalone environments

Identity Providers and Federation

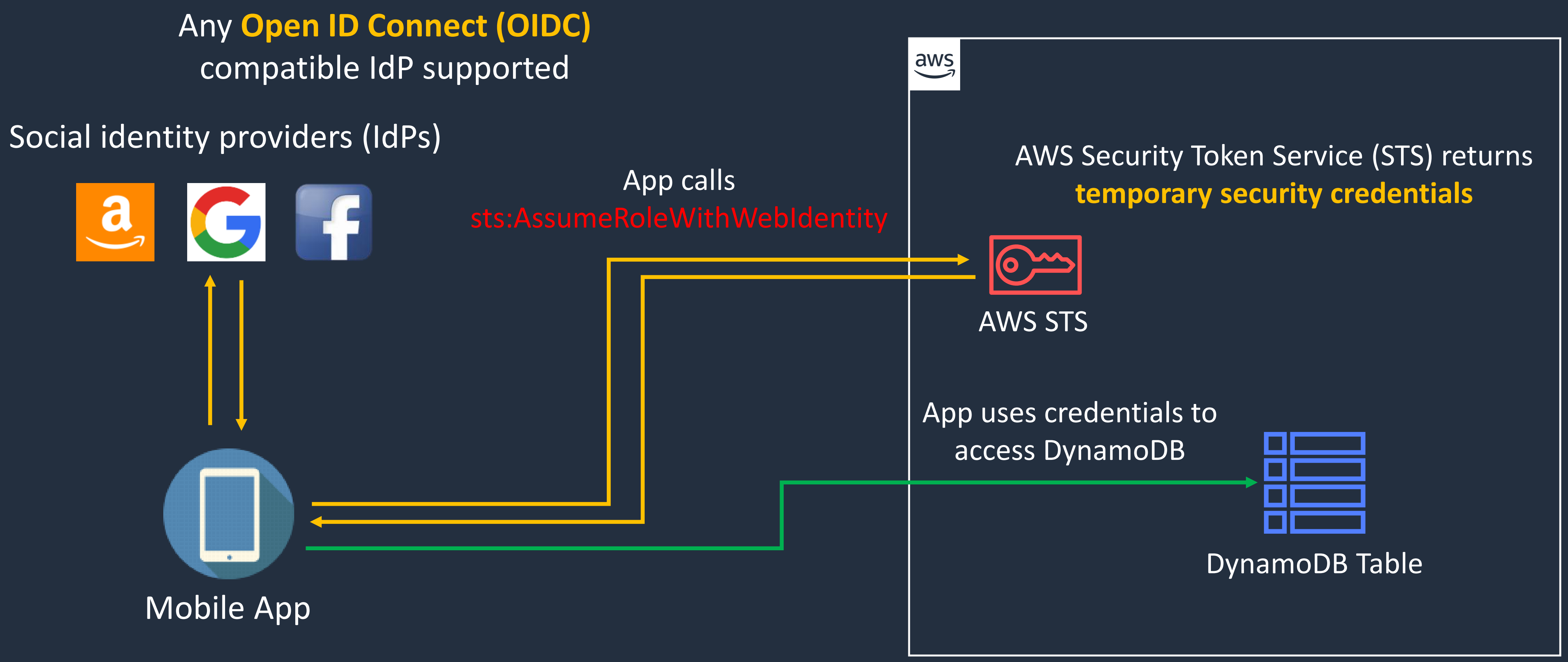
IAM – SAML 2.0 Identity Federation

Active Directory is an LDAP **Identity Store**

Active Directory Federation Services is an **Identity Provider** (IdP)



IAM – Web Identity Federation



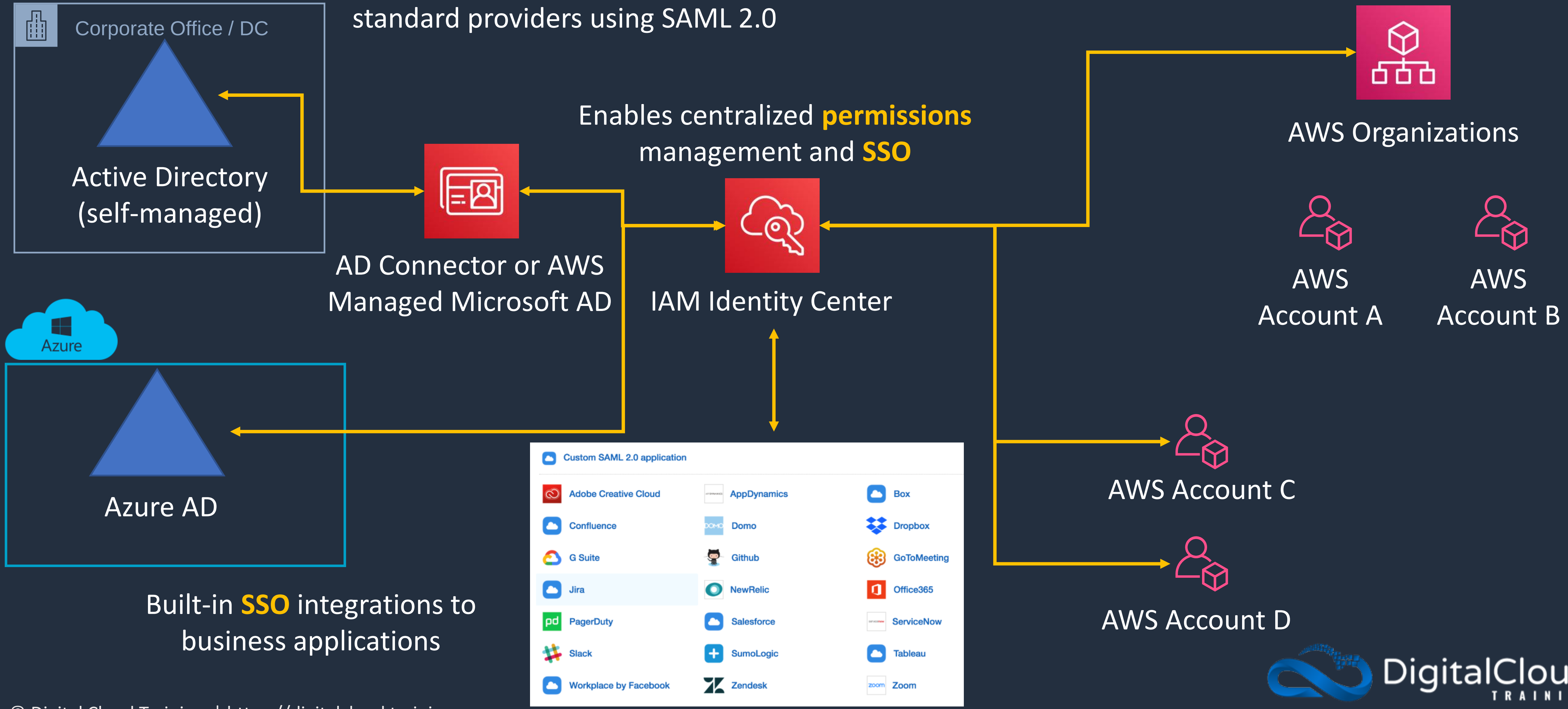
AWS recommend to use **Cognito** for **web identity federation** in most cases

IAM Identity Center

Identity sources can be Identity Center directory, Active Directory and standard providers using SAML 2.0

IAM Identity Center is the successor to **AWS Single Sign-On** (SSO)

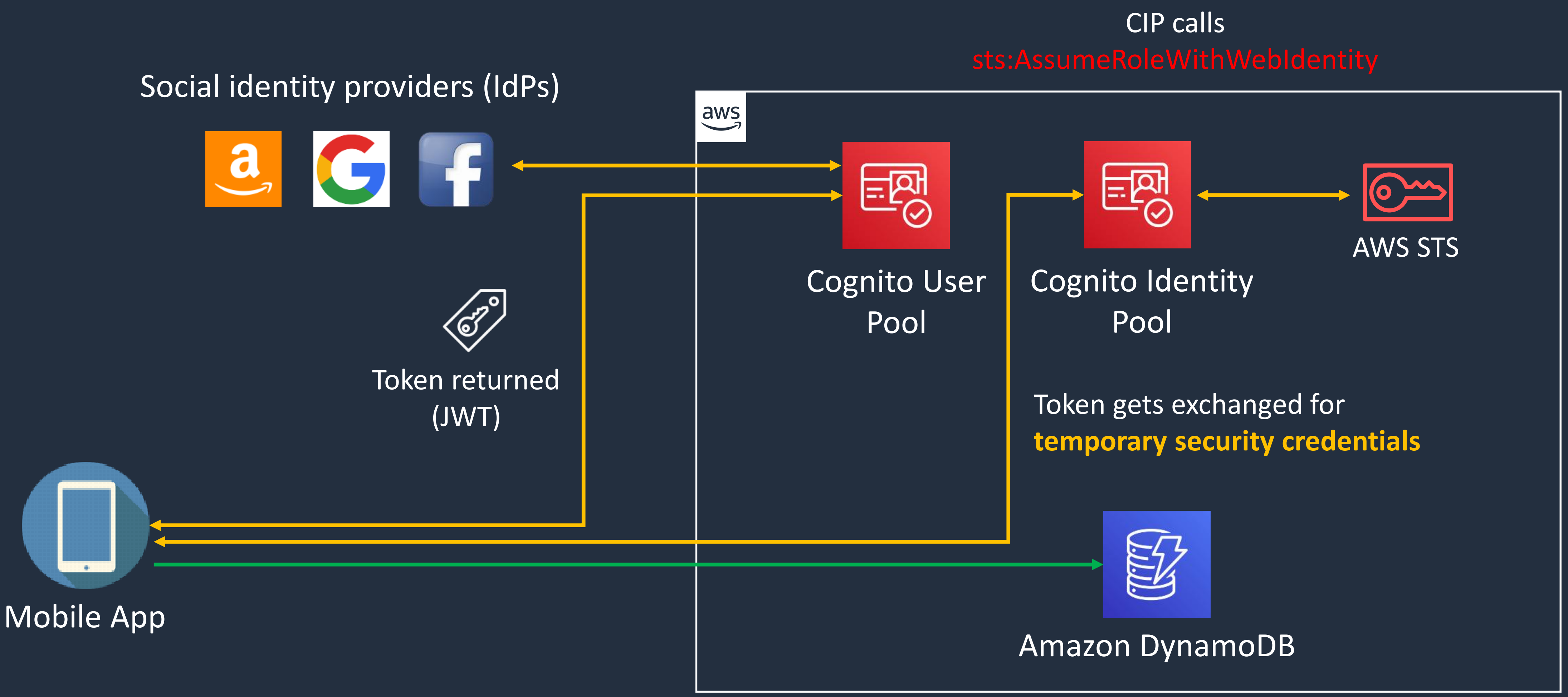
Enables centralized **permissions** management and **SSO**



Built-in **SSO** integrations to business applications

DigitalCloud TRAINING

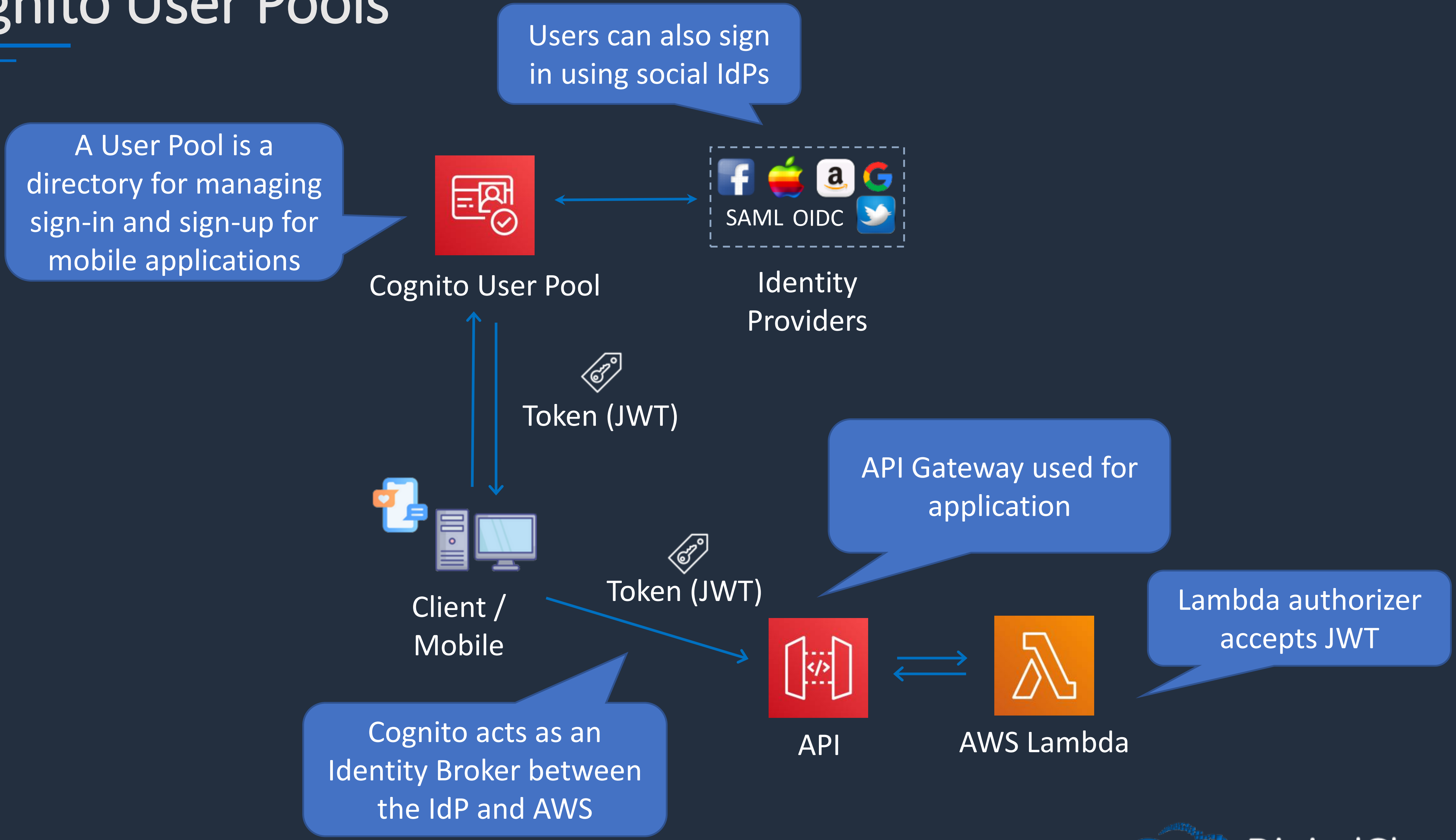
Amazon Cognito



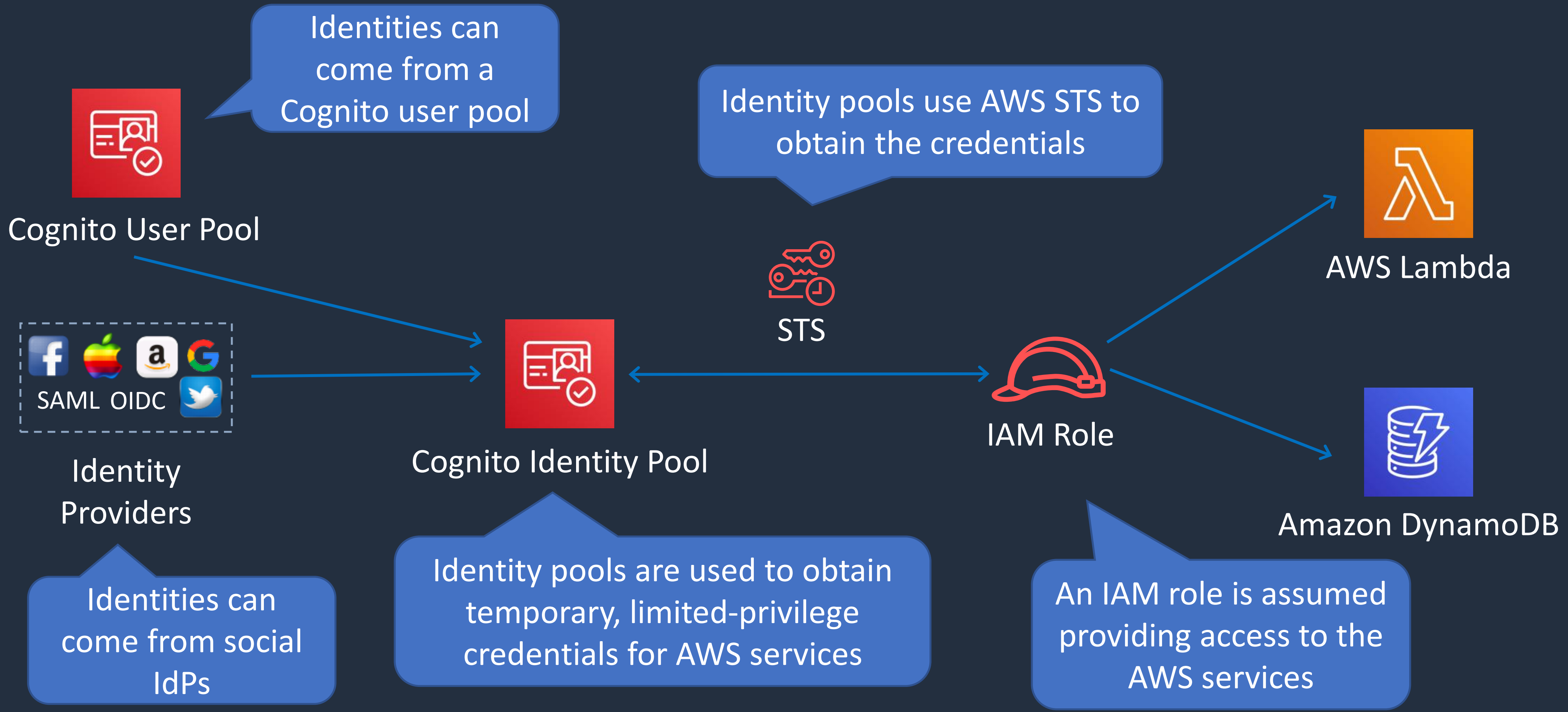
IAM Identity Center

Amazon Cognito

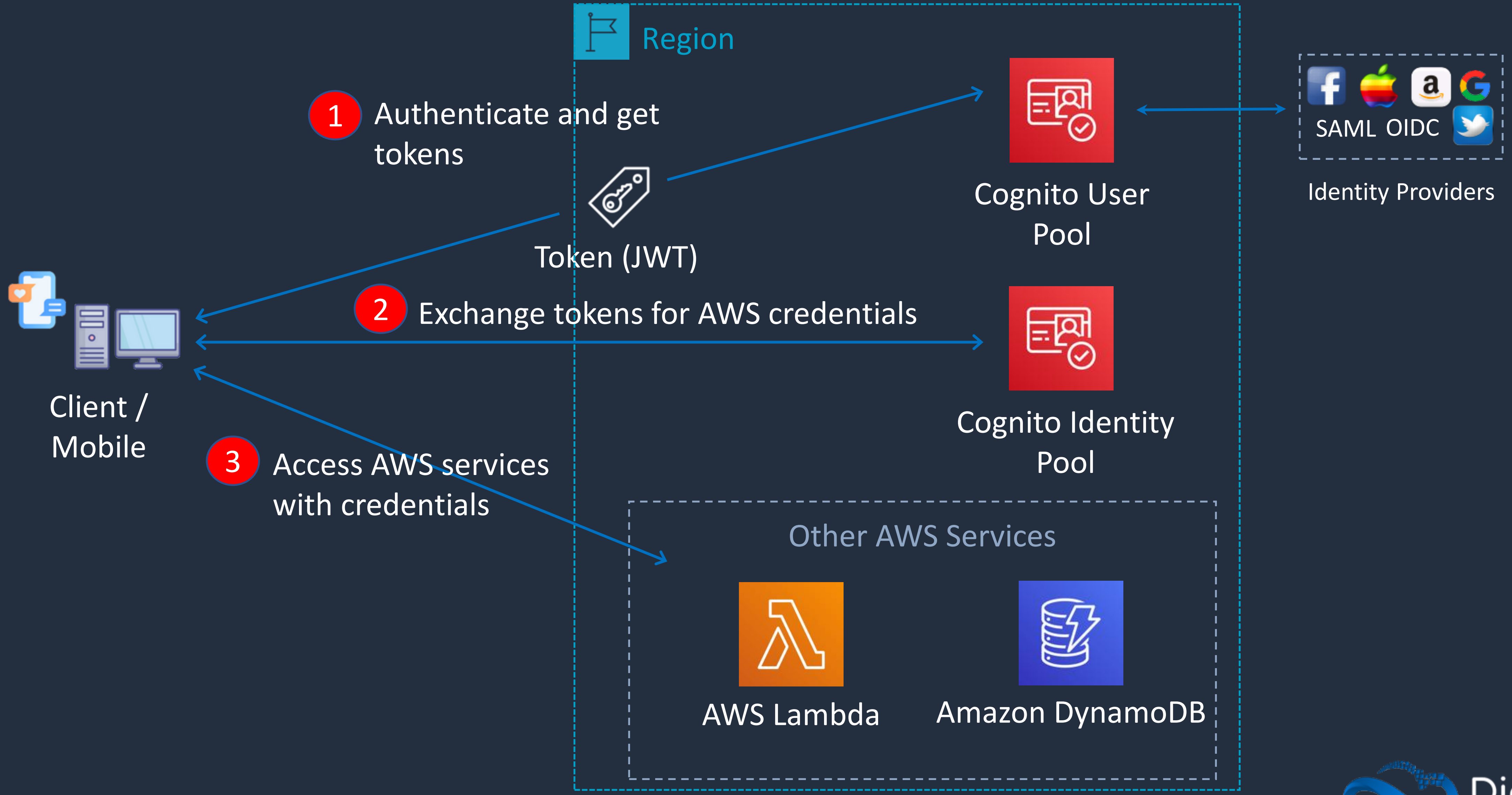
Cognito User Pools



Cognito Identity Pool



User Pools + Identity Pools



SECTION 5

Advanced VPC

The AWS Global Infrastructure

AWS Global Infrastructure

Region – us-east-1

Availability Zone

Availability Zone

Availability Zone

Every region is **connected** via a high bandwidth, fully redundant network

There are many **regions** around the world

Region – eu-west-1

Availability Zone

Availability Zone

Availability Zone

Each region consists of **multiple** Availability Zones

Region – ap-southeast-2

Availability Zone

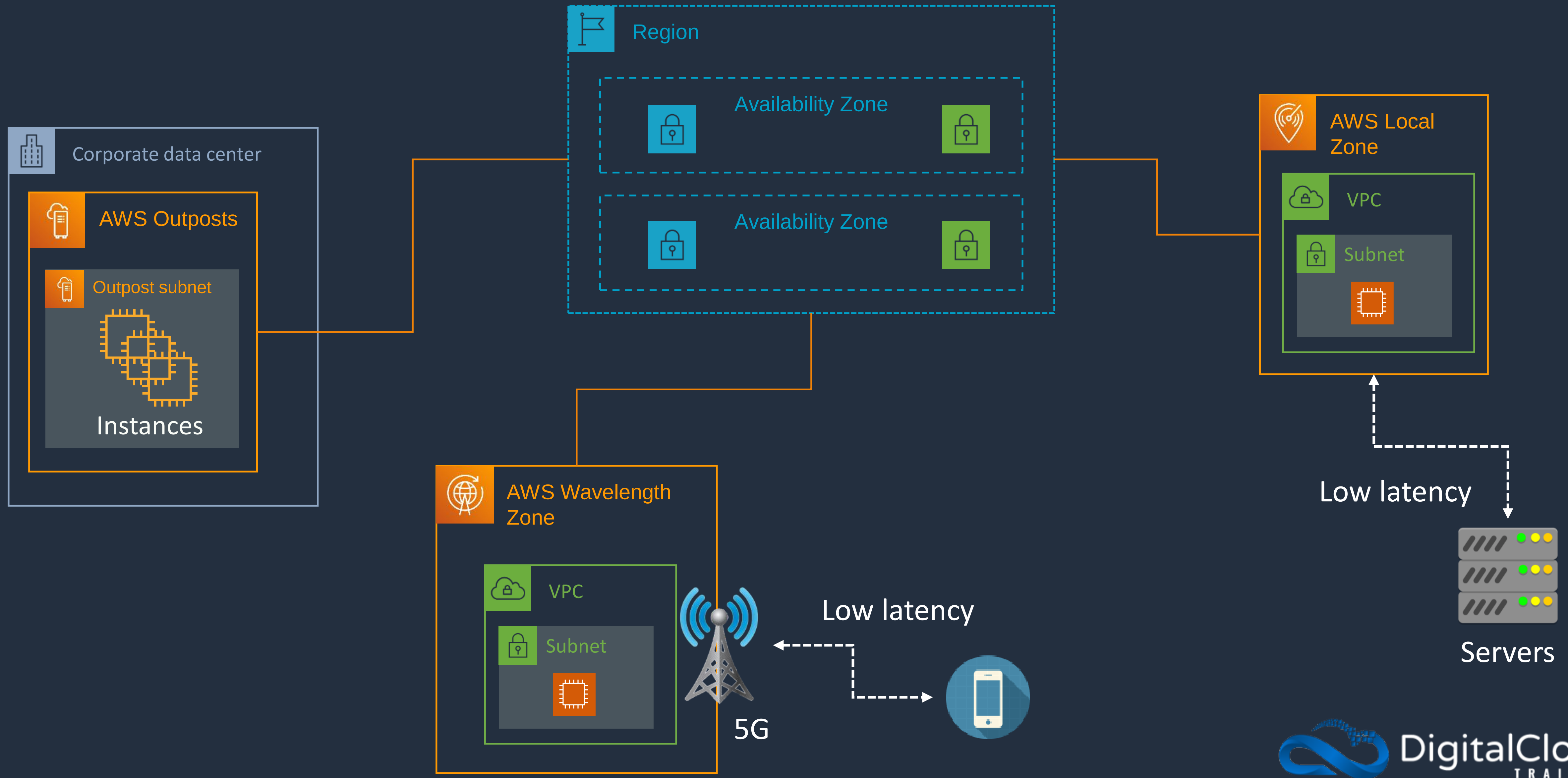
Availability Zone

Availability Zone

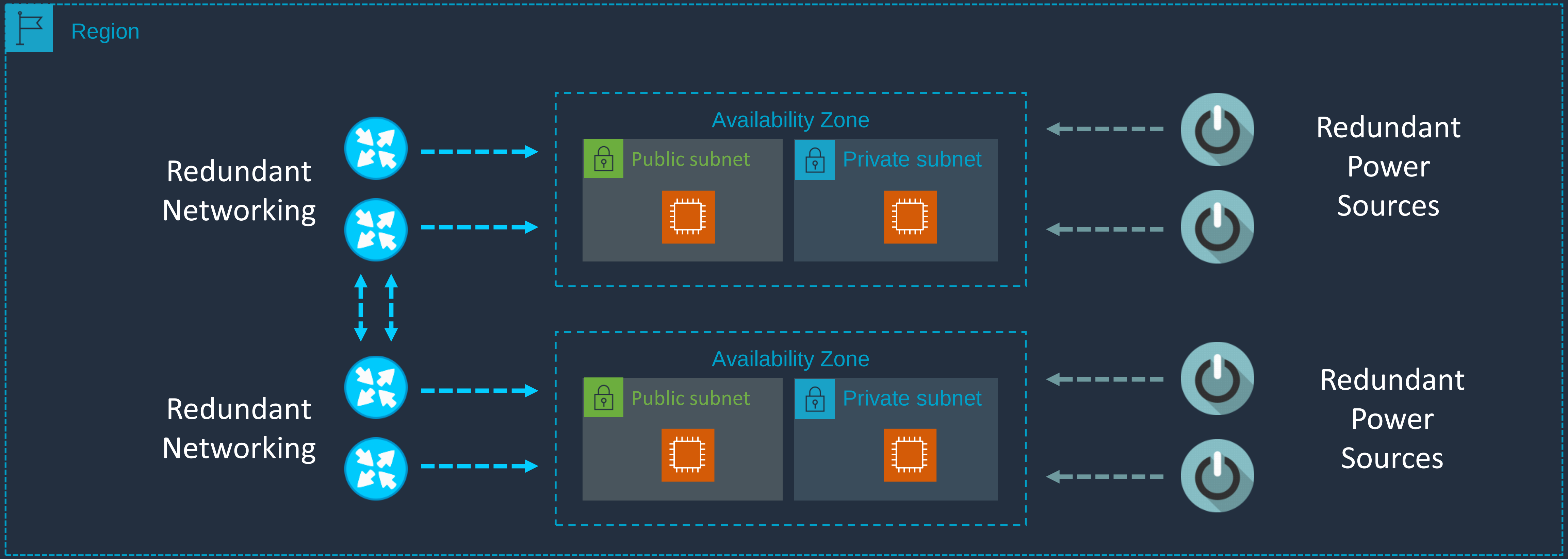
A **Region** is a physical location in the world and is **independent**

An Availability Zone is composed of **one** or **more** data centers

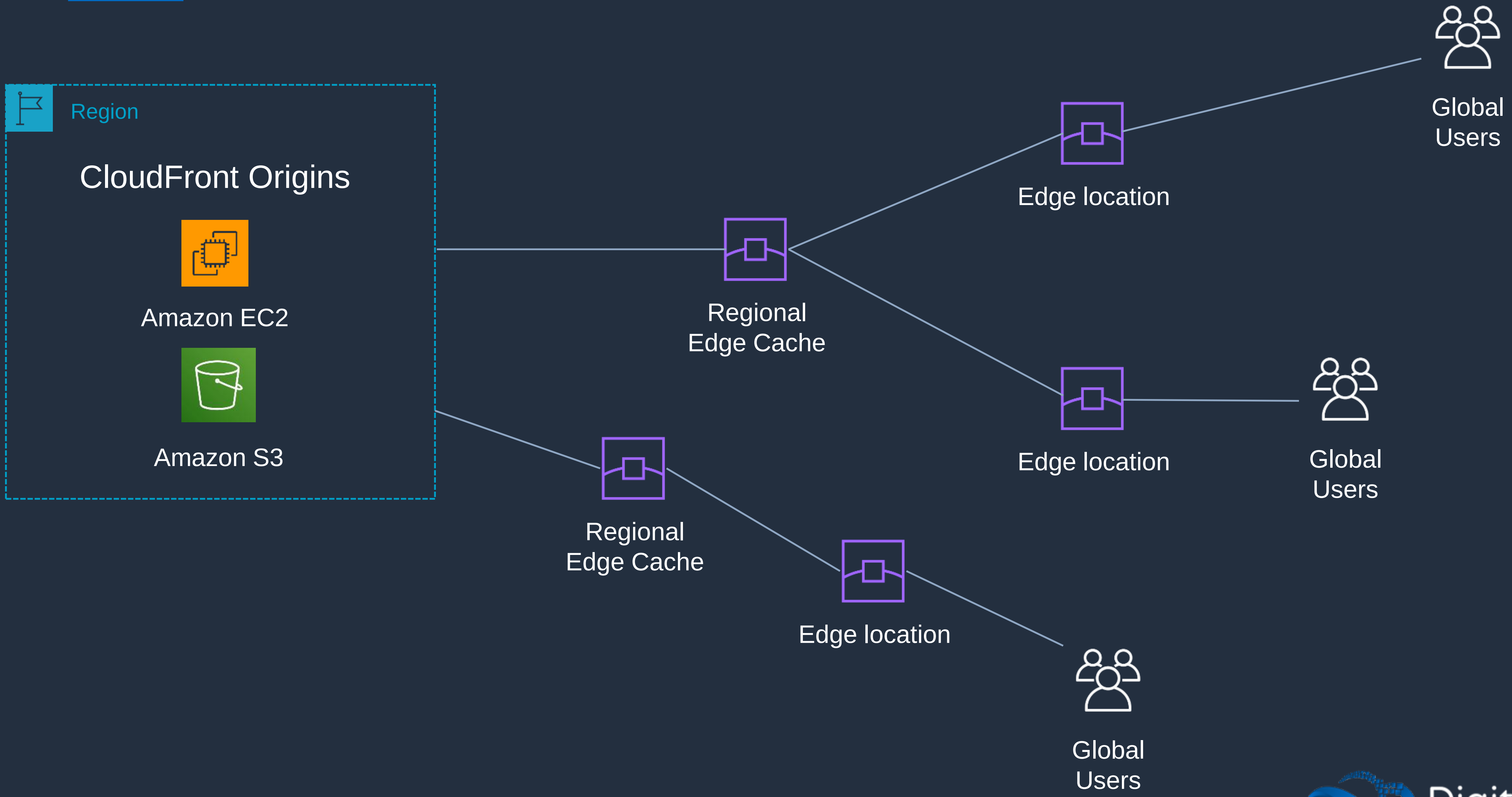
AWS Global Infrastructure



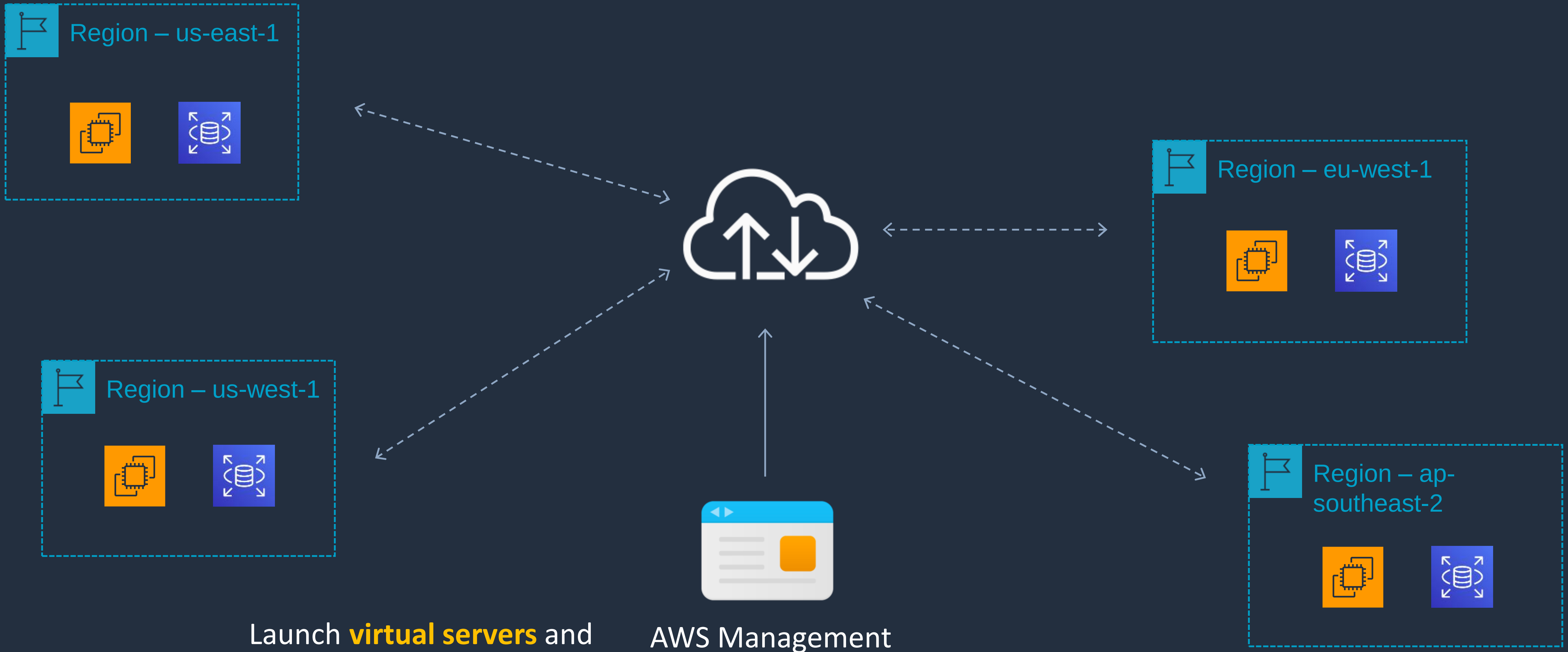
AWS Global Infrastructure



Amazon CloudFront



Deploying Services Globally



Launch **virtual servers** and **databases**

AWS Management Console

Defining VPC CIDR Blocks

DigitalCloud TRAINING

Defining VPC CIDR Blocks

Network	192	168	0	0
/24 Subnet Mask	255	255	255	0
/16 Subnet Mask	255	255	0	0
/20 Subnet Mask	255	255	0	0

First Address	Last Address
192.168.0.1	192.168.0.254

8 host bits = 256 addresses

16 host bits = 65536 addresses

First Address	Last Address
192.168.0.1	192.168.255.254

12 host bits = 4096 addresses

First Address	Last Address
192.168.0.1	192.168.15.254

Classless Interdomain Routing (CIDR) uses **variable length subnets masks** (VLSM)

DigitalCloud TRAINING

Rules and Guidelines

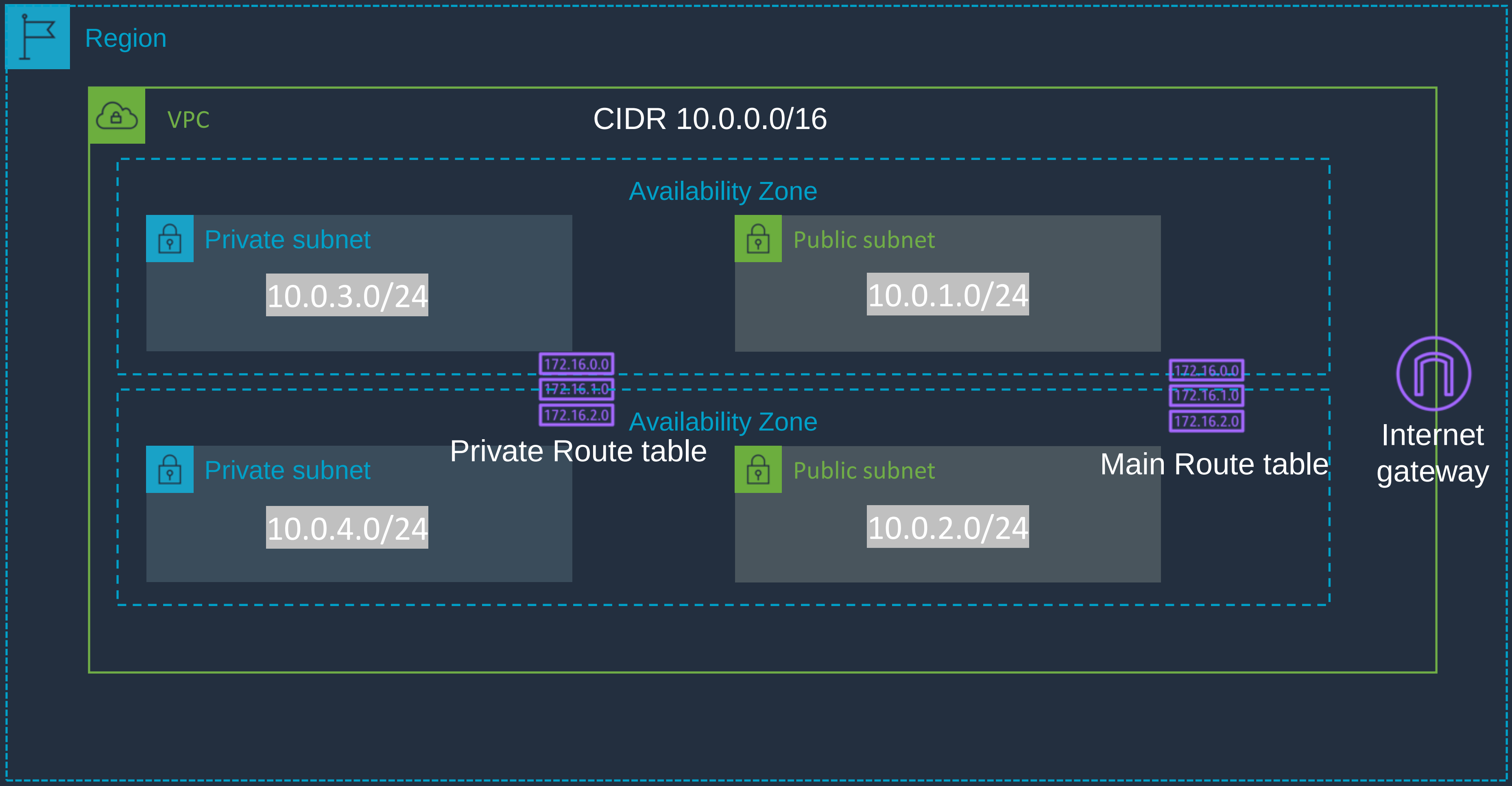
- CIDR block size can be between /16 and /28
- The CIDR block must not overlap with any existing CIDR block that's associated with the VPC
- You cannot increase or decrease the size of an existing CIDR block
- The first four and last IP address are not available for use
- AWS recommend you use CIDR blocks from the RFC 1918 ranges:

RFC 1918 Range	Example CIDR Block
10.0.0.0 - 10.255.255.255 (10/8 prefix)	Your VPC must be /16 or smaller, for example, 10.0.0.0/16
172.16.0.0 - 172.31.255.255 (172.16/12 prefix)	Your VPC must be /16 or smaller, for example, 172.31.0.0/16
192.168.0.0 - 192.168.255.255 (192.168/16 prefix)	Your VPC can be smaller, for example 192.168.0.0/20

DigitalCloud TRAINING

Create a Custom VPC

Create a Custom VPC



Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

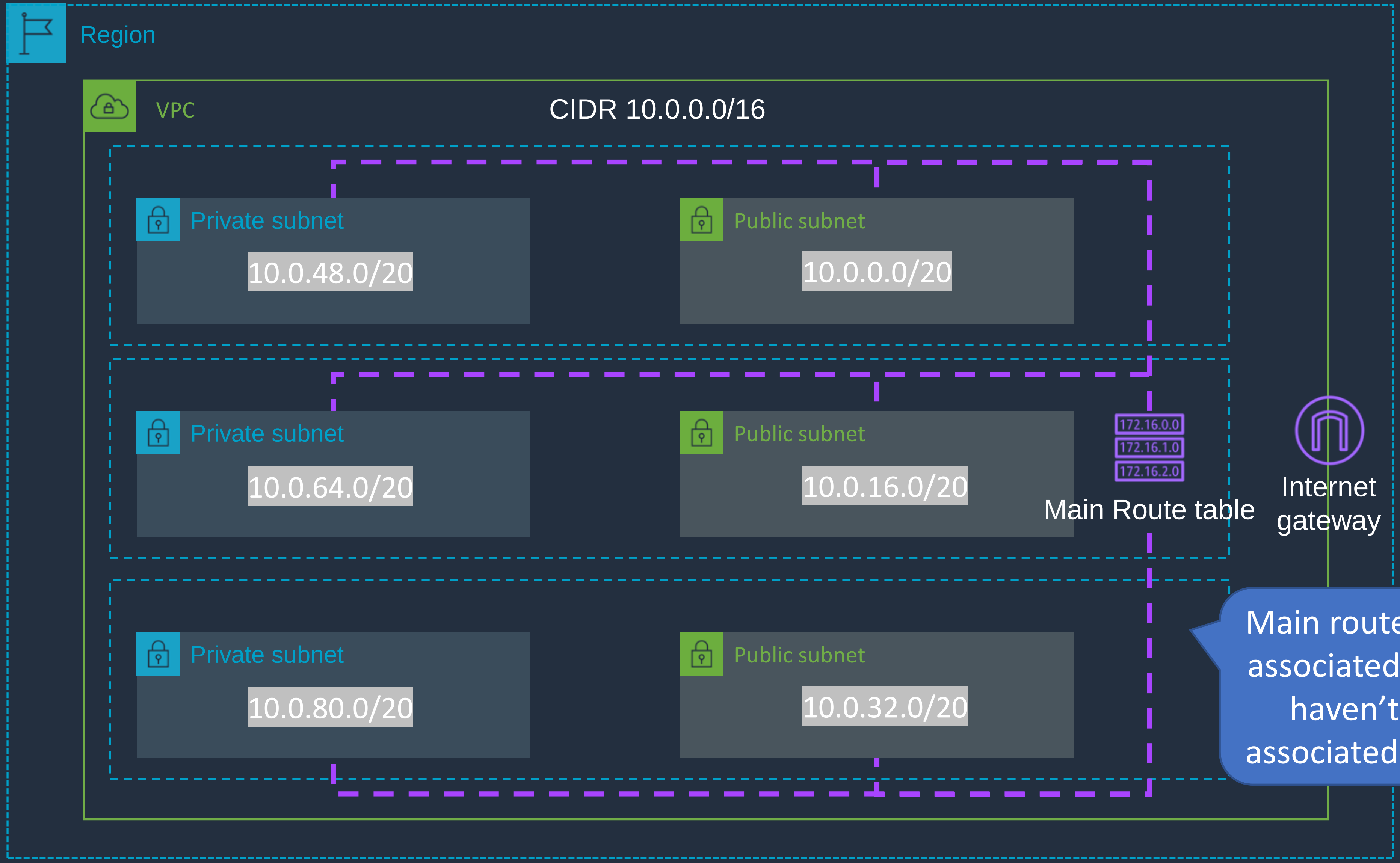
Private Route Table

Destination	Target
10.0.0.0/16	Local

VPC Routing Deep Dive

172.16.0.0
172.16.1.0
172.16.2.0

VPC Routing Deep Dive

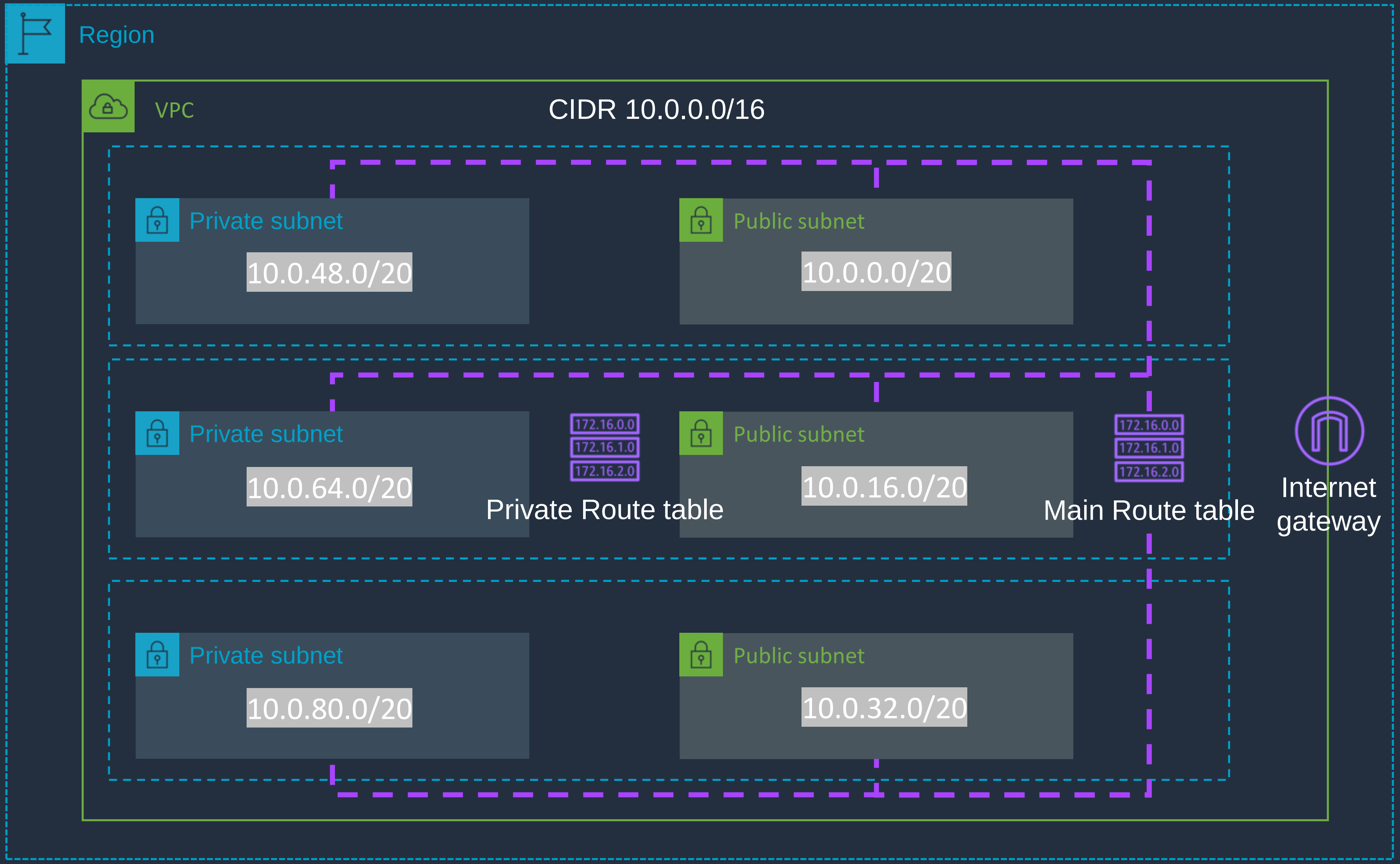


Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Main route table is **implicitly** associated with subnets that haven't been **explicitly** associated with a route table

VPC Routing Deep Dive



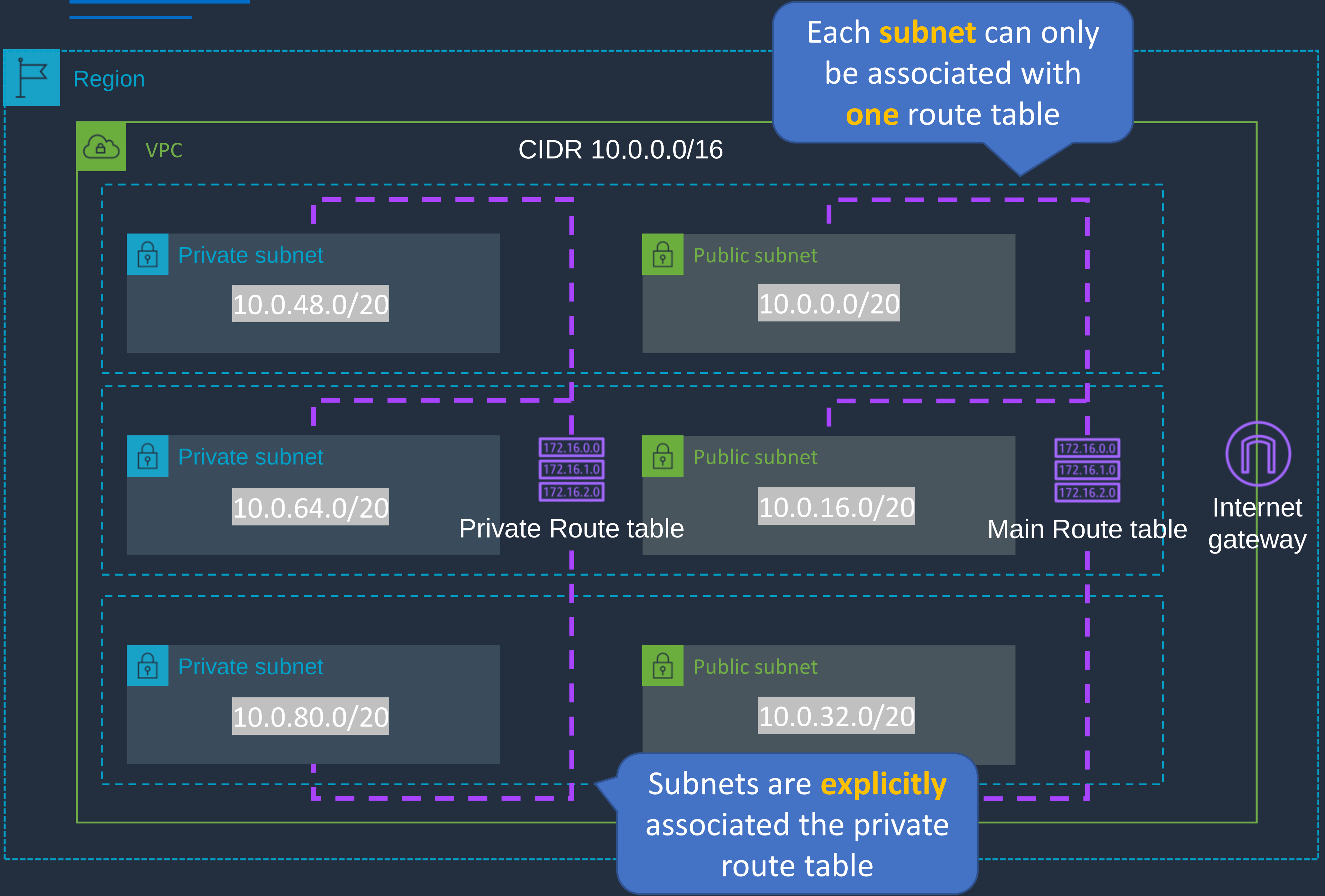
Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

Destination	Target
10.0.0.0/16	Local

VPC Routing Deep Dive



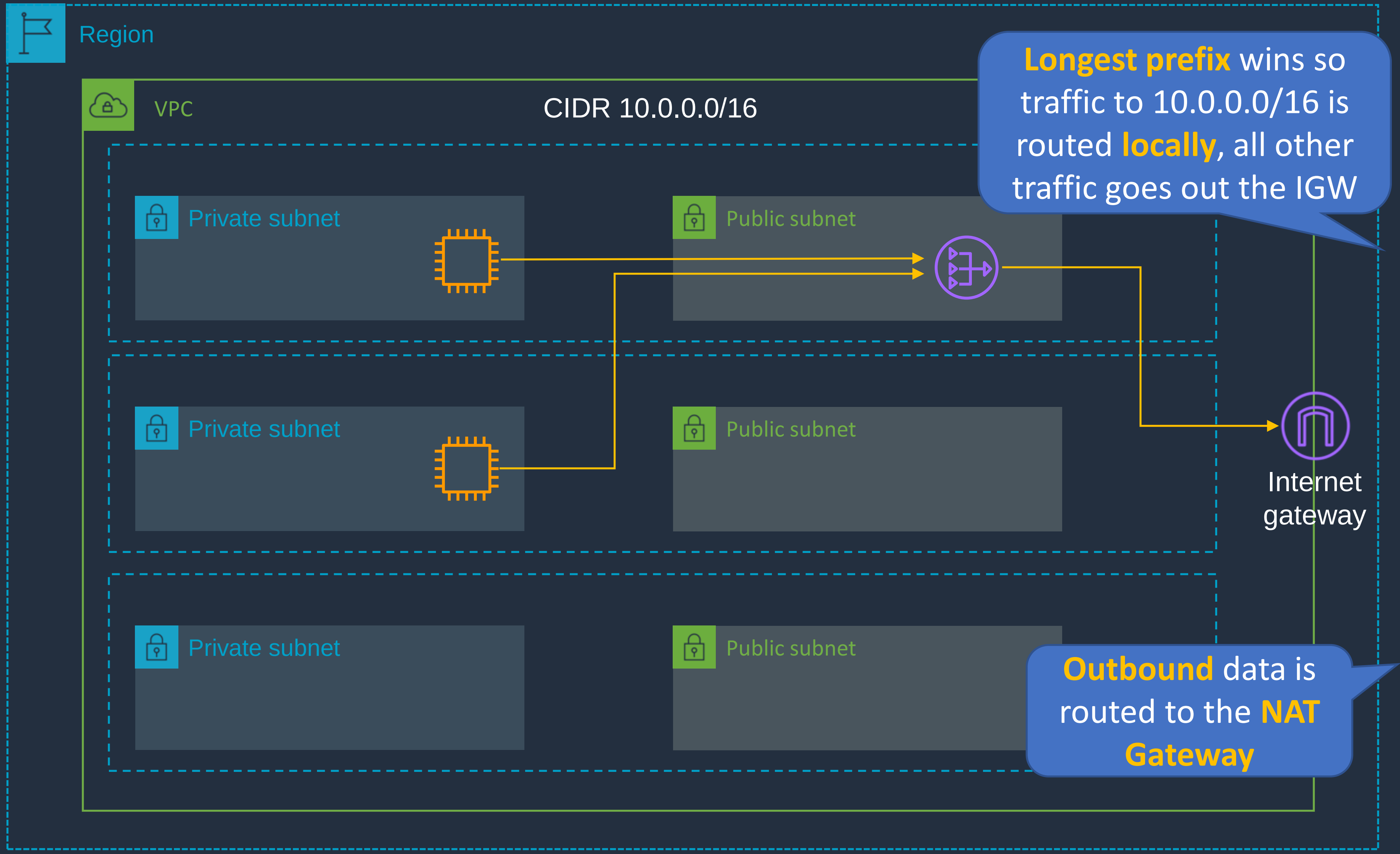
Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

Destination	Target
10.0.0.0/16	Local

VPC Routing Deep Dive



Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

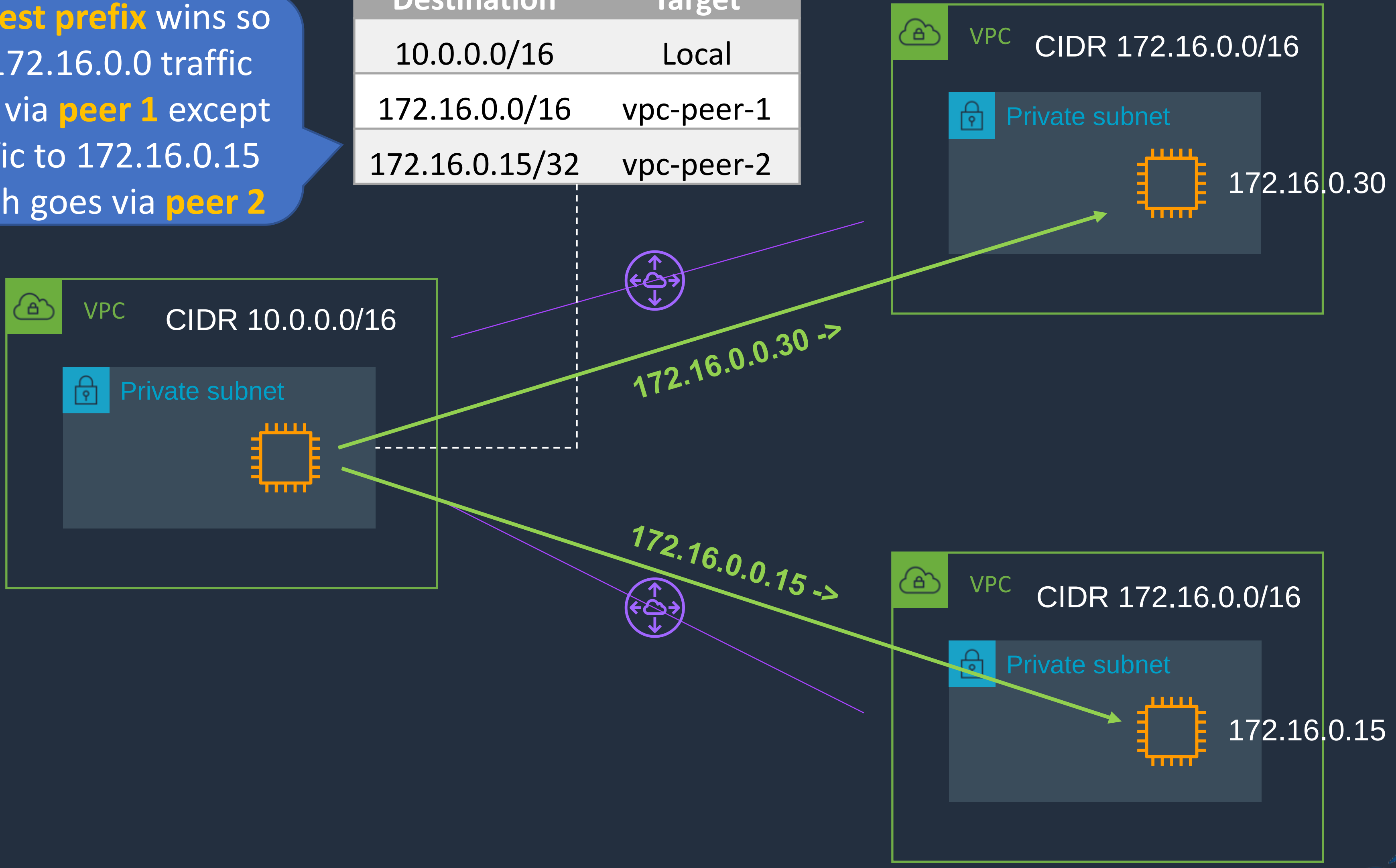
Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	nat-gw-id

DigitalCloud TRAINING

VPC Routing Deep Dive

Longest prefix wins so all 172.16.0.0 traffic goes via **peer 1** except traffic to 172.16.0.15 which goes via **peer 2**

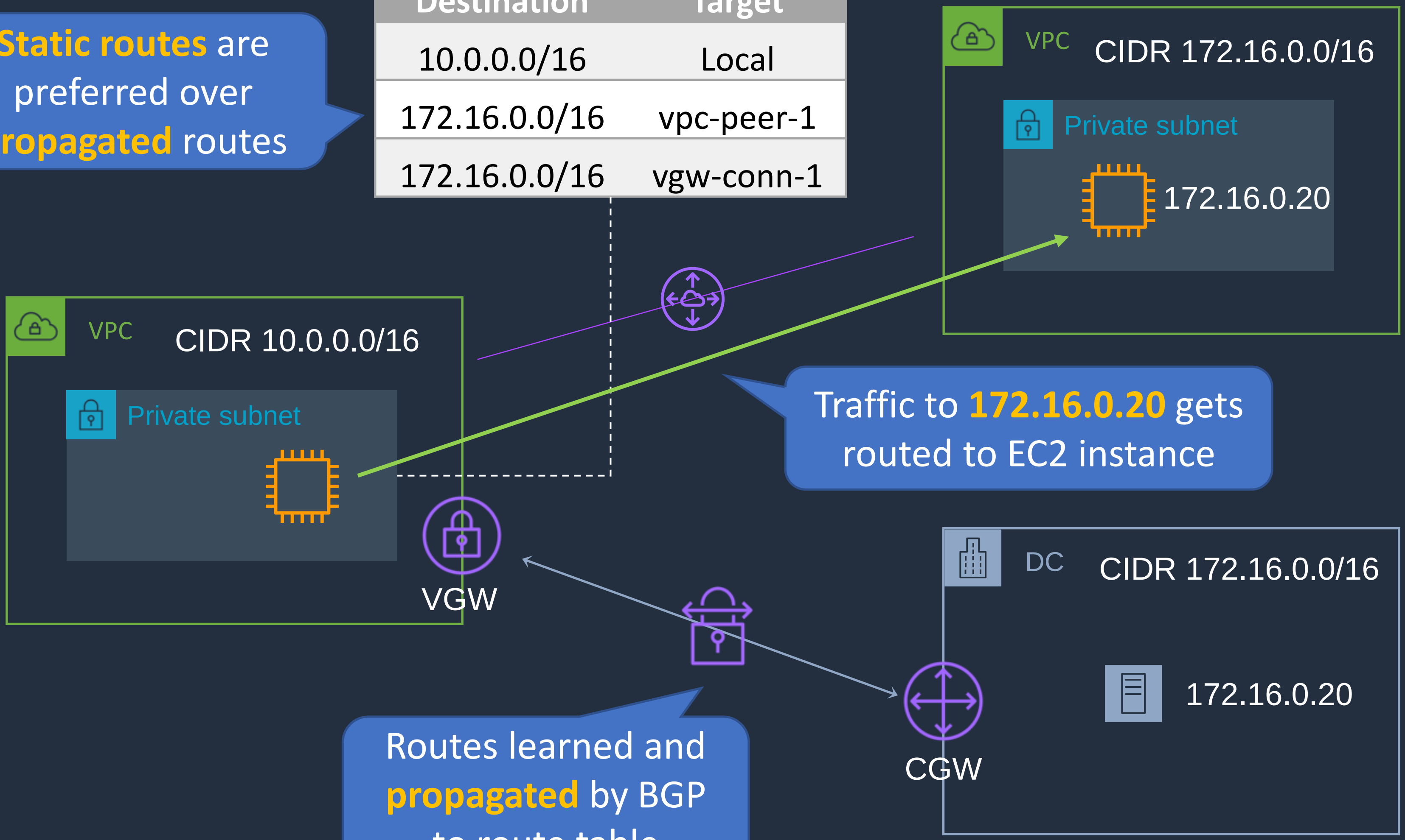
Destination	Target
10.0.0.0/16	Local
172.16.0.0/16	vpc-peer-1
172.16.0.15/32	vpc-peer-2



VPC Routing Deep Dive

Static routes are preferred over propagated routes

Destination	Target
10.0.0.0/16	Local
172.16.0.0/16	vpc-peer-1
172.16.0.0/16	vgw-conn-1



Traffic to **172.16.0.20** gets routed to EC2 instance

Routes learned and **propagated** by BGP to route table

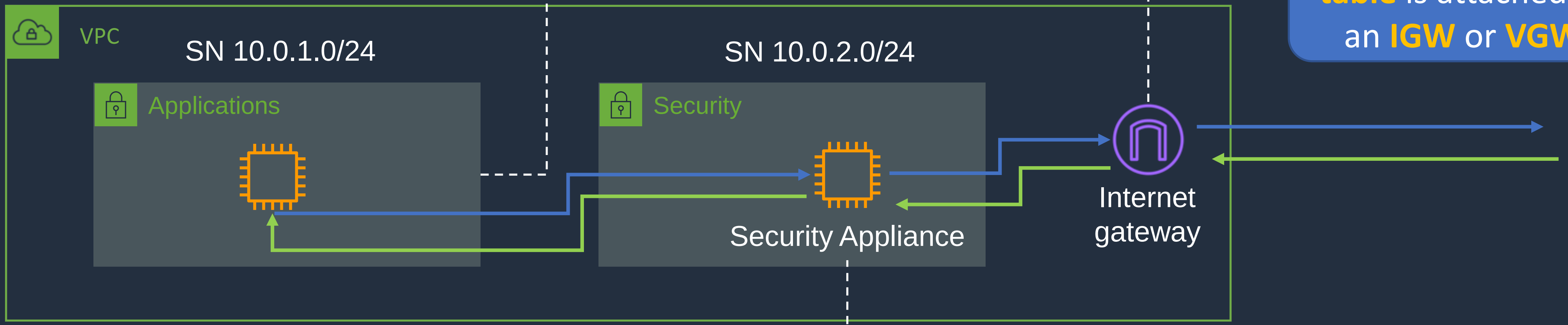
Gateway Route Tables

0.0.0.0/0 points to the ENI ID of the **security appliance**

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	eni-id-sec

Destination	Target
10.0.0.0/16	Local
10.0.1.0/24	eni-id-sec

A **Gateway route table** is attached to an **IGW** or **VGW**



All **outbound** traffic forwarded to **IGW**

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

IPv4 and IPv6 Routing

Destination	Target
10.0.0.0/16	Local
2001:db8:1234:1a00::/56	Local
172.31.0.0/16	pcx-11223344556677889
0.0.0.0/0	igw-12345678901234567
::/0	eigw-aabbccddee1122334

IPv6 traffic within the VPC is routed **locally**

IPv4 traffic within the VPC is routed **locally**

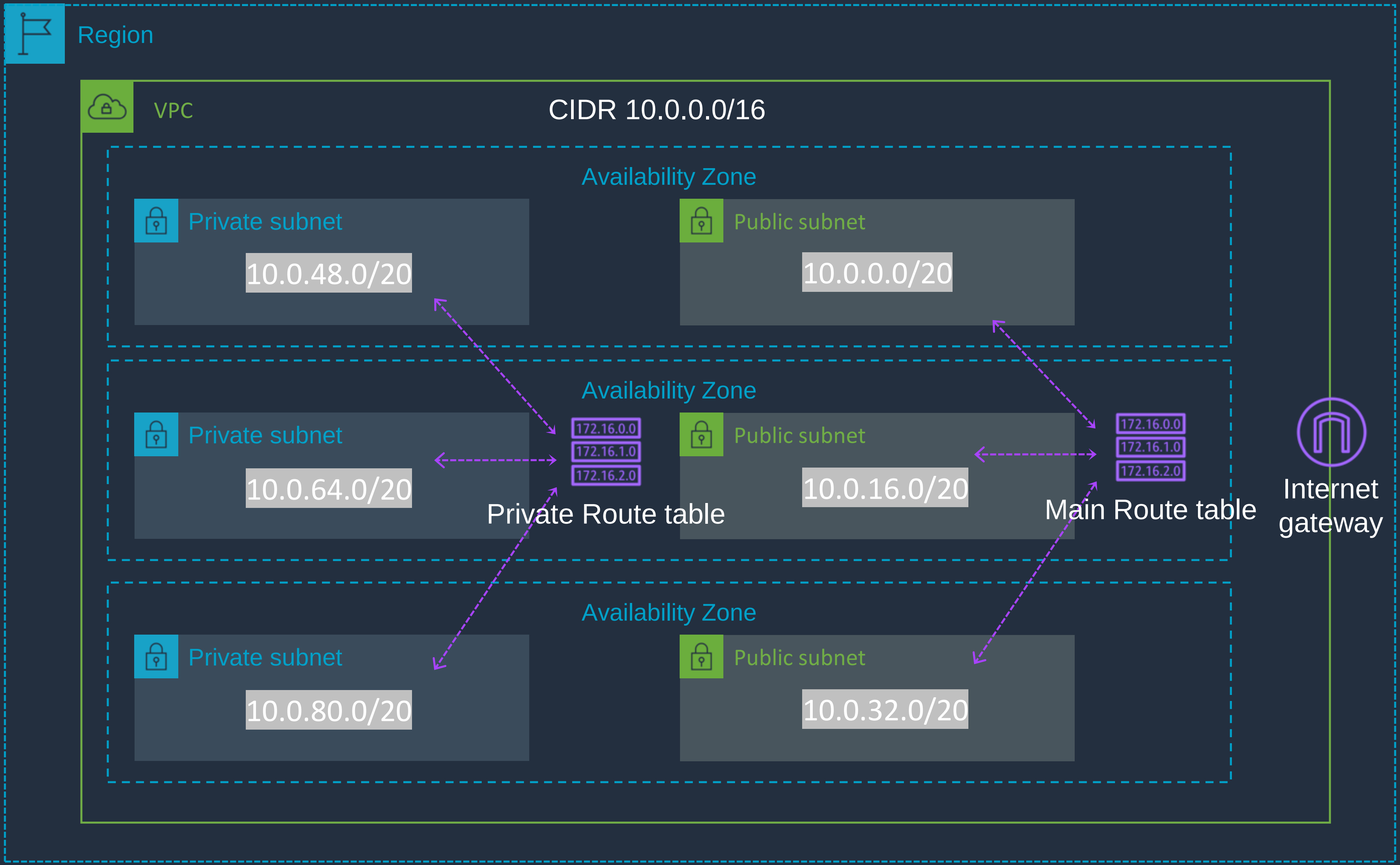
IPv4 traffic for 172.31.0.0/16 network goes via a **peering connection**

Traffic that doesn't match a more specific route goes via the **IGW**

IPv6 traffic that doesn't match a more specific route goes via the **EIGW**

Configure Routing

Create a Custom VPC



Main Route Table

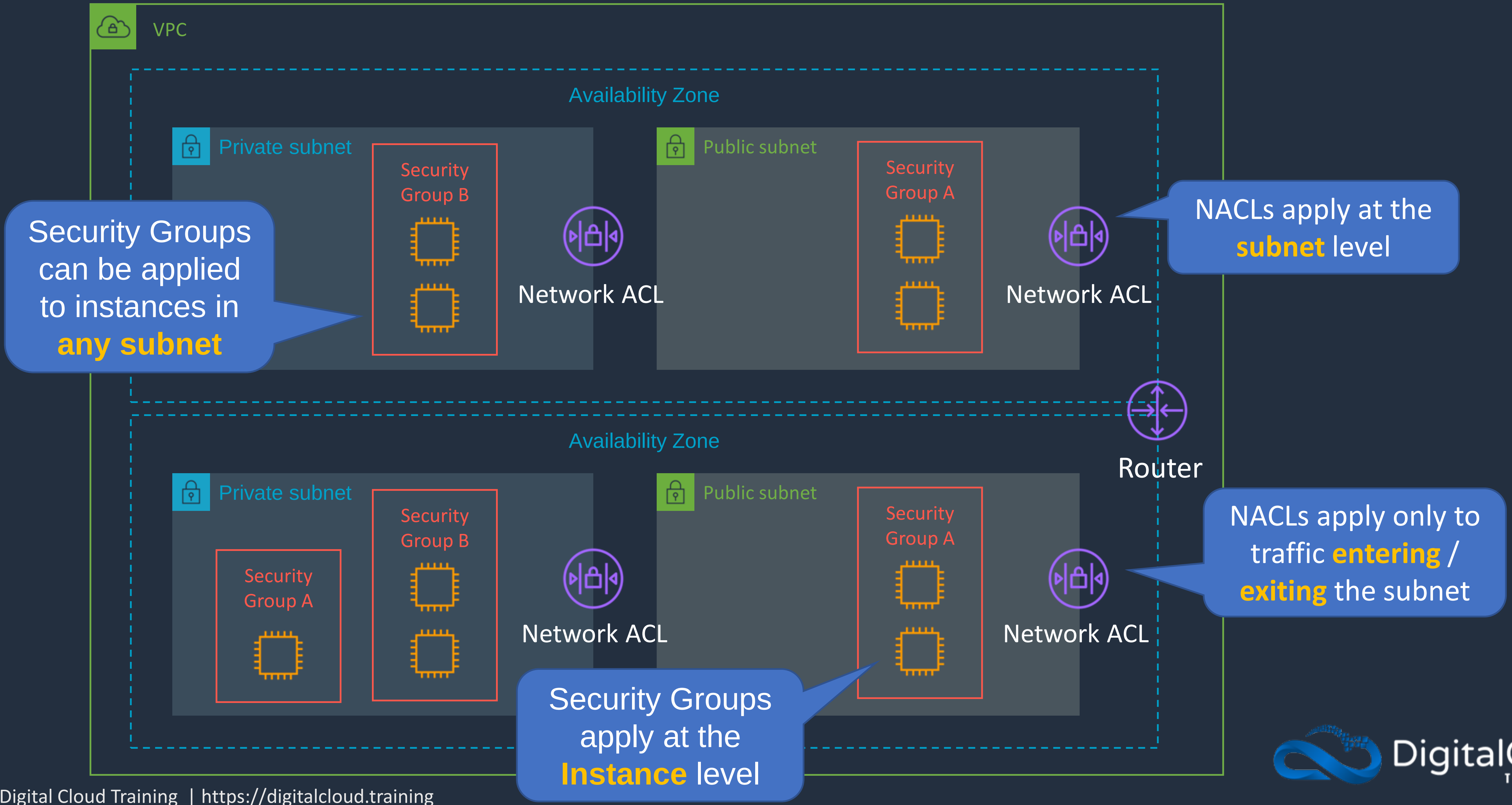
Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

Destination	Target
10.0.0.0/16	Local

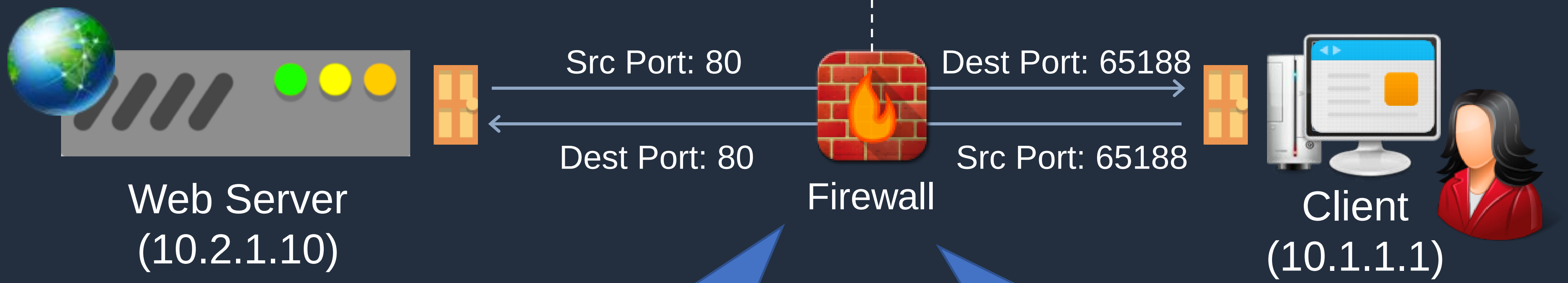
Security Groups and Network ACLs

Security Groups and Network ACLs



Stateful vs Stateless Firewalls

PROTOCOL	SOURCE IP	DESTINATION IP	SOURCE PORT	DESTINATION PORT
HTTP	10.1.1.1	10.2.1.10	65188	80
HTTP	10.2.1.10	10.1.1.1	80	65188



A stateful firewall allows the return traffic automatically

A stateless firewall checks for an allow rule for both connections

Security Group Rules

Security groups support **allow** rules only

Separate rules are defined for outbound traffic

A source can be an **IP address** or **security group ID**

Inbound rules

Type	Protocol	Port range	Source
SSH	TCP	22	0.0.0.0/0
RDP	TCP	3389	0.0.0.0/0
RDP	TCP	3389	::/0
HTTPS	TCP	443	0.0.0.0/0
HTTPS	TCP	443	::/0
All ICMP - IPv4	ICMP	All	0.0.0.0/0

Security Groups Best Practice

Public subnet(s)

Security group – PublicALB

Inbound: Protocol/Port HTTP/80 Source: 0.0.0.0/0
Outbound: Protocol/Port HTTPS:80 Destination: PublicEC2

Internet-facing ALB

Security group – PublicEC2

Inbound: Protocol/Port HTTP/80 Source: PublicALB
Outbound: Protocol/Port HTTPS/8080 Destination: PrivateALB

Web Front-End

Private subnet(s)

Security group – PrivateALB

Inbound: Protocol/Port HTTP/8080 Source: PublicEC2
Outbound: Protocol/Port HTTPS/8080 Destination: PrivateEC2

Internal ALB

Security group – PrivateEC2

Inbound: Protocol/Port HTTP/8080 Source: PrivateALB

Application Layer

Network ACLs

Inbound Rules

Rule #	Type	Protocol	Port Range	Source	Allow / Deny
100	ALL Traffic	ALL	ALL	0.0.0.0/0	ALLOW
101	ALL Traffic	ALL	ALL	::/0	ALLOW
*	ALL Traffic	ALL	ALL	0.0.0.0/0	DENY
*	ALL Traffic	ALL	ALL	::/0	DENY

NACLs have an explicit deny

Outbound Rules

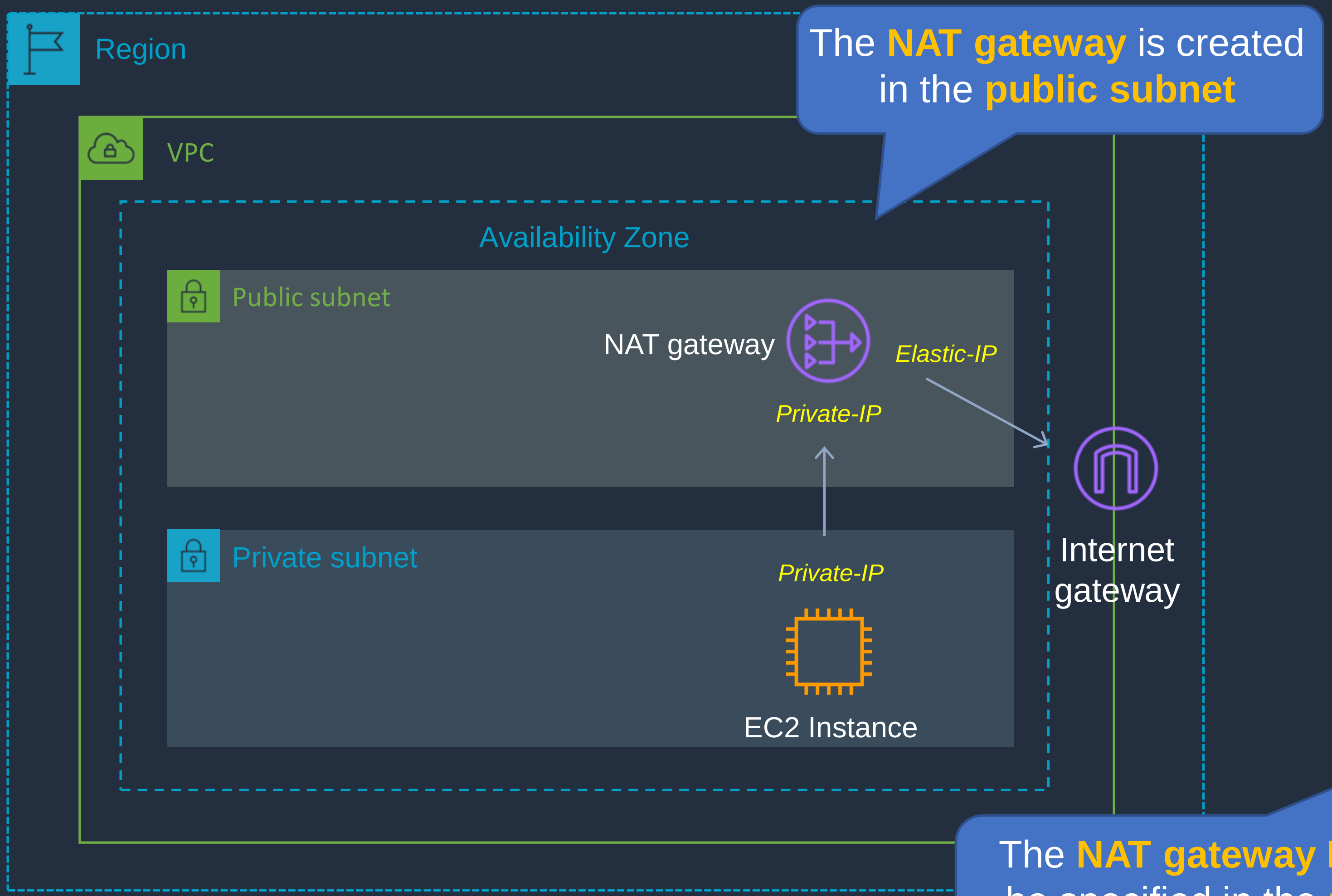
Rule #	Type	Protocol	Port Range	Destination	
100	ALL Traffic	ALL	ALL	0.0.0.0/0	ALLOW
101	ALL Traffic	ALL	ALL	::/0	ALLOW
*	ALL Traffic	ALL	ALL	0.0.0.0/0	DENY
*	ALL Traffic	ALL	ALL	::/0	DENY

Rules are processed in order

Configure Security Groups and NACLs

NAT Gateways and NAT Instances

NAT Gateways



Main Route Table

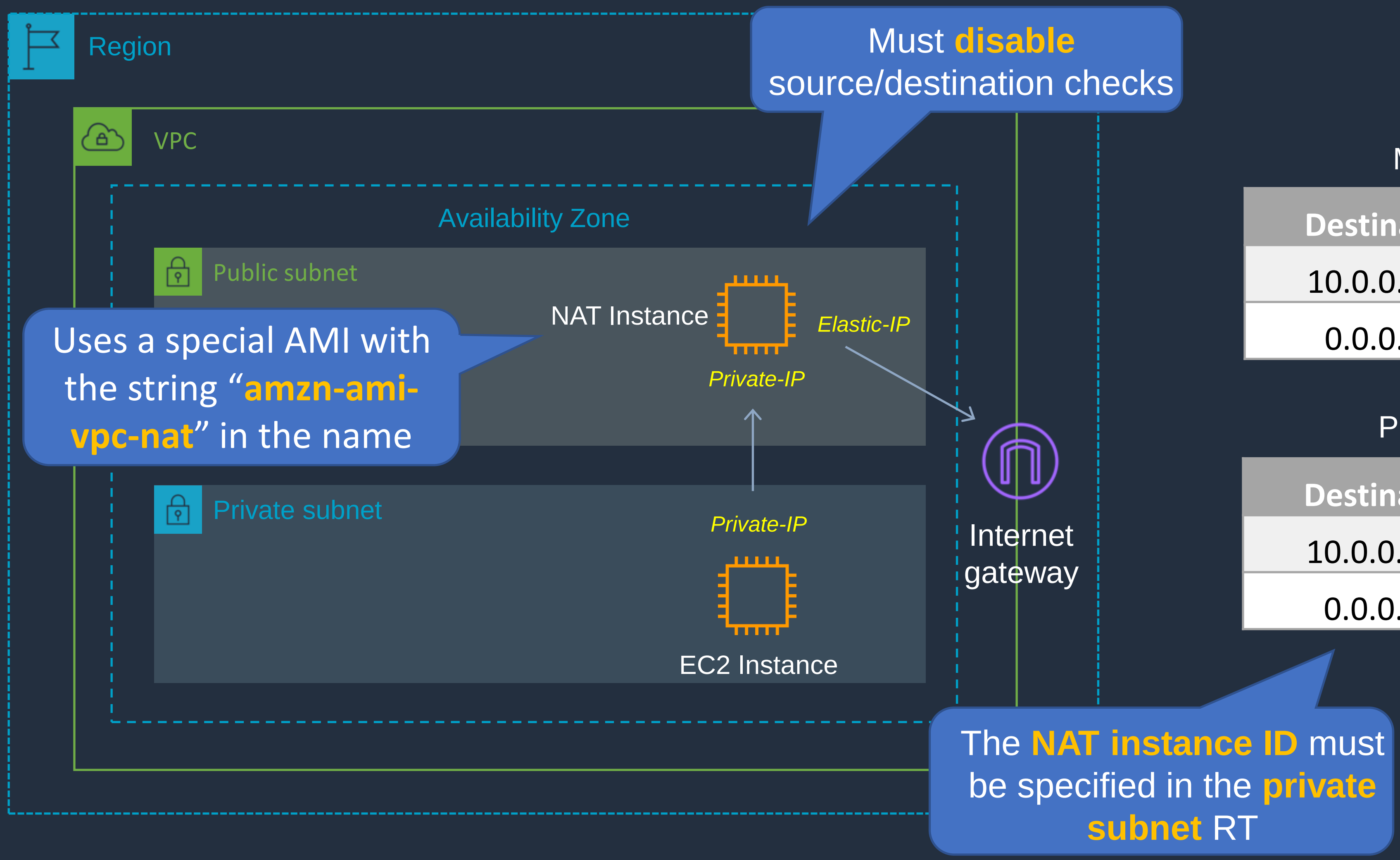
Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	nat-gateway-id

The **NAT gateway ID** must be specified in the **private subnet** RT

NAT Instances



Main Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

Private Route Table

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	nat-instance-id

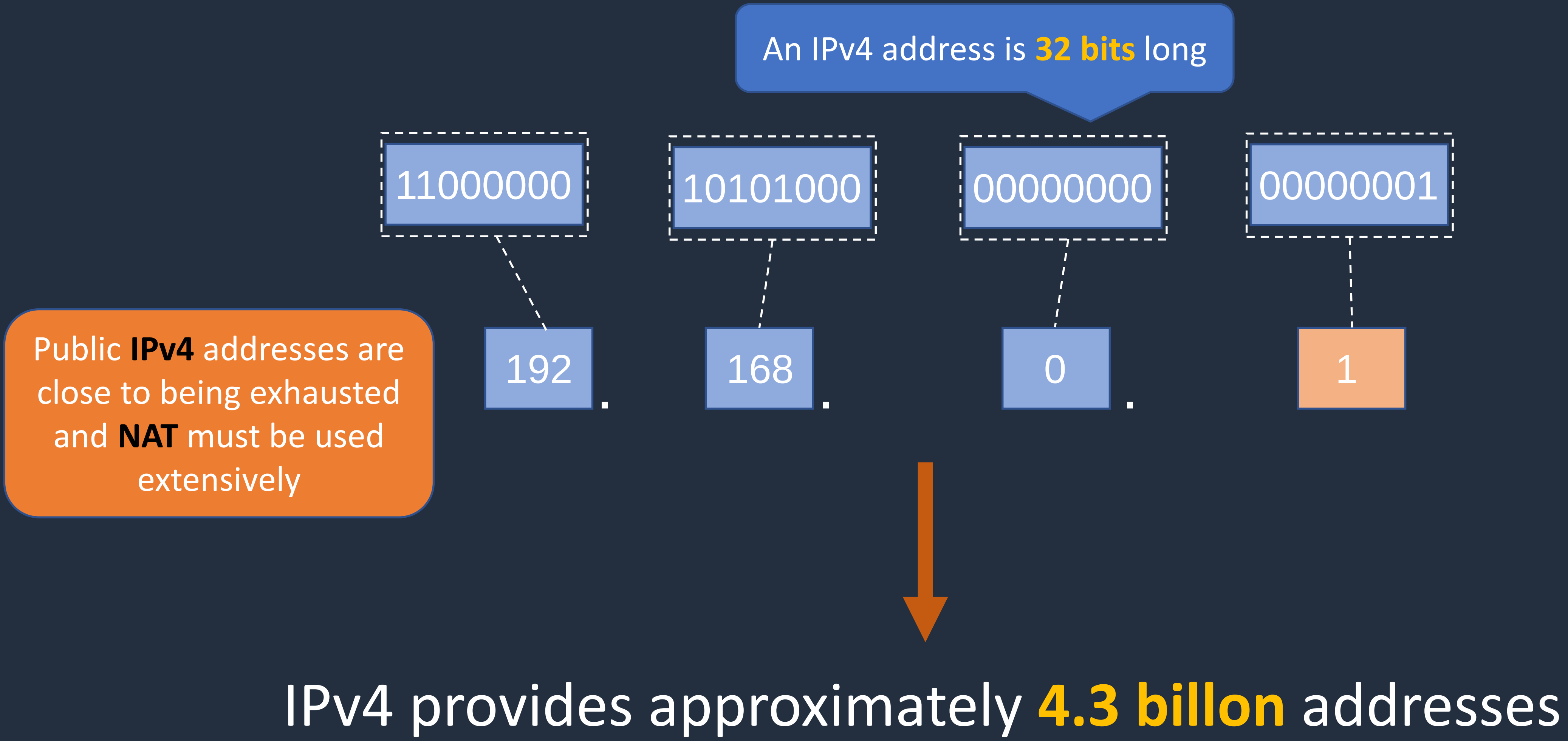
NAT Instance vs NAT Gateway

NAT Instance	NAT Gateway
Managed by you (e.g. software updates)	Managed by AWS
Scale up (instance type) manually and use enhanced networking	Elastic scalability up to 45 Gbps
No high availability – scripted/auto-scaled HA possible using multiple NATs in multiple subnets	Provides automatic high availability within an AZ and can be placed in multiple AZs
Need to assign Security Group	No Security Groups
Can use as a bastion host	Cannot access through SSH
Use an Elastic IP address or a public IP address with a NAT instance	Choose the Elastic IP address to associate with a NAT gateway at creation
Can implement port forwarding through manual customisation	Does not support port forwarding

Private Subnet with NAT Gateway

Using IPv6 in a VPC

Using IPv6 in a VPC



Using IPv6 in a VPC

An IPv6 address is **128 bits** long

2020 : 0001 : 9d32 : 5bc2 : 1c48 : 32c1 : a93b : b12c

Network Part | Node Part

An **IPv6** addresses use **hexadecimal** whereas **IPv4** addresses use **dotted decimal**

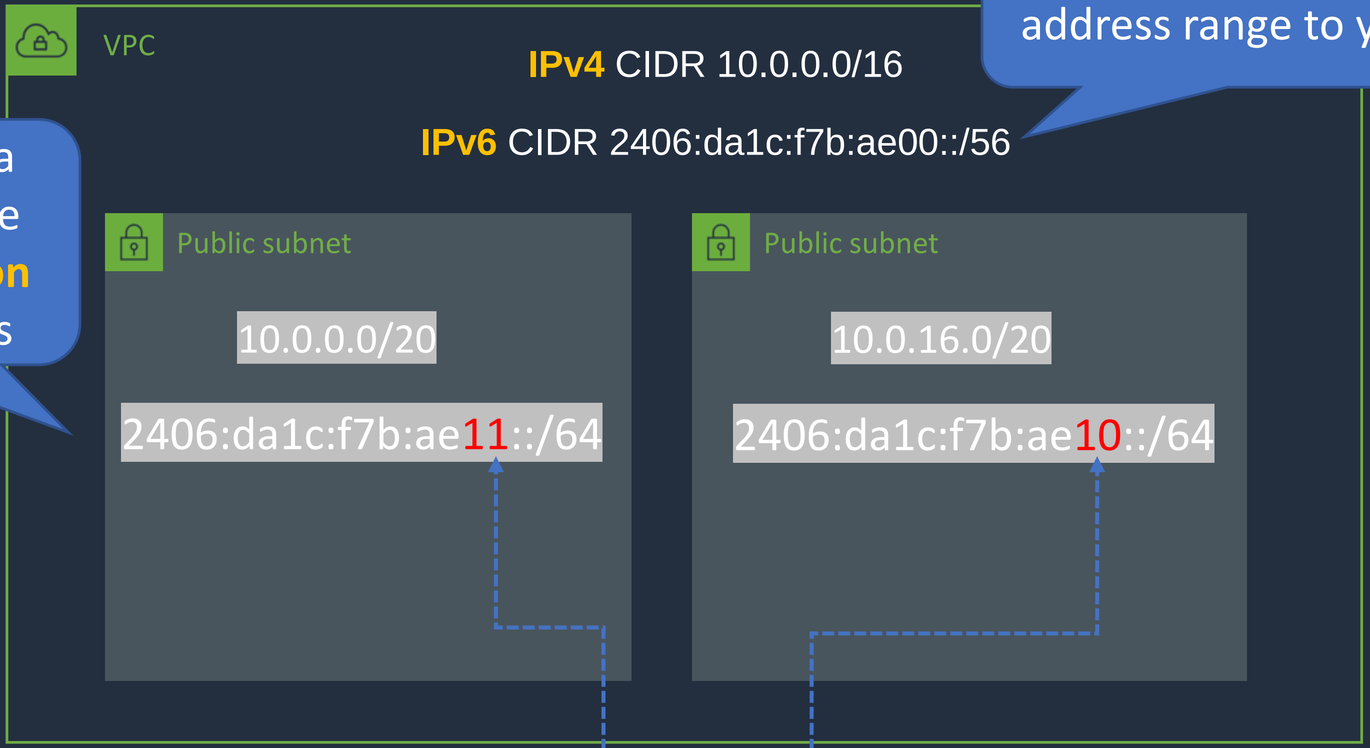
That's enough to assign more than **100 IPv6 addresses** to **every atom** on earth!!!

IPv6 provides **340,282,366,920,938,463,463,374,607,431,768,211,456** addresses

Using IPv6 in a VPC

AWS assign a **/56** IPv6 address range to your VPC

Subnets receive a **/64** address range allowing **18 million trillion** addresses



Route Table

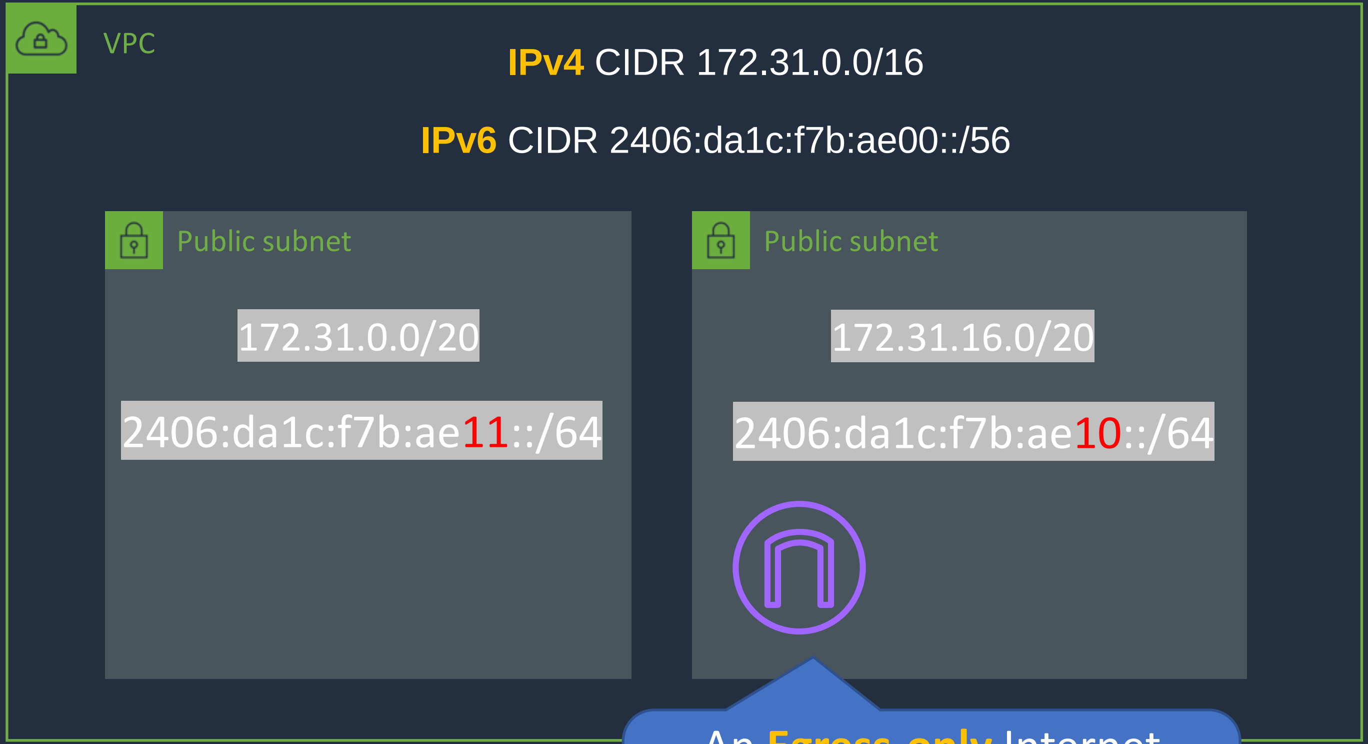
Destination	Target
10.0.0.0/16	Local
2406:da1c:f7b:ae00::/56	Local
0.0.0.0/0	igw-id
::/0	igw-id

Route table has an entry to send all **external IPv6 traffic** to the **IGW**

All **IPv6** addresses are **publicly routable** (no NAT)

A **hexadecimal pair** is assigned for each subnet – values from **00 – FF** = **256 /64** subnets

Using IPv6 in a VPC



Route Table

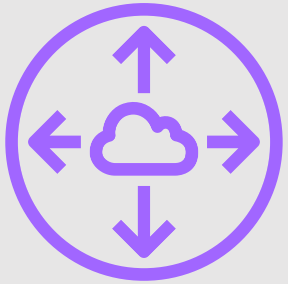
Destination	Target
172.31.0.0/16	Local
0.0.0.0/0	igw-id
::/0	eo-igw-id

An **Egress-only** Internet Gateway allows IPv6 traffic **outbound** but **not inbound**

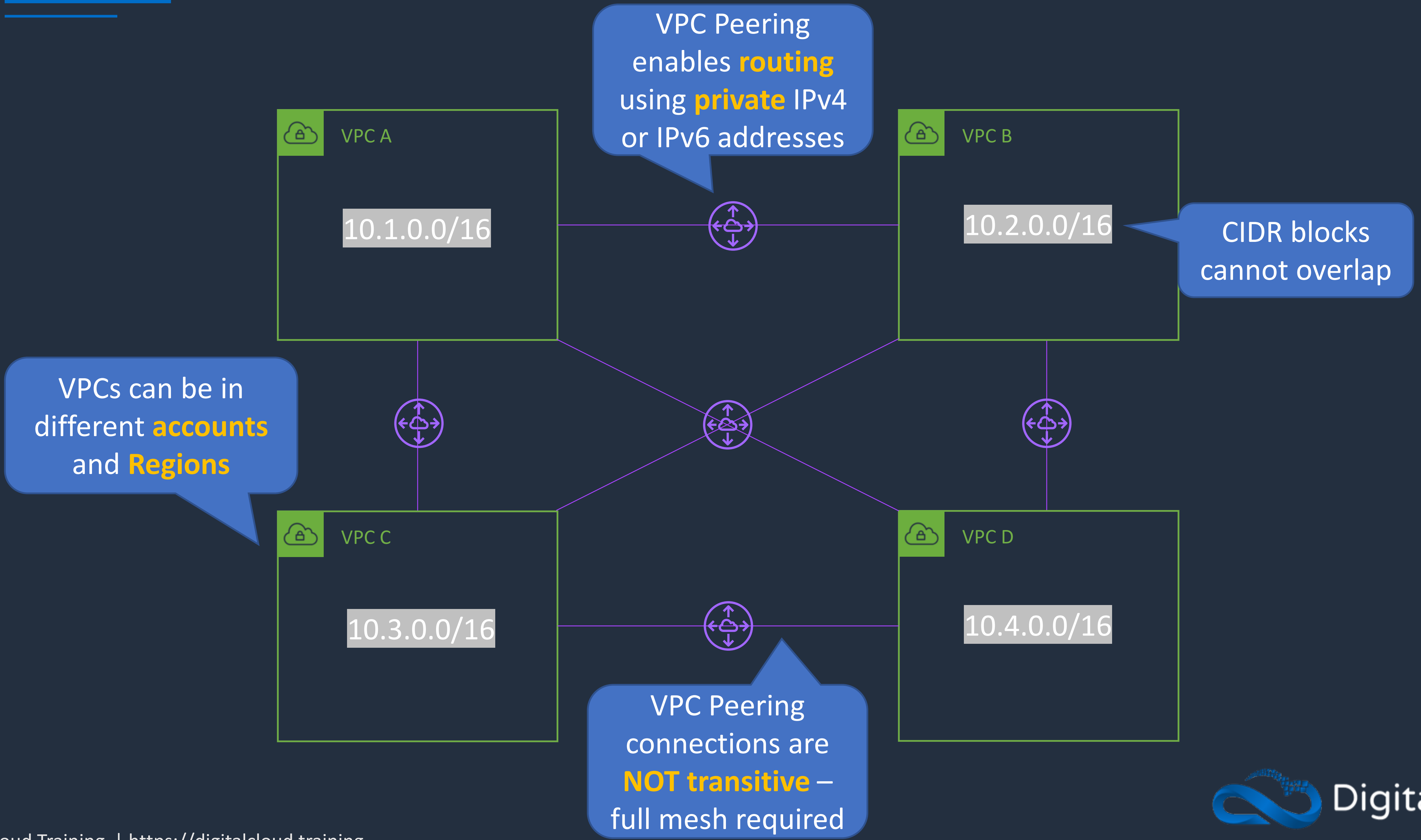
All **IPv6** addresses are **publicly routable** (no NAT)

Configure IPv6

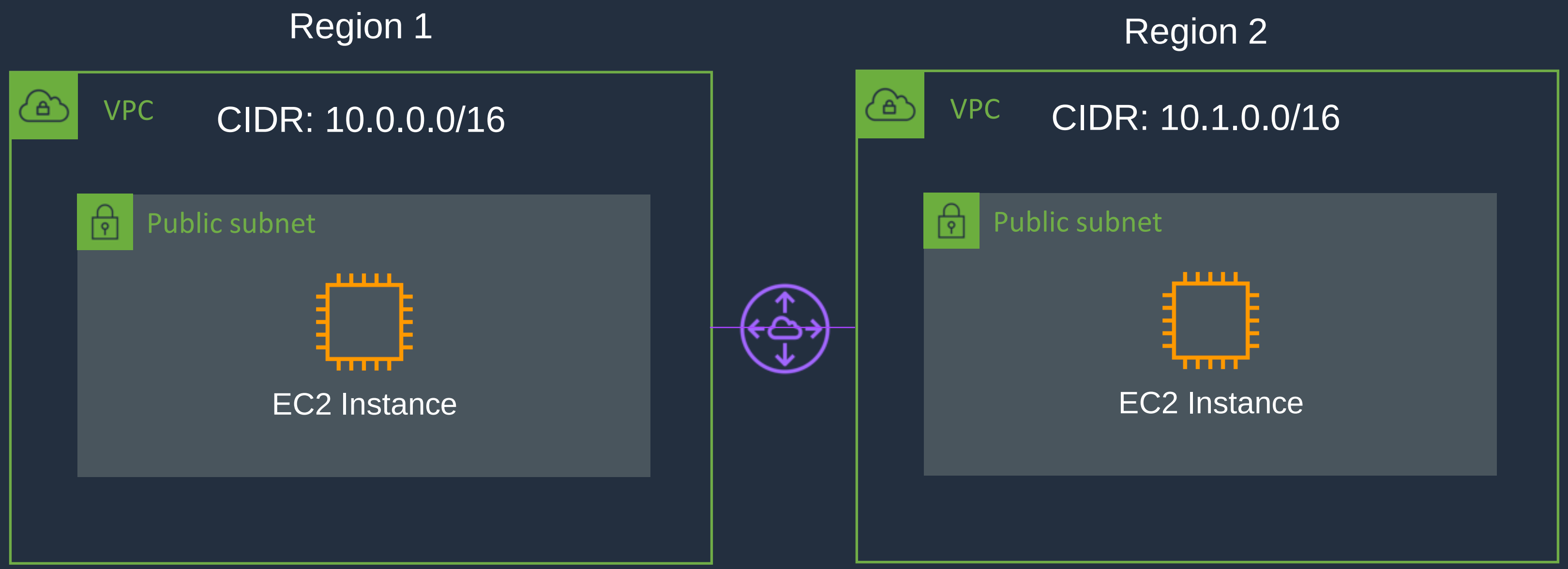
VPC Peering



VPC Peering


VPC Peering enables **routing** using **private** IPv4 or IPv6 addresses
VPC A
10.1.0.0/16
VPC B
10.2.0.0/16
CIDR blocks cannot overlap
VPCs can be in different **accounts** and **Regions**
VPC C
10.3.0.0/16
VPC D
10.4.0.0/16
VPC Peering connections are **NOT transitive** – full mesh required

VPC Peering



Security group (Region1-SG)

Protocol	Port	Source
ICMP	All	10.1.0.0/16
TCP	22	0.0.0.0/0

Security group (Region2-SG)

Protocol	Port	Source
ICMP	All	10.0.0.0/16
TCP	22	0.0.0.0/0

Route Table

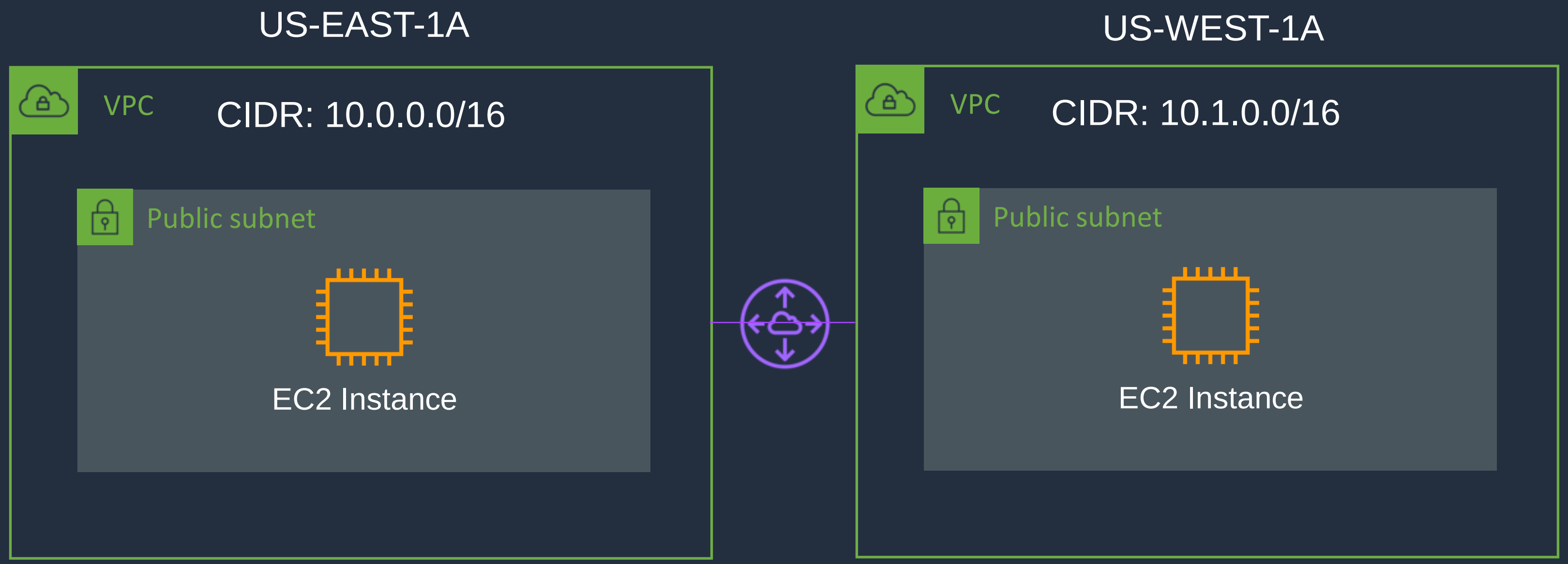
Destination	Target
10.1.0.0/16	peering-id

Route Table

Destination	Target
10.0.0.0/16	peering-id

Configure VPC Peering

VPC Peering



Security group

Protocol	Port	Source
ICMP	All	10.1.0.0/16
TCP	22	0.0.0.0/0

Security group

Protocol	Port	Source
ICMP	All	10.0.0.0/16
TCP	22	0.0.0.0/0

Route Table

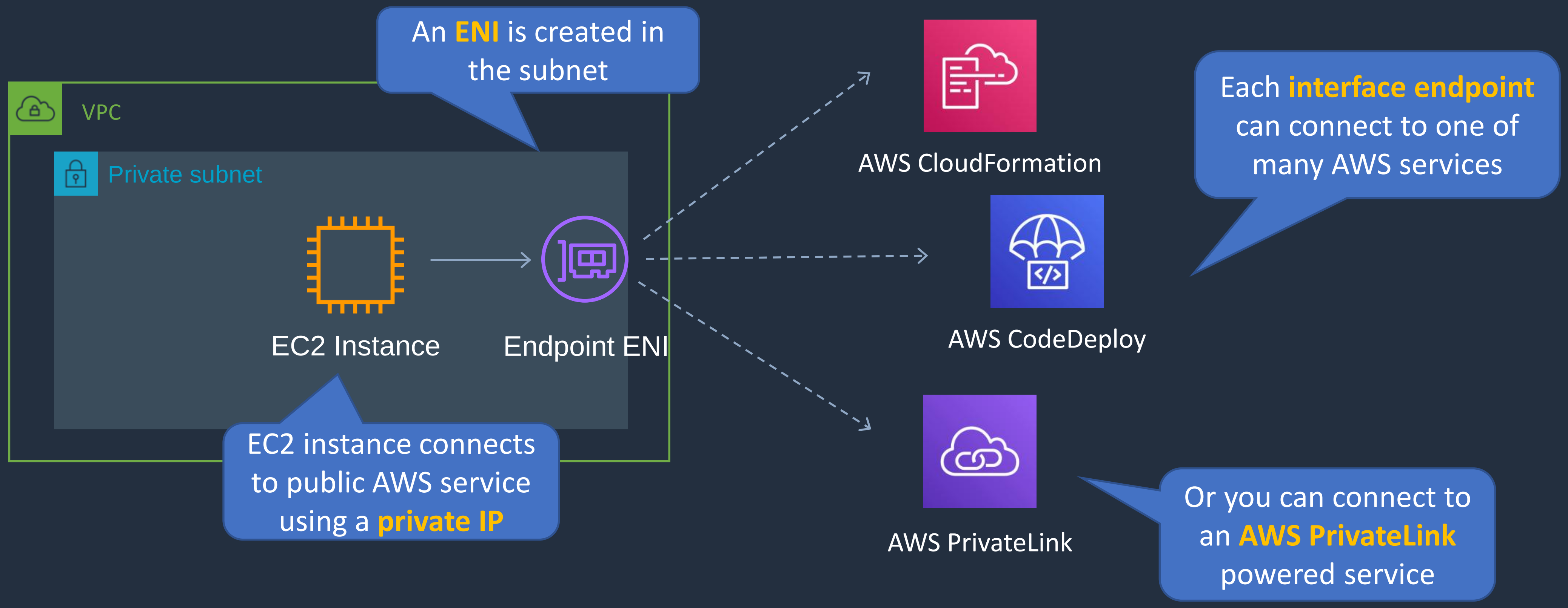
Destination	Target
10.1.0.0/16	peering-id

Route Table

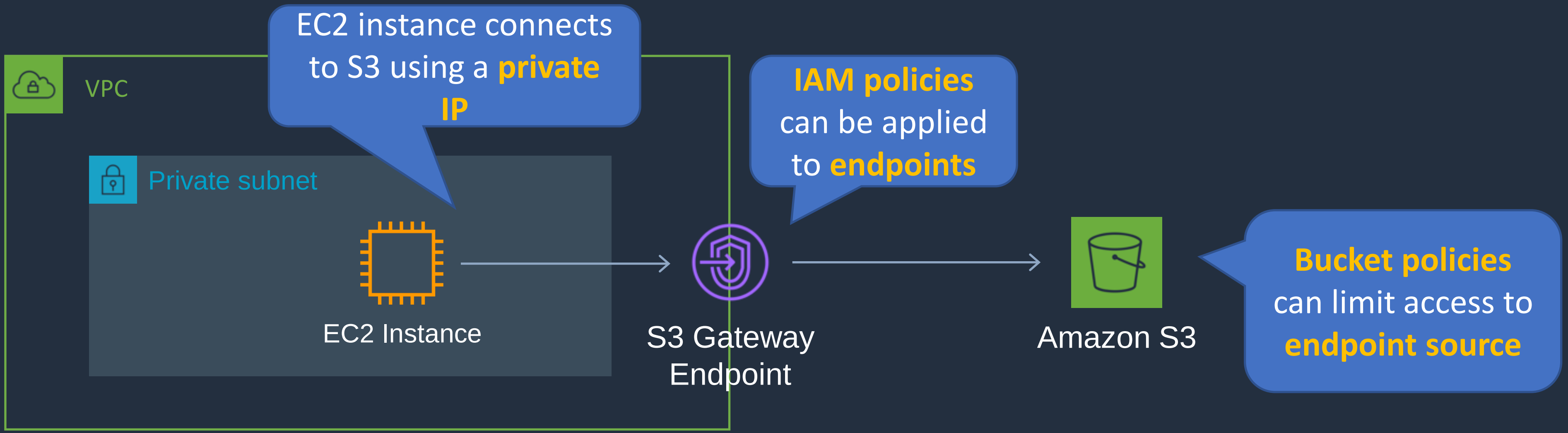
Destination	Target
10.0.0.0/16	peering-id

VPC Endpoints

VPC Interface Endpoints



VPC Gateway Endpoints



Route Table

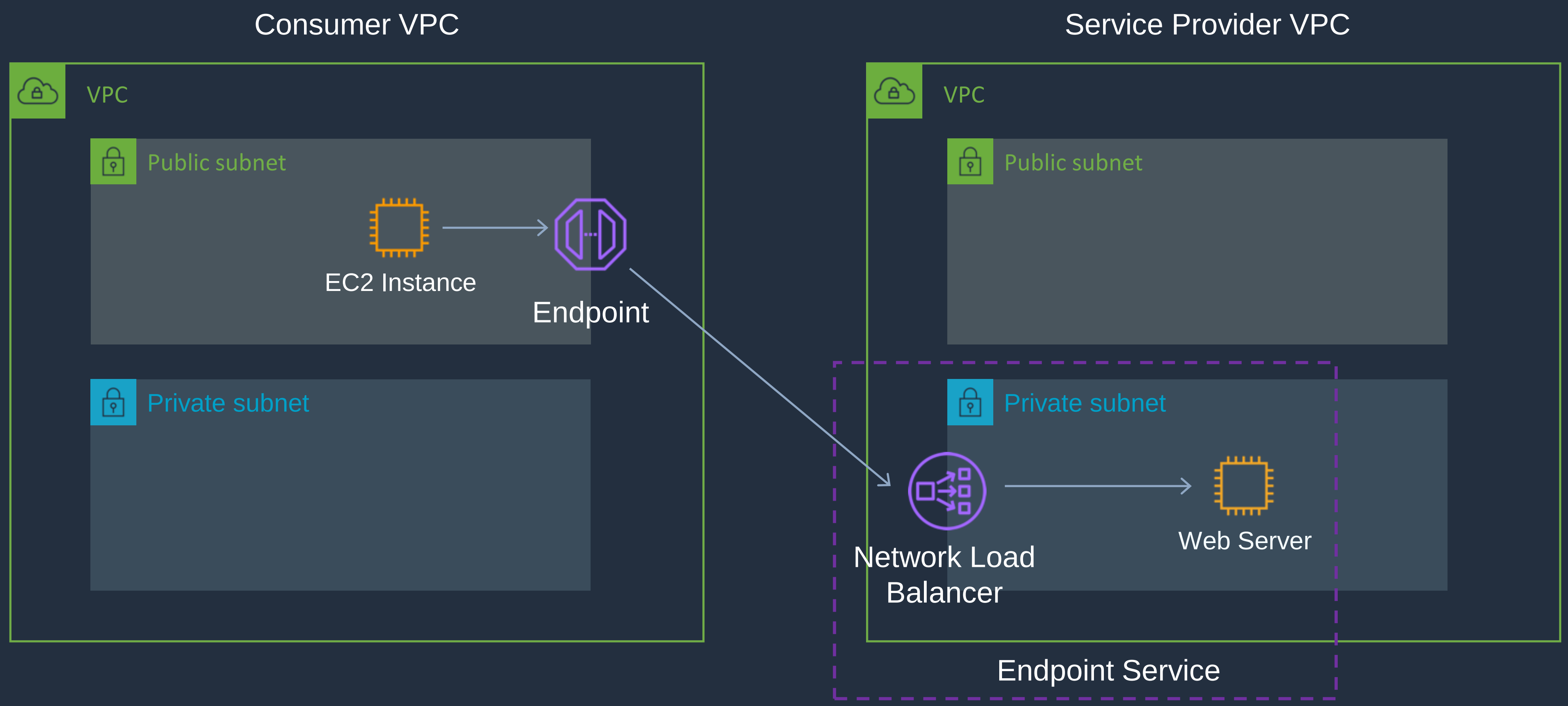
Destination	Target
pl-6ca54005 (com.amazonaws.ap-southeast-2.s3, 54.231.248.0/22, 54.231.252.0/24, 52.95.128.0/21)	*vpce-ID*

A **route table** entry is required with the prefix list for S3 and the **gateway ID**

VPC Endpoints

	Interface Endpoint	Gateway Endpoint
What	Elastic Network Interface with a Private IP	A gateway that is a target for a specific route
How	Uses DNS entries to redirect traffic	Uses prefix lists in the route table to redirect traffic
Which services	API Gateway, CloudFormation, CloudWatch etc.	Amazon S3, DynamoDB
Security	Security Groups	VPC Endpoint Policies

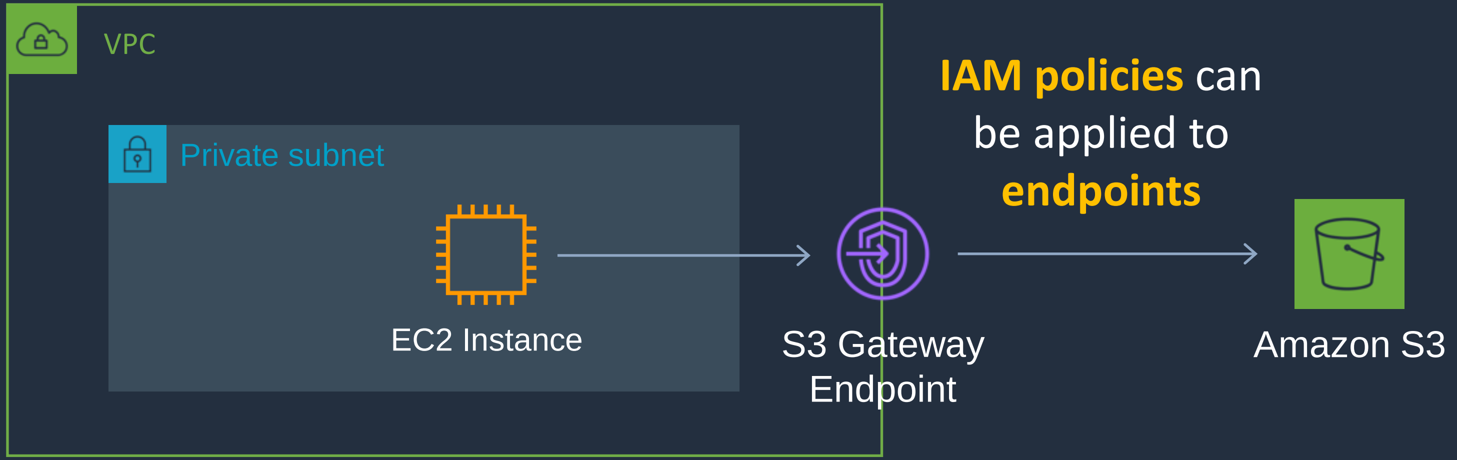
Service Provider Model



Create VPC Endpoint

VPC Gateway Endpoints

EC2 instance connects to S3 using a **private IP**



IAM policies can be applied to **endpoints**

Bucket policies can limit access to **endpoint source**

Route Table

Destination	Target
pl-6ca54005 (com.amazonaws.ap-southeast-2.s3, 54.231.248.0/22, 54.231.252.0/24, 52.95.128.0/21)	*vpce-ID*

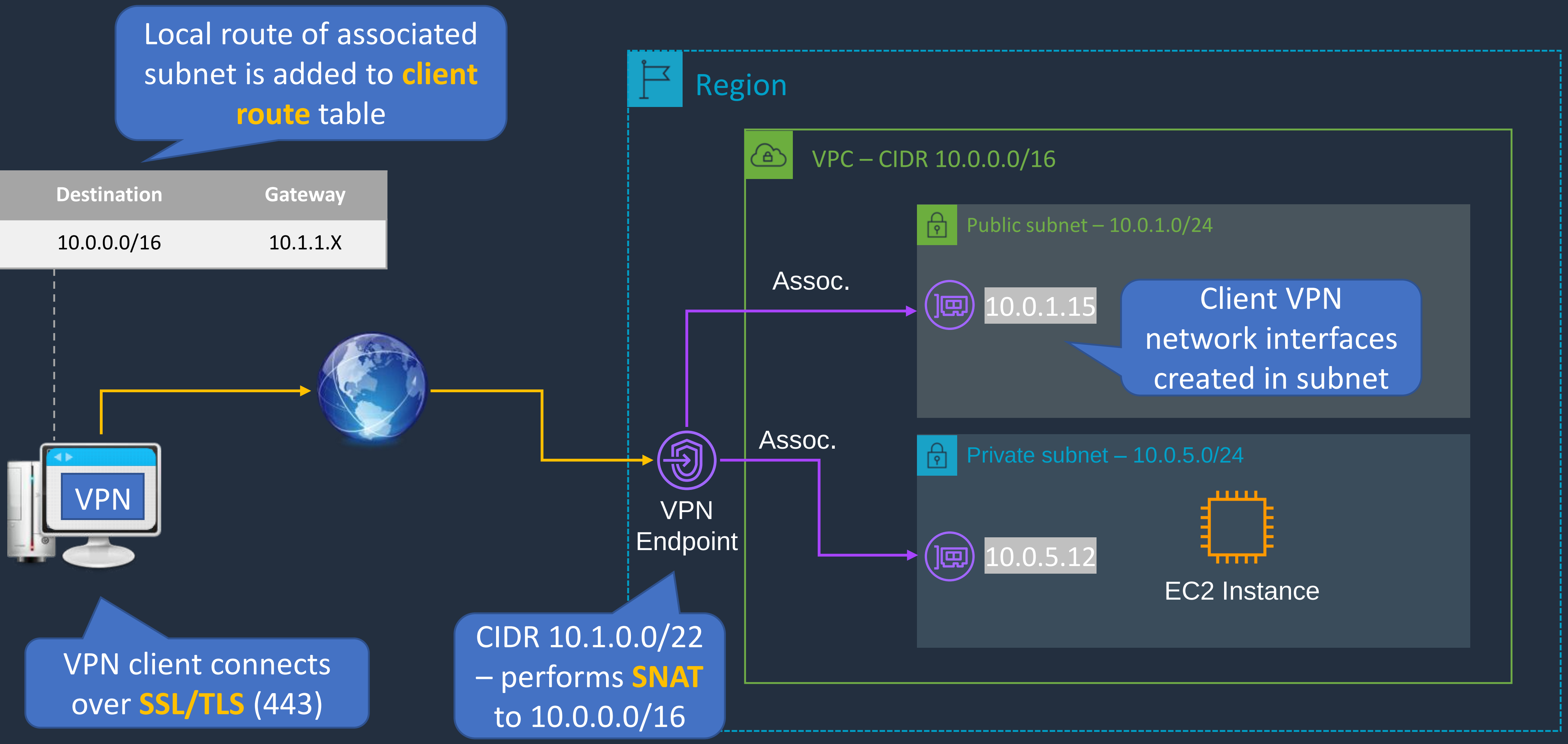
A **route table** entry is required with the prefix list for S3 and the **gateway ID**

SECTION 6

Hybrid Connectivity

AWS Client VPN

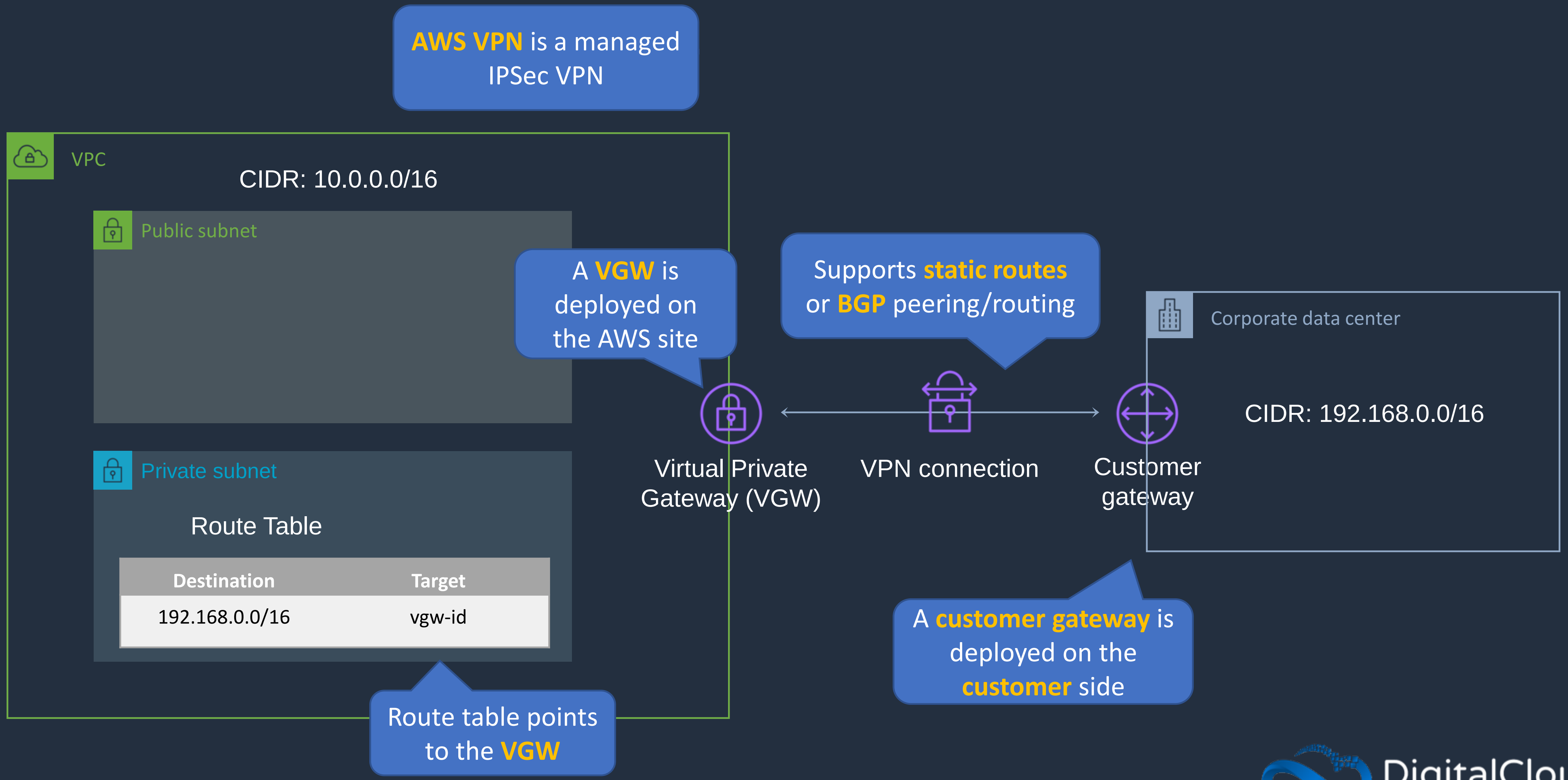
AWS Client VPN



Destination	Gateway
10.0.0.0/16	10.1.1.X

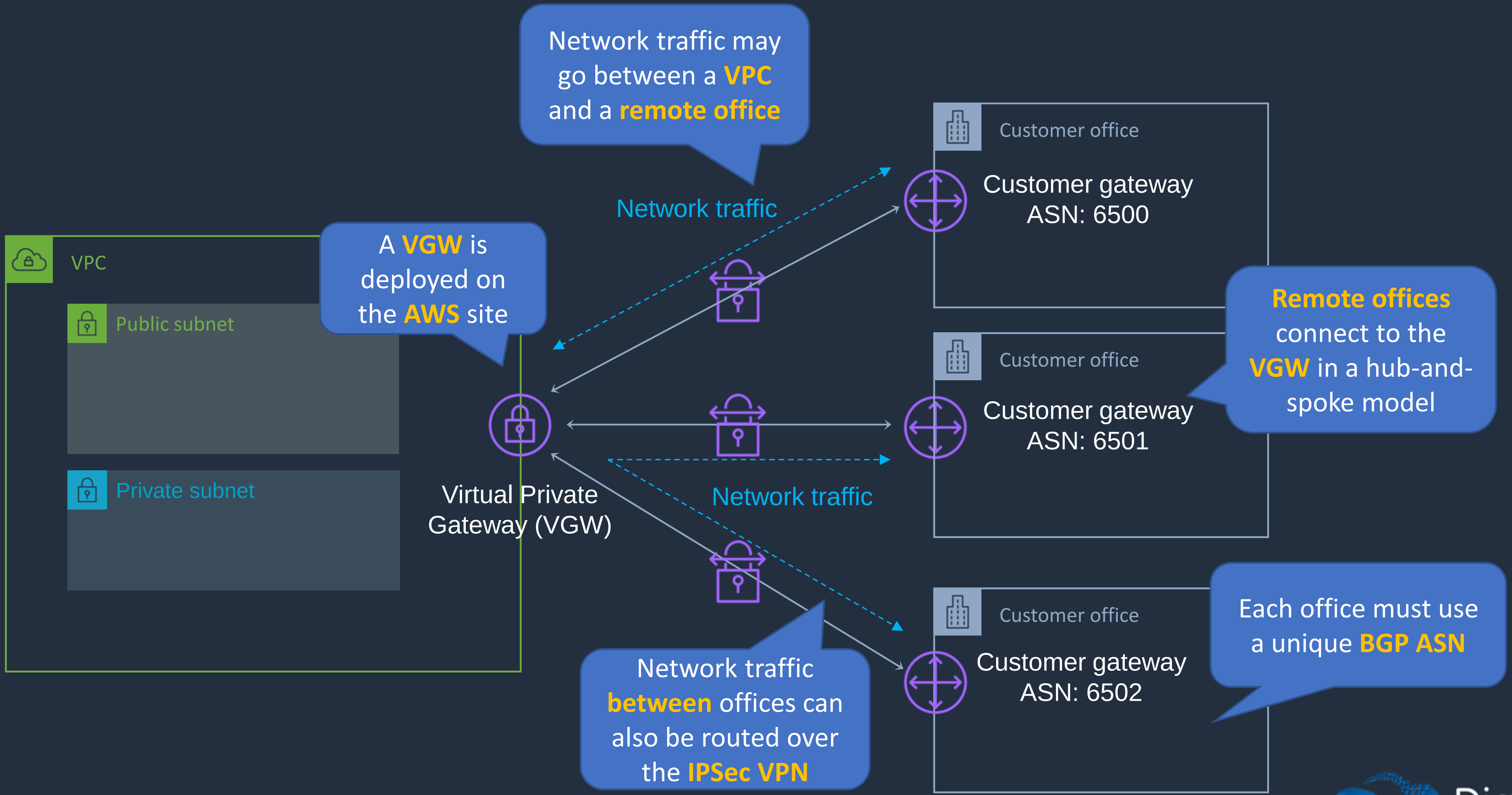
AWS Site-to-Site VPN

AWS Site-to-Site VPN



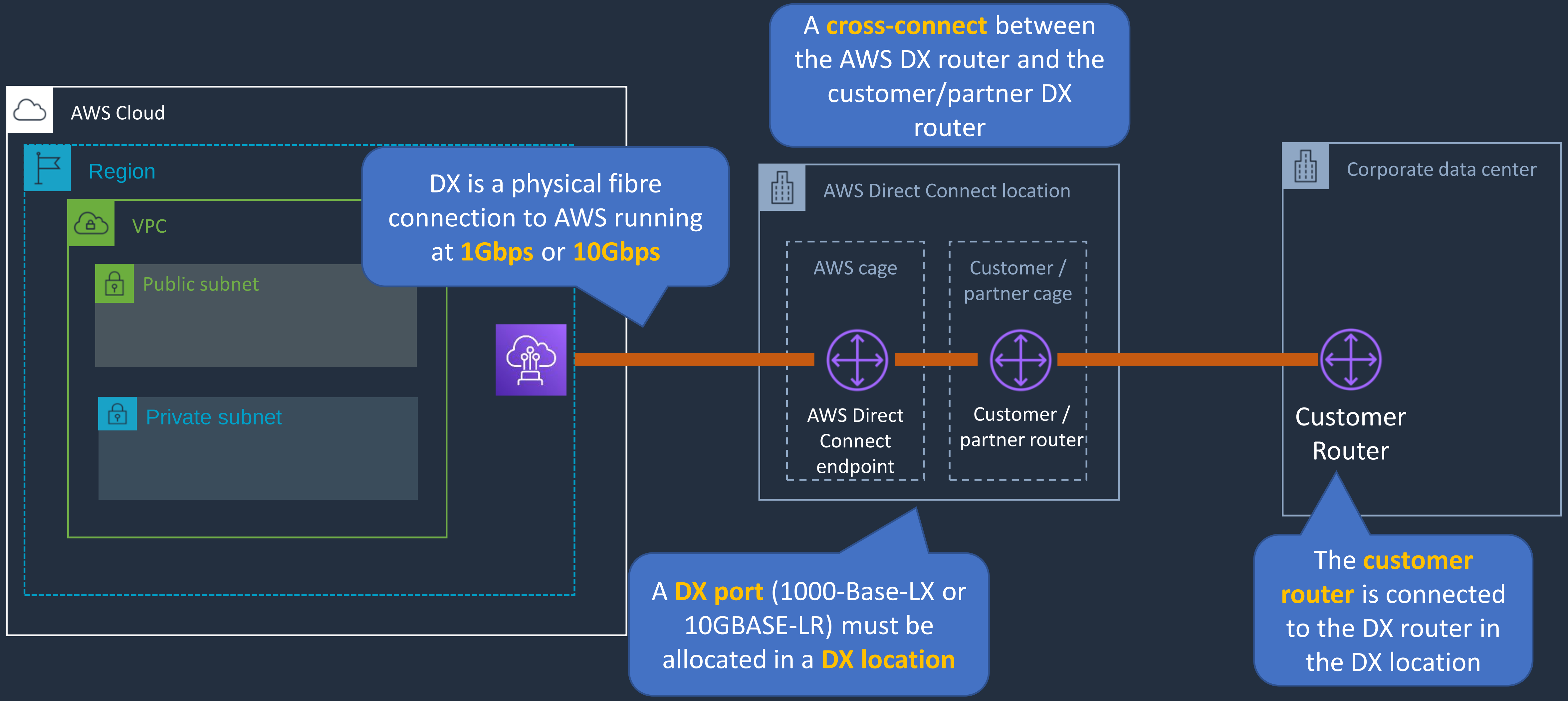
AWS VPN CloudHub

AWS VPN CloudHub



AWS Direct Connect (DX)

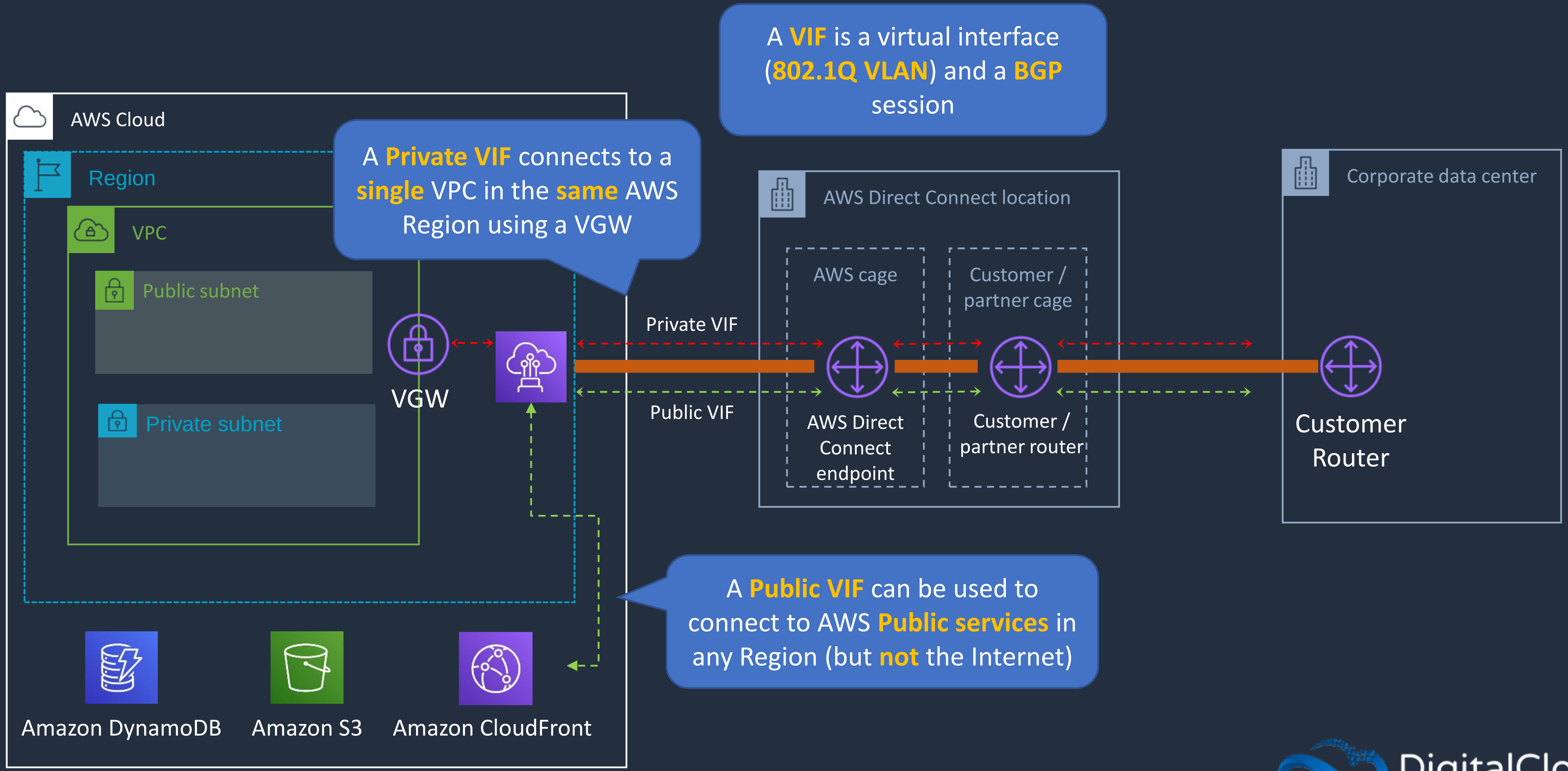
AWS Direct Connect (DX)



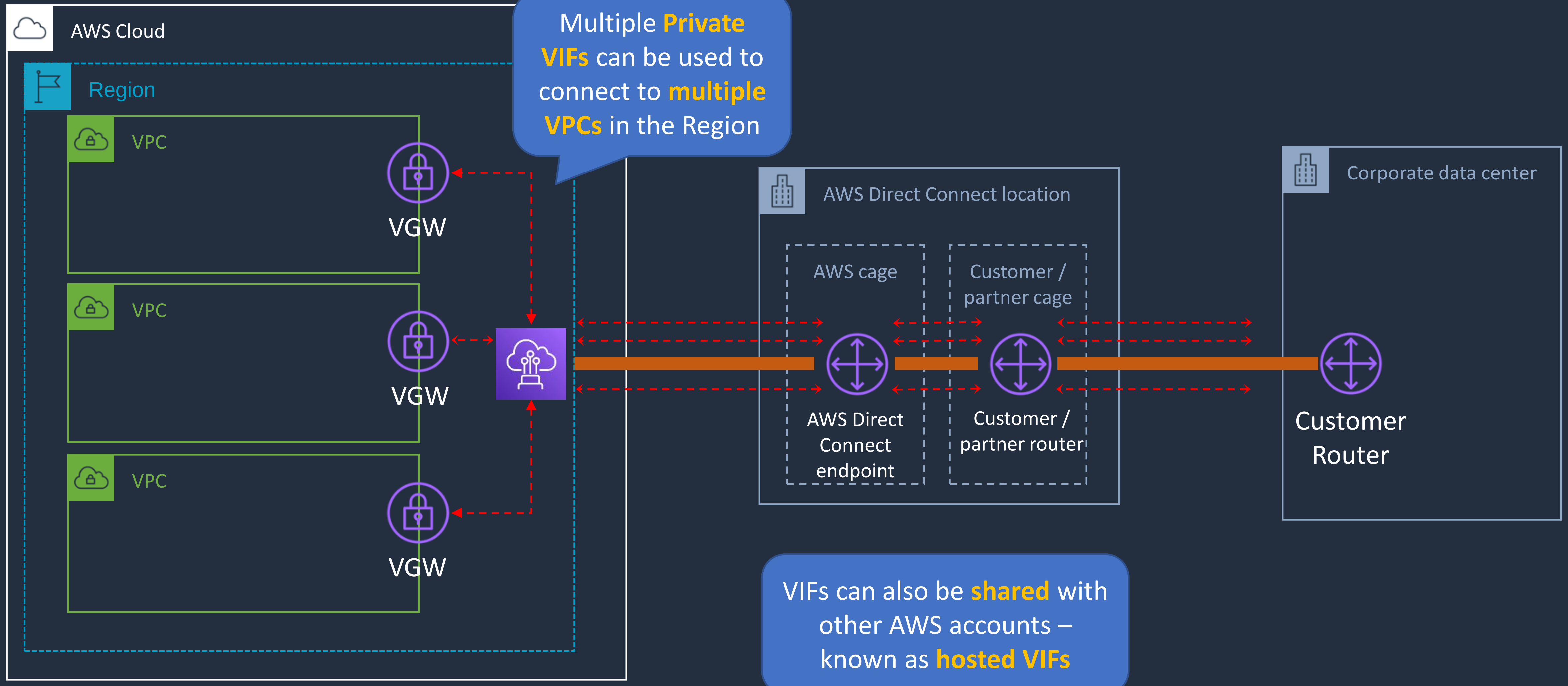
AWS Direct Connect Benefits

- **Private** connectivity between AWS and your data center / office
- Consistent network experience – increased **speed/latency** & **bandwidth/throughput**
- Lower costs for organizations that transfer **large** volumes of data

AWS Direct Connect (DX)



AWS Direct Connect (DX)



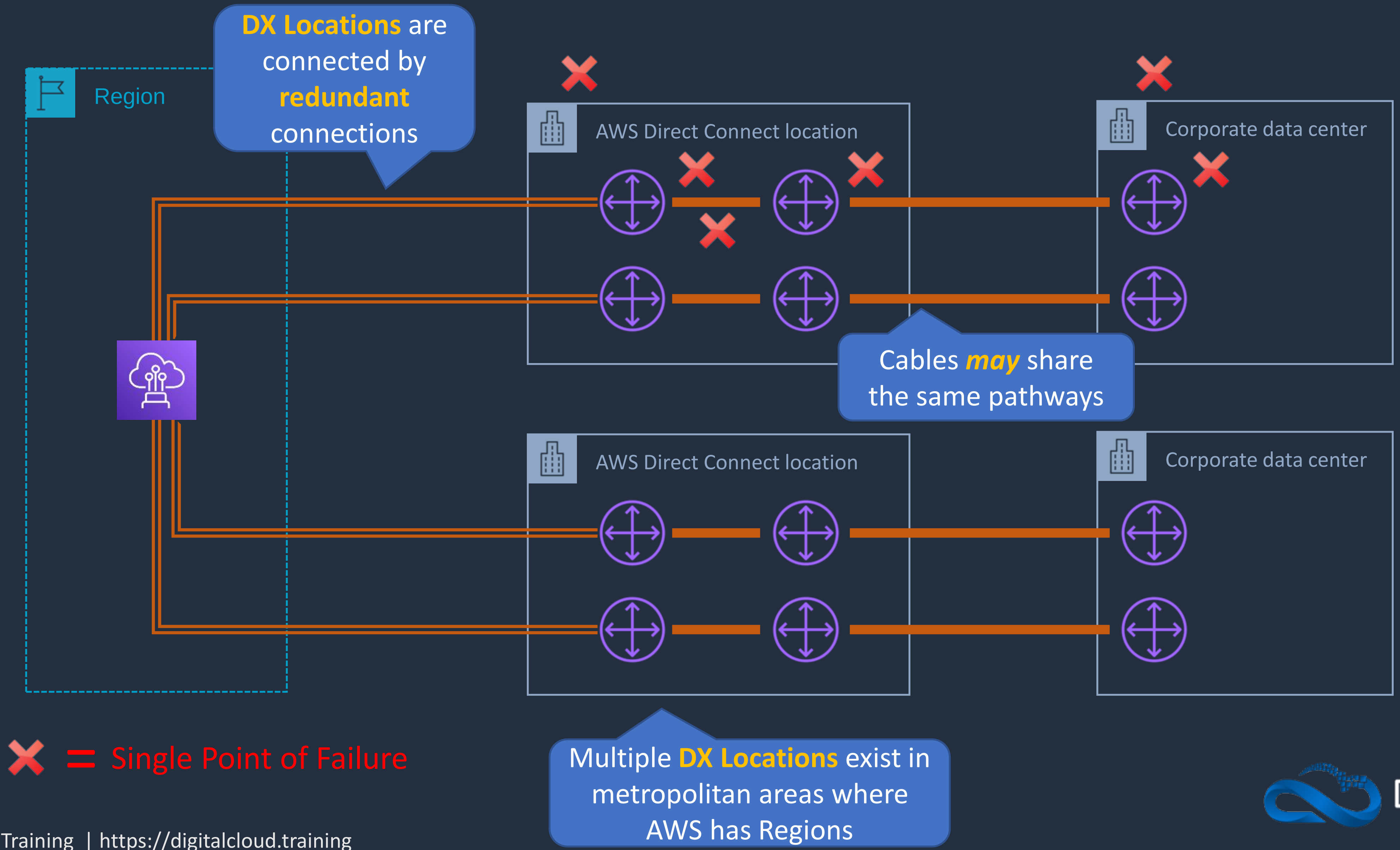
Multiple **Private VIFs** can be used to connect to **multiple VPCs** in the Region

VIFs can also be **shared** with other AWS accounts – known as **hosted VIFs**

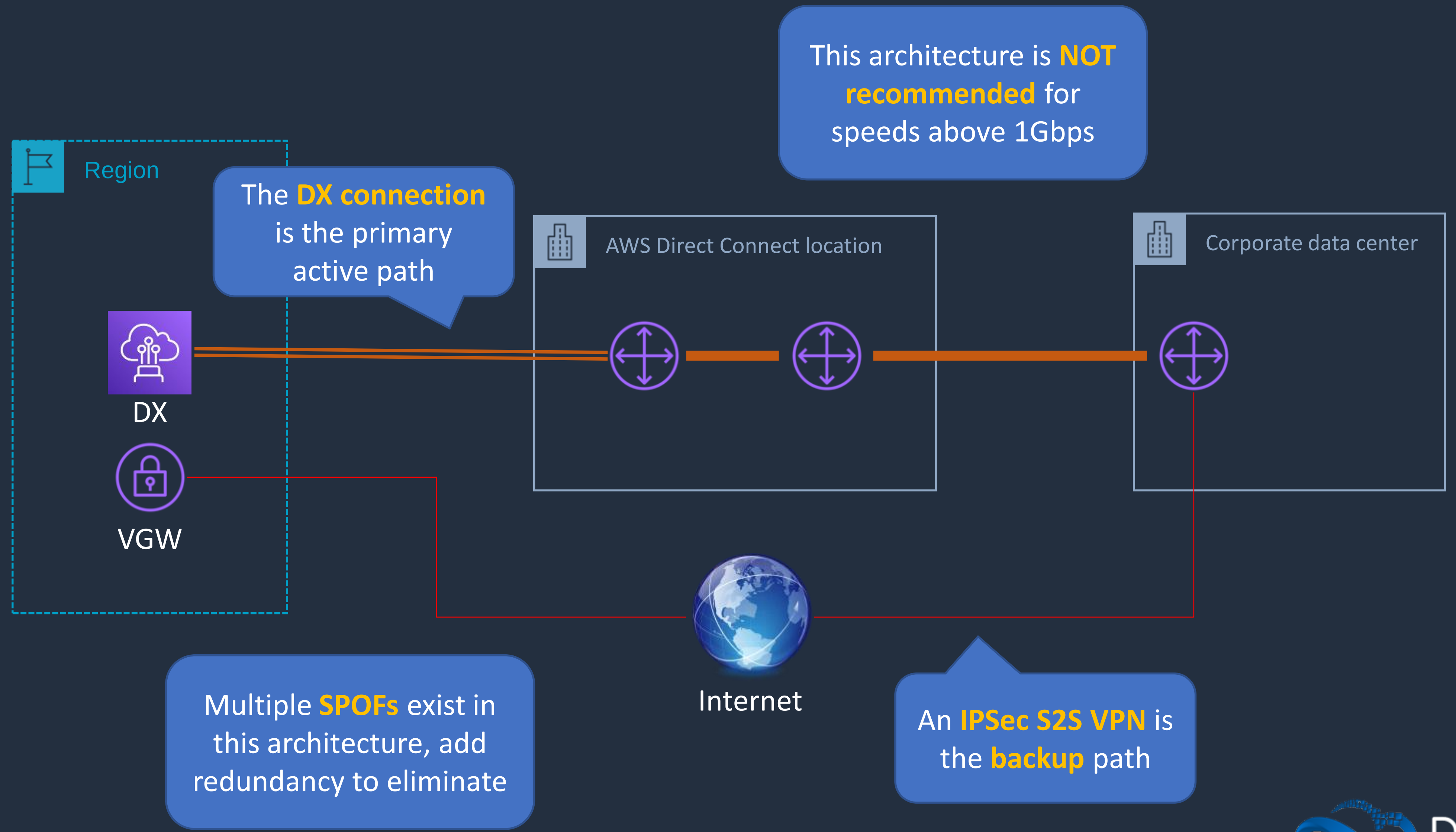
AWS Direct Connect (DX)

- Speeds from 50Mbps to 500Mbps can also be accessed via an APN partner (uses hosted VIFs or hosted connections):
  - A ***hosted VIF*** is a single VIF that is shared with other customers (shared bandwidth)
  - A ***hosted connection*** is a DX connection with a single VIF dedicated to you
- DX Connections are **NOT** encrypted!
- Use an **IPSec S2S VPN** connection over a VIF to add encryption in transit
- Link aggregation groups (**LAG**s) can be used to combine multiple **physical** connections into a single **logical** connection using **LACP** – provides improved **speed**

DX - Native High Availability

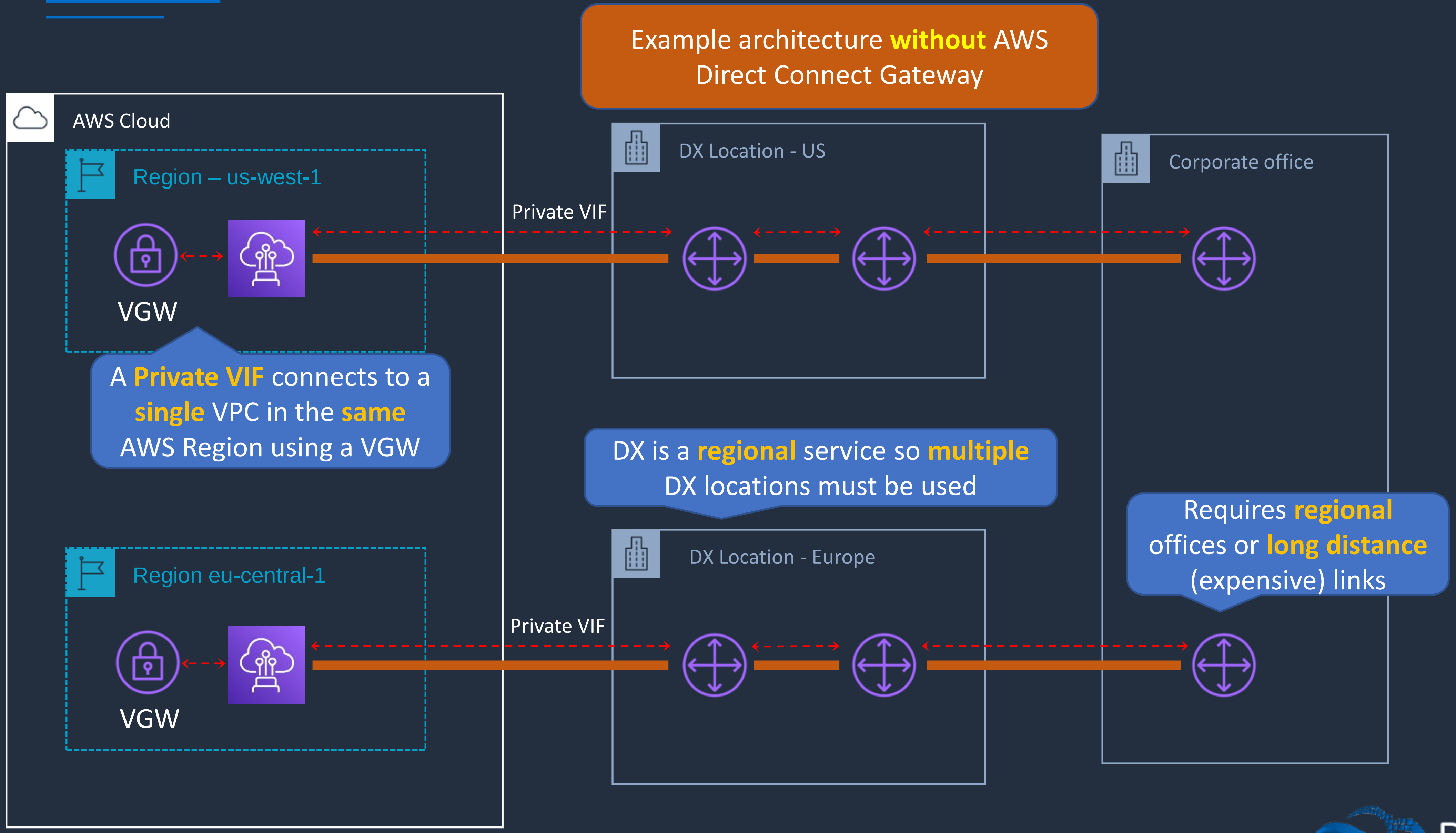


Direct Connect + IPSec S2S VPN



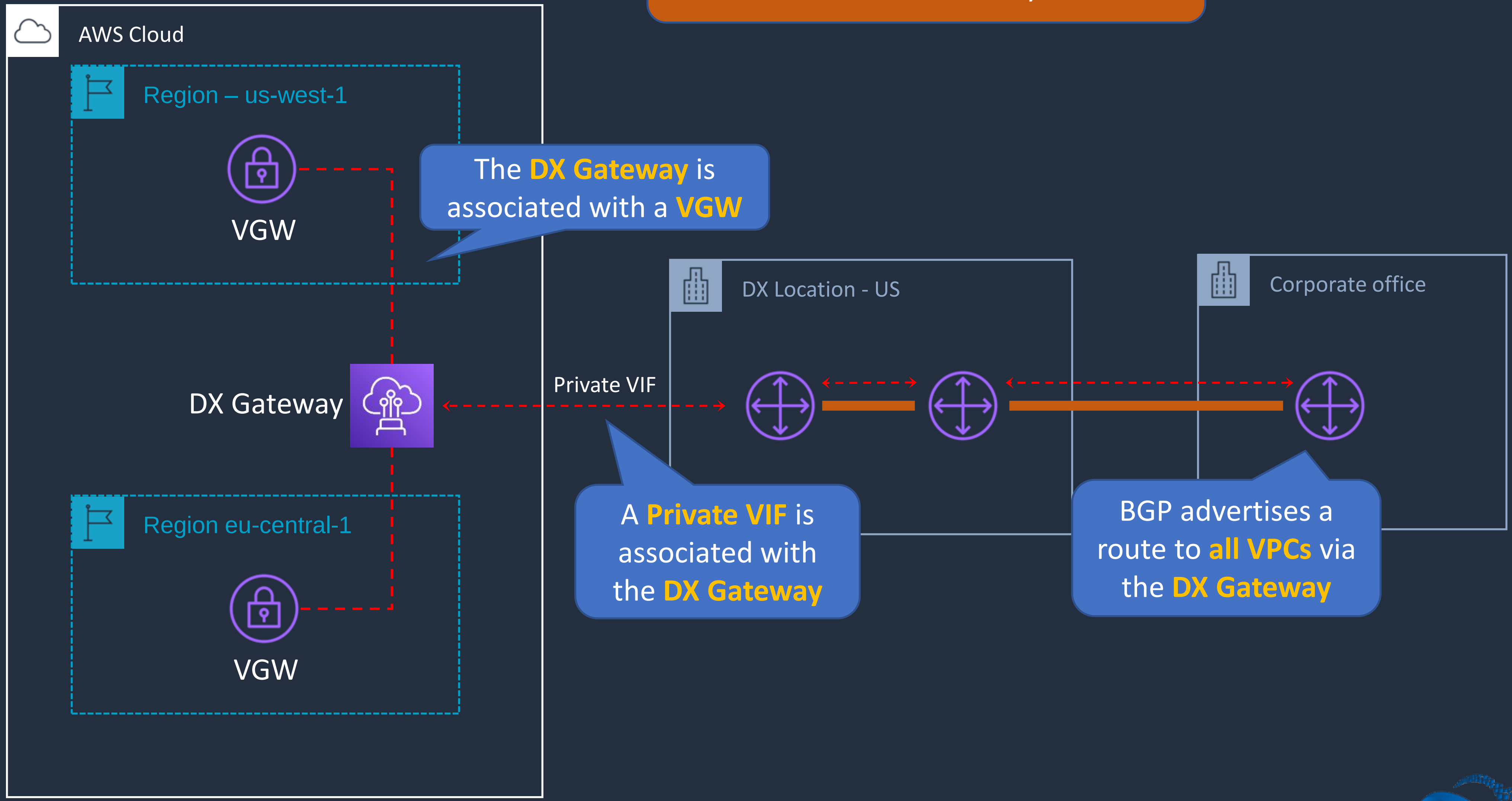
AWS Direct Connect Gateway

Direct Connect - Multiple Regions

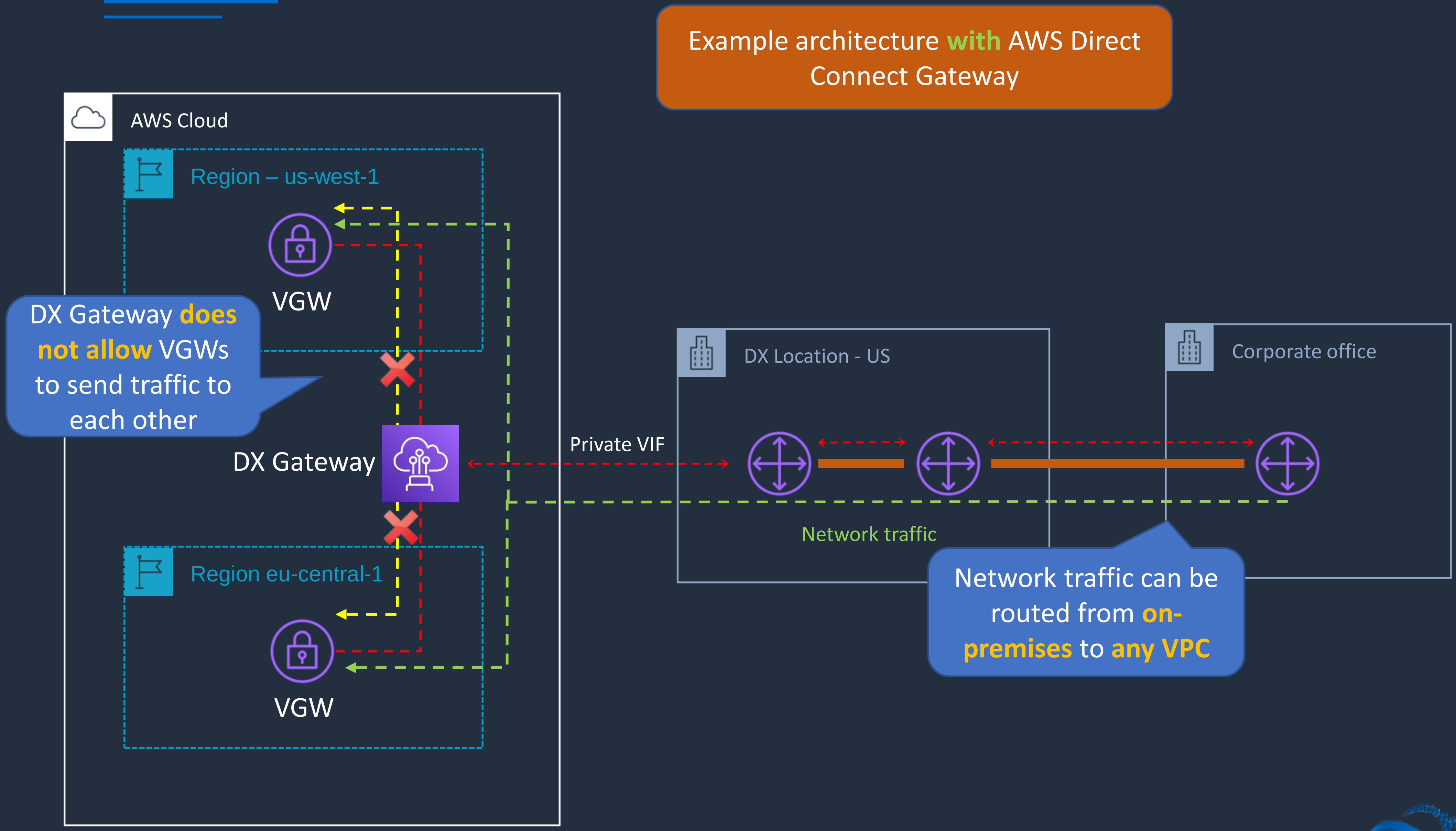


Direct Connect - Multiple Regions

Example architecture with AWS Direct Connect Gateway

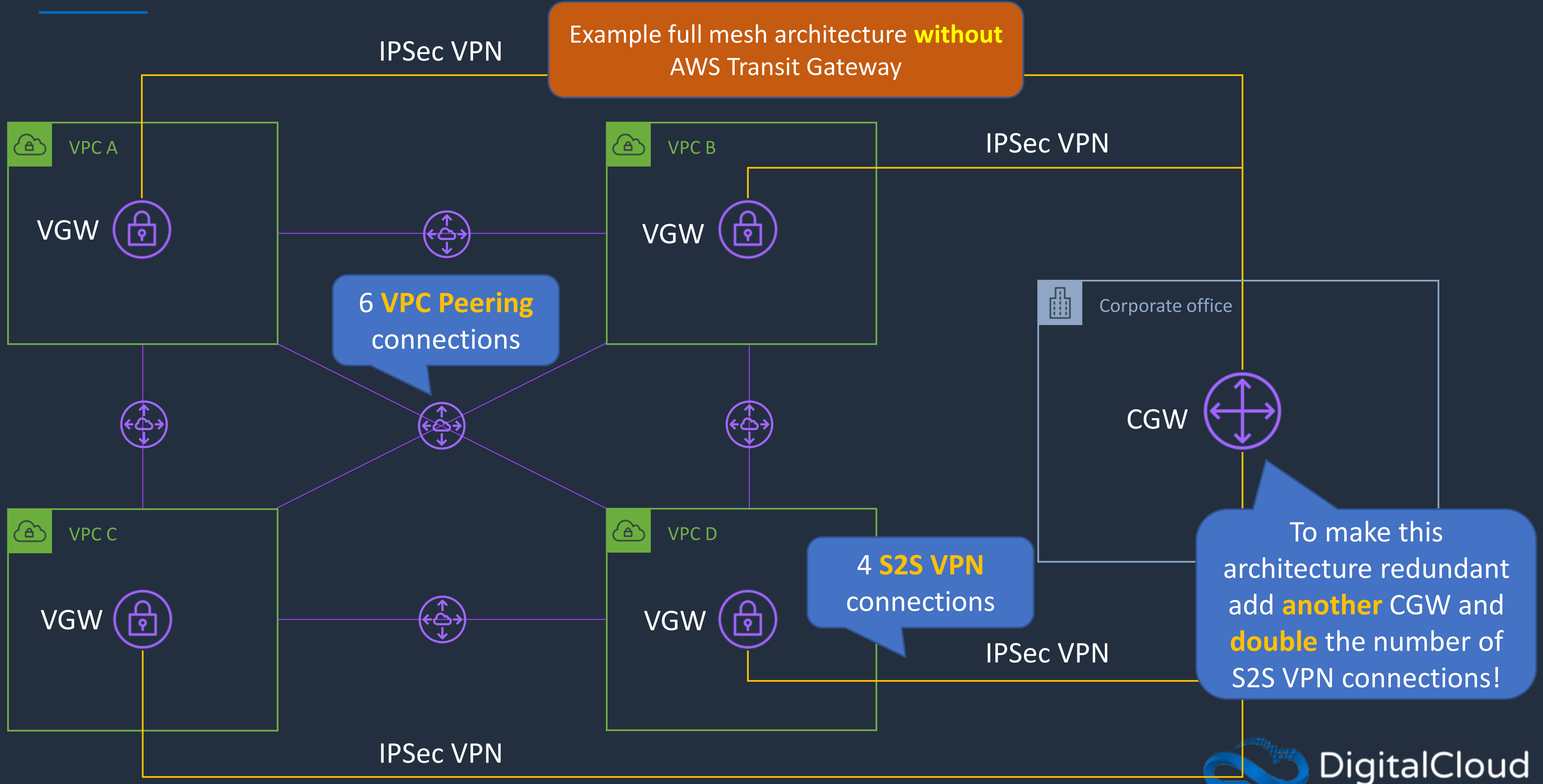


Direct Connect - Multiple Regions

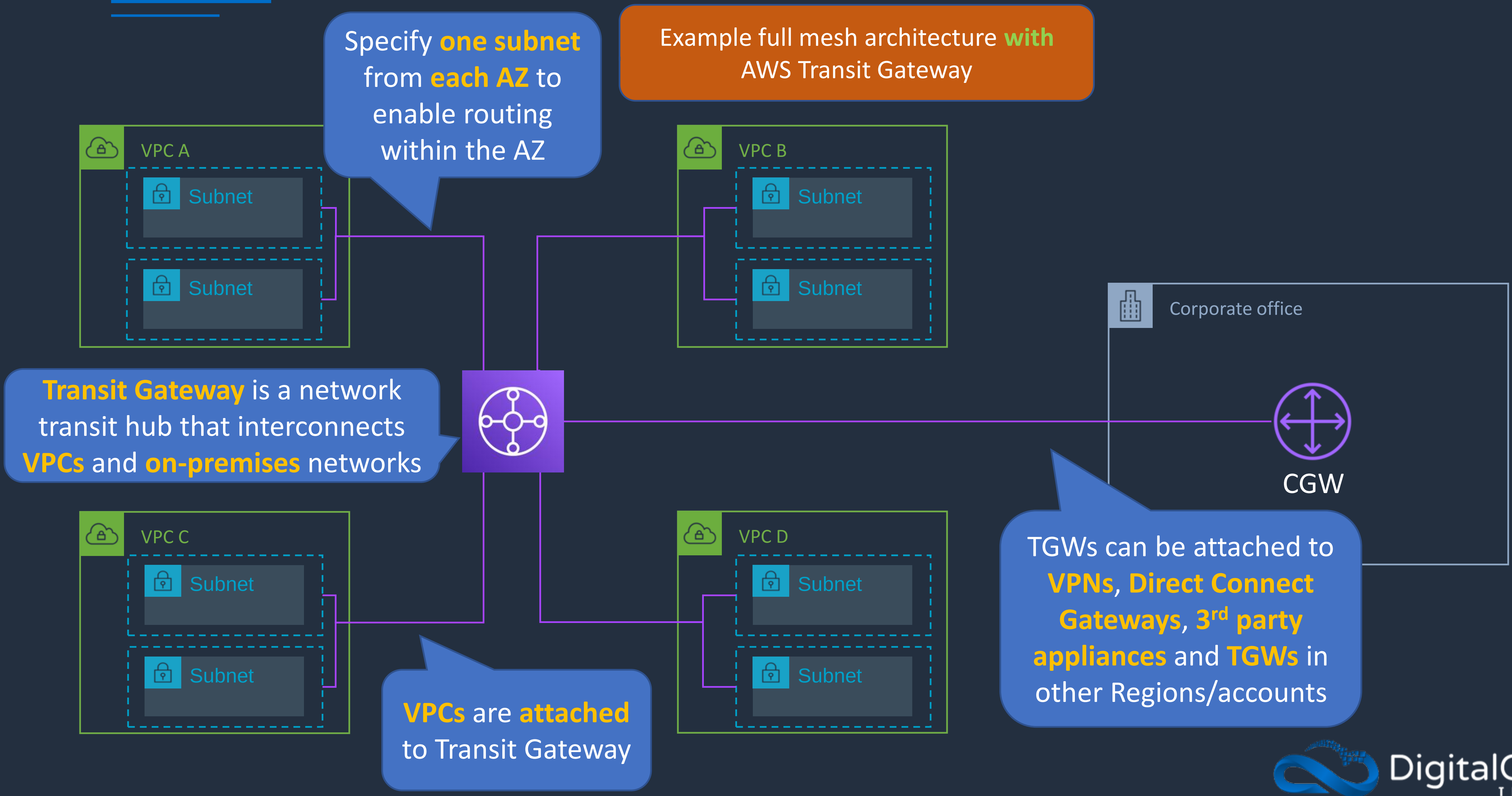


AWS Transit Gateway

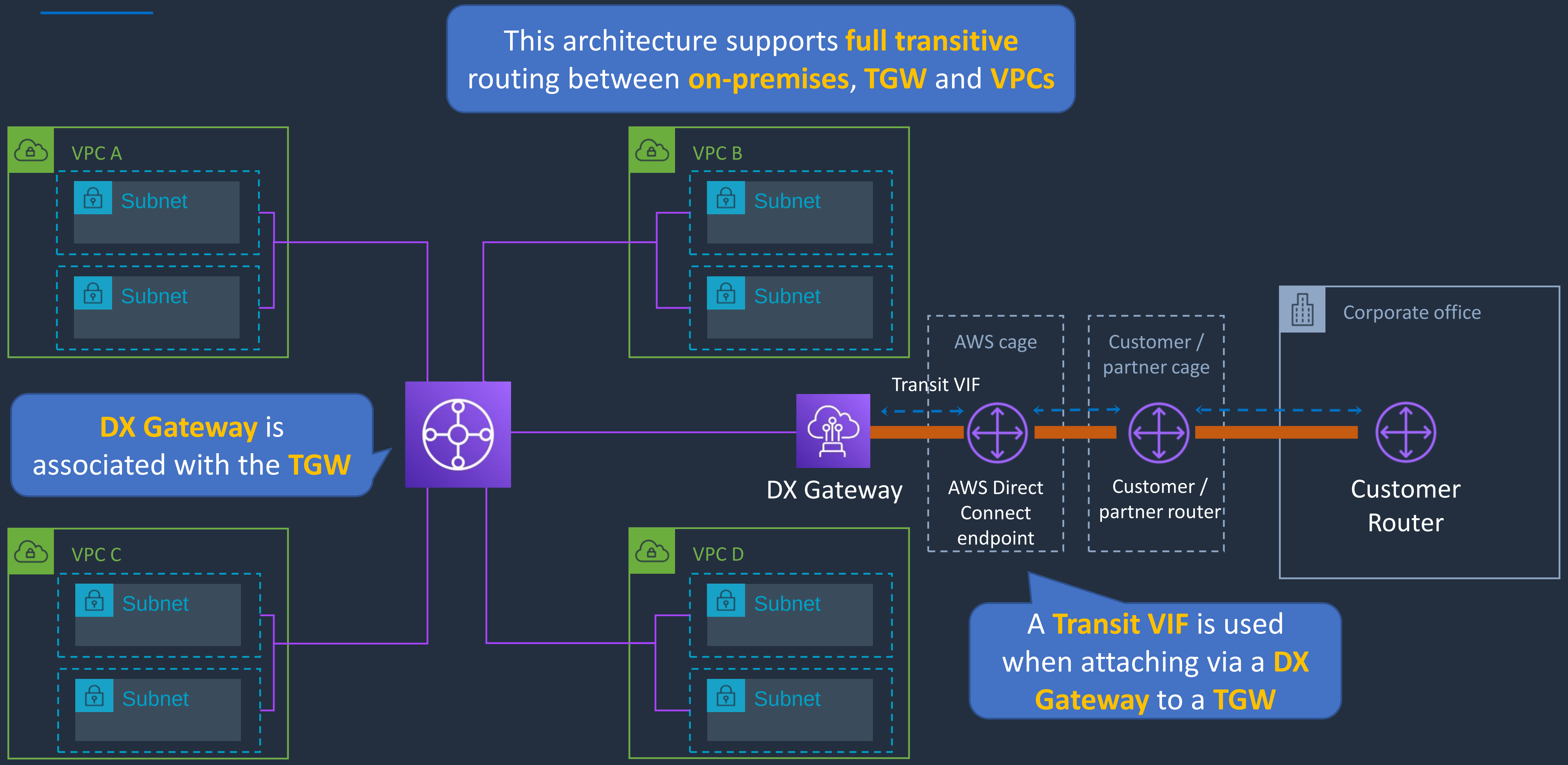
AWS Transit Gateway



AWS Transit Gateway



AWS TGW + DX Gateway



SECTION 7

Compute, Auto Scaling, and Load Balancing

Amazon EC2 Pricing Options

Amazon EC2 Pricing Options

On-Demand

Standard rate - no discount; no commitments; dev/test, short-term, or unpredictable workloads

Reserved

1 or 3-year commitment; up to 75% discount; steady-state, predictable workloads and reserved capacity

Spot Instances

Get discounts of up to 90% for unused capacity. Can be terminated at any time

Dedicated Instances

Physical isolation at the host hardware level from instances belonging to other customers; pay per instance

Dedicated Hosts

Physical server dedicated for your use; Socket/core visibility, host affinity; pay per host; workloads with server-bound software licenses

Savings Plans

Commitment to a consistent amount of usage (EC2 + Fargate + Lambda); Pay by $/hour; 1 or 3-year commitment

Amazon EC2 Billing

Commercial Linux distros such as **Red Hat EL** and **SUSE ES** use **hourly** pricing

Billed per hour;
Minimum of 1 hour

Billed per second;
Minimum of 1 minute

Per-second billing is for **Amazon Linux, Windows** and **Ubuntu** in **On-Demand**, **Reserved**, and **Spot** forms

Volumes billed per second;
Minimum of 1 minute

DigitalCloud TRAINING

Amazon EC2 Reserved Instances (RIs)

Can pay **All Upfront**, **Partial Upfront**, **No Upfront**

Standard RI

Change **AZ**, **instance size** (Linux), **networking type**

Use **ModifyReservedInstances** API

Convertible RI

Change **AZ**, **instance size** (Linux), **networking type**

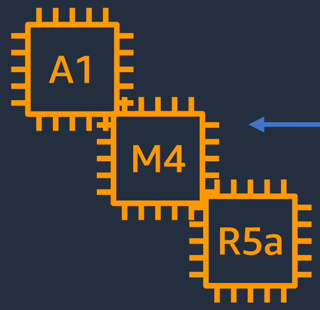
\+

Change **family**, **OS**, **tenancy**, **payment option**

Use **ExchangeReservedInstances** API

Amazon EC2 Reserved Instances (RIs)

Tenancy: **Default** or **Dedicated**



When the **attributes** of a used instance match the **attributes** of an RI the discount is applied

Reserves capacity in specified AZ

Availability Zone

Region

Does **not** reserve capacity; discount applies to all AZs

DigitalCloud
TRAINING

Amazon EC2 Reserved Instances (RIs)

Scheduled RI

- Match capacity reservation to **recurring schedule**
- Minimum **1200 hours** per year
- Example: Reporting app that runs 6 hours a day 4 days a week = 1248 hours per year

Important

We do not have any capacity for purchasing Scheduled Reserved Instances or any plans to make it available in the future. To reserve capacity, use On-Demand Capacity Reservations instead. For discounted rates, use Savings Plans.

This message started showing recently but exam may **not** reflect this yet

AWS Savings Plans

Compute Savings Plan

1 or 3-year; hourly commitment to usage of **Fargate**, **Lambda**, and **EC2**; Any Region, family, size, tenancy, and OS

EC2 Savings Plan

1 or 3-year; hourly commitment to usage of **EC2** within a **selected Region** and **Instance Family**; Any size, tenancy and OS

Amazon EC2 Spot Instances

Spot Instance: One or more EC2 instances

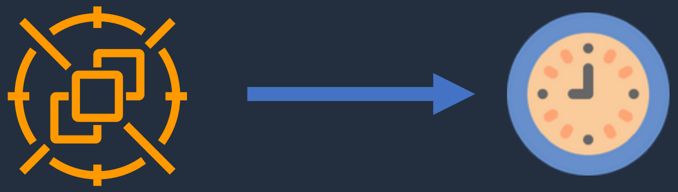
Spot Fleet: launches and maintains the number of Spot / On-Demand instances to meet specified target capacity

EC2 Fleet: launches and maintains specified number of Spot / On-Demand / Reserved instances in a **single API call**

Can define separate OD/Spot **capacity targets**, **bids**, **instance types**, and **AZs**

2-minute warning if AWS need to reclaim capacity – available via **instance metadata** and **CloudWatch Events**

Spot Block



Requirement:
Uninterrupted for
1-6 hours

Solution: **Spot Block**

Pricing is **30% - 45%** less than On-Demand

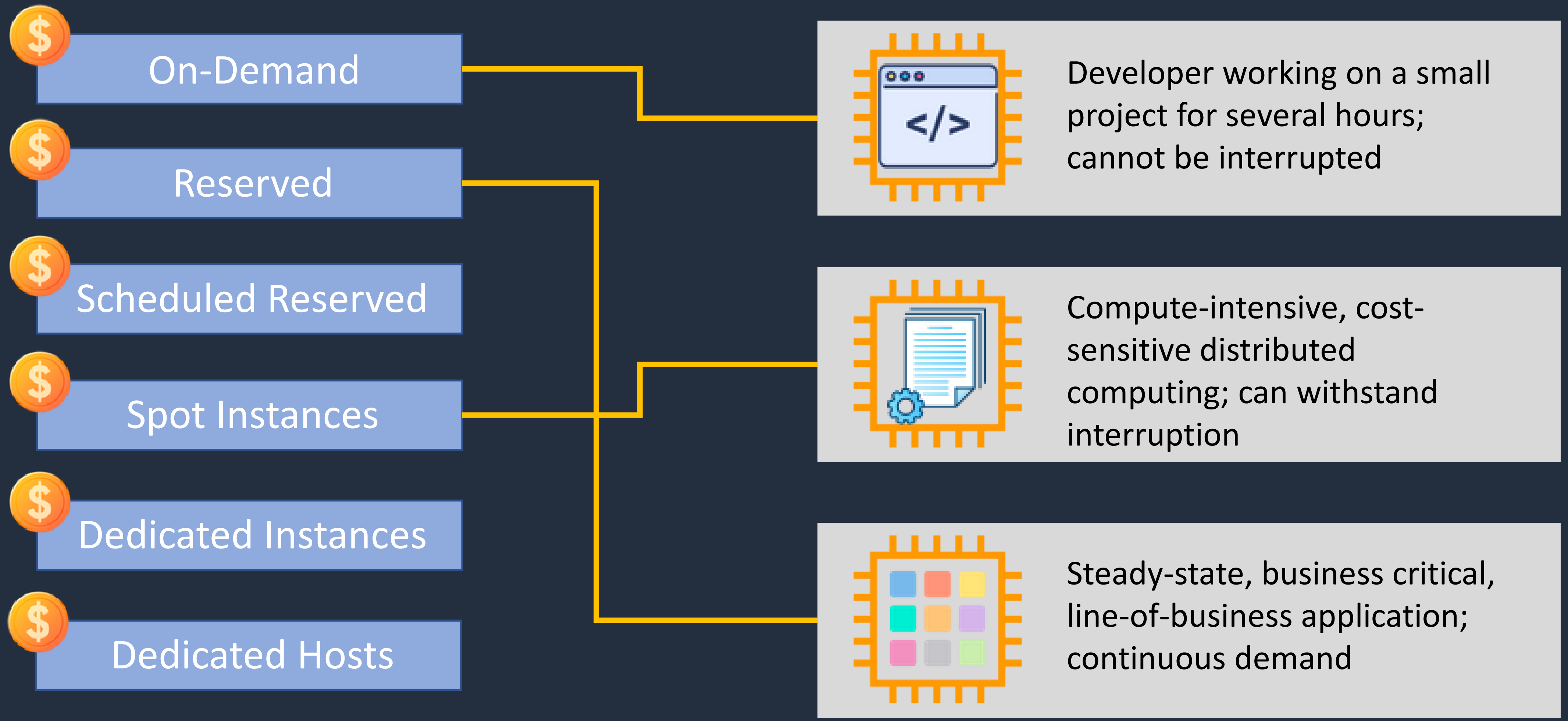
```
$ aws ec2 request-spot-instances \
  --block-duration-minutes 360 \
  --instance-count 5 \
  --spot-price "0.25" ...
```

Dedicated Instances and Dedicated Hosts

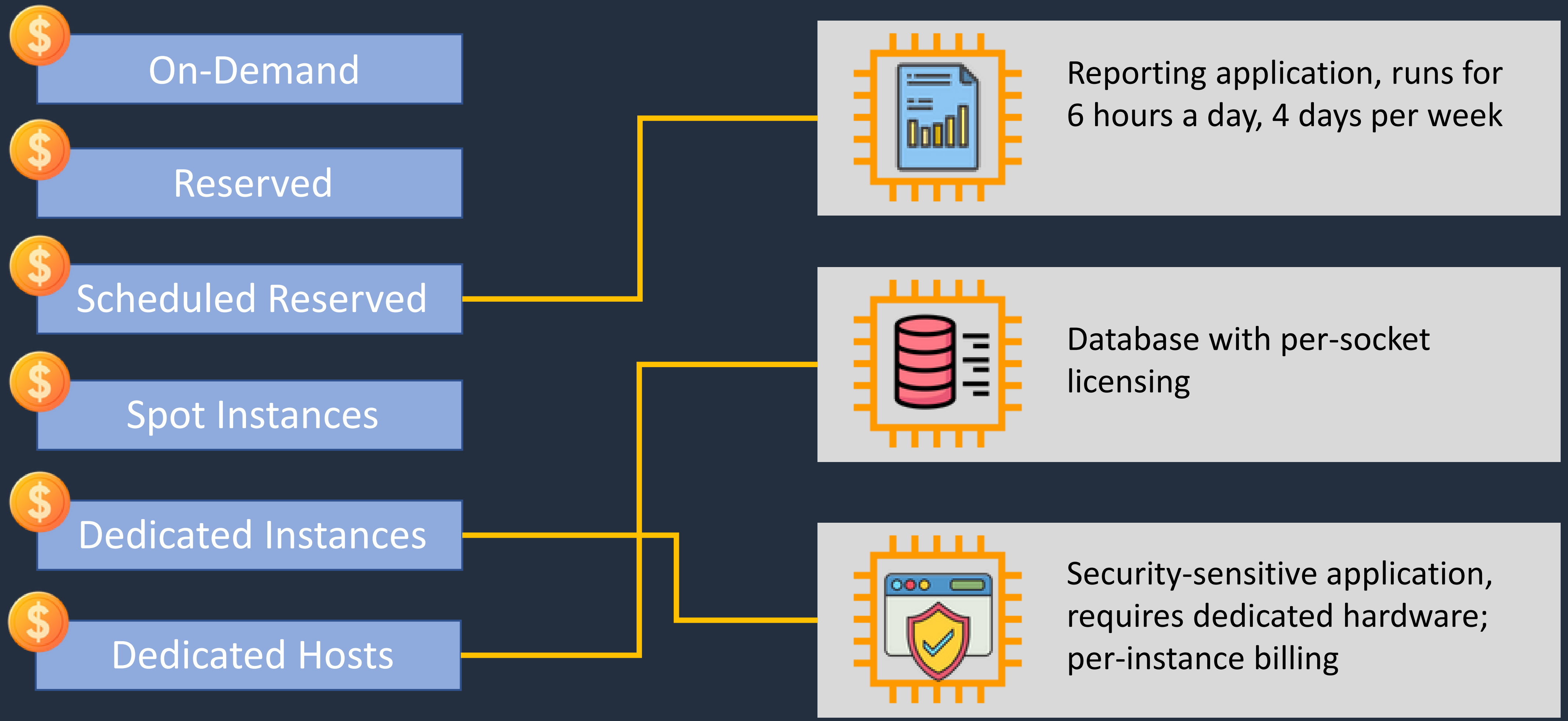
Characteristic	Dedicated Instances	Dedicated Hosts
Enables the use of dedicated physical servers	X	X
Per instance billing (subject to a $2 per region fee)	X	
Per host billing		X
Visibility of sockets, cores, host ID		X
Affinity between a host and instance		X
Targeted instance placement		X
Automatic instance placement	X	X
Add capacity using an allocation request		X

Amazon EC2 Pricing Use Cases

Amazon EC2 Pricing Use Cases



Amazon EC2 Pricing Use Cases



Bootstrapping AMIs

AMIs and Bootstrapping

How can we launch our EC2 instances and install dependencies, applications, software and security updates, and configure customizations quickly?

Customized AMIs

EC2 Instance

Select an **instance type**

Family	Type	vCPUs	Memory (GiB)
General purpose	t2.micro	1	1
Compute optimized	c5n.large	2	5.25
Memory optimized	r5ad.large	2	16
Storage optimized	d2.xlarge	4	30.5
GPU instances	g2.2xlarge	8	15

Amazon Machine Image (AMI)

EBS Snapshot

Linux

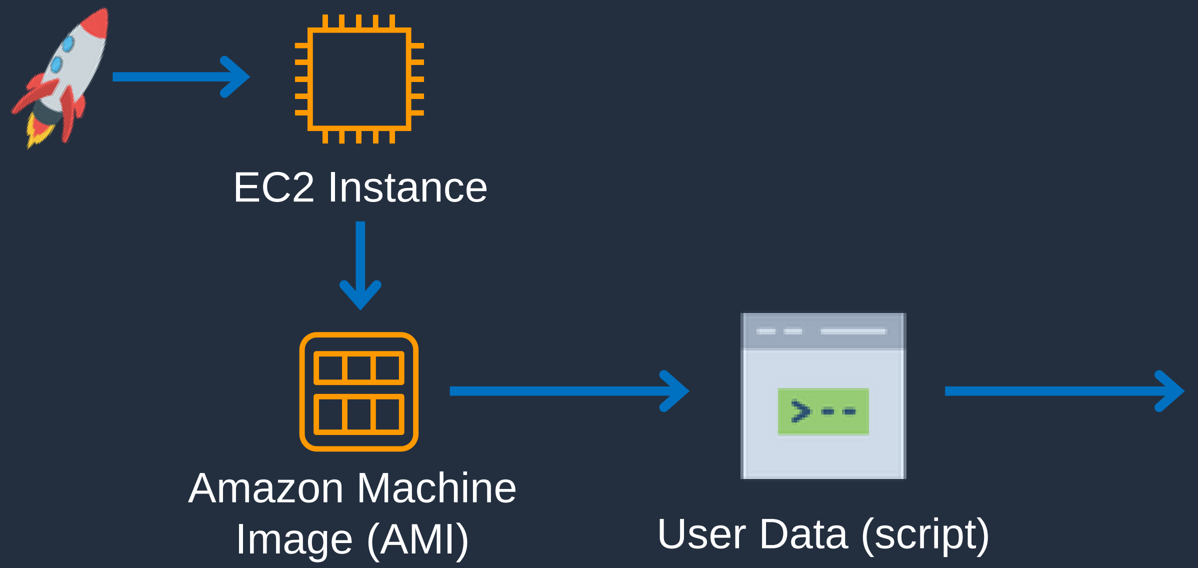
Microsoft Windows

- OS Customizations
- Application Dependencies
- Application and Configuration
- Software and Security updates

Customized AMI

DigitalCloud TRAINING

User Data



- OS Customizations
- Application Dependencies
- Application and Configuration
- Software and Security updates

A combined approach would use a **customized AMI** + **User Data**

Automation and Configuration Management Tools

Use **automation** and **configuration** tools

- AWS CloudFormation
- AWS OpsWorks
- AWS Systems Manager
- AWS CodePipeline, CodeDeploy etc.
- Chef and Puppet
- Jenkins

EC2 Placement Group Use Cases

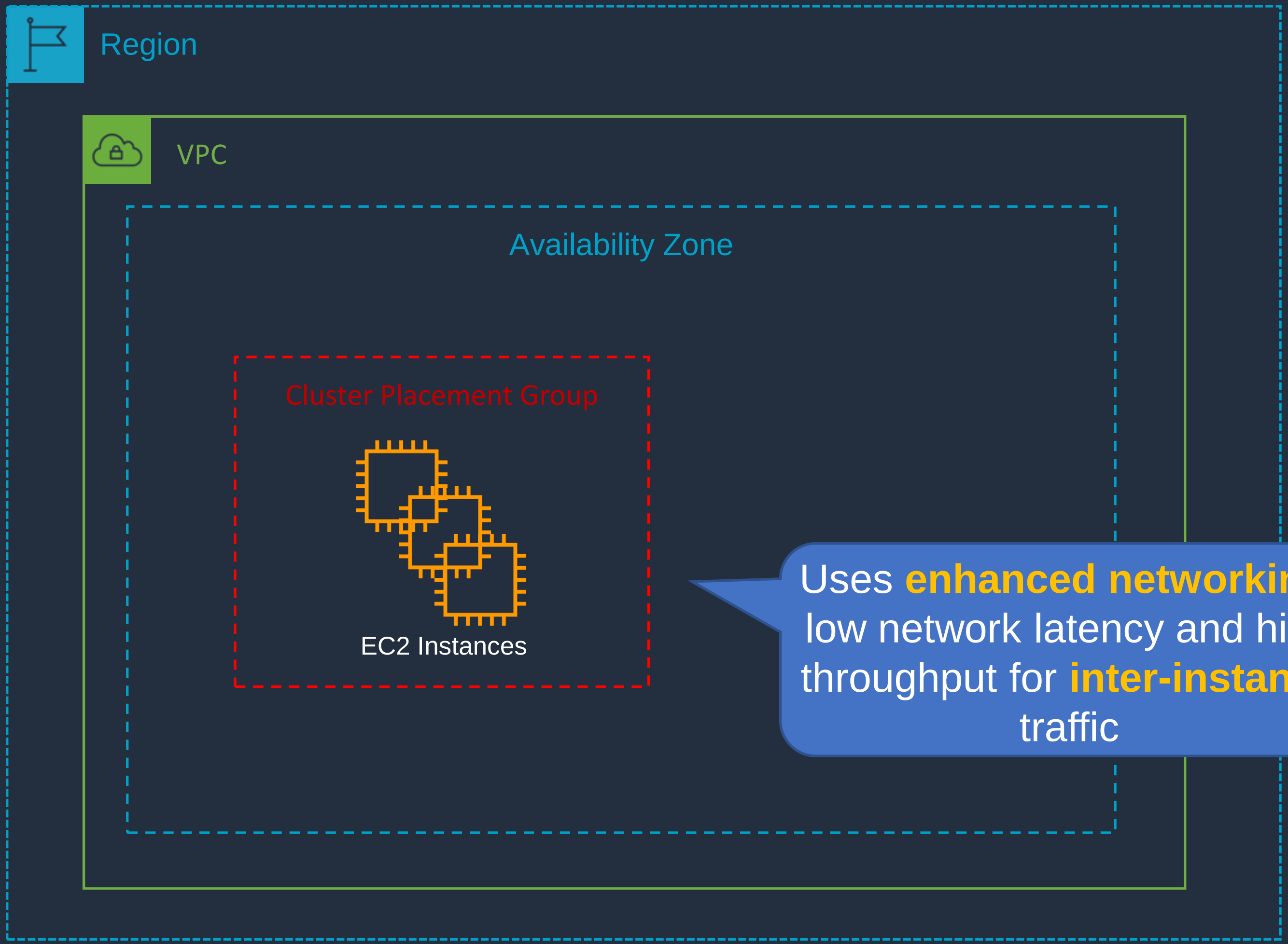
EC2 Placement Groups

- Cluster – packs instances close together inside an Availability Zone. This strategy enables workloads to achieve the low-latency network performance necessary for tightly-coupled node-to-node communication that is typical of HPC applications

- Partition – spreads your instances across logical partitions such that groups of instances in one partition do not share the underlying hardware with groups of instances in different partitions. This strategy is typically used by large distributed and replicated workloads, such as Hadoop, Cassandra, and Kafka

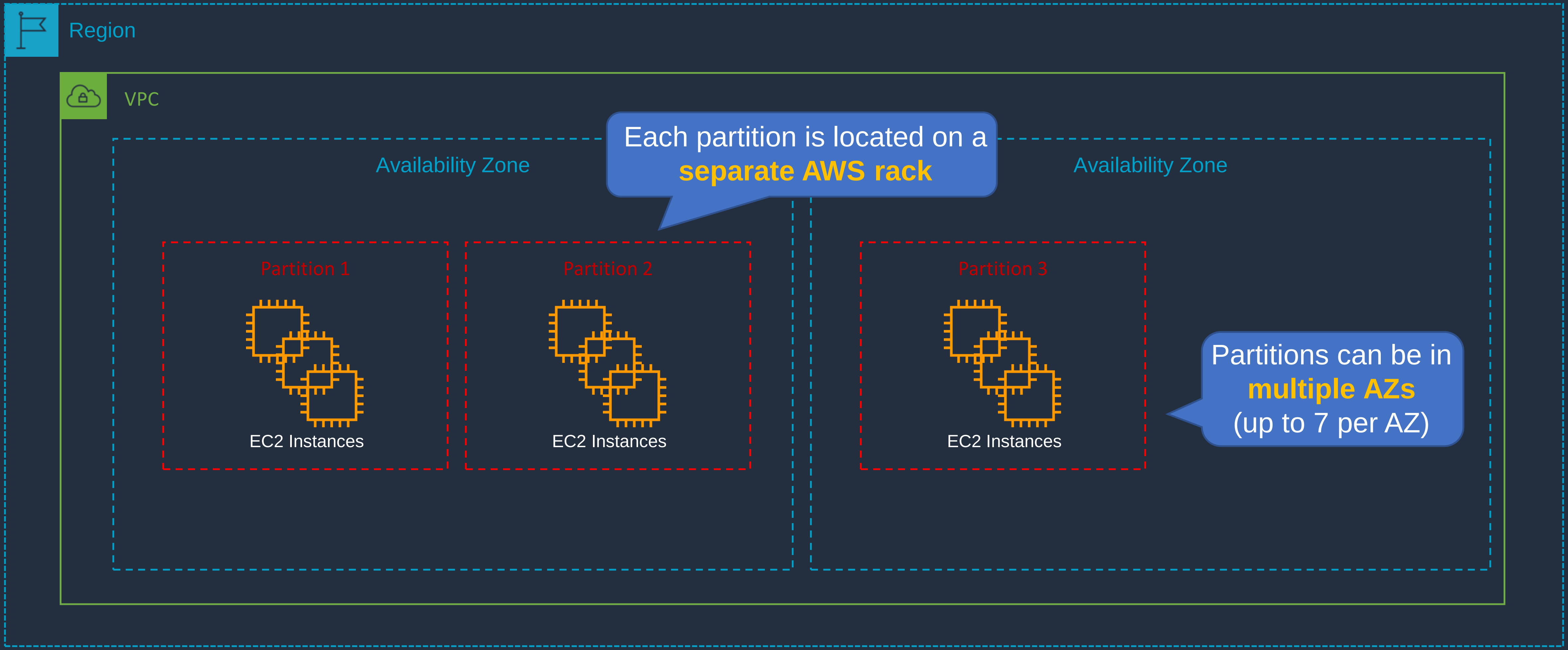
- Spread – strictly places a small group of instances across distinct underlying hardware to reduce correlated failures

Cluster Placement Group

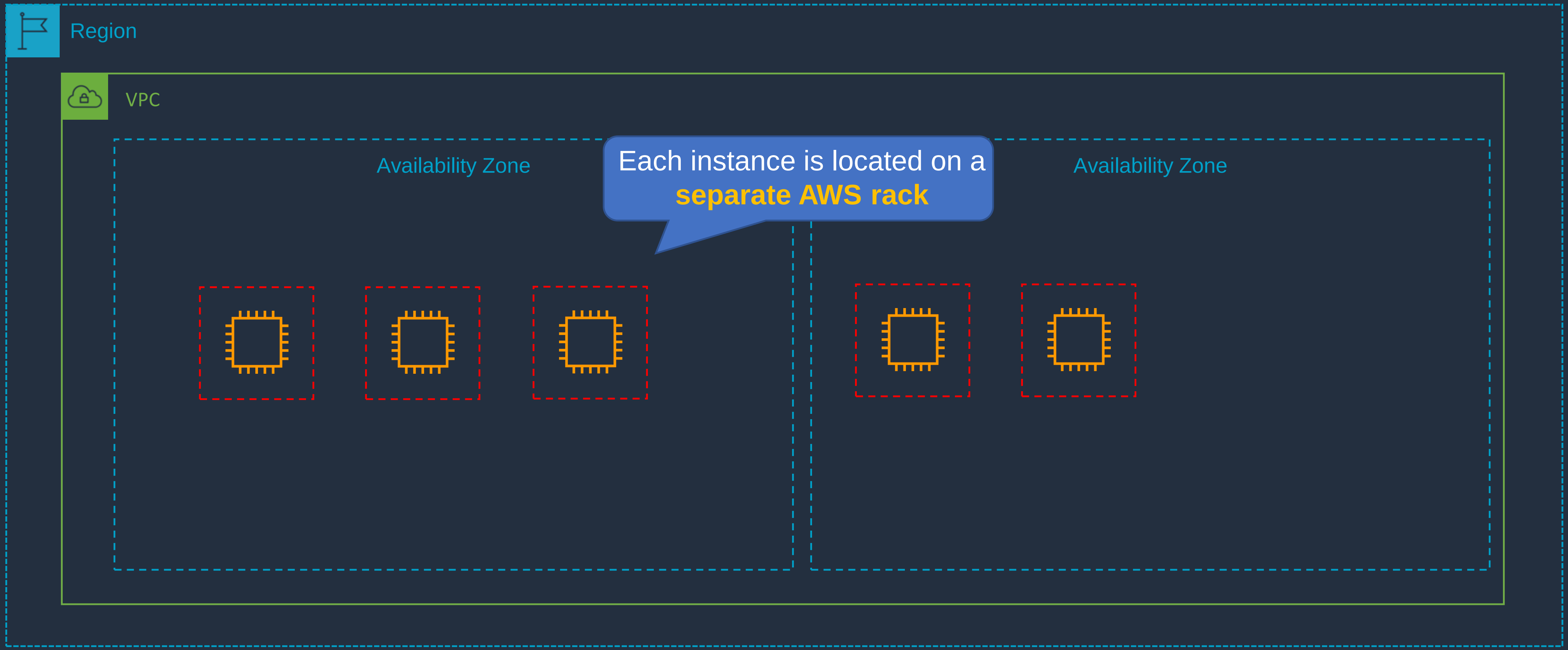


Uses **enhanced networking**, low network latency and high throughput for **inter-instance** traffic

Partition Placement Group



Spread Placement Group

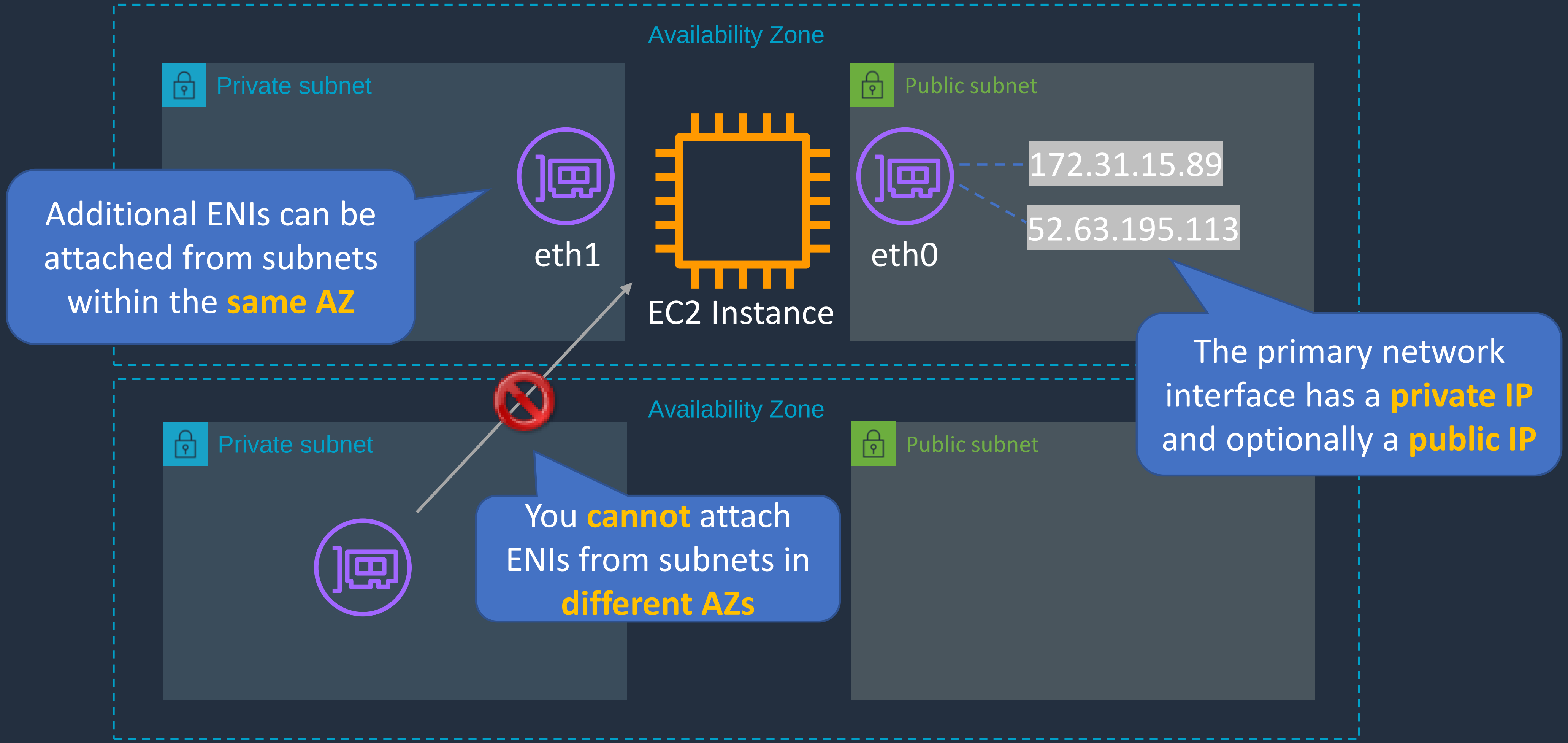


EC2 Placement Group Use Cases

- **Cluster** – Tightly-coupled application that requires low-latency, high throughput network traffic between instances
- **Partition** – Distributed and replicated NoSQL database; requires separate hardware for node groups
- **Spread** – Small number of critical instances that should be kept separate from each other

Network Interfaces (ENI, ENA, EFA)

Network Interfaces (ENI, ENA, EFA)



Network Interfaces (ENI, ENA, EFA)

Elastic network interface

- Basic adapter type for when you don't have any high-performance requirements
- Can use with all instance types

Elastic network adapter

- Enhanced networking performance
- Higher bandwidth and lower inter-instance latency
- Must choose supported instance type

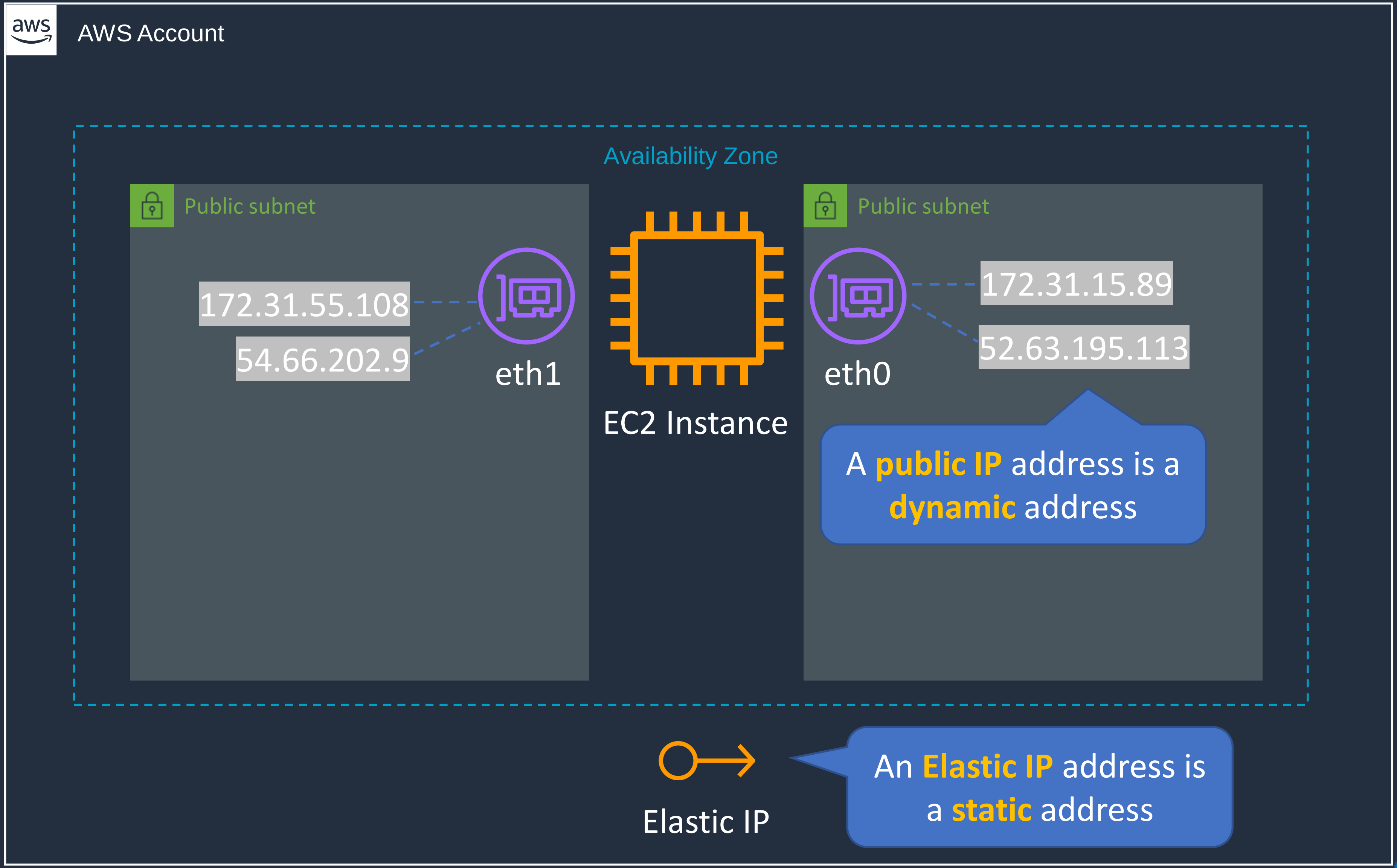
Elastic Fabric Adapter

- Use with High Performance Computing and MPI and ML use cases
- Tightly coupled applications
- Can use with all instance types

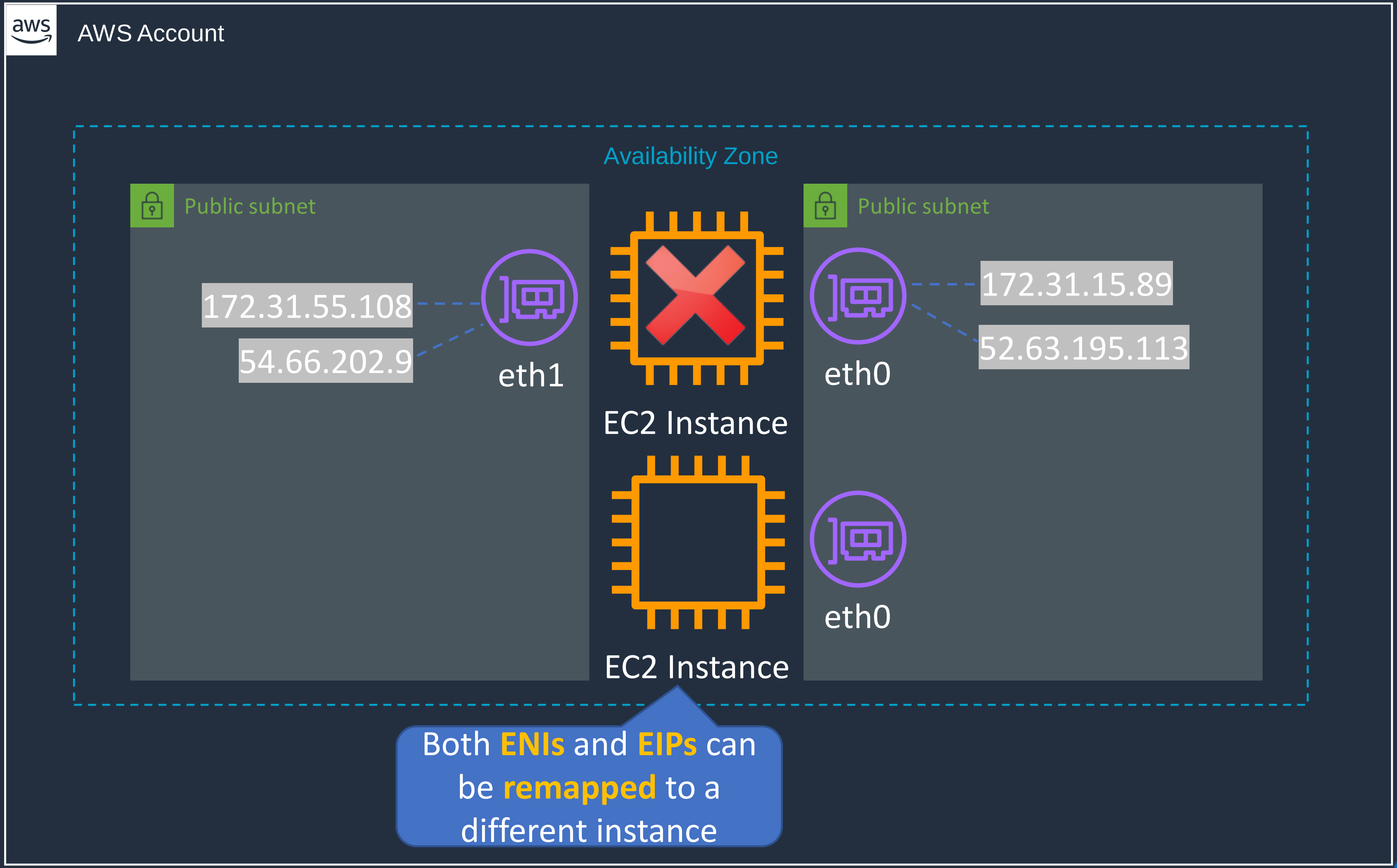
Working with ENIs and IP Addresses

Public, Private and Elastic IP Addresses

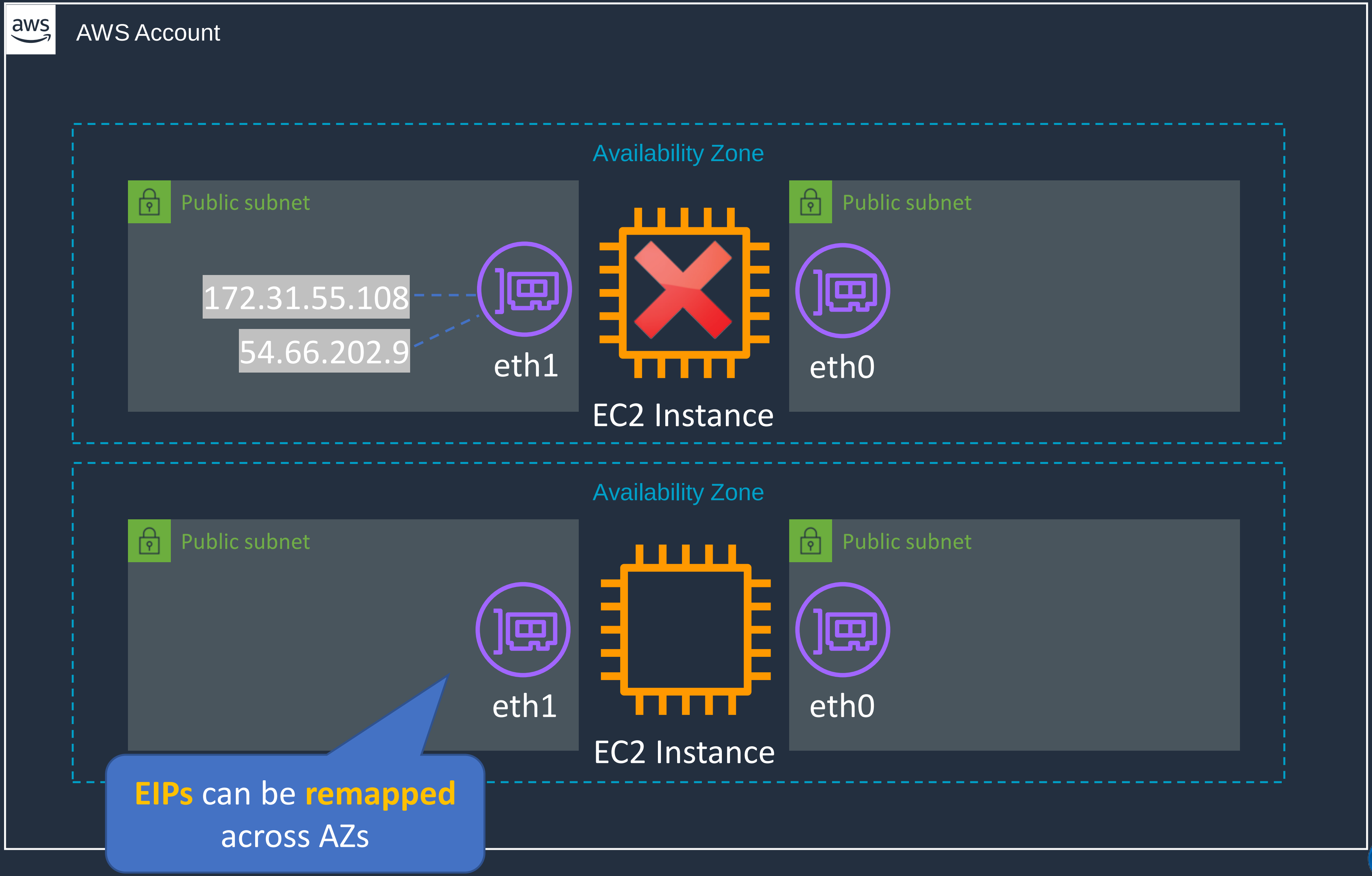
Public, Private and Elastic IP Addresses



Public, Private and Elastic IP Addresses



Public, Private and Elastic IP Addresses

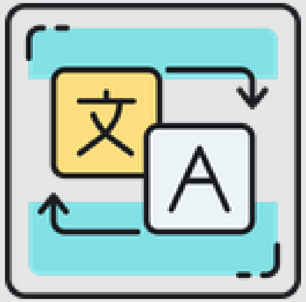


Public, Private and Elastic IP addresses

Name	Description
Public IP address	Lost when the instance is stopped Used in Public Subnets No charge Associated with a private IP address on the instance Cannot be moved between instances
Private IP address	Retained when the instance is stopped Used in Public and Private Subnets
Elastic IP address	Static Public IP address You are charged if not used Associated with a private IP address on the instance Can be moved between instances and Elastic Network Adapters

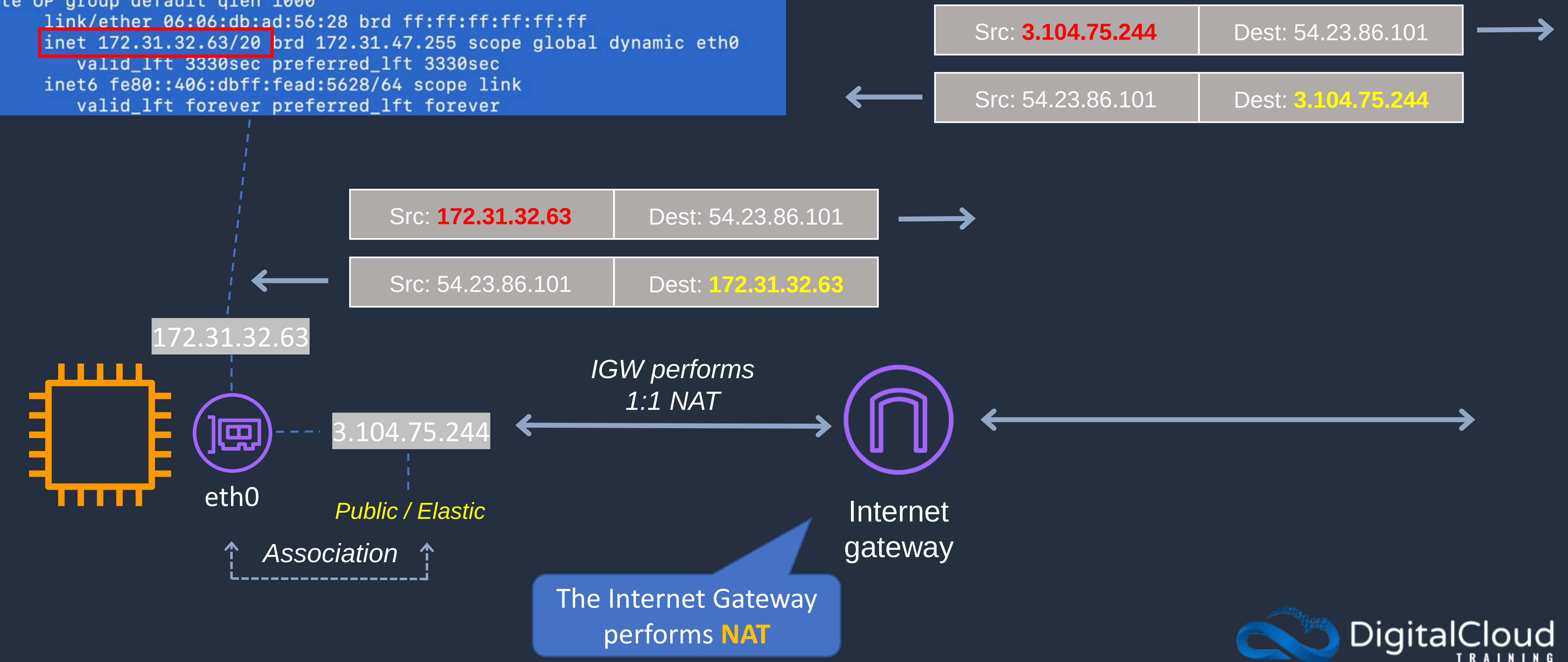
DigitalCloud TRAINING

NAT for Public Addresses



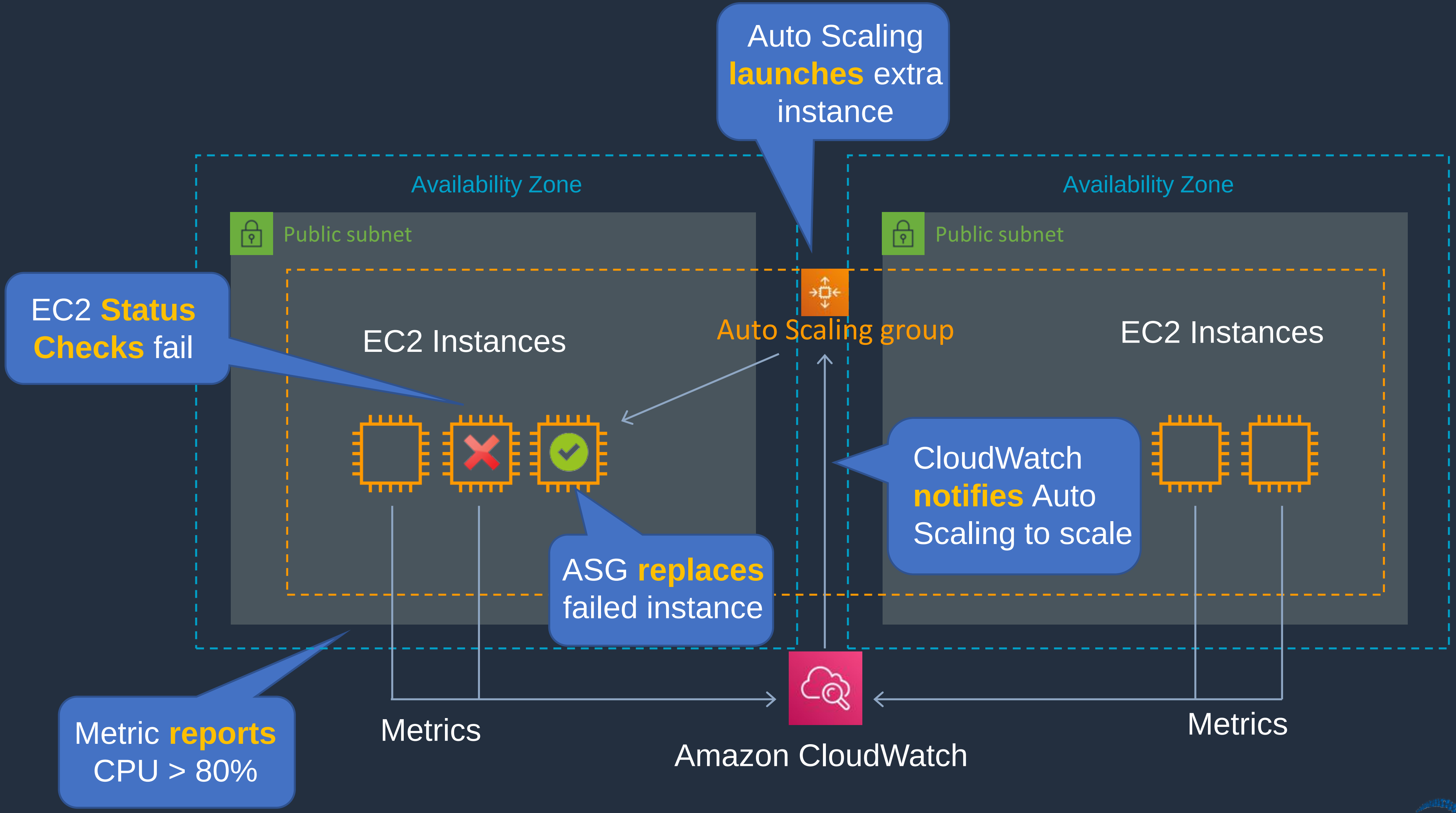
NAT for Public Addresses

```
[ec2-user@ip-172-31-32-63 ~]$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc pfifo_fast sta
te UP group default qlen 1000
    link/ether 06:06:db:ad:56:28 brd ff:ff:ff:ff:ff:ff
    inet 172.31.32.63/20 brd 172.31.47.255 scope global dynamic eth0
       valid_lft 3330sec preferred_lft 3330sec
    inet6 fe80::406:dbff:fead:5628/64 scope link
       valid_lft forever preferred_lft forever
```



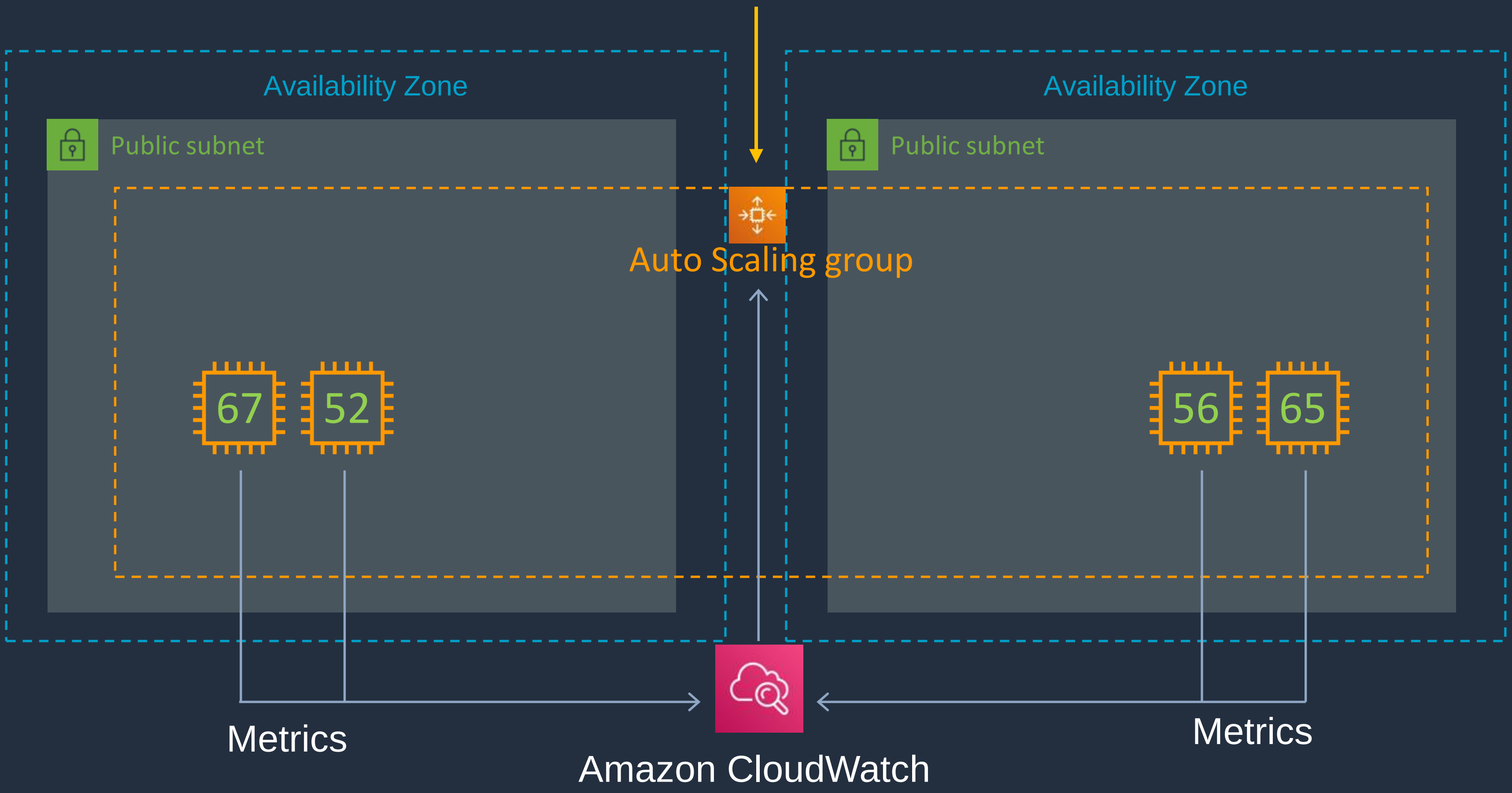
Advanced Auto Scaling

Refresher: Auto Scaling Basics



Dynamic Scaling – Target Tracking

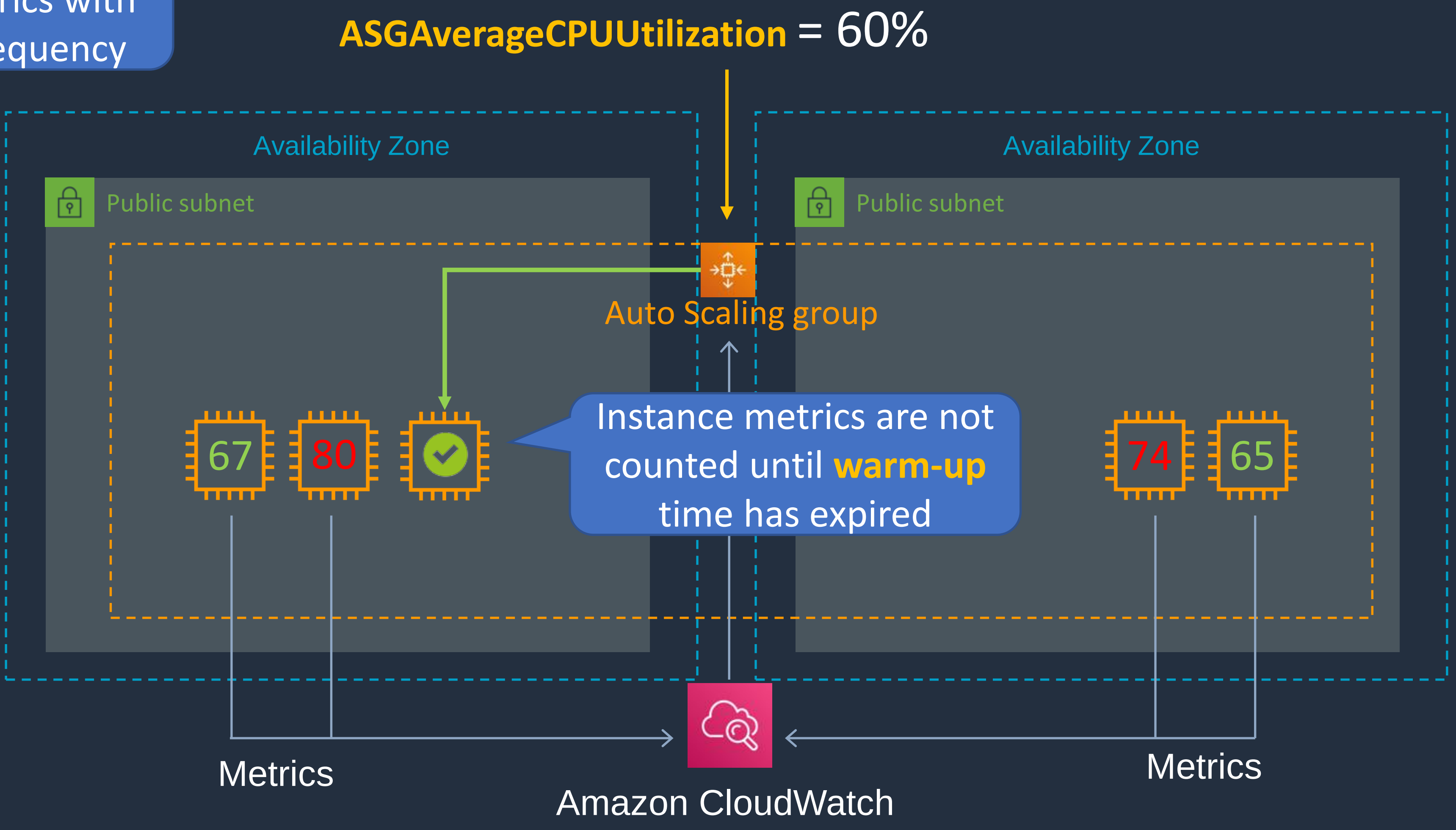
ASGAverageCPUUtilization = 60%



Average CPU = 60%

Dynamic Scaling – Target Tracking

AWS recommend scaling on metrics with a **1-minute** frequency



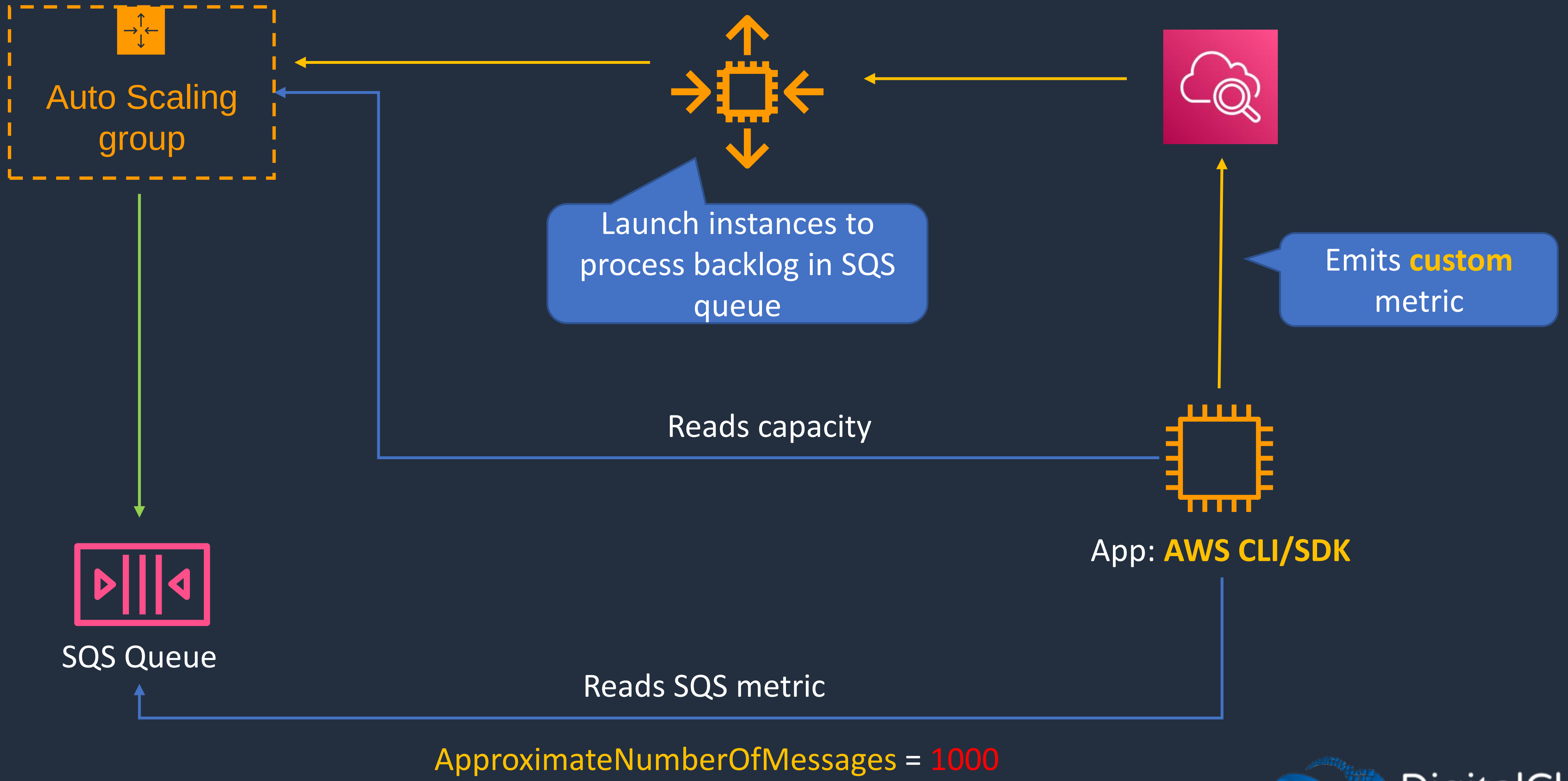
Average CPU = 71.5%

Dynamic Scaling – Target Tracking

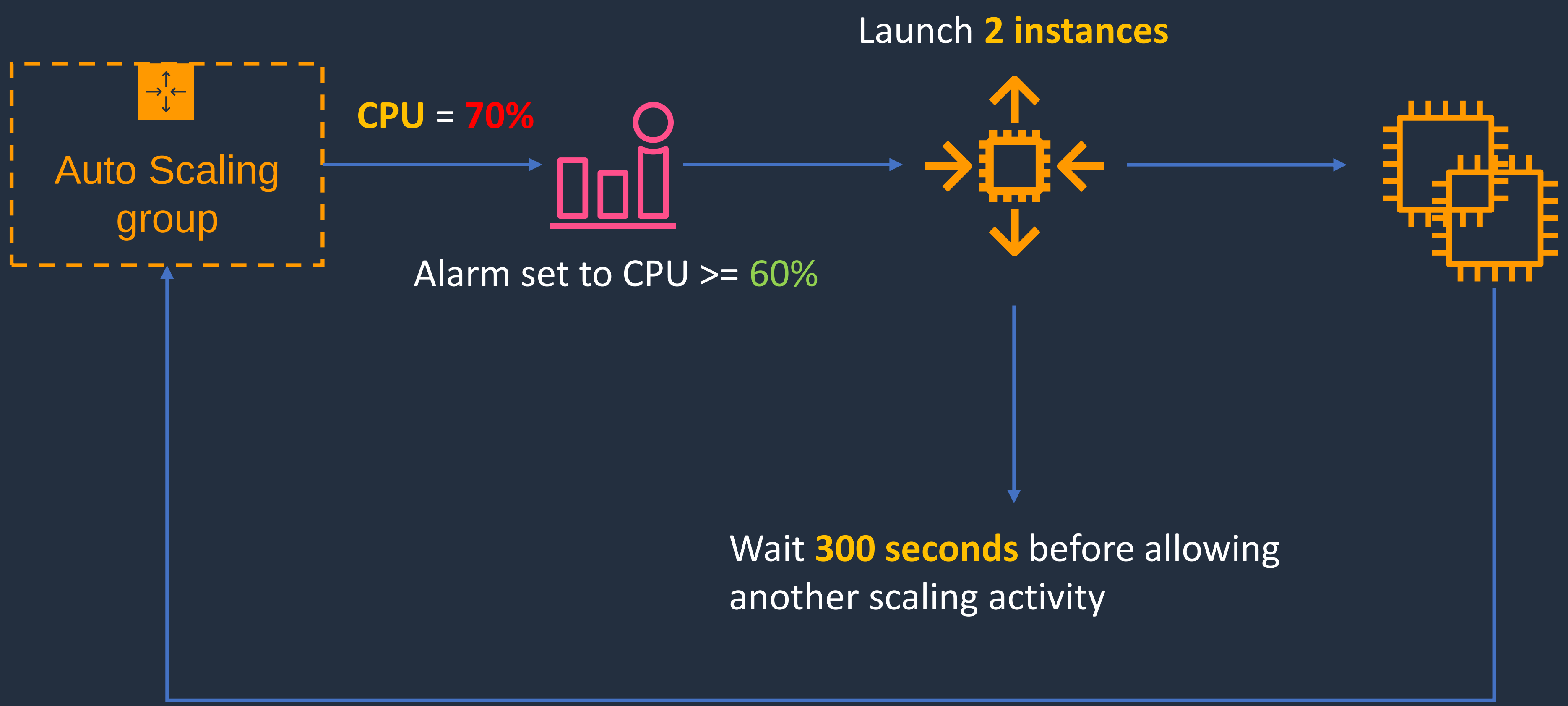
Metrics:

- **ASGAverageCPUUtilization**—Average CPU utilization of the Auto Scaling group
- **ASGAverageNetworkIn**—Average number of bytes received on all network interfaces by the Auto Scaling group
- **ASGAverageNetworkOut**—Average number of bytes sent out on all network interfaces by the Auto Scaling group
- **ALBRequestCountPerTarget**—Number of requests completed per target in an Application Load Balancer target group

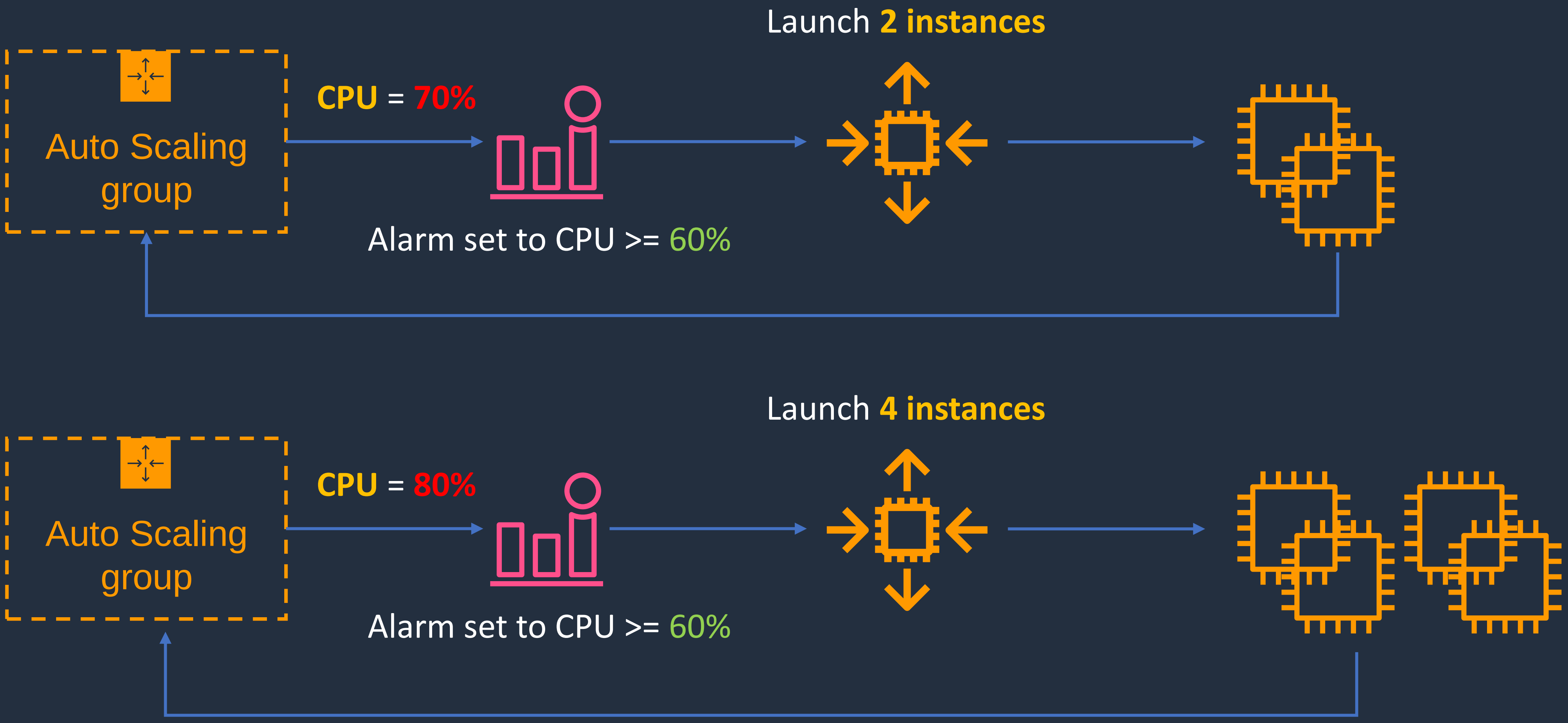
Dynamic Scaling – Target Tracking with SQS



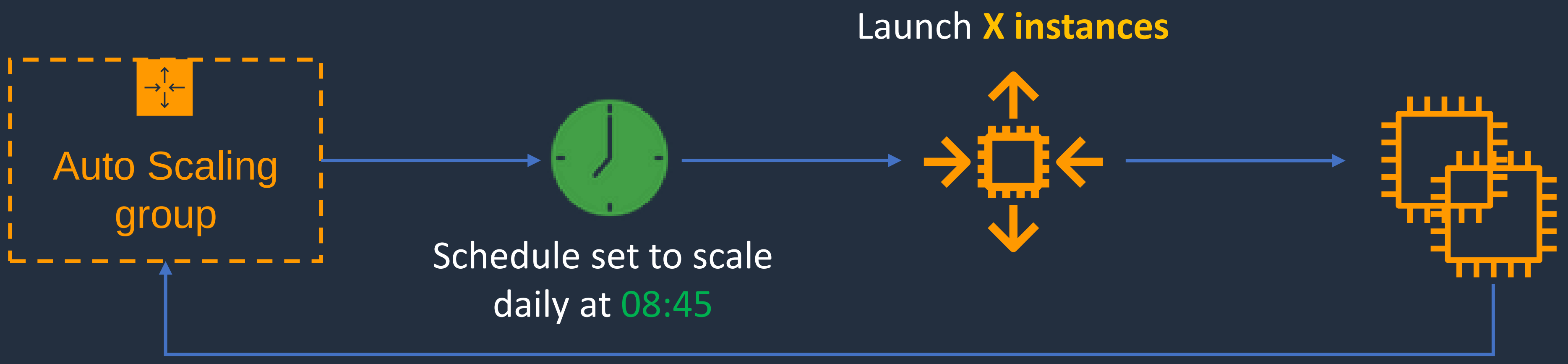
Dynamic Scaling – Simple Scaling



Dynamic Scaling – Step Scaling



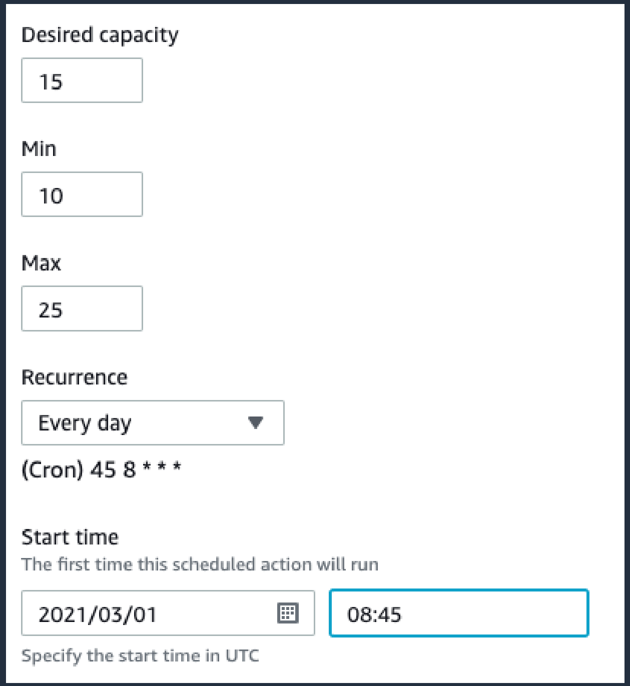
Scheduled Scaling



Attempts to maintain **desired** count ->

The **minimum** instances running at any time ->

The **maximum** instances that can run ->


Desired capacity
15
Min
10
Max
25
Recurrence
Every day
(Cron) 45 8 * * *
Start time
The first time this scheduled action will run
2021/03/01 08:45
Specify the start time in UTC

Scaling Processes

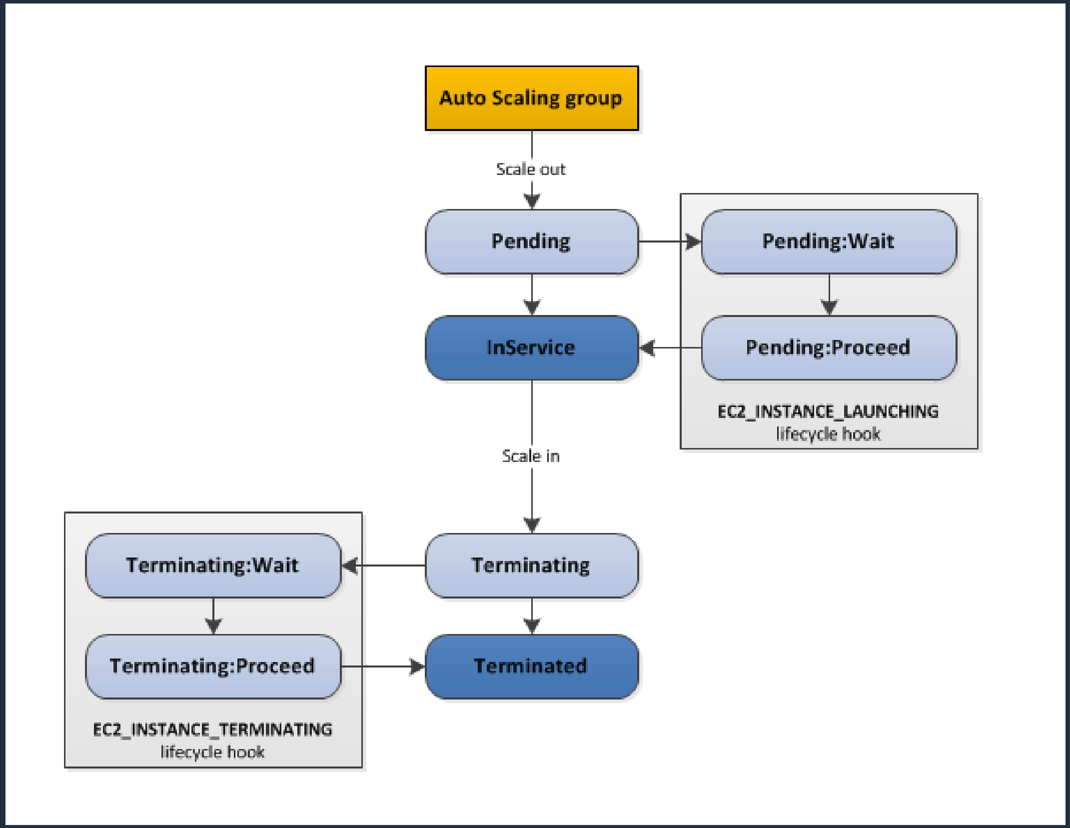
- **Launch** – Adds a new EC2 instance to an Auto Scaling group
- **Terminate** – Removes an EC2 instance from the group
- **AddToLoadBalancer** – Adds instances to an attached ELB or TG
- **AlarmNotification** – Accepts notifications from CloudWatch alarms that are associated with the group's scaling policies
- **AZRebalance** – Balances the number of EC2 instances in the group evenly across all of the specified Availability Zones
- **HealthCheck** – Checks the health of the instances and marks an instance as unhealthy if Amazon EC2 or Elastic Load Balancing tells Amazon EC2 Auto Scaling that the instance is unhealthy
- **ReplaceUnhealthy** – Terminates instances that are marked as unhealthy and then creates new instances to replace them
- **ScheduledActions** – Performs scheduled scaling actions

Additional Scaling Settings

- **Cooldowns** – Used with simple scaling policy to prevent Auto Scaling from launching or terminating before effects of previous activities are visible. Default value is 300 seconds (5 minutes)
- **Termination Policy** – Controls which instances to terminate first when a scale-in event occurs.
- **Termination Protection** – Prevents Auto Scaling from terminating protected instances
- **Standby State** – Used to put an instance in the InService state into the Standby state, update or troubleshoot the instance

Additional Scaling Settings

- **Lifecycle Hooks** – Used to perform custom actions by pausing instances as the ASG launches or terminates them.

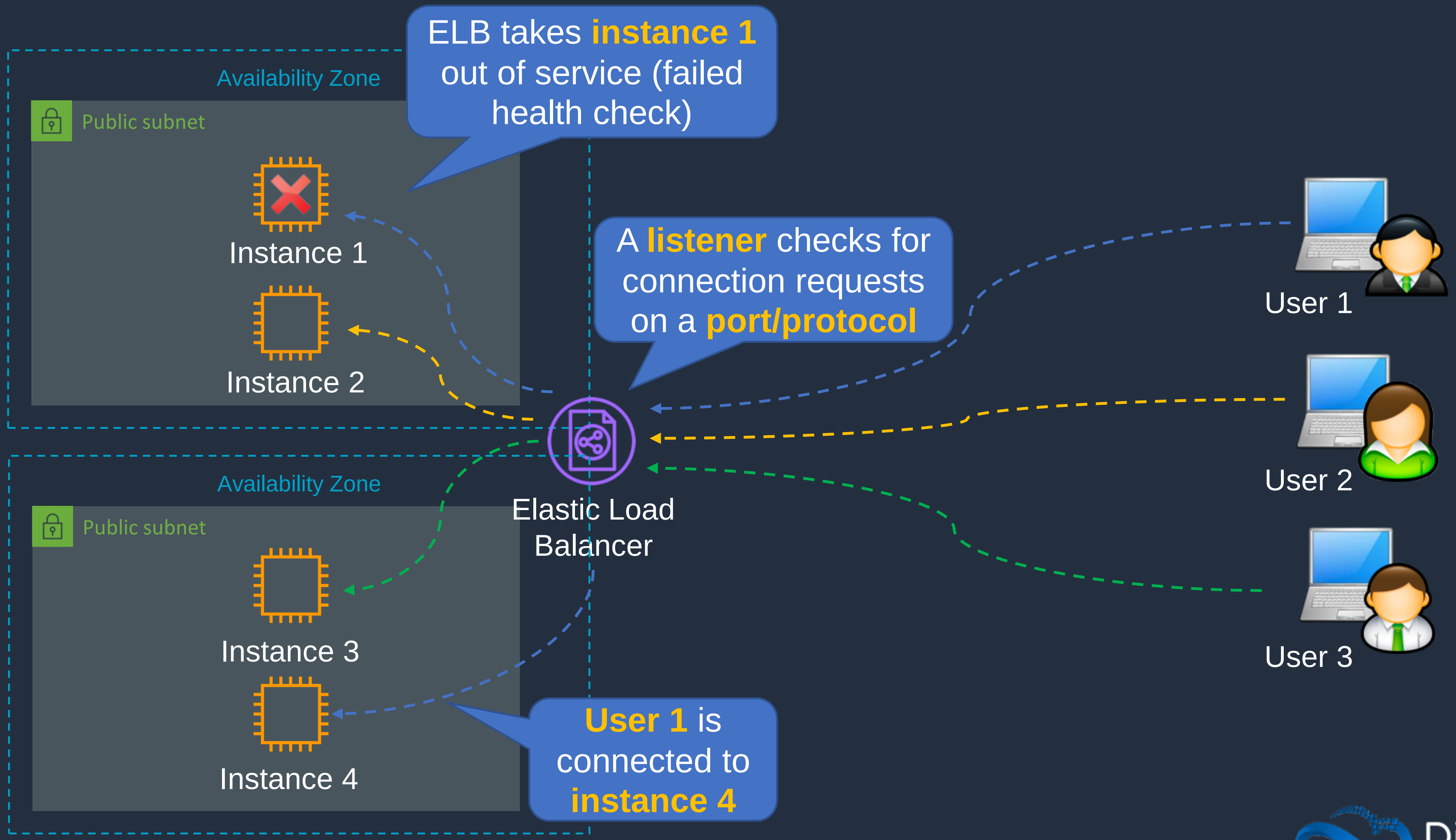


Create an ASG and ALB with the CLI

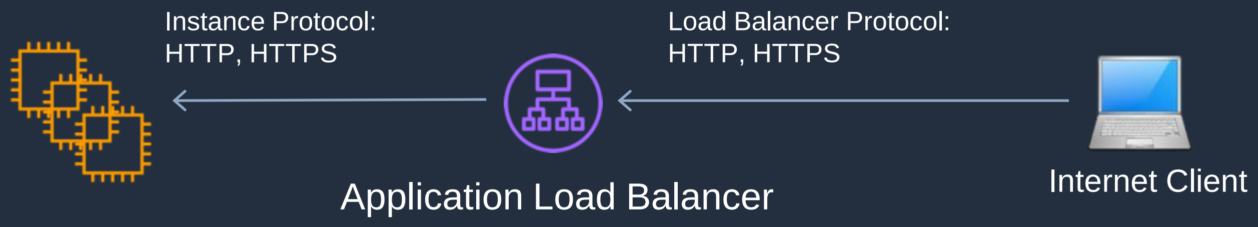
Create a Lifecycle Hook

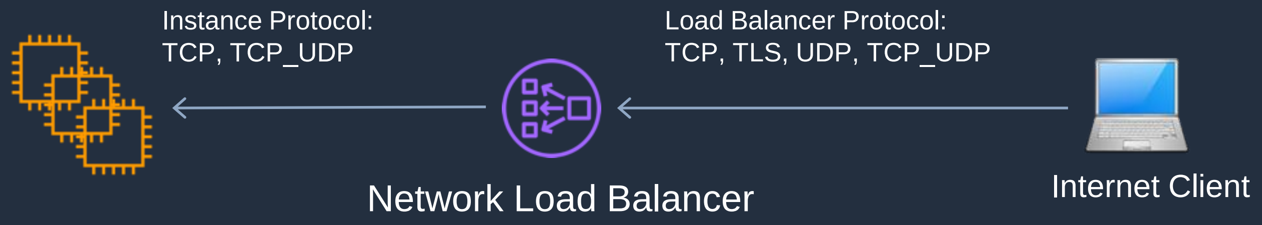
Types of Elastic Load Balancer (ELB)

Refresher: ELB Basics



Types of Elastic Load Balancer (ELB)





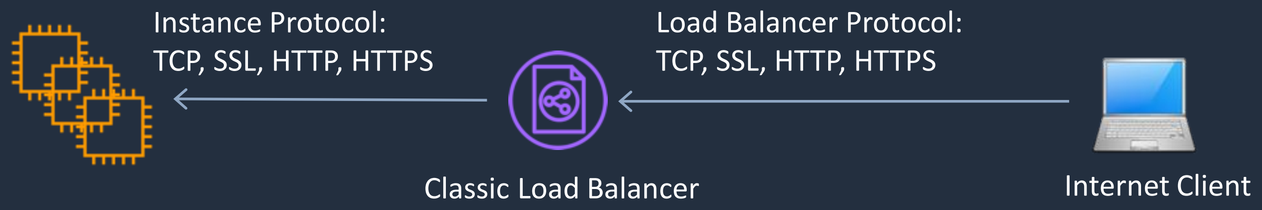
Application Load Balancer

- Operates at the request level
- Routes based on the content of the request (layer 7)
- Supports path-based routing, host-based routing, query string parameter-based routing, and source IP address-based routing
- Supports instances, IP addresses, Lambda functions and containers as targets

Network Load Balancer

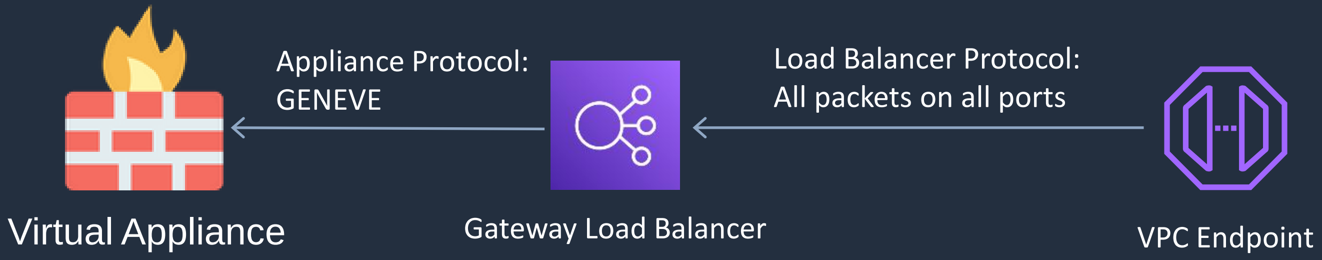
- Operates at the connection level
- Routes connections based on IP protocol data (layer 4)
- Offers ultra high performance, low latency and TLS offloading at scale
- Can have a static IP / Elastic IP
- Supports UDP and static IP addresses as targets

Types of Elastic Load Balancer (ELB)



Classic Load Balancer

- Old generation; not recommended for new applications
- Performs routing at Layer 4 and Layer 7
- Use for existing applications running in EC2-Classic



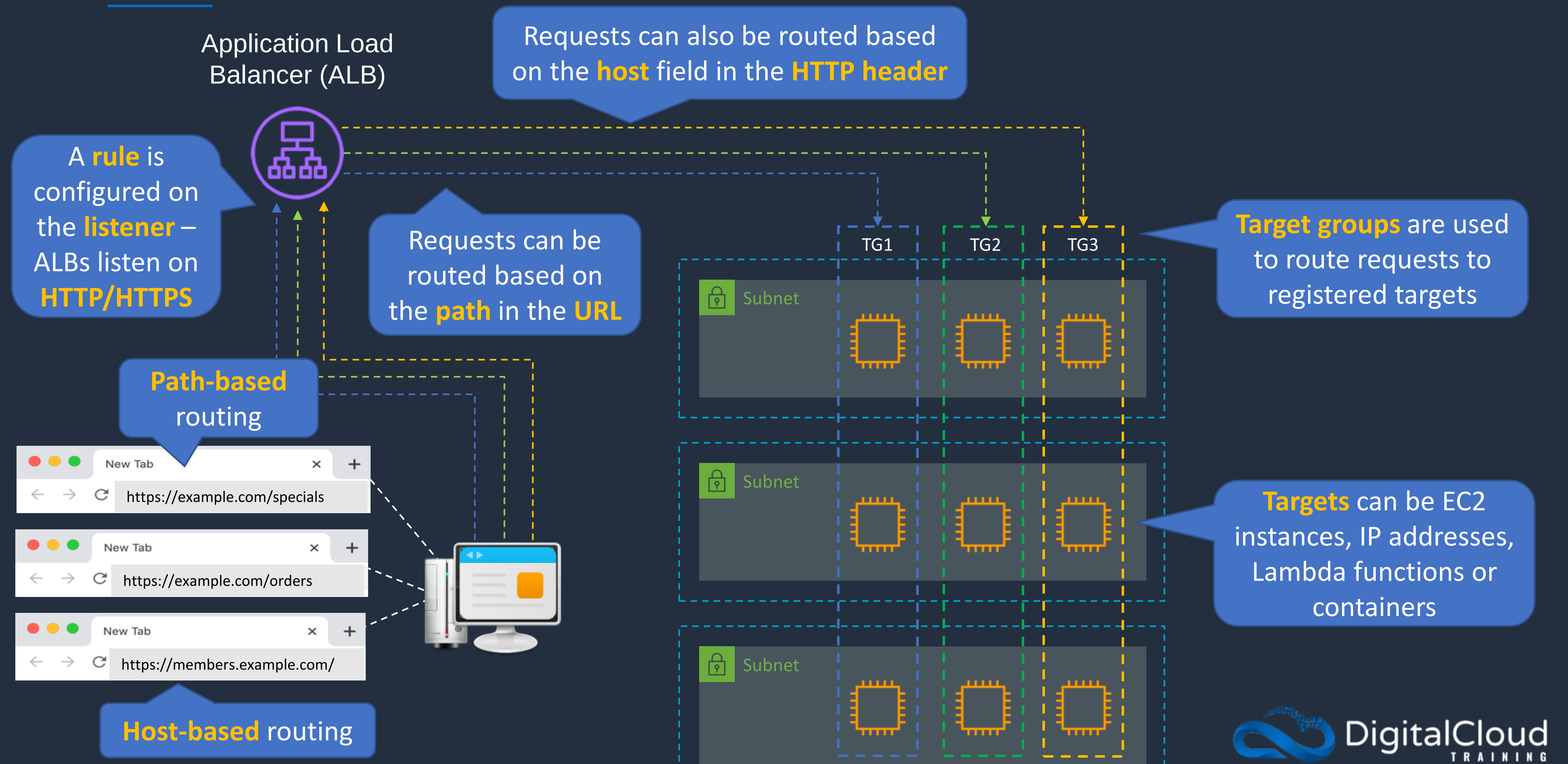
New and **not** yet on the exam

Gateway Load Balancer

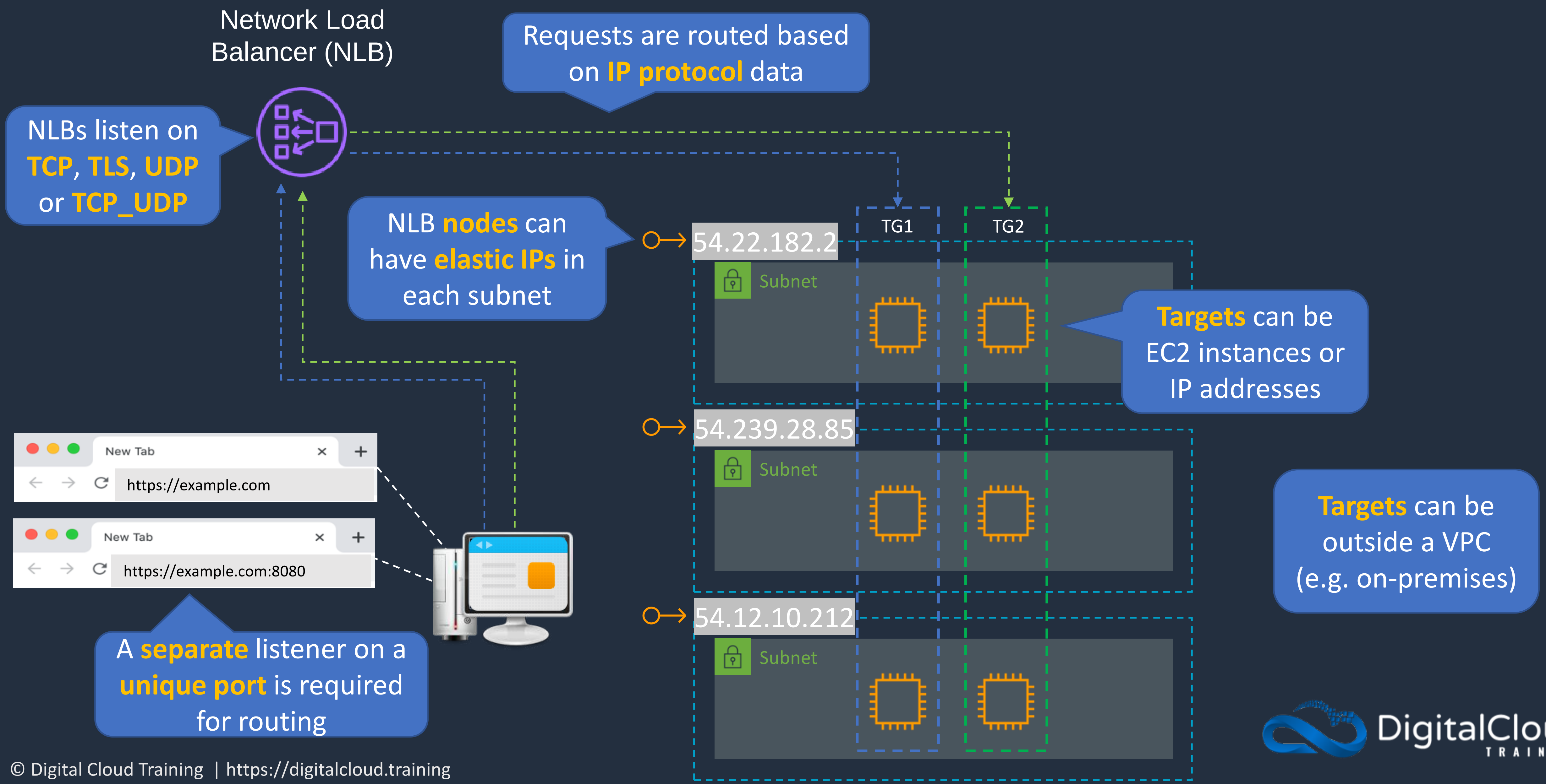
- Used in front of virtual appliances such as firewalls, IDS/IPS, and deep packet inspection systems.
- Operates at Layer 3 – listens for all packets on all ports
- Forwards traffic to the TG specified in the listener rules
- Exchanges traffic with appliances using the GENEVE protocol on port 6081

Routing with ALB and NLB

Application Load Balancer (ALB)

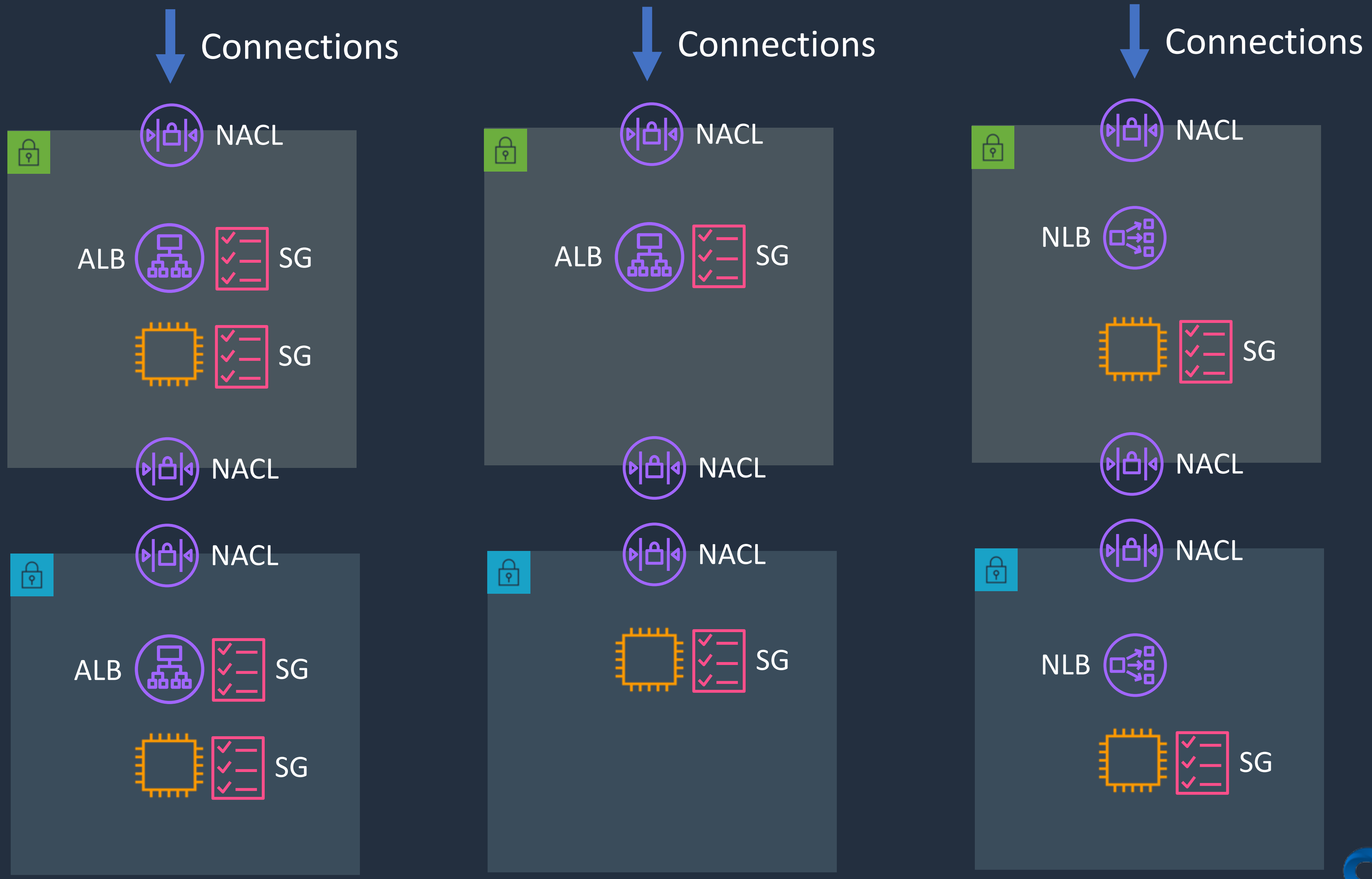


Network Load Balancer (NLB)



ALB and NLB Access Control and SSL/TLS

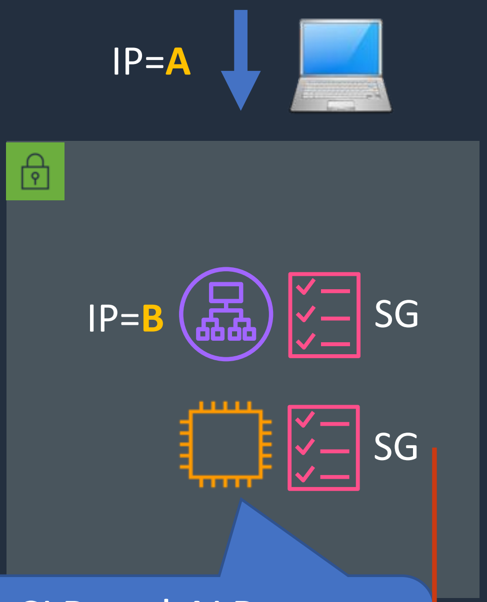
Access Control with ALB and NLB



What's the Source IP Address the App sees?

Note: X-forwarded-for can be used with ALB to capture client IP

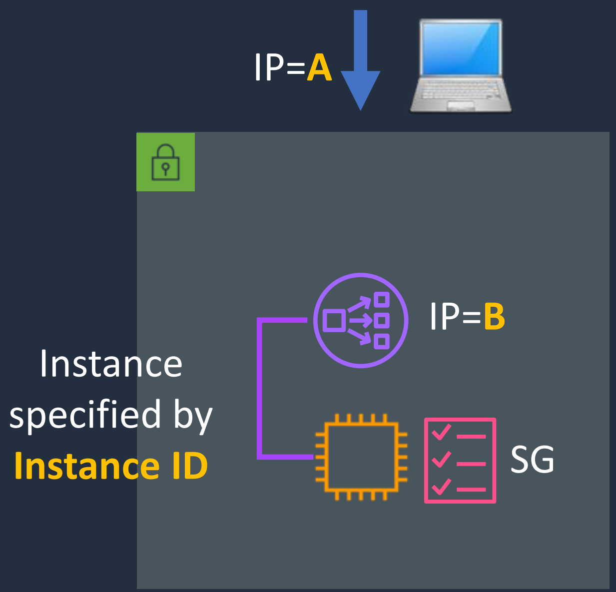
AWS ALB



CLB and ALB use **private IP** of their **ENIs** as source address

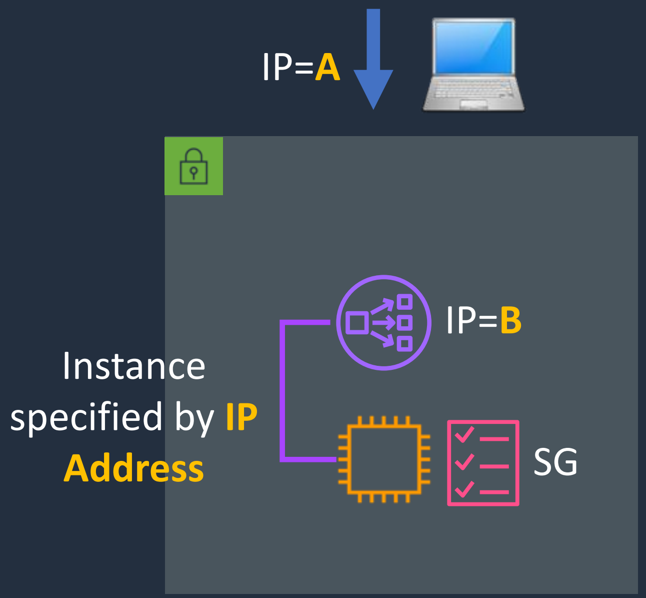
Source	Protocol	Port
IP=B	TCP	80

AWS NLB



Source	Protocol	Port
IP=A	TCP	80

AWS NLB

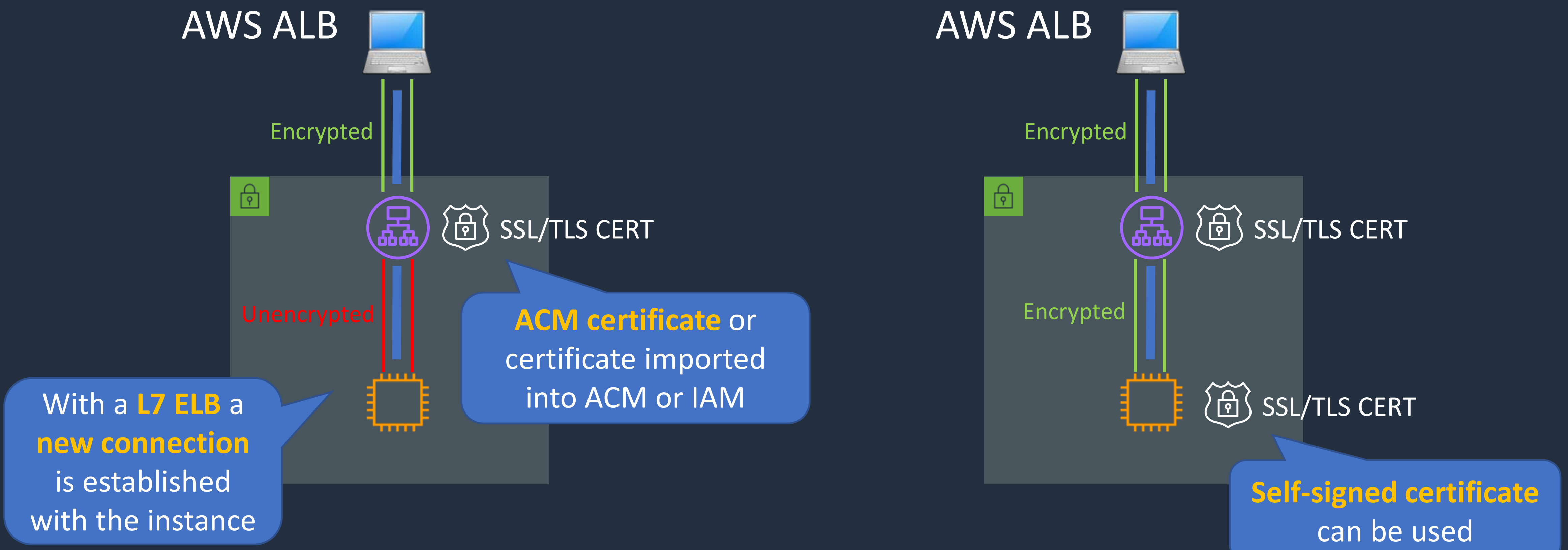


Applicable to **TCP and TLS** – for **UDP and TCP_UDP** should be IP=**A**

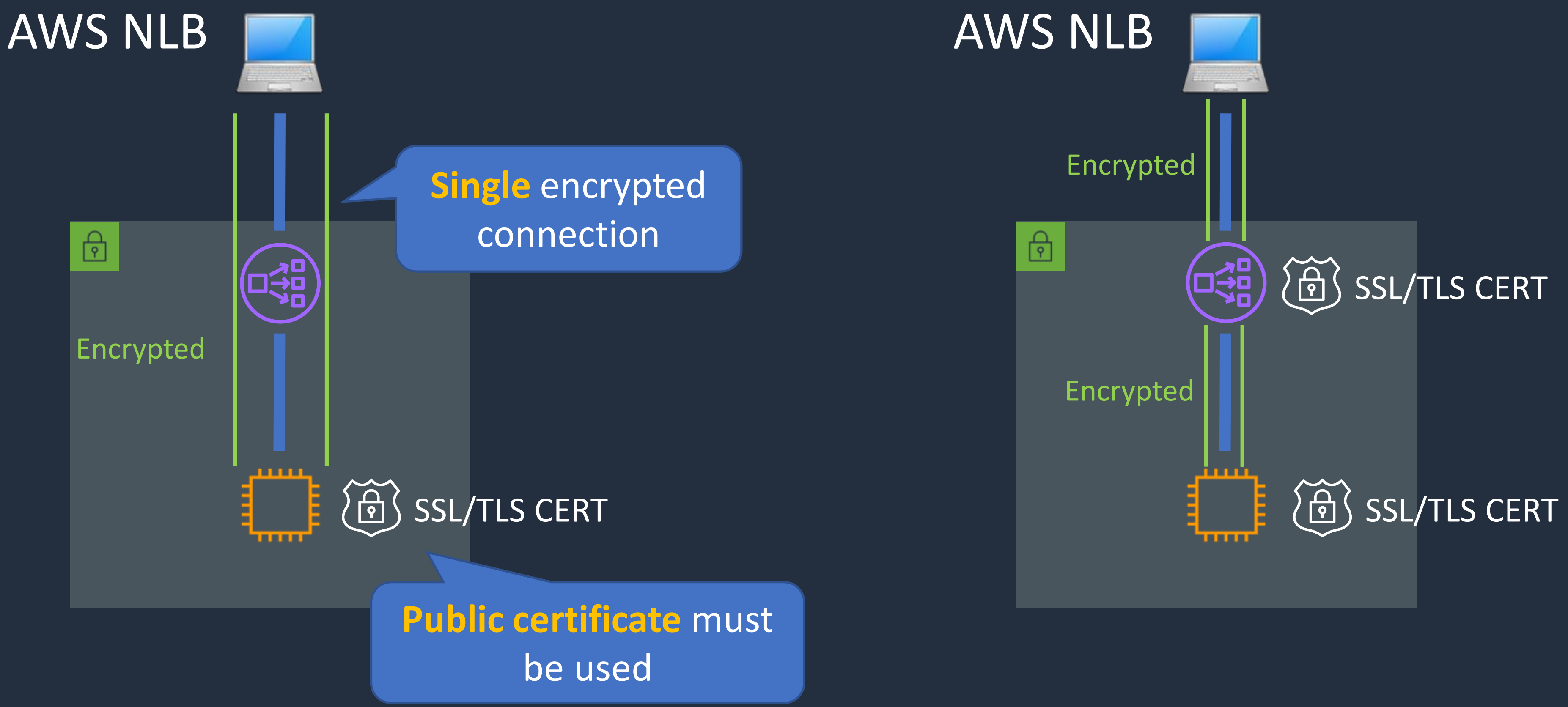
Source	Protocol	Port
IP=B	TCP	80

When using an NLB with a **VPC Endpoint** or **AWS GA** source IPs are **private IPs** of **NLB nodes**

SSL/TLS Termination



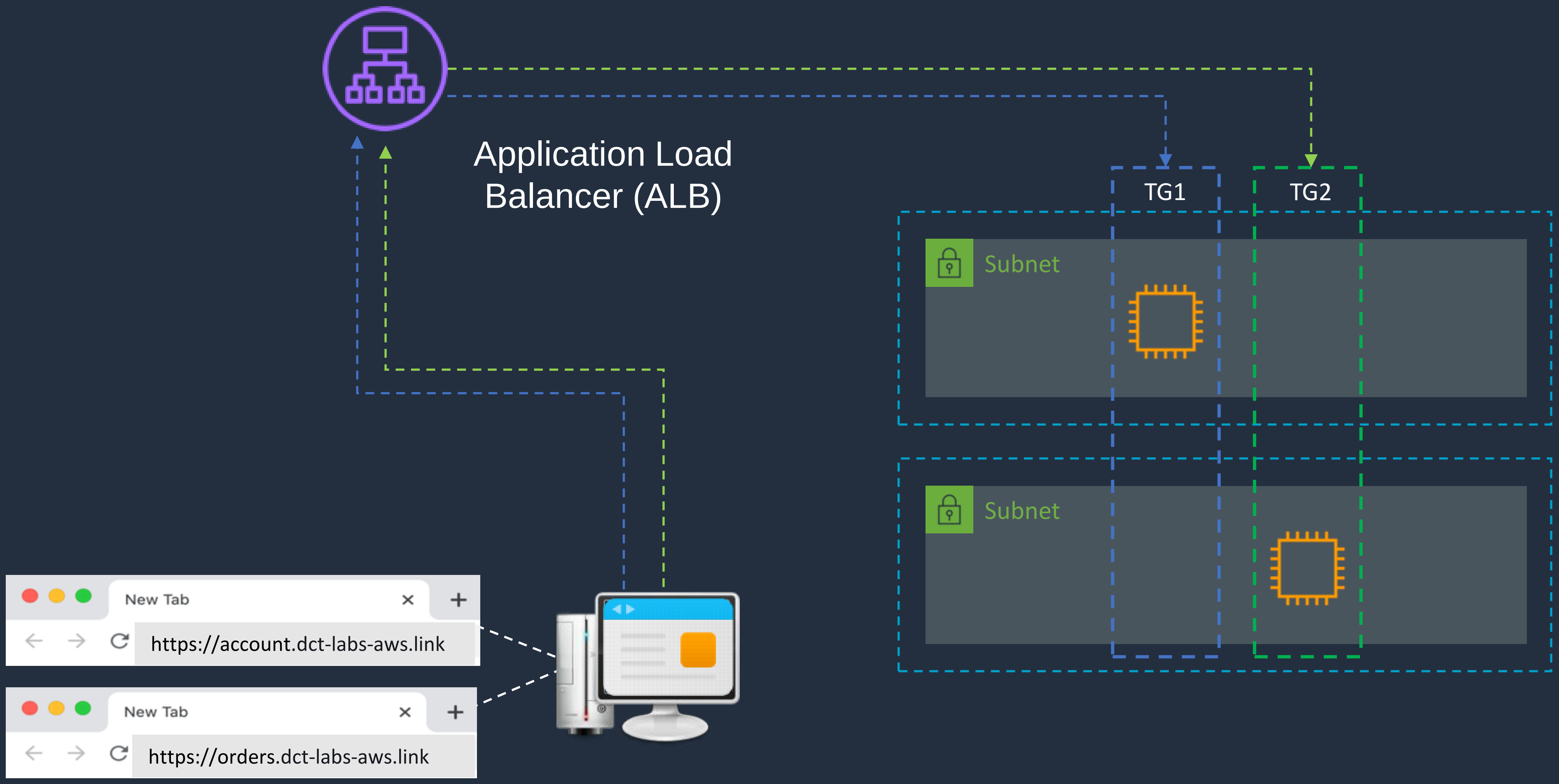
SSL/TLS Termination



Register Domain with Route 53 (Optional)

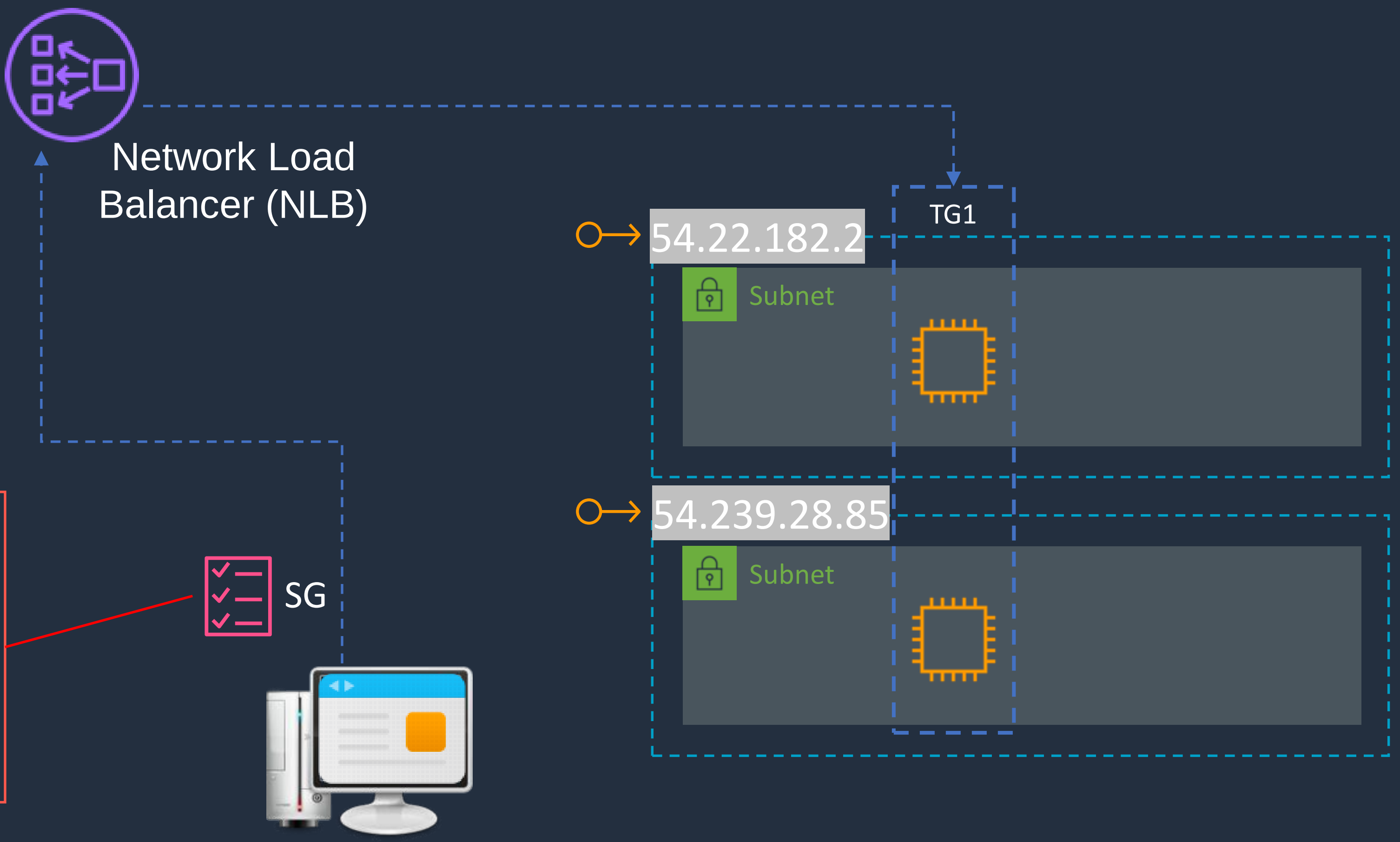
Request Routing with ALB

Application Load Balancer (ALB)



NLB Static IPs and Whitelisting

Network Load Balancer (NLB)

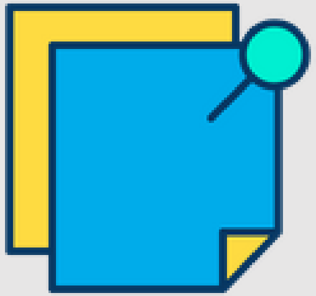


Security Group

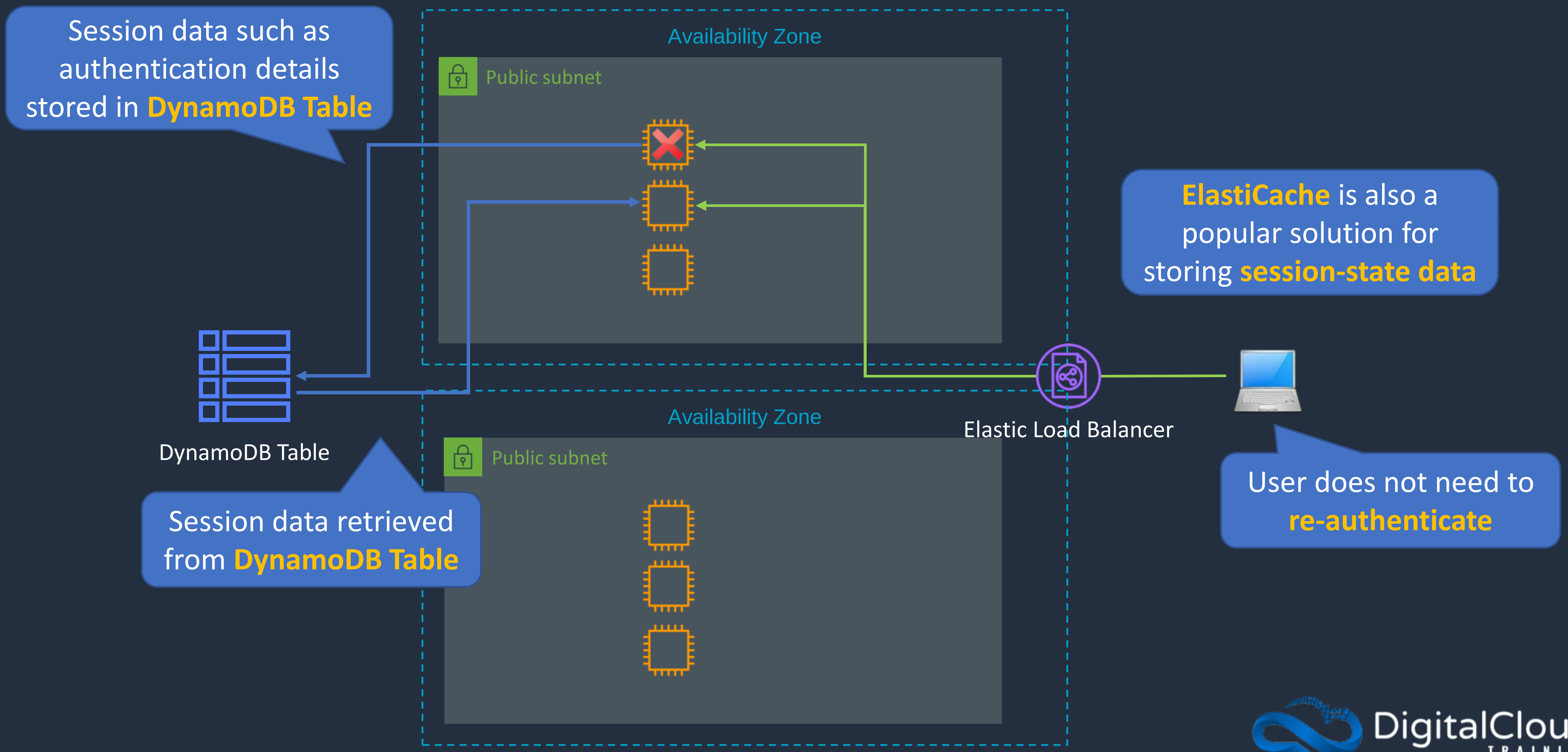
Protocol	Port	Destination
HTTP	80	54.22.182.2
		54.239.28.85

DigitalCloud TRAINING

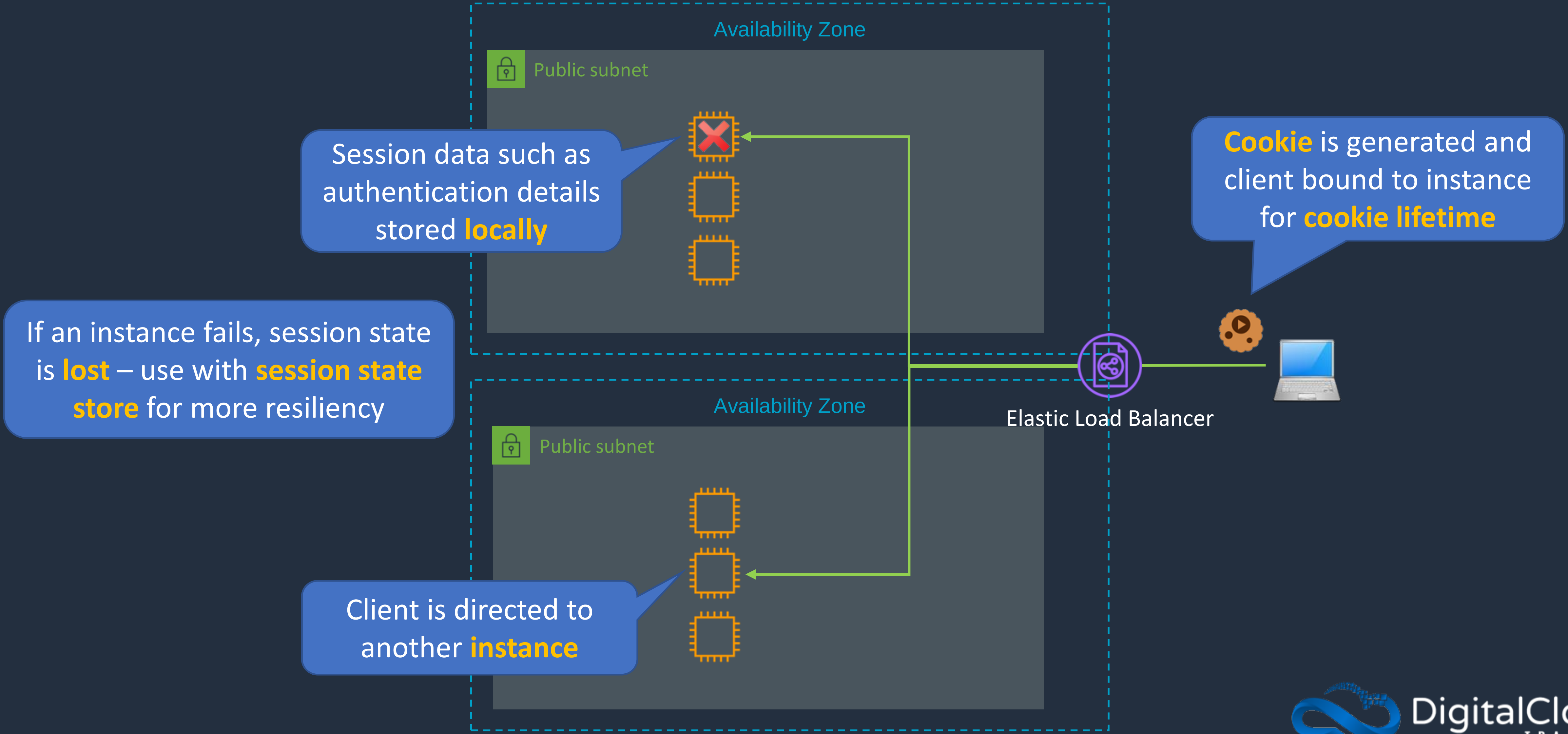
Session State and Session Stickiness



Storing Session State

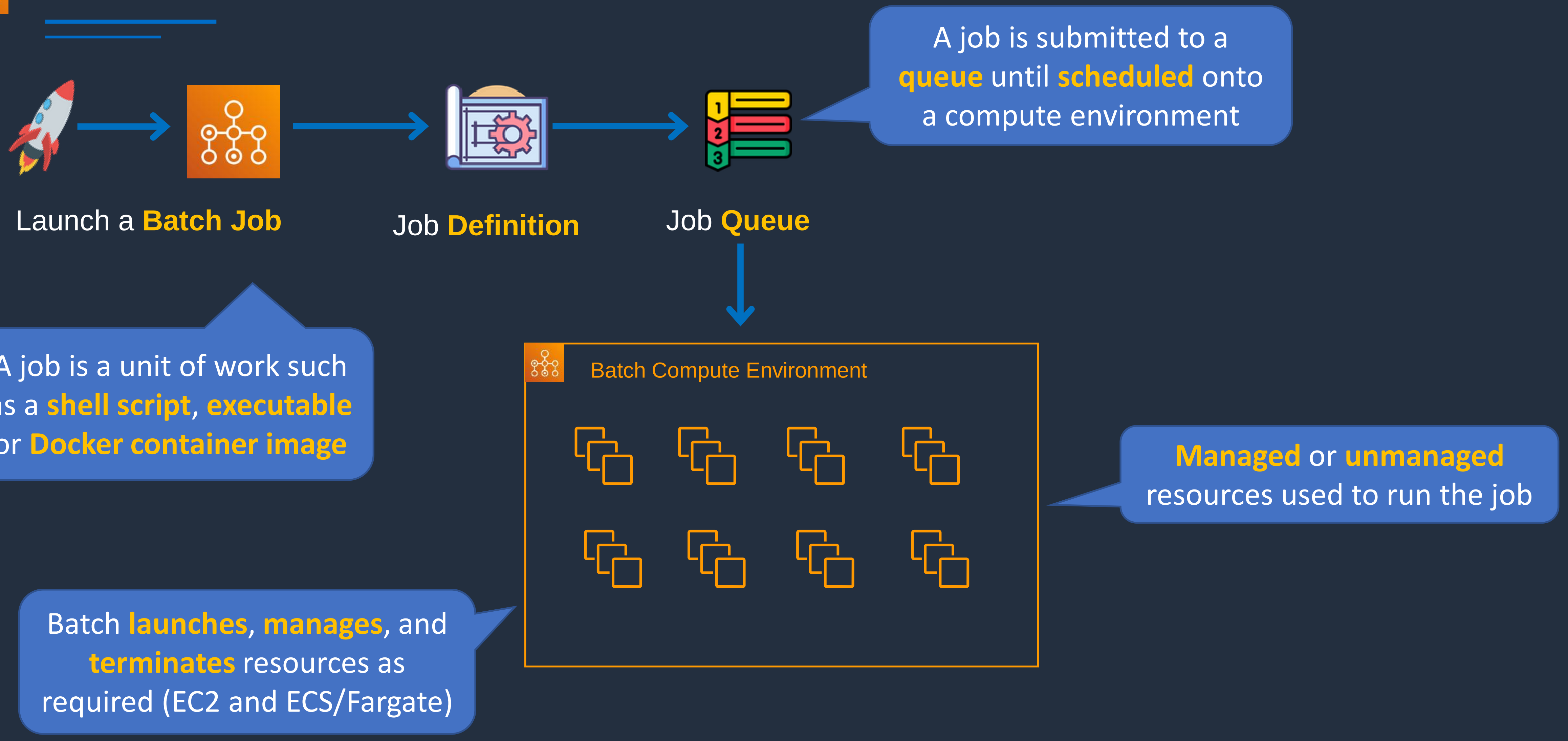


Sticky Sessions



AWS Batch

AWS Batch



Launch a **Batch Job**
Job **Definition**
Job **Queue**
A job is submitted to a **queue** until **scheduled** onto a compute environment
A job is a unit of work such as a **shell script**, **executable** or **Docker container image**
Batch Compute Environment
Managed or **unmanaged** resources used to run the job
Batch **launches**, **manages**, and **terminates** resources as required (EC2 and ECS/Fargate)

Amazon LightSail

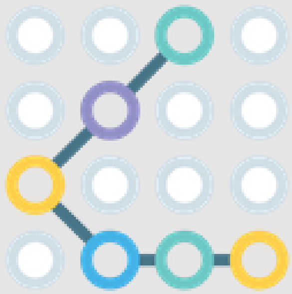
Amazon LightSail

- Low cost and ideal for users with less technical expertise
- Compute, storage, and network
- Preconfigured virtual servers

- Virtual servers, databases and load balancers
- SSH and RDP access
- Can access Amazon VPC

Exam tip: typically comes up in use cases where an easy method of deploying a virtual server is required by a user with little or no AWS expertise

Architecture Patterns - Compute



Architecture Patterns - Compute

Requirement	Solution
High availability and elastic scalability for web servers	Use Amazon EC2 Auto Scaling and an Application Load Balancer across multiple AZs
Low-latency connections over UDP to a pool of instances running a gaming application	Use a Network Load Balancer with a UDP listener
Clients need to whitelist static IP addresses for a highly available load balanced application in an AWS Region.	Use an NLB and create static IP addresses in each AZ

DigitalCloud TRAINING

Architecture Patterns - Compute

Requirement	Solution
Application on EC2 in an Auto Scaling group requires disaster recovery across Regions	Create an ASG in a second Region with the capacity set to 0. Take snapshots and copy them across Regions (Lambda or DLM)
Application on EC2 must scale in larger increments if a big increase in traffic occurs, compared to small increases in traffic	Use Auto Scaling with a Step Scaling policy and configure a larger capacity increase
Need to scale EC2 instances behind an ALB based on the number of requests completed by each instance	Configure a target tracking policy using the ALBRequestCountPerTarget metric

Architecture Patterns - Compute

Requirement	Solution
Need to run a large batch computing job at the lowest cost. Must be managed. Nodes can pick up where others left off in case of interruption	Use a managed AWS Batch job and use EC2 Spot instances
A tightly coupled High Performance Computing (HPC) workload requires low-latency between nodes and optimum network performance	Launch EC2 instances in a single AZ in a cluster placement group and use an Elastic Fabric Adapter (EFA)
LOB application receives weekly burst of traffic and must scale for short periods – need the most cost-effective solution	Use reserved instances for minimum required workload and then use Spot instances for the bursts in traffic

Architecture Patterns - Compute

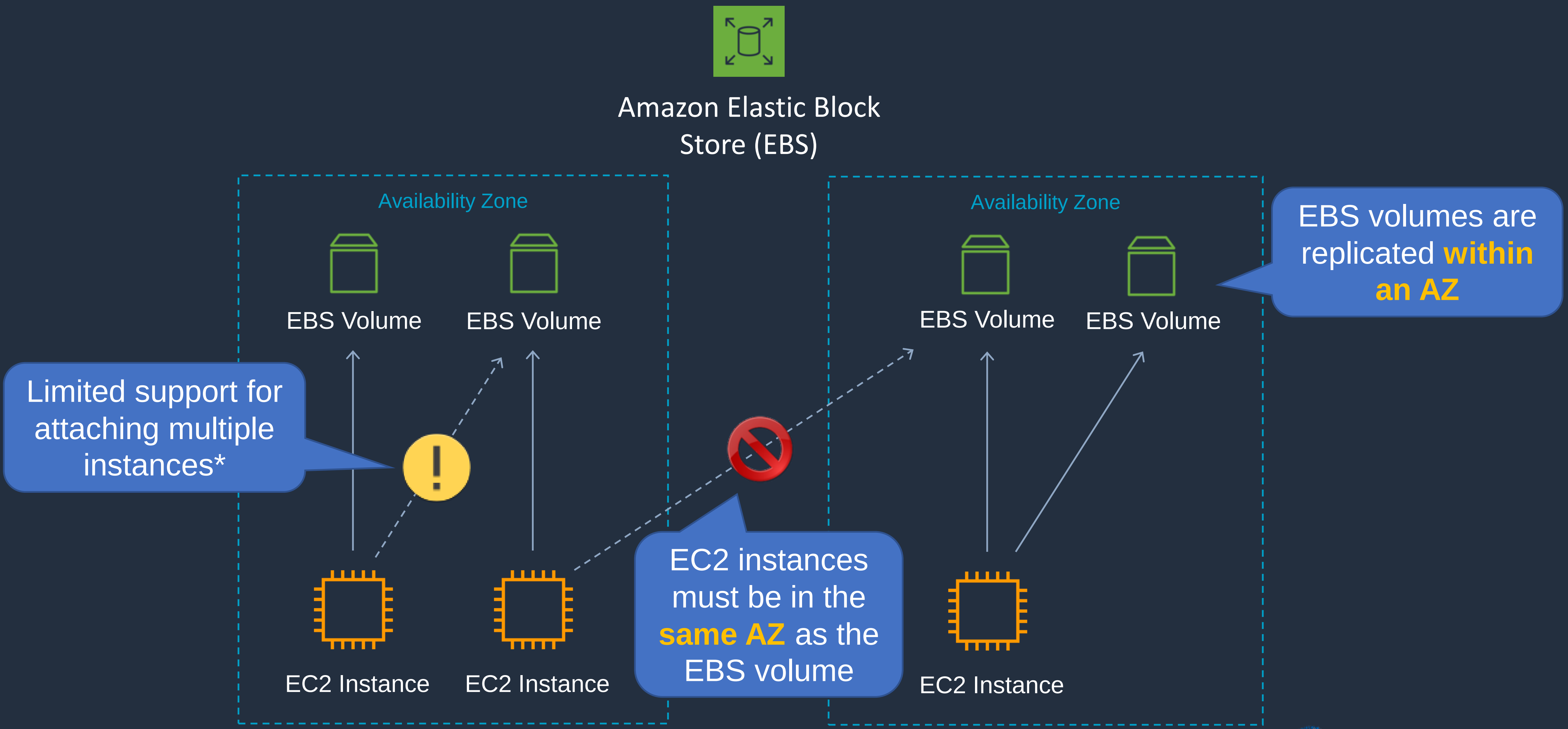
Requirement	Solution
Application must startup quickly when launched by ASG but requires app dependencies and code to be installed	Create an AMI that includes the application dependencies and code
Application runs on EC2 behind an ALB. Once authenticated users should not need to reauthenticate if an instance fails	Enable Sticky session for the target group or alternatively use a session store such as DynamoDB

SECTION 8

AWS Storage Services

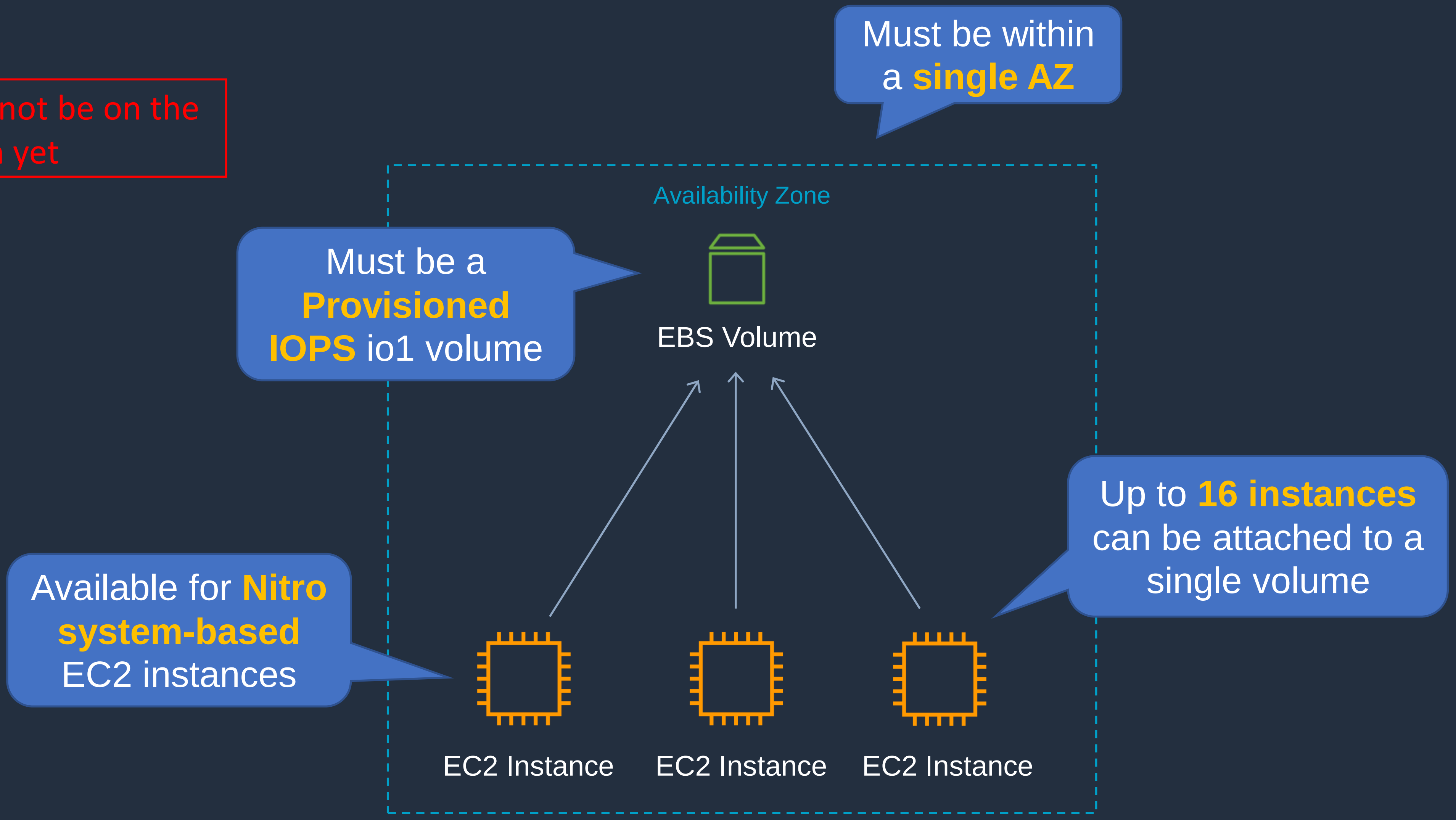
Amazon EBS Deployment and Volume Types

Amazon EBS Deployment



Amazon EBS Multi-Attach

May not be on the exam yet



Amazon EBS SSD-Backed Volumes

	General Purpose SSD		Provisioned IOPS SSD		
Volume type	gp3	gp2	io2 Block Express ‡	io2	io1
Durability	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)	99.999% durability (0.001% annual failure rate)		99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)
Use cases	• Low-latency interactive apps • Development and test environments		Workloads that require sub-millisecond latency, and sustained IOPS performance or more than 64,000 IOPS or 1,000 MiB/s of throughput	• Workloads that require sustained IOPS performance or more than 16,000 IOPS • I/O-intensive database workloads	
Volume size	1 GiB - 16 TiB		4 GiB - 64 TiB	4 GiB - 16 TiB	
Max IOPS per volume (16 KiB I/O)	16,000		256,000	64,000 †	
Max throughput per volume	1,000 MiB/s	250 MiB/s *	4,000 MiB/s	1,000 MiB/s †	
Amazon EBS Multi-attach	Not supported		Not supported	Supported	
Boot volume	Supported				

New and **not** on the exam yet

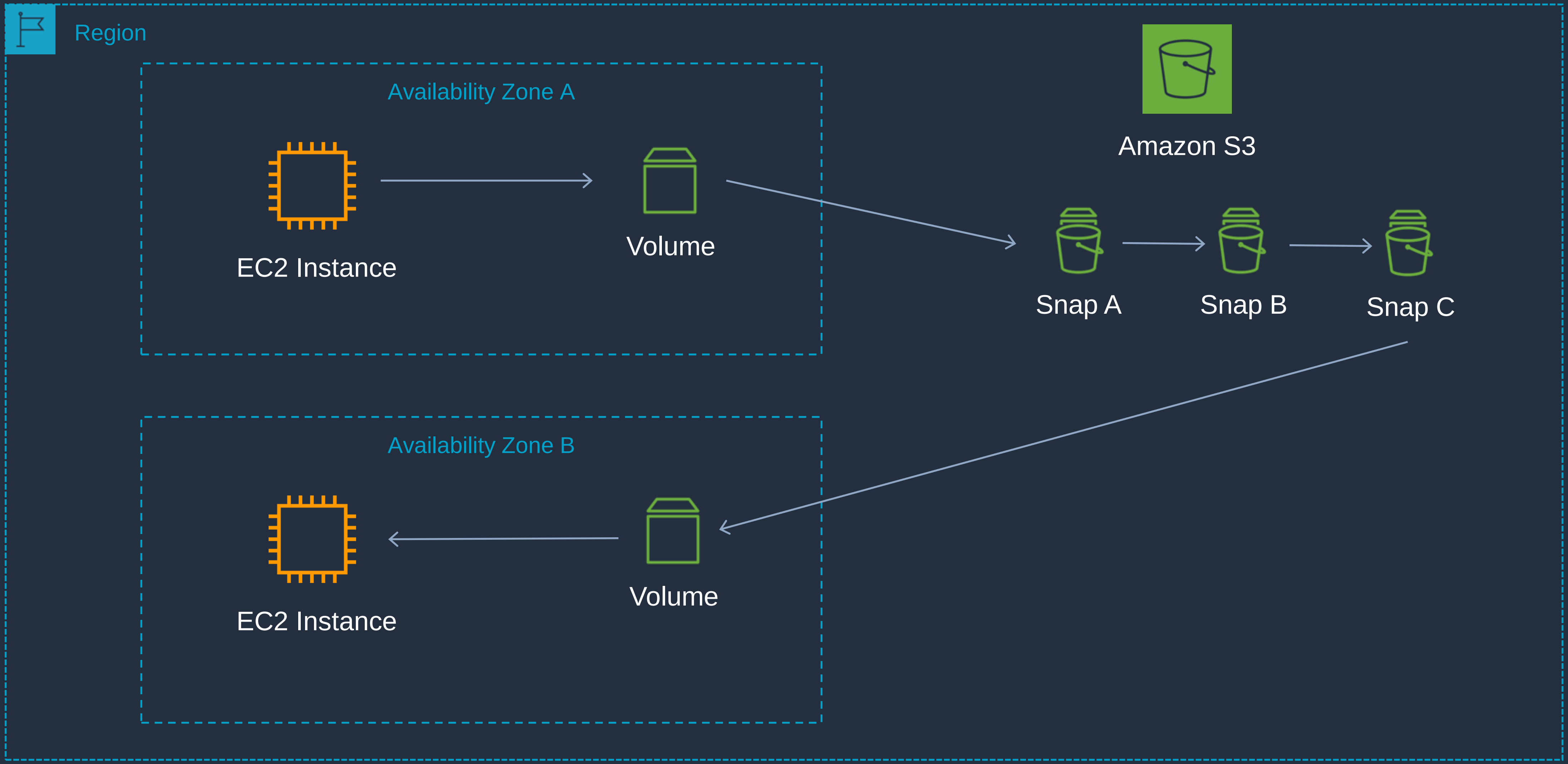
New and **not** on the exam yet

Amazon EBS HDD-Backed Volumes

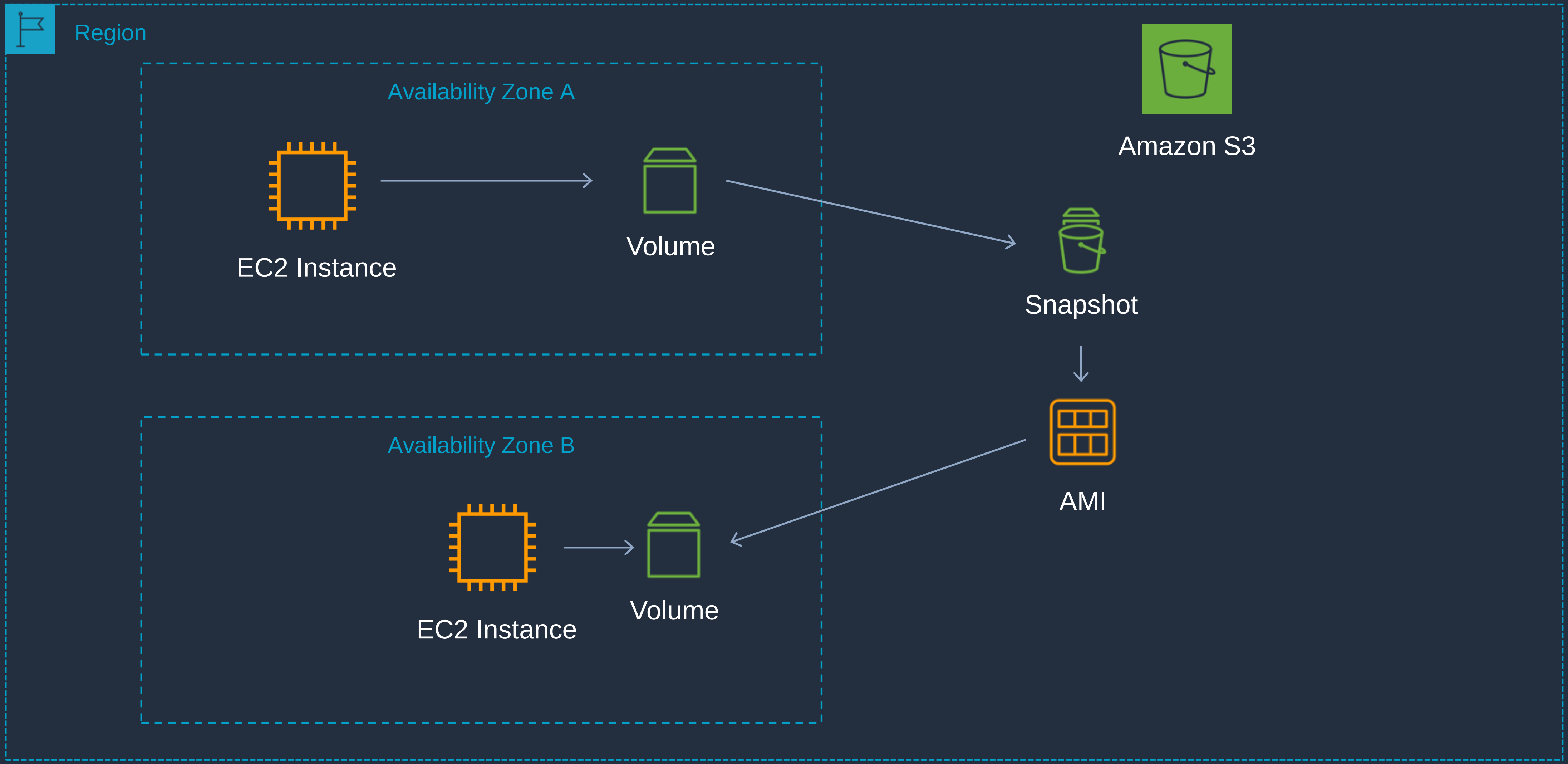
	Throughput Optimized HDD	Cold HDD
Volume type	`st1`	`sc1`
Durability	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)
Use cases	• Big data • Data warehouses • Log processing	• Throughput-oriented storage for data that is infrequently accessed • Scenarios where the lowest storage cost is important
Volume size	125 GiB - 16 TiB	125 GiB - 16 TiB
Max IOPS per volume (1 MiB I/O)	500	250
Max throughput per volume	500 MiB/s	250 MiB/s
Amazon EBS Multi-attach	Not supported	Not supported
Boot volume	Not supported	Not supported

Amazon EBS Copying, Sharing and Encryption

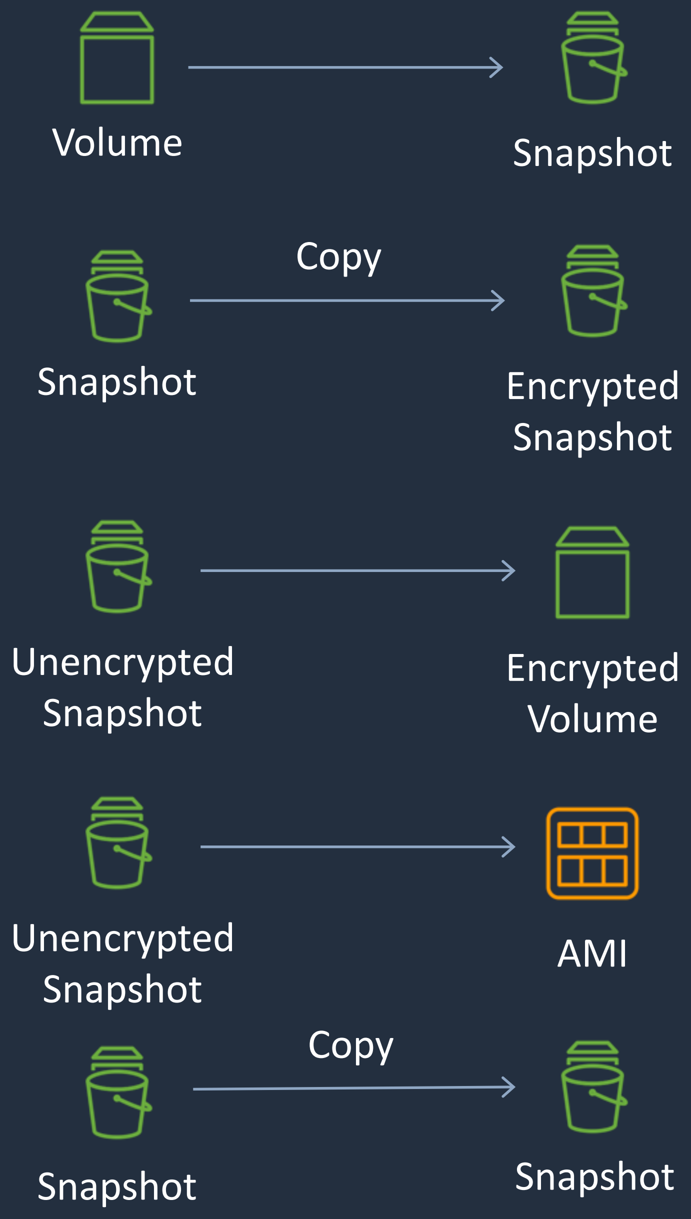
Amazon EBS Copying, Sharing and Encryption



Take Snapshot, Create AMI, Launch New Instance



Copying and Sharing AMIs and Snapshots



- Encryption state retained
- Same region

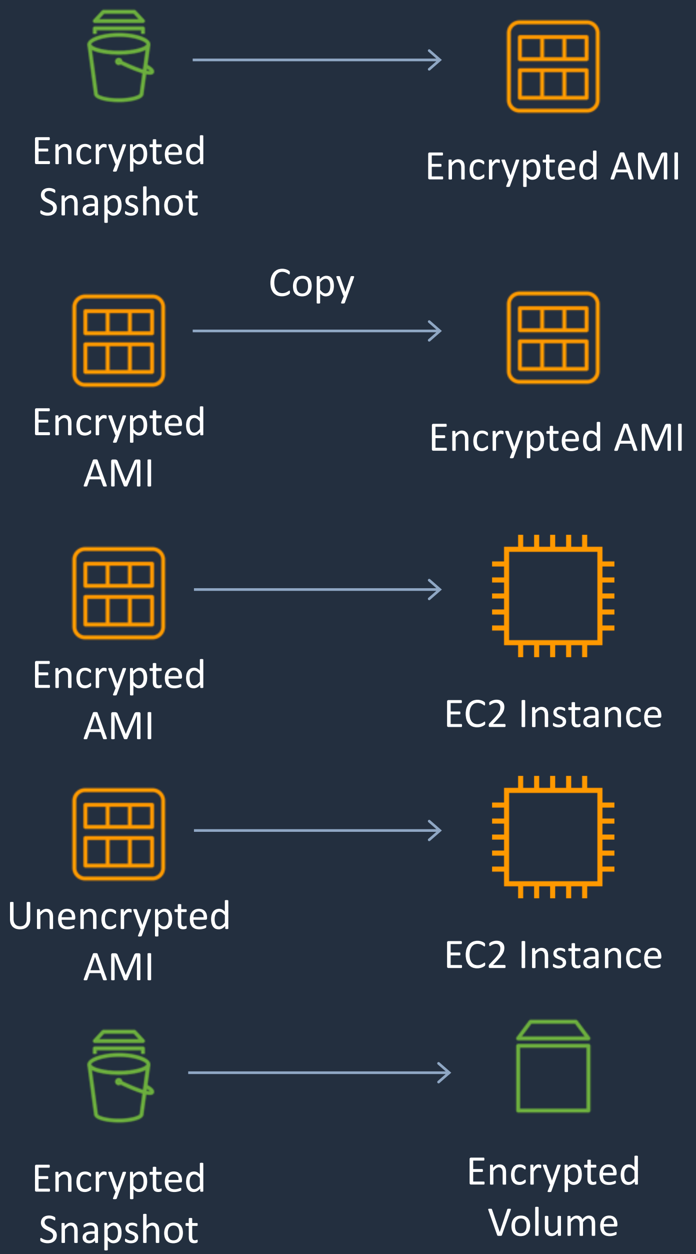
- Can be encrypted
- Can change regions

- Can be encrypted
- Can change AZ

- Cannot be encrypted
- Can be shared with other accounts
- Can be shared publicly

- Can change encryption key
- Can change regions

Copying and Sharing AMIs and Snapshots



- Can be shared with other accounts (custom key only)
- Cannot be shared publicly

- Can change encryption key
- Can change region

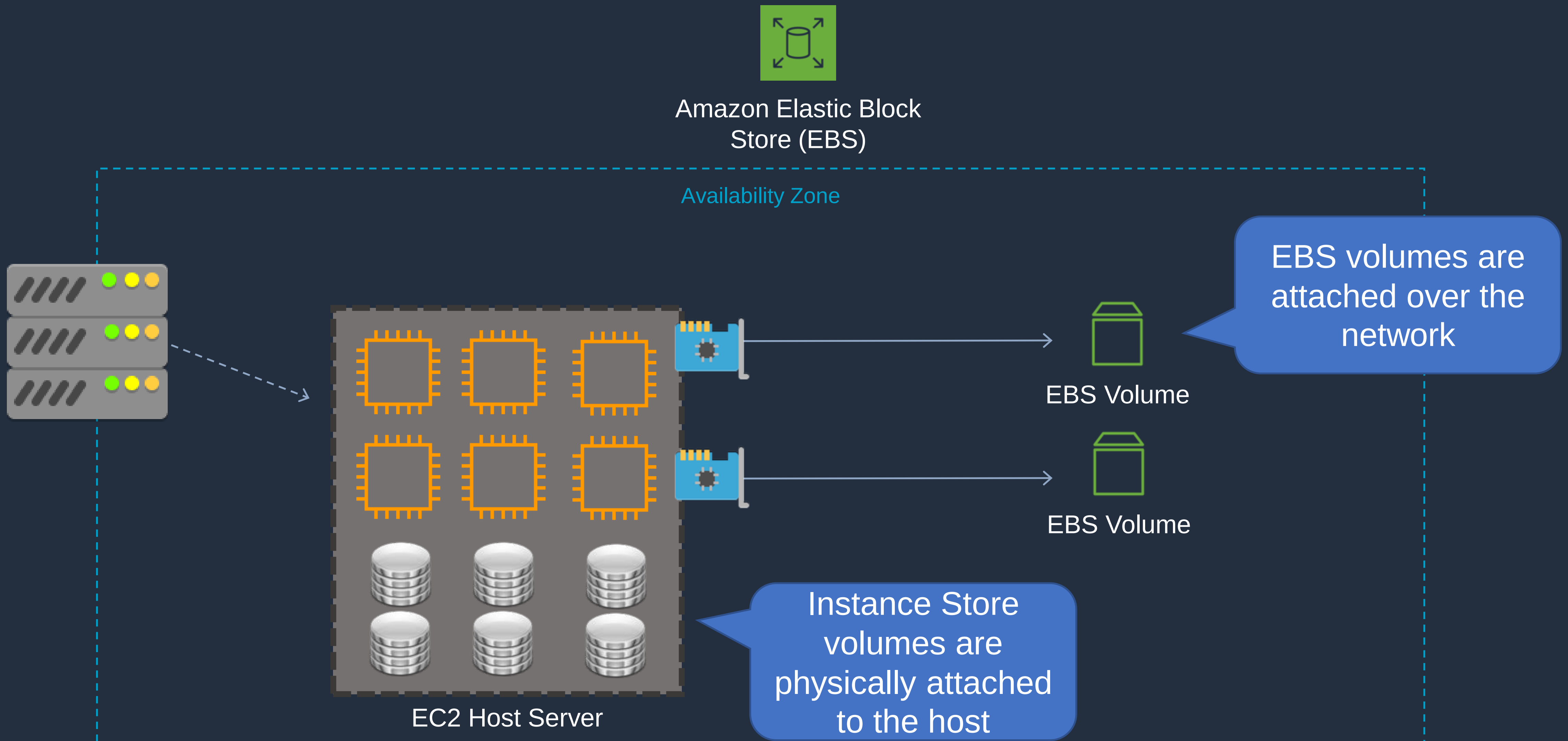
- Can change encryption key
- Can change AZ

- Can change encryption state
- Can change AZ

- Can be encrypted
- Can change AZ

EBS vs instance store

EBS vs instance store

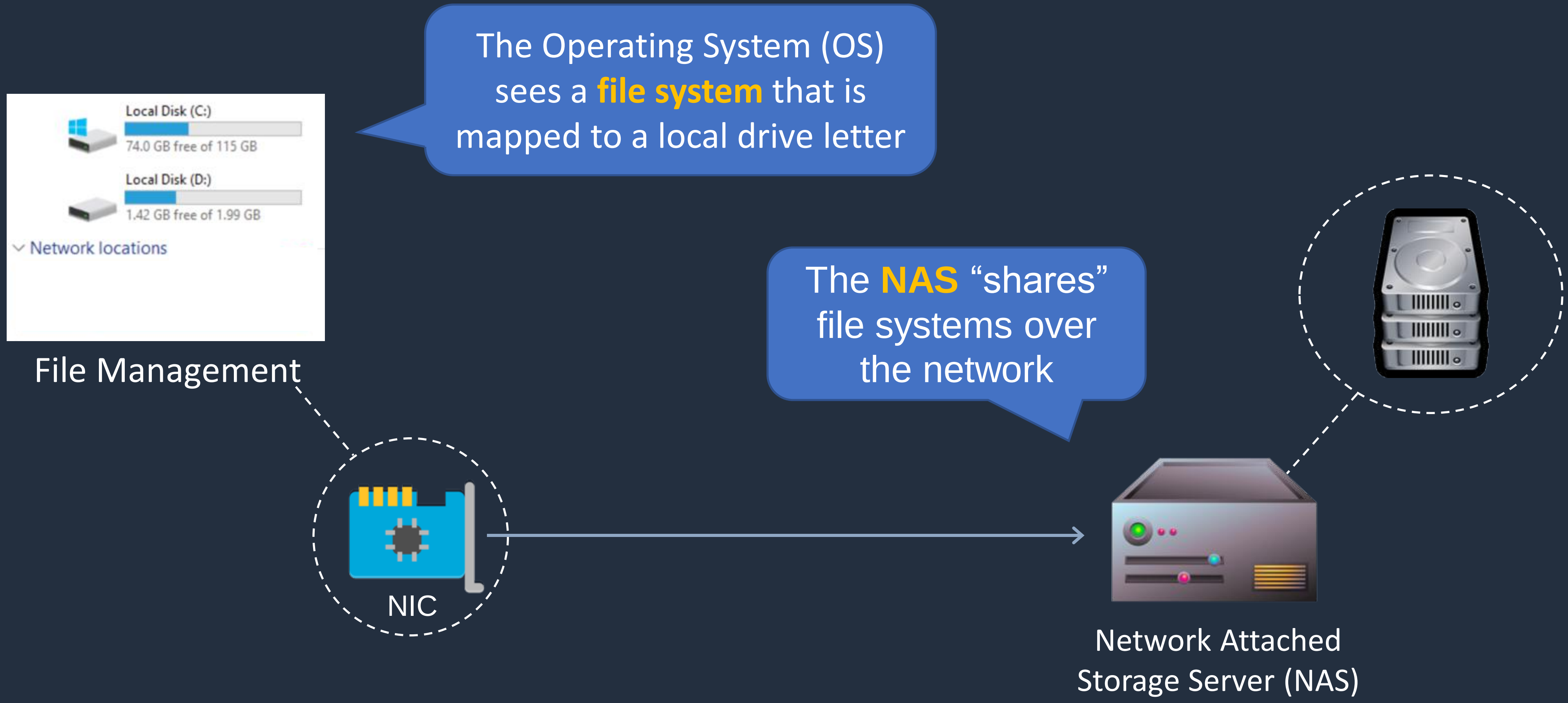


EBS vs instance store

- Instance store volumes are high performance local disks that are physically attached to the host computer on which an EC2 instance runs
- Instance stores are ephemeral which means the data is lost when powered off (non-persistent)
- Instance stores are ideal for temporary storage of information that changes frequently, such as buffers, caches, or scratch data
- Instance store volume root devices are created from AMI templates stored on S3
- Instance store volumes cannot be detached/reattached

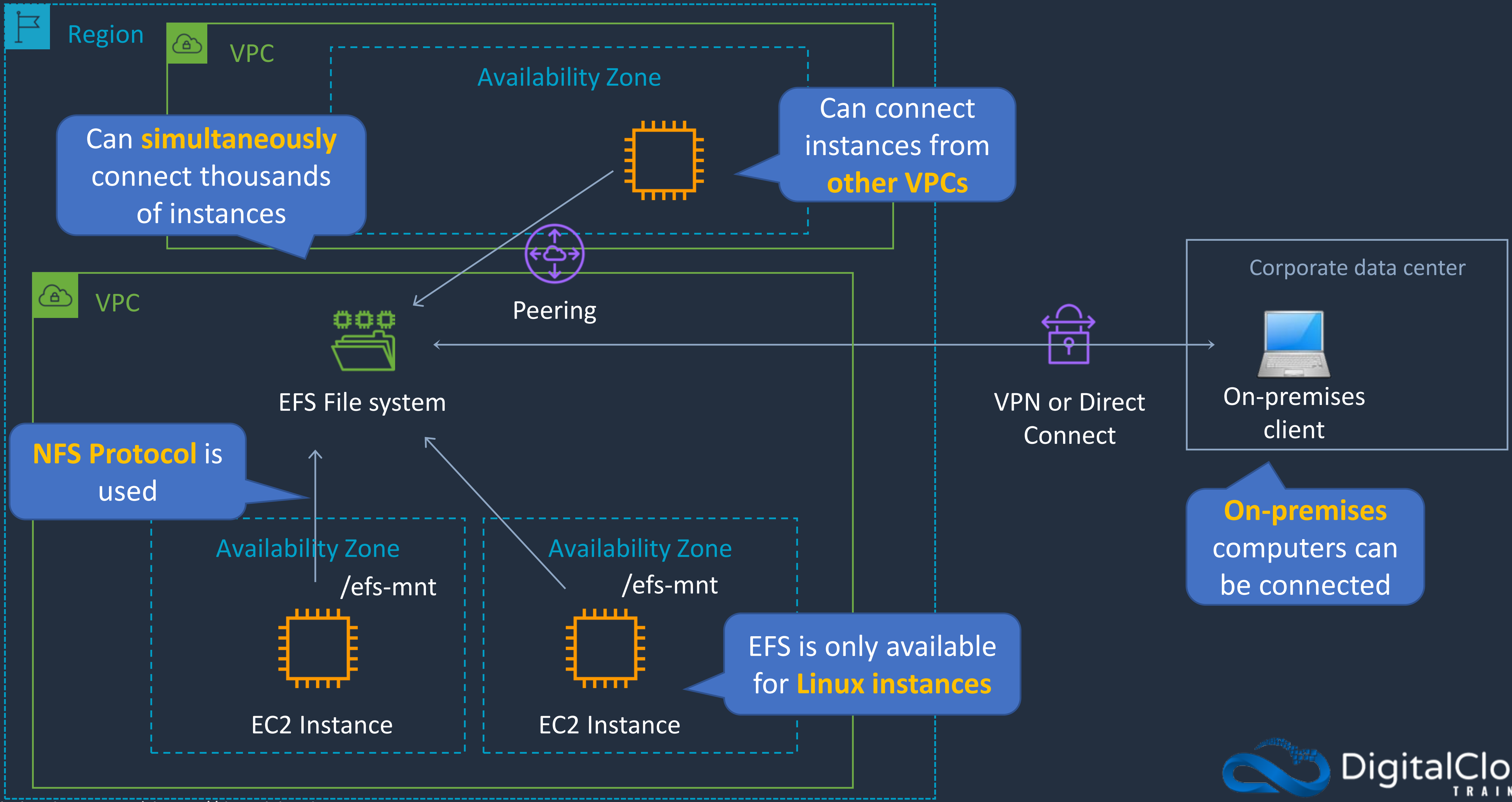
Amazon EFS Refresher

Network Attached Storage

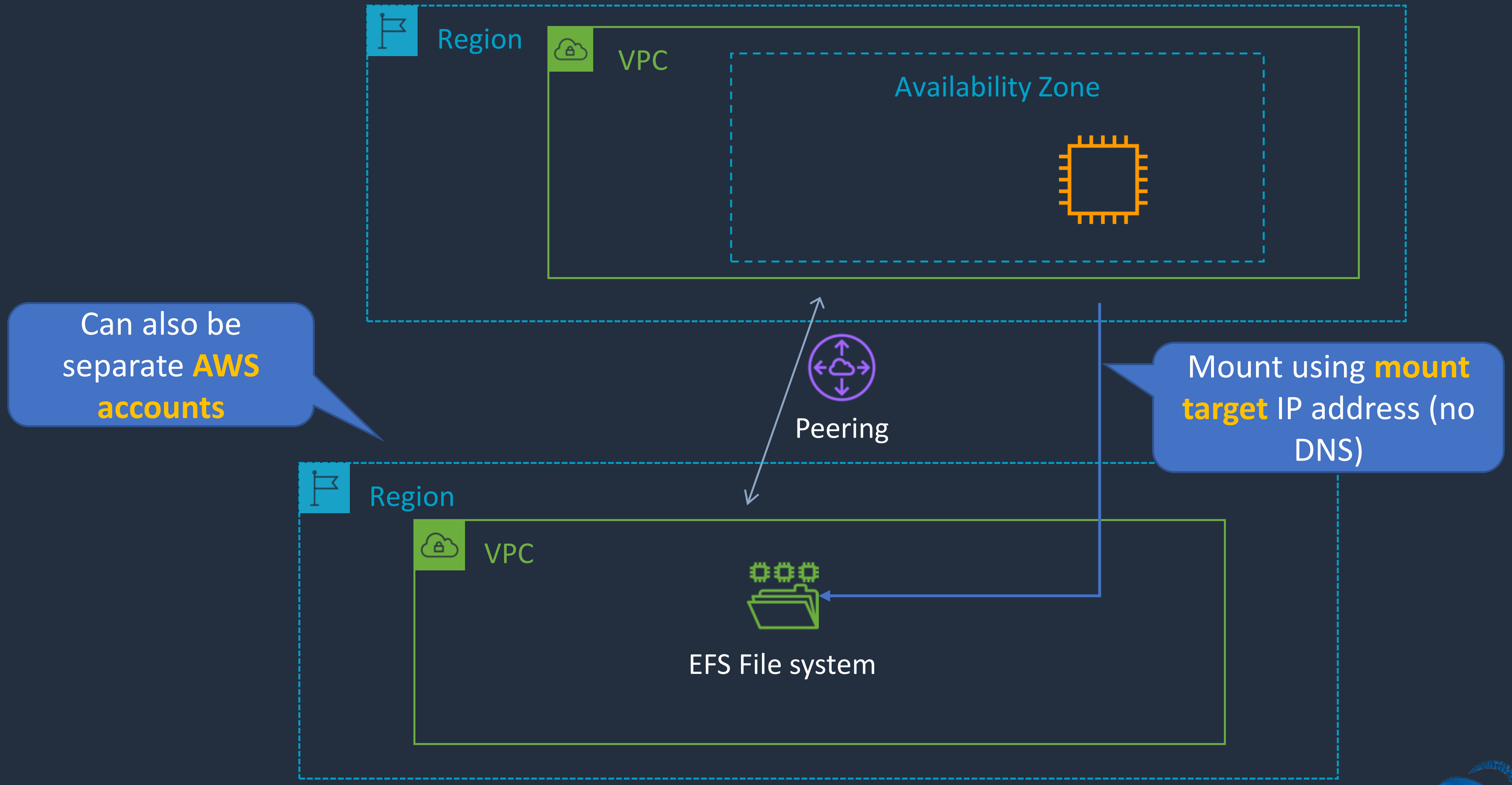


NAS devices are file-based storage systems

Amazon Elastic File System (EFS) Overview

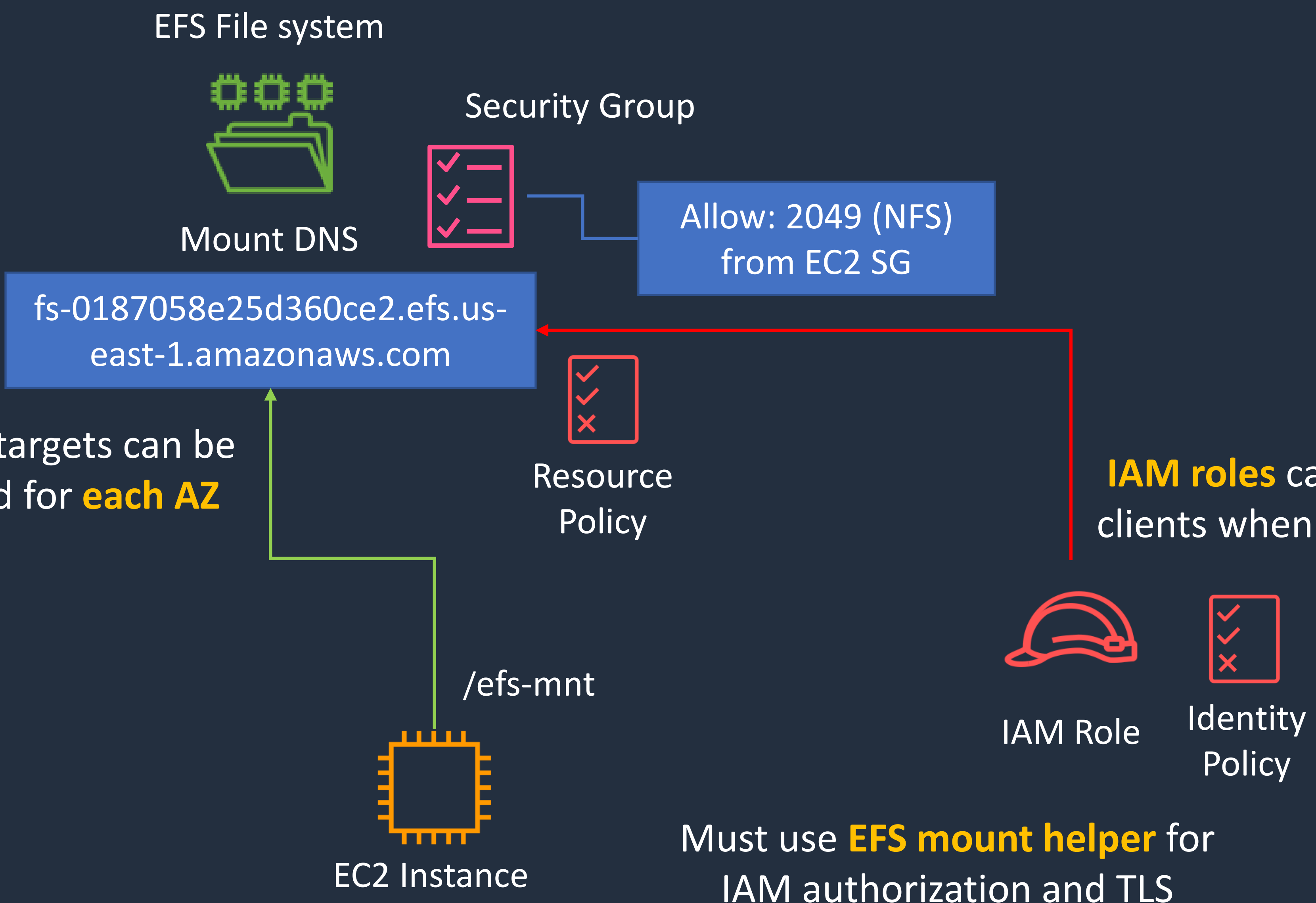


Amazon Elastic File System (EFS) Overview



Working with Files on EFS

Amazon EFS



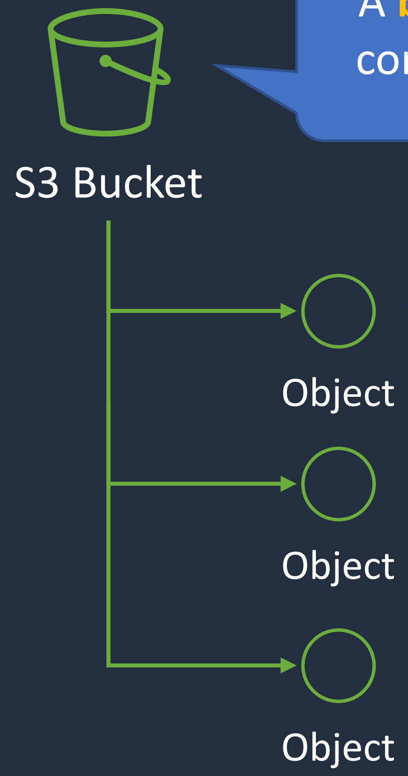
Mount targets can be created for **each AZ**

IAM roles can be used by NFS clients when connecting to EFS

Must use **EFS mount helper** for IAM authorization and TLS

Amazon S3 Overview

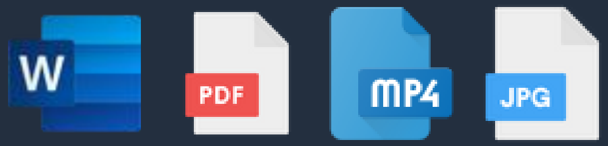
Amazon Simple Storage Service (S3)



A **bucket** is a container for objects

An **object** is a file you upload

You can store millions of **objects** in a **bucket**



Accessing objects in a bucket:

`https://bucket.s3.aws-region.amazonaws.com/key`

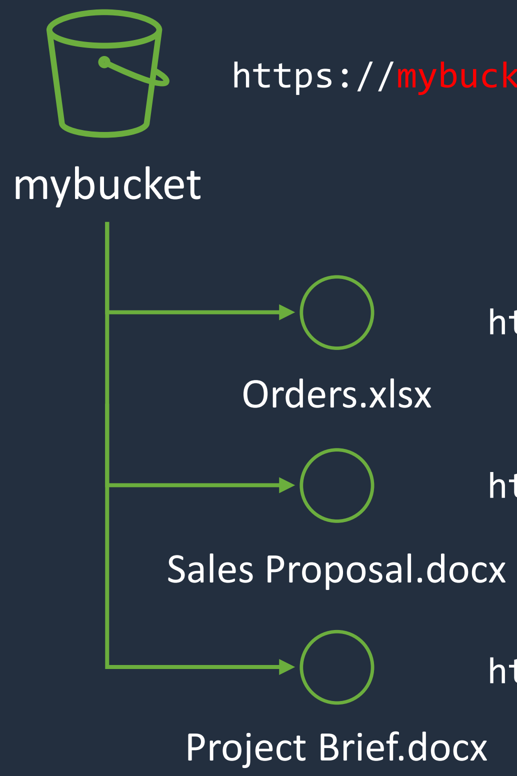
`https://s3.aws-region.amazonaws.com/bucket/key`

The **HTTP protocol** is used with a **REST API** (e.g. GET, PUT, POST, SELECT, DELETE)

Amazon Simple Storage Service (S3)

- You can store any type of file in S3
- Files can be anywhere from 0 bytes to 5 TB
- There is unlimited storage available
- S3 is a universal namespace so **bucket names** must be **unique globally**
- However, you create your buckets within a **REGION**
- It is a best practice to create buckets in regions that are physically closest to your users to reduce latency
- There is no hierarchy for objects within a bucket
- Delivers strong read-after-write consistency

Buckets, Folders, and Objects



https://mybucket.s3-website-us-west-1.amazonaws.com

https://mybucket.s3.us-west-1.amazonaws.com/Orders.xlsx

https://mybucket.s3.us-west-1.amazonaws.com/Project+Alpha/Sales+Proposal.docx

https://mybucket.s3.us-west-1.amazonaws.com/Project+Alpha/Project+Brief.docx

The object name is the **Key** the data is the **Value**

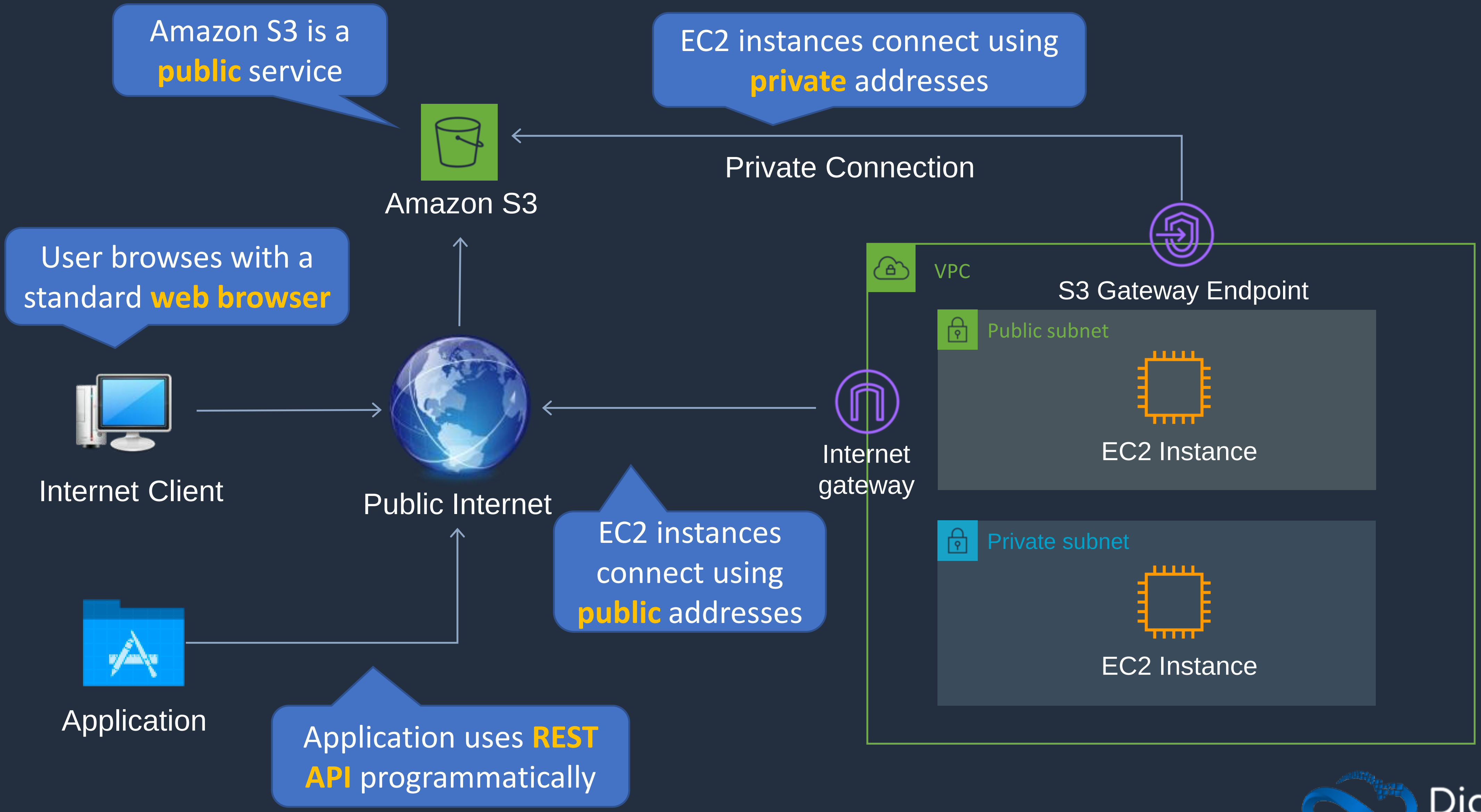
A **Folder** is a shared **prefix** for grouping objects

Buckets, Folders, and Objects

- Folders **can** be created within folders
- Buckets **cannot** be created within other buckets
- An objects consists of:
  - Key (the name of the object)
  - Version ID
  - Value (actual data)
  - Metadata
  - Subresources
  - Access control information

DigitalCloud TRAINING

Accessing Amazon S3



Amazon S3 Storage Classes

Durability and Availability in S3

Durability

Durability is protection against:

- Data loss
- Data corruption
- S3 offers 11 9s durability (99.999999999)

If you store 10 million objects, then you expect to lose one object every 10,000 years!

Availability

Availability is a measurement of:

- The amount of time the data is available to you
- Expressed as a percent of time per year
- E.g. 99.99%

S3 Storage Classes

	S3 Standard	S3 Intelligent Tiering	S3 Standard-IA	S3 One Zone-IA	S3 Glacier Instant Retrieval	S3 Glacier Flexible Retrieval	S3 Glacier Deep Archive
Designed for durability	99.999999999%	99.999999999%	99.999999999%	99.999999999%	99.999999999%	99.999999999%	99.999999999%
Designed for availability	99.99%	99.9%	99.9%	99.5%	99.9%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99%	99.9%	99.9%
Availability Zones	≥3	≥3	≥3	1	≥3	≥3	≥3
Minimum capacity charge per object	N/A	N/A	128KB	128KB	128KB	40KB	40KB
Minimum storage duration charge	N/A	N/A	30 days	30 days	90 days	90 days	180 days
Retrieval fee	N/A	N/A	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved
First byte latency	milliseconds	milliseconds	milliseconds	milliseconds	milliseconds	minutes or hours	hours
Storage type	Object	Object	Object	Object	Object	Object	Object
Lifecycle transitions	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Amazon S3 Lifecycle Policies

S3 Lifecycle Management

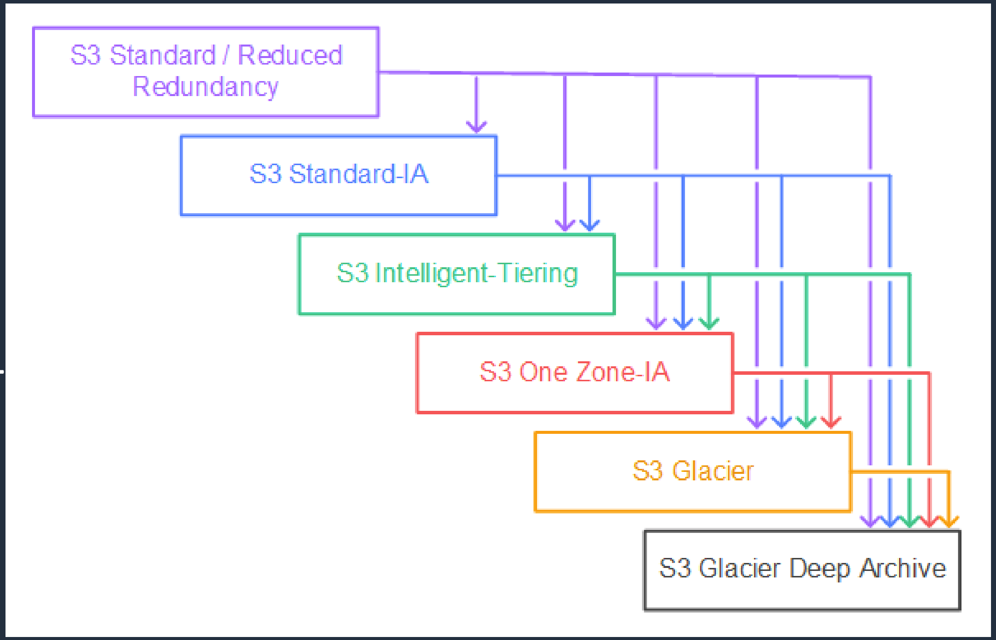
There are two types of actions:

- **Transition actions** - Define when objects transition to another storage class
- **Expiration actions** - Define when objects expire (deleted by S3)

S3 LM: Supported Transitions

You can transition from the following:

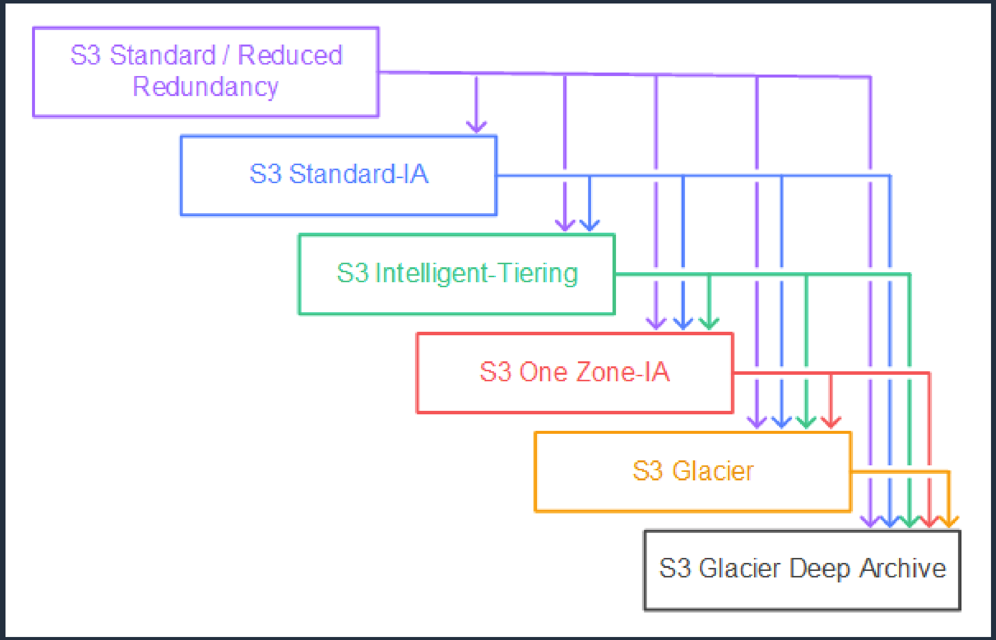
- The S3 Standard storage class to any other storage class
- Any storage class to the S3 Glacier or S3 Glacier Deep Archive storage classes
- The S3 Standard-IA storage class to the S3 Intelligent-Tiering or S3 One Zone-IA storage classes
- The S3 Intelligent-Tiering storage class to the S3 One Zone-IA storage class
- The S3 Glacier storage class to the S3 Glacier Deep Archive storage class



S3 LM: Unsupported Transitions

You can't transition from the following:

- Any storage class to the S3 Standard storage class
- Any storage class to the Reduced Redundancy storage class
- The S3 Intelligent-Tiering storage class to the S3 Standard-IA storage class
- The S3 One Zone-IA storage class to the S3 Standard-IA or S3 Intelligent-Tiering storage classes



S3 Lifecycle Management

- Can create a lifecycle policy through the console or CLI/API
- When configured using the CLI/API an XML or JSON file must be supplied
- API actions to create/update/delete lifecycle policies:
  - PutBucketLifecycleConfiguration - Creates a new lifecycle configuration for the bucket or replaces an existing lifecycle configuration
  - GetBucketLifecycleConfiguration - Returns the lifecycle configuration information set on the bucket
  - DeleteBucketLifecycle - Deletes the lifecycle configuration from the specified bucket

Example S3 Lifecycle Policy (XML)

```xml
<LifecycleConfiguration>
    <Rule>
        <ID>ExampleRule</ID>
        <Filter>
            <Prefix>documents/</Prefix>
        </Filter>
        <Status>Enabled</Status>
        <Transition>
            <Days>365</Days>
            <StorageClass>GLACIER</StorageClass>
        </Transition>
        <Expiration>
            <Days>3650</Days>
        </Expiration>
    </Rule>
</LifecycleConfiguration>
```

Configure Replication and Lifecycle

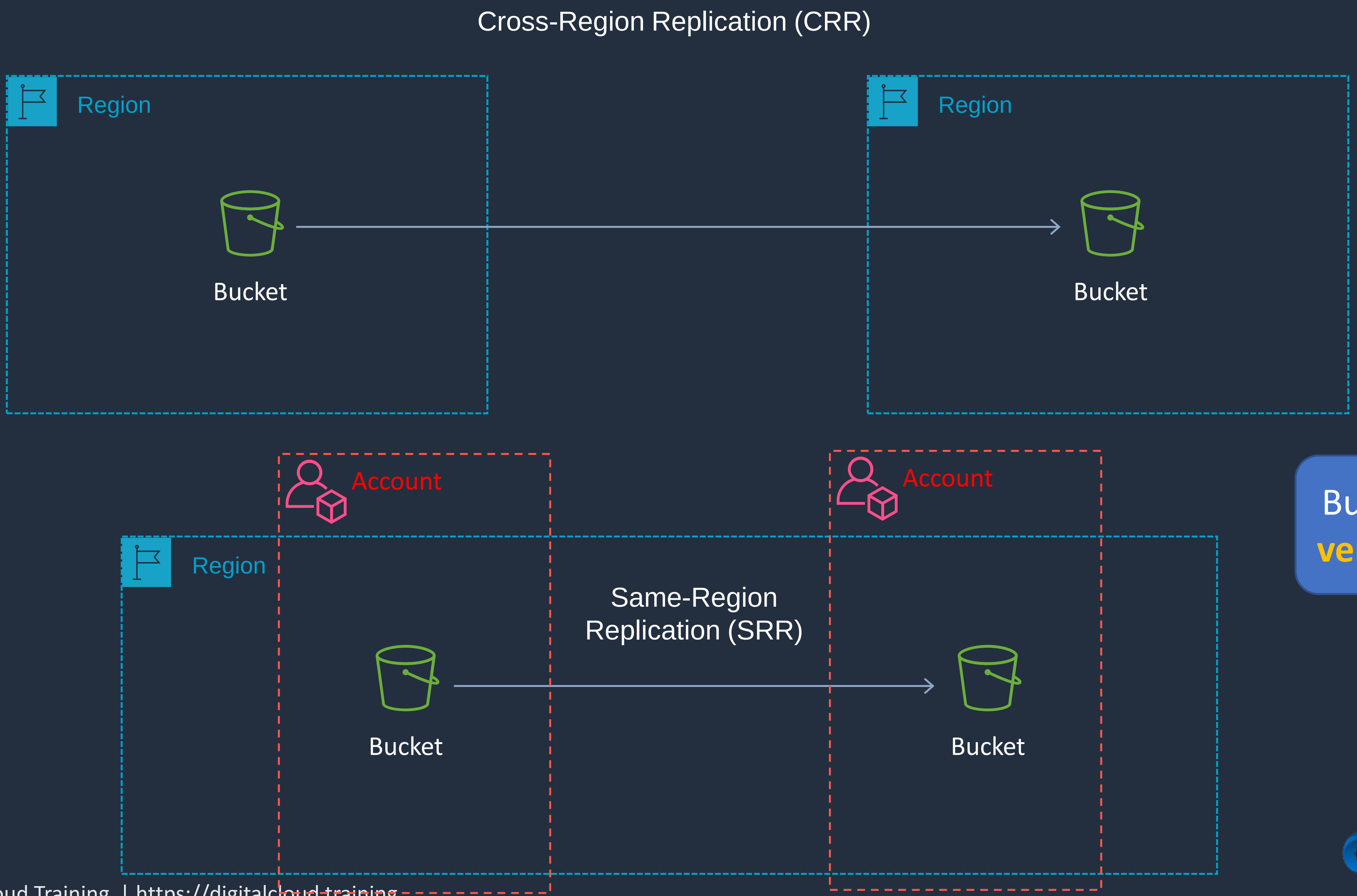
S3 Versioning and Replication

DigitalCloud TRAINING

S3 Versioning

- Versioning is a means of keeping multiple variants of an object in the same bucket
- Use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket
- Versioning-enabled buckets enable you to recover objects from accidental deletion or overwrite

S3 Replication



Buckets must have **versioning** enabled

Amazon S3 Encryption

Amazon S3 Encryption

Server-side encryption with S3 managed keys (SSE-S3)

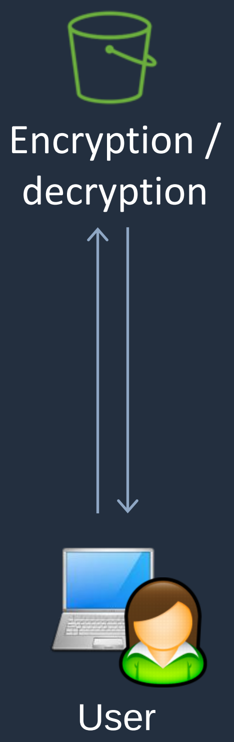
- S3 managed keys
- Unique object keys
- Master key
- AES 256

Server-side encryption with AWS KMS managed keys (SSE-KMS)

- KMS managed keys
- Can be AWS managed keys
- Or customer managed KMS keys

Amazon S3 Encryption

Server-side encryption with client provided keys (SSE-C)



- Client managed keys
- Not stored on AWS

Client-side encryption

- Client managed keys
- Not stored on AWS
- Or you can use a KMS Key

Amazon S3 Default Encryption

- All Amazon S3 buckets have encryption configured by default
- All new object uploads to Amazon S3 are automatically encrypted
- There is no additional cost and no impact on performance
- Objects are automatically encrypted by using server-side encryption with Amazon S3 managed keys (SSE-S3)
- To encrypt existing unencrypted Amazon S3 objects, you can use Amazon S3 Batch Operations
- You can also encrypt existing objects by using the CopyObject API operation or the copy-object AWS CLI command

Enforce Encryption with Bucket Policy

```
{
    "Version": "2012-10-17",
    "Id": "PutObjPolicy",
    "Statement": [
        {
            "Sid": "DenyIncorrectEncryptionHeader",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:PutObject",
            "Resource": "arn:aws:s3:::<bucket_name>/*",
            "Condition": {
                "StringNotEquals": {
                    "s3:x-amz-server-side-encryption": "aws:kms"
                }
            }
        },
        {
            "Sid": "DenyUnEncryptedObjectUploads",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:PutObject",
            "Resource": "arn:aws:s3:::<bucket_name>/*",
            "Condition": {
                "Null": {
                    "s3:x-amz-server-side-encryption": true
                }
            }
        }
    ]
}
```

Enforces encryption using **SSE-KMS**

Example PUT request

```
PUT /example-object HTTP/1.1
Host: myBucket.s3.amazonaws.com
Date: Wed, 8 Jun 2016 17:50:00 GMT
Authorization: authorization string
Content-Type: text/plain
Content-Length: 11434
x-amz-meta-author: Janet
Expect: 100-continue
x-amz-server-side-encryption: aws:kms
[11434 bytes of object data]
```

DigitalCloud TRAINING

Enforce Encryption with AWS KMS

Enforce Encryption with Bucket Policy

```json
{
    "Version": "2012-10-17",
    "Id": "PutObjPolicy",
    "Statement": [
        {
            "Sid": "DenyIncorrectEncryptionHeader",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:PutObject",
            "Resource": "arn:aws:s3:::<bucket_name>/*",
            "Condition": {
                "StringNotEquals": {
                    "s3:x-amz-server-side-encryption": "aws:kms"
                }
            }
        },
        {
            "Sid": "DenyUnEncryptedObjectUploads",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:PutObject",
            "Resource": "arn:aws:s3:::<bucket_name>/*",
            "Condition": {
                "Null": {
                    "s3:x-amz-server-side-encryption": true
                }
            }
        }
    ]
}
```

Enforces encryption using **SSE-KMS**

Example PUT request

```
PUT /example-object HTTP/1.1
Host: myBucket.s3.amazonaws.com
Date: Wed, 8 Jun 2016 17:50:00 GMT
Authorization: authorization string
Content-Type: text/plain
Content-Length: 11434
x-amz-meta-author: Janet
Expect: 100-continue
x-amz-server-side-encryption: aws:kms
[11434 bytes of object data]
```

S3 Presigned URLs

S3 Presigned URLs

AWS S3 CLI command to generate a presigned URL

```
$ aws s3 presign s3://dct-data-bucket/cool_image.jpeg
```

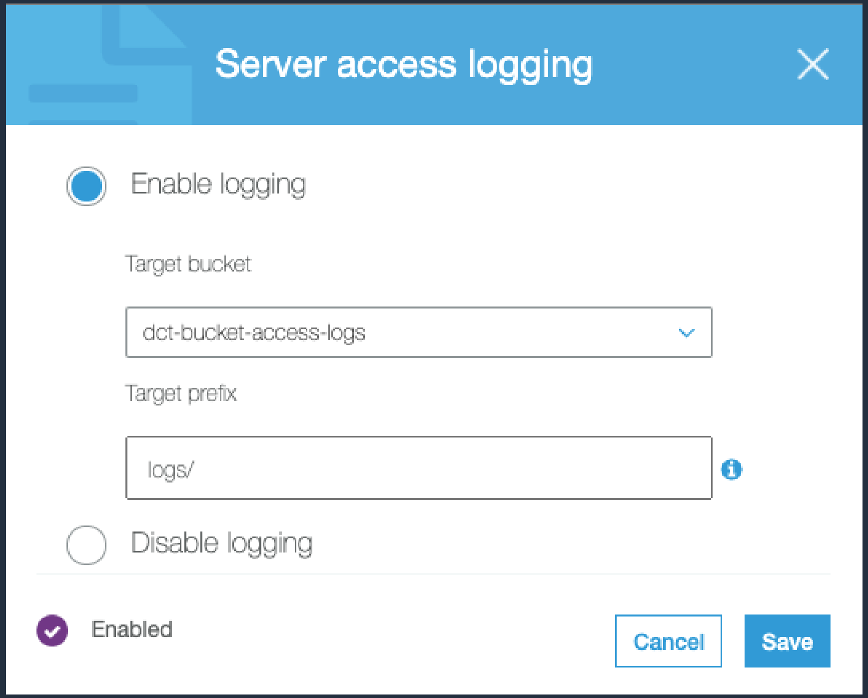
```
$ https://dct-data-bucket.s3.ap-southeast-2.amazonaws.com/cool_image.jpeg?X-Amz-
Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIA3KSVPHP6MAHNW5YH%2F20200909%2Fap-southeast-
2%2Fs3%2Faws4_request&X-Amz-Date=20200909T053538Z&X-Amz-Expires=3600&X-Amz-
SignedHeaders=host&X-Amz-
Signature=8b74653beee371da07a73dfdb4ff6883742383afa528aecd5c95c326c97764db
```

This is the response; the URL expires after 1 hour

Server Access Logging

Server Access Logging

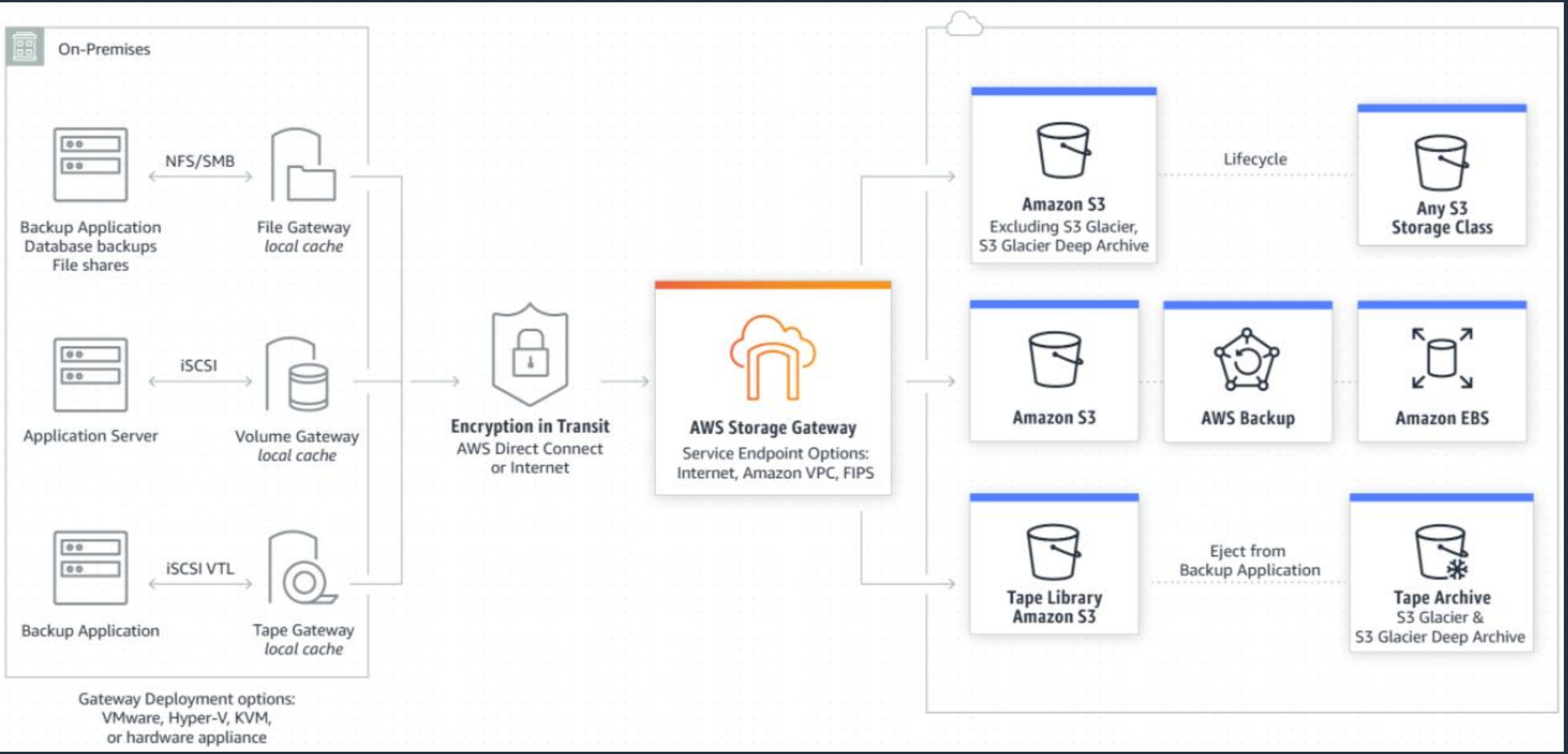
- Provides detailed records for the requests that are made to a bucket
- Details include the requester, bucket name, request time, request action, response status, and error code (if applicable)
- Disabled by default
- Only pay for the storage space used
- Must configure a separate bucket as the destination (can specify a prefix)
- Must grant write permissions to the Amazon S3 Log Delivery group on destination bucket



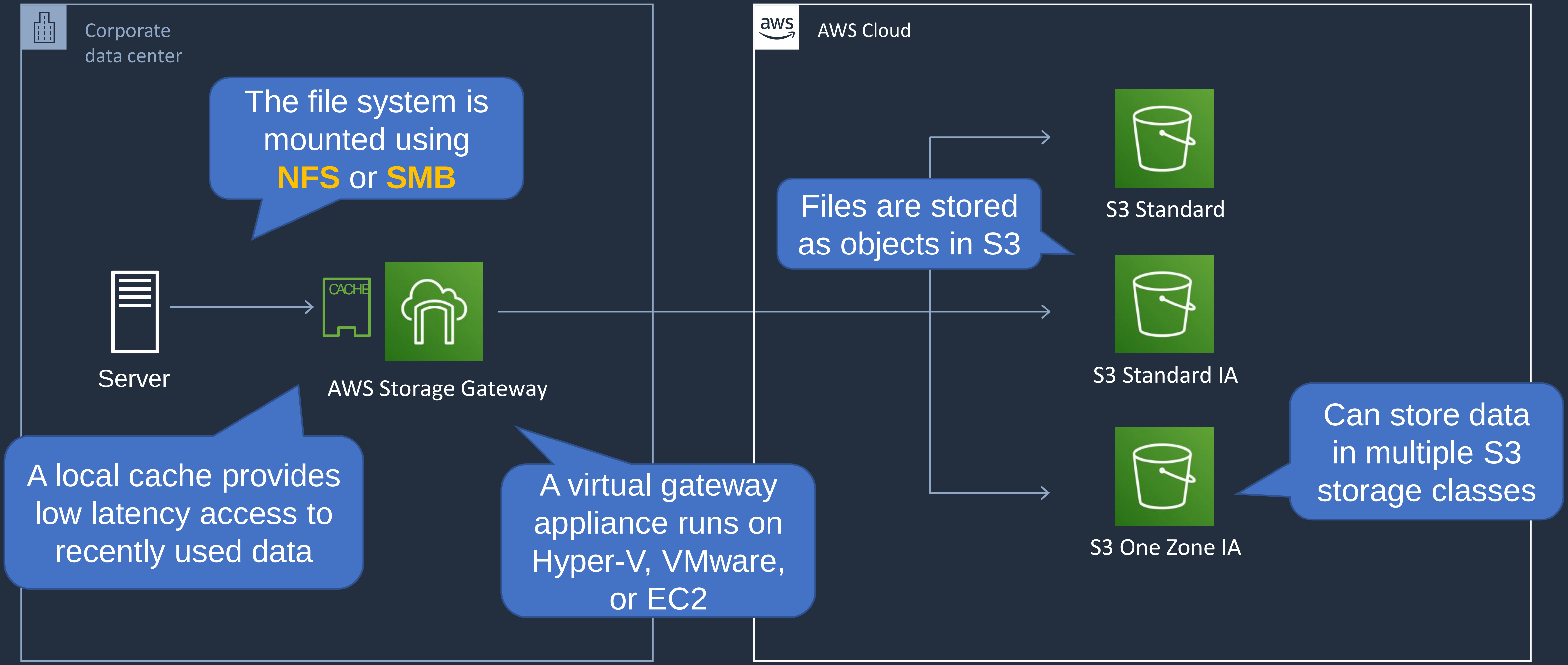
S3 Event Notifications

AWS Storage Gateway

AWS Storage Gateway



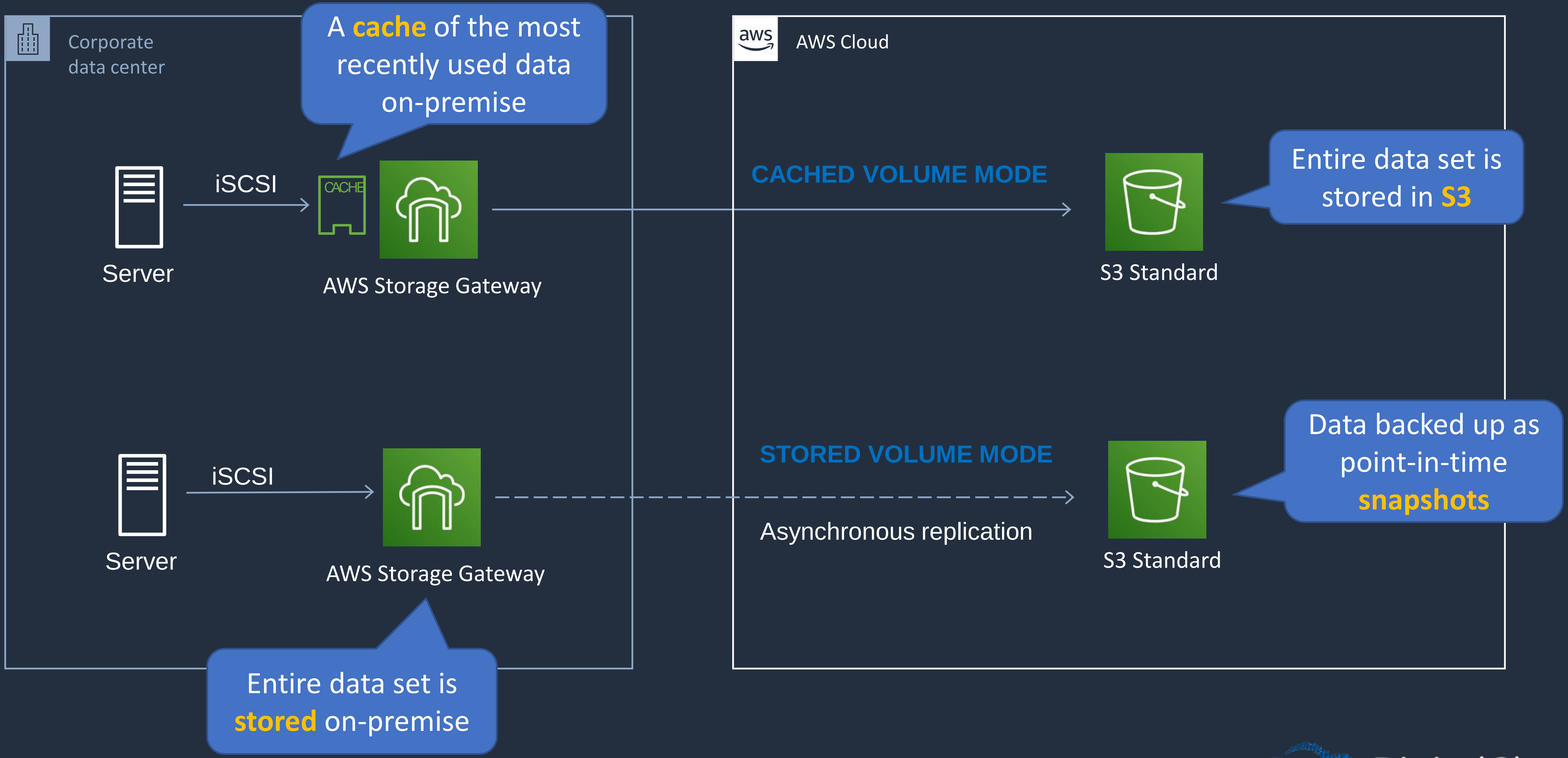
AWS Storage Gateway – File Gateway



AWS Storage Gateway – File Gateway

- File gateway provides a virtual **on-premises file server**
- Store and retrieve files as objects in Amazon S3
- Use with on-premises applications, and EC2-based applications that need file storage in S3 for object-based workloads
- File gateway offers **SMB** or **NFS**-based access to data in Amazon S3 with local caching

AWS Storage Gateway - Volume Gateway

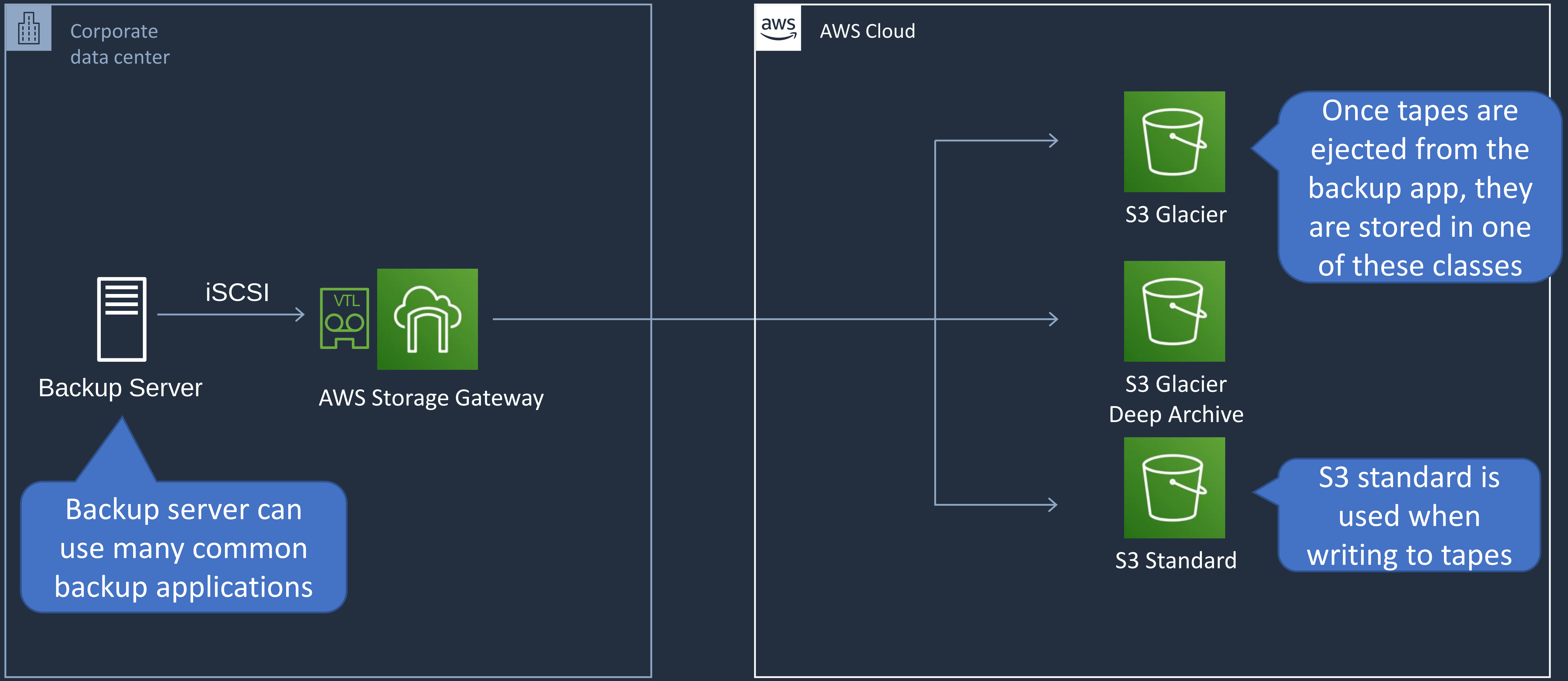


DigitalCloud TRAINING

AWS Storage Gateway - Volume Gateway

- The volume gateway supports block-based volumes
- Block storage – iSCSI protocol
- **Cached Volume mode** – the entire dataset is stored on S3 and a cache of the most frequently accessed data is cached on-site
- **Stored Volume mode** – the entire dataset is stored on-site and is asynchronously backed up to S3 (EBS point-in-time snapshots). Snapshots are incremental and compressed

AWS Storage Gateway - Tape Gateway



AWS Storage Gateway - Tape Gateway

- Used for backup with popular backup software
- Each gateway is preconfigured with a media changer and tape drives. Supported by NetBackup, Backup Exec, Veeam etc.
- When creating virtual tapes, you select one of the following sizes: 100 GB, 200 GB, 400 GB, 800 GB, 1.5 TB, and 2.5 TB
- A tape gateway can have up to 1,500 virtual tapes with a maximum aggregate capacity of 1 PB
- All data transferred between the gateway and AWS storage is encrypted using SSL
- All data stored by tape gateway in S3 is encrypted server-side with Amazon S3-Managed Encryption Keys (SSE-S3)

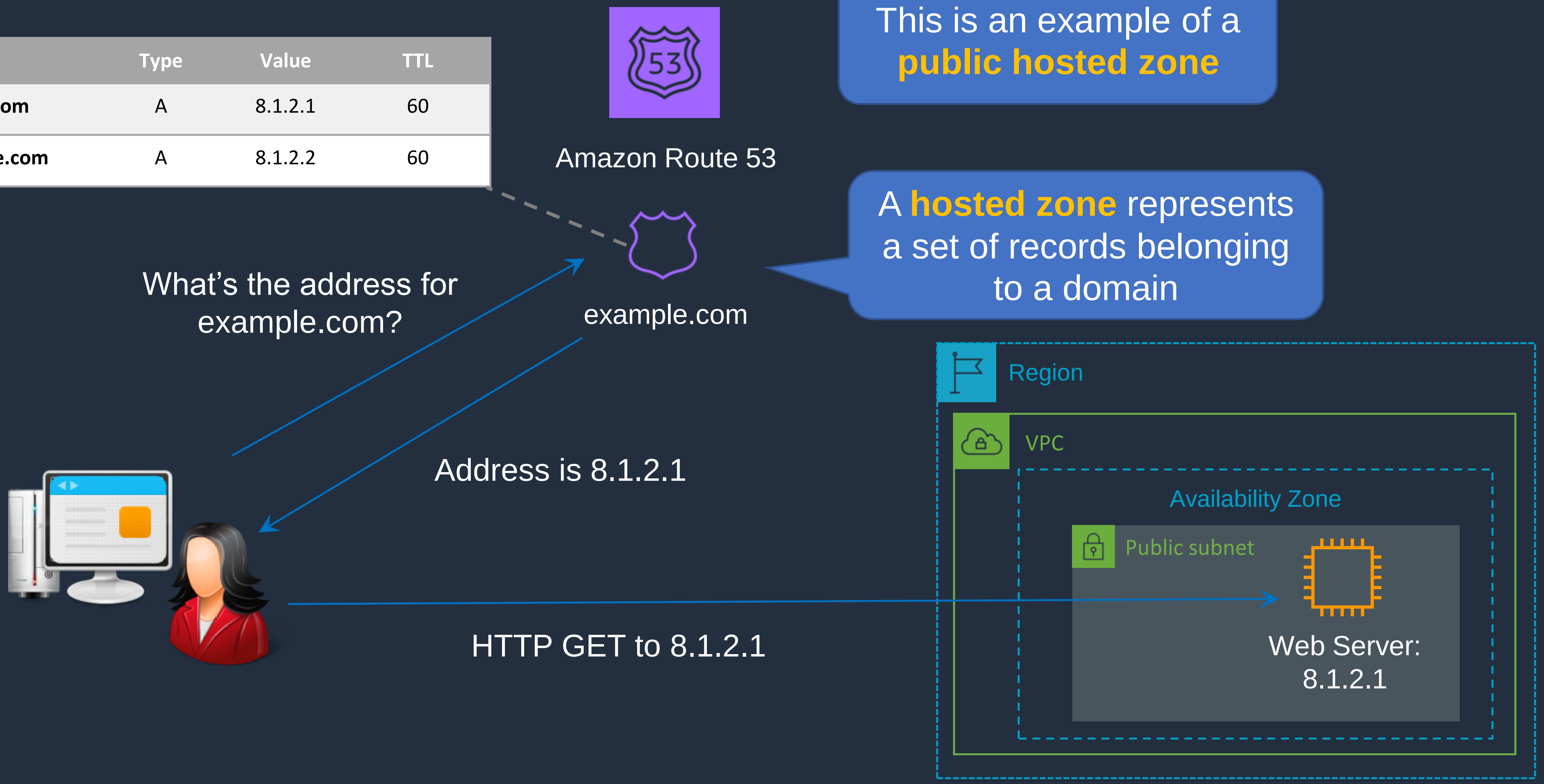
SECTION 9

DNS, Caching, and Performance Optimization

Amazon Route 53 Hosted Zones

Amazon Route 53 Hosted Zones

Name	Type	Value	TTL
example.com	A	8.1.2.1	60
dev.example.com	A	8.1.2.2	60

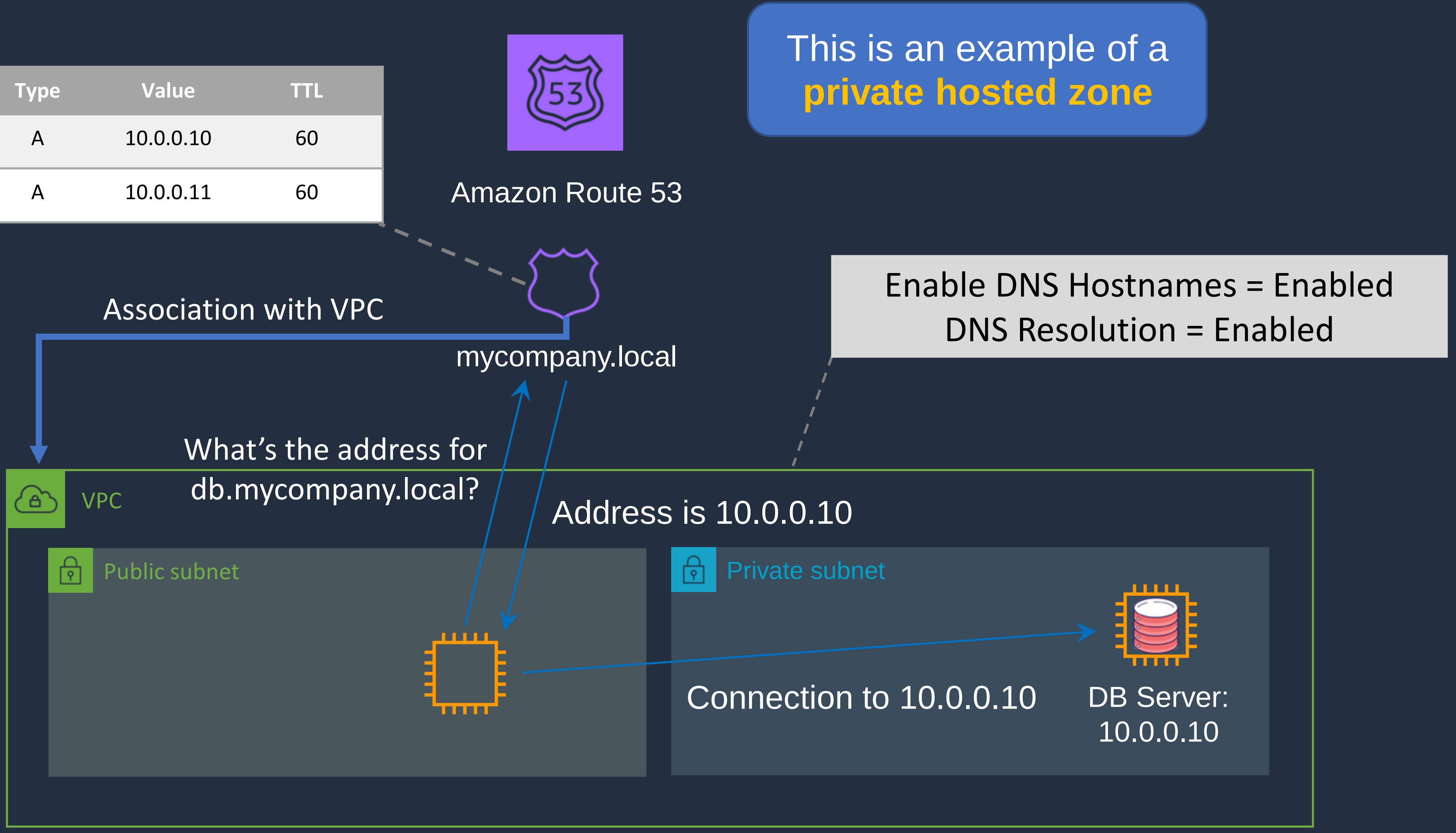


This is an example of a **public hosted zone**

A **hosted zone** represents a set of records belonging to a domain

Amazon Route 53 Hosted Zones

Name	Type	Value	TTL
db.mycompany.local	A	10.0.0.10	60
app.mycompany.local	A	10.0.0.11	60



Migration to/from Route 53

- You can migrate from **another DNS provider** and can import records
- You can migrate a hosted zone to **another AWS account**
- You can migrate from Route 53 to **another registrar**
- You can also associate a Route 53 hosted zone with a **VPC in another account**
  - Authorize association with VPC in the second account.
  - Create an association in the second account

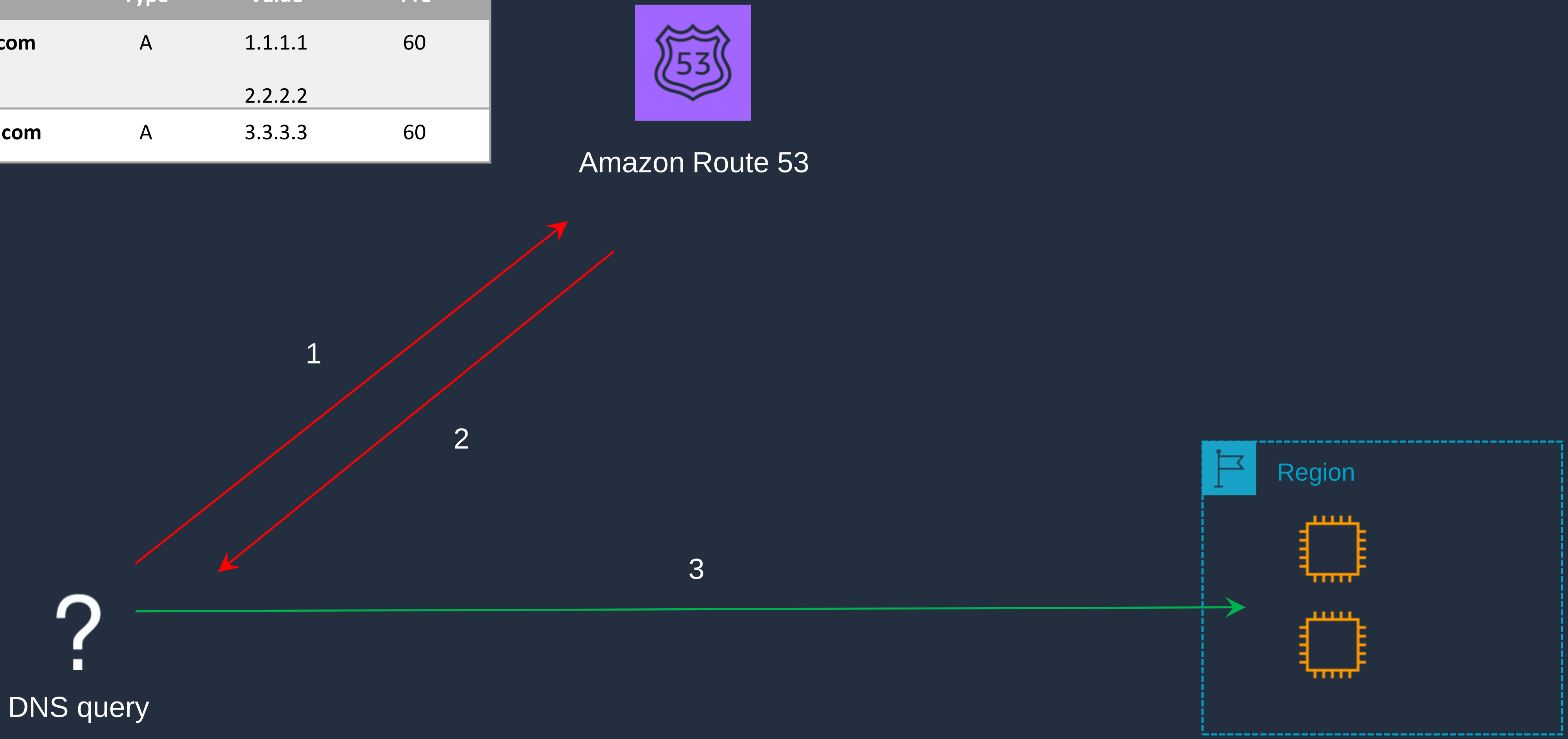
Route 53 Routing Policies

Amazon Route 53 Routing Policies

Routing Policy	What it does
Simple	Simple DNS response providing the IP address associated with a name
Failover	If primary is down (based on health checks), routes to secondary destination
Geolocation	Uses geographic location you're in (e.g. Europe) to route you to the closest region
Geoproximity	Routes you to the closest region within a geographic area
Latency	Directs you based on the lowest latency route to resources
Multivalue answer	Returns several IP addresses and functions as a basic load balancer
Weighted	Uses the relative weights assigned to resources to determine which to route to

Amazon Route 53 – Simple Routing Policy

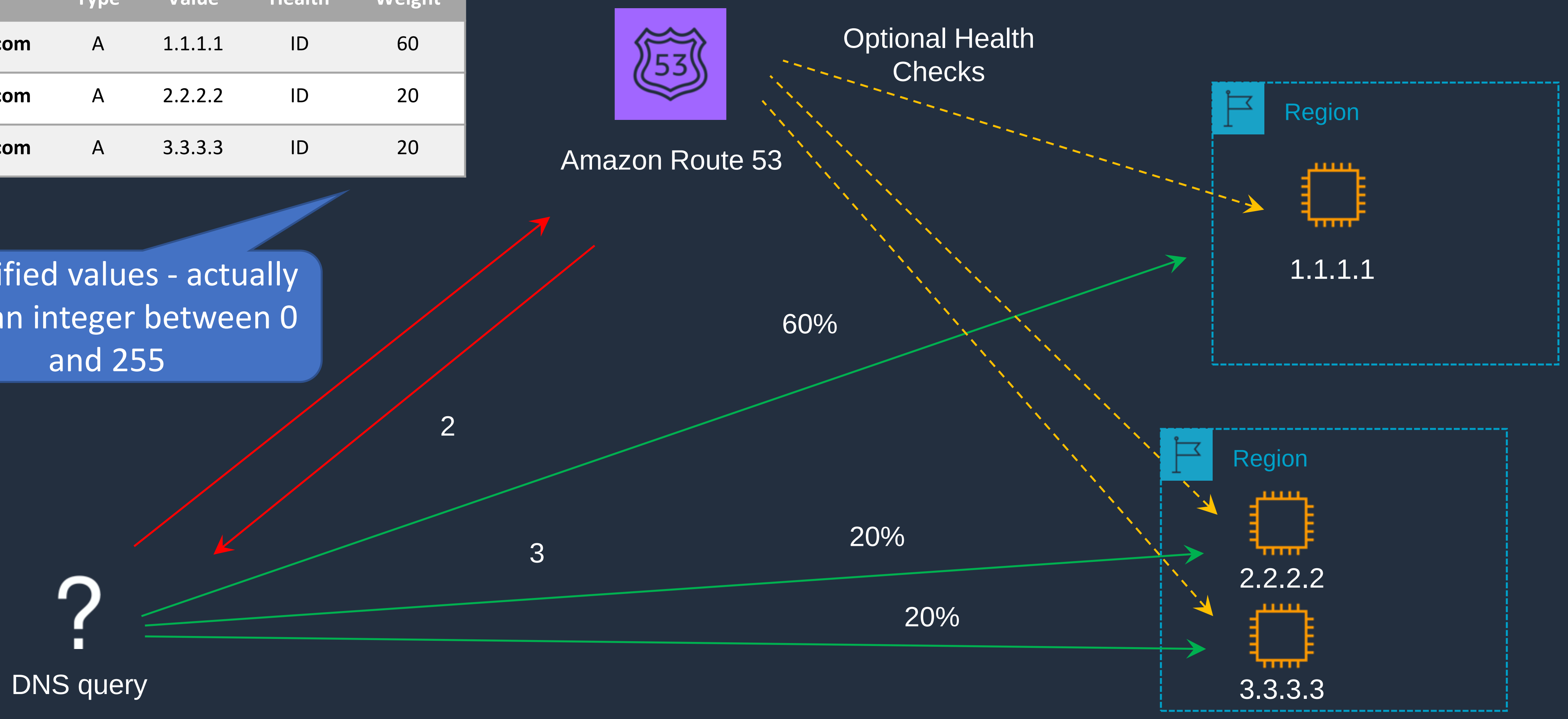
Name	Type	Value	TTL
simple.dctlabs.com	A	1.1.1.1 2.2.2.2	60
simple2.dctlabs.com	A	3.3.3.3	60



Amazon Route 53 – Weighted Routing Policy

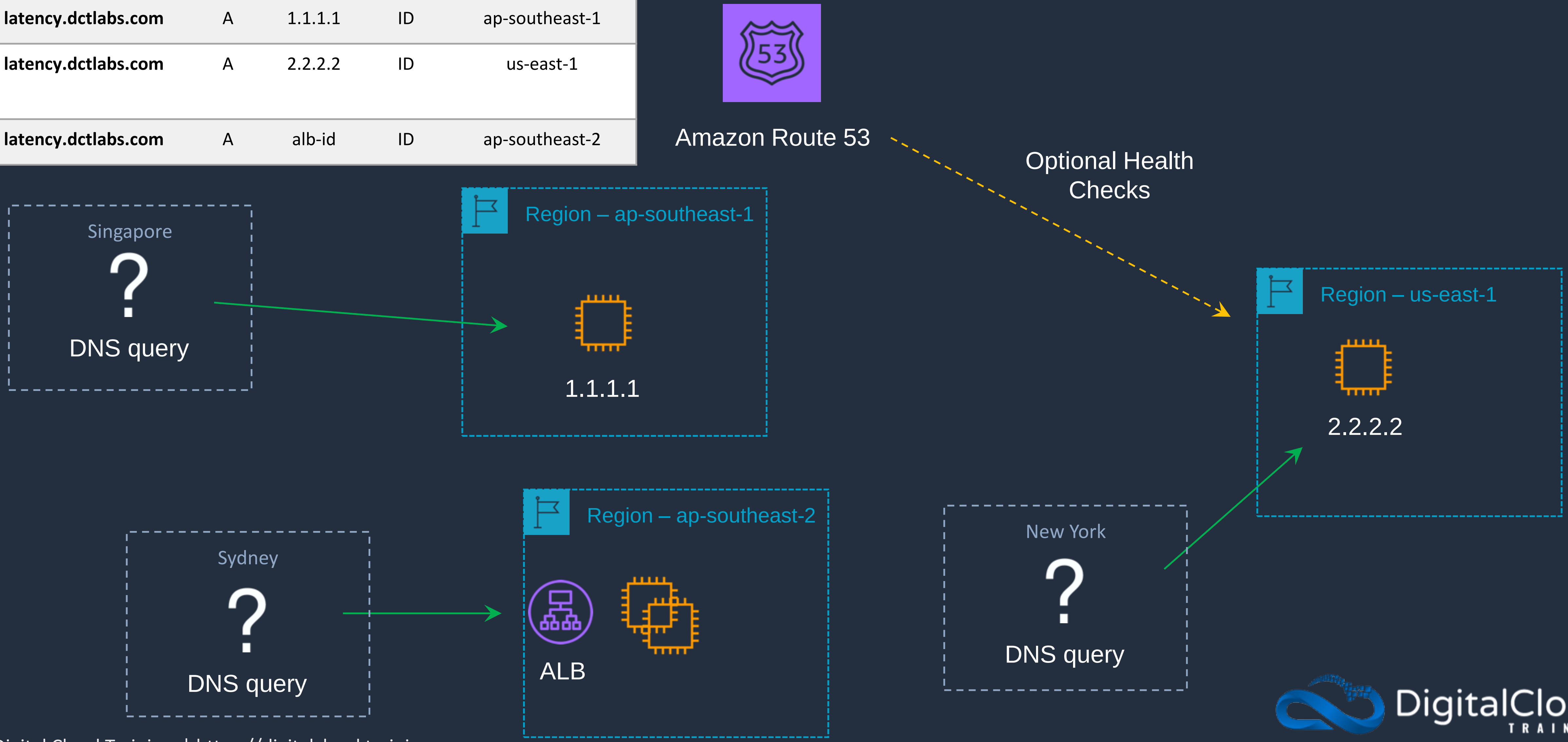
Name	Type	Value	Health	Weight
weighted.dctlabs.com	A	1.1.1.1	ID	60
weighted.dctlabs.com	A	2.2.2.2	ID	20
weighted.dctlabs.com	A	3.3.3.3	ID	20

Simplified values - actually uses an integer between 0 and 255



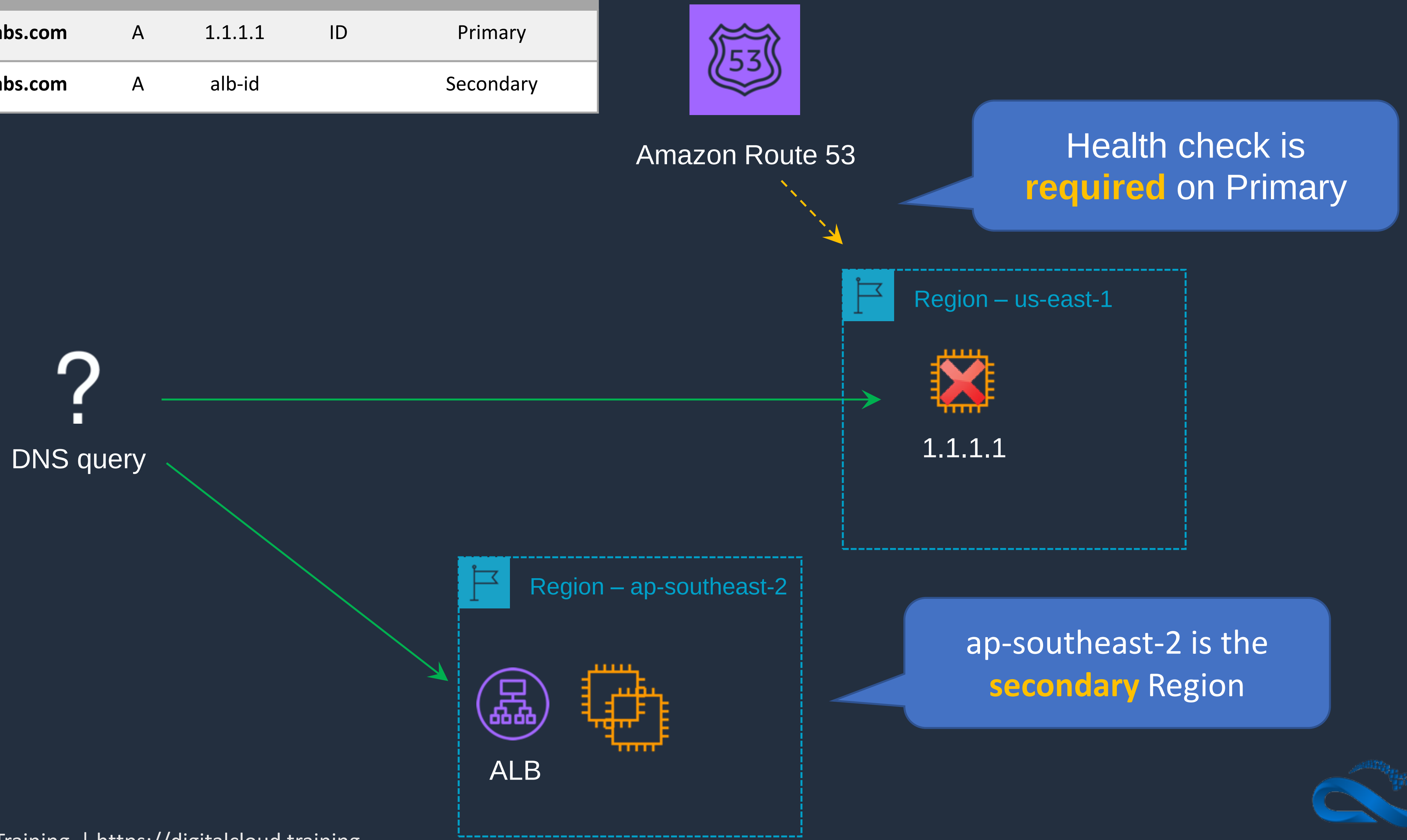
Amazon Route 53 – Latency Routing Policy

Name	Type	Value	Health	Region
latency.dctlabs.com	A	1.1.1.1	ID	ap-southeast-1
latency.dctlabs.com	A	2.2.2.2	ID	us-east-1
latency.dctlabs.com	A	alb-id	ID	ap-southeast-2



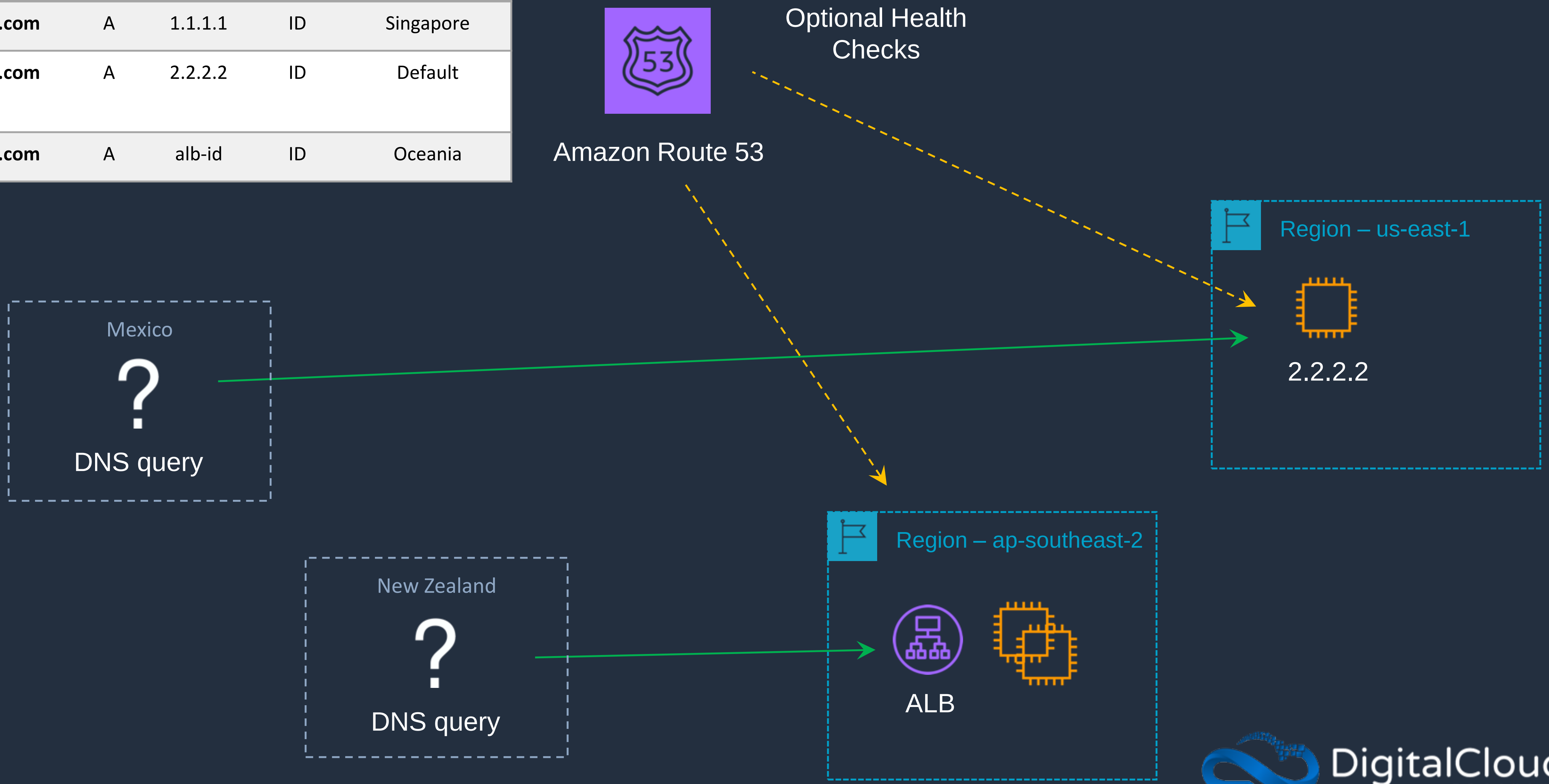
Amazon Route 53 – Failover Routing Policy

Name	Type	Value	Health	Record Type
failover.dctlabs.com	A	1.1.1.1	ID	Primary
failover.dctlabs.com	A	alb-id		Secondary



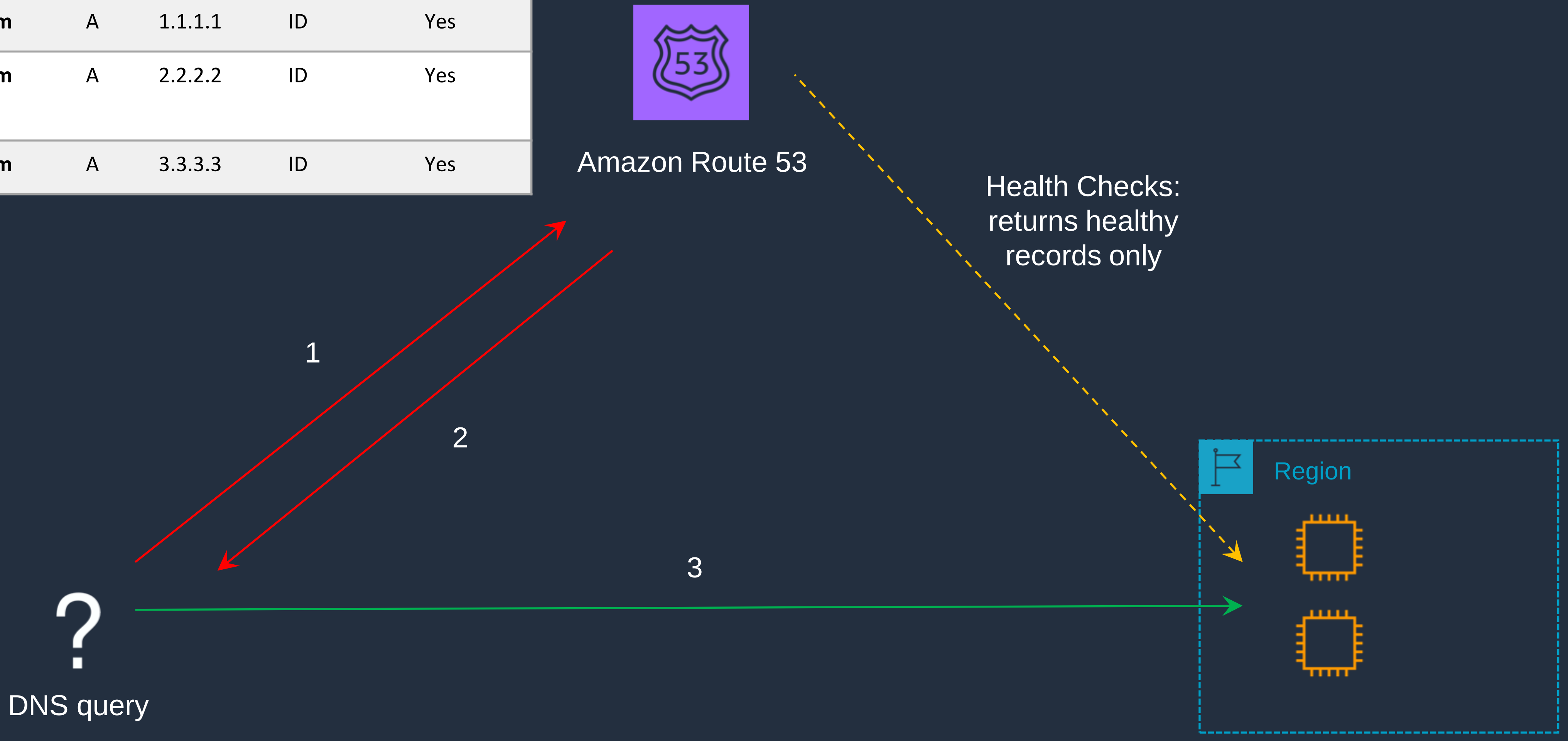
Amazon Route 53 – Geolocation Routing Policy

Name	Type	Value	Health	Geolocation
geolocation.dctlabs.com	A	1.1.1.1	ID	Singapore
geolocation.dctlabs.com	A	2.2.2.2	ID	Default
geolocation.dctlabs.com	A	alb-id	ID	Oceania



Amazon Route 53 – Multivalue Routing Policy

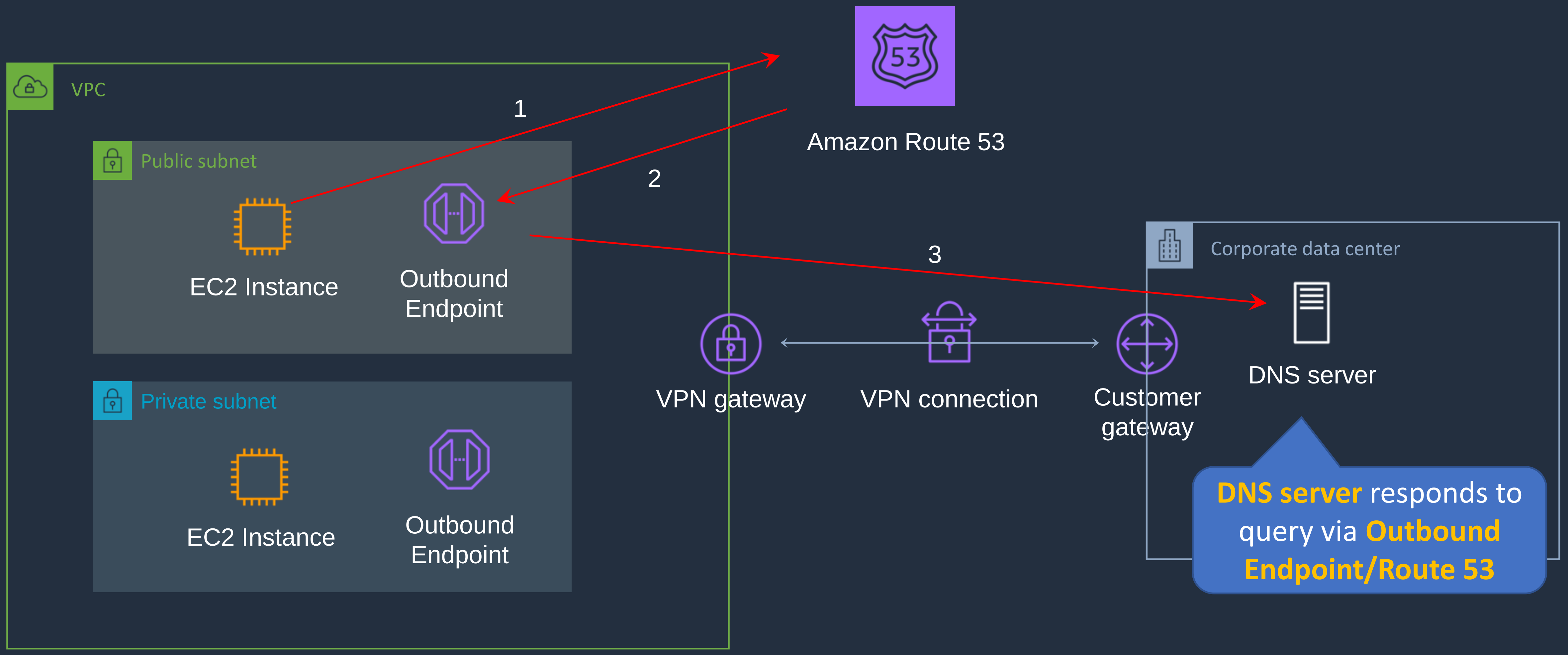
Name	Type	Value	Health	Multi Value
multivalue.dctlabs.com	A	1.1.1.1	ID	Yes
multivalue.dctlabs.com	A	2.2.2.2	ID	Yes
multivalue.dctlabs.com	A	3.3.3.3	ID	Yes



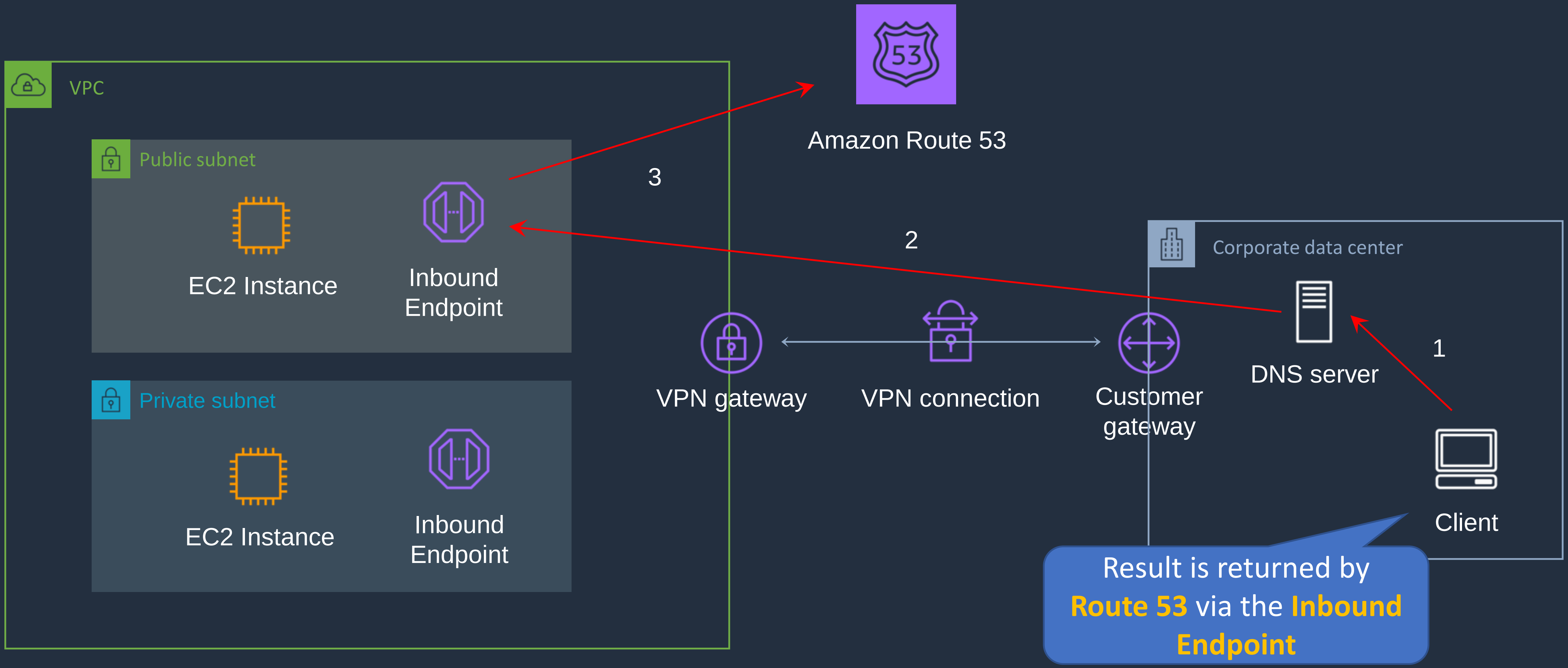
Failover Routing Policy with ALB

Amazon Route 53 Resolver

Route 53 Resolver – Outbound Endpoints

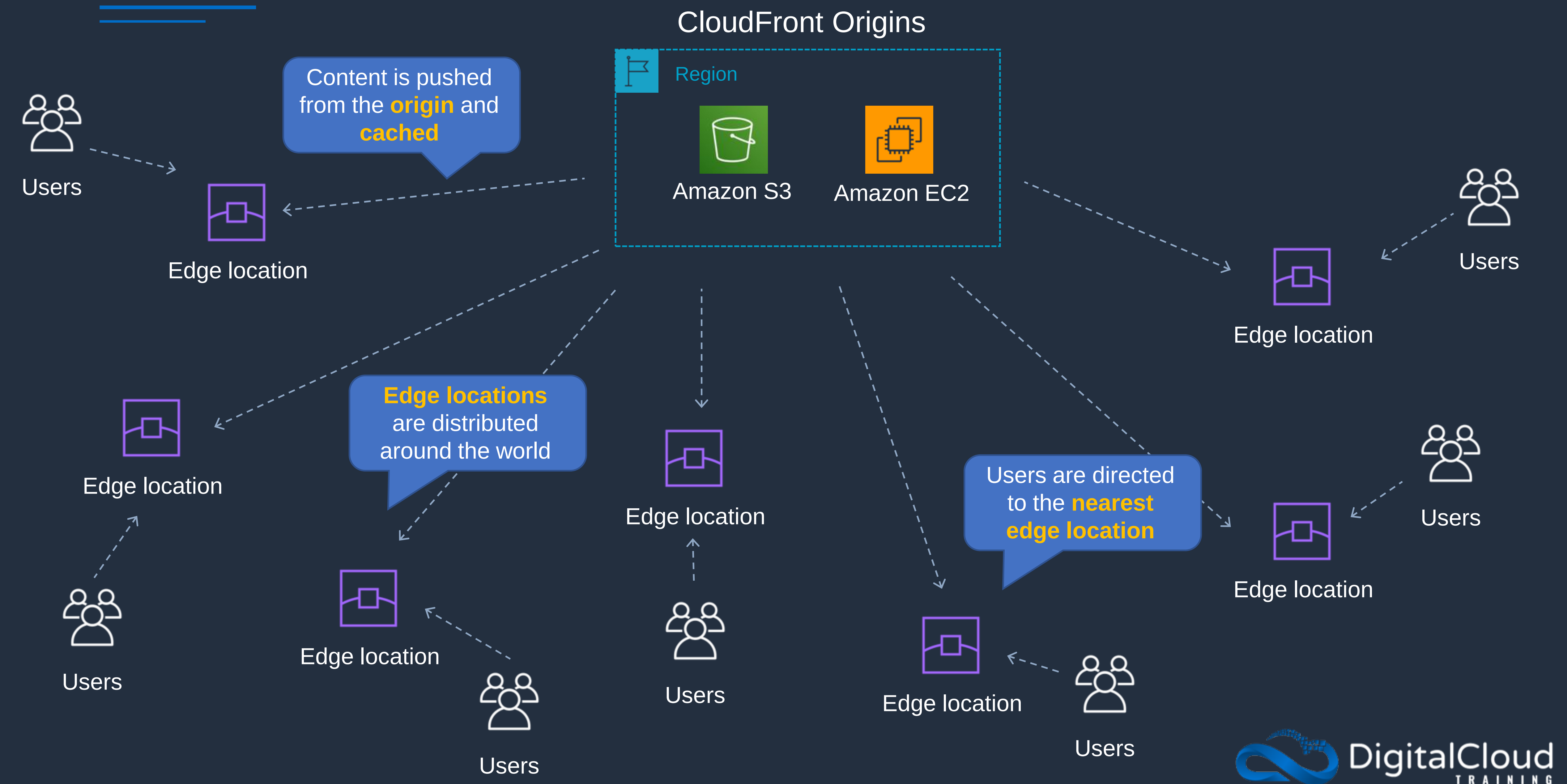


Route 53 Resolver – Inbound Endpoints



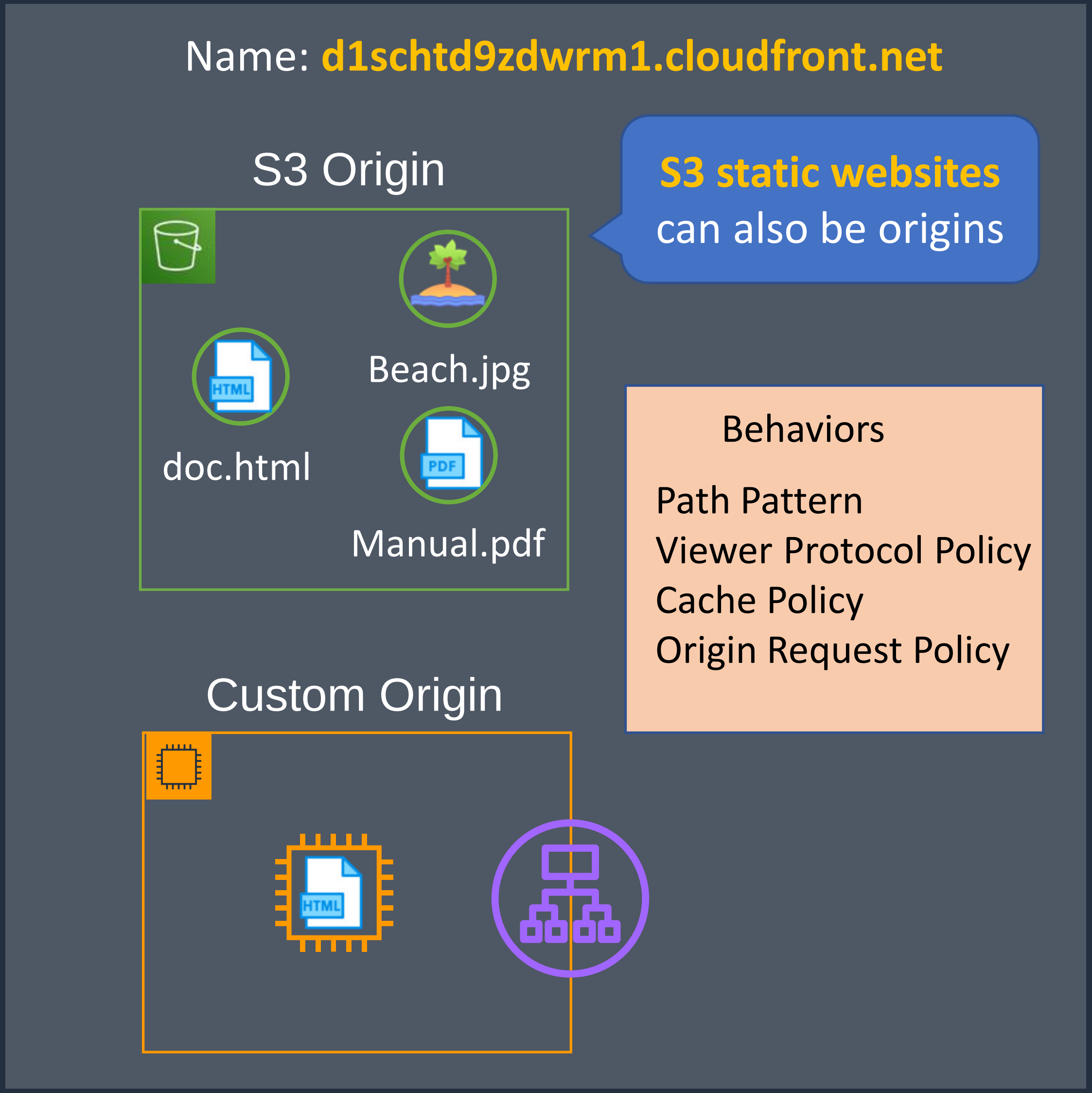
Amazon CloudFront Origins and Distributions

Amazon CloudFront



CloudFront Origins and Distributions

CloudFront Distribution



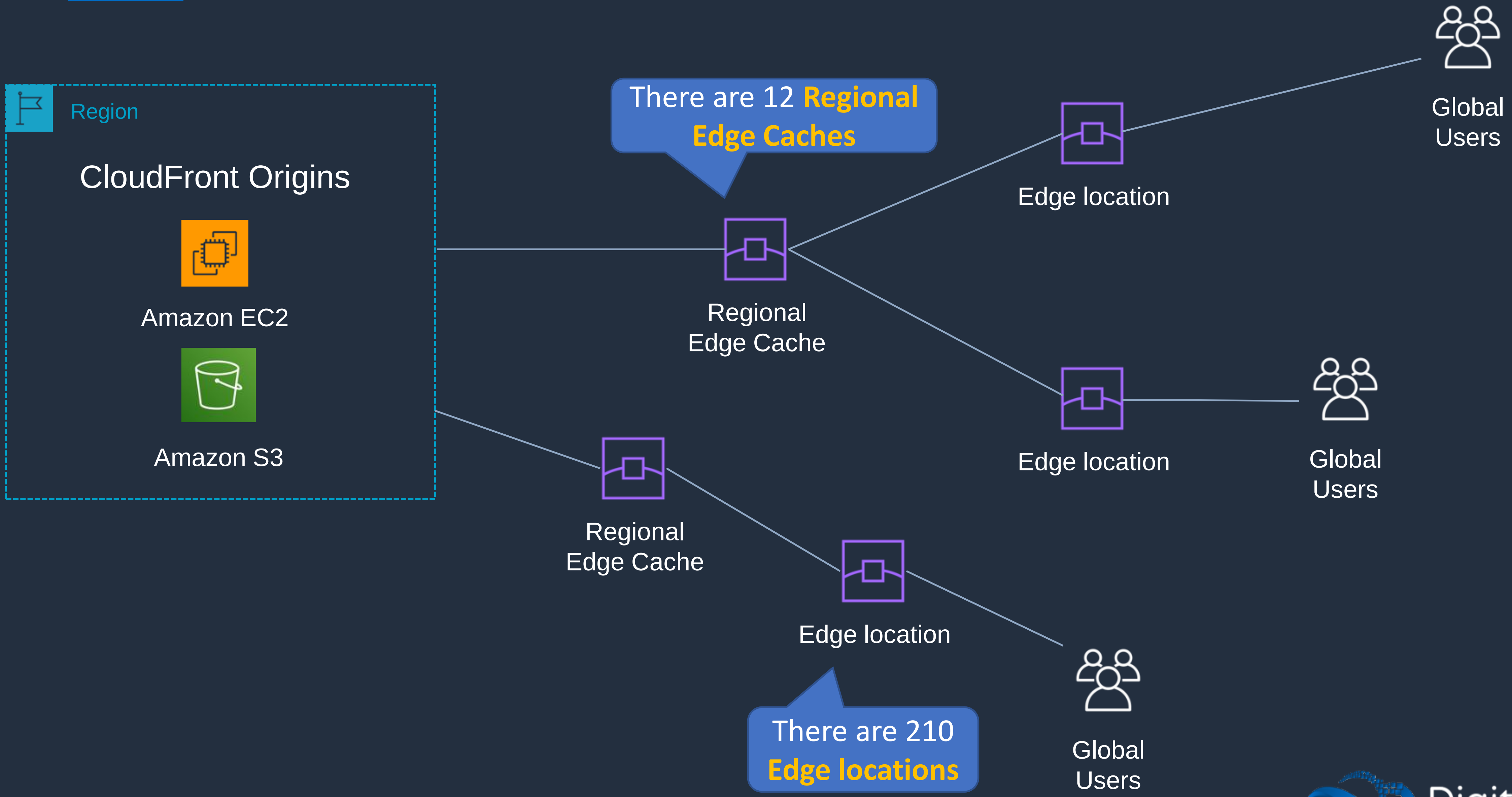
RTMP distributions were discontinued so only **web distributions** are currently available

CloudFront Web Distribution:

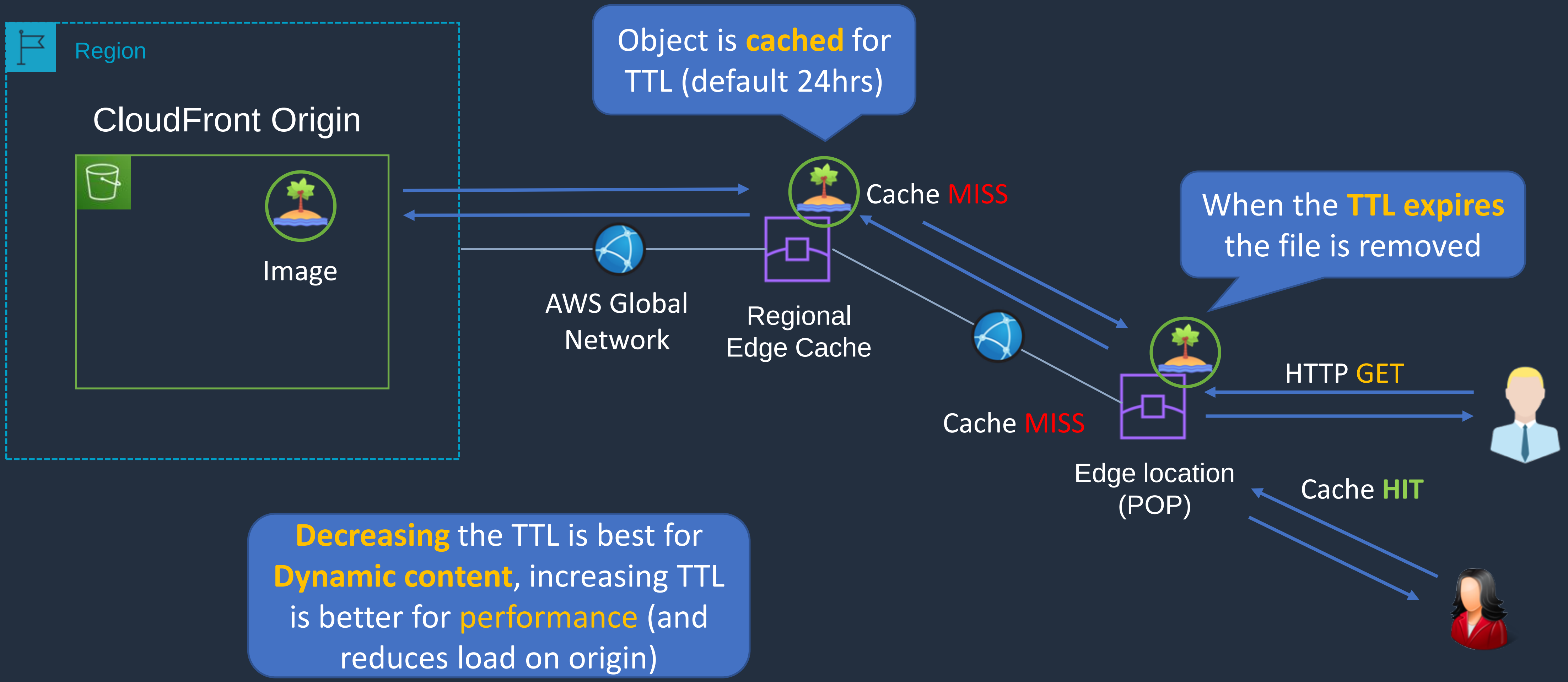
- Speed up distribution of static and dynamic content, for example, .html, .css, .php, and graphics files.
- Distribute media files using HTTP or HTTPS.
- Add, update, or delete objects, and submit data from web forms.
- Use live streaming to stream an event in real time.

Amazon CloudFront Caching and Behaviors

Amazon CloudFront Caching



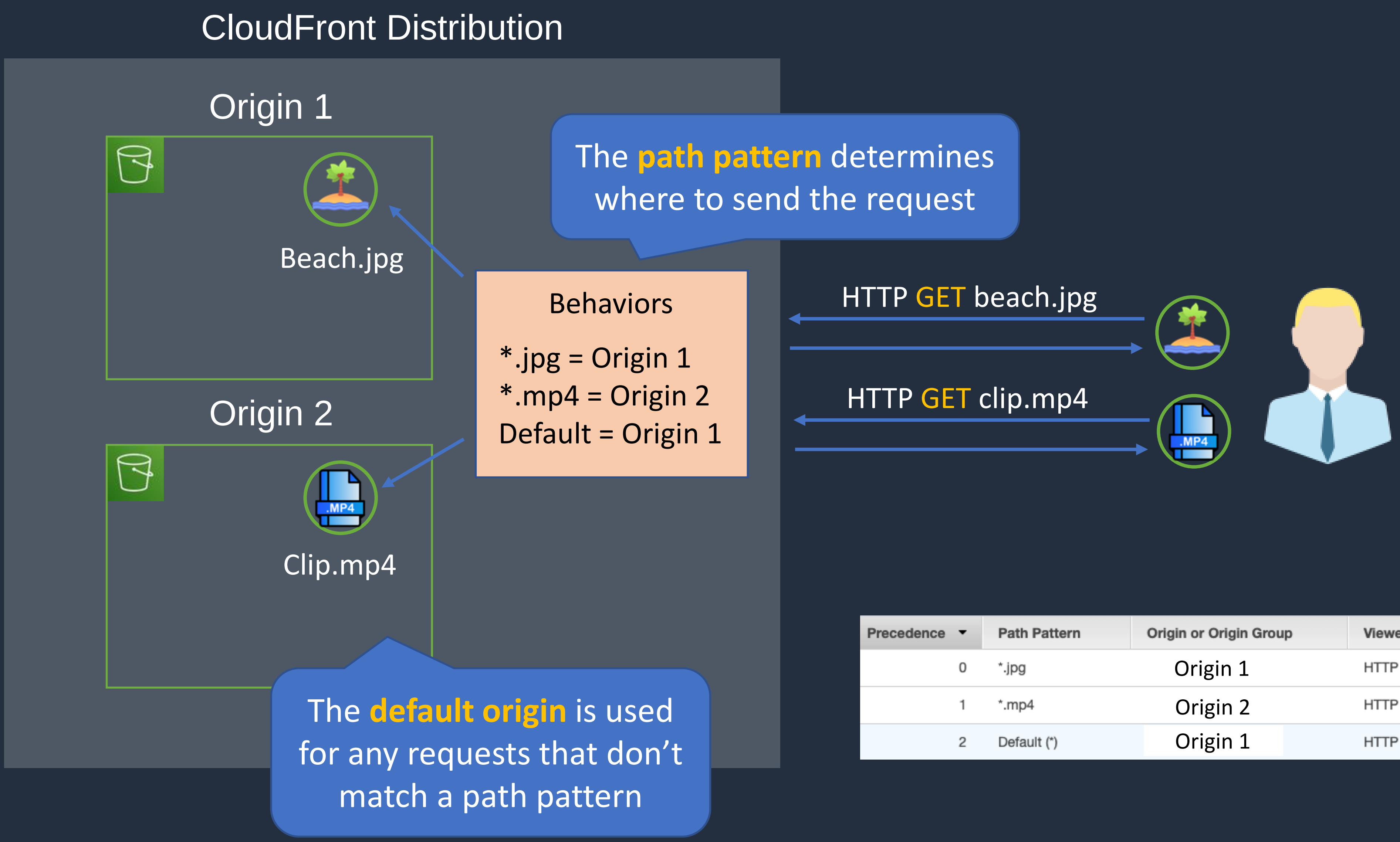
Amazon CloudFront Caching



Amazon CloudFront Caching

- You can define a maximum **Time To Live** (TTL) and a default TTL
- TTL is defined at the **behavior** level
- This can be used to define different TTLs for different file types (e.g. png vs jpg)
- After expiration, CloudFront checks the origin for any new requests (check the file is the latest version)
- Headers can be used to control the cache:
  - Cache-Control max-age=*(seconds)* - specify how long before CloudFront gets the object again from the origin server
  - Expires – specify an expiration date and time

CloudFront Path Patterns



Precedence	Path Pattern	Origin or Origin Group	Viewer Protocol Policy	Cache Policy Name
0	*.jpg	Origin 1	HTTP and HTTPS	Managed-CachingOptimized
1	*.mp4	Origin 2	HTTP and HTTPS	Managed-CachingOptimized
2	Default (*)	Origin 1	HTTP and HTTPS	Managed-CachingOptimized

Caching Based on Request Headers

- You can configure CloudFront to forward **headers** in the **viewer request** to the origin
- CloudFront can then cache multiple versions of an object based on the values in one or more request headers
- Controlled in a behavior to do one of the following:
  - Forward all headers to your origin (objects are not cached)
  - Forward a whitelist of headers that you specify
  - Forward only the default headers (doesn't cache objects based on values in request headers)

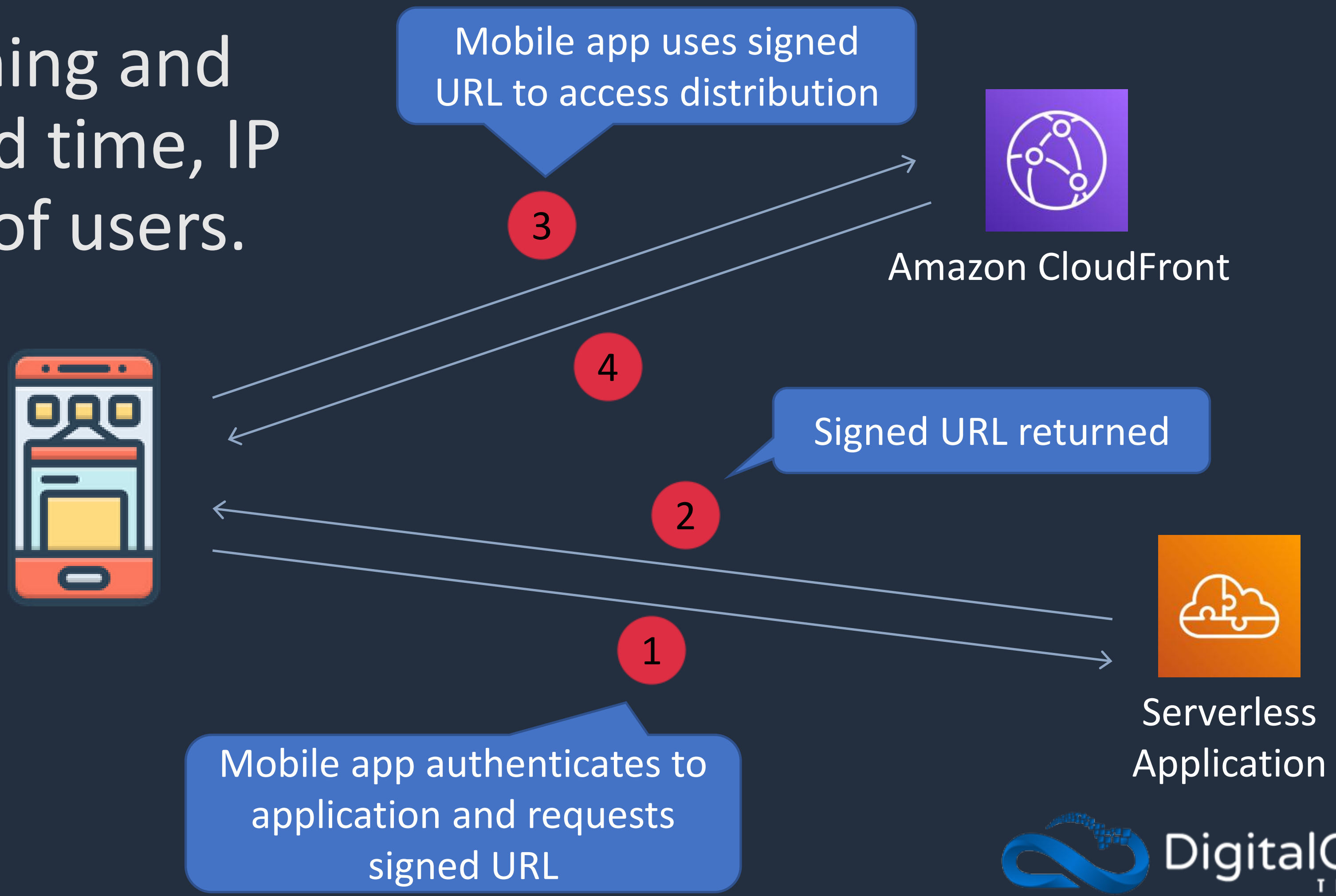
DigitalCloud TRAINING

CloudFront Signed URLs and OAI

CloudFront Signed URLs

- Signed URLs provide more control over access to content.
- Can specify beginning and expiration date and time, IP addresses/ranges of users.

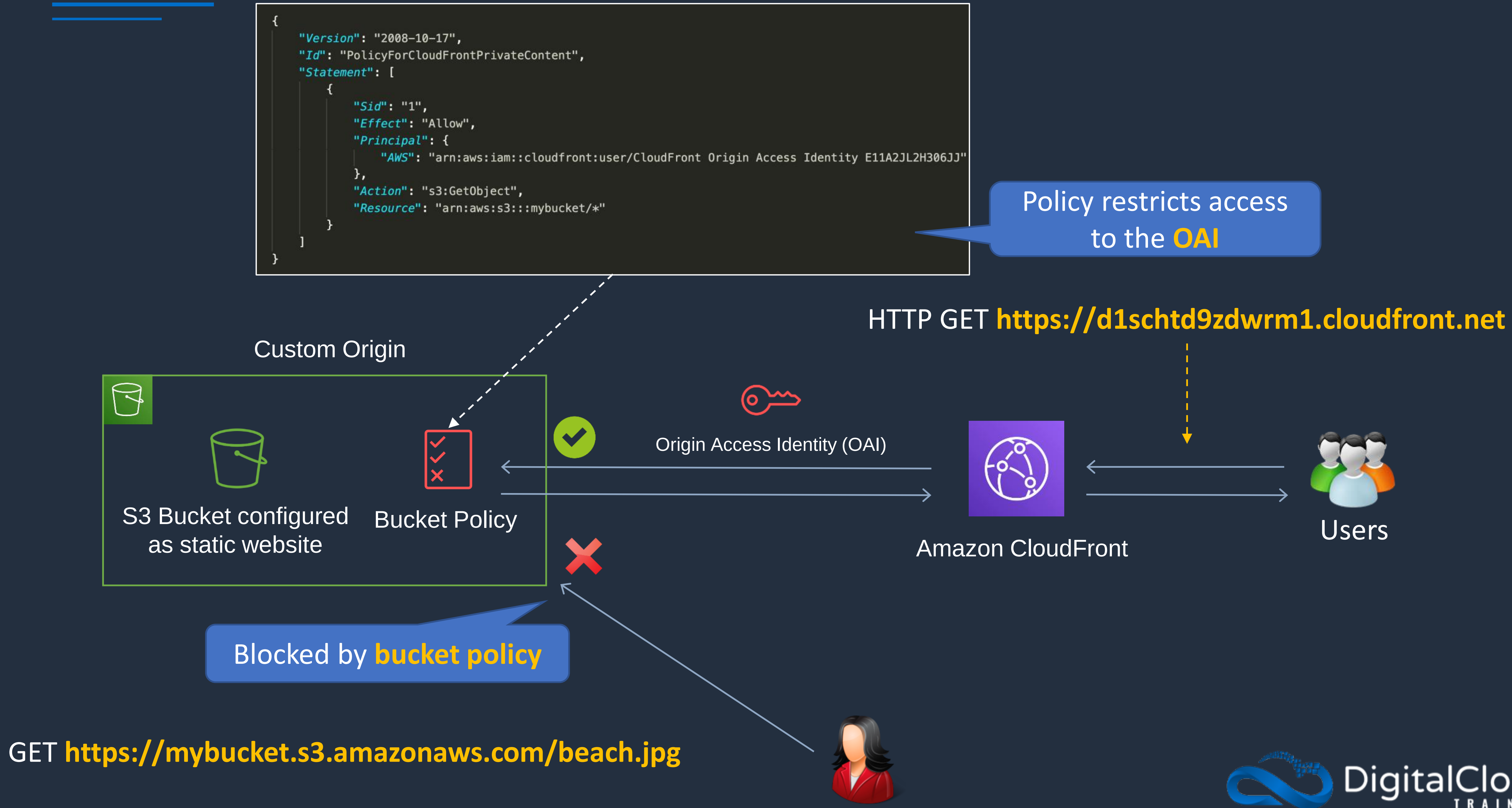
Signed URLs should be used for **individual files** and clients that don’t support **cookies**.



CloudFront Signed Cookies

- Similar to Signed URLs
- Use signed cookies when you don't want to change URLs
- Can also be used when you want to provide access to **multiple restricted files** (Signed URLs are for individual files)

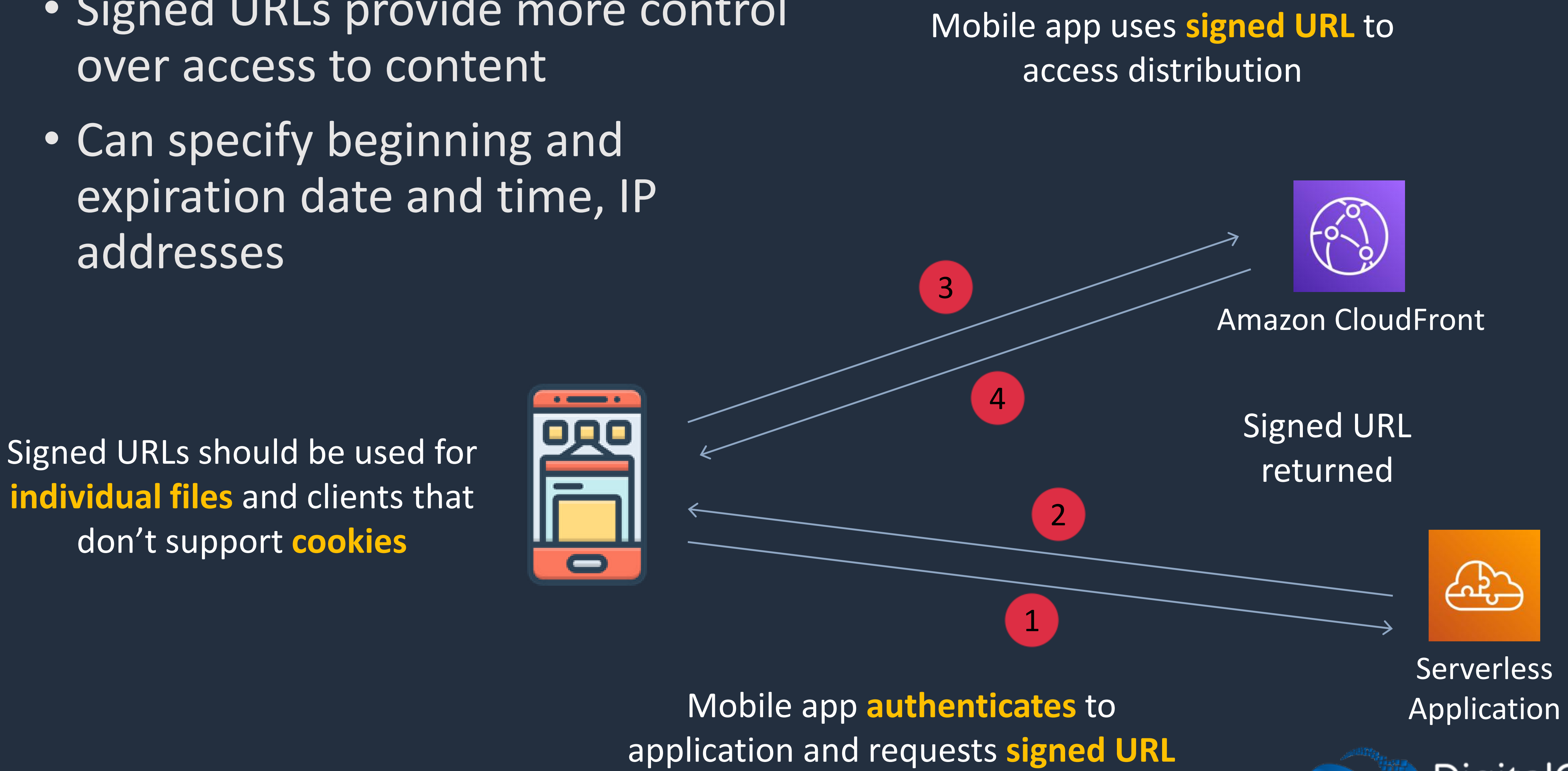
CloudFront Origin Access Identity (OAI)



CloudFront Signed URLs and OAI/OAC

CloudFront Signed URLs

- Signed URLs provide more control over access to content
- Can specify beginning and expiration date and time, IP addresses



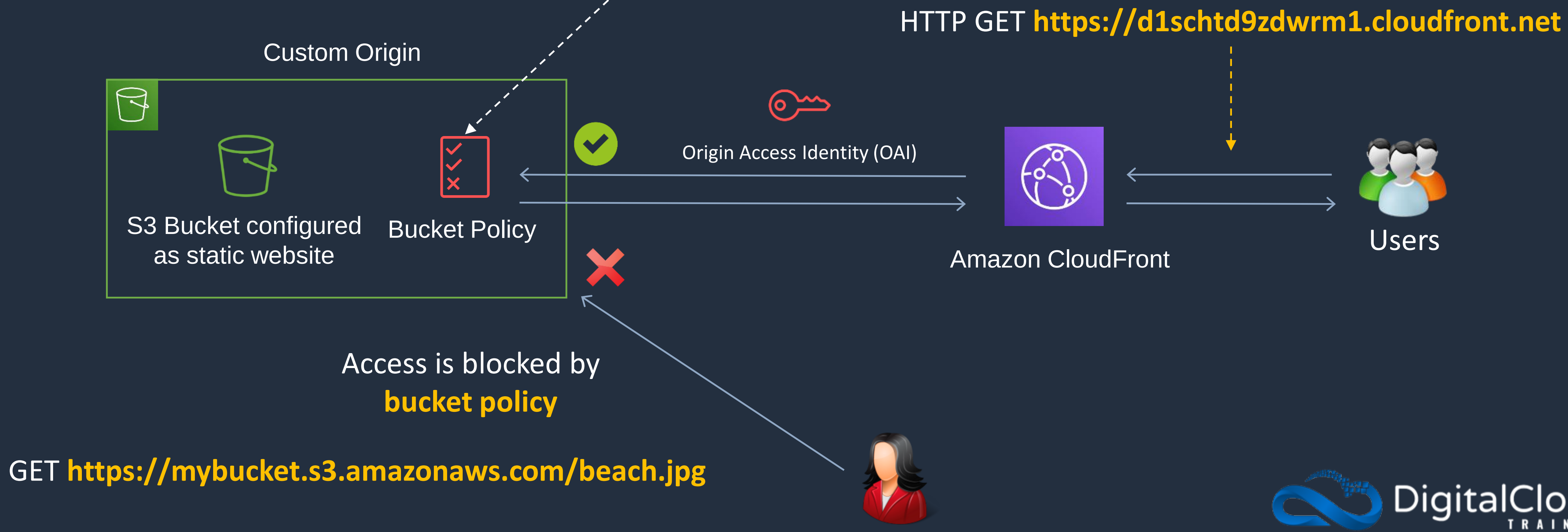
CloudFront Signed Cookies

- Similar to Signed URLs
- Use signed cookies when you don't want to change URLs
- Can also be used when you want to provide access to **multiple restricted files** (Signed URLs are for individual files)

CloudFront Origin Access Identity (OAI)

```
{
    "Version": "2008-10-17",
    "Id": "PolicyForCloudFrontPrivateContent",
    "Statement": [
        {
            "Sid": "1",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::cloudfront:user/CloudFront Origin Access Identity E11A2JL2H306JJ"
            },
            "Action": "s3:GetObject",
            "Resource": "arn:aws:s3:::mybucket/*"
        }
    ]
}
```

The policy restricts access to the **OAI**



CloudFront Origin Access Control (OAC)

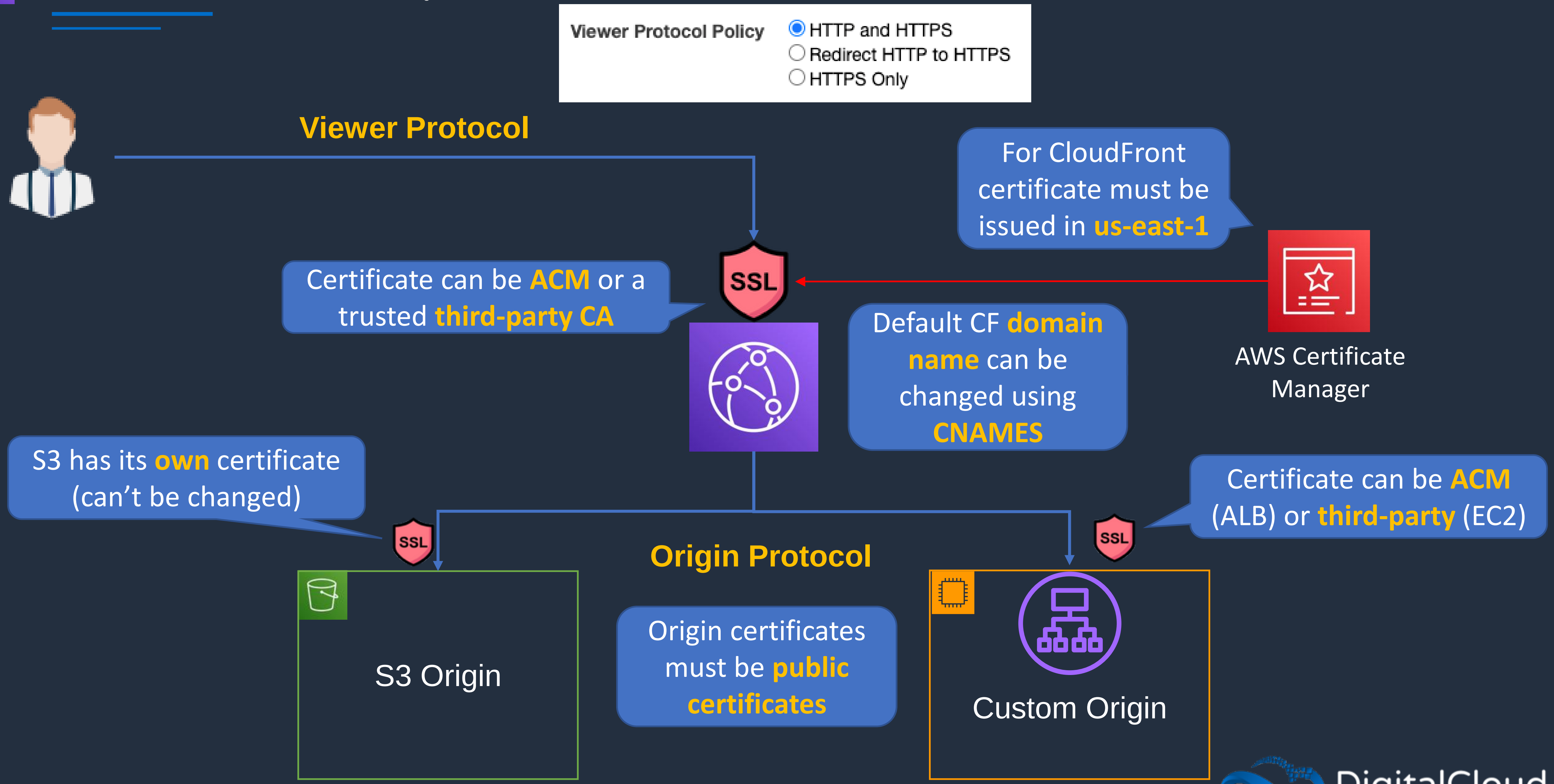
- Like a OAI but supports additional use cases
- AWS recommend using the OAC instead of an OAI
- Requires an S3 bucket policy that allows the CloudFront service principal

```
{
    "Version": "2012-10-17",
    "Statement": {
        "Sid": "AllowCloudFrontServicePrincipalReadOnly",
        "Effect": "Allow",
        "Principal": {
            "Service": "cloudfront.amazonaws.com"
        },
        "Action": "s3:GetObject",
        "Resource": "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*",
        "Condition": {
            "StringEquals": {
                "AWS:SourceArn": "arn:aws:cloudfront::111122223333:distribution/EDFDVBD6EXAMPLE"
            }
        }
    }
}
```

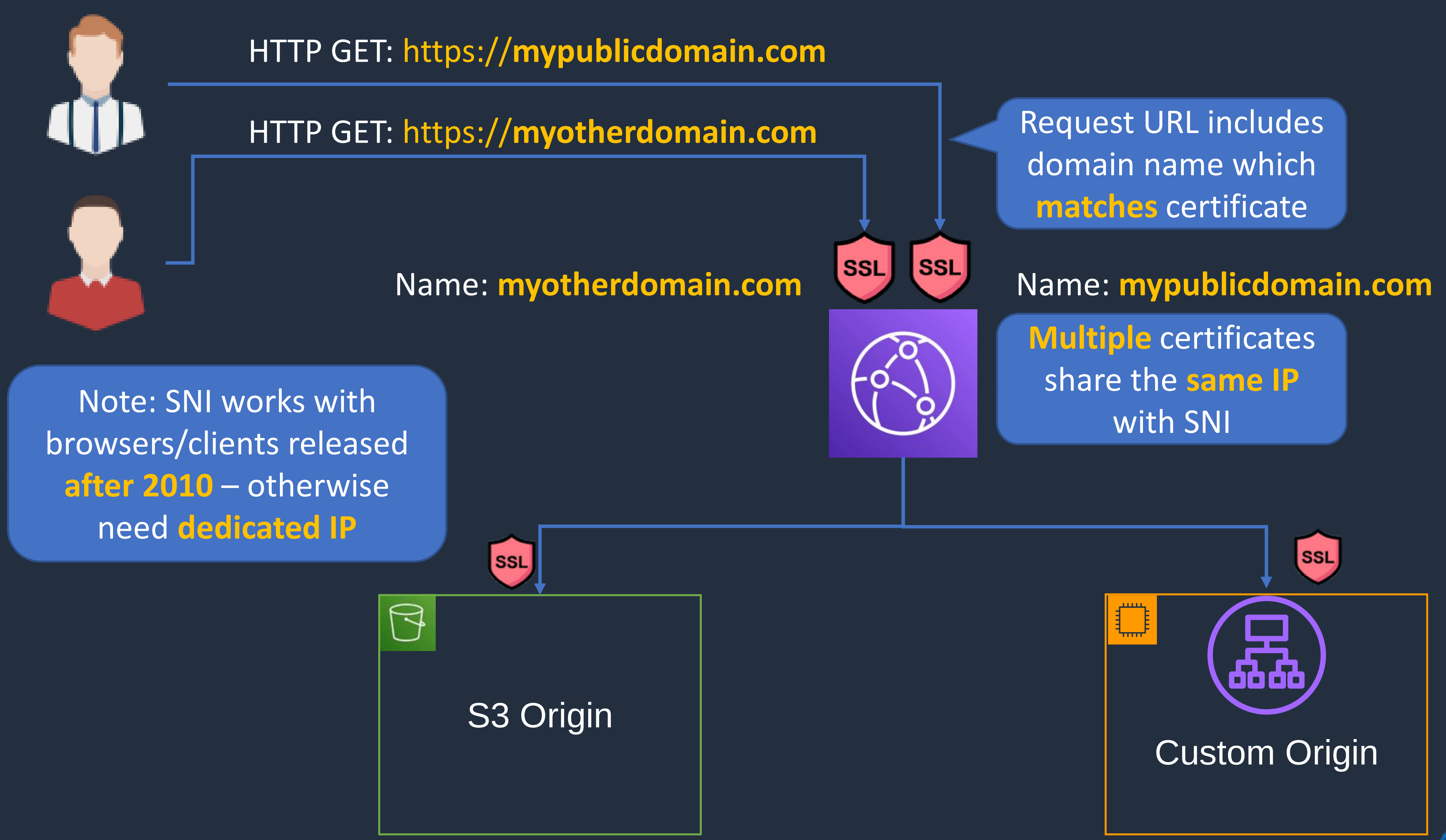
CloudFront Cache and Behavior Settings

CloudFront SSL/TLS and SNI

CloudFront SSL/TLS



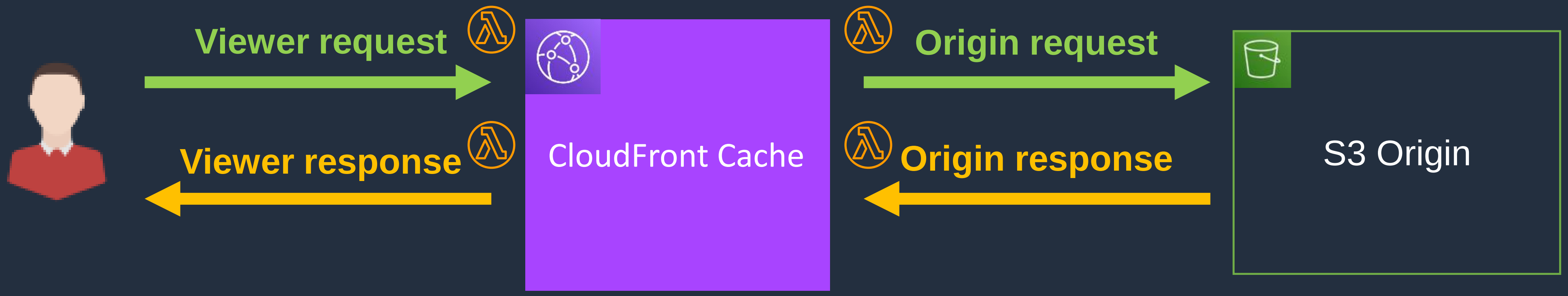
CloudFront Server Name Indication (SNI)



Lambda@Edge

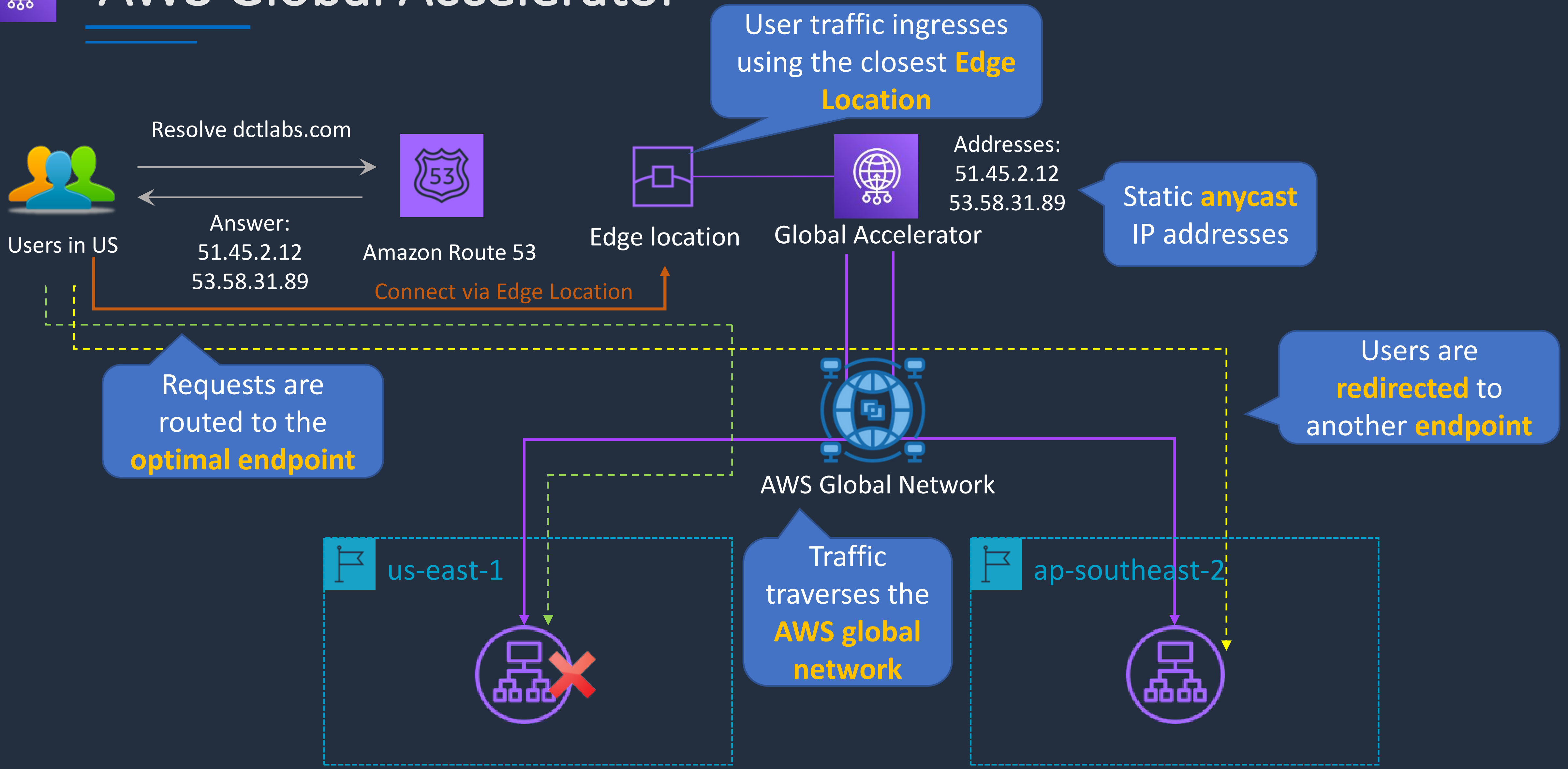
Lambda@Edge

- Run Node.js and Python Lambda functions to customize the content CloudFront delivers
- Executes functions closer to the viewer
- Can be run at the following points
  - After CloudFront receives a request from a viewer (viewer request)
  - Before CloudFront forwards the request to the origin (origin request)
  - After CloudFront receives the response from the origin (origin response)
  - Before CloudFront forwards the response to the viewer (viewer response)



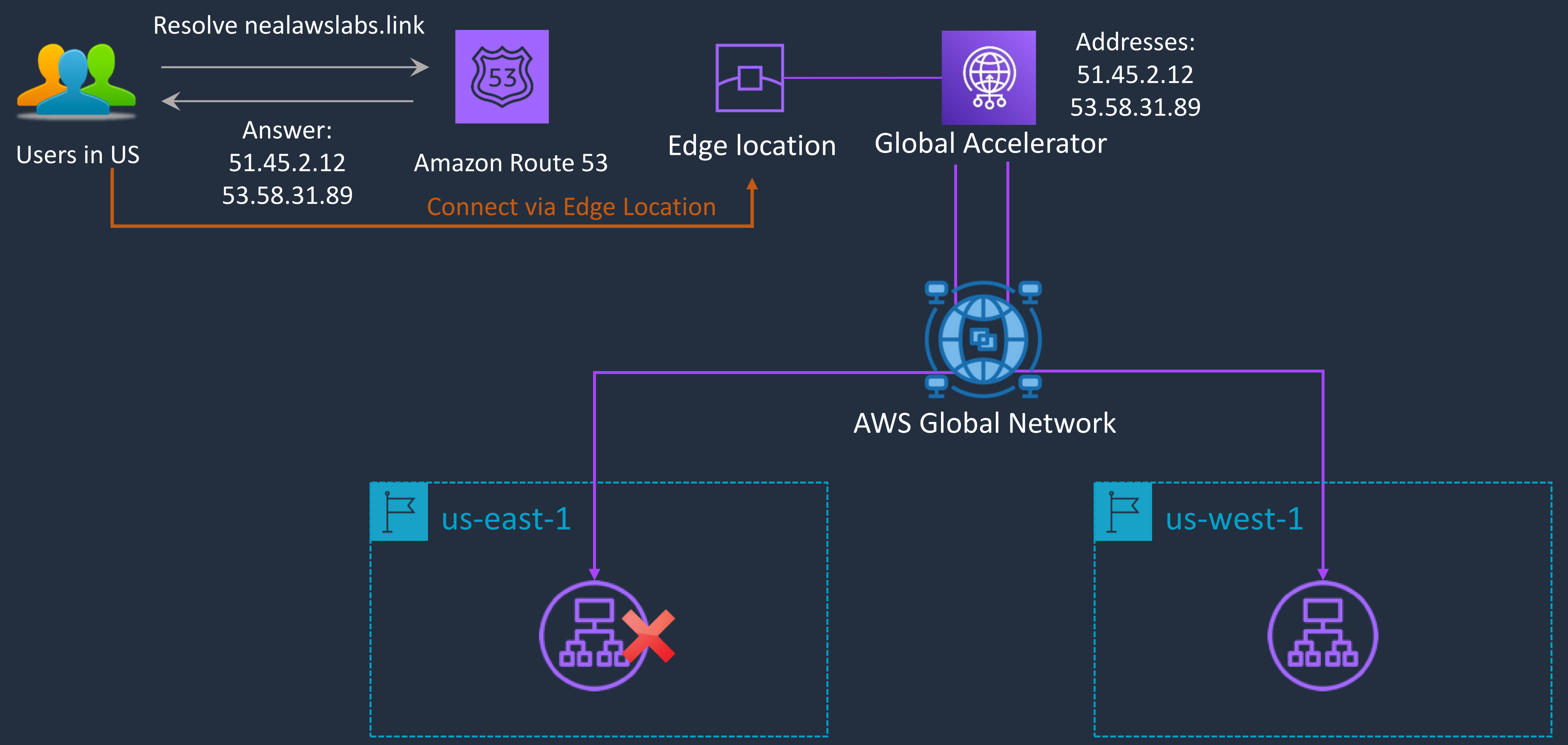
AWS Global Accelerator

AWS Global Accelerator



Create a Global Accelerator

AWS Global Accelerator



SECTION 10

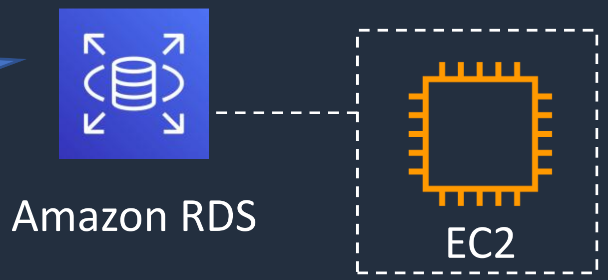
AWS Database Services

Amazon RDS Scaling and Deployment

Amazon RDS Overview

RDS runs on **EC2 instances**, so you must choose an **instance type**

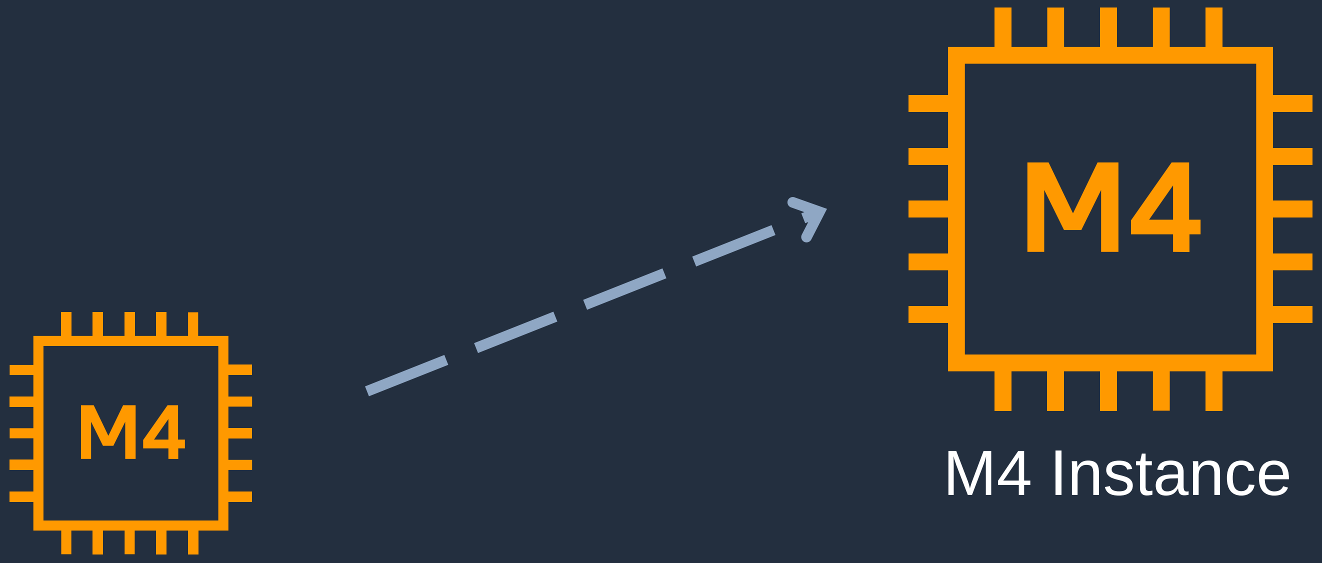
RDS is a **managed**, relational database



RDS supports the following database engines:

- Amazon Aurora
- MySQL
- MariaDB
- Oracle
- Microsoft SQL Server
- PostgreSQL

Amazon RDS Scaling Up (vertically)



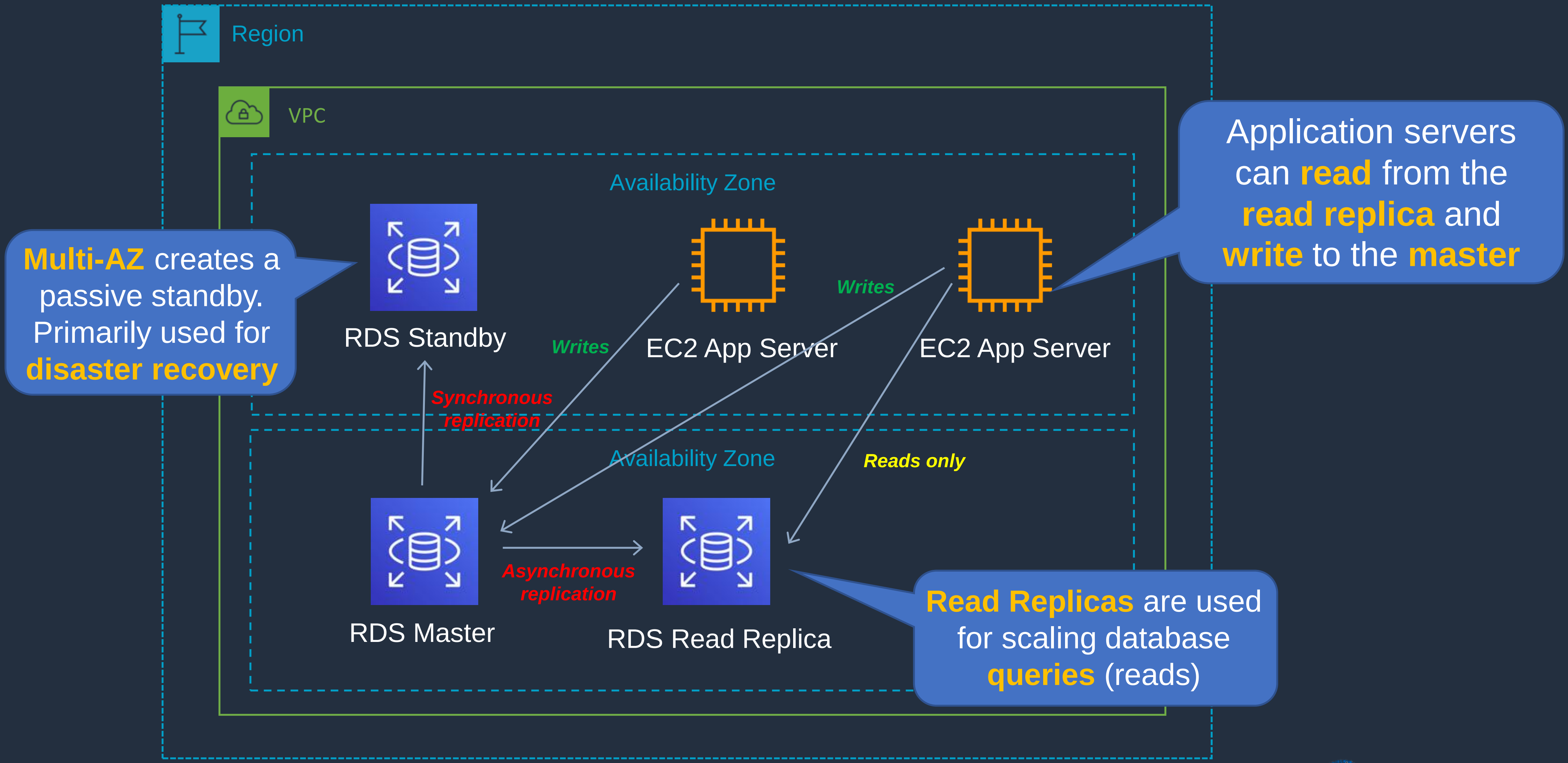
M4 instance

db.m4.large 2 vCPUs, 8 GiB RAM

M4 Instance

db.m4.2xlarge 4 vCPUs, 32 GiB RAM

Disaster Recovery (DR) and Scaling Out (Horizontally)

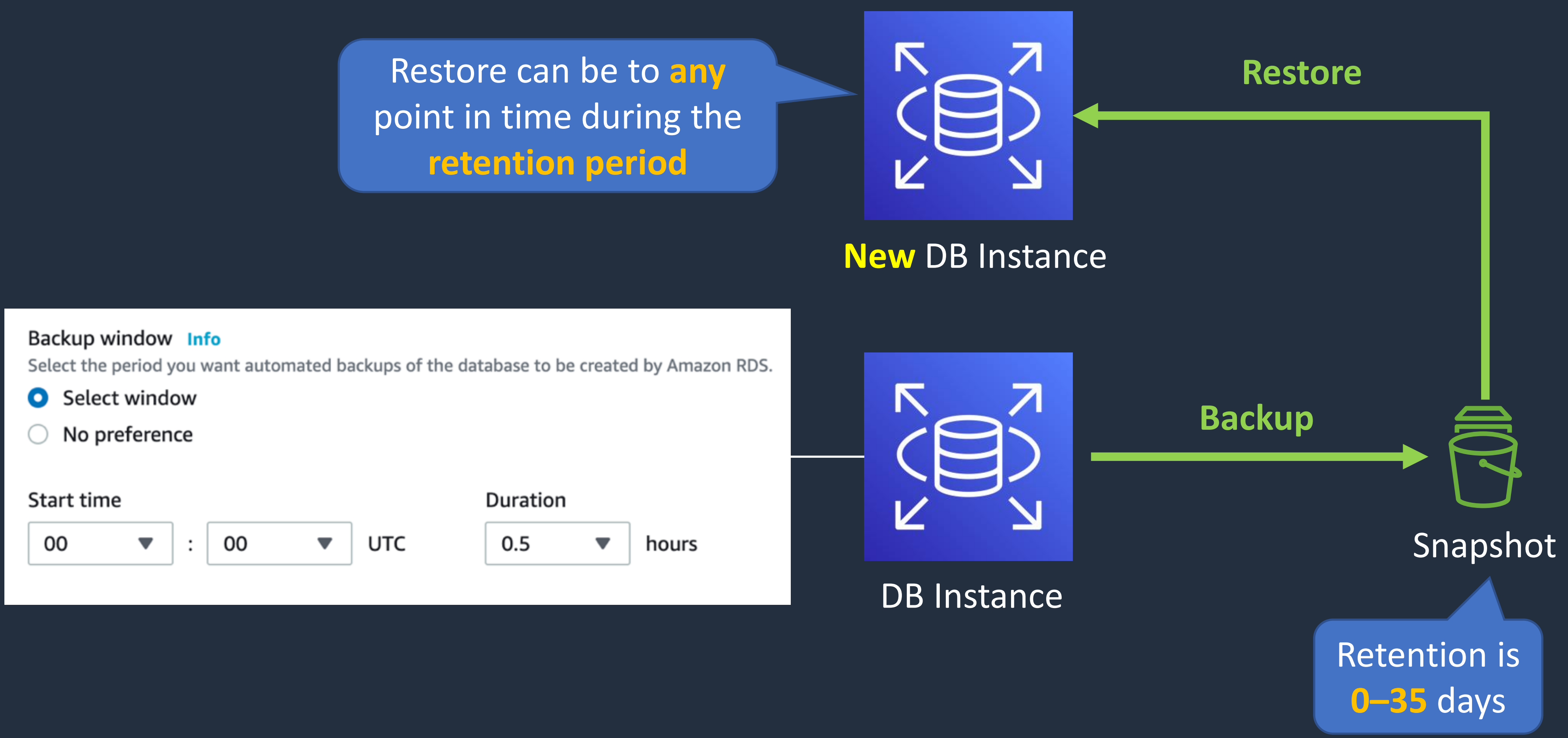


Amazon RDS Multi-AZ and Read Replicas

Multi-AZ Deployments	Read Replicas
Synchronous replication – highly durable	Asynchronous replication – highly scalable
Only database engine on primary instance is active	All read replicas are accessible and can be used for read scaling
Automated backups are taken from standby	No backups configured by default
Always span two Availability Zones within a single Region	Can be within an Availability Zone, Cross-AZ, or Cross-Region
Database engine version upgrades happen on primary	Database engine version upgrade is independent from source instance
Automatic failover to standby when a problem is detected	Can be manually promoted to a standalone database instance

Amazon RDS Backup and Recovery

Amazon RDS Automated Backups

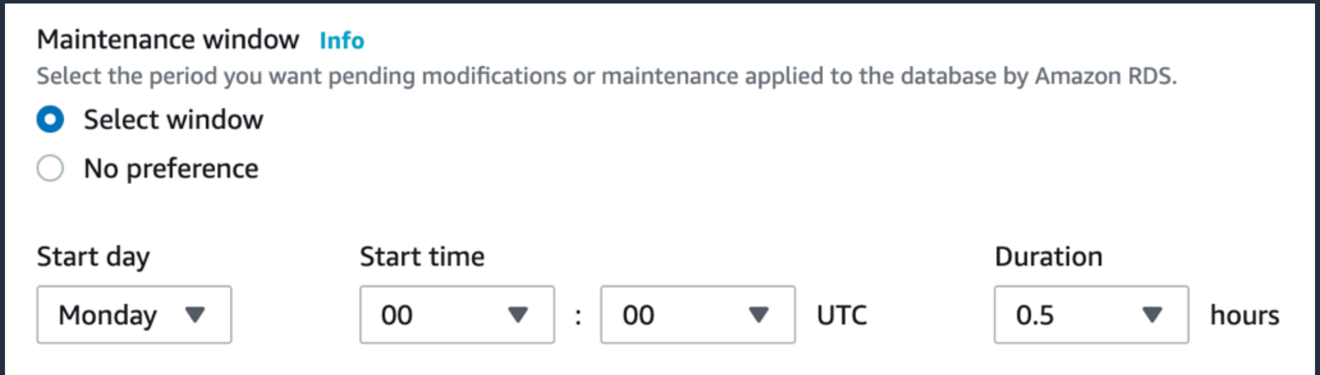


Amazon RDS Manual Backups (Snapshot)

- Backs up the entire DB instance, not just individual databases
- For single-AZ DB instances there is a brief suspension of I/O
- For Multi-AZ SQL Server, I/O activity is briefly suspended on primary
- For Multi-AZ MariaDB, MySQL, Oracle and PostgreSQL the snapshot is taken from the standby
- Snapshots do not expire (no retention period)

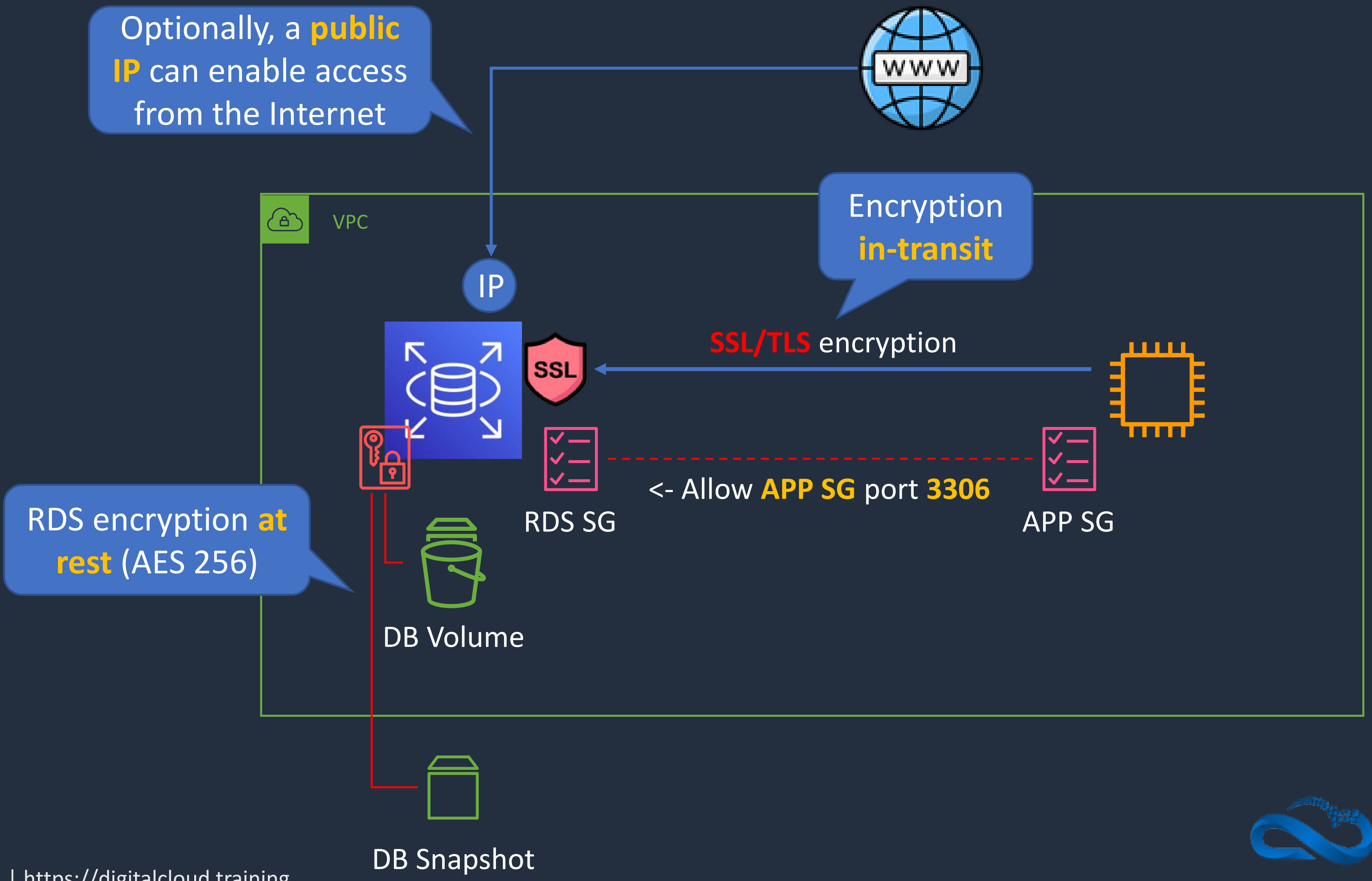
Amazon RDS Maintenance Windows

- Operating system and DB patching can require taking the database offline
- These tasks take place during a maintenance window
- By default a weekly maintenance window is configured
- You can choose your own maintenance window



Amazon RDS Security

Amazon RDS Security



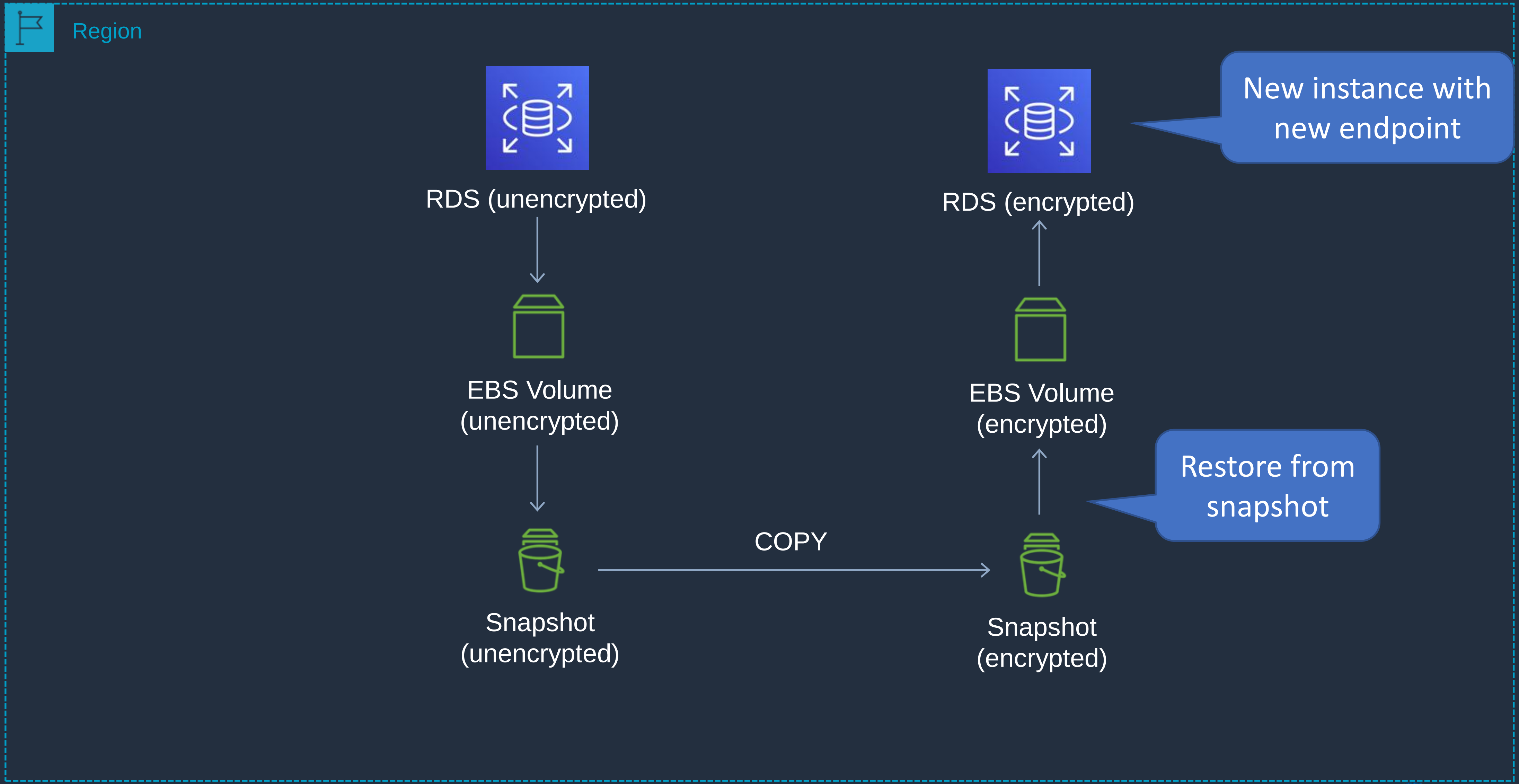
Amazon RDS Security

- Encryption at rest can be enabled – includes DB storage, backups, read replicas and snapshots
- You can only enable encryption for an Amazon RDS DB instance when you create it, not after the DB instance is created
- DB instances that are encrypted can't be modified to disable encryption
- Uses AES 256 encryption and encryption is transparent with minimal performance impact
- RDS for Oracle and SQL Server is also supported using Transparent Data Encryption (TDE) (may have performance impact)
- AWS KMS is used for managing encryption keys

Amazon RDS Security

- You can't have an encrypted read replica of an unencrypted DB instance or an unencrypted read replica of an encrypted DB instance
- Read replicas of encrypted master instances are encrypted
- The same key is used if in the same Region as the master
- If the read replica is in a different Region, a different key is used
- You can't restore an unencrypted backup or snapshot to an encrypted DB instance

Amazon RDS Security



Amazon Aurora Core Knowledge

Amazon Aurora Key Features

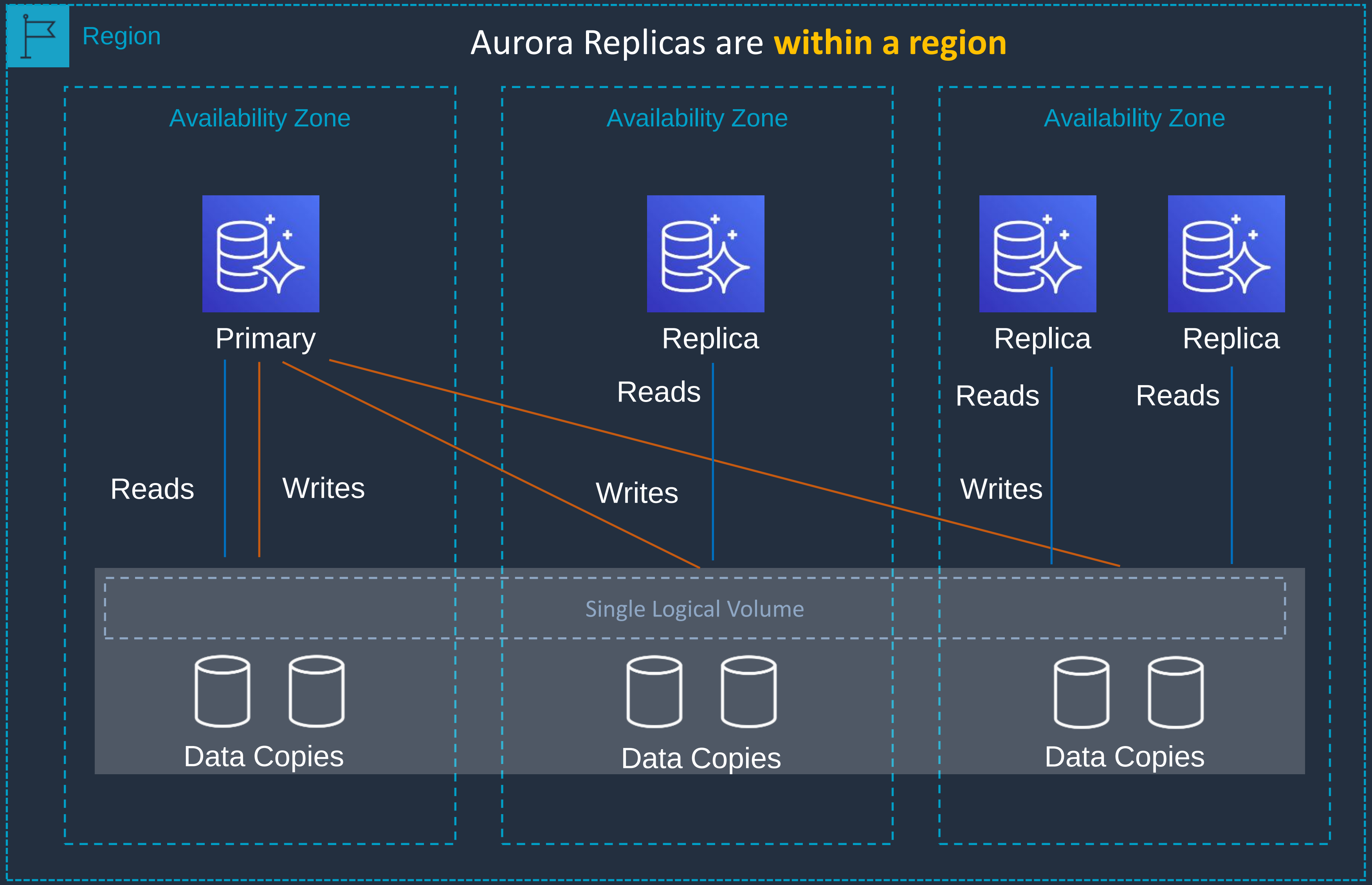
Aurora Feature	Benefit
High performance and scalability	Offers high performance, self-healing storage that scales up to 64TB, point-in-time recovery and continuous backup to S3
DB compatibility	Compatible with existing MySQL and PostgreSQL open source databases
Aurora Replicas	In-region read scaling and failover target – up to 15 (can use Auto Scaling)
MySQL Read Replicas	Cross-region cluster with read scaling and failover target – up to 5 (each can have up to 15 Aurora Replicas)
Global Database	Cross-region cluster with read scaling (fast replication / low latency reads). Can remove secondary and promote
Multi-Master	Scales out writes within a region. In preview currently and will not appear on the exam
Serverless	On-demand, autoscaling configuration for Amazon Aurora - does not support read replicas or public IPs (can only access through VPC or Direct Connect - not VPN)

Amazon Aurora Replicas

Feature	Aurora Replica	MySQL Replica
Number of replicas	Up to 15	Up to 5
Replication type	Asynchronous (milliseconds)	Asynchronous (seconds)
Performance impact on primary	Low	High
Replica location	In-region	Cross-region
Act as failover target	Yes (no data loss)	Yes (potentially minutes of data loss)
Automated failover	Yes	No
Support for user-defined replication delay	No	Yes
Support for different data or schema vs. primary	No	Yes

Amazon Aurora Deployment Options

Aurora Fault Tolerance and Aurora Replicas

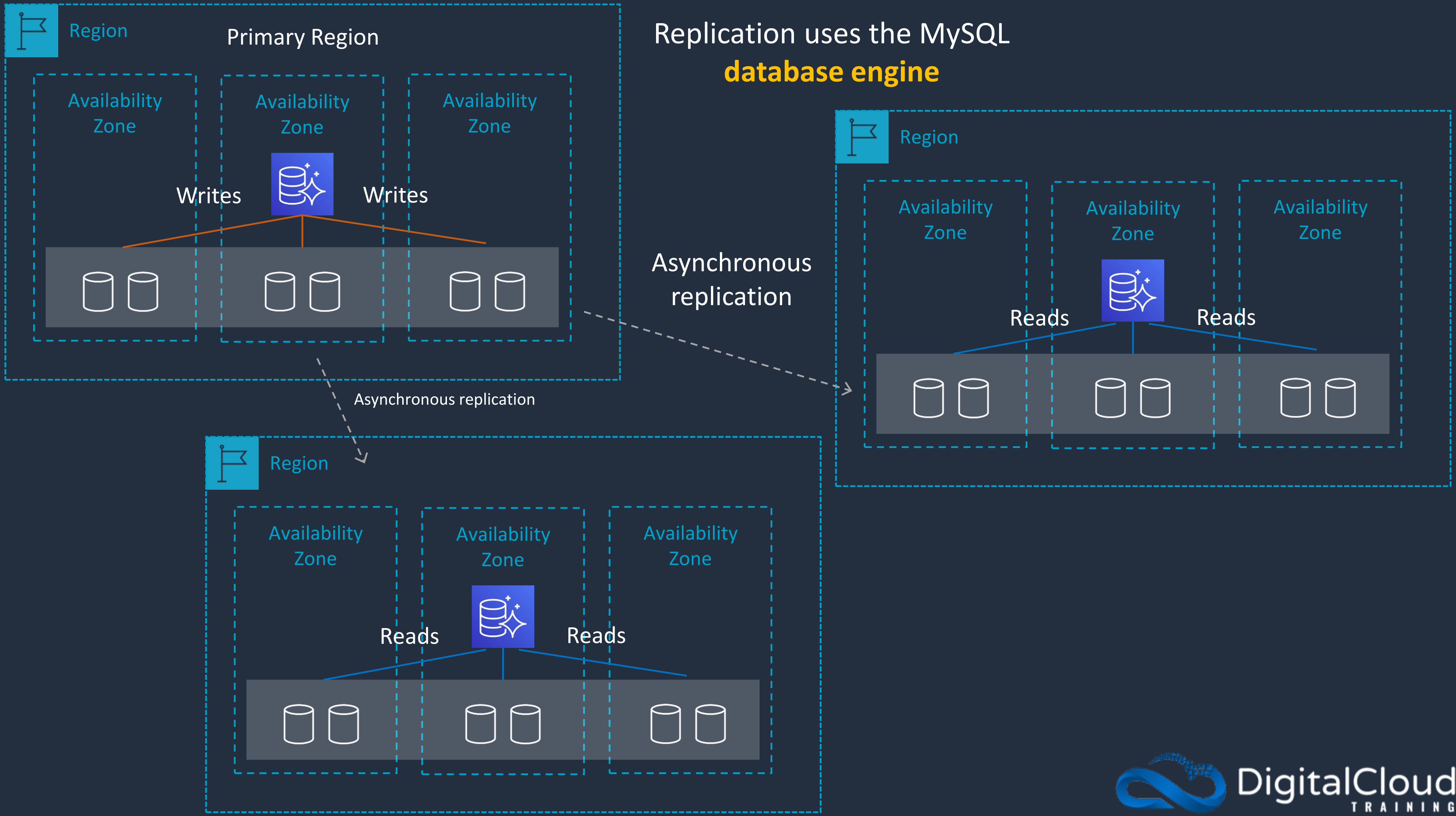


Aurora Fault Tolerance

- Fault tolerance across 3 AZs
- Single logical volume
- Aurora Replicas scale-out read requests
- Up to 15 Aurora Replicas with **sub-10ms** replica lag
- Aurora Replicas are independent endpoints
- Can **promote** Aurora Replica to be a new primary or create new primary
- Set priority (tiers) on Aurora Replicas to control order of promotion
- Can use **Auto Scaling** to add replicas

Cross-Region Replica with Aurora MySQL

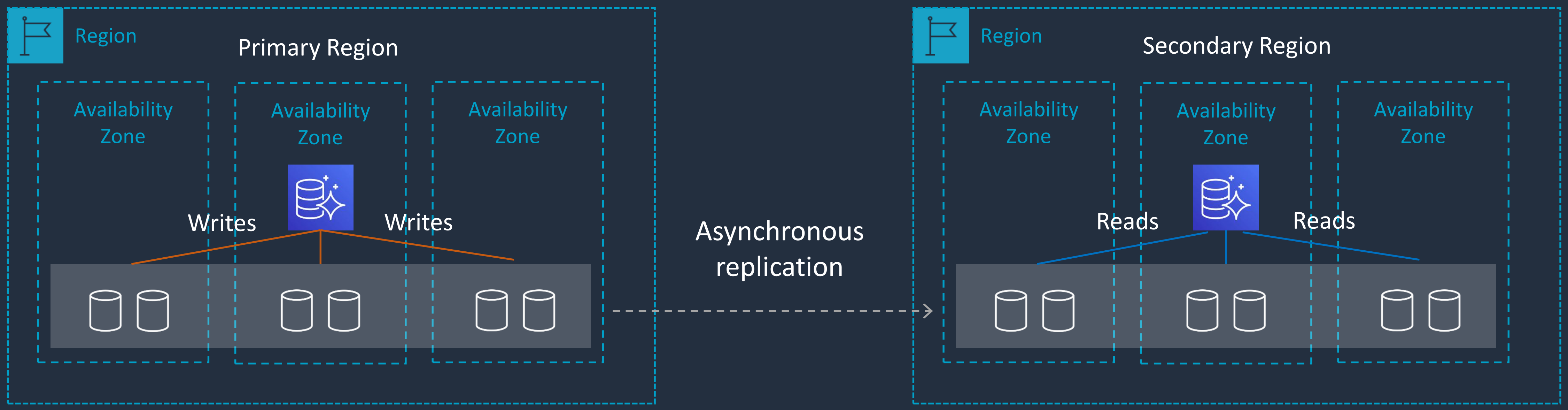
Replication uses the MySQL **database engine**



Aurora Global Database

A database in a secondary Region can be promoted to full read/write capabilities in less than **1 minute**

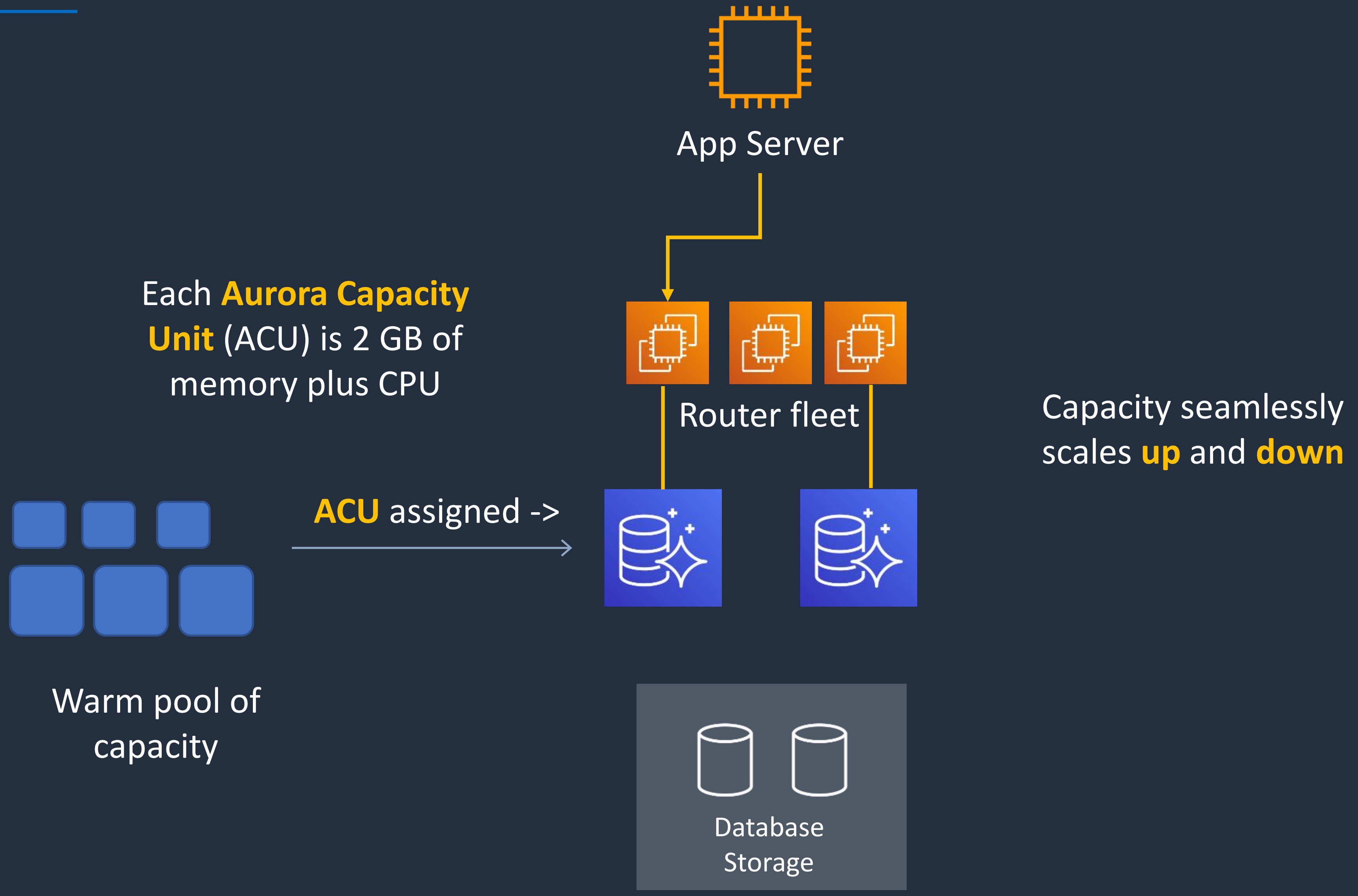
Applications can connect to the cluster **Reader Endpoint**



Replication uses the **Aurora storage layer**

Secondary clusters can be configured with **write-forwarding**

Aurora Serverless



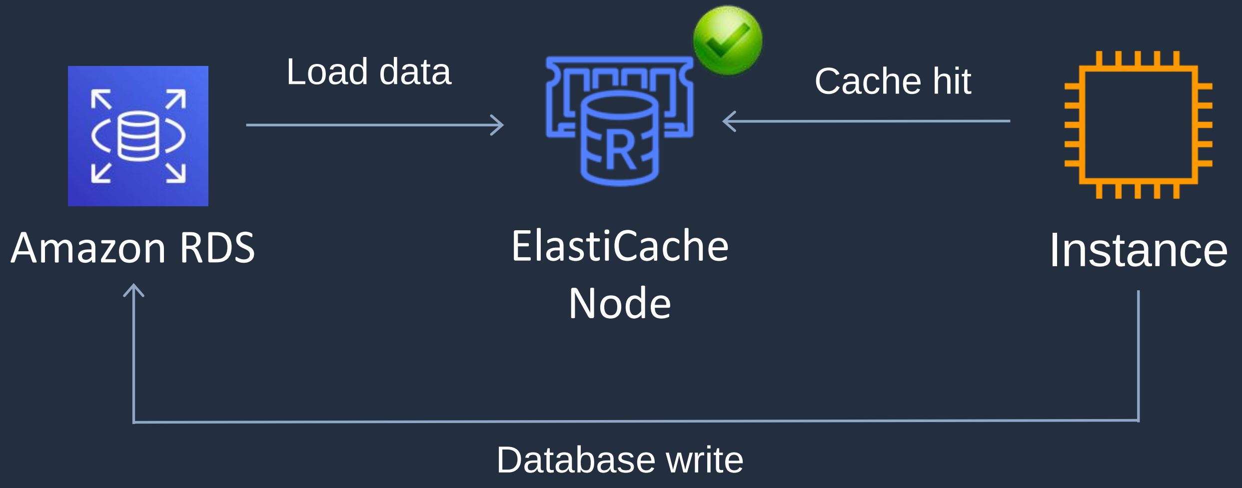
Aurora Serverless Use Cases

- Infrequently used applications
- New applications
- Variable workloads
- Unpredictable workloads
- Development and test databases
- Multi-tenant applications

Amazon ElastiCache Core Knowledge

Amazon ElastiCache Core Knowledge

- Fully managed implementations **Redis** and **Memcached**
- ElastiCache is a **key/value** store
- In-memory database offering high performance and low latency
- Can be put in front of databases such as RDS and DynamoDB



Amazon ElastiCache Core Knowledge

Feature	Memcached	Redis (cluster mode disabled)	Redis (cluster mode enabled)
Data persistence	No	Yes	Yes
Data types	Simple	Complex	Complex
Data partitioning	Yes	No	Yes
Encryption	No	Yes	Yes
High availability (replication)	No	Yes	Yes
Multi-AZ	Yes, place nodes in multiple AZs. No failover or replication	Yes, with auto-failover. Uses read replicas (0-5 per shard)	Yes, with auto-failover. Uses read replicas (0-5 per shard)
Scaling	Up (node type); out (add nodes)	Up (node type); out (add replica)	Up (node type); out (add shards)
Multithreaded	Yes	No	No
Backup and restore	No (and no snapshots)	Yes, automatic and manual snapshots	Yes, automatic and manual snapshots

Amazon ElastiCache Use Cases

- Data that is relatively **static** and **frequently accessed**
- Applications that are tolerant of stale data
- Data is slow and expensive to get compared to cache retrieval
- Require push-button scalability for memory, writes and reads
- Often used for storing session state

Scaling ElastiCache

Amazon ElastiCache - Scalability

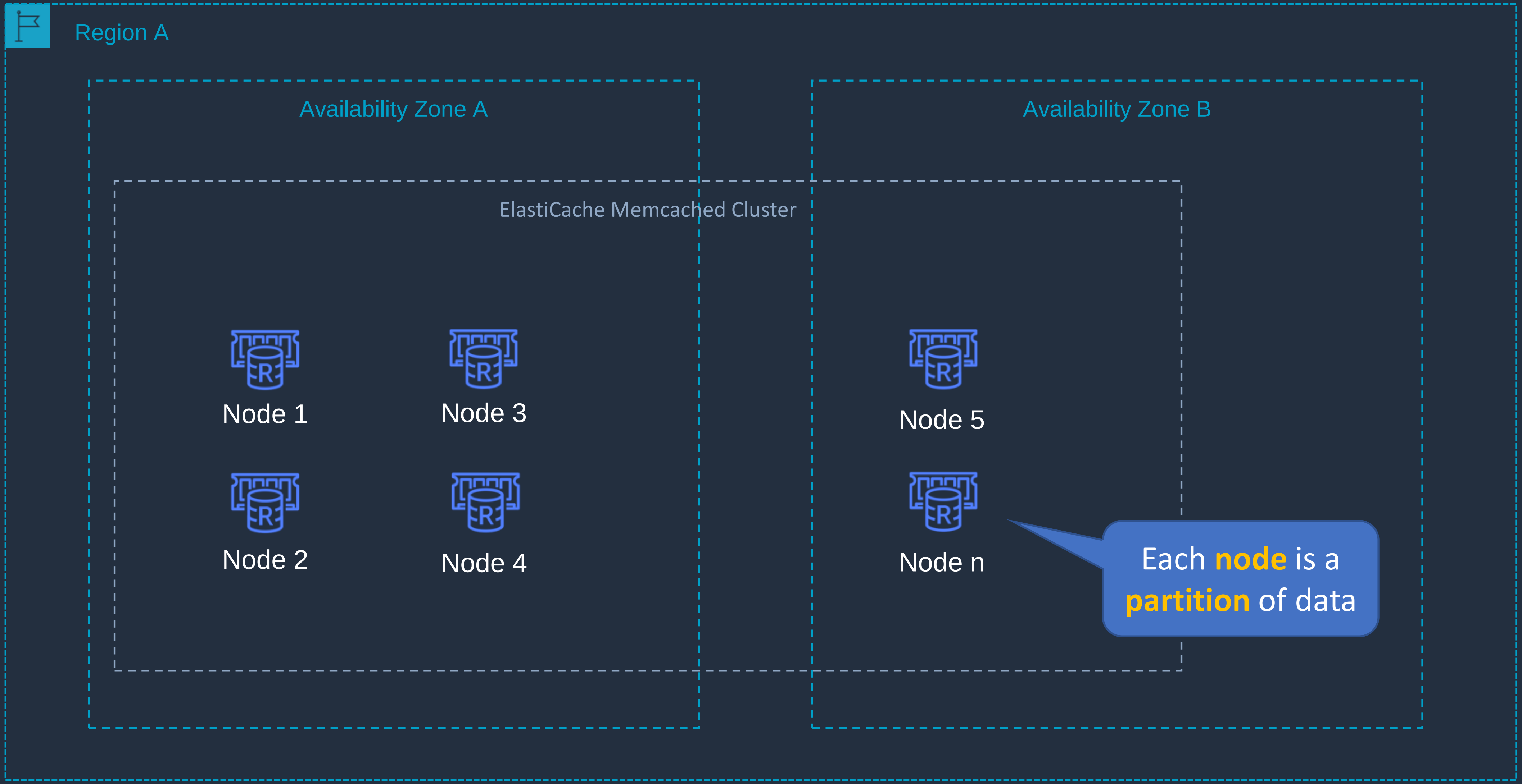
Memcached

- Add nodes to a cluster
- Scale vertically (node type) – must create a **new cluster** manually

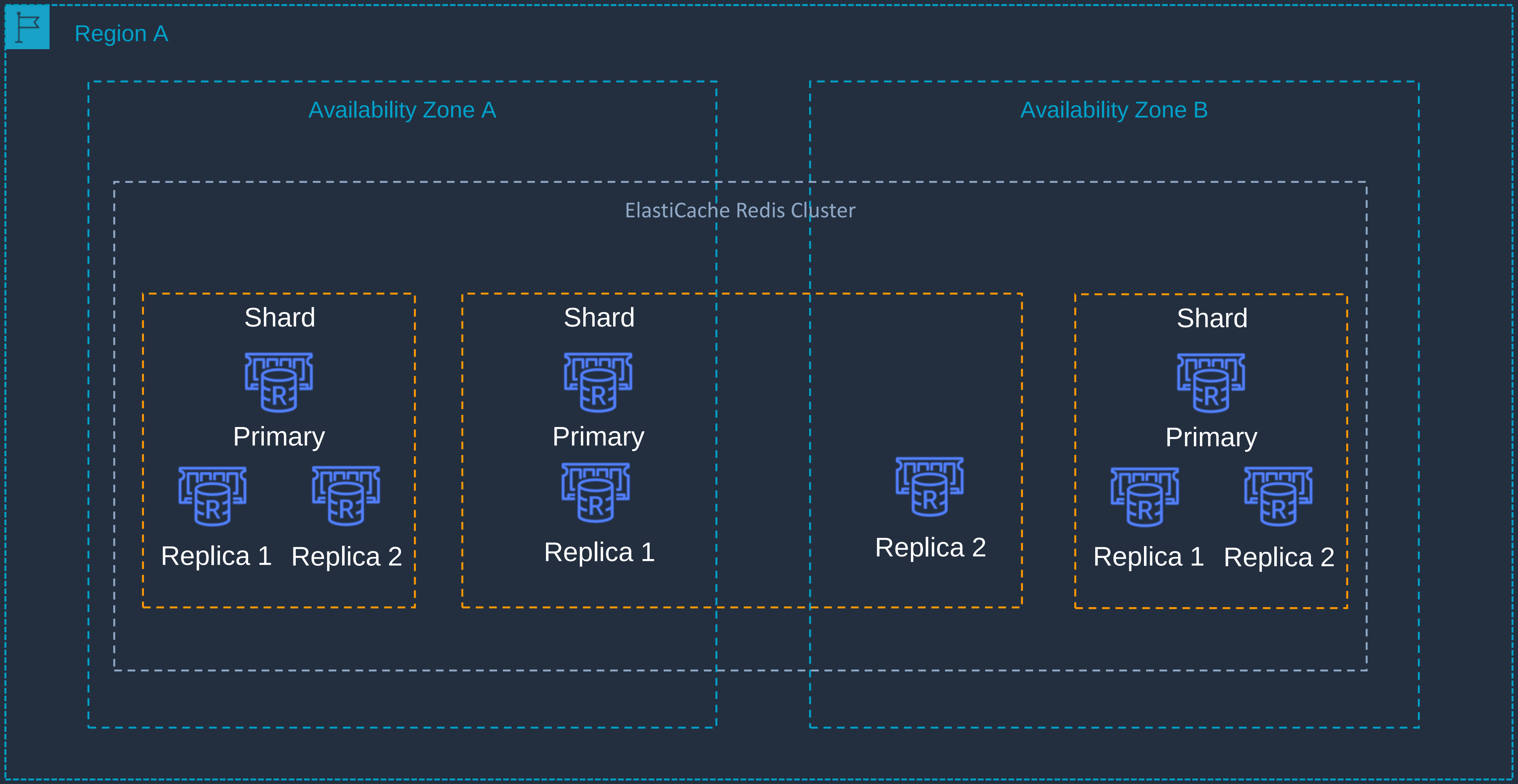
Redis

- Cluster mode **disabled**:
  - Add replica or change node type – creates a new cluster and migrates data
- Cluster mode **enabled**:
  - Online resharding to add or remove shards; vertical scaling to change node type
  - Offline resharding to add or remove shards change node type or upgrade engine (more flexible than online)

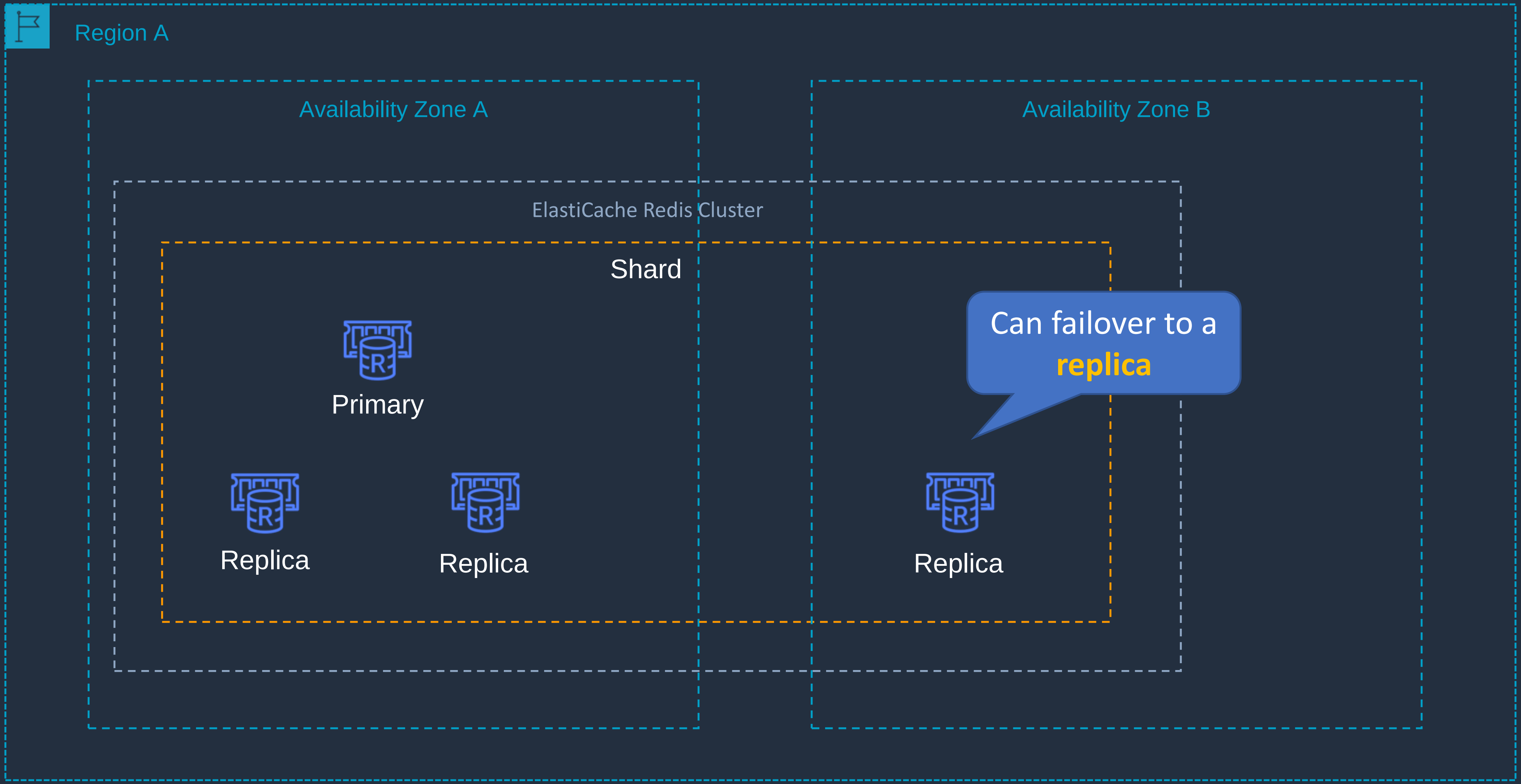
Amazon ElastiCache Memcached



Amazon ElastiCache Redis (Cluster mode enabled)



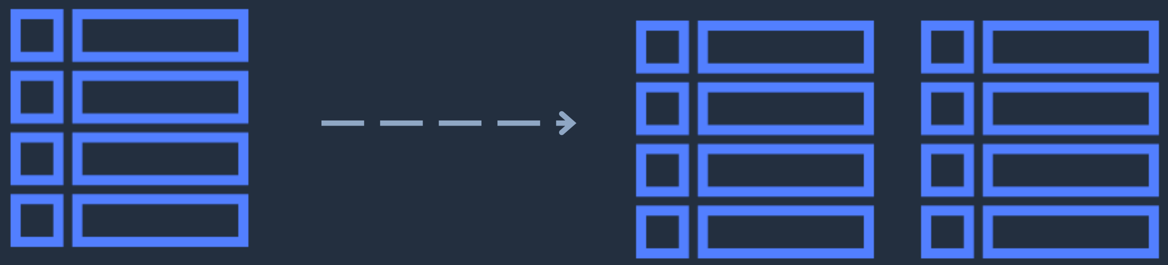
Amazon ElastiCache Redis (Cluster mode disabled)



Amazon DynamoDB Core Knowledge

Amazon DynamoDB Core Knowledge

- Fully managed NoSQL database service
- Key/value store and document store
- It is a non-relational, key-value type of database
- Fully serverless service
- Push button scaling



DynamoDB Table

Amazon DynamoDB Core Knowledge

- DynamoDB is made up of:
  - Tables
  - Items
  - Attributes

userid	orderid	book	price	date
user001	1000092	ISBN100..	9.99	2020.04..
user002	1000102	ISBN100..	24.99	2020.03..
user003	1000168	ISBN2X0..	12.50	2020.04..

DigitalCloud TRAINING

DynamoDB Time to Live (TTL)

- TTL lets you define when items in a table expire so that they can be automatically deleted from the database
- With TTL enabled on a table, you can set a timestamp for deletion on a per-item basis
- No extra cost and does not use WCU / RCU
- Helps reduce storage and manage the table size over time

Amazon DynamoDB Core Knowledge

This is known as a **composite key** as it has a **partition key** + **sort key**

Primary Key

Partition Key

Sort Key

Attributes

clientid	created	sku	category	size	colour	weight
john@example.com	1583975308	SKU-S523	T-Shirt	Small	Red	Light
chris@example.com	1583975613	SKU-J091	Pen		Blue	
chris@example.com	1583975449	SKU-A234	Mug			
sarah@example.com	1583976311	SKU-R873	Chair			4011
jenny@example.com	1583976323	SKU-I019	Plate	30		

Useful when the data structure is **unpredictable**

DigitalCloud TRAINING

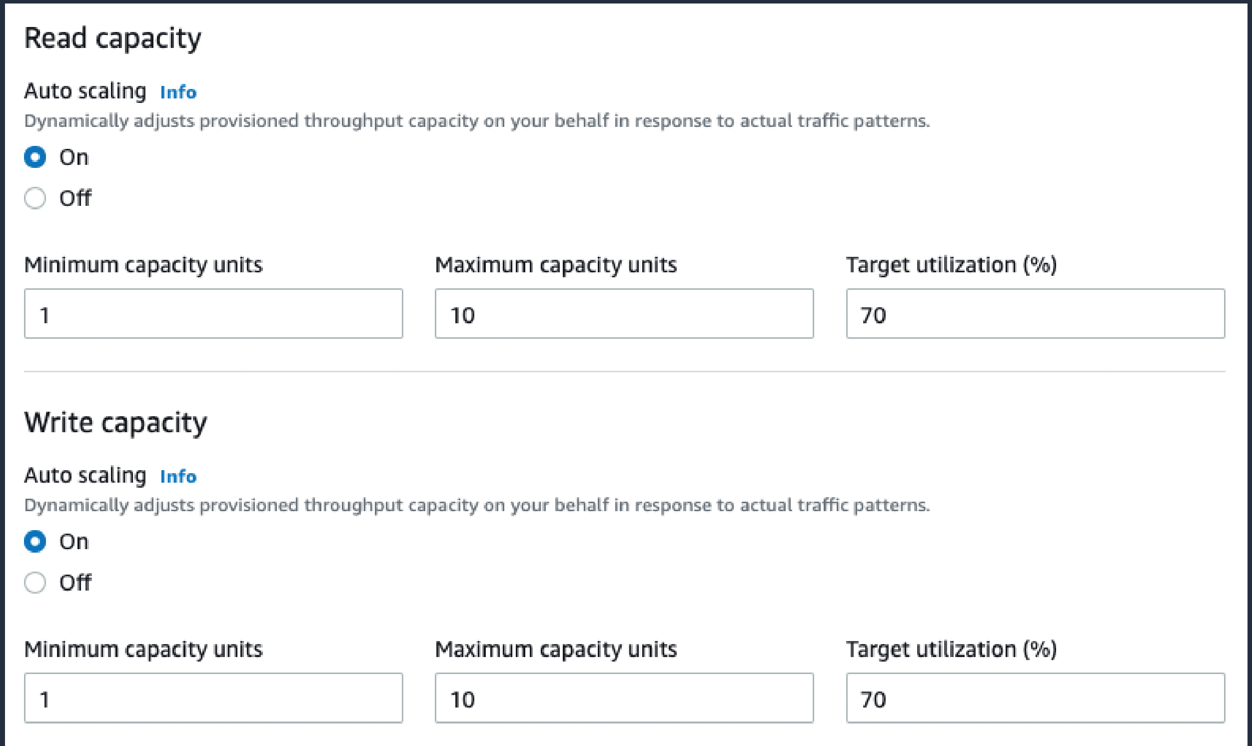
Amazon DynamoDB Core Knowledge

DynamoDB Feature	Benefit
Serverless	Fully managed, fault tolerant, service
Highly available	99.99% availability SLA – 99.999% for Global Tables!
NoSQL type of database with Name / Value structure	Flexible schema, good for when data is not well structured or unpredictable
Horizontal scaling	Seamless scalability to any scale with push button scaling or Auto Scaling
DynamoDB Streams	Captures a time-ordered sequence of item-level modifications in a DynamoDB table and durably stores the information for up to 24 hours. Often used with Lambda and the Kinesis Client Library (KCL)
DynamoDB Accelerator (DAX)	Fully managed in-memory cache for DynamoDB that increases performance (microsecond latency)
Transaction options	Strongly consistent or eventually consistent reads, support for ACID transactions
Backup	Point-in-time recovery down to the second in last 35 days; On-demand backup and restore
Global Tables	Fully managed multi-region, multi-master solution

DynamoDB Capacity Units (RCU/WCU)

DynamoDB Provisioned Capacity

- Provisioned capacity is the default setting
- You specify the reads and write per second
- Can enabled auto scaling for dynamic adjustments
- Capacity is specified using:
  - **Read Capacity Units** (RCUs)
  - **Write Capacity Units** (WCUs)



DynamoDB Provisioned Capacity

Read Capacity Units (RCUs):

- Each API call to read data from your table is a read request
- Read requests can be strongly consistent, eventually consistent, or transactional
- For items up to 4 KB in size, one RCU equals:
  - One strongly consistent read request per second
  - Two eventually consistent read requests per second
  - 0.5 transactional read requests per second
- Items larger than 4 KB require additional RCUs

DynamoDB Provisioned Capacity

Calculating RCUs (examples):

Requirement	RCUs Needed
10 strongly consistent reads per second of 4 KB each	10 * 4 KB / 4 KB = **10 RCU**
10 strongly consistent reads per second of 11 KB each	10 * 12 KB / 4 KB = **30 RCU**
20 eventually consistent reads per second of 12 KB each	(20 / 2) * (12 / 4) – **30 RCU**
36 eventually consistent reads per second of 16 KB each	(36 / 2) * (16 / 4) = **72 RCU**

DynamoDB Provisioned Capacity

Write Capacity Units (WCUs):

- Each API call to write data to your table is a write request
- For items up to 1 KB in size, one WCU can perform:
  - One standard write request per second
  - 0.5 transactional writes requests (one transactional write requires two WCUs)
- Items larger than 1 KB require additional WCUs

DynamoDB Provisioned Capacity

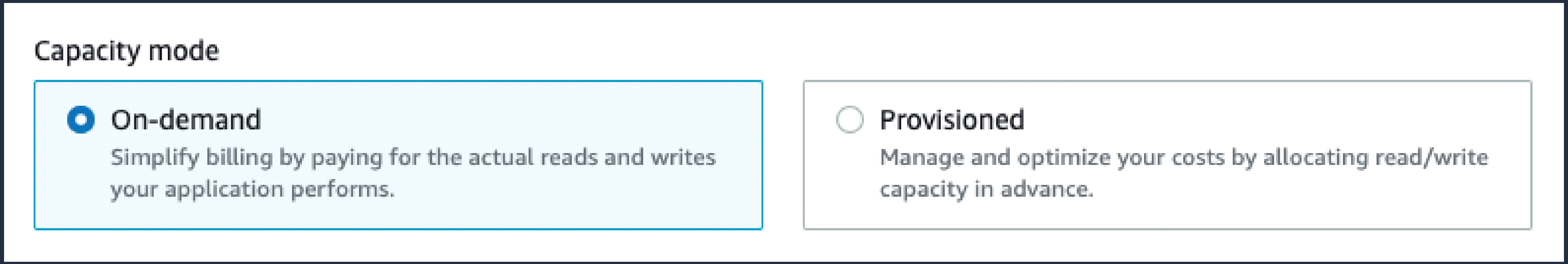
Calculating WCUs (examples):

Requirement	WCUs Needed
10 standard writes per second of 4 KB each	10 * 4 = **40 WCU**
12 standard writes per second of 9.5 KB each (9.5 gets rounded up)	12 * 10 = **120 WCU**
10 transactional writes per second of 4 KB each	10 * 2 * 4 = **80 WCU**
12 transactional writes per second of 9.5 KB each	12 * 2 * 10 = **240 WCU**

DigitalCloud TRAINING

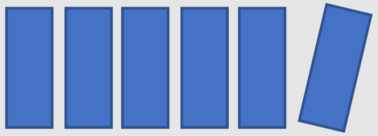
DynamoDB On-Demand Capacity

- With on-demand, you don't need to specify your requirements
- DynamoDB instantly scales up and down based on the activity of your application
- Great for unpredictable / spikey workloads or new workloads that aren't well understood
- You pay for what you use (pay per request)

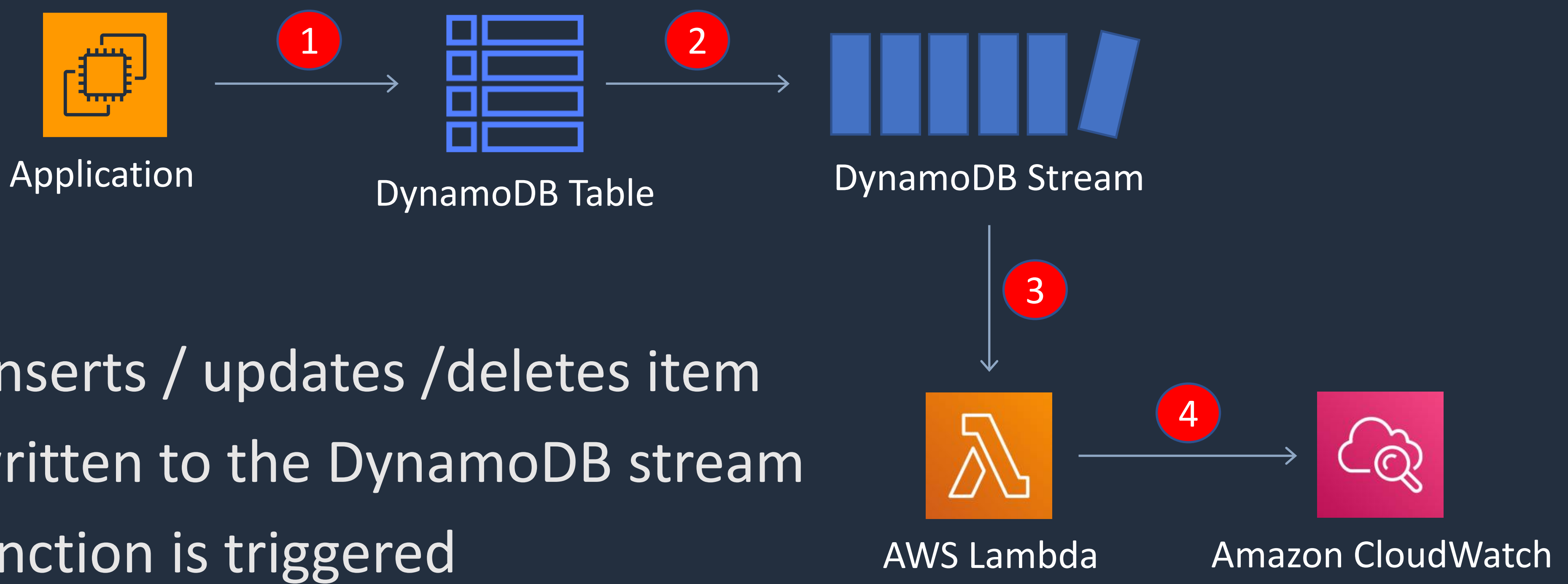


Practice Creating DynamoDB Tables

DynamoDB Streams



DynamoDB Streams



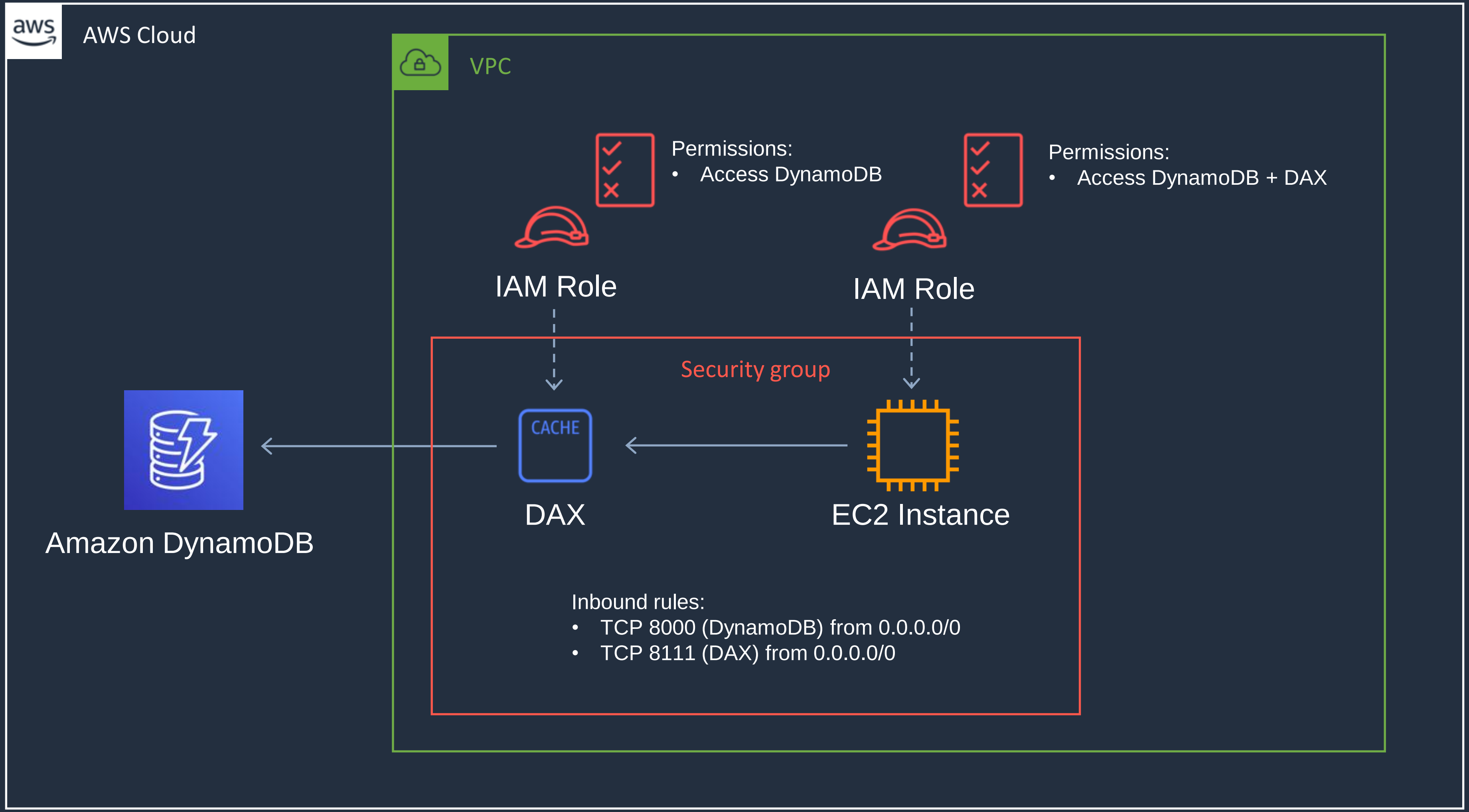
1. Application inserts / updates /deletes item
2. A record is written to the DynamoDB stream
3. A Lambda function is triggered
4. The Lambda function writes to CloudWatch Logs

DynamoDB Streams

- Captures a **time-ordered** sequence of **item-level** modifications in any DynamoDB table and stores this information in a log for up to **24 hours**
- Can configure the information that is written to the stream:
  - **KEYS_ONLY** — Only the key attributes of the modified item
  - **NEW_IMAGE** — The entire item, as it appears after it was modified
  - **OLD_IMAGE** — The entire item, as it appeared before it was modified
  - **NEW_AND_OLD_IMAGES** — Both the new and the old images of the item

DynamoDB Accelerator (DAX)

DynamoDB Accelerator (DAX)



DynamoDB Accelerator (DAX)

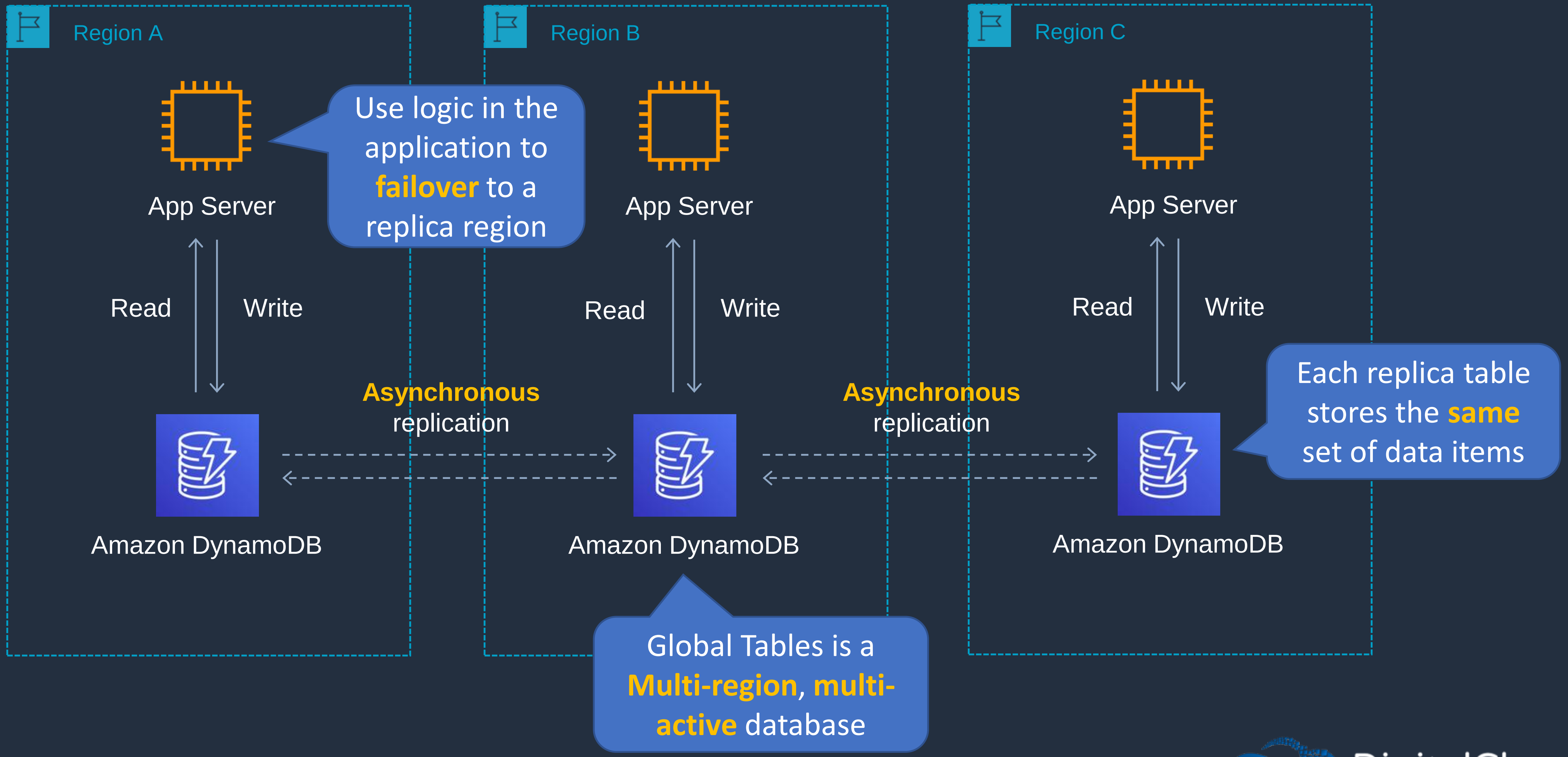
- DAX is a fully managed, highly available, **in-memory** cache for DynamoDB
- Improves performance from milliseconds to microseconds
- DAX is used to improve **READ** performance (not writes)
- You do not need to modify application logic, since DAX is compatible with existing DynamoDB API calls

DAX vs ElastiCache

- DAX is **optimized** for DynamoDB
- DAX does not support lazy loading (uses write-through caching)
- With ElastiCache you have more **management overhead** (e.g. invalidation)
- With ElastiCache you need to **modify application code** to point to cache
- ElastiCache supports more datastores

DynamoDB Global Tables

DynamoDB Global Tables



Enable Global Table

Other Database Services

Amazon DocumentDB

- Amazon DocumentDB provides MongoDB compatibility
- It is a database service that is purpose-built for JSON data management at scale

JSON

```json
[
    {
        "year" : 2013,
        "title" : "Turn It Down, Or Else!",
        "info" : {
            "directors" : [ "Alice Smith", "Bob Jones"],
            "release_date" : "2013-01-18T00:00:00Z",
            "rating" : 6.2,
            "genres" : ["Comedy", "Drama"],
            "image_url" : "http://ia.media-imdb.com/images/N/O9ERWAU7FS797AJ7LU8HN09AMUP908RLlo5JF90EWR7LJKQ7@@._V1_SX400_.jpg",
            "plot" : "A rock band plays their music at high volumes, annoying the neighbors.",
            "actors" : ["David Matthewman", "Jonathan G. Neff"]
        }
    },
    {
        "year": 2015,
        "title": "The Big New Movie",
        "info": {
            "plot": "Nothing happens at all.",
            "rating": 0
        }
    }
]
```

DigitalCloud TRAINING

Amazon DocumentDB

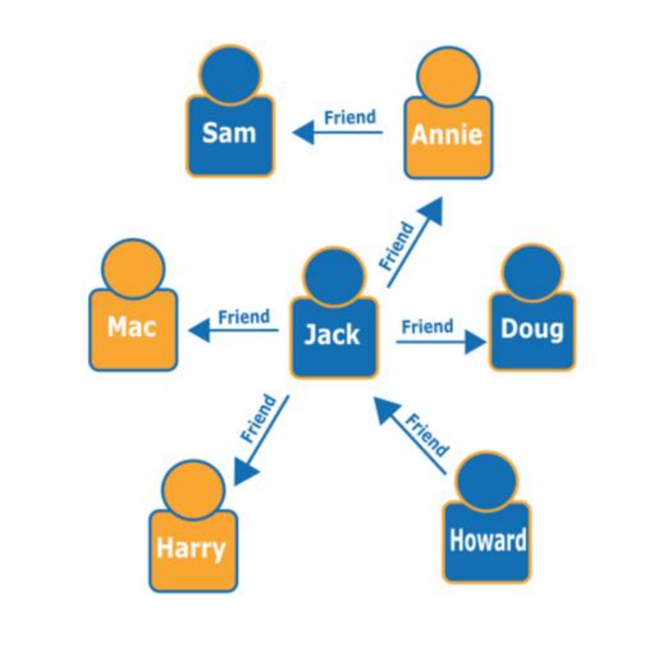
- Fully managed service
- Storage scales automatically up to 64 TB without any impact to your application
- Supports millions of requests per second with up to 15 low latency read replicas
- Designed for 99.99% availability and replicates six copies of your data across three AZs
- Can migrate from MongoDB using the AWS Database Migration Service (AWS DMS)

Amazon Keyspaces (for Apache Cassandra)

- A scalable, highly available, and managed Apache Cassandra–compatible database service
- Keyspaces enables you to use the Cassandra Query Language (CQL) API code
- Keyspaces is serverless and fully managed
- Scales automatically in response to application traffic
- Can serve thousands of requests per second with virtually unlimited throughput and storage
- Consistent, single-digit-millisecond response times at any scale
- 99.99% availability SLA within an AWS Region

Amazon Neptune

- Fully managed graph database service
- Build and run identity, knowledge, fraud graph, and other applications
- Deploy high performance graph applications using popular open-source APIs including:
  - Gremlin
  - openCypher
  - SPARQL
- Offers greater than 99.99% availability
- Storage is fault-tolerant and self-healing
- DB volumes grow in increments of 10 GB up to a maximum of 64 TB
- Create up to 15 database read replicas



Amazon Quantum Ledger Database

- Amazon QLDB is a fully managed ledger database that provides a transparent, immutable, and cryptographically verifiable transaction log
- Amazon QLDB has a built-in immutable journal that stores an accurate and sequenced entry of every data change
- The journal is append-only, meaning that data can only be added to a journal, and it cannot be overwritten or deleted
- Amazon QLDB uses cryptography to create a concise summary of your change history
- Generated using a cryptographic hash function (SHA-256)
- Serverless and offers automatic scalability

Architecture Patterns - AWS Databases

Architecture Patterns - AWS Databases

Requirement	Solution
Relational database running on MySQL must be migrated to AWS and must be highly available	Use Amazon RDS MySQL and configure a Multi-AZ standby node for HA
Amazon RDS DB has high query traffic that is causing performance degradation	Create a Read Replica and configure the application to use the reader endpoint for database queries
Amazon RDS DB is approaching its storage capacity limits and/or is suffering from high write latency	Scale up the DB instance to an instance type that has more storage / CPU

Architecture Patterns - AWS Databases

Requirement	Solution
Amazon RDS database is unencrypted and a cross-Region read replica must be created with encryption	Encrypt a snapshot of the main DB and create a new encrypted DB instance from the encrypted snapshot. Create a encrypted cross-Region read replica
Amazon Aurora DB deployed and requires a cross-Region replica	Deploy an Aurora MySQL Replica in the second Region
Amazon Aurora DB deployed and requires a read replica in the same Region with minimal synchronization latency	Deploy an Aurora Replica in the Region in a different Availability Zone

Architecture Patterns - AWS Databases

Requirement	Solution
Aurora deployed and app in another Region requires read-only access with low latency – synchronization latency must also be minimized	Use Aurora Global Database and configure the app in the second Region to use the reader endpoint
Application and DB migrated to Aurora and requires the ability to write to the DB across multiple nodes	Use Aurora Multi-Master for an in-Region multi-master database
Application requires a session-state data store that provides low-latency	Use either Amazon ElastiCache or DynamoDB

Architecture Patterns - AWS Databases

Requirement	Solution
Multi-threaded in-memory datastore required for unstructured data	Use Amazon ElastiCache Memcached
In-memory datastore required that offers microsecond performance for unstructured data	Use Amazon DynamoDB DAX (DAX)
In-memory datastore required that supports data persistence and high availability	Use Amazon ElastiCache Redis

Architecture Patterns - AWS Databases

Requirement	Solution
Serverless database required that supports No-SQL key-value store workload	Use Amazon DynamoDB
Serverless database required that supports MySQL or PostgreSQL	Use Amazon Aurora Serverless
Relational database required for a workload with an unknown usage pattern (usage expected to be low and variable)	Use Amazon Aurora Serverless

Architecture Patterns - AWS Databases

Requirement

Application requires a key-value database that can be written to from multiple AWS Regions

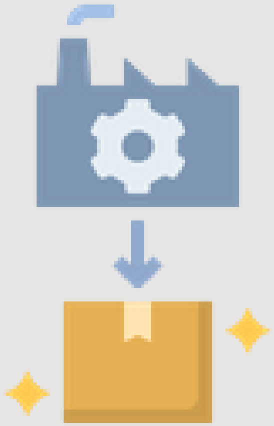
Solution

Use DynamoDB Global Tables

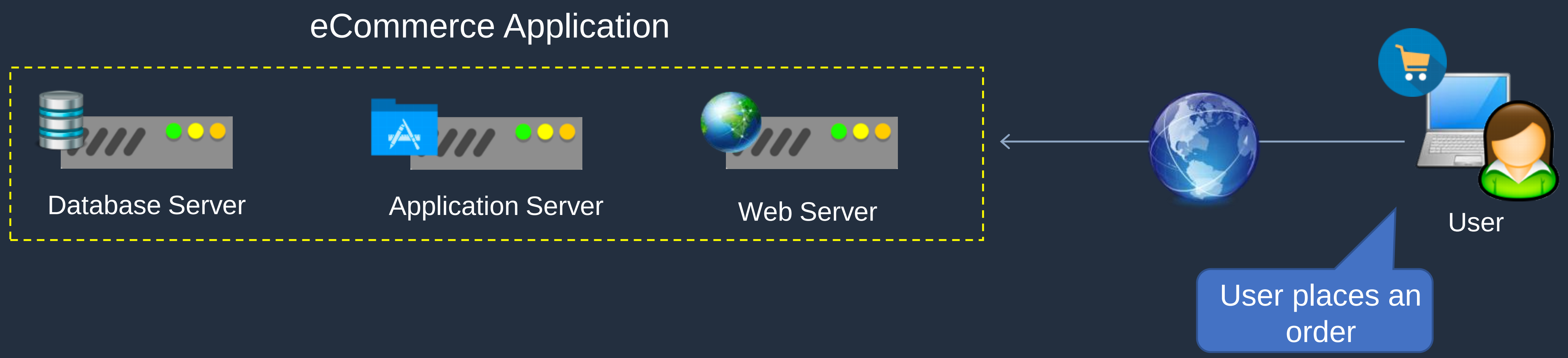
SECTION 11

Serverless Applications

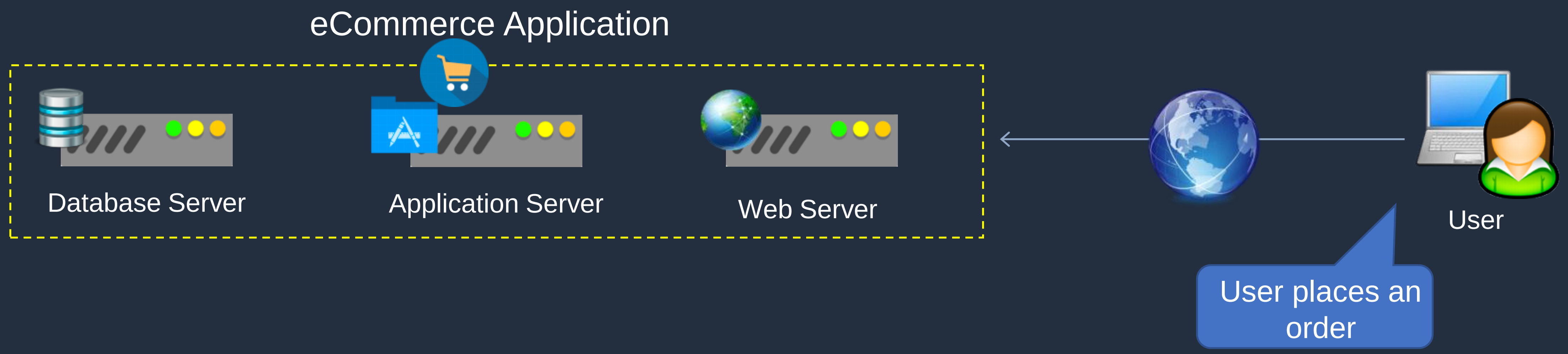
Event-Driven Architectures



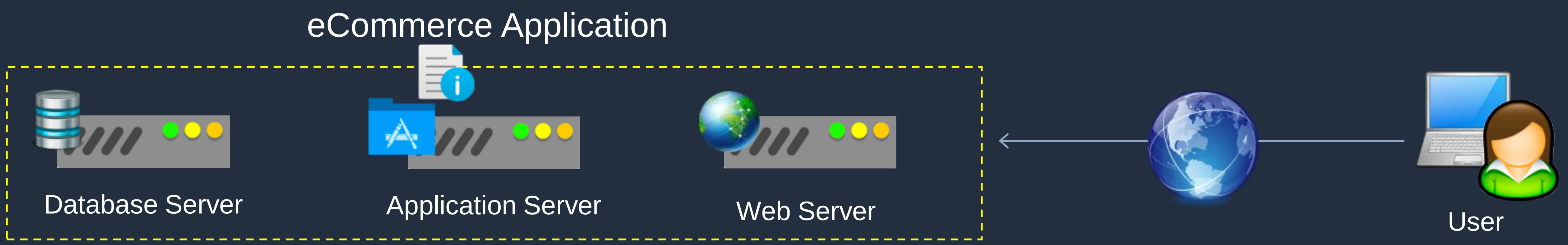
Event-Driven Architectures



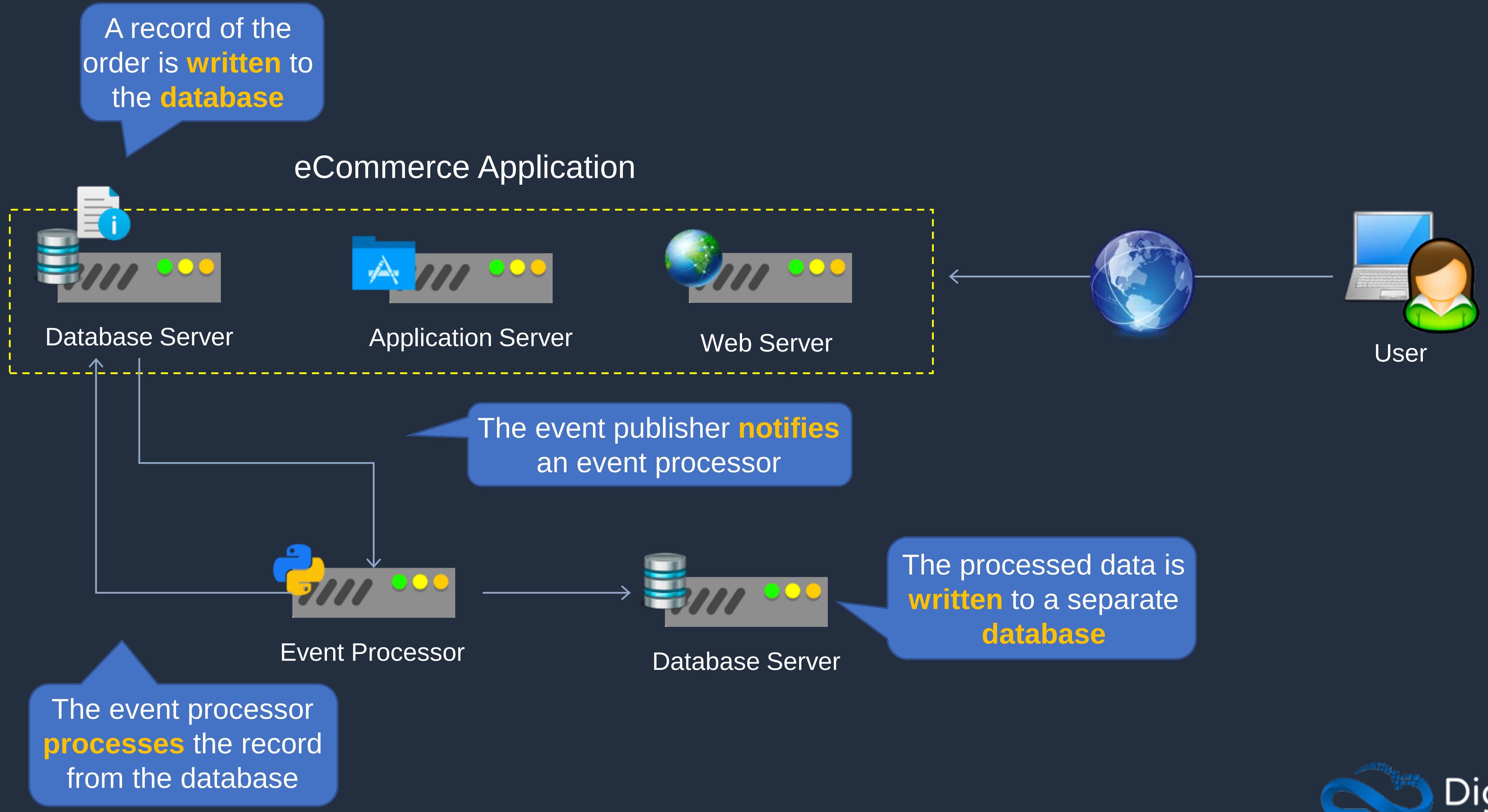
Event-Driven Architectures



Event-Driven Architectures

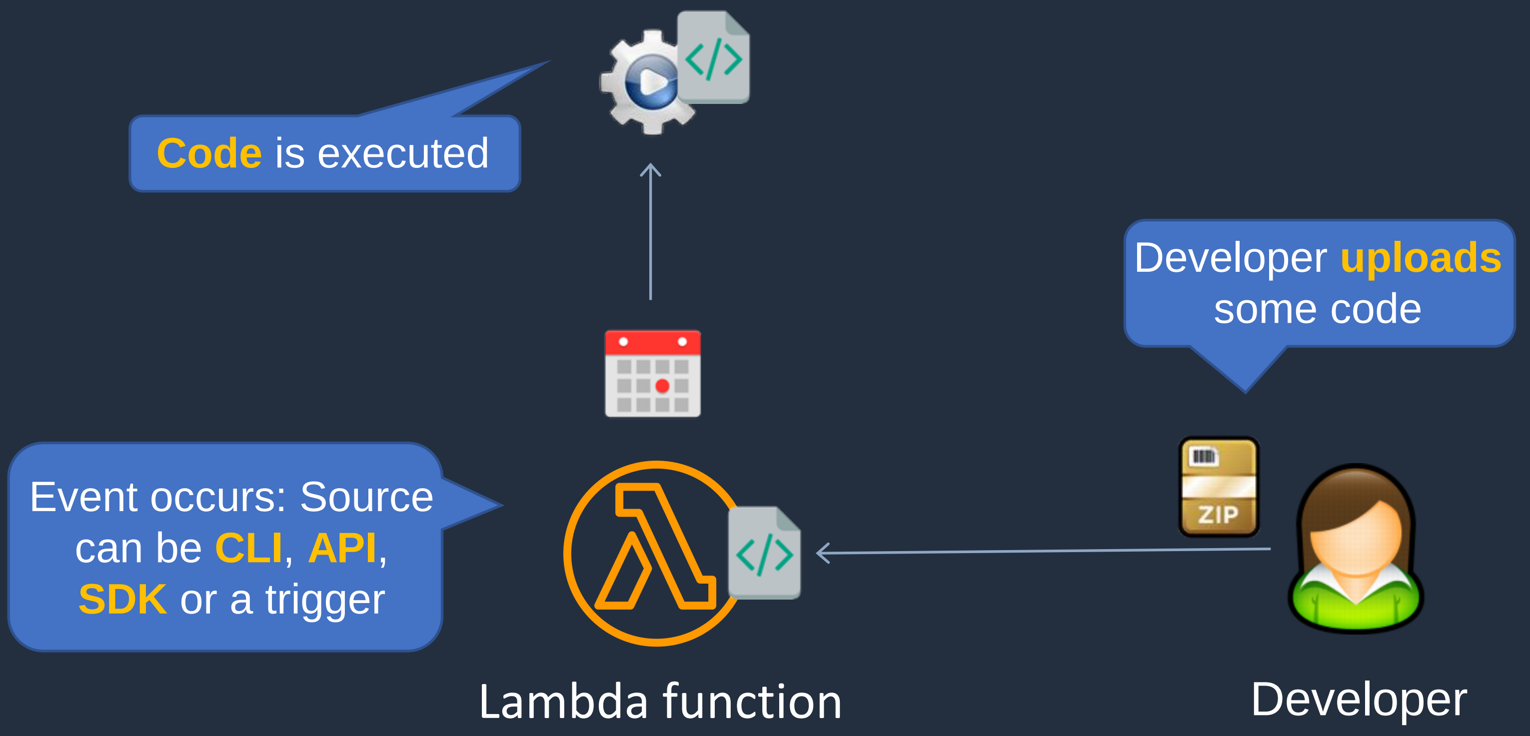


Event-Driven Architectures



AWS Lambda Invocations and Concurrency

AWS Lambda Invocations



Lambda Function Invocations

Synchronous:
- CLI, SDK, API Gateway
- Result returned immediately
- Error handling happens client side (retries, exponential backoff etc.)

Asynchronous:
- S3, SNS, CloudWatch Events etc.
- Lambda retries up to 3 times
- Processing must be idempotent (due to retries)

Event source mapping:
- SQS, Kinesis Data Streams, DynamoDB Streams
- Lambda does the polling (polls the source)
- Records are processed in order (except for SQS standard)

SQS can also **trigger** Lambda

Lambda Function Concurrency

Additional functions are initialized up to the **burst** or **account limit**

If the concurrency limit is **exceeded** throttling occurs with error "Rate exceeded" and 429 "TooManyRequestsException"

Burst **concurrency** quotas:

- 3000 – US West (Oregon), US East (N. Virginia), Europe (Ireland)
- 1000 – Asia Pacific (Tokyo), Europe (Frankfurt), US East (Ohio)
- 500 – Other Regions

Lambda Function Concurrency

- Throttling can result in the error: "Rate exceeded" and 429 "TooManyRequestsException"
- If the above error occurs, verify if you see throttling messages in Amazon CloudWatch Logs but no corresponding data points in the Lambda Throttles metrics
- If there are no Lambda Throttles metrics, the throttling is happening on API calls in your Lambda function code
- For asynchronous invocations Lambda retries up to 3 times then goes to a Dead Letter Queue
- DLQ can be SNS topic or SQS queue

Lambda Function Concurrency

Methods to resolve throttling include:

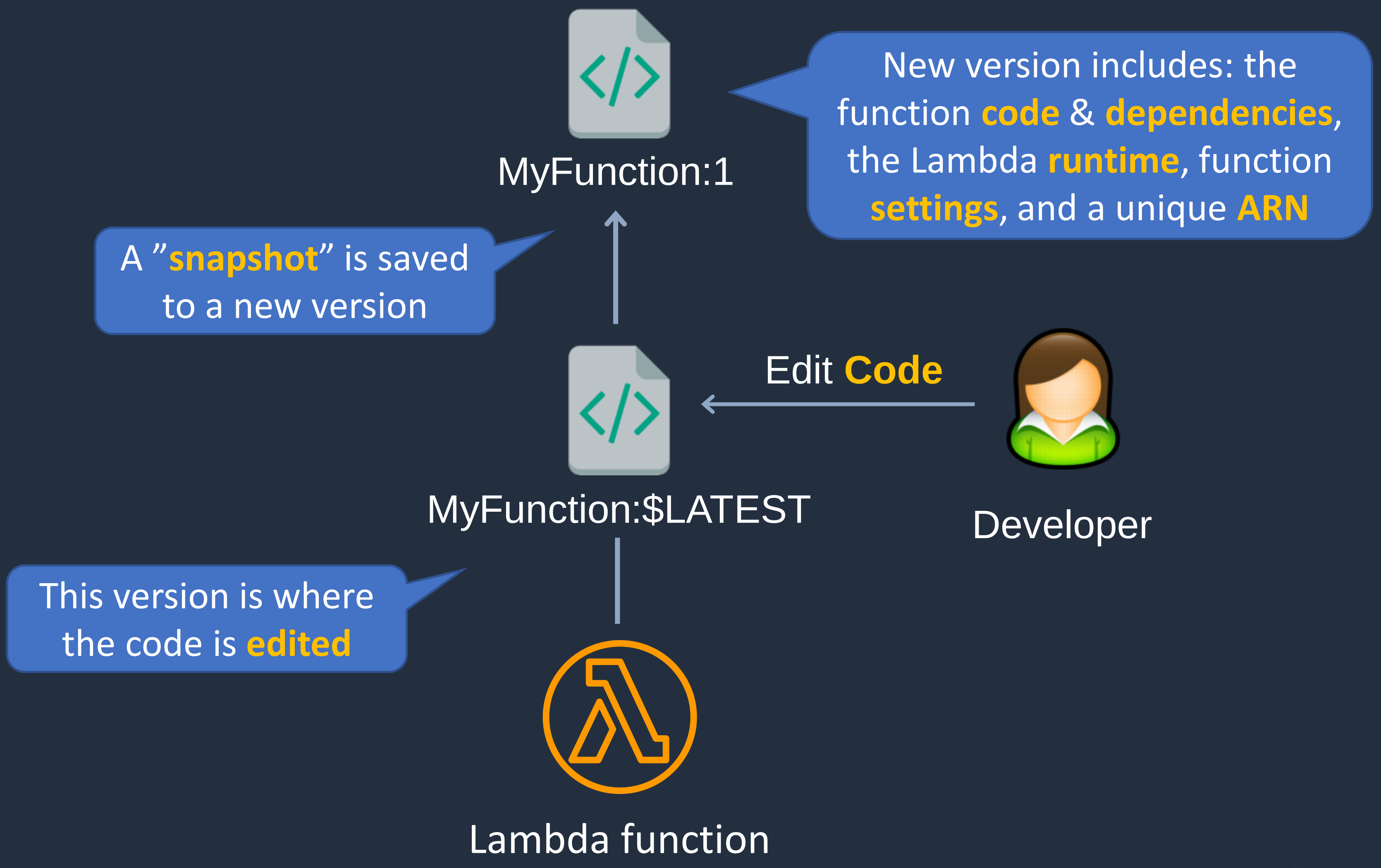
- Configure reserved concurrency
- Use exponential backoff in your application code

Concurrency metrics:

- ConcurrentExecutions
- UnreservedConcurrentExecutions
- ProvisionedConcurrentExecutions
- ProvisionedConcurrencyInvocations
- ProvisionedConcurrencySpilloverInvocations
- ProvisionedConcurrencyUtilization

Lambda Versions and Aliases

Lambda Versions



Lambda Versions

- You work on $LATEST which is the latest version of the code - this is mutable (changeable)

MyFunction:$LATEST

- When you're ready to publish a Lambda function you create a version (these are numbered)

MyFunction:1

- Numbered versions are assigned a number starting with 1 and subsequent versions are incremented by 1

MyFunction:2

- Versions are immutable (code cannot be edited)

Lambda Versions

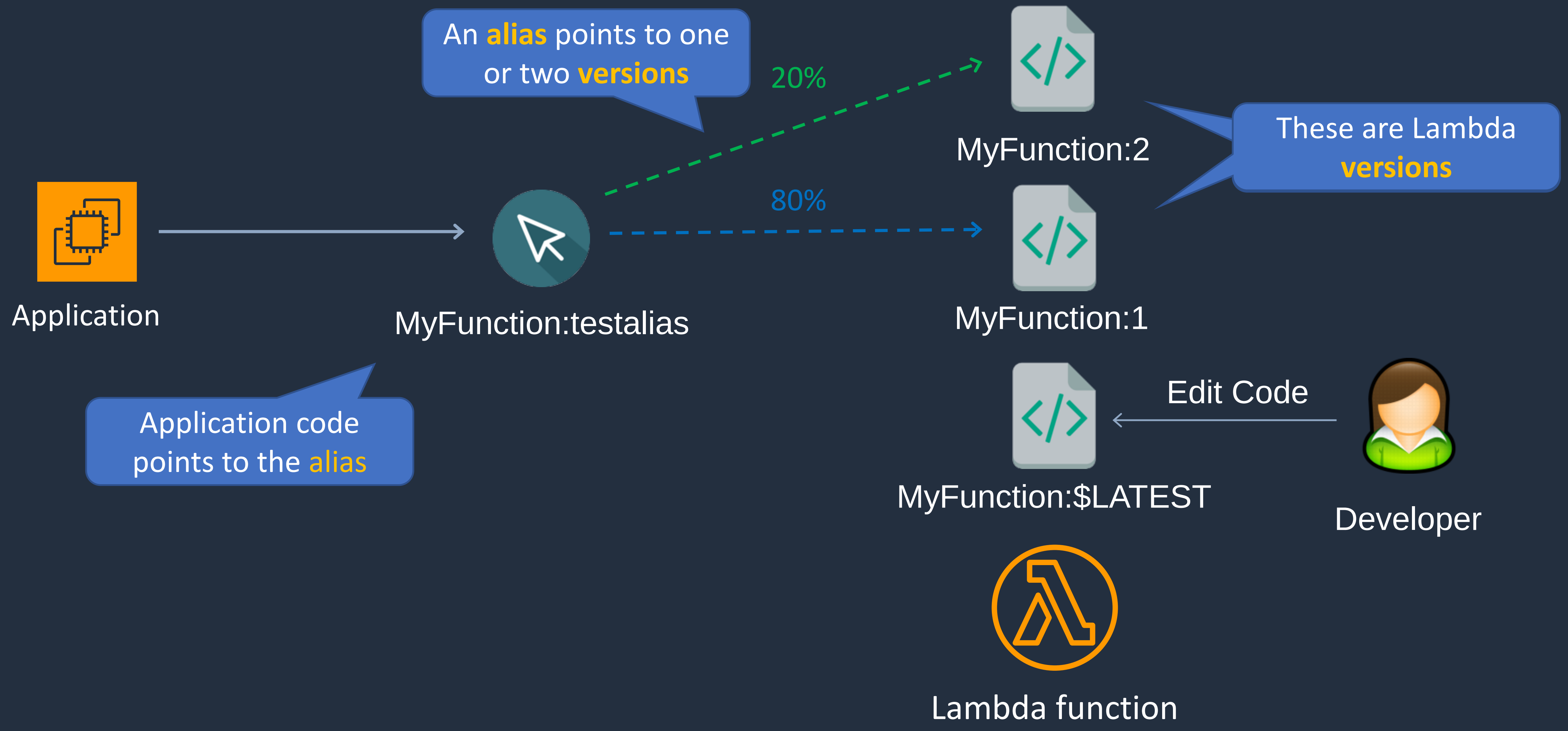
- Each version has its own ARN

ARN - arn:aws:lambda:ap-southeast-2:515148227241:function:MyFunction:$LATEST

ARN - arn:aws:lambda:ap-southeast-2:515148227241:function:MyFunction:1

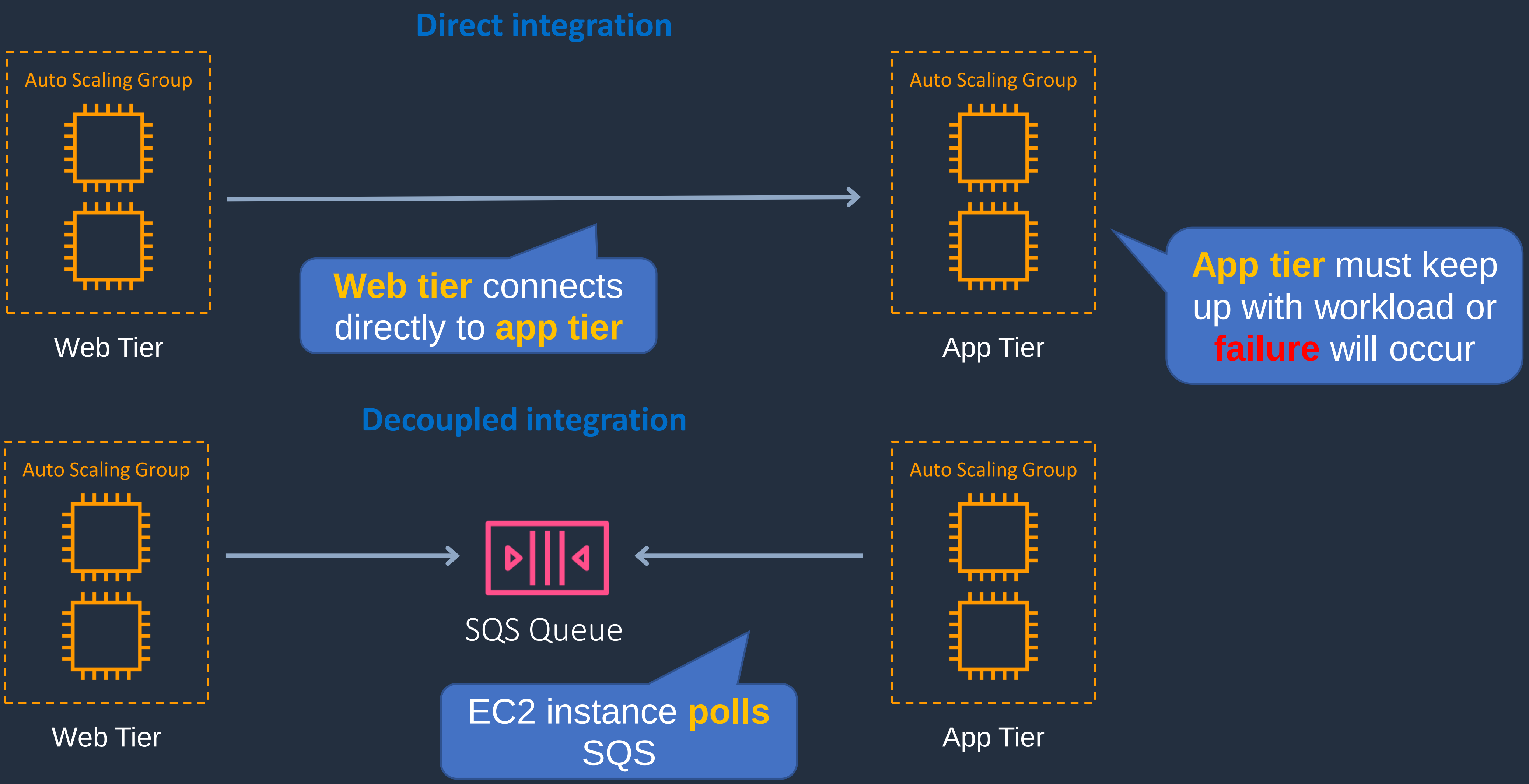
- Because different versions have unique ARNs this allows you to effectively manage them for different environments like Production, Staging or Development
- A qualified ARN has a version suffix
- An unqualified ARN does not have a version suffix
- You cannot create an alias from an unqualified ARN

Lambda Aliases

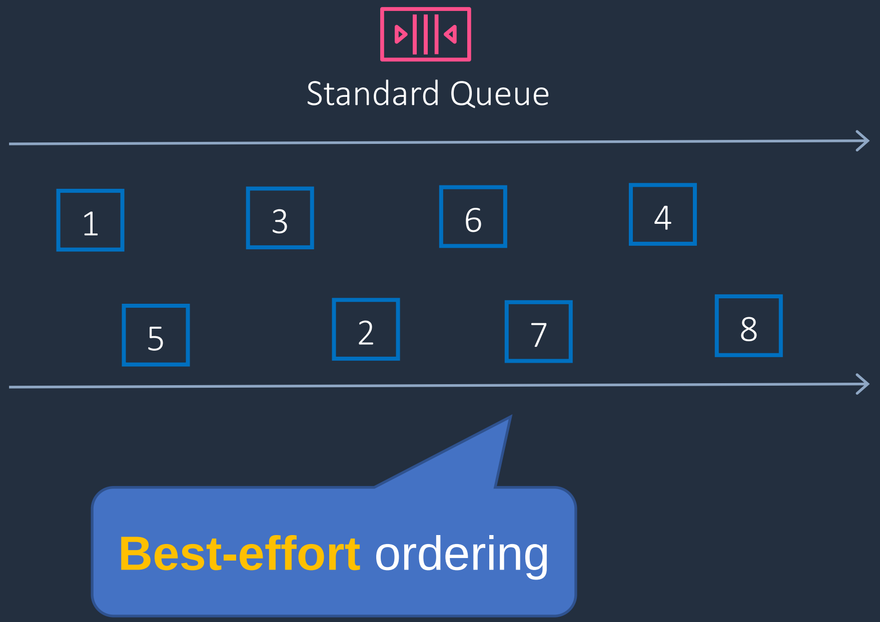


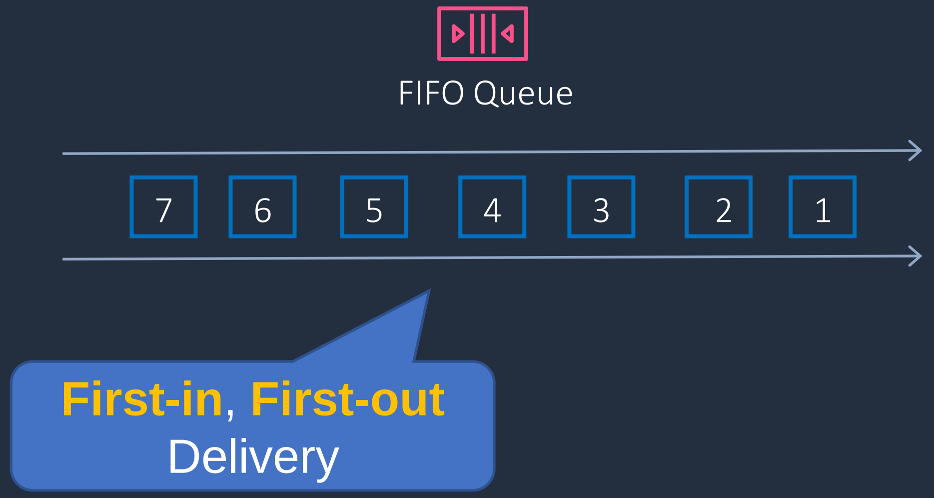
Advanced Amazon SQS

Decoupling with SQS Queues



SQS Queue Types





SQS Queue Types

Standard Queue	FIFO Queue
Unlimited Throughput: Standard queues support a nearly unlimited number of transactions per second (TPS) per API action.	**High Throughput:** FIFO queues support up to 300 messages per second (300 send, receive, or delete operations per second). When you batch 10 messages per operation (maximum), FIFO queues can support up to 3,000 messages per second
Best-Effort Ordering: Occasionally, messages might be delivered in an order different from which they were sent	**First-In-First-out Delivery:** The order in which messages are sent and received is strictly preserved
At-Least-Once Delivery: A message is delivered at least once, but occasionally more than one copy of a message is delivered	**Exactly-Once Processing:** A message is delivered once and remains available until a consumer processes and deletes it. Duplicates are not introduced into the queue

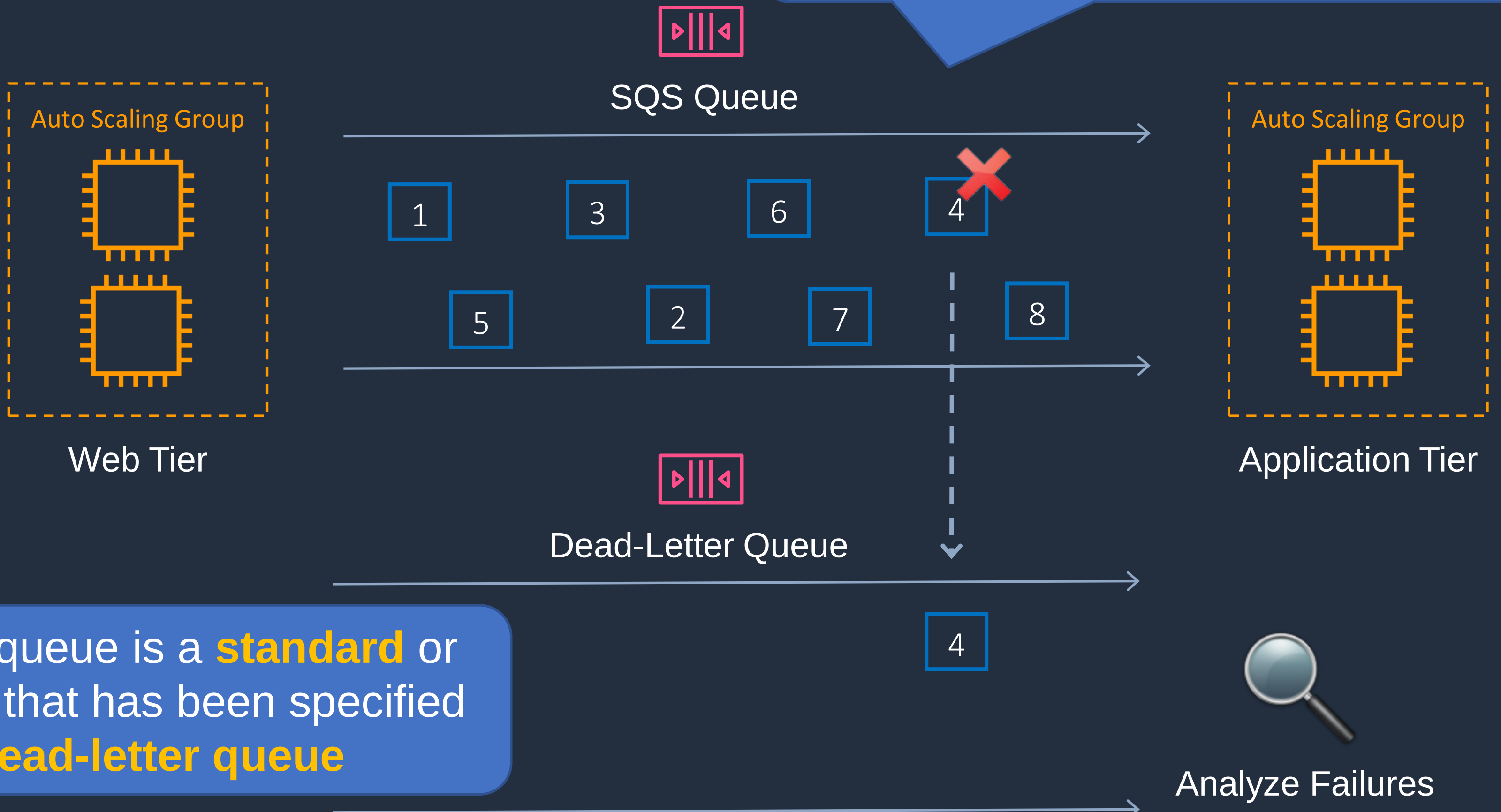
DigitalCloud TRAINING

SQS Queue Types

- FIFO queues require the **Message Group ID** and **Message Deduplication ID** parameters to be added to messages
- **Message Group ID**:
  - The tag that specifies that a message belongs to a specific message group Messages that belong to the same message group are guaranteed to be processed in a FIFO manner
- **Message Deduplication ID**:
  - The token used for deduplication of messages within the deduplication interval

SQS – Dead Letter Queue

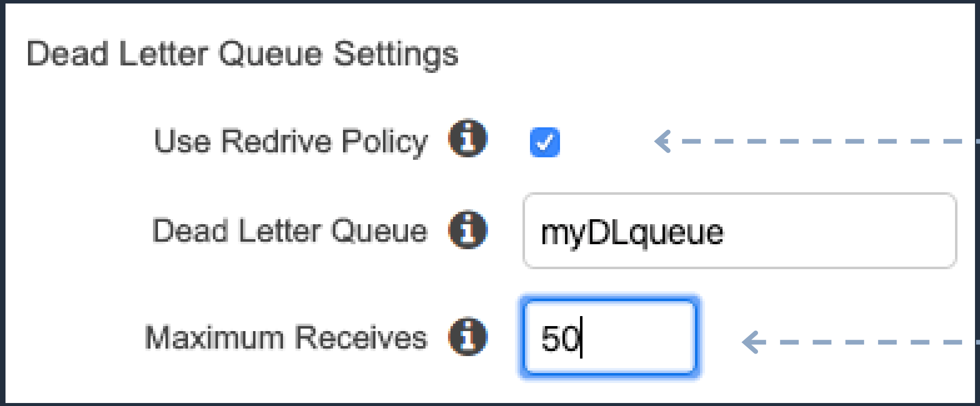
Message not processed successfully (**ReceiveCount** exceeds **maxReceiveCount** for queue)



Dead-letter queue is a **standard** or **FIFO** queue that has been specified as a **dead-letter queue**

SQS – Dead Letter Queue

- The main task of a dead-letter queue is handling message failure
- A dead-letter queue lets you set aside and isolate messages that can't be processed correctly to determine why their processing didn't succeed
- It is not a queue type, it is a **standard** or **FIFO** queue that has been specified as a dead-letter queue in the configuration of **another** standard or FIFO queue

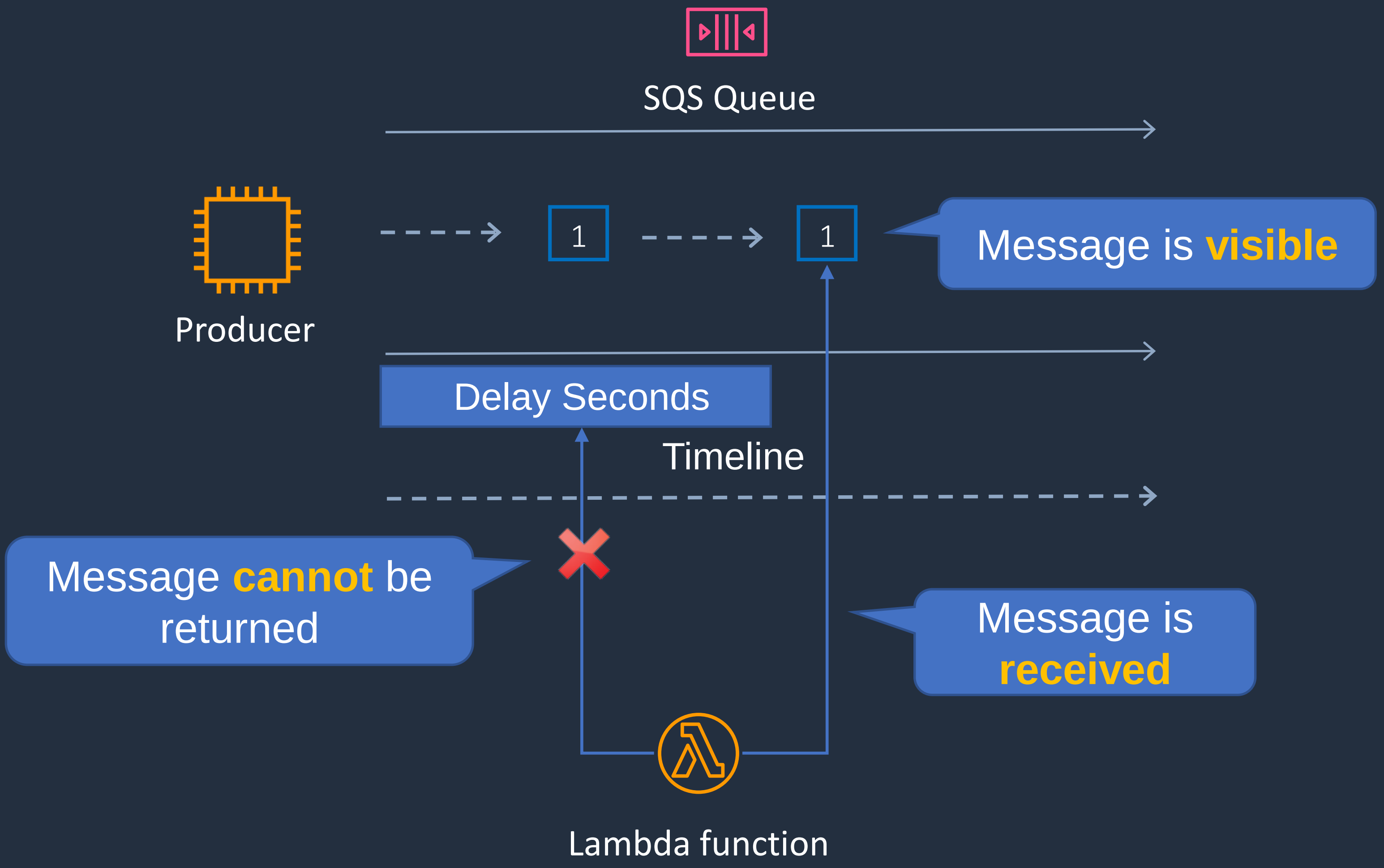


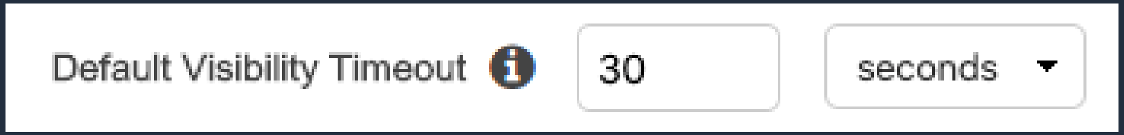
Enable Redrive Policy

Specify the queue to use as a dead-letter queue

Specify the maximum receives before a message is sent to the dead-letter queue

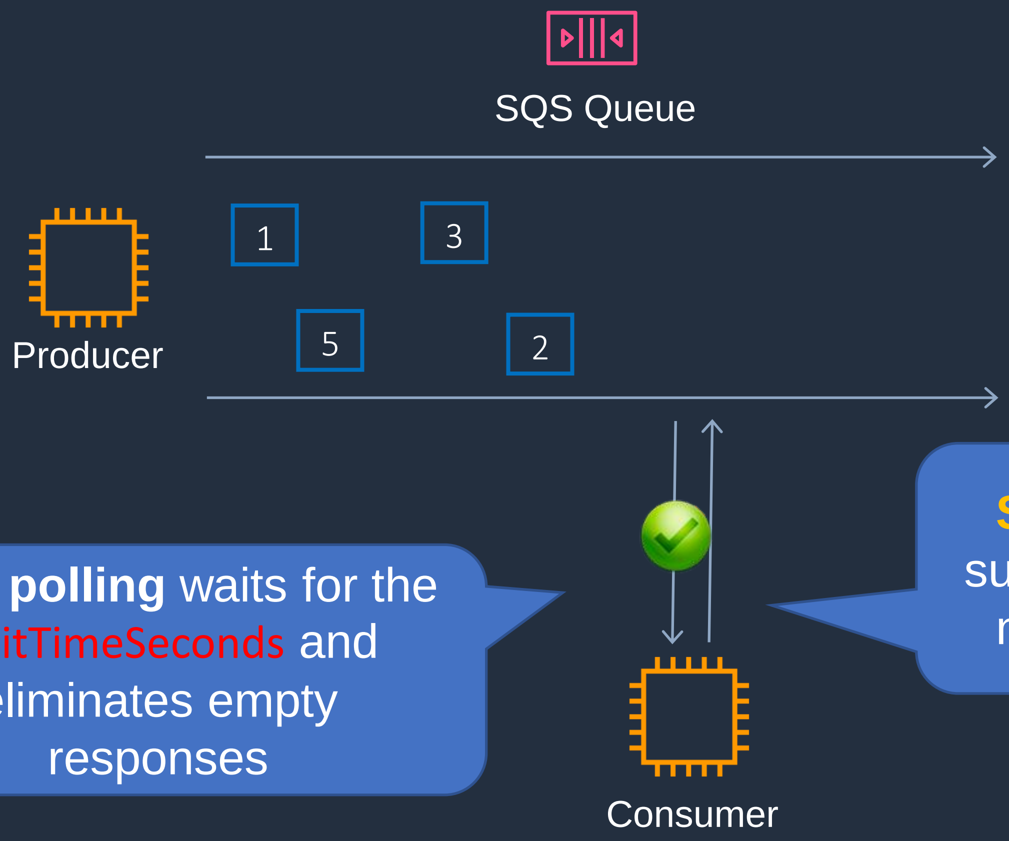
SQS – Delay Queue





Default is **30 seconds**, max is **15 minutes**

SQS Long Polling vs Short Polling



Long polling waits for the WaitTimeSeconds and eliminates empty responses

Short polling checks a subset of servers and may not return all messages

SQS Long Polling vs Short Polling

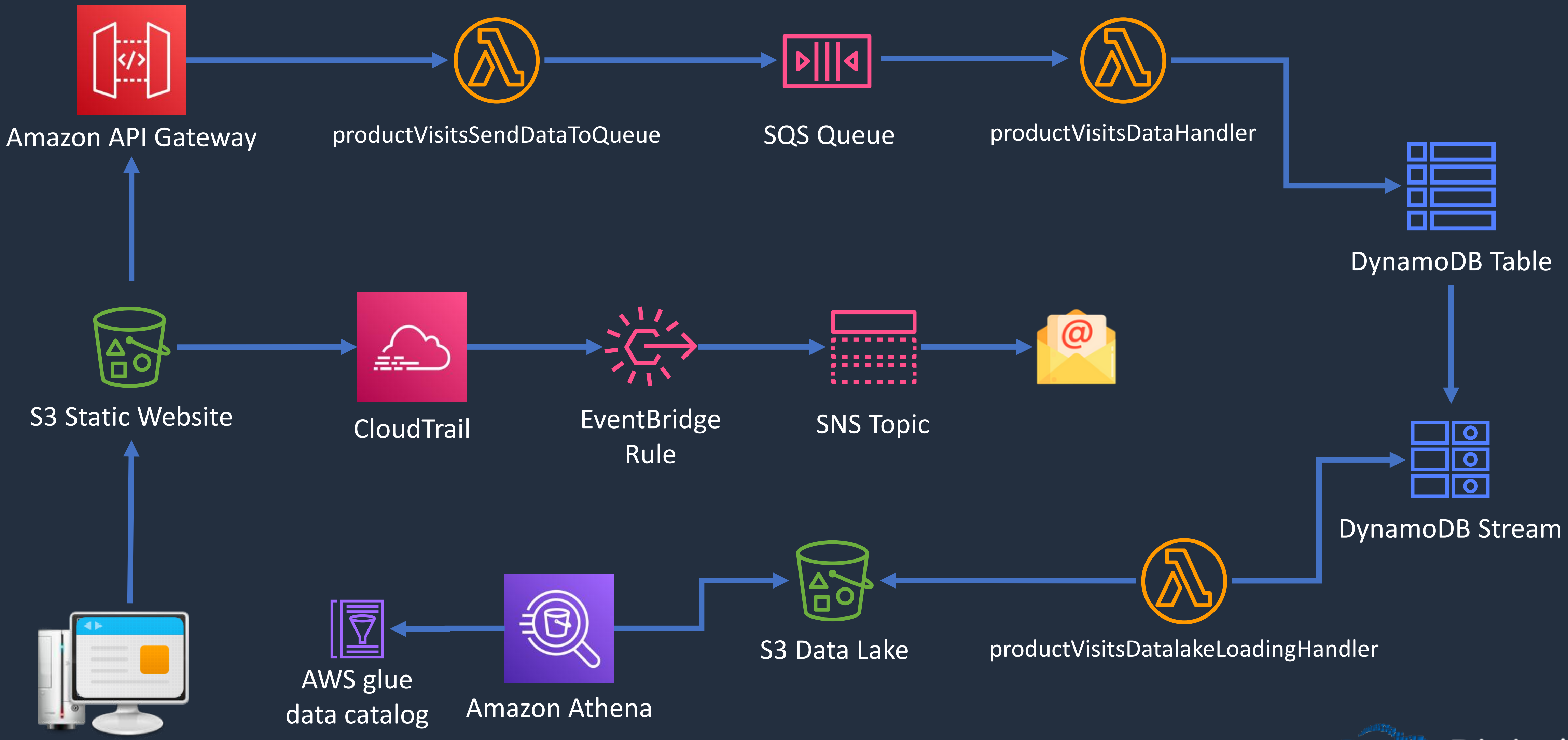
- SQS **Long polling** is a way to retrieve messages from SQS queues – waits for messages to arrive
- SQS **Short polling** returns immediately (even if the message queue is empty)
- SQS Long polling can lower costs
- SQS Long polling can be enabled at the queue level or at the API level using WaitTimeSeconds
- SQS Long polling is in effect when the Receive Message Wait Time is a value greater than 0 seconds and up to 20 seconds

The maximum amount of time that a long polling receive call will wait for a message to become available before returning an empty response.

DigitalCloud TRAINING

Serverless App Architecture for HOL

Serverless App Architecture

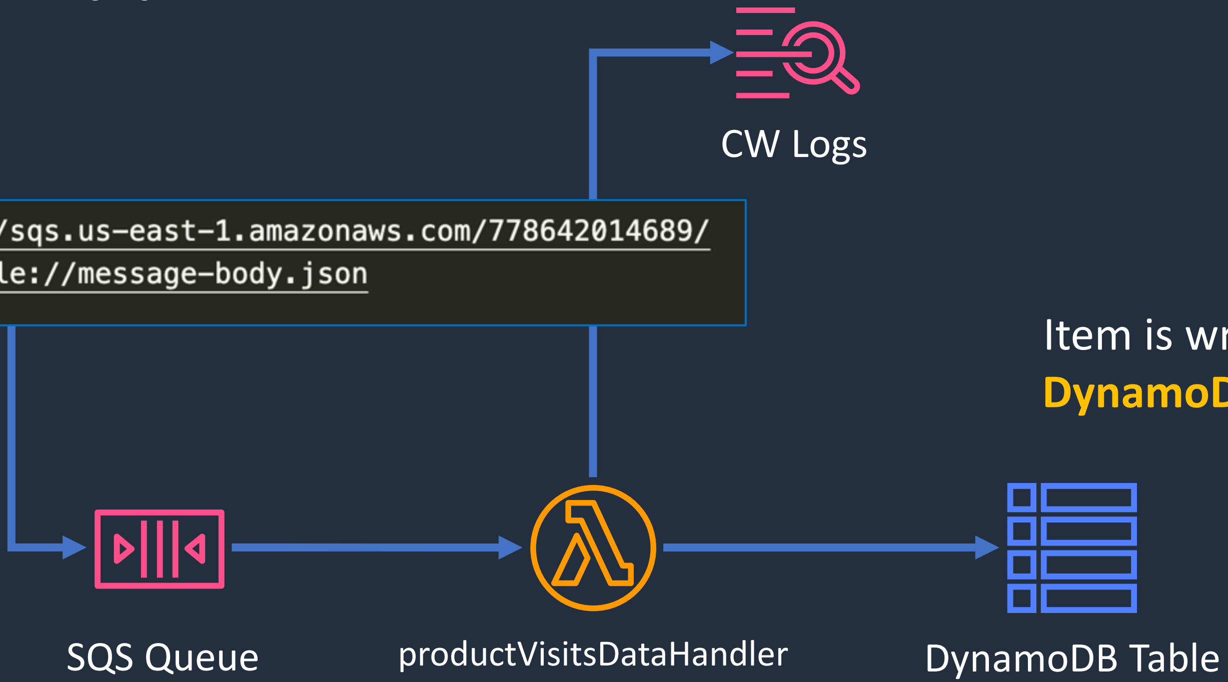


Build a Serverless App – Part 1

Build a Serverless App - Part 1

```
aws sqs send-message --queue-url https://sqs.us-east-1.amazonaws.com/778642014689/
ProductVisitsDataQueue --message-body file://message-body.json
```

Manually add message to queue with **AWS CLI**



Item is written to **DynamoDB table**

SQS triggers Lambda

```json
{
  "ProductId": "c96b49bb-c378-4a15-b2e3-842a9850b23d",
  "ProductName": "Gloves",
  "Category": "Accessories",
  "PricePerUnit": "10",
  "CustomerId": "be44af0a-74f9-438e-a3ac-e3e21d84259f",
  "CustomerName": "John Doe",
  "TimeOfVisit": "2023-01-31T16:23:42.389Z"
}
```

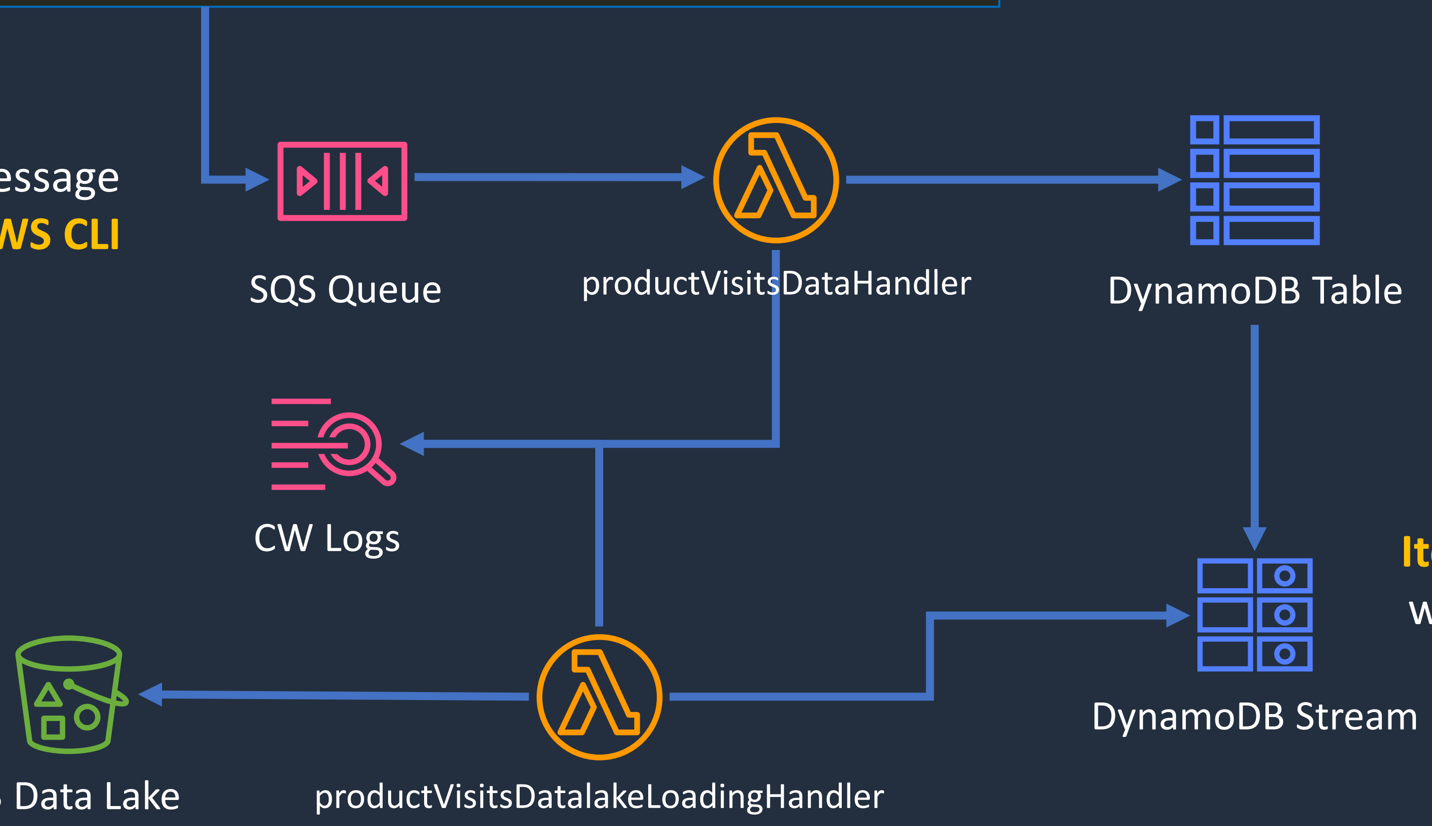
This is the **message body**

Build a Serverless App – Part 2

Build a Serverless App - Part 2

```
aws sqs send-message --queue-url https://sqs.us-east-1.amazonaws.com/778642014689/ProductVisitsDataQueue --message-body file://message-body.json
```

Manually add message to queue with **AWS CLI**



Item is written to **DynamoDB table**

Item-level modifications written to **DDB streams**

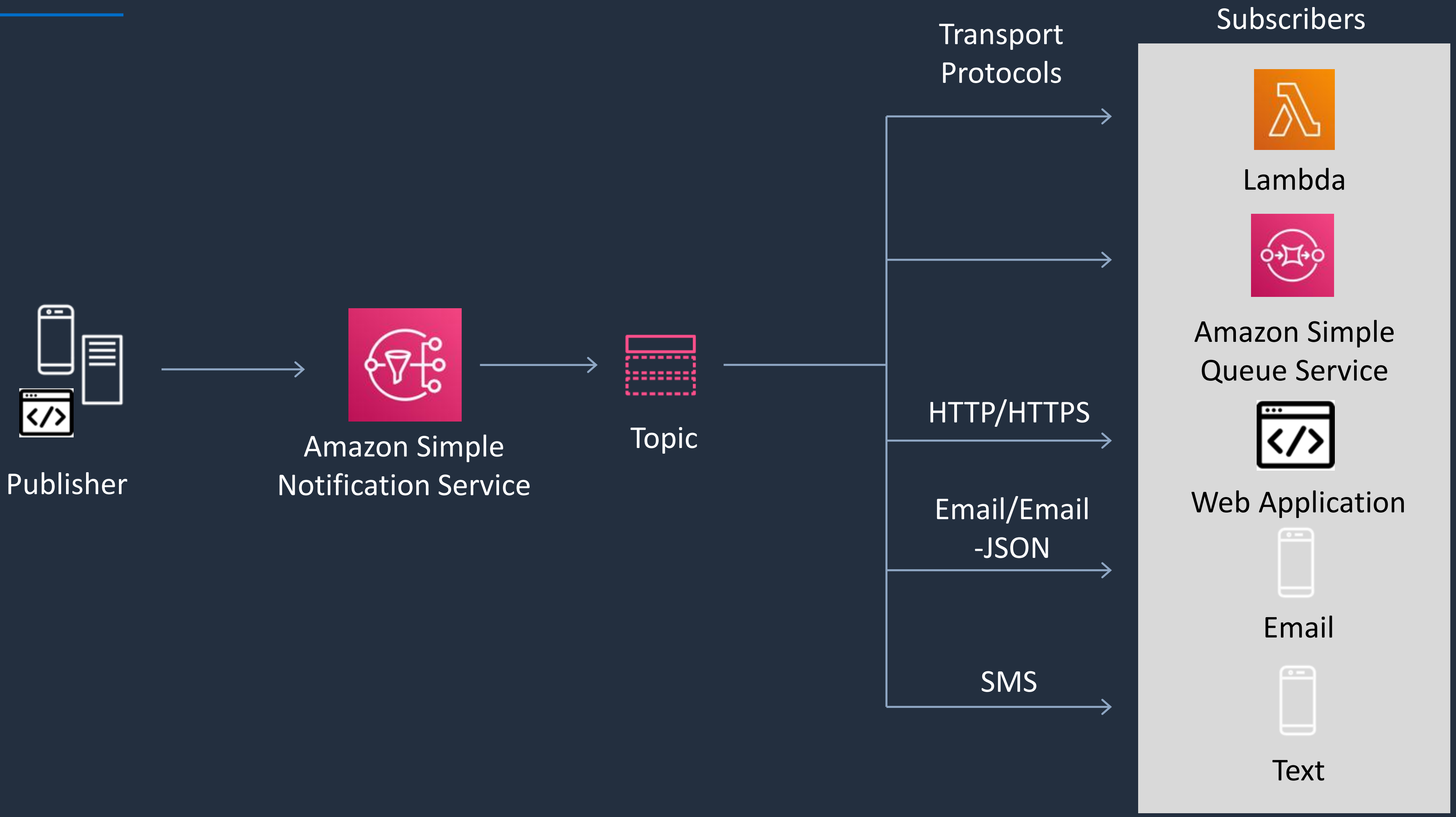
Lambda function **polls** the stream and writes data to the **S3 bucket**

Application Integration Services Comparison

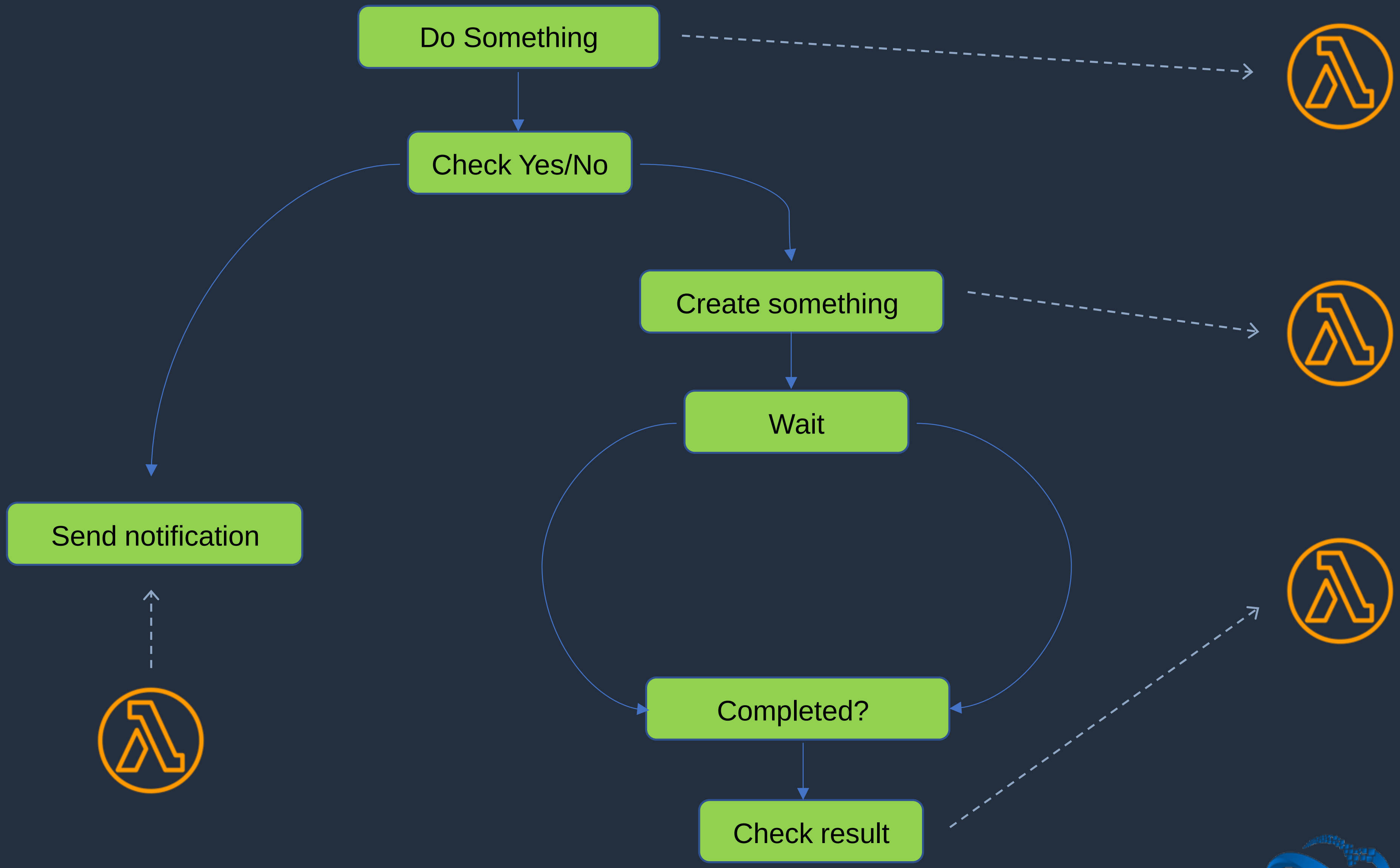
Application Integration Services Comparison

Service	What it does	Example use cases
Simple Queue Service	Messaging queue; store and forward patterns	Building distributed / decoupled applications
Simple Notification Service	Set up, operate, and send notifications from the cloud	Send email notification when CloudWatch alarm is triggered
Step Functions	Out-of-the-box coordination of AWS service components with visual workflow	Order processing workflow
Simple Workflow Service	Need to support external processes or specialized execution logic	Human-enabled workflows like an order fulfilment system or for procedural requests Note: AWS recommends that for new applications customers consider Step Functions instead of SWF
Amazon MQ	Message broker service for Apache Active MQ and RabbitMQ	Need a message queue that supports industry standard APIs and protocols; migrate queues to AWS
Amazon Kinesis	Collect, process, and analyze streaming data.	Collect data from IoT devices for later processing

Amazon SNS Notifications



AWS Step Functions



AWS Step Functions

- AWS Step Functions is used to build distributed applications as a series of steps in a visual workflow.
- You can quickly build and run state machines to execute the steps of your application

How it works:

1. Define the steps of your workflow in the **JSON-based Amazon States Language**. The visual console automatically graphs each step in the order of execution
2. Start an execution to visualize and verify the steps of your application are operating as intended. The console highlights the real-time status of each step and provides a detailed history of every execution
3. AWS Step Functions **operates and scales** the steps of your **application** and **underlying compute** for you to help ensure your application executes reliably under increasing demand

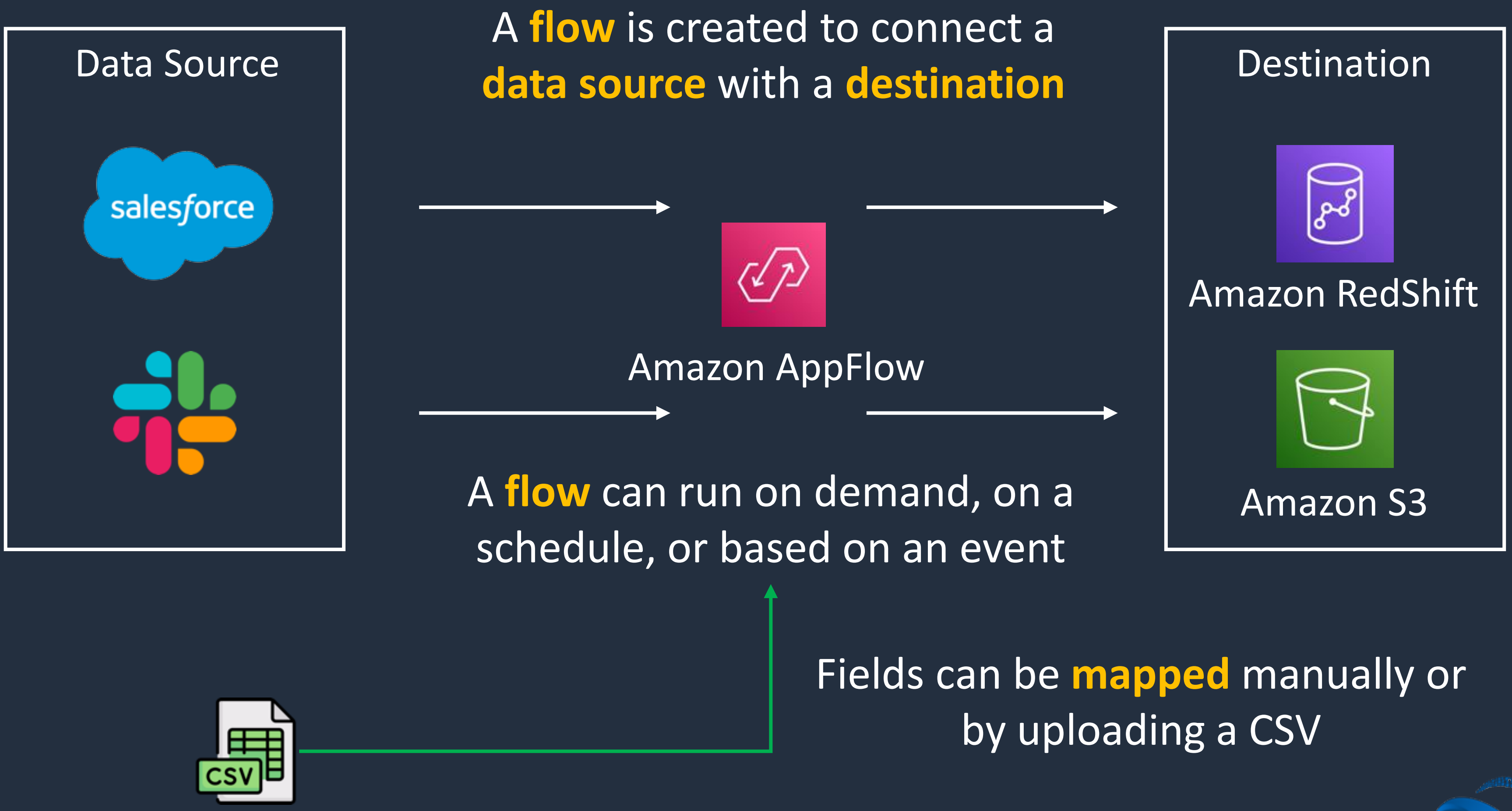
Kinesis vs SQS vs SNS

Amazon Kinesis	Amazon SQS	Amazon SNS
Consumers pull data	Consumers pull data	Push data to many subscribers
As many consumers as you need	Data is deleted after being consumed	Publisher / subscriber model
Routes related records to same record processor	Can have as many workers (consumers) as you need	Integrates with SQS for fan-out architecture pattern
Multiple applications can access stream concurrently	No ordering guarantee (except with FIFO queues)	Up to 10,000,000 subscribers
Ordering at the shard level	Provides messaging semantics	Up to 100,000 topics
Can consume records in correct order at later time	Individual message delay	Data is not persisted
Must provision throughput	Dynamically scales	No need to provision throughput

Amazon AppFlow

Amazon AppFlow

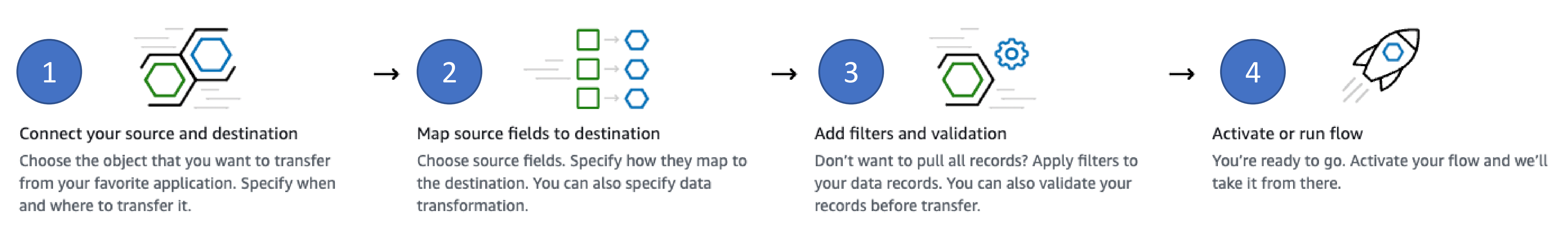
Amazon AppFlow securely **transfers data** between **SaaS** applications



Amazon AppFlow

- Fully managed service with a pay as you go pricing model
- Securely transfer data between Software-as-a-Service (SaaS) applications like Salesforce, SAP, Zendesk, Slack
- AWS destinations include S3, RedShift, Snowflake and more
- Can run up to 100 GB of data per flow
- Performs data transformations like mapping, merging, masking, filtering, and validation as part of the flow
- Leverages AWS PrivateLink for secure transfer of data using private endpoints
- Encryption through AWS KMS and fine-grained permissions can be applied using AWS IAM policies

Creating a Flow in AppFlow



Choose how to **trigger** the flow

On Demand	On a Schedule	On an Event
Flow will run immediately when you trigger it	Flow will run at specified times	Flow will run when an event occurs

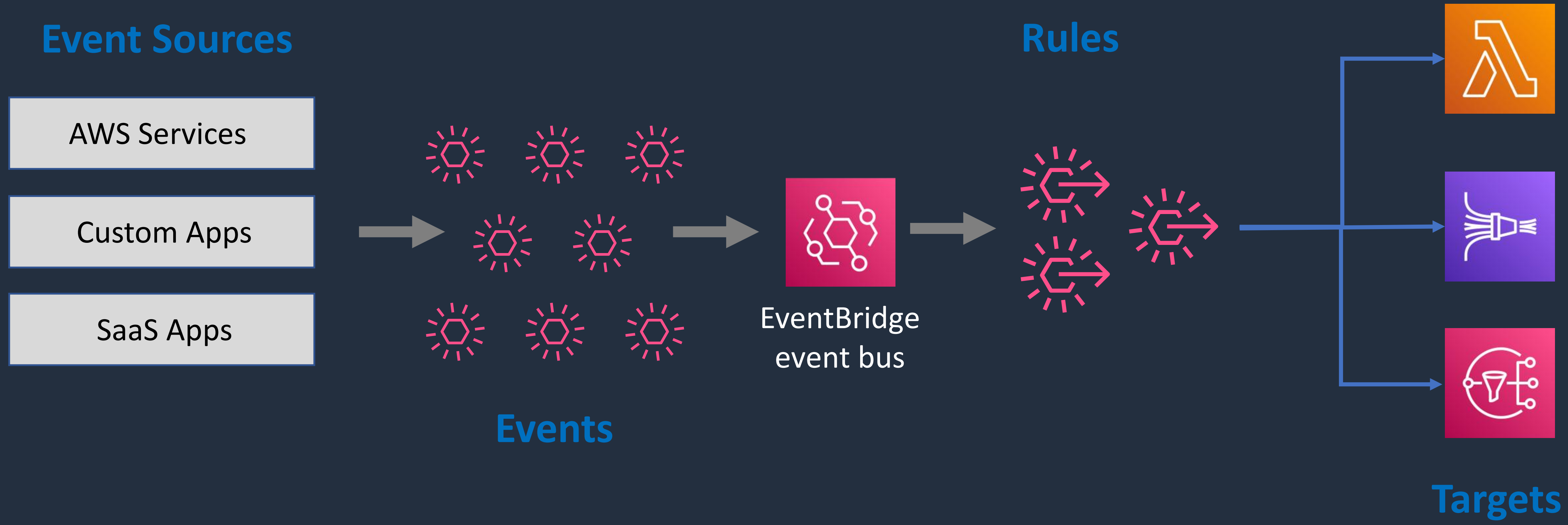
Amazon AppFlow Use Cases

- Synchronizing data from multiple sources for analytics
- Data enrichment – e.g. pull data from SaaS applications, add metadata and feed back to the SaaS application
- Event-based workflows – automatically trigger processes in one app based on data from another
- Transfer opportunity data from Salesforce to populate real-time dashboards
- Automate data backups from SaaS applications to AWS services such as Amazon S3
- Analyze Slack events using business intelligence tools

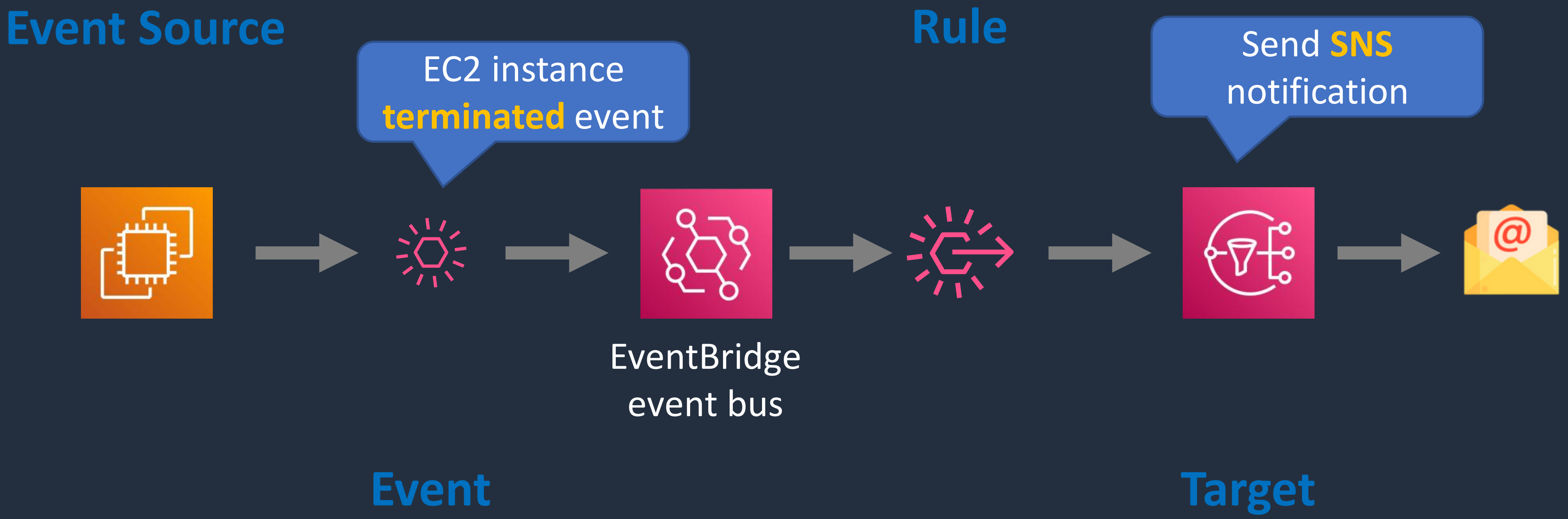
Amazon EventBridge

Amazon EventBridge

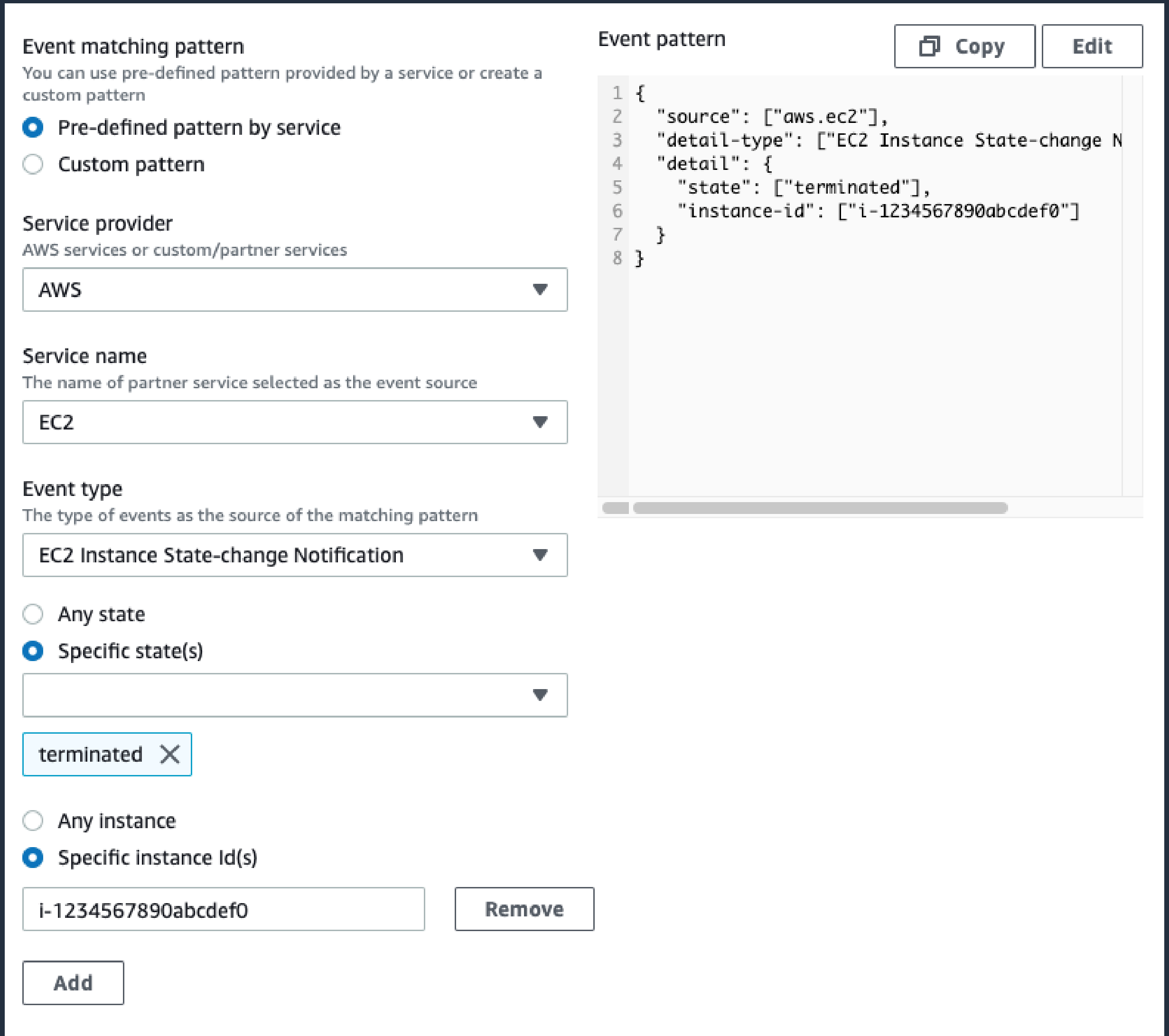
EventBridge used to be known as **CloudWatch Events**



Amazon EventBridge Example 1

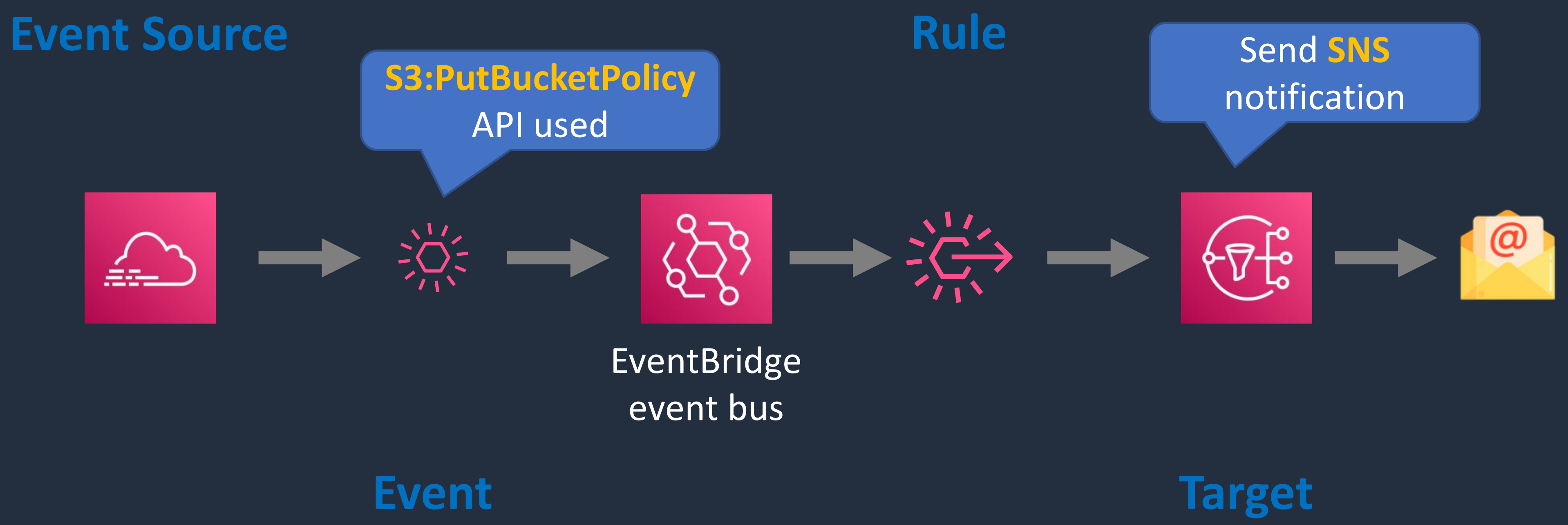


Amazon EventBridge Example 1




```
{
   "version": "0",
   "id": "6a7e8feb-b491-4cf7-a9f1-bf3703467718",
   "detail-type": "EC2 Instance State-change Notification",
   "source": "aws.ec2",
   "account": "111122223333",
   "time": "2017-12-22T18:43:48Z",
   "region": "us-west-1",
   "resources": [
     "arn:aws:ec2:us-west-1:123456789012:instance/i-1234567890abcdef0"
  ],
  "detail": {
   "instance-id": "i-1234567890abcdef0",
   "state": "terminated"
   }
 }
```

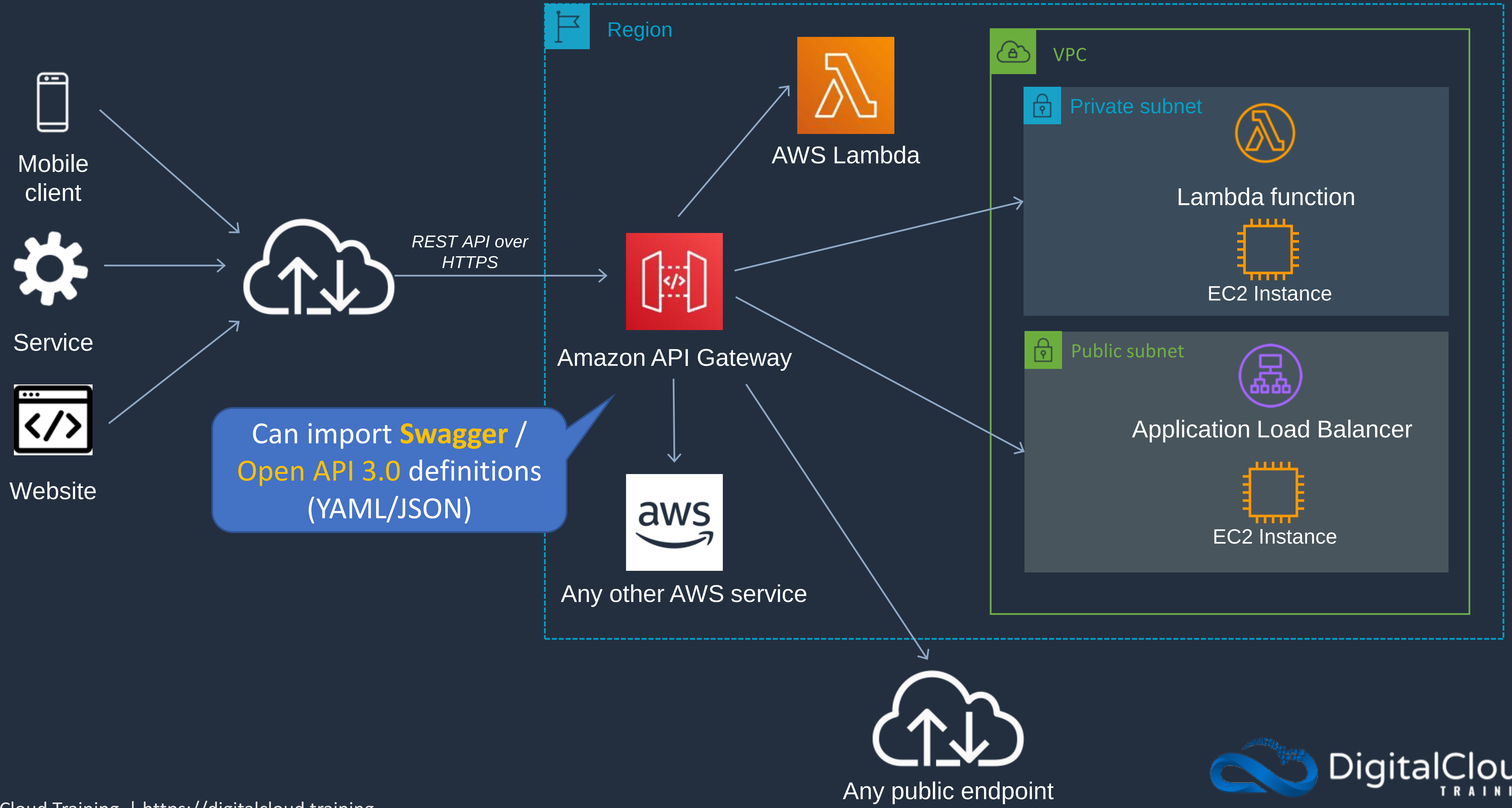
Amazon EventBridge Example 2




```
{
  "source": ["aws.cloudtrail"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["cloudtrail.amazonaws.com"],
    "eventName": ["s3:PutBucketPolicy"]
  }
}
```

Amazon API Gateway Core Knowledge

Amazon API Gateway Overview

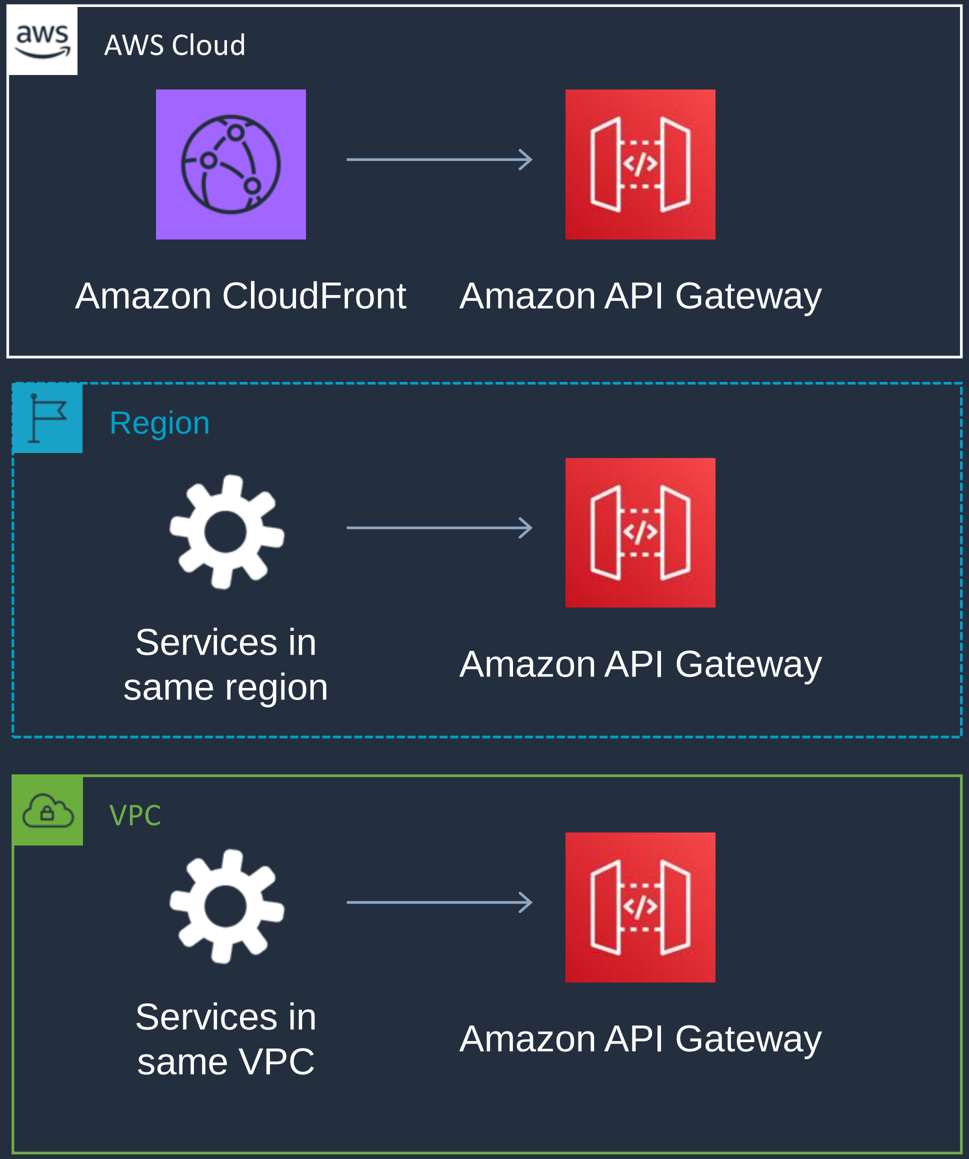


Amazon API Gateway Deployment Types

Edge-optimized endpoint

Regional endpoint

Private endpoint

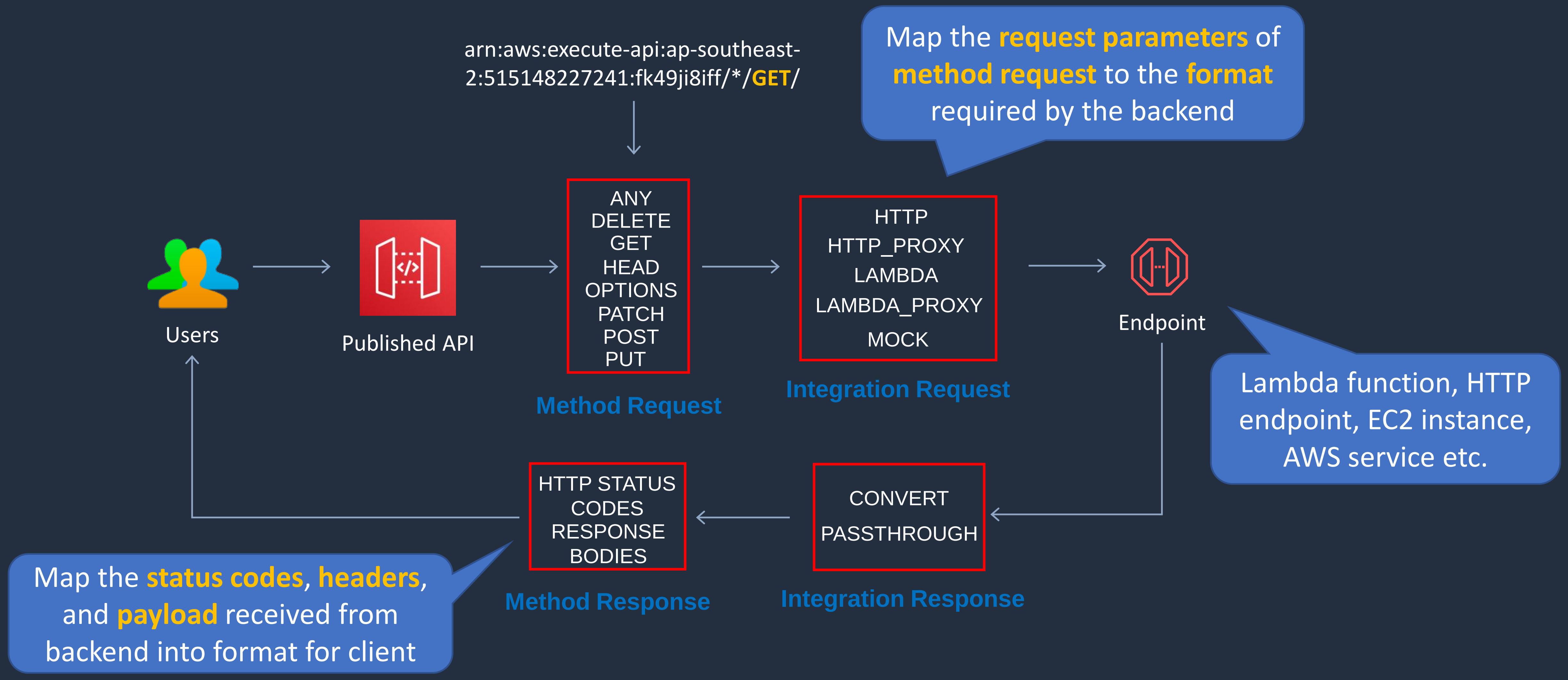


Key benefits:
- Reduced latency for requests from around the world

Key benefits:
- Reduced latency for requests that originate in the same region
- Can also configure your own CDN and protect with WAF

Key benefits:
- Securely expose your REST APIs only to other services within your VPC or connect via Direct Connect

Amazon API Gateway – Structure of an API

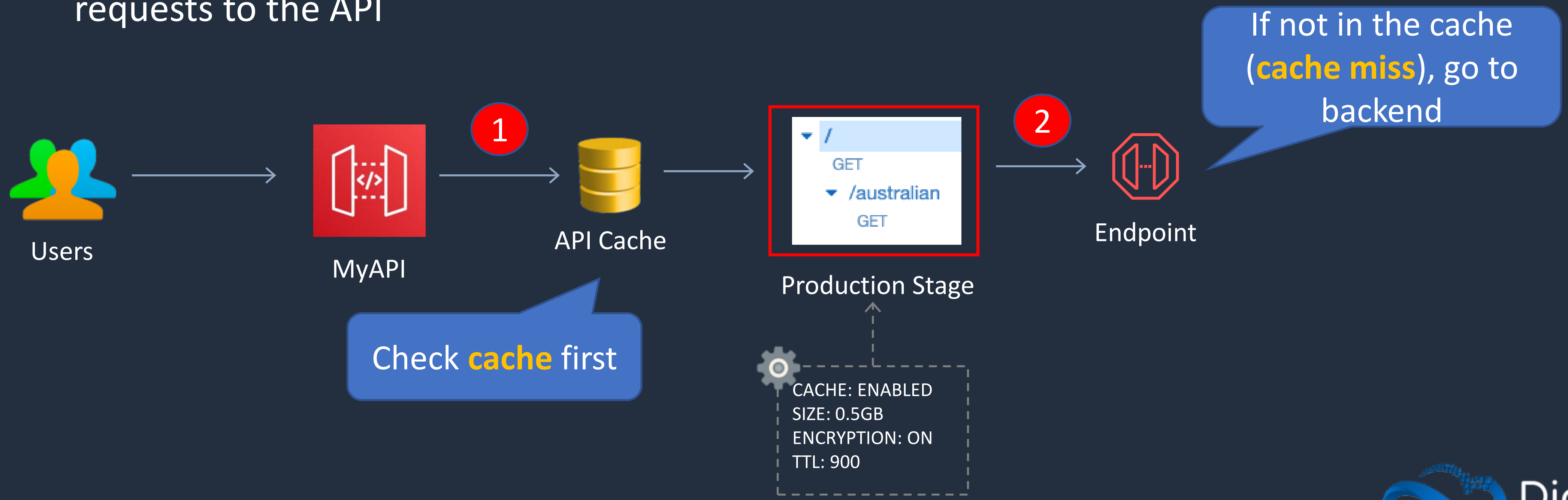


API Gateway Integrations

- For a Lambda function, you can have the Lambda proxy integration, or the Lambda custom integration
- For an HTTP endpoint, you can have the HTTP proxy integration or the HTTP custom integration
- For an AWS service action, you have the AWS integration of the non-proxy type only

API Gateway - Caching

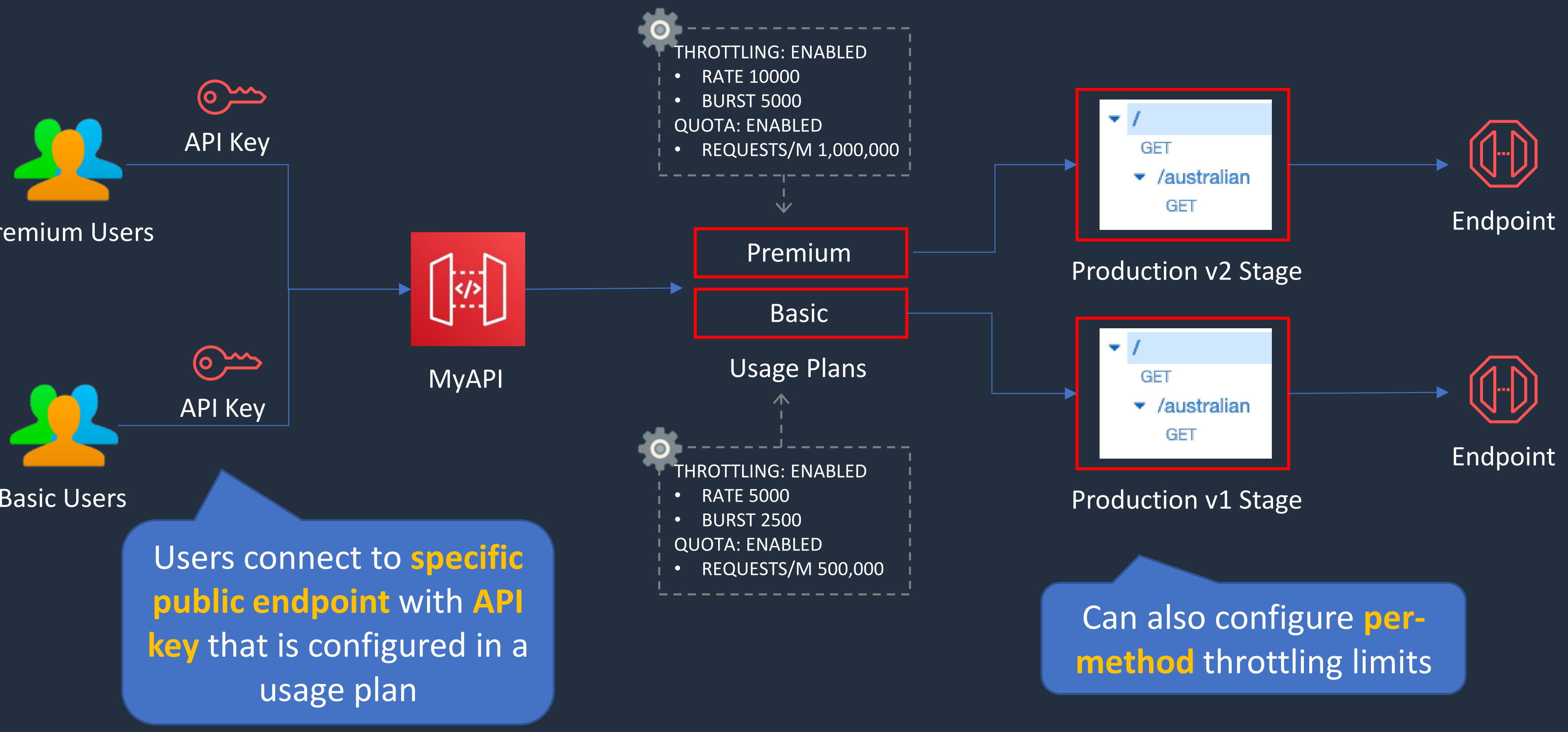
- You can add caching to API calls by provisioning an Amazon API Gateway cache and specifying its size in gigabytes
- Caching allows you to cache the endpoint's response
- Caching can reduce number of calls to the backend and improve latency of requests to the API



API Gateway - Throttling

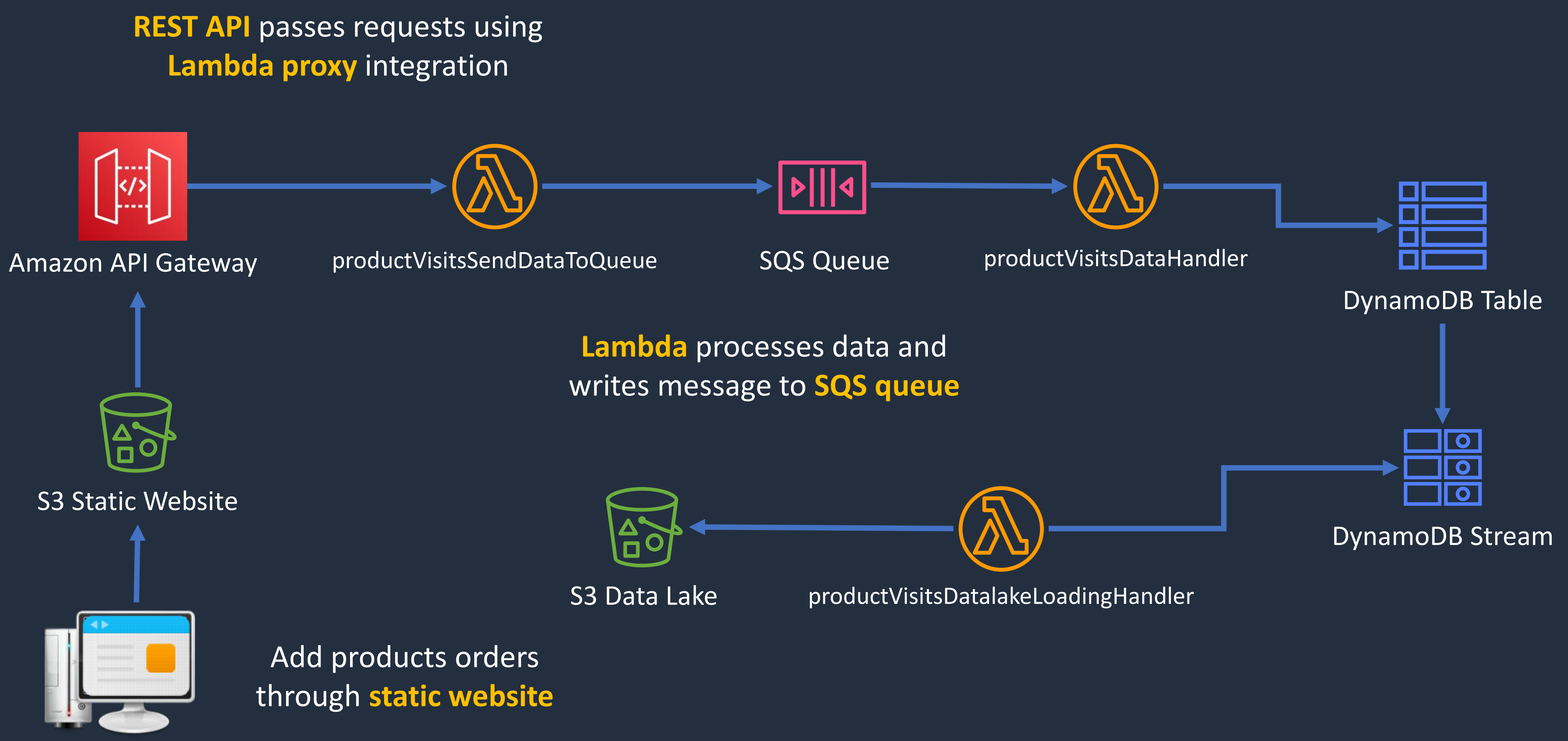
- API Gateway sets a limit on a steady-state rate and a burst of request submissions against all APIs in your account
- Limits:
  - By default API Gateway limits the steady-state request rate to 10,000 requests per second
  - The maximum concurrent requests is 5,000 requests across all APIs within an AWS account
  - If you go over 10,000 requests per second or 5,000 concurrent requests you will receive a 429 Too Many Requests error response
- Upon catching such exceptions, the client can resubmit the failed requests in a way that is rate limiting, while complying with the API Gateway throttling limits

API Gateway – Usage Plans and API Keys



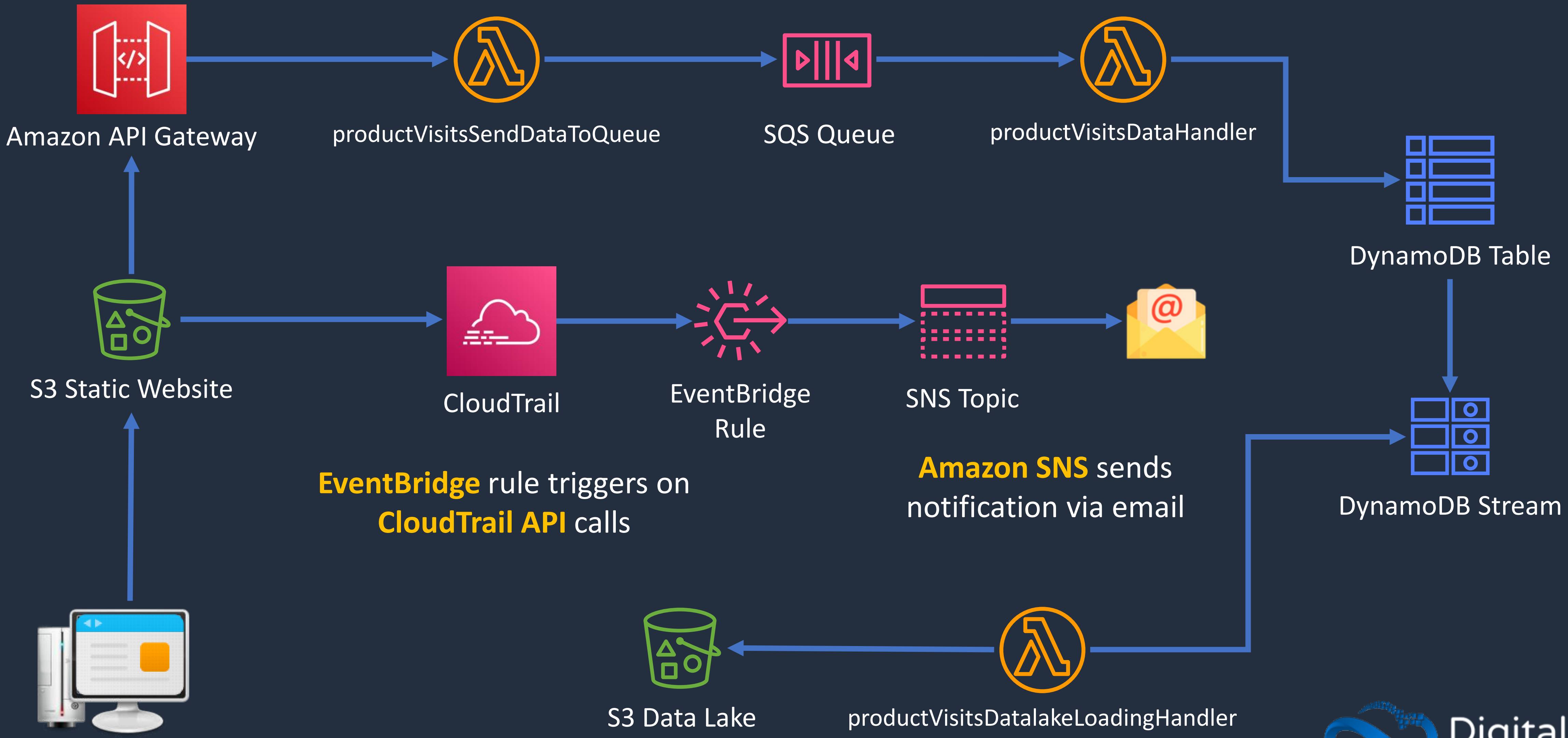
Build a Serverless App – Part 3

Build a Serverless App - Part 3



Build a Serverless App – Part 4

Build a Serverless App - Part 4



Amazon Simple Email Service (Amazon SES)

Amazon SES

- Amazon SES is a cloud email service provider
- Used for delivering high volume email campaigns
- Delivers hundreds of billions of emails per year
- No need for an on-premises SMTP server
- Supports owned, dedicated, or shared IP addresses
- Offers security with SPF, DMARC, and DKIM
- Engagement data can be sent to AWS services such as S3, RedShift, EC2, and Lambda

Amazon SES vs SNS

	Amazon SES	Amazon SNS
Use case	Email	Notifications
Direction	Inbound/outbound	Push only
Model	Email service provider	Pub/Sub
Use Case	Transactional, marketing, notifications, announcements	Notifications
Format	Email	Multiple formats including email

DigitalCloud TRAINING

Architecture Patterns - Serverless

Architecture Patterns - Serverless

Requirement	Solution
Application includes EC2 and RDS. Spikes in traffic causing writes to be dropped by RDS	Decouple EC2 and RDS database with an SQS queue; use Lambda to process records in the queue
Migrate decoupled on-premises web app. Users upload files and processing tier processes and stores in NFS file system. Should scale dynamically	Migrate to EC2 instances, SQS and EFS. Use Auto Scaling and scale the processing tier based on the SQS queue length
Lambda function execution time has increased significantly as the number of records in the data to process has increased	Optimize execution time by increasing memory available to the function which will proportionally increase CPU

Architecture Patterns - Serverless

Requirement	Solution
API Gateway forwards streaming data to AWS Lambda to process and TooManyRequestsException is experienced	Send the data to a Kinesis Data Stream and then configure Lambda to process in batches
Migrating app with highly variable load to AWS. Must be decoupled and orders must be processed in the order they are received	Implement an Amazon SQS FIFO queue to preserve the record order
Company needs to process large volumes of media files with Lambda which takes +2hrs. Need to optimize time and automate the whole process	Configure a Lambda function to write jobs to queue. Configure queue as input to Step Functions which will coordinate multiple functions to process in parallel

DigitalCloud TRAINING

Architecture Patterns - Serverless

Requirement	Solution
Objects uploaded to an S3 bucket must be processed by AWS Lambda	Create an event source notification to notify Lambda function to process new objects
Company requires API events that involve the root user account to be captured in a third-party ticketing system	Create a CloudTrail trail and an EventBridge rule that looks for API events that involve root, put events on SQS queue; process queue with Lambda
Legacy application uses many batch scripts that process data and pass on to next script. Complex and difficult to maintain	Migrate scripts to AWS Lambda functions and use AWS Step Functions to coordinate components

Architecture Patterns - Serverless

Requirement	Solution
Lambda processes objects created in bucket. Large volumes of objects can be uploaded. Must ensure function does not affect other critical functions	Configure reserved concurrency to set the maximum limit for the function. Monitor critical functions’ CloudWatch alarms for the Throttles Lambda metric
EC2 instance processes images using JavaScript code and stores in S3. Load is highly variable. Need a more cost-effective solution	Replace EC2 with AWS Lambda function
Solutions Architect needs to update Lambda function code using canary strategy; traffic should be routed based on weights	Create an Alias for the Lambda function and configure weights to Lambda versions

Architecture Patterns - Serverless

Requirement	Solution
App uses API Gateway regional REST API. Just gone global and performance has suffered	Convert API to an edge-optimized API to optimize for the global user base
App uses API Gateway and Lambda. During busy periods many requests fail multiple times before succeeding. No errors reported in Lambda	Throttle limit could be configured a value that is too low. Increase the throttle limit
Need to ensure only authorized IAM users can access REST API on API Gateway	Set authorization to AWS_IAM for API Gateway method. Grant execute-api:Invoke permissions in IAM policy

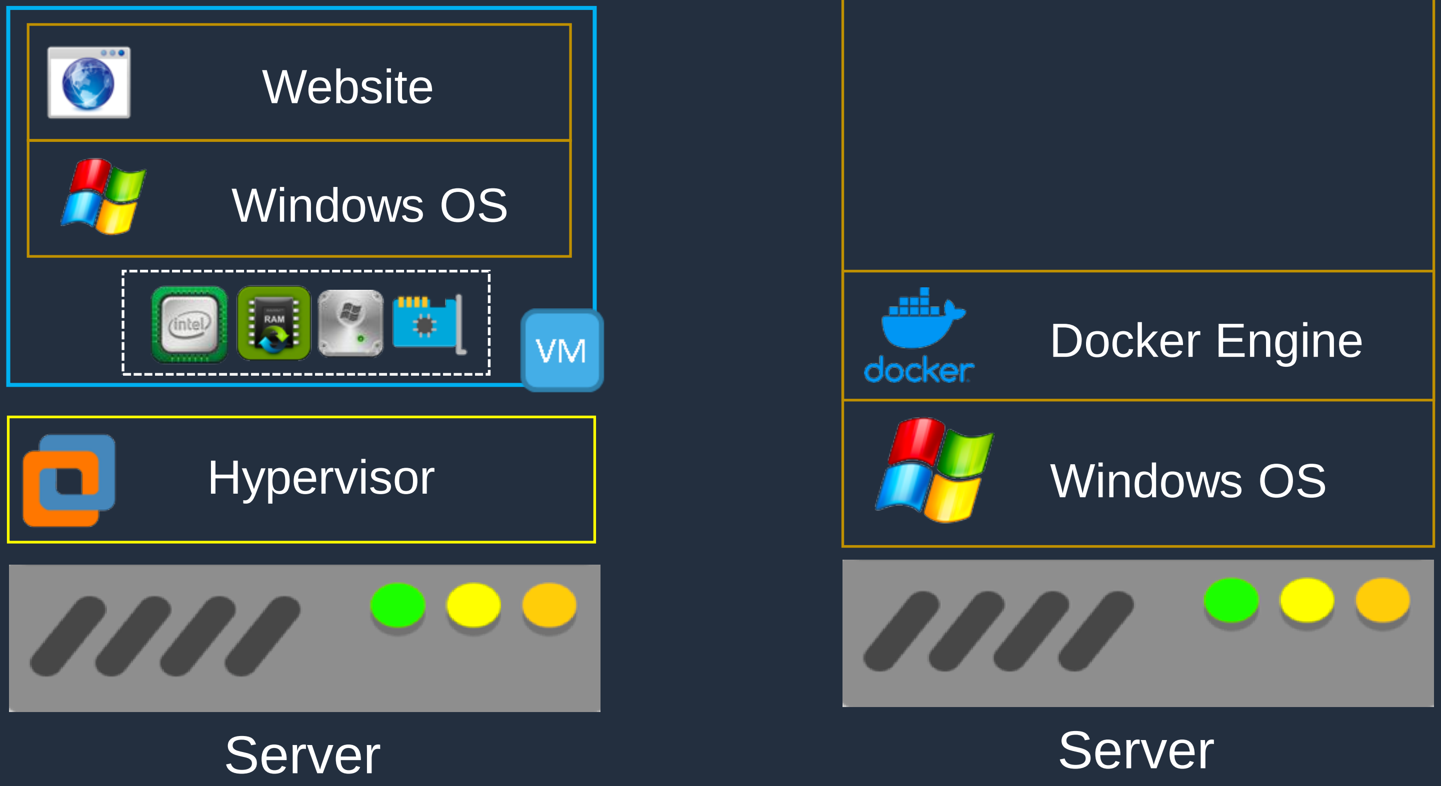
SECTION 12

Docker Containers and PaaS

Docker Containers and Microservices

Server Virtualization vs Containers

Every VM/instance needs an **operating system** which uses significant resources



Docker Containers

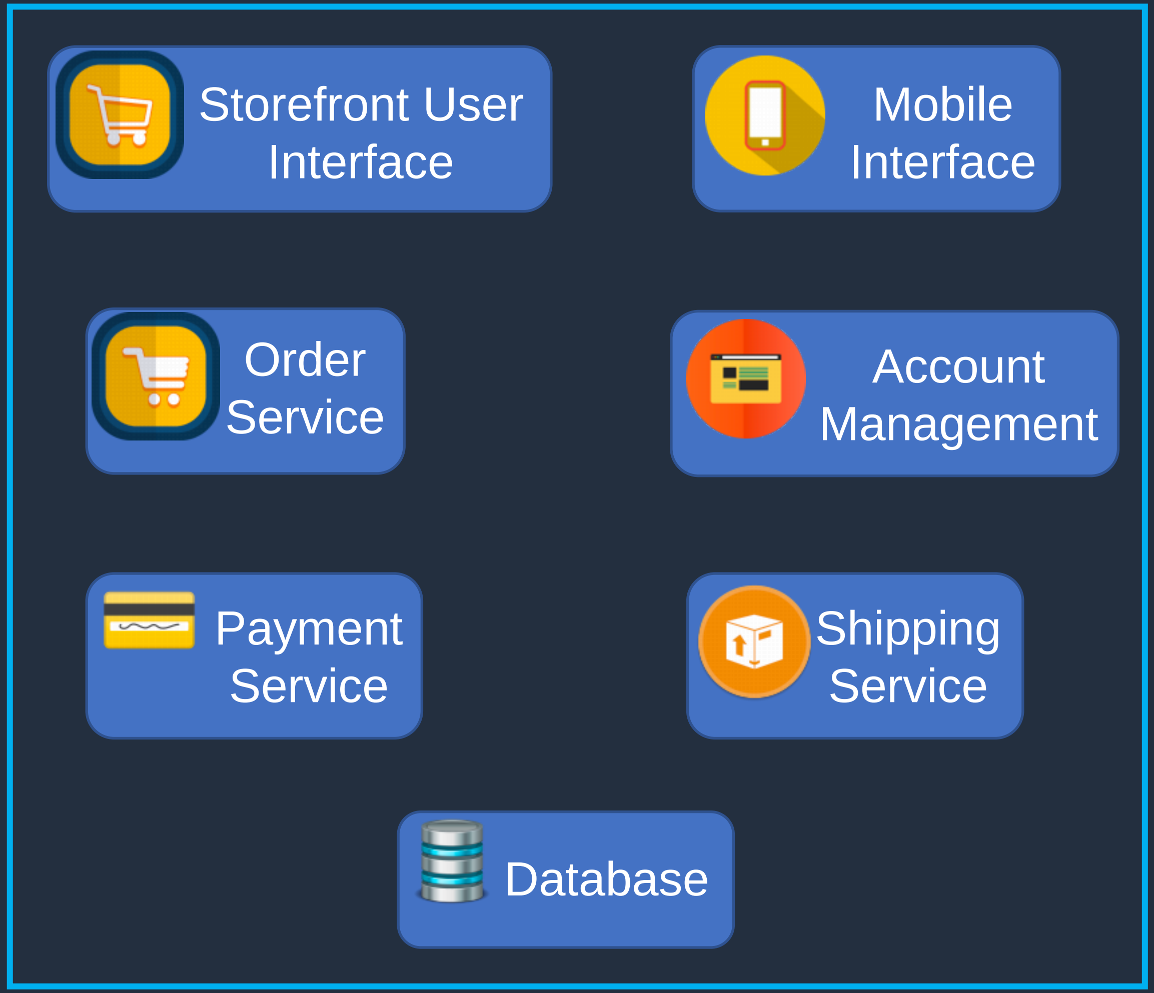
Containers **start up** very **quickly**

A **container** includes all the code, settings, and dependencies for running the application

Containers are very resource **efficient**

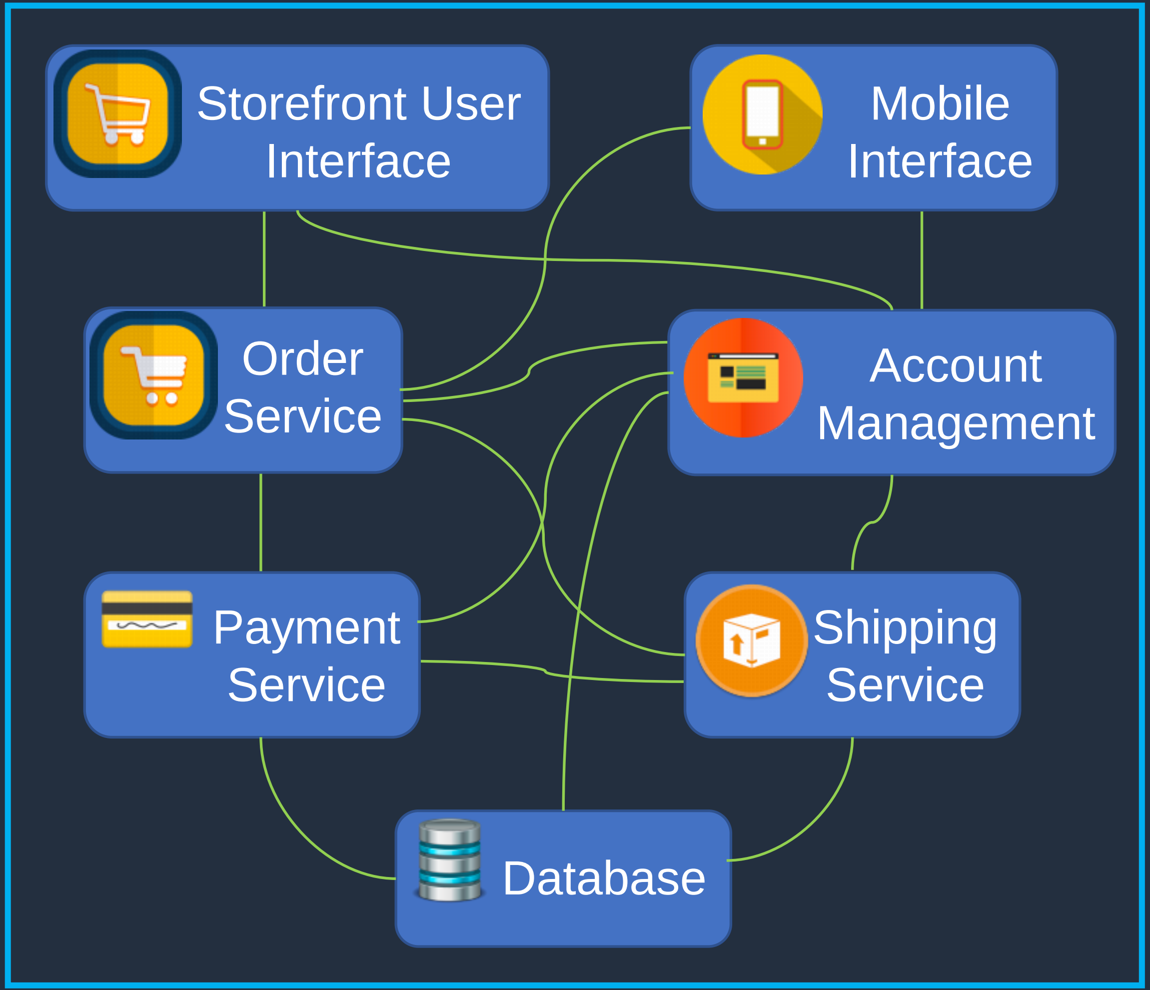
Each container is **isolated** from other containers

Monolithic Application



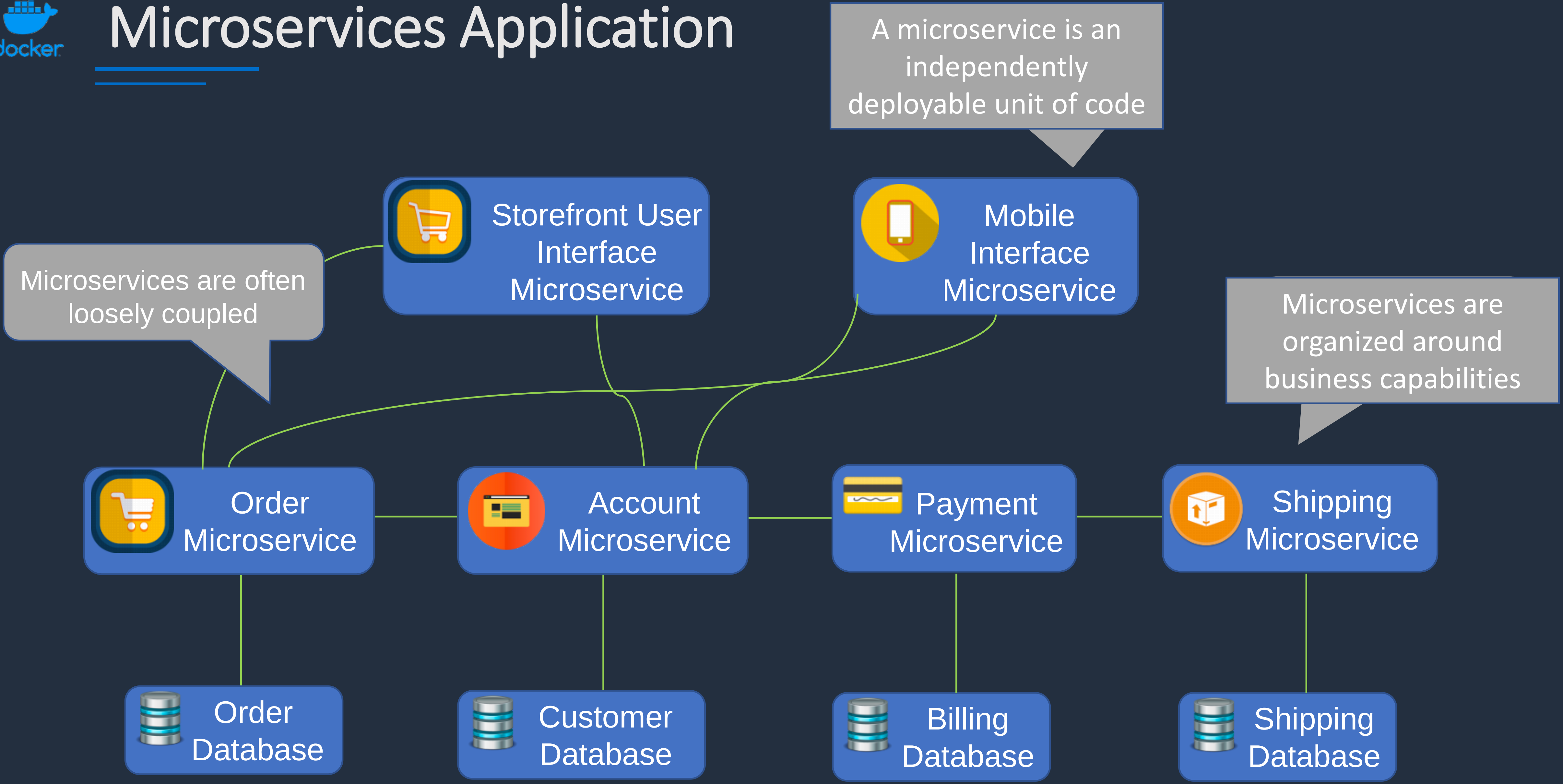
Monolithic Application

Updates to, or failures of, any single component can take down the whole application



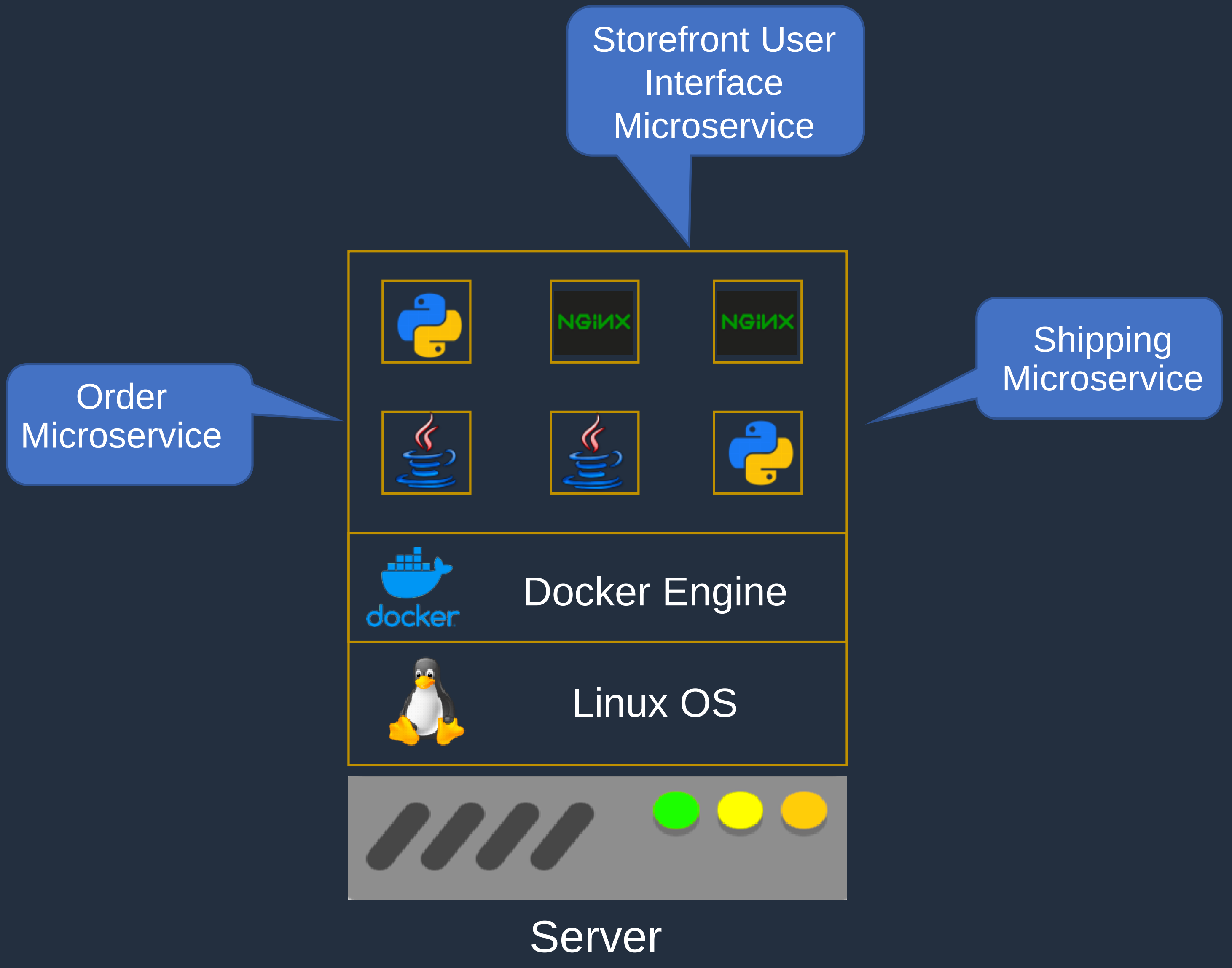
The user interface, business logic, and data access layer are combined on a single platform

Microservices Application



DigitalCloud TRAINING

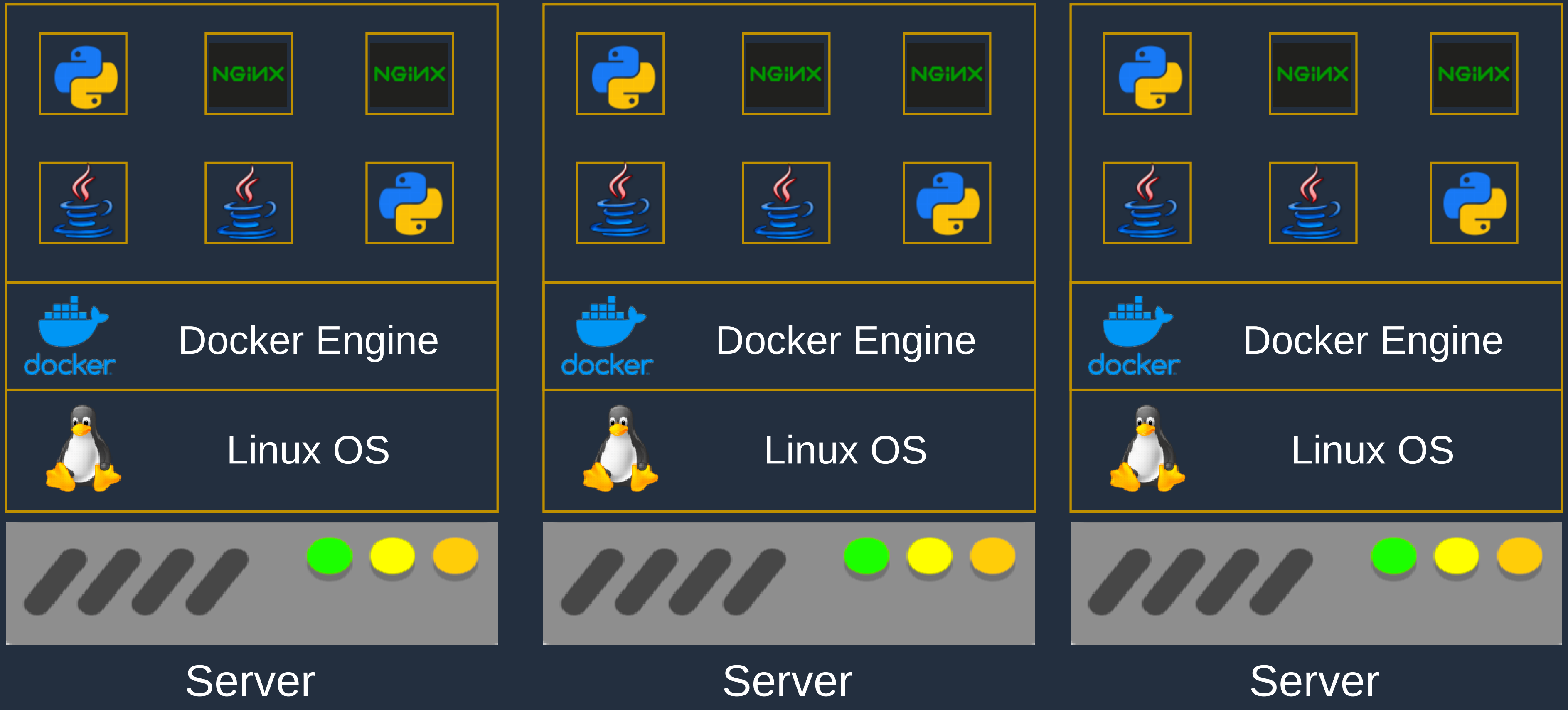
Microservices Application



Microservices Application

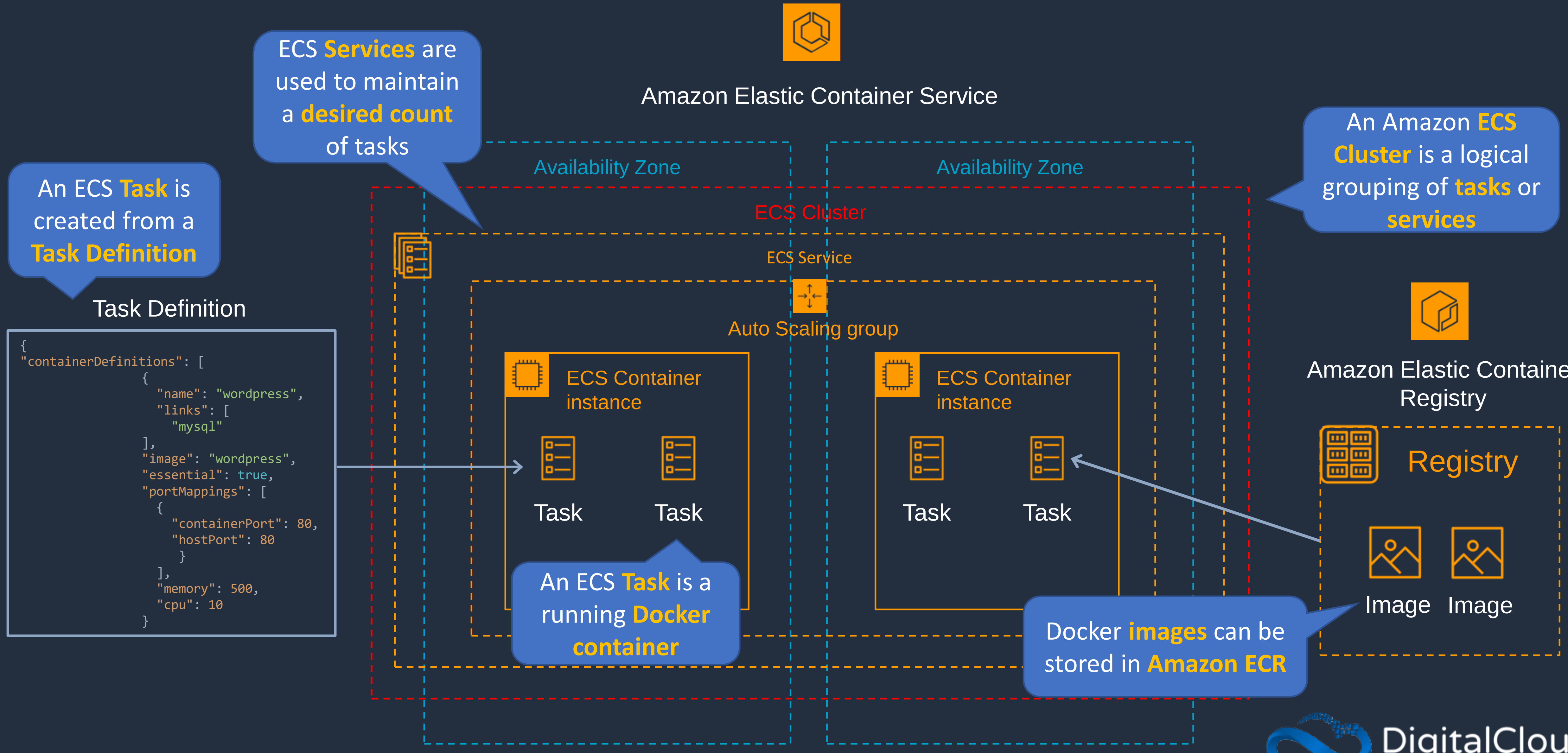
Microservices can also be **spread** across hosts

Many instances of each microservice can run on each host



Amazon Elastic Container Service (ECS)

Amazon ECS



ECS **Services** are used to maintain a **desired count** of tasks
Amazon Elastic Container Service
An Amazon **ECS Cluster** is a logical grouping of **tasks** or **services**
An ECS **Task** is created from a **Task Definition**
Availability Zone
Availability Zone
ECS Cluster
ECS Service
Auto Scaling group
Task Definition
{
"containerDefinitions": [
{
"name": "wordpress",
"links": [
"mysql"
],
"image": "wordpress",
"essential": true,
"portMappings": [
{
"containerPort": 80,
"hostPort": 80
}
],
"memory": 500,
"cpu": 10
}
ECS Container instance
ECS Container instance
Task
Task
Task
Task
Amazon Elastic Container Registry
Registry
Image
Image
An ECS **Task** is a running **Docker container**
Docker **images** can be stored in **Amazon ECR**

Amazon ECS Key Features

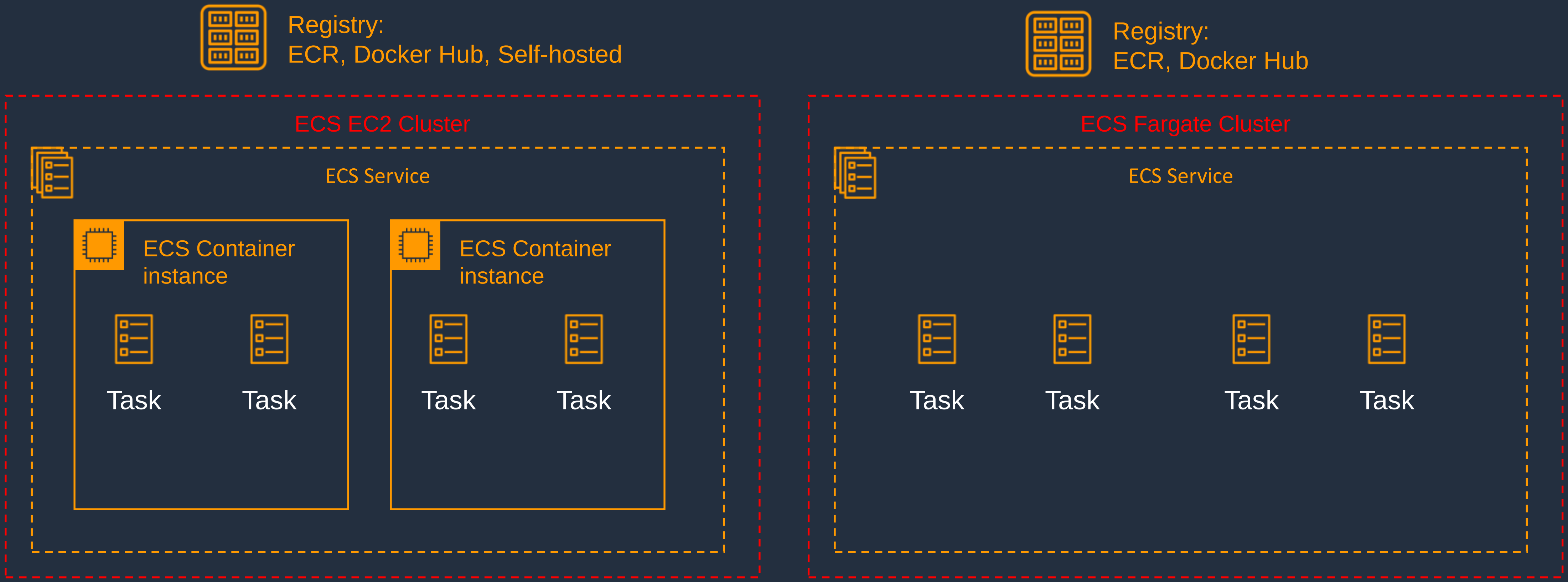
- **Serverless with AWS Fargate** – managed for you and fully scalable
- **Fully managed container orchestration** – control plane is managed for you
- **Docker support** – run and manage Docker containers with integration into the Docker Compose CLI
- **Windows container support** – ECS supports management of Windows containers
- **Elastic Load Balancing integration** – distribute traffic across containers using ALB or NLB
- **Amazon ECS Anywhere (NEW)** – enables the use of Amazon ECS control plane to manage on-premises implementations

Amazon ECS Components

Elastic Container Service (ECS)	Description
Cluster	Logical grouping of EC2 instances
Container instance	EC2 instance running the the ECS agent
Task Definition	Blueprint that describes how a docker container should launch
Task	A running container using settings in a Task Definition
Service	Defines long running tasks – can control task count with Auto Scaling and attach an ELB

Amazon ECS Launch Types

Launch Types – EC2 and Fargate



EC2 Launch Type

- You explicitly provision EC2 instances
- You're responsible for managing EC2 instances
- Charged per running EC2 instance
- Docker volumes, EFS, and FSx for Windows File Server
- You handle cluster optimization
- More granular control over infrastructure

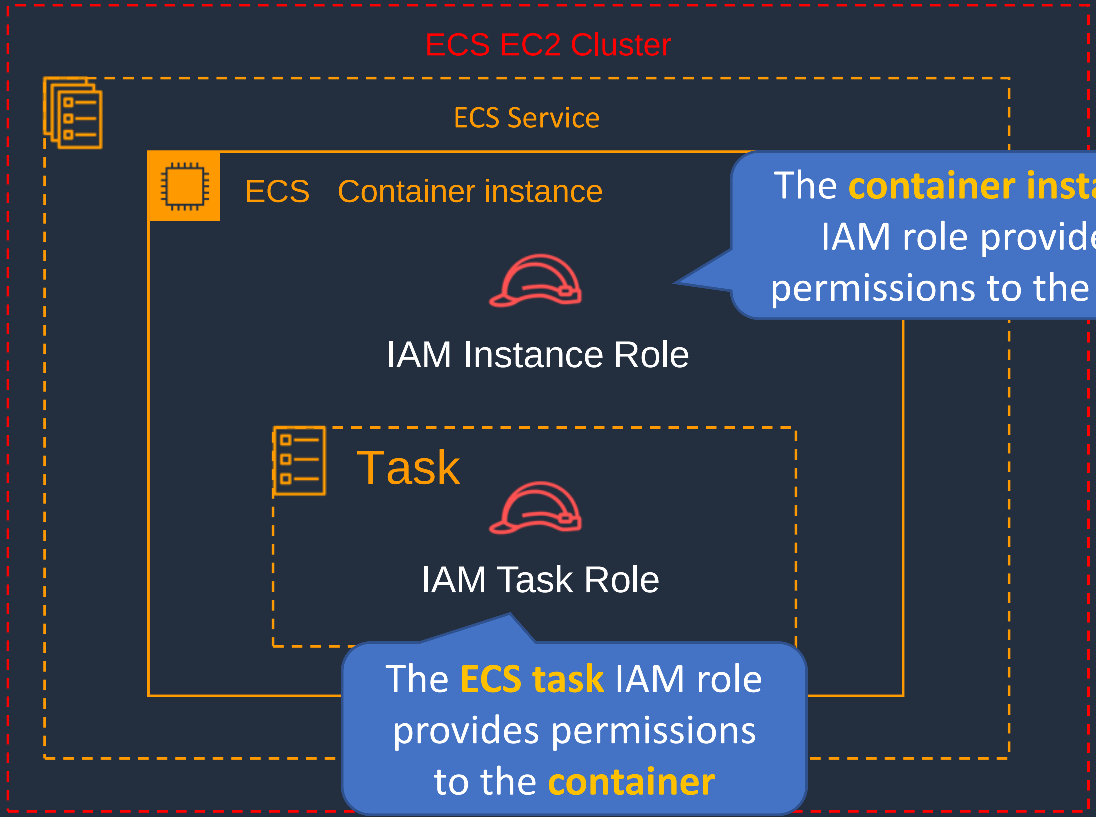
Fargate Launch Type

- Fargate automatically provisions resources
- Fargate provisions and manages compute
- Charged for running tasks
- EFS integration
- Fargate handles cluster optimization
- Limited control, infrastructure is automated

Deploy Tasks on Amazon ECS

Amazon ECS and IAM Roles

ECS and IAM Roles



The **container instance** IAM role provides permissions to the **host**

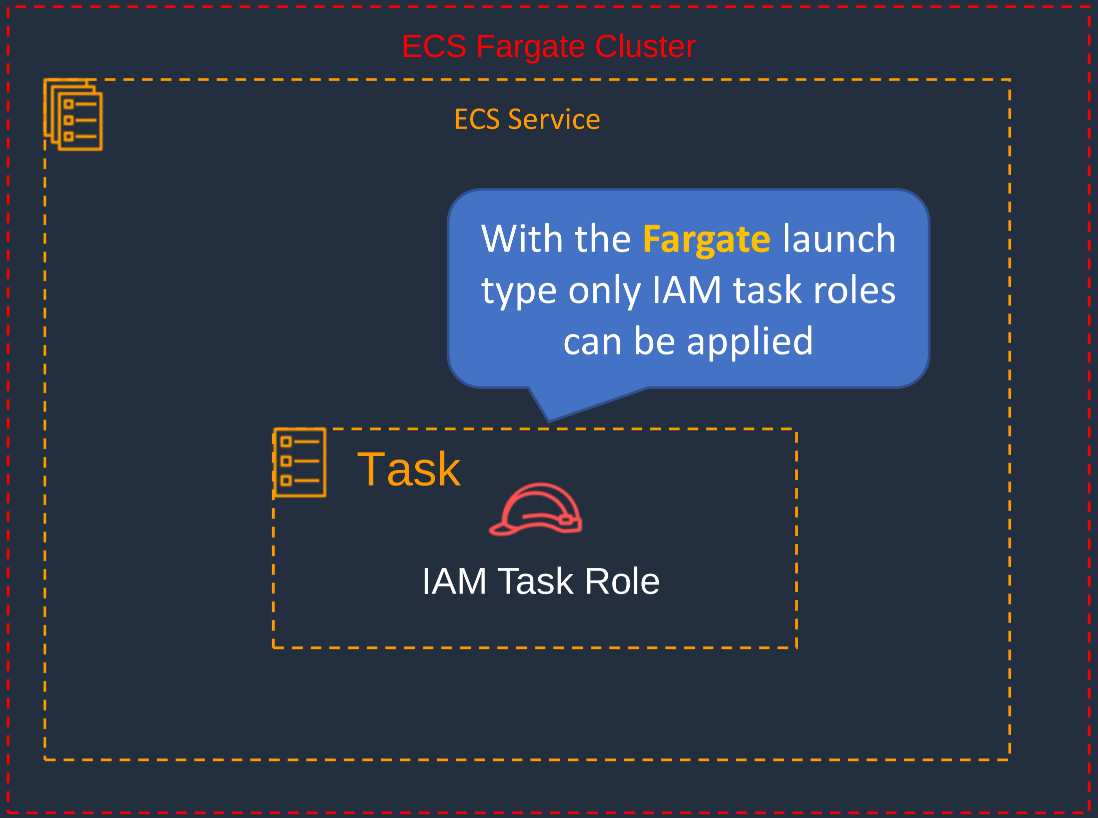
The **ECS task** IAM role provides permissions to the **container**

NOTE: container instances have access to **all of the permissions** that are supplied to the **container instance role** through instance metadata

AmazonEC2ContainerServiceforEC2Role

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:DescribeTags",
                "ecs:CreateCluster",
                "ecs:DeregisterContainerInstance",
                "ecs:DiscoverPollEndpoint",
                "ecs:Poll",
                "ecs:RegisterContainerInstance",
                "ecs:StartTelemetrySession",
                "ecs:UpdateContainerInstancesState",
                "ecs:Submit*",
                "ecr:GetAuthorizationToken",
                "ecr:BatchCheckLayerAvailability",
                "ecr:GetDownloadUrlForLayer",
                "ecr:BatchGetImage",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": "*"
        }
    ]
}
```

ECS and IAM Roles



Scaling Amazon ECS

Auto Scaling for ECS

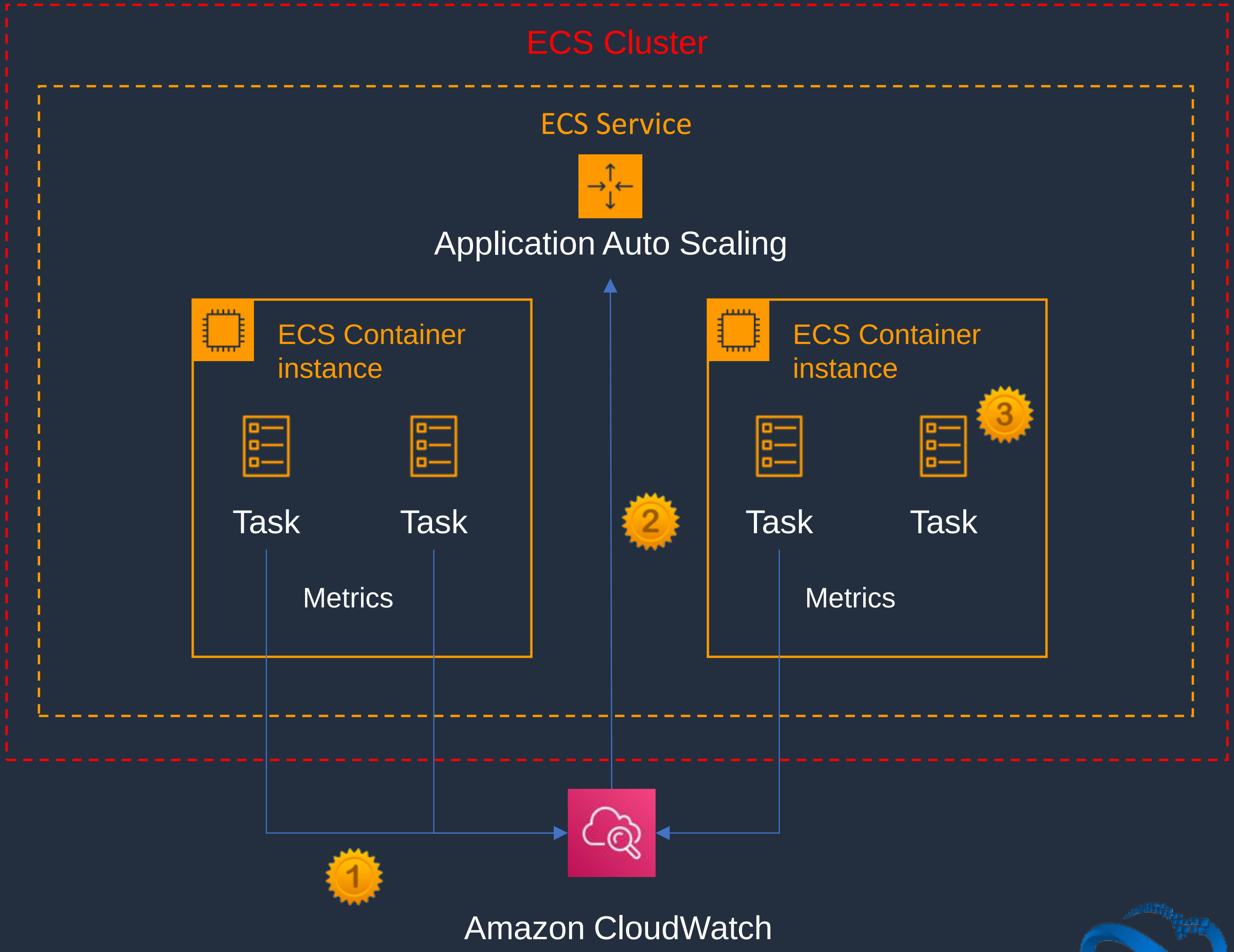
Two types of scaling:

1. Service auto scaling
2. Cluster auto scaling

- **Service auto scaling** automatically adjusts the desired task count up or down using the Application Auto Scaling service
- **Service auto scaling** supports target tracking, step, and scheduled scaling policies
- **Cluster auto scaling** uses a Capacity Provider to scale the number of EC2 cluster instances using EC2 Auto Scaling

Service Auto Scaling

- 1. Metric reports CPU > 80%
- 2. CloudWatch notifies Application Auto Scaling
- 3. ECS launches additional task



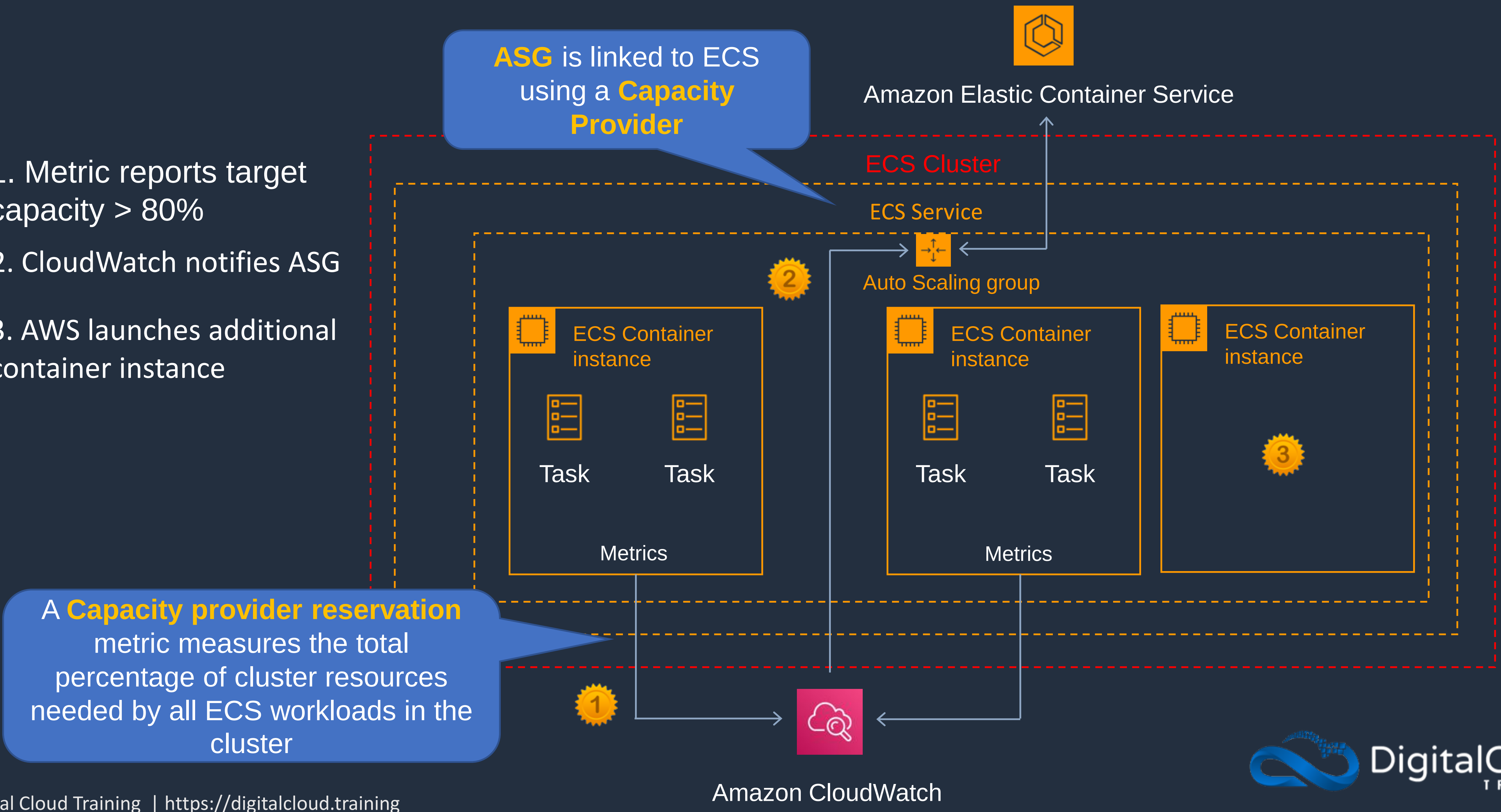
Service Auto Scaling

- Amazon ECS Service Auto Scaling supports the following types of scaling policies:
  - **Target Tracking Scaling Policies**—Increase or decrease the number of tasks that your service runs based on a target value for a specific CloudWatch metric
  - **Step Scaling Policies**—Increase or decrease the number of tasks that your service runs in response to CloudWatch alarms. Step scaling is based on a set of scaling adjustments, known as step adjustments, which vary based on the size of the alarm breach
  - **Scheduled Scaling**—Increase or decrease the number of tasks that your service runs based on the date and time

DigitalCloud TRAINING

Cluster Auto Scaling

- 1. Metric reports target capacity > 80%
- 2. CloudWatch notifies ASG
- 3. AWS launches additional container instance

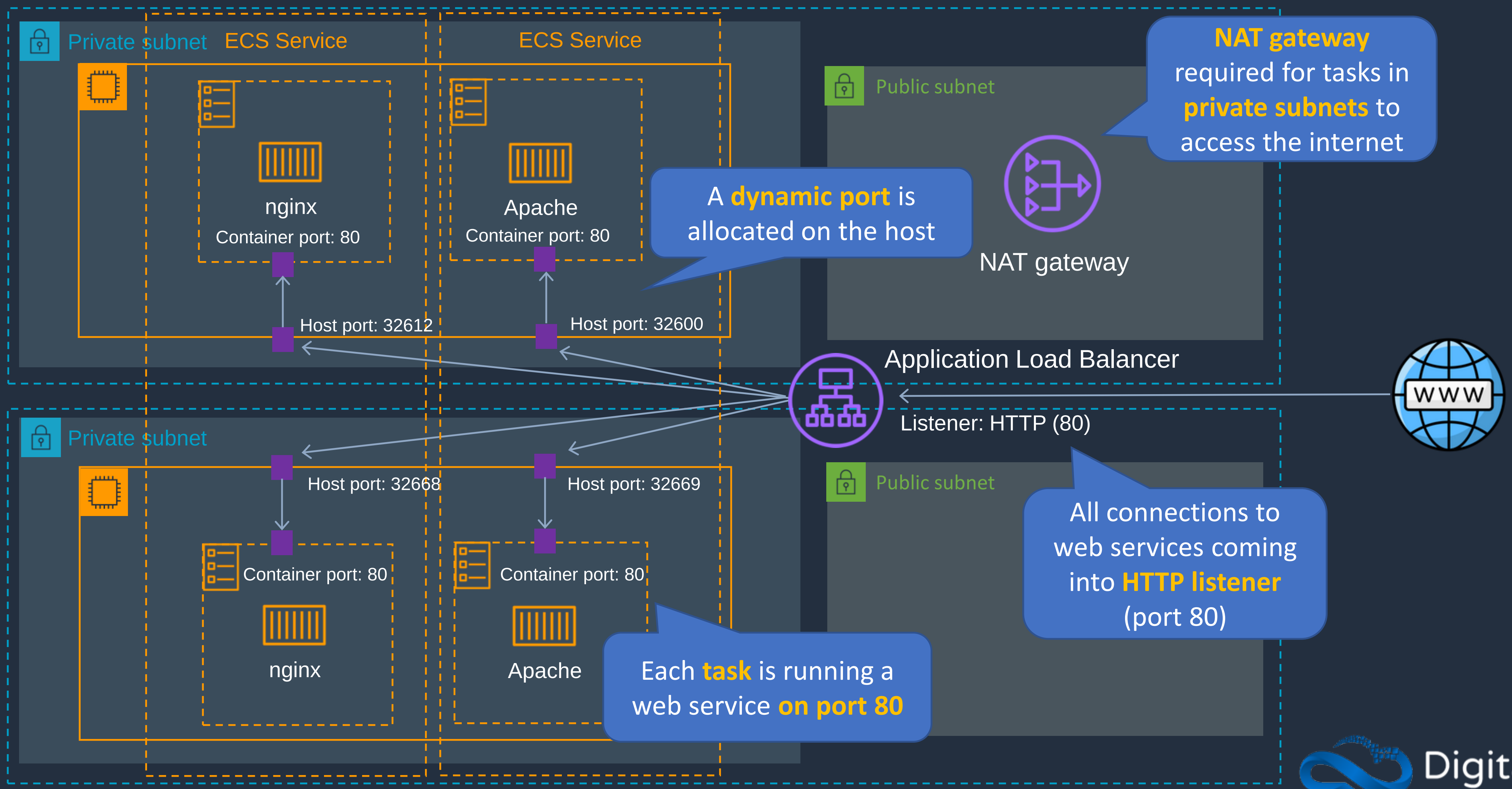


Cluster Auto Scaling

- Uses an ECS resource type called a **Capacity Provider**
- A Capacity Provider can be associated with an EC2 **Auto Scaling Group** (ASG)
- ASG can automatically scale using:
  - **Managed scaling** - with an automatically-created scaling policy on your ASG
  - **Managed instance termination protection** - which enables container-aware termination of instances in the ASG when scale-in happens

Amazon ECS with ALB

Amazon ECS with ALB



DigitalCloud TRAINING

AWS Fargate Blue-Green CI/CD Pipeline – Part 1

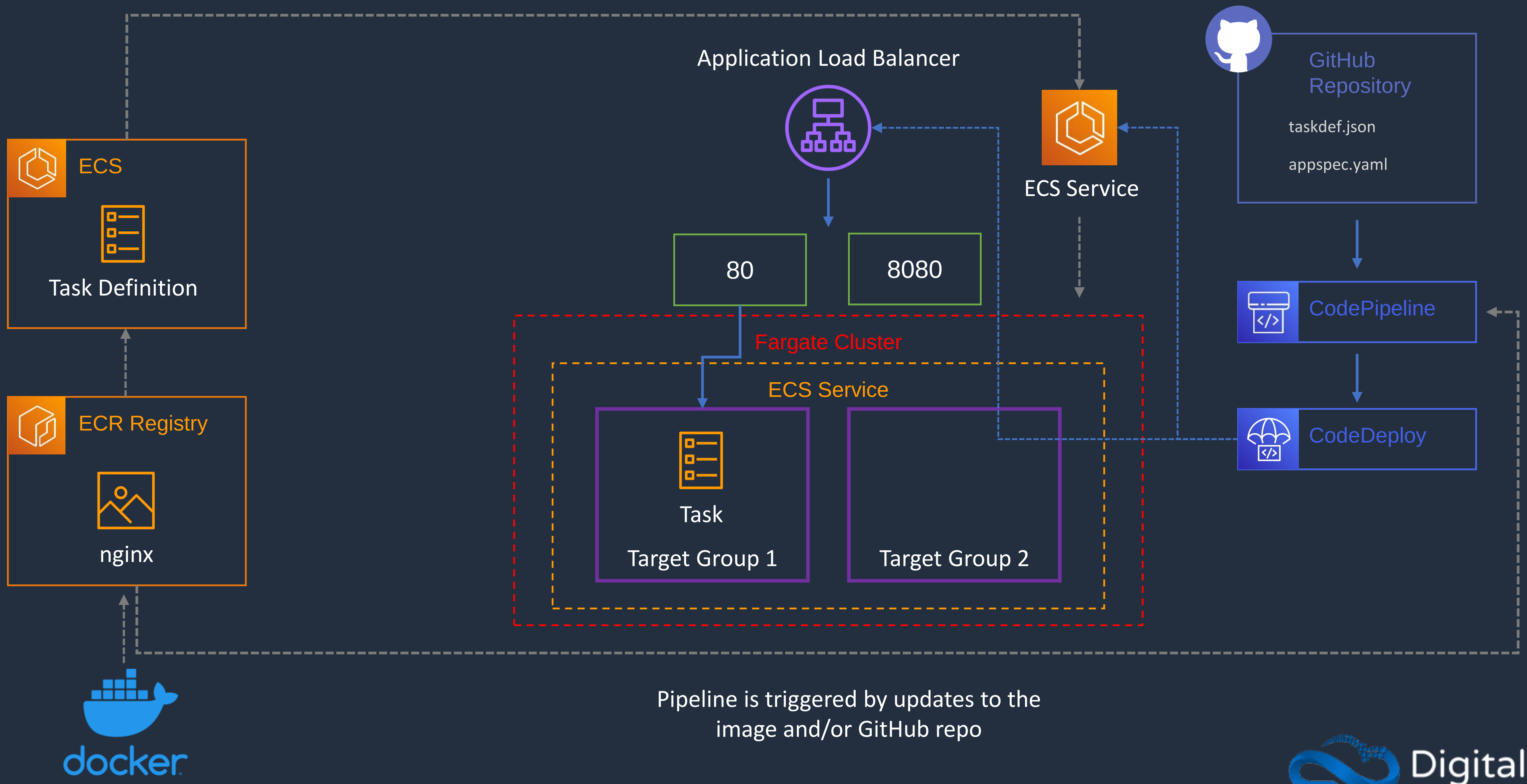
DigitalCloud TRAINING

AWS Fargate Blue-Green CI/CD Pipeline

Tasks we'll perform:

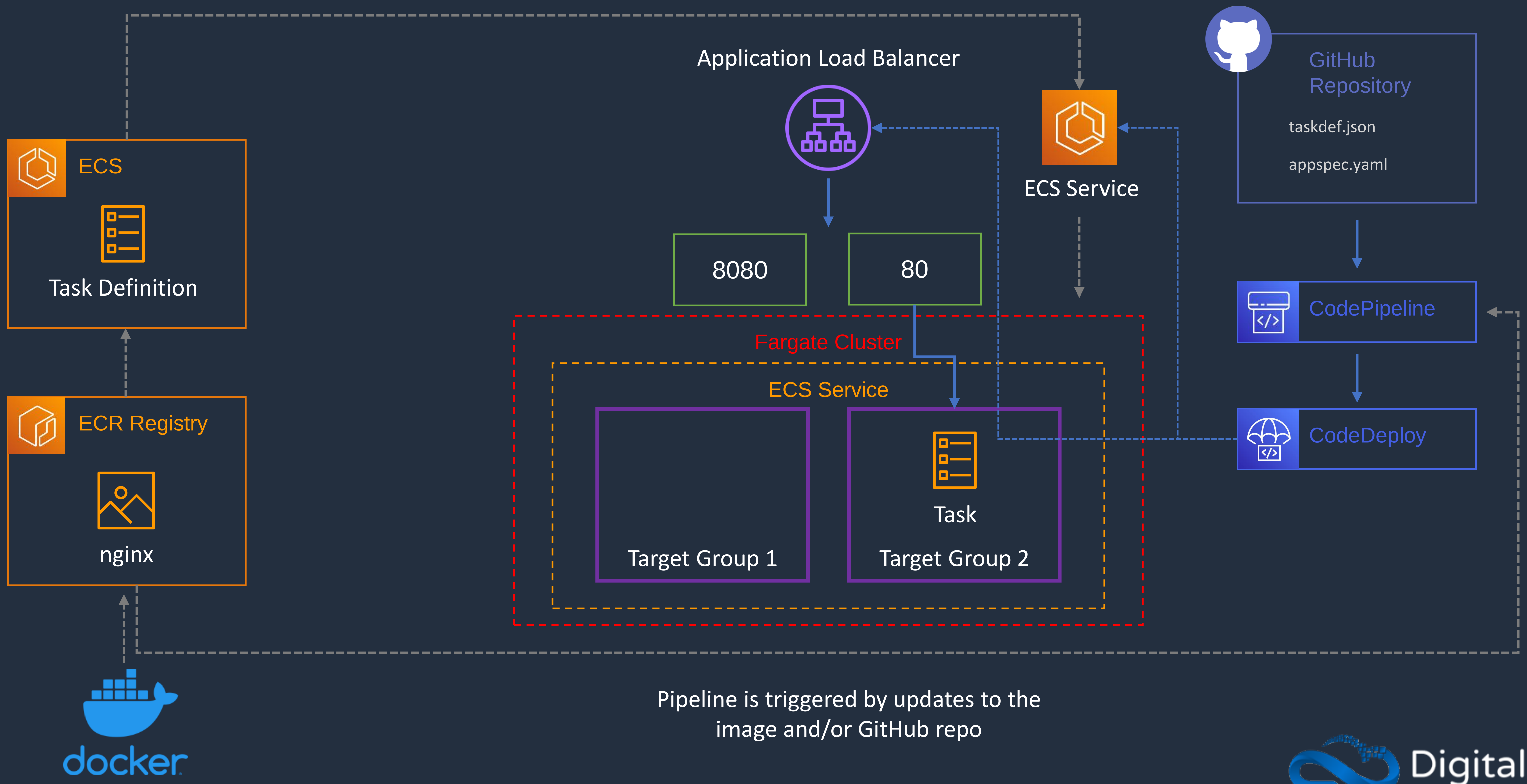
- Create an nginx container image using Docker and push to Amazon ECR
- Create an ALB with multiple listeners for blue/green updates
- Create an AWS Fargate cluster and service fronted by the ALB
- Create a pipeline for pushing code updates via GitHub using CodePipeline and CodeDeploy
- Implement a blue/green updates for the ECS application

AWS Fargate Blue-Green CI/CD Pipeline



Pipeline is triggered by updates to the image and/or GitHub repo

AWS Fargate Blue-Green CI/CD Pipeline

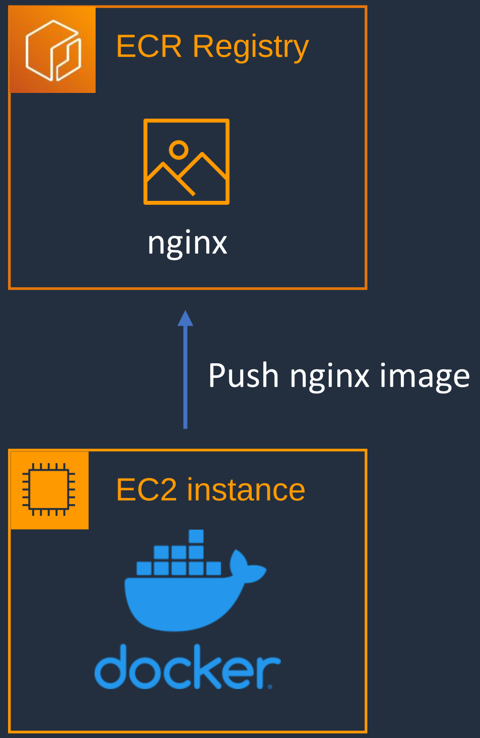


Pipeline is triggered by updates to the image and/or GitHub repo

AWS Fargate Blue-Green CI/CD Pipeline

1. **Create Image and Push to ECR Repository**
2. Create Task Definition and ALB
3. Create Fargate Cluster and Service
4. CodeDeploy Application and Pipeline
5. Implement Blue/Green Update to ECS

AWS Fargate Blue-Green CI/CD Pipeline



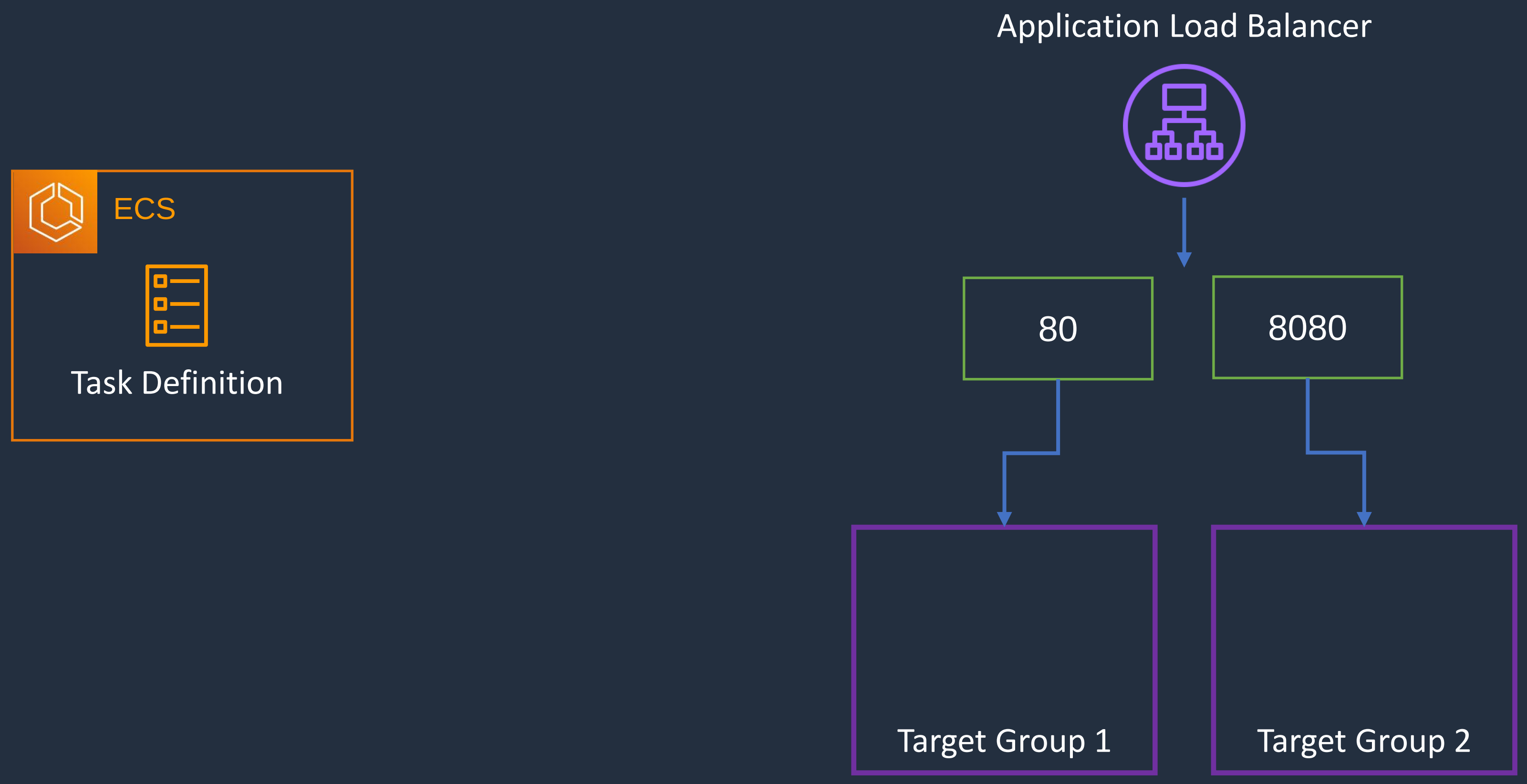
Can use AWS CloudShell instead of EC2 (non-persistent)

AWS Fargate Blue-Green CI/CD Pipeline – Part 2

AWS Fargate Blue-Green CI/CD Pipeline

1. Create Image and Push to ECR Repository
2. **Create Task Definition and ALB**
3. Create Fargate Cluster and Service
4. CodeDeploy Application and Pipeline
5. Implement Blue/Green Update to ECS

AWS Fargate Blue-Green CI/CD Pipeline

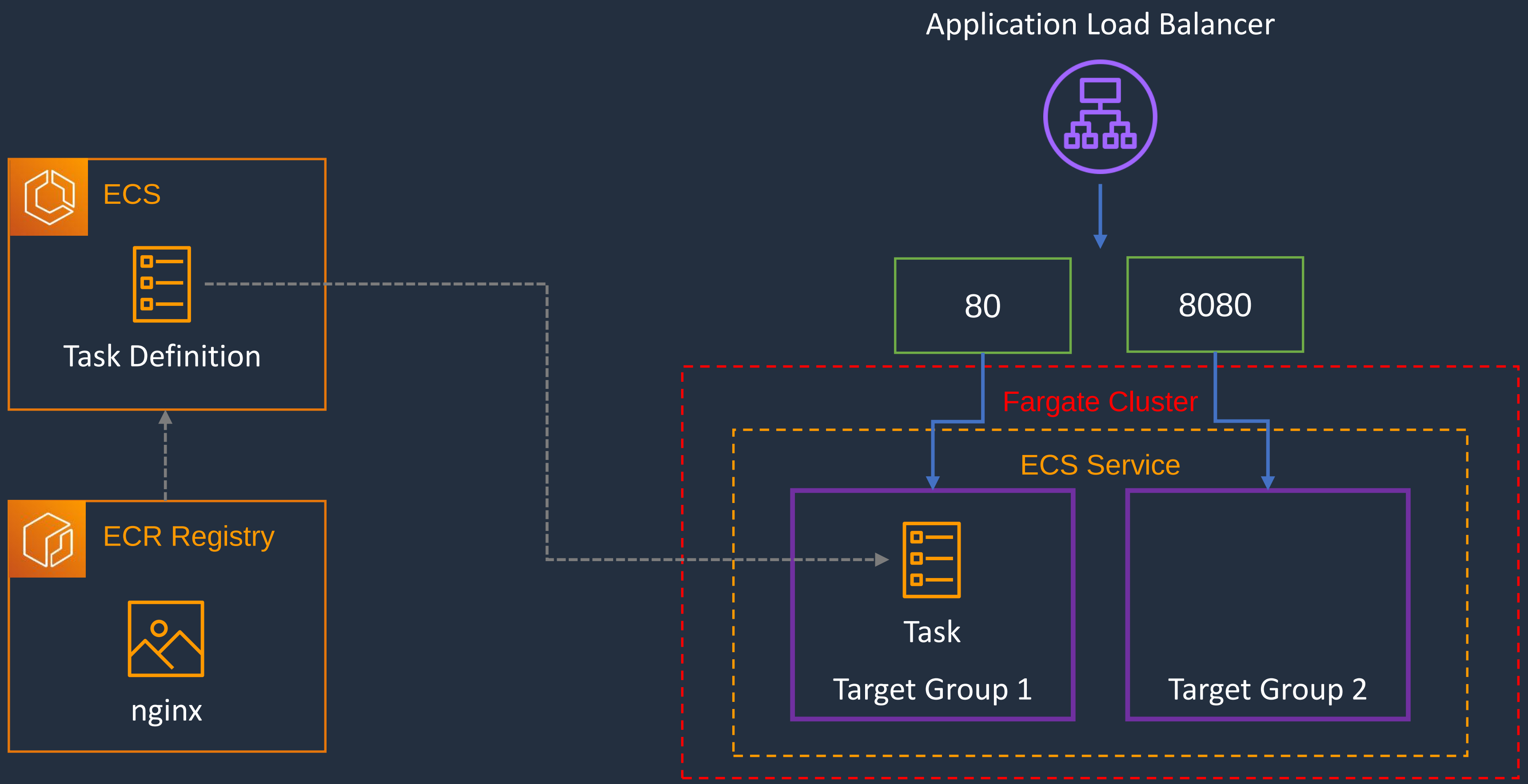


AWS Fargate Blue-Green CI/CD Pipeline – Part 3

AWS Fargate Blue-Green CI/CD Pipeline

1. Create Image and Push to ECR Repository
2. Create Task Definition and ALB
3. **Create Fargate Cluster and Service**
4. CodeDeploy Application and Pipeline
5. Implement Blue/Green Update to ECS

AWS Fargate Blue-Green CI/CD Pipeline

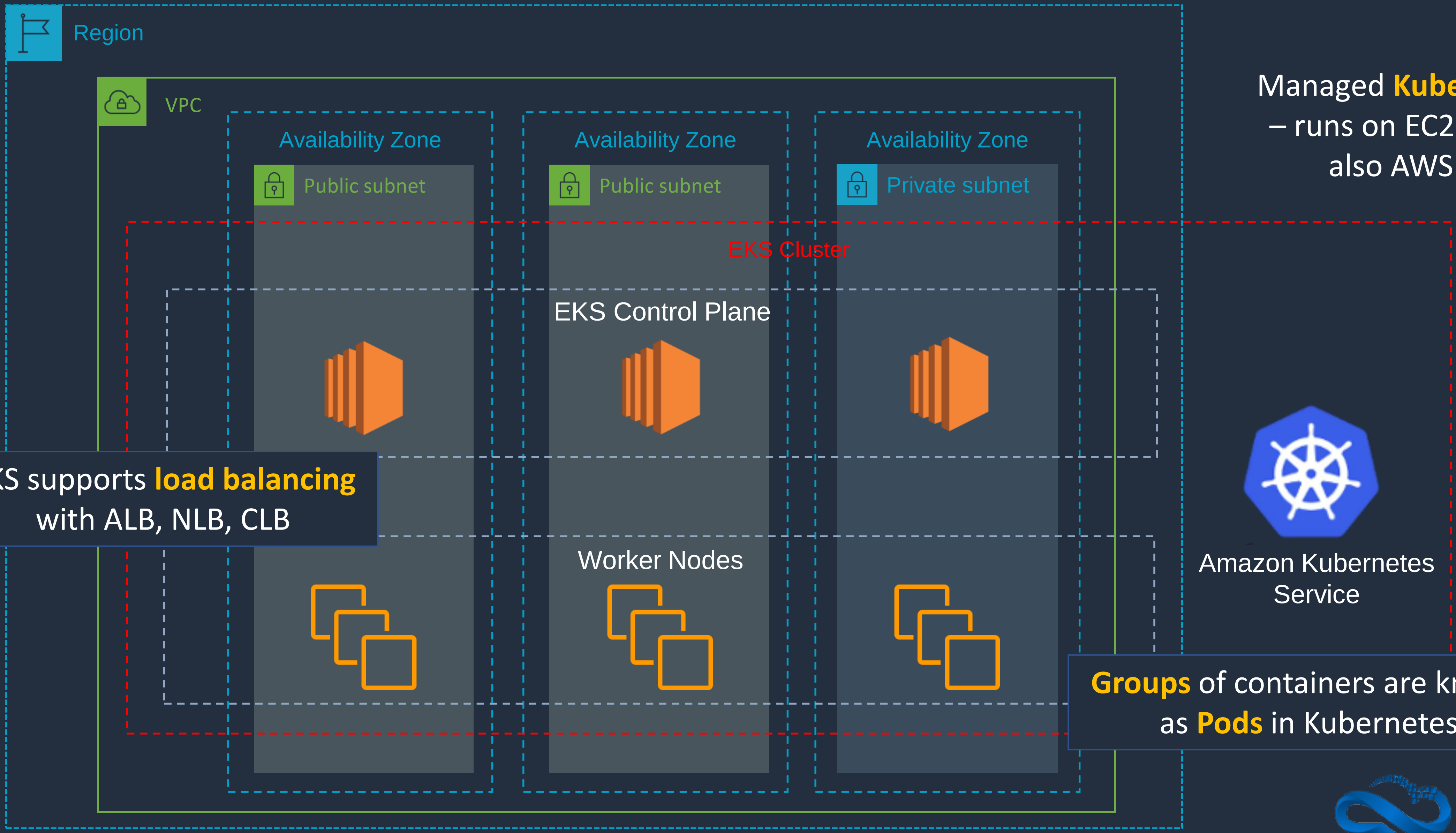


Amazon Elastic Kubernetes Service (EKS)

Amazon EKS

- Amazon Elastic Kubernetes Service (Amazon EKS) is a managed service for running Kubernetes applications in the cloud or on-premises
- Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications
- Use when you need to **standardize** container orchestration across multiple environments using a **managed Kubernetes** implementation
- Features:
  - **Hybrid Deployment** - manage Kubernetes clusters and applications across hybrid environments (AWS + On-premises)
  - **Batch Processing** - run sequential or parallel batch workloads on your EKS cluster using the Kubernetes Jobs API. Plan, schedule and execute batch workloads
  - **Machine Learning** - use Kubeflow with EKS to model your machine learning workflows and efficiently run distributed training jobs using the latest EC2 GPU-powered instances, including Inferentia
  - **Web Applications** - build web applications that automatically scale up and down and run in a highly available configuration across multiple Availability Zones

Amazon EKS



Managed **Kubernetes service** – runs on EC2 / Fargate and also AWS Outposts

EKS supports **load balancing** with ALB, NLB, CLB

Groups of containers are known as **Pods** in Kubernetes

Amazon EKS Auto Scaling

Workload Auto Scaling:

- **Vertical Pod Autoscaler** - automatically adjusts the CPU and memory reservations for your pods to help "right size" your applications
- **Horizontal Pod Autoscaler** - automatically scales the number of pods in a deployment, replication controller, or replica set based on that resource's CPU utilization

Cluster Auto Scaling:

- Amazon EKS supports two **autoscaling** products:
  - Kubernetes Cluster Autoscaler
  - Karpenter open source autoscaling project
- The cluster autoscaler uses AWS scaling groups, while Karpenter works directly with the Amazon EC2 fleet

Amazon EKS and Elastic Load Balancing

- Amazon EKS supports Network Load Balancers and Application Load Balancers
- The AWS Load Balancer Controller manages AWS Elastic Load Balancers for a Kubernetes cluster
- Install the AWS Load Balancer Controller using Helm V3 or later or by applying a Kubernetes manifest
- The controller provisions the following resources:
  - An AWS Application Load Balancer (ALB) when you create a Kubernetes Ingress
  - An AWS Network Load Balancer (NLB) when you create a Kubernetes service of type LoadBalancer
- In the past, the Kubernetes network load balancer was used for instance targets, but the AWS Load balancer Controller was used for IP targets
- With the AWS Load Balancer Controller version 2.3.0 or later, you can create NLBs using either target type

Amazon EKS Distro

- Amazon EKS Distro is a distribution of Kubernetes with the same dependencies as Amazon EKS
- Allows you to manually run Kubernetes clusters anywhere
- EKS Distro includes binaries and containers of open-source Kubernetes, etcd, networking, and storage plugins, tested for compatibility
- You can securely access EKS Distro releases as open source on GitHub or within AWS via Amazon S3 and Amazon ECR
- Amazon EKS Distro alleviates the need to track updates, determine compatibility, and standardize on a common Kubernetes version across distributed teams
- You can create Amazon EKS Distro clusters in AWS on Amazon EC2 and on your own on-premises hardware using the tooling of your choice

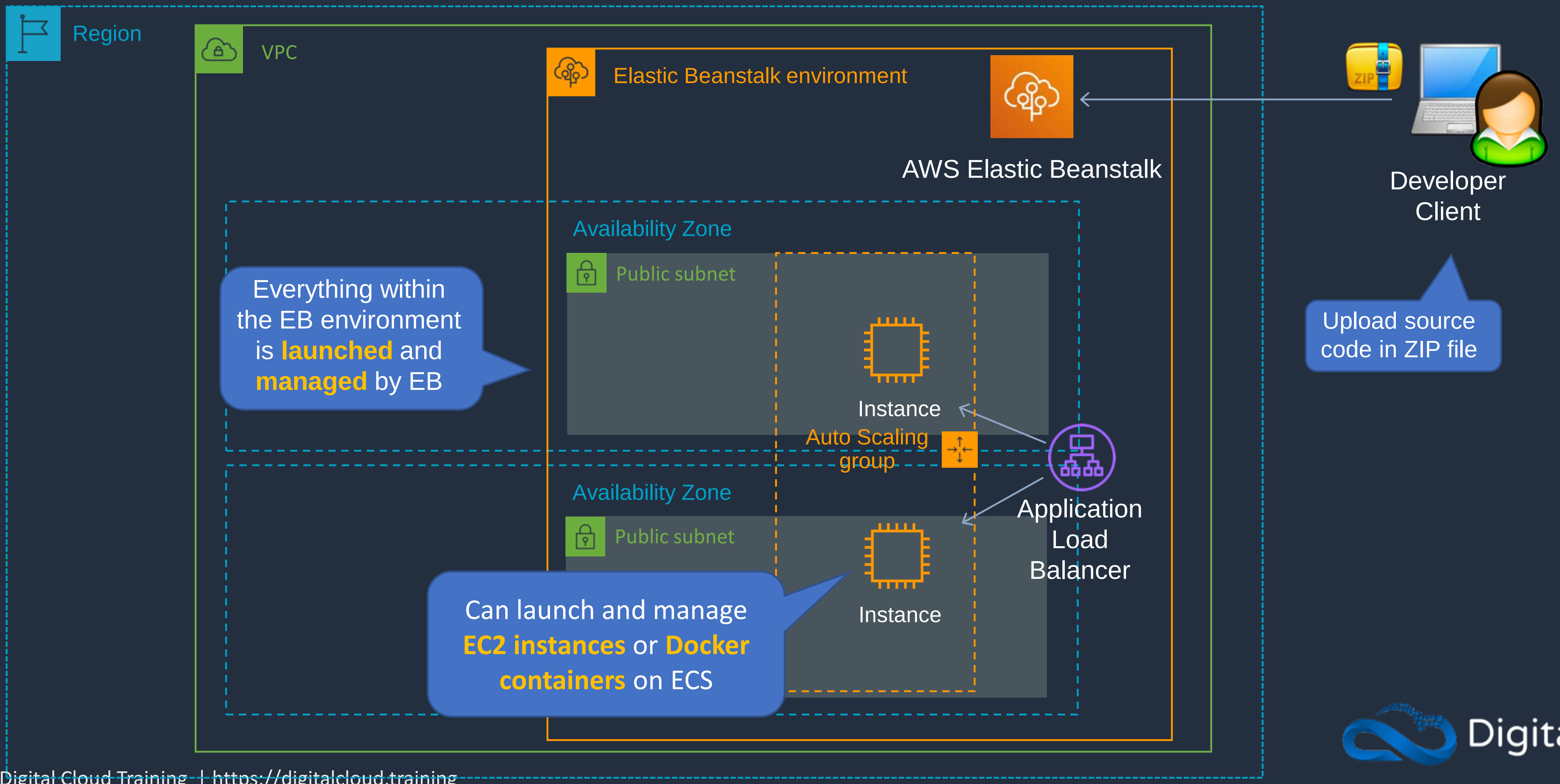
Amazon ECS and EKS Anywhere

- Run ECS or EKS on customer-managed infrastructure, supported by AWS
- Customers can run Amazon ECS/EKS Anywhere on their own on-premises infrastructure on bare metal servers
- You can also deploy ECS/EKS Anywhere using VMware vSphere

Deploying and Scaling an Amazon EKS Cluster

AWS Elastic Beanstalk Core Knowledge

AWS Elastic Beanstalk



AWS Elastic Beanstalk

- Supports Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker web applications
- Supports the following languages and development stacks:
  - Apache Tomcat for Java applications
  - Apache HTTP Server for PHP applications
  - Apache HTTP Server for Python applications
  - Nginx or Apache HTTP Server for Node.js applications
  - Passenger or Puma for Ruby applications
  - Microsoft IIS 7.5, 8.0, and 8.5 for .NET applications
  - Java SE
  - Docker
  - Go

AWS Elastic Beanstalk

There are several **layers**

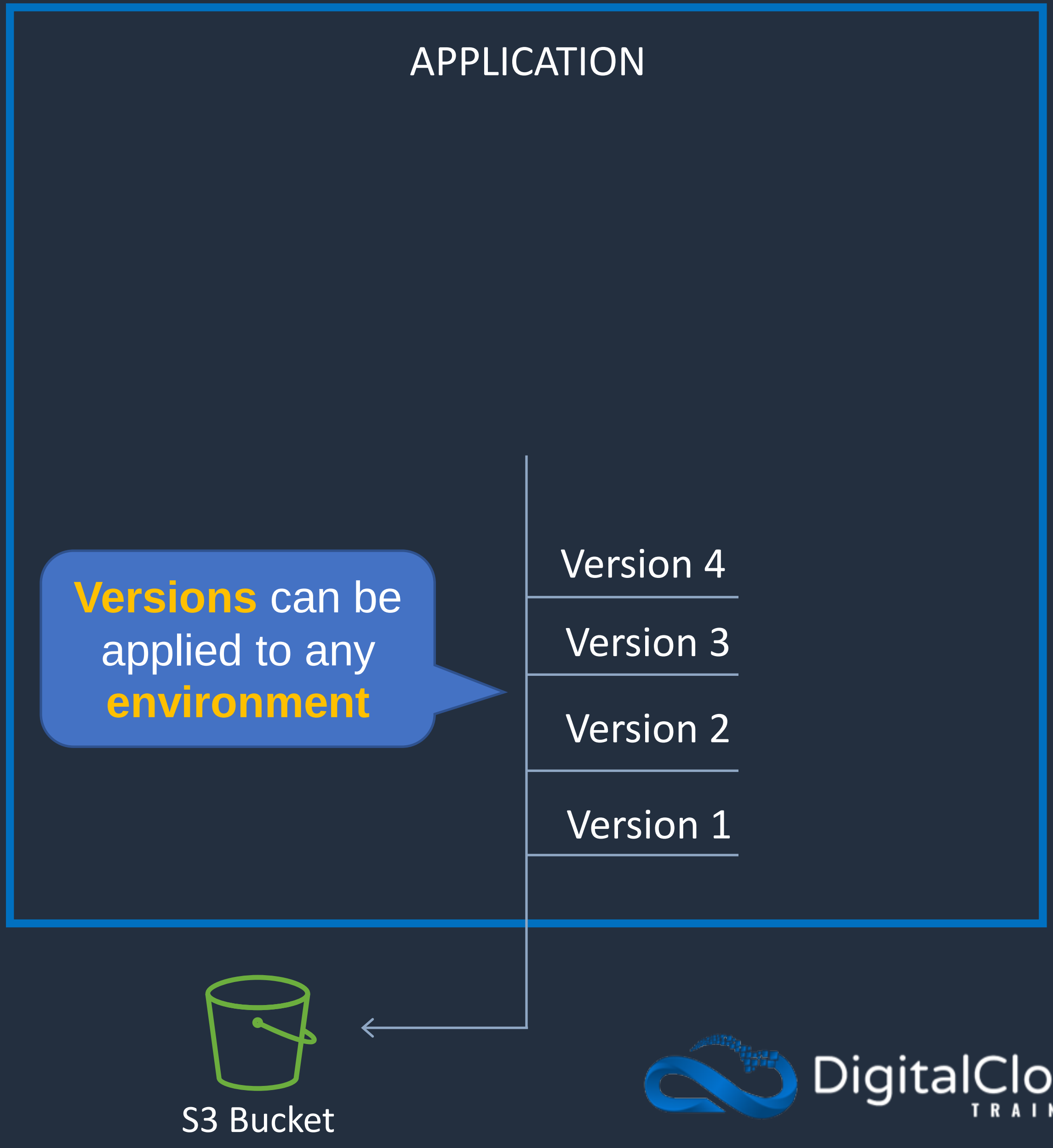
Applications:

- Contain environments, environment configurations, and application versions
- You can have multiple application versions held within an application

AWS Elastic Beanstalk

Application version

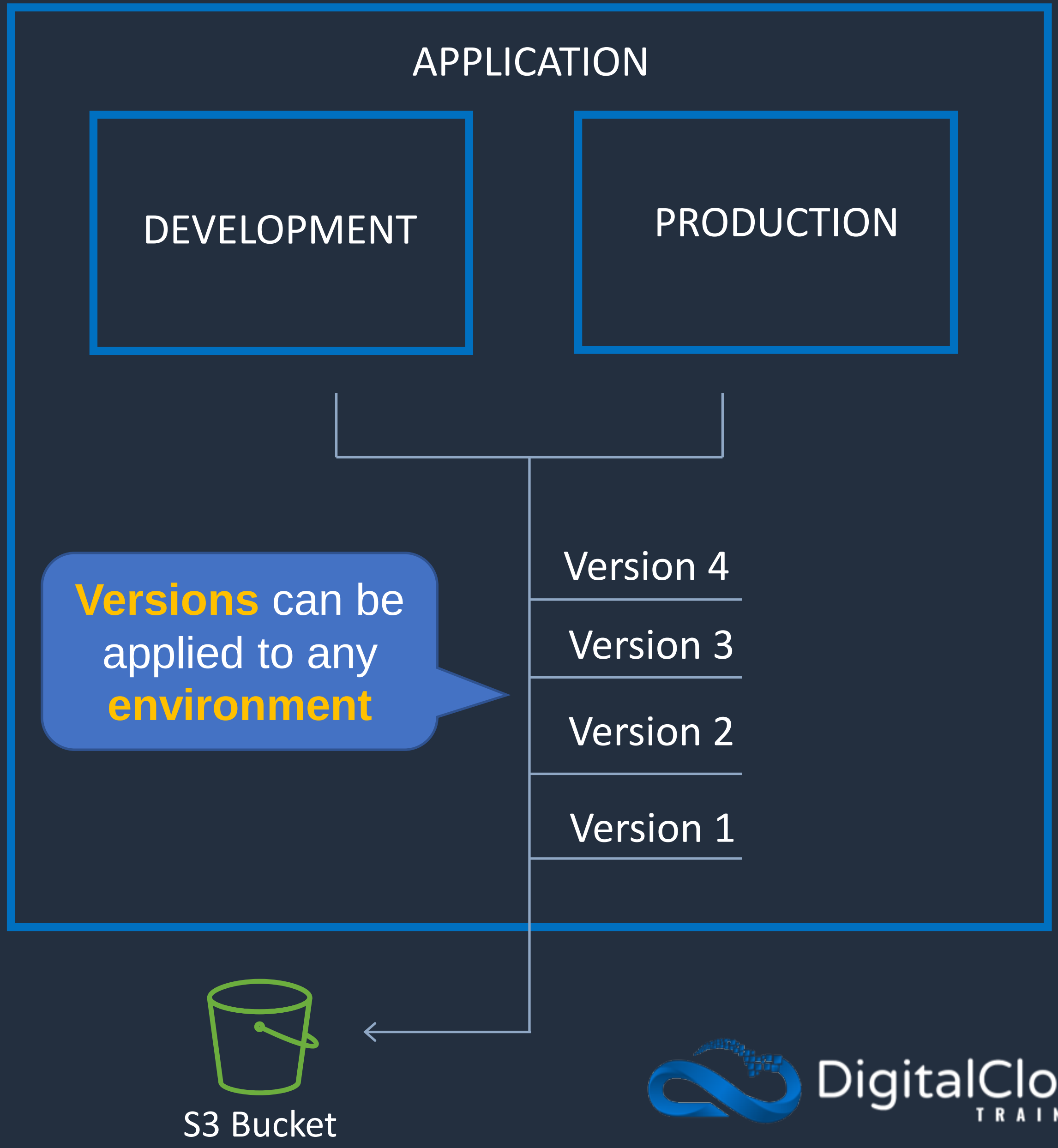
- A specific reference to a section of deployable code
- The application version will point typically to an Amazon S3 bucket containing the code



AWS Elastic Beanstalk

Environments:

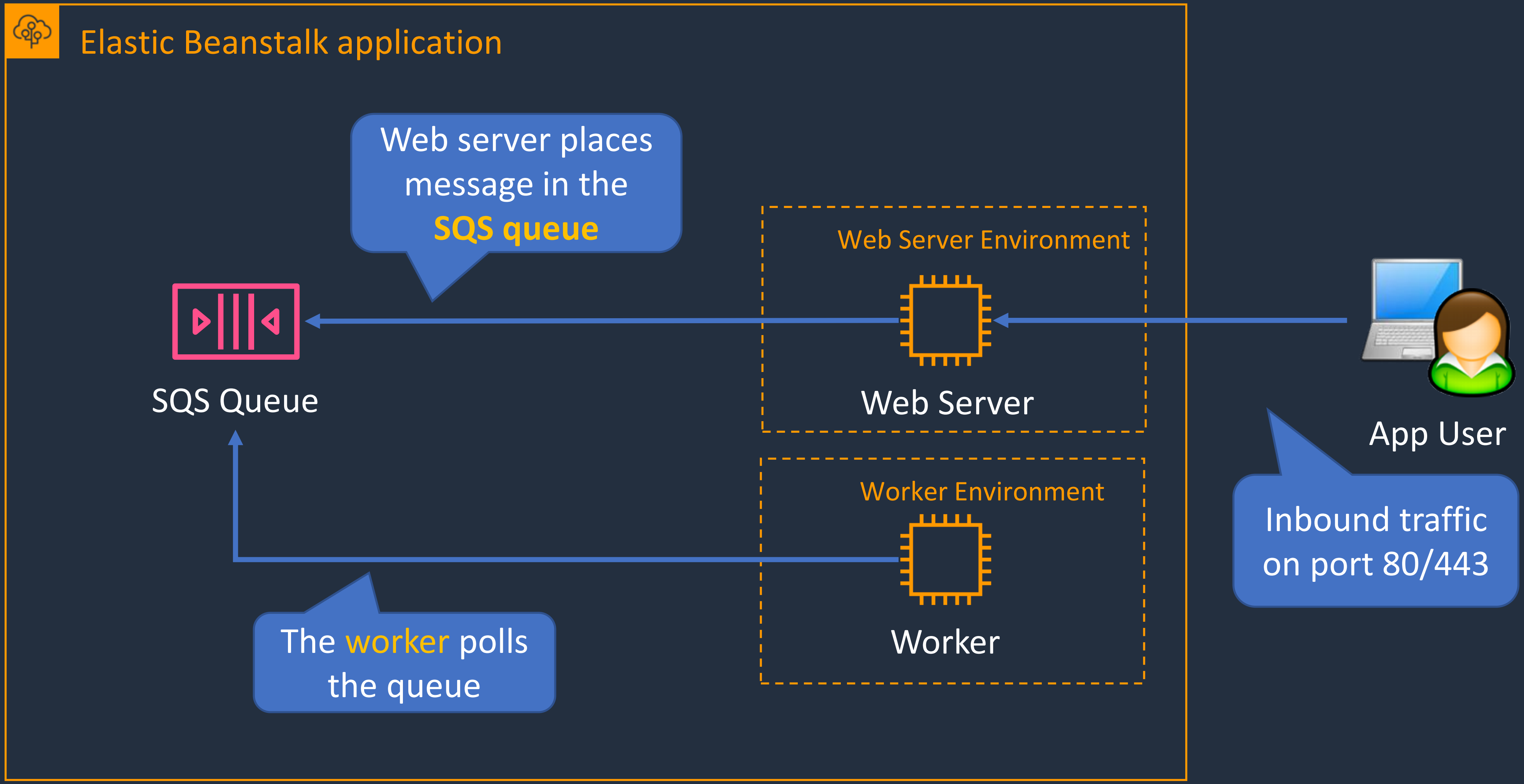
- An application version that has been deployed on AWS resources
- The resources are configured and provisioned by AWS Elastic Beanstalk
- The environment is comprised of all the resources created by Elastic Beanstalk and not just an EC2 instance with your uploaded code



Web Servers and Workers

- **Web servers** are standard applications that listen for and then process HTTP requests, typically over port 80
- **Workers** are specialized applications that have a background processing task that listens for messages on an Amazon SQS queue
- **Workers** should be used for long-running tasks

AWS Elastic Beanstalk

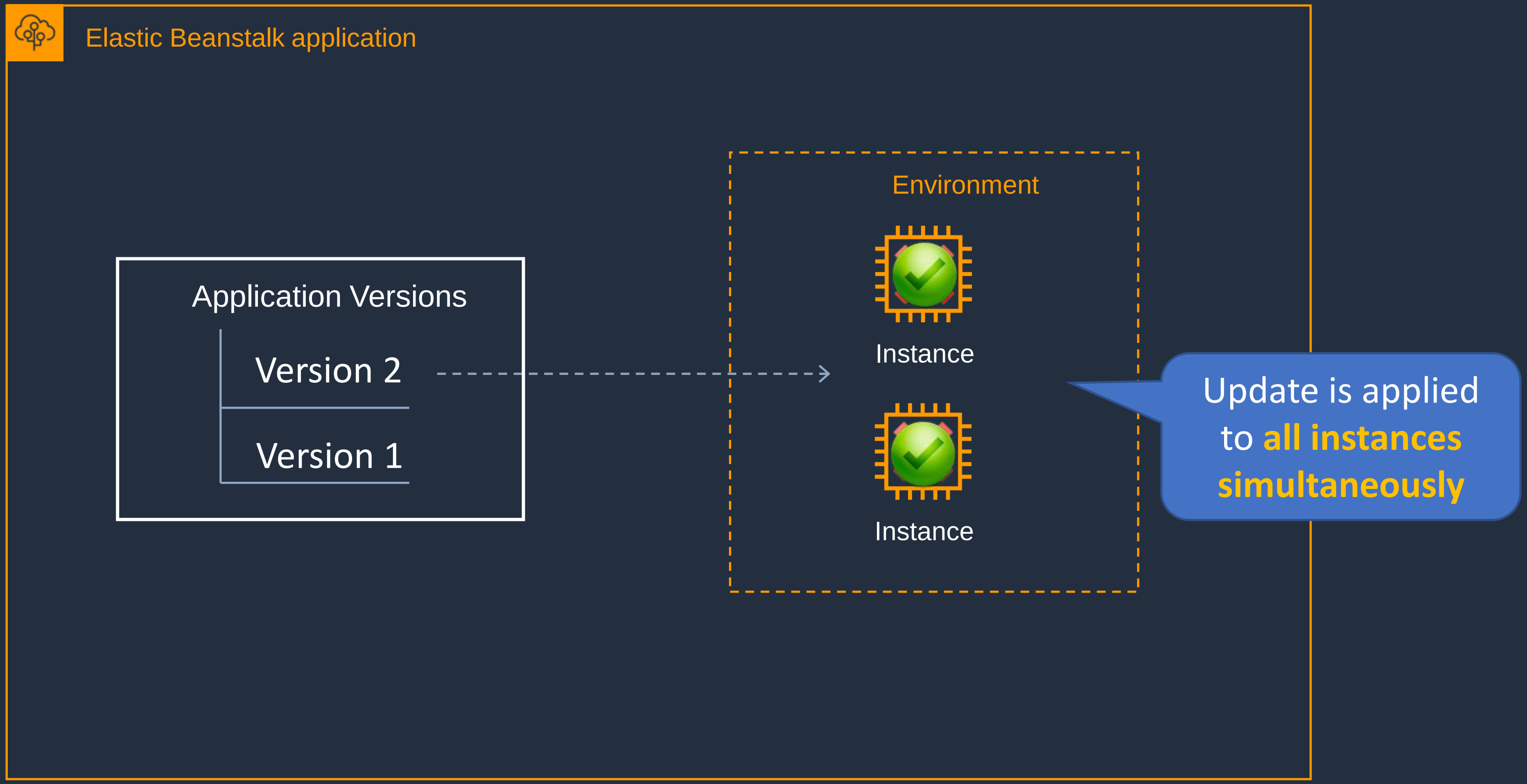


Updating Elastic Beanstalk Applications

Elastic Beanstalk Deployment Policies

- **All at once** - Deploys the new version to all instances simultaneously
- **Rolling** - Update a batch of instances, and then move onto the next batch once the first batch is healthy
- **Rolling with additional batch** - Like Rolling but launches new instances in a batch ensuring that there is full availability
- **Immutable** - Launches new instances in a new ASG and deploys the version update to these instances before swapping traffic to these instances once healthy
- **Blue/green** - Create a new "stage" environment and deploy updates there

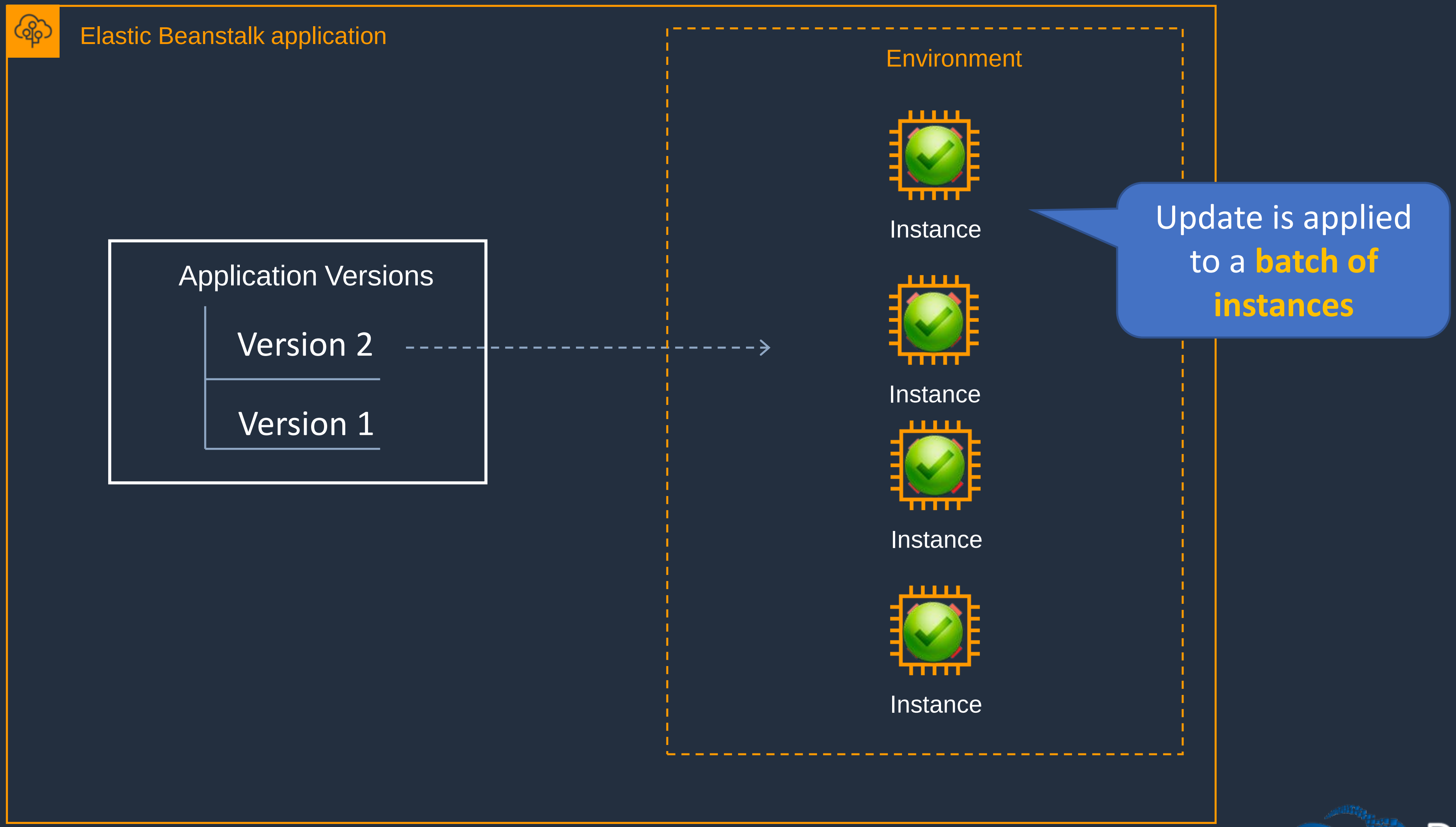
All at once update



All at once update

- Deploys the new version to all instances simultaneously
- All of your instances are out of service while the deployment takes place
- Fastest deployment
- Good for quick iterations in development environment
- You will experience an outage while the deployment is taking place - not ideal for mission-critical systems
- If the update fails, you need to roll back the changes by re-deploying the original version to all of your instances
- No additional cost

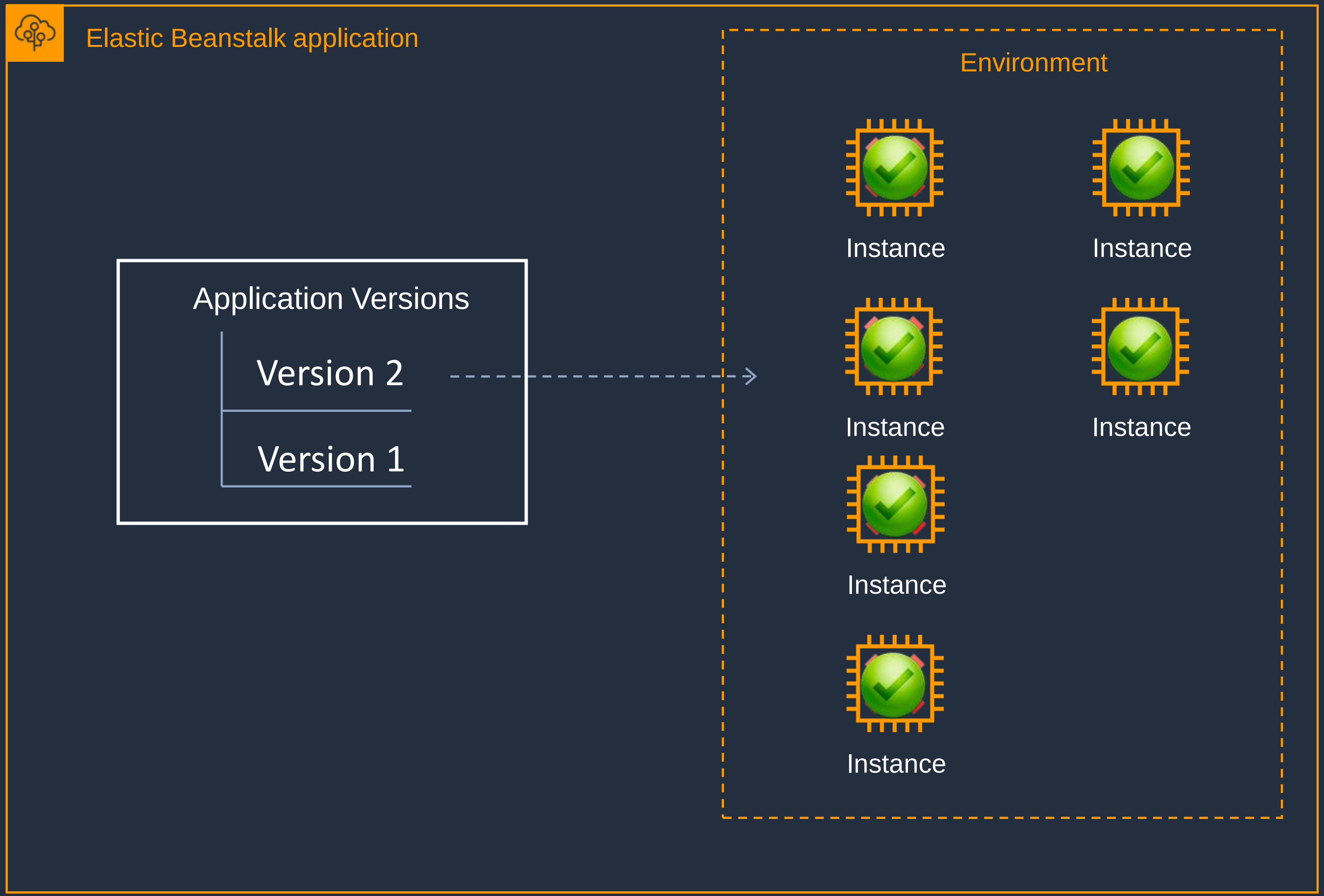
Rolling Update



Rolling Update

- Update a few instances at a time (batch), and then move onto the next batch once the first batch is healthy (downtime for 1 batch at a time)
- Application is running both versions simultaneously
- Each batch of instances is taken out of service while the deployment takes place
- Your environment capacity will be reduced by the number of instances in a batch while the deployment takes place
- Not ideal for performance-sensitive systems
- If the update fails, you need to perform an additional rolling update to roll back the changes
- No additional cost
- Long deployment time

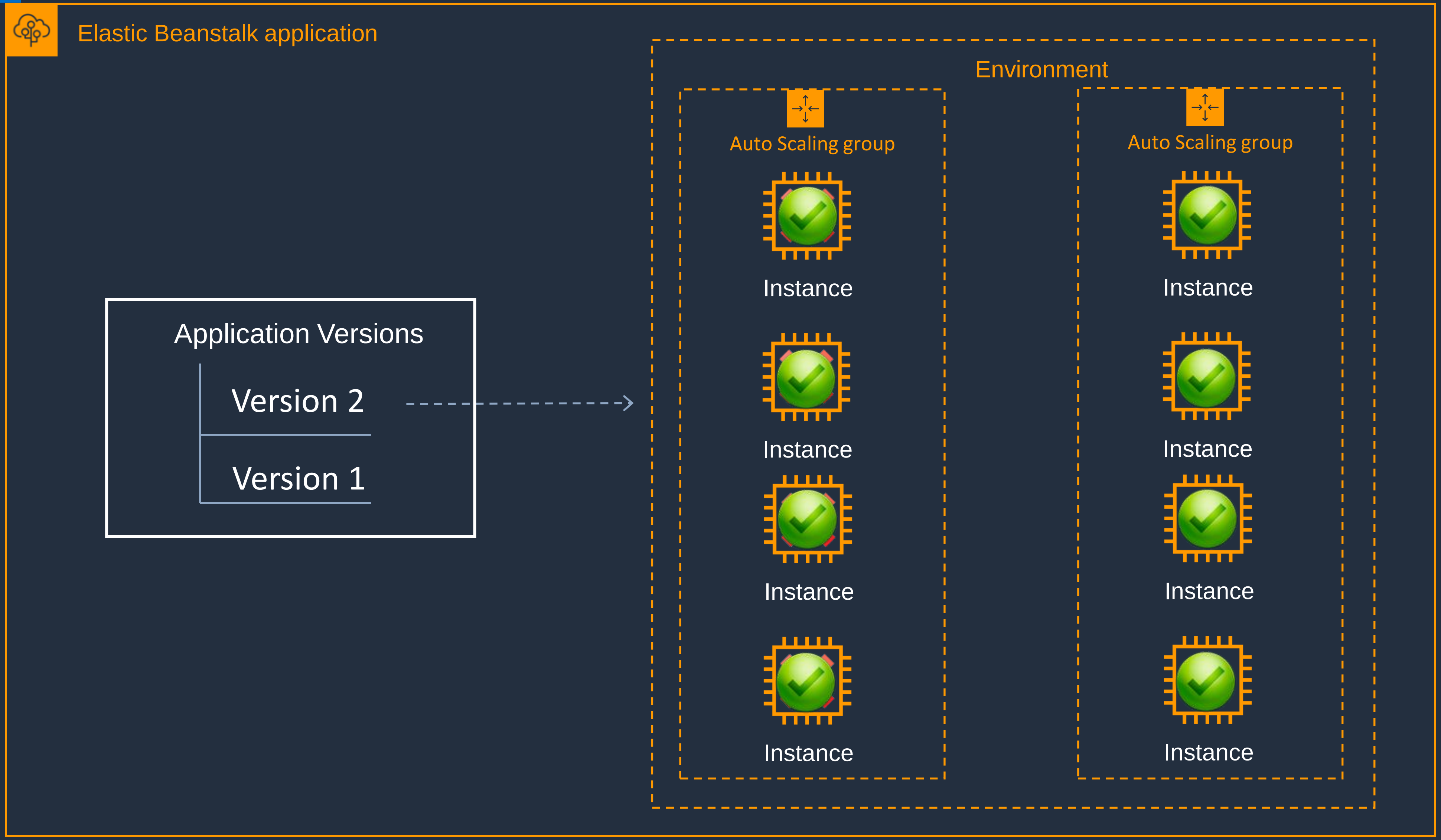
Rolling with Additional Batch Update



Rolling with Additional Batch Update

- Like Rolling but launches new instances in a batch ensuring that there is full availability
- Application is running at capacity
- Can set the batch size
- Application is running both versions simultaneously
- Small additional cost
- Additional batch is removed at the end of the deployment
- Longer deployment
- Good for production environments

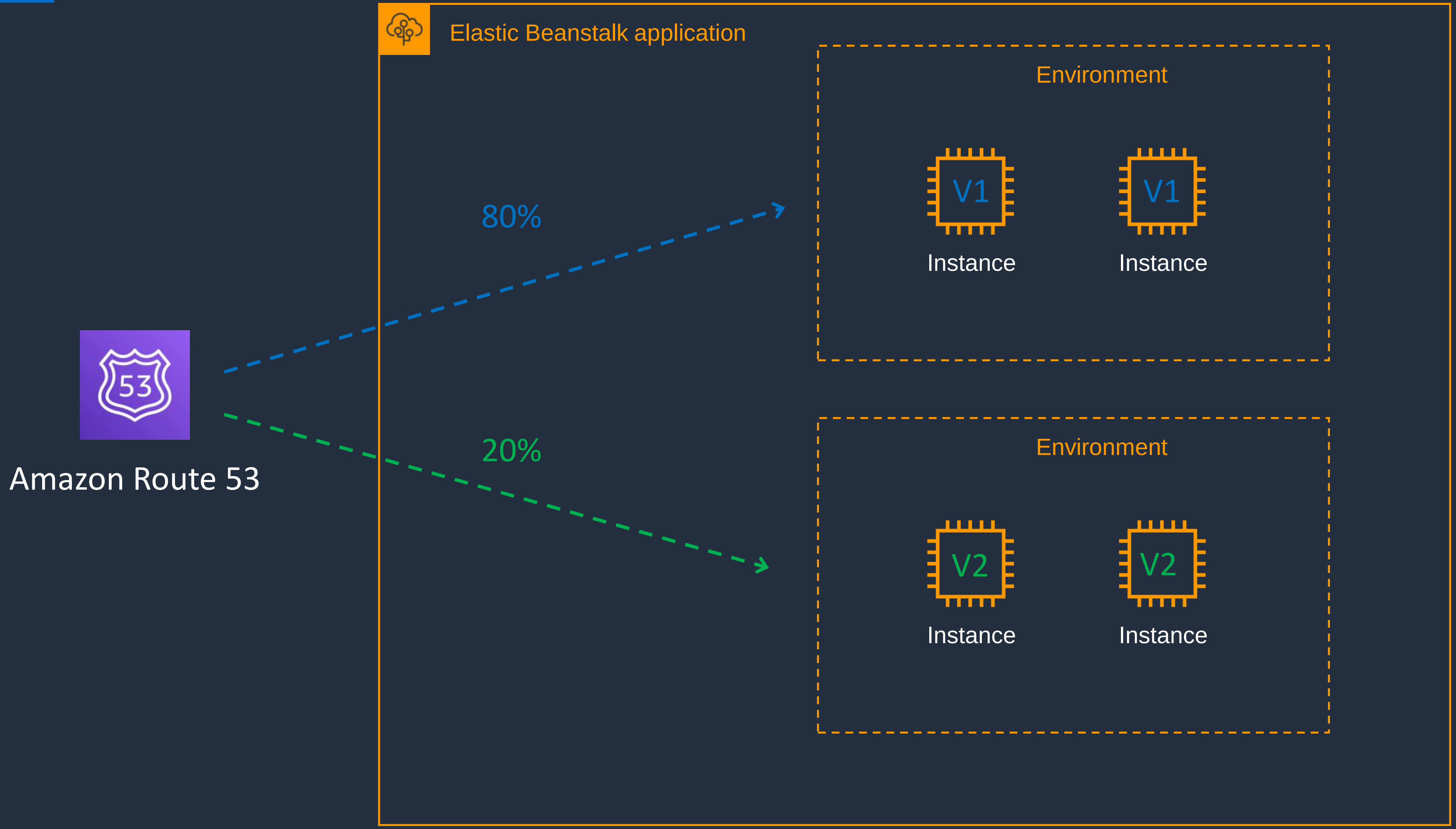
Immutable Update



Immutable Update

- Launches new instances in a new ASG and deploys the version update to these instances before swapping traffic to these instances once healthy
- Zero downtime
- New code is deployed to new instances using an ASG
- High cost as double the number of instances running during updates
- Longest deployment
- Quick rollback in case of failures
- Great for production environments

Blue/Green Update



Blue/Green Update

- This is not a feature within Elastic Beanstalk
- You create a new "staging" environment and deploy updates there
- The new environment (green) can be validated independently, and you can roll back if there are issues
- Route 53 can be setup using weighted policies to redirect a a percentage of traffic to the staging environment
- Using Elastic Beanstalk, you can "swap URLs" when done with the environment test
- Zero downtime

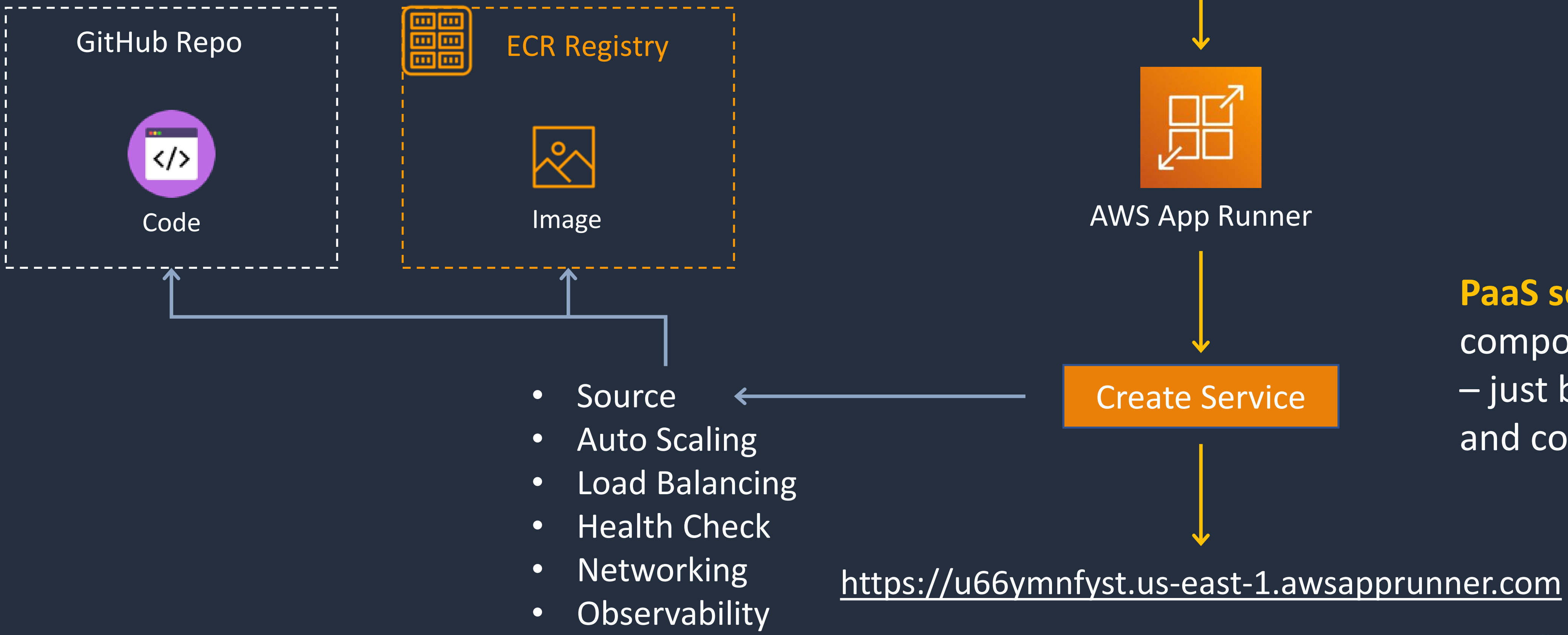
Creating and Updating Environments

AWS App Runner

AWS App Runner

Fully managed service for deploying **containerized** web apps and APIs

```
aws apprunner create-service --service-name my-simple-service --source-configuration '
{"ImageRepository": {"ImageIdentifier": "111111111111.dkr.ecr.us-east-1.amazonaws.com/nginx:latest",
"ImageRepositoryType": "ECR_PUBLIC"}}'
```



PaaS solution with all components managed – just bring your code and container image!

Architecture Patterns – Containers and PaaS

Architecture Patterns – Containers and PaaS

Requirement	Solution
Company plans to deploy Docker containers on AWS at the lowest cost	Use Amazon ECS with a cluster of Spot instances and enable Spot instance draining
Company plans to migrate Docker containers to AWS and does not want to manage operating systems	Migrate to Amazon ECS using the Fargate launch type
Fargate task is launched in a private subnet and fails with error "CannotPullContainer"	Disable auto-assignment of public IP addresses and configure a NAT gateway

Architecture Patterns – Containers and PaaS

Requirement	Solution
Application will be deployed on Amazon ECS and must scale based on memory	Use service auto-scaling and use the memory utilization
Application will run on Amazon ECS tasks across multiple hosts and needs access to an Amazon S3 bucket	Use a task execution IAM role to provide permissions to S3 bucket.
Company requires standard Docker container automation and management service to be used across multiple environments	Deploy Amazon EKS

Architecture Patterns – Containers and PaaS

Requirement	Solution
Company needs to move many simple web apps running on PHP, Java, and Ruby to AWS. Utilization is very low	Deploy to single-instance Elastic Beanstalk environments
Business critical application running on Elastic Beanstalk must be updated. Require zero downtime and quick and complete rollback	Update using an immutable update with a new ASG and swap traffic
A development application running on Elastic Beanstalk needs a cost-effective and quick update. Downtime is acceptable	Use an all-at-once update

Architecture Patterns – Containers and PaaS

Requirement

Need a managed environment for running a simple web application. App processes incoming data which can take several minutes per task

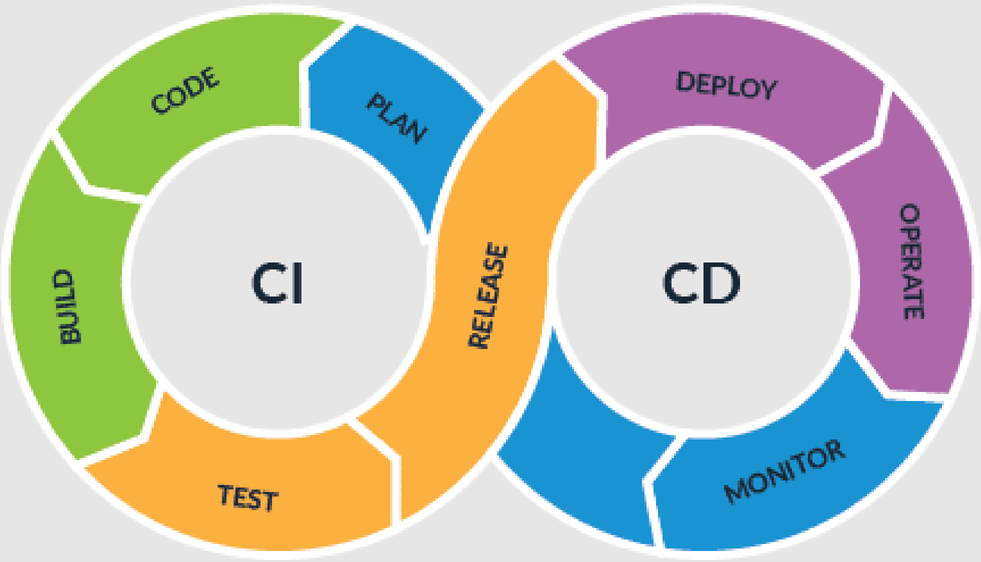
Solution

Use an Elastic Beanstalk environment with a web server for the app front-end and a decoupled worker tier for the long running process

SECTION 13

Deployment and Management

CI/CD Overview



DigitalCloud TRAINING

Continuous Integration

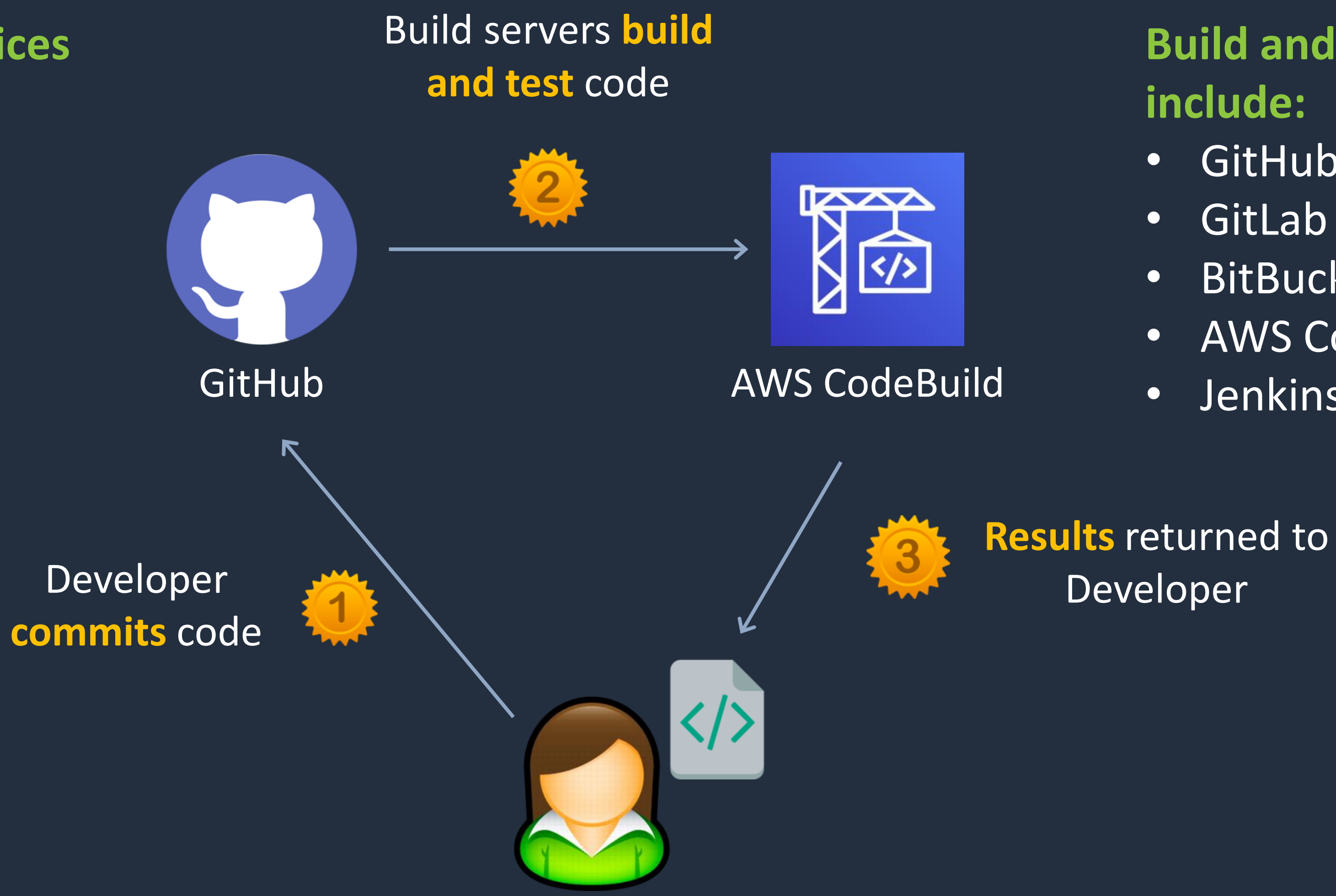
Continuous Integration (CI):

- Developers frequently merge code into a shared repository
- Each merge triggers automated builds and tests to catch issues early
- Helps detect integration problems sooner rather than later

Continuous Integration

Source code services include:

- GitHub
- GitLab
- Bitbucket
- Azure Repos



Build and test services include:

- GitHub Actions
- GitLab CI/CD
- BitBucket Pipelines
- AWS CodeBuild
- Jenkins

Continuous Delivery / Deployment

Continuous Delivery (CD)

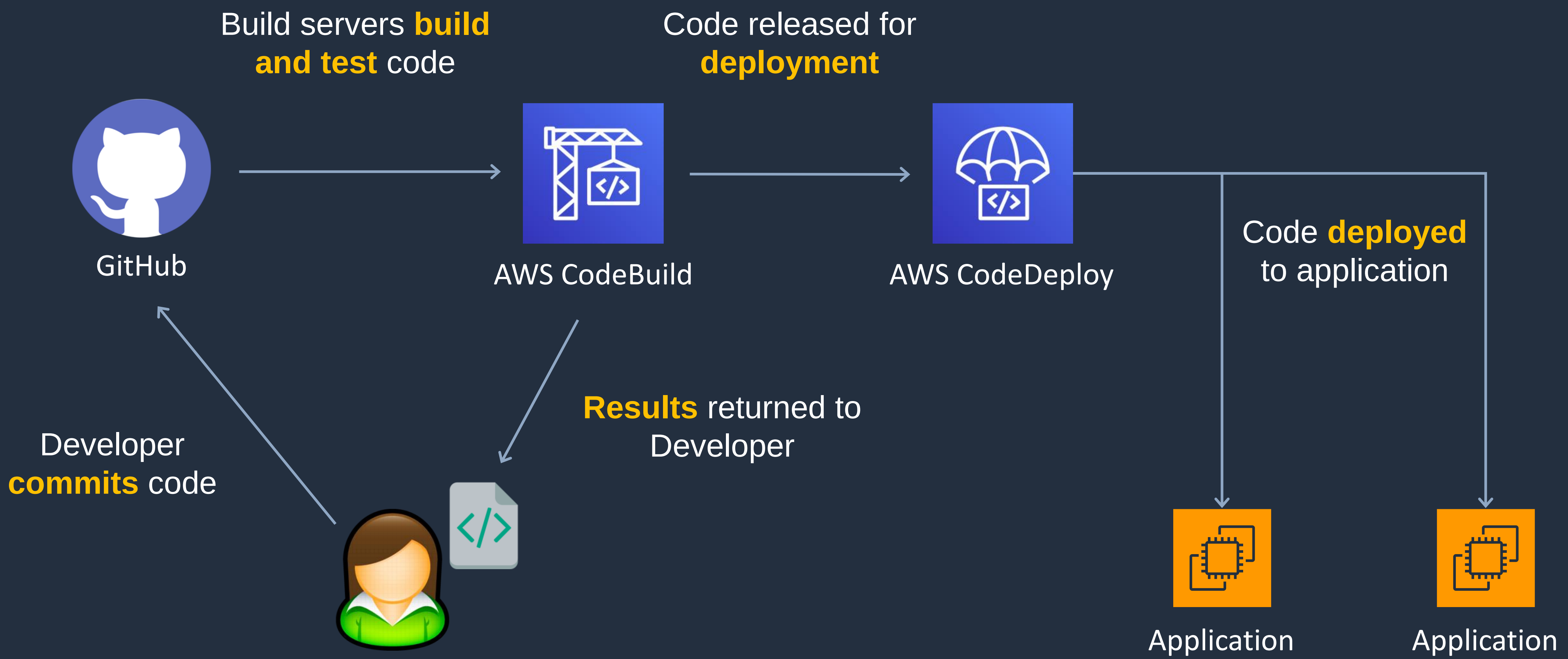
- Ensures that code is always in a deployable state after passing automated tests
- After passing tests, code must be manually approved before deployment to production or staging
- Automates everything except the final release step
- Allows teams to decide when to deploy rather than deploying automatically

Continuous Delivery / Deployment

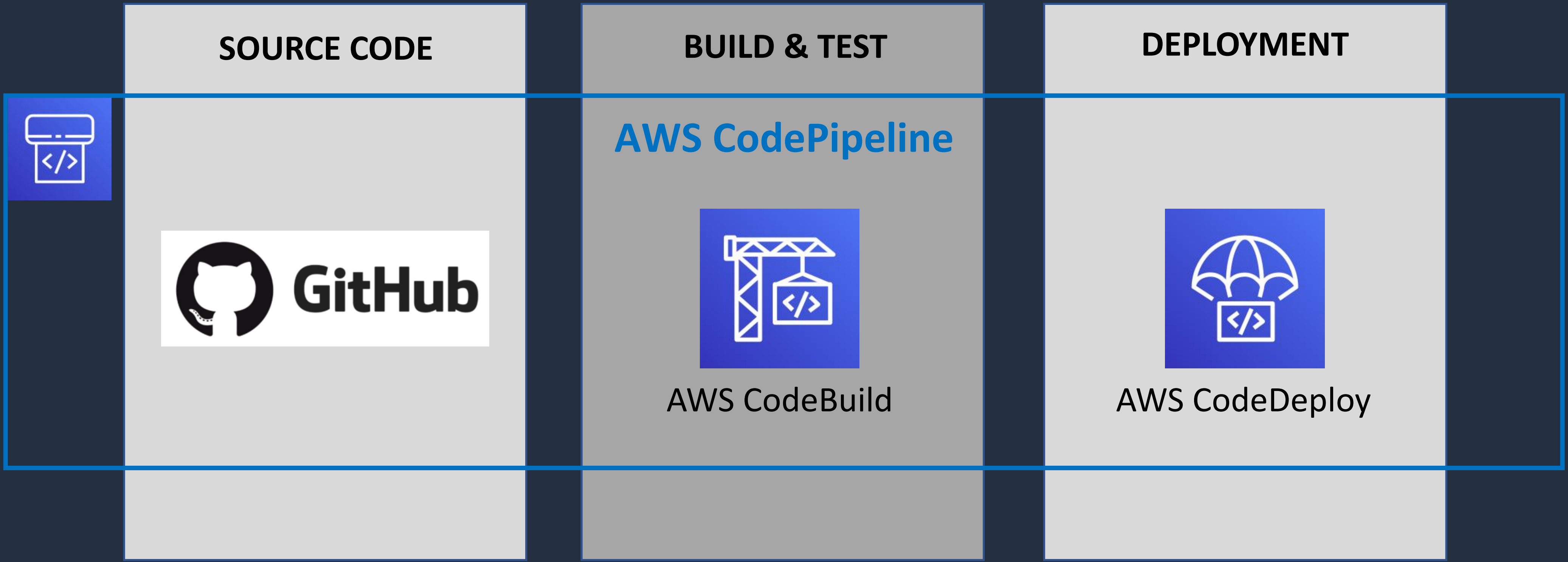
Continuous Deployment (CD)

- Fully automates deployment to production without manual approval
- Every successful change that passes tests is automatically released
- Requires a high level of automation and confidence in testing
- Best suited for teams that need rapid updates with minimal risk

Continuous Integration and Continuous Deployment



Continuous Integration and Continuous Delivery



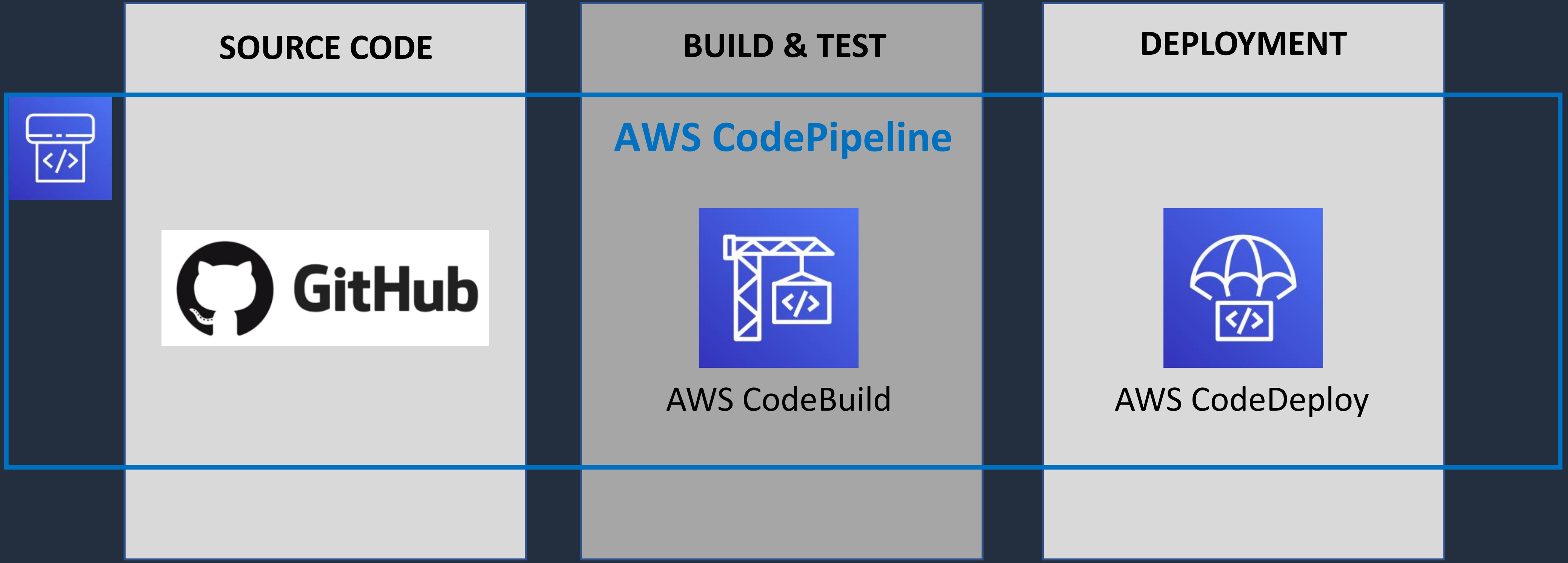
Git and GitHub

AWS CodePipeline

AWS CodePipeline Overview

Fully managed continuous delivery service you can use to **model**, **visualize**,

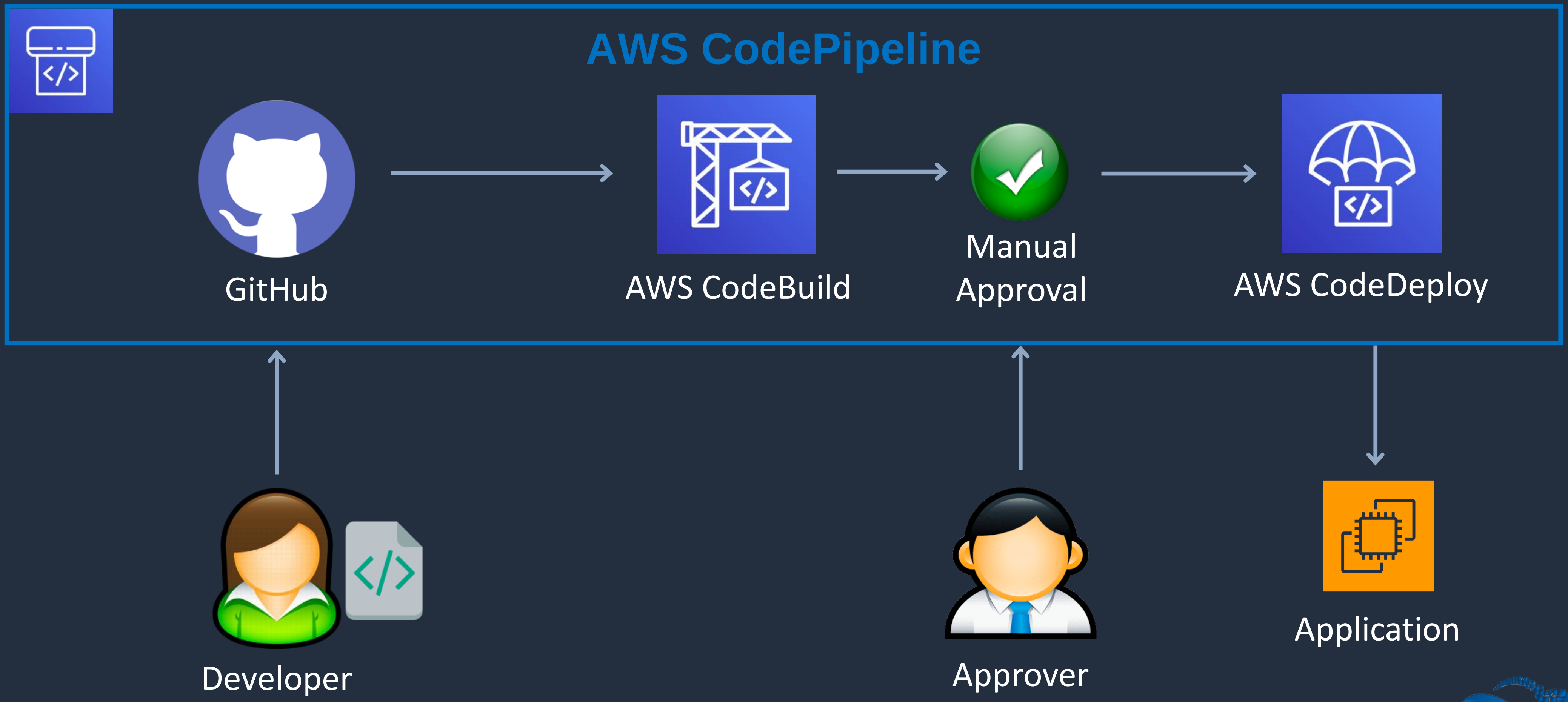
and **automate** the steps required to release software in CI/CD



Source stage | **Build stage** | **Deploy stage**

AWS CodePipeline Overview

CodePipeline automates the **build**, **test**, and **deploy** phases of your release process every time there is a code change, based on the release model you define



AWS CodePipeline Integrations

AWS CodePipeline provides tooling integrations for many AWS and third-party software at each stage of the pipeline *including*:

- **Source stage** – S3, ECR, Github, GitLab, Bitbucket etc.
- **Build** – Run shell commands or use AWS CodeBuild, ECR, or Jenkins
- **Test** – CodeBuild, Device Farm, Jenkins, Ghost Inspector UI Testing, Micro Focus StormRunner Load, Runscope API monitoring
- **Deploy stage** – AppConfig, CloudFormation, CodeDeploy, ECS, Elastic Beanstalk, AWS Service Catalog, S3, Alexa Skills Kit

AWS CodePipeline Concepts

Pipelines

- A workflow that describes how software changes go through the release process
- Each pipeline is made up of a series of stages
- There are two types of pipeline:
  - V1 type pipelines have a JSON structure that contains standard pipeline, stage, and action-level parameters
  - V2 type pipelines have the same structure as a V1 type, along with additional parameters for release safety and trigger configuration

AWS CodePipeline Concepts

Artifacts

- Files or changes that will be worked on by the actions and stages in the pipeline
- Each pipeline stage can create "artifacts"
- Artifacts are passed, stored in Amazon S3 and then passed on to the next stage
- Examples include application source code, built applications, dependencies, definitions files, and templates

AWS CodePipeline Concepts

Stages

- Pipelines are broken up into stages
- Each stage can have sequential actions and/or parallel actions
- Stage examples would be source, build, test, deploy, load test etc.
- Manual approval can be defined at any stage

Source	→	Build	→	Staging	→	Deployment
Code changes committed		Changes built		Code deployed / tested		Code deployed to production

AWS CodePipeline Concepts

Actions

- Stages contain at least one action
- For example, a source action from a code change, or an action for deploying the application to instances
- Actions affect artifacts and will have artifacts as either an input, and output, or both

Transitions

- The progression from one stage to another inside of a pipeline

AWS CodePipeline Concepts

Pipeline executions

- An execution is a set of changes released by a pipeline. Each pipeline execution is unique and has its own ID
- An execution corresponds to a set of changes, such as a merged commit or a manual release of the latest commit
- Two executions can release the same set of changes at different times

AWS CodePipeline

Source Stage

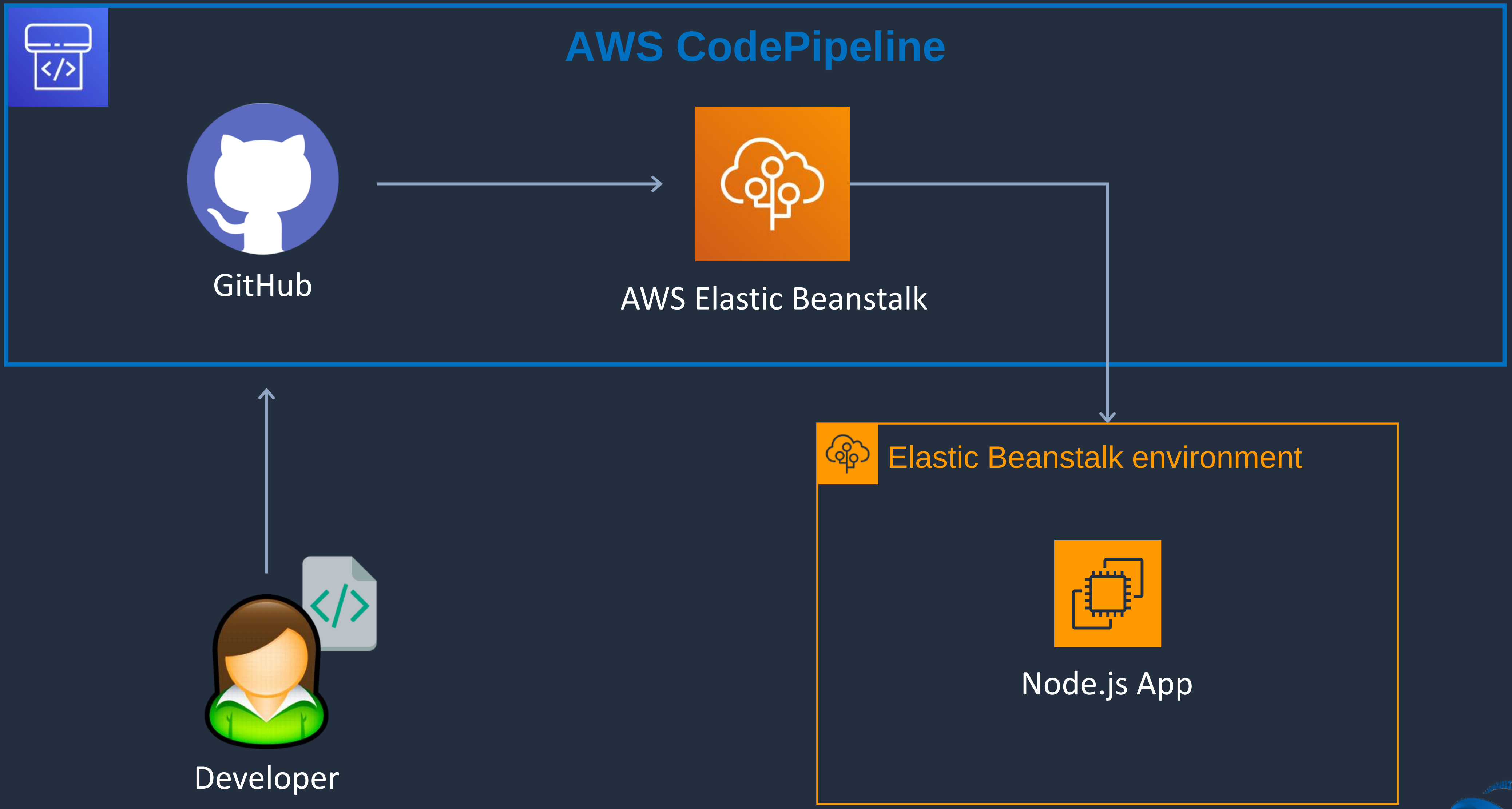
Build Stage

Test Stage

Deployment Stage

Create Pipeline and Application

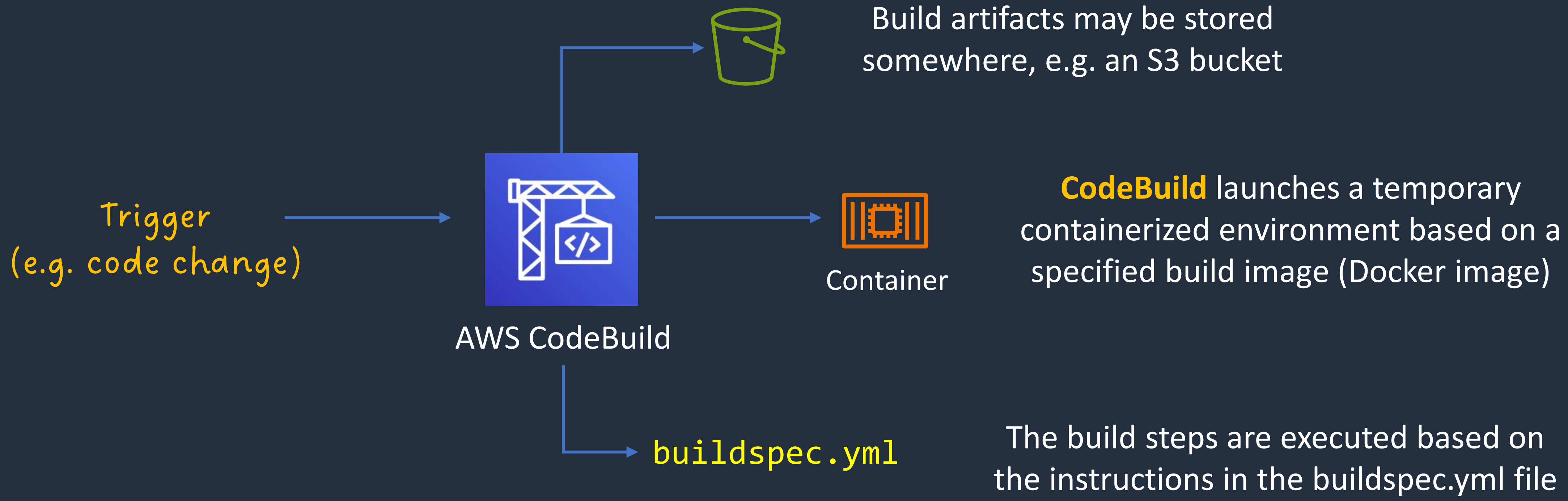
CodePipeline with Elastic Beanstalk



AWS CodeBuild

AWS CodeBuild

- AWS CodeBuild is a fully managed continuous integration (CI) service
- It compiles source code, runs tests, and produces software packages that are ready to deploy



AWS CodeBuild

- CodeBuild scales continuously and processes multiple builds concurrently
- You pay based on the time it takes to complete the builds
- CodeBuild takes source code from GitHub, Bitbucket, CodePipeline, S3 etc.
- Build instructions can be defined in the code (**buildspec.yml**)
- Output logs can be sent to Amazon S3 & Amazon CloudWatch Logs

AWS CodeBuild Components

Build project - defines how CodeBuild will run a build defines settings including:

- Location of the source code
- The build environment to use
- The build commands to run
- Where to store the output of the build

Build environment - the operating system, language runtime, and tools that CodeBuild uses for the build

Build Specification - a YAML file that describes the collection of commands and settings for CodeBuild to run a build

AWS CodeBuild – buildspec.yml file

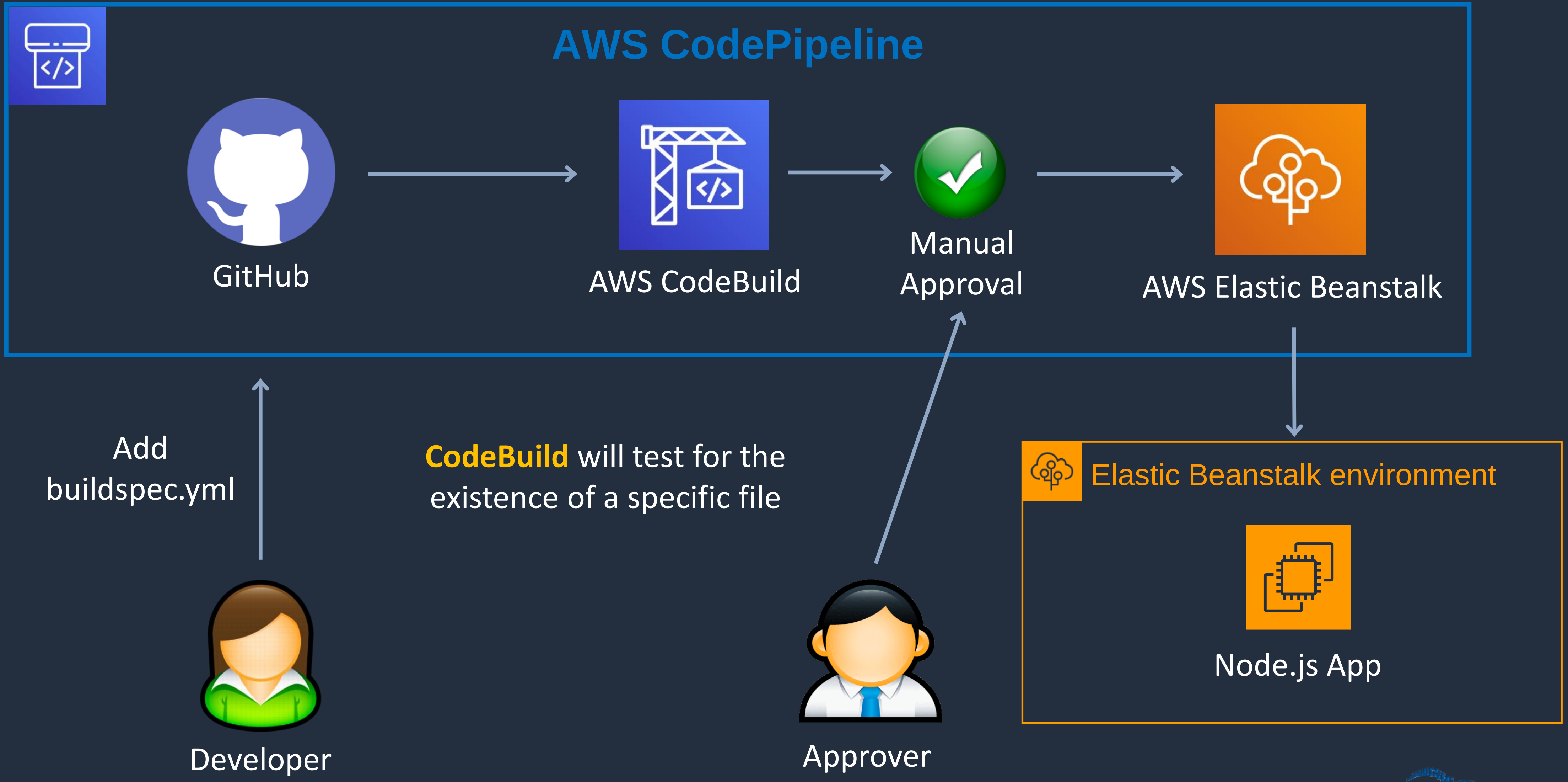
```yaml
version: 0.2

phases:
    install:
        commands:
            - echo "Entered the install phase..."
    pre_build:
        commands:
            - echo "Entered the pre_build phase..."
    build:
        commands:
            - echo "Entered the build phase..."
            - echo "Build started on `date`"
            - find production.txt
    post_build:
        commands:
            - echo "Entered the post_build phase..."
            - echo "Build completed on `date`"
```

Buildspec Phases

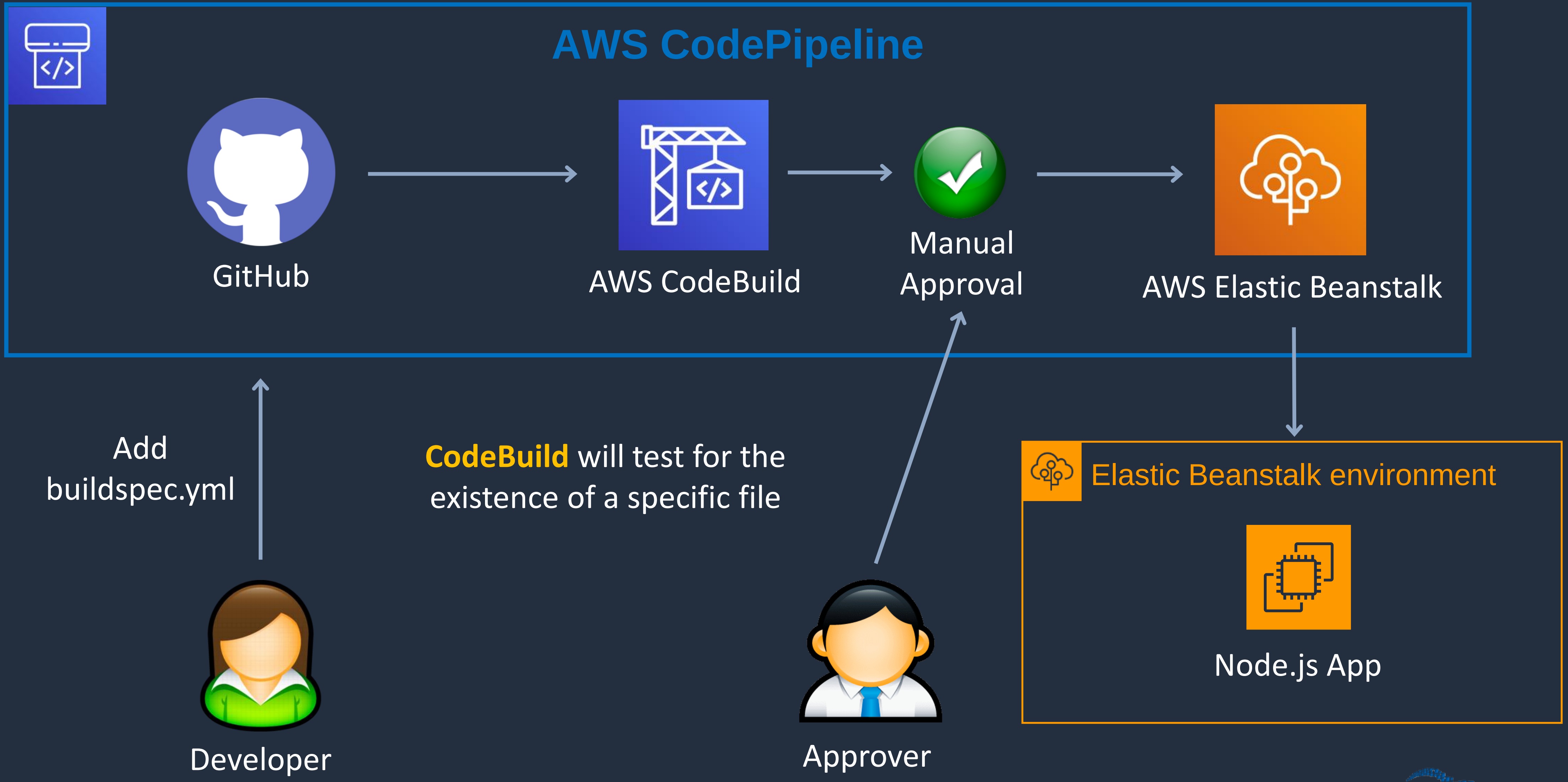
Phases	Required?	Description
phases/*/run-as	Optional	Specifies a Linux user that runs its commands
phases/*/on-failure	Optional	Specifies the action to take if failure occurs during the phase (ABORT or CONTINUE)
phases/*/finally	Optional	Runs commands after commands in the commands block (even if the the commands in the commands block fail)
phases/install	Optional	Commands to run during installation
phases/pre_build	Optional	Commands to run before the build
phases/build	Optional	Commands to run during the build
phases/post_build	Optional	Commands to run after the build

CI/CD with CodeBuild



Add Build Stage to Pipeline

CI/CD with CodeBuild



AWS CodeDeploy

AWS CodeDeploy

- CodeDeploy is a deployment service that automates application deployments
- Deploys to Amazon EC2 instances, on-premises instances, serverless Lambda functions, and Amazon ECS
- You can deploy a nearly unlimited variety of application content, including:
  - Serverless AWS Lambda functions
  - Web and configuration files
  - Executables
  - Packages
  - Scripts
  - Multimedia files

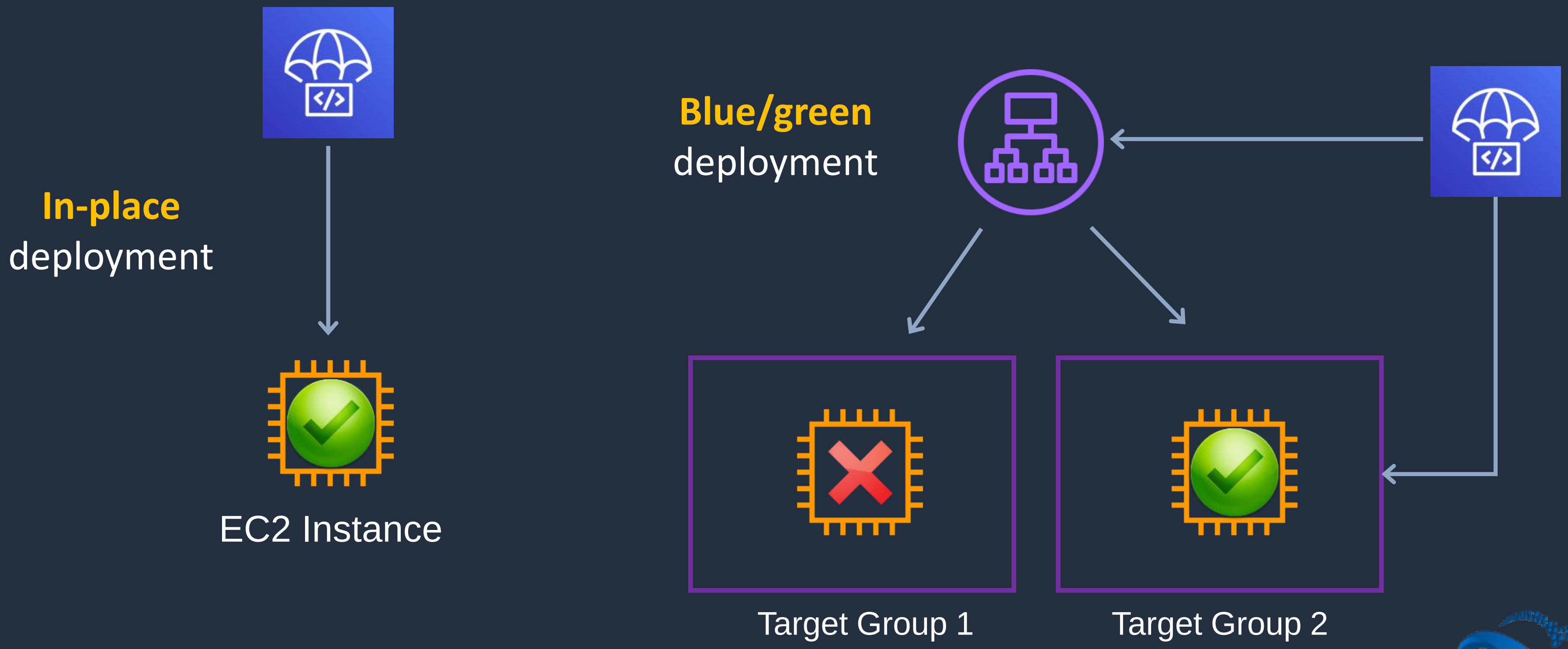
AWS CodeDeploy

- An AWS CodeDeploy application contains information about what to deploy and how to deploy it
- Need to choose the compute platform:
  - EC2/On-premises
  - AWS Lambda
  - Amazon ECS

AWS CodeDeploy

EC2/On-Premises:

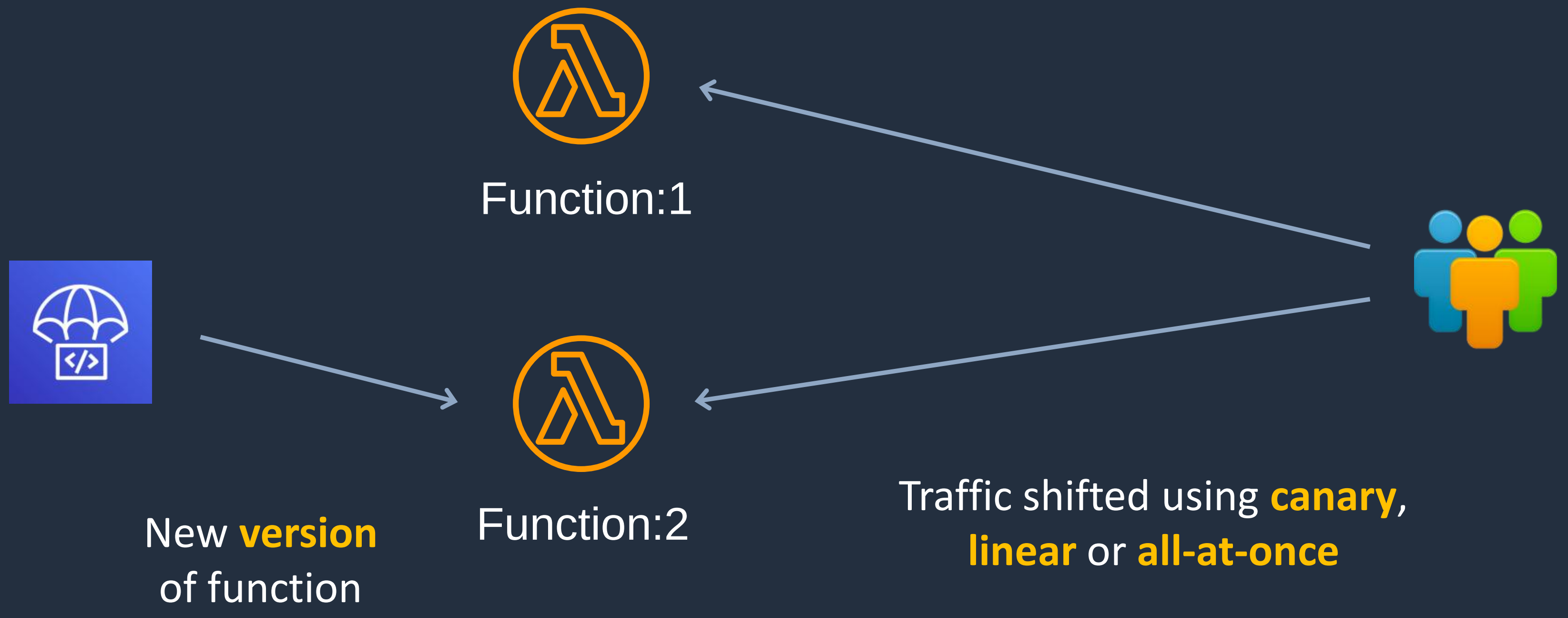
- Amazon EC2, on-premises servers, or both
- Traffic is directed using an **in-place** or **blue/green** deployment type



AWS CodeDeploy

AWS Lambda:

- Used to deploy applications that consist of an updated version of a Lambda function
- You can manage the way in which traffic is shifted to the updated Lambda function versions during a deployment by choosing a **canary**, **linear**, or **all-at-once** configuration

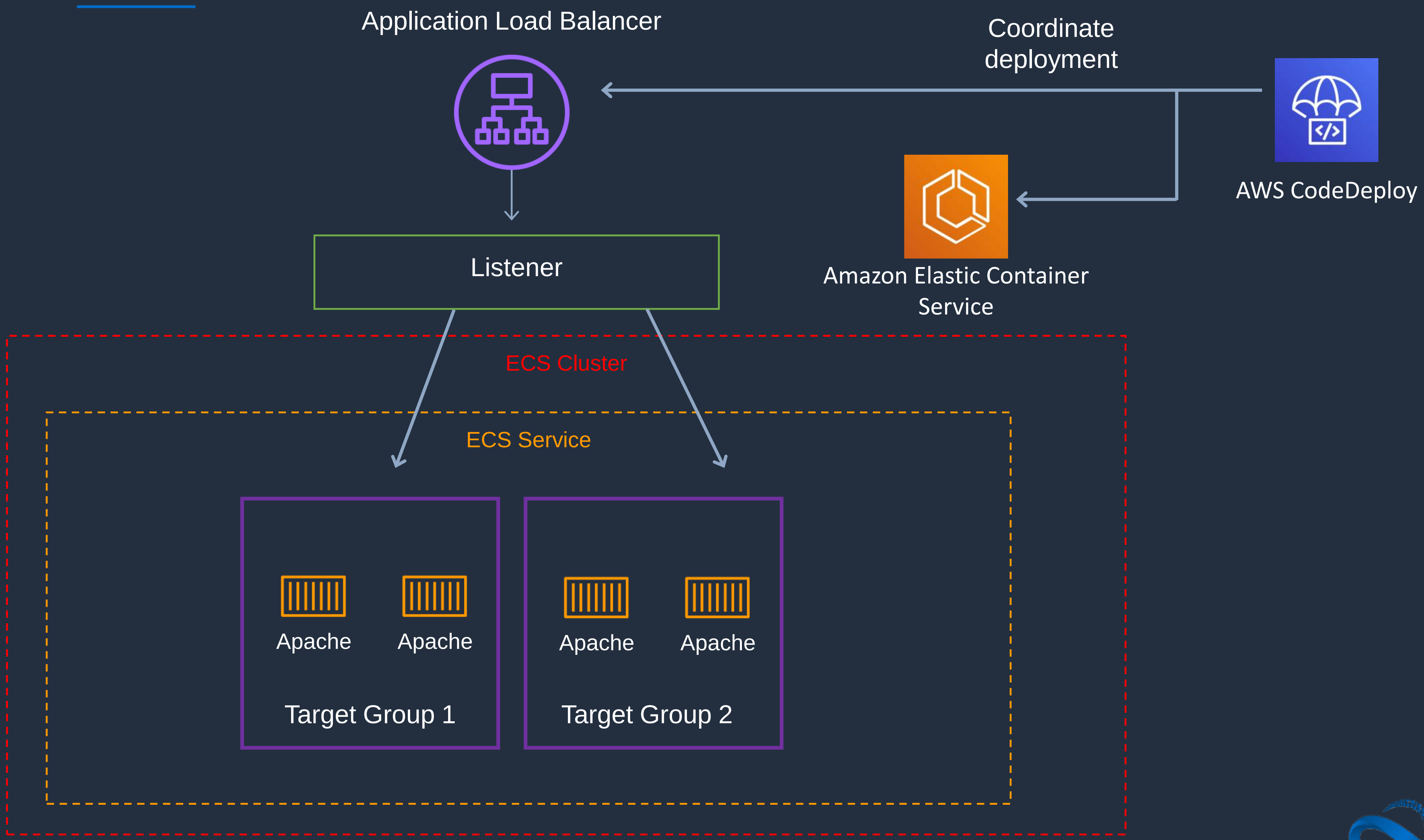


AWS CodeDeploy

Amazon ECS:

- Used to deploy an Amazon ECS containerized application as a task set
- CodeDeploy performs a **blue/green** deployment by installing an updated version of the application as a new replacement task set
- CodeDeploy reroutes production traffic from the original application task set to the replacement task set
- The original task set is terminated after a successful deployment
- You can manage the way in which traffic is shifted to the updated task set during a deployment by choosing a **canary**, **linear**, or **all-at-once** configuration

AWS CodeDeploy



Blue/Green Traffic Shifting

- **AWS Lambda:** Traffic is shifted from one version of a Lambda function to a new version of the same Lambda function
- **Amazon ECS:** Traffic is shifted from a task set in your Amazon ECS service to an updated, replacement task set in the same Amazon ECS service
- **EC2/On-Premises:** Traffic is shifted from one set of instances in the original environment to a replacement set of instances
- **Note:** All AWS Lambda and Amazon ECS deployments are blue/green. An EC2/On-Premises deployment can be in-place or blue/green

Blue/Green Traffic Shifting

For Amazon ECS and AWS Lambda there are three ways traffic can be shifted during a deployment:

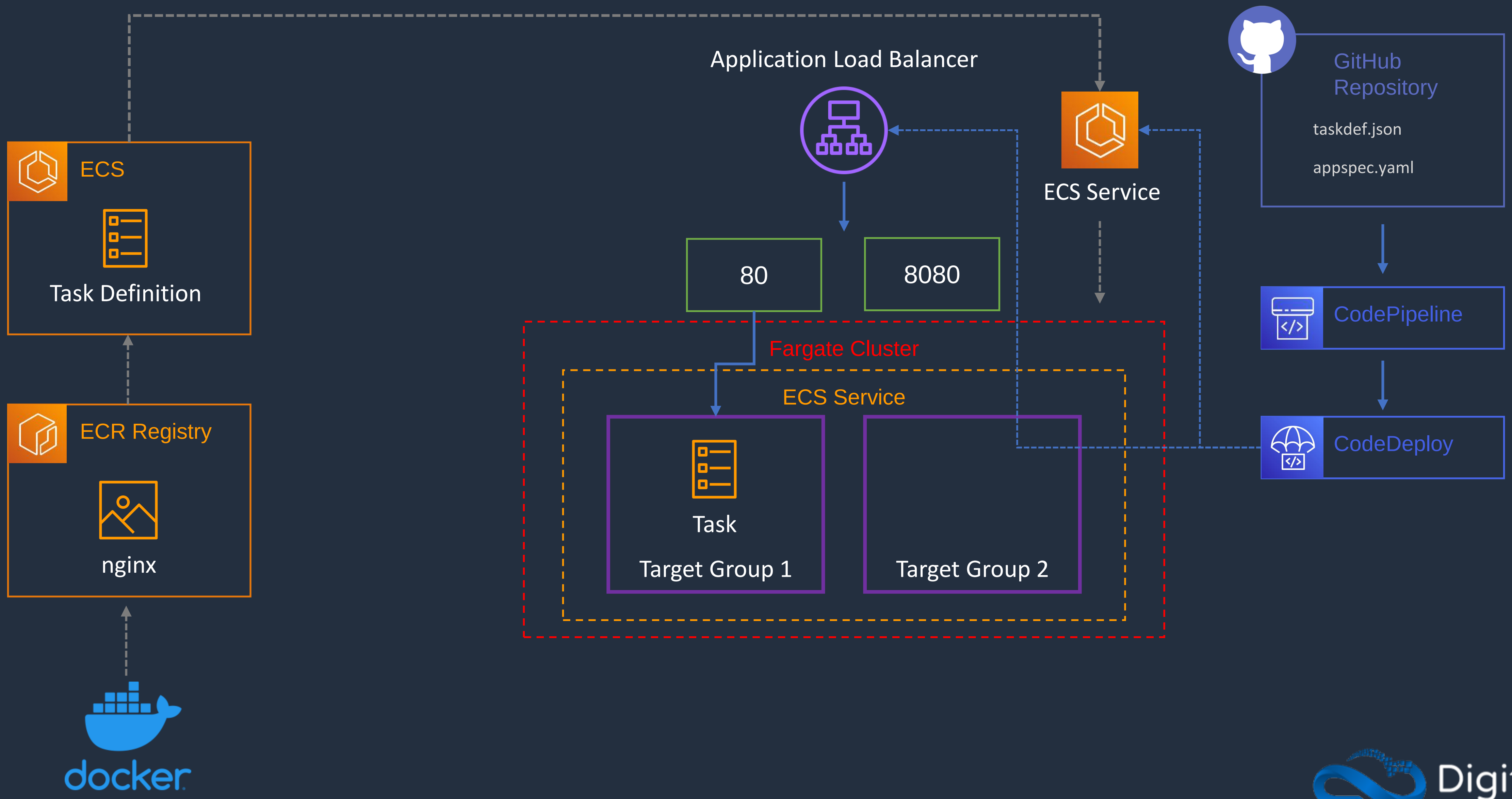
- **Canary** - Traffic is shifted in two increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Amazon ECS task set / Lambda function in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment
- **Linear** - Traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment
- **All-at-once** - All traffic is shifted from the original Amazon ECS task set / Lambda function to the updated Amazon ECS task set / Lambda function all at once

AWS Fargate Blue-Green CI/CD Pipeline – Part 4

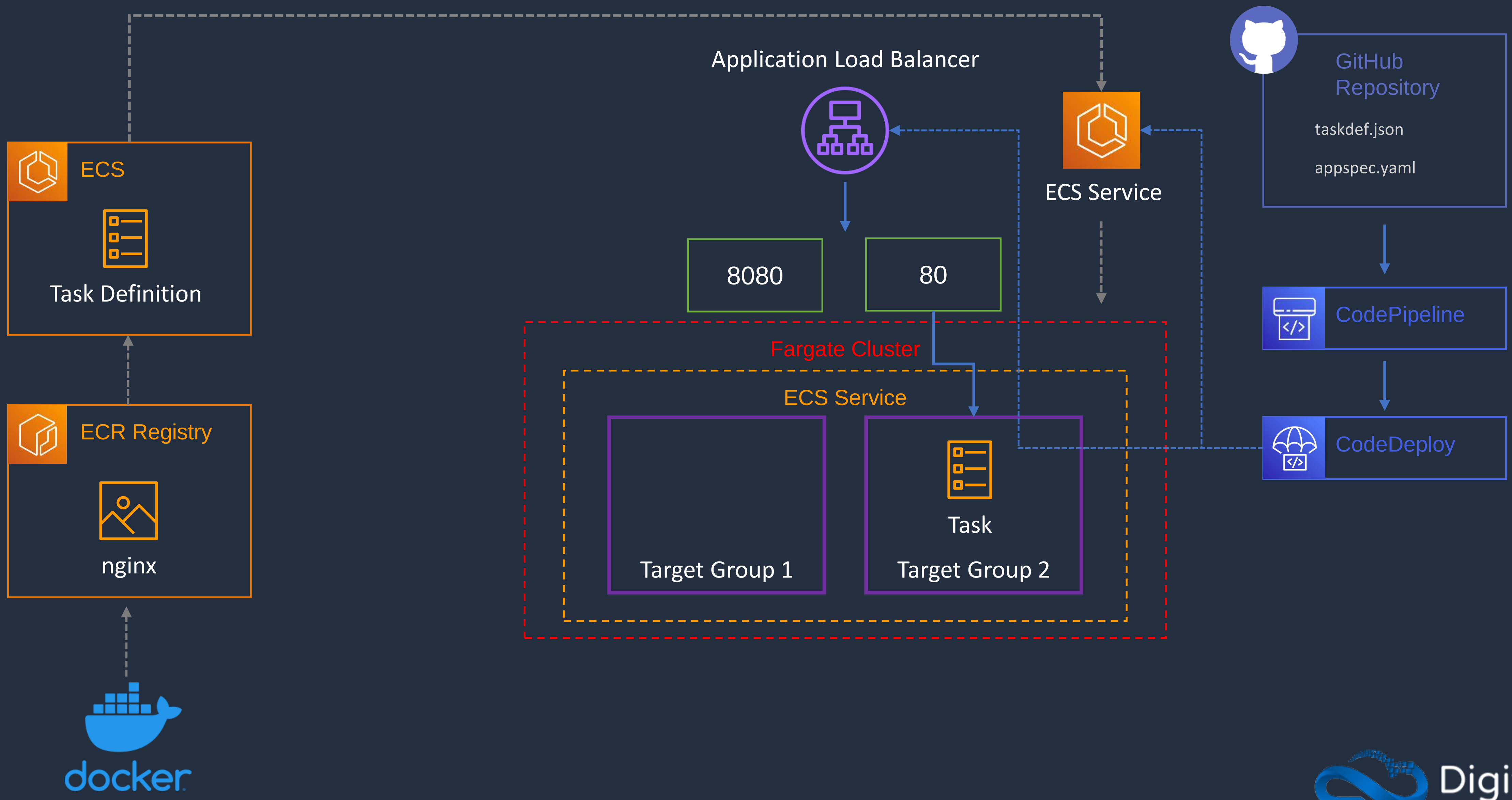
AWS Fargate Blue-Green CI/CD Pipeline

1. Create Image and Push to ECR Repository
2. Create Task Definition and ALB
3. Create Fargate Cluster and Service
4. **CodeDeploy Application and Pipeline**
5. Implement Blue/Green Update to ECS

AWS Fargate Blue-Green CI/CD Pipeline



AWS Fargate Blue-Green CI/CD Pipeline

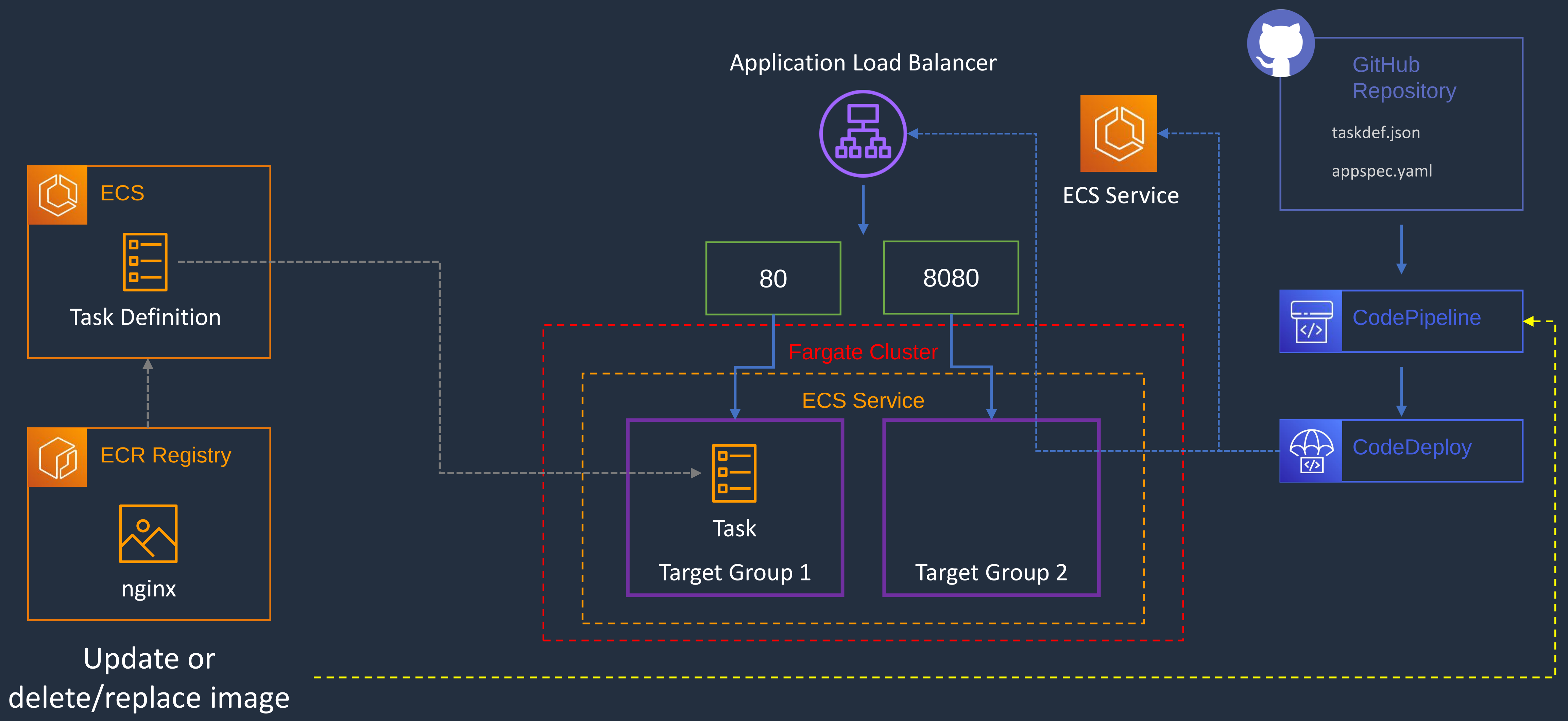


AWS Fargate Blue-Green CI/CD Pipeline – Part 5

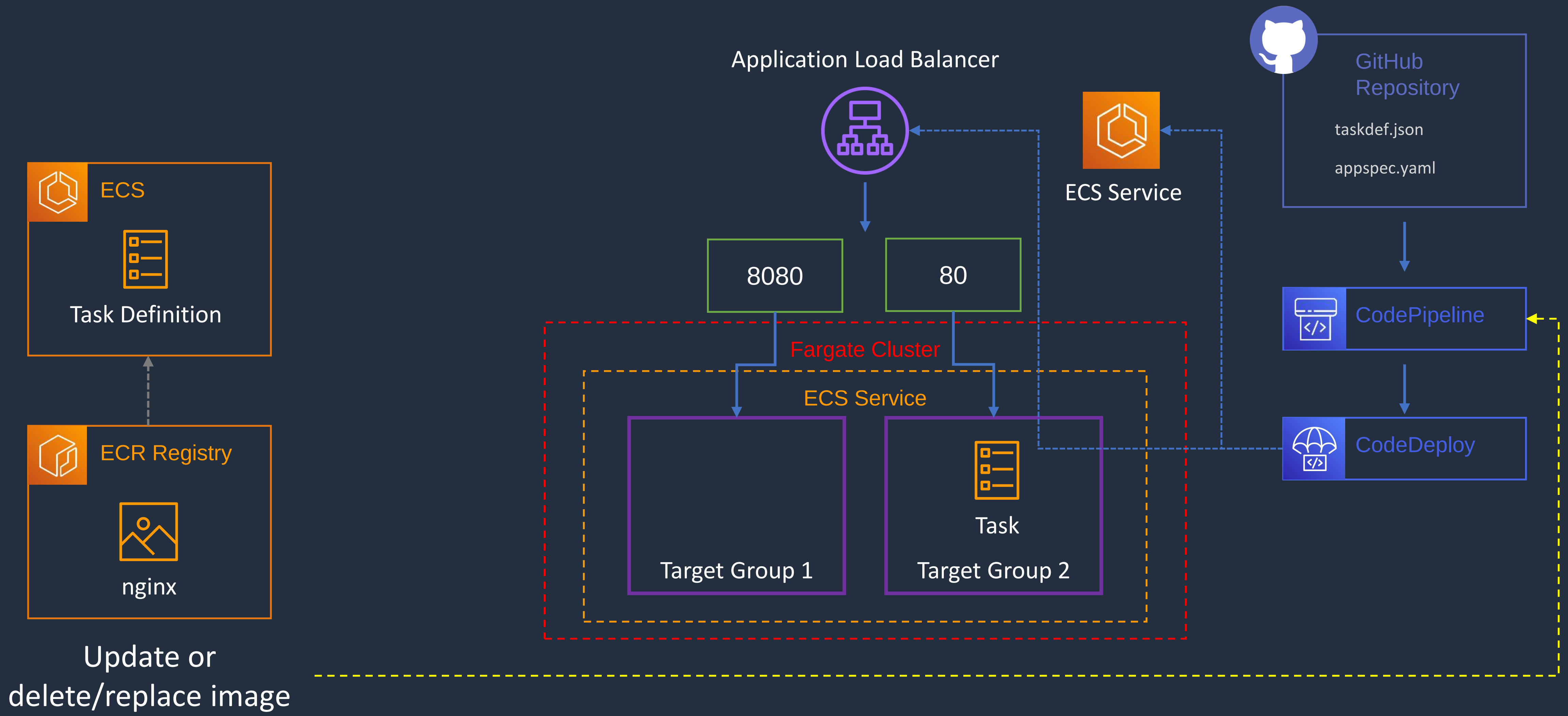
AWS Fargate Blue-Green CI/CD Pipeline

1. Create Image and Push to ECR Repository
2. Create Task Definition and ALB
3. Create Fargate Cluster and Service
4. CodeDeploy Application and Pipeline
5. **Implement Blue/Green Update to ECS**

AWS Fargate Blue-Green CI/CD Pipeline

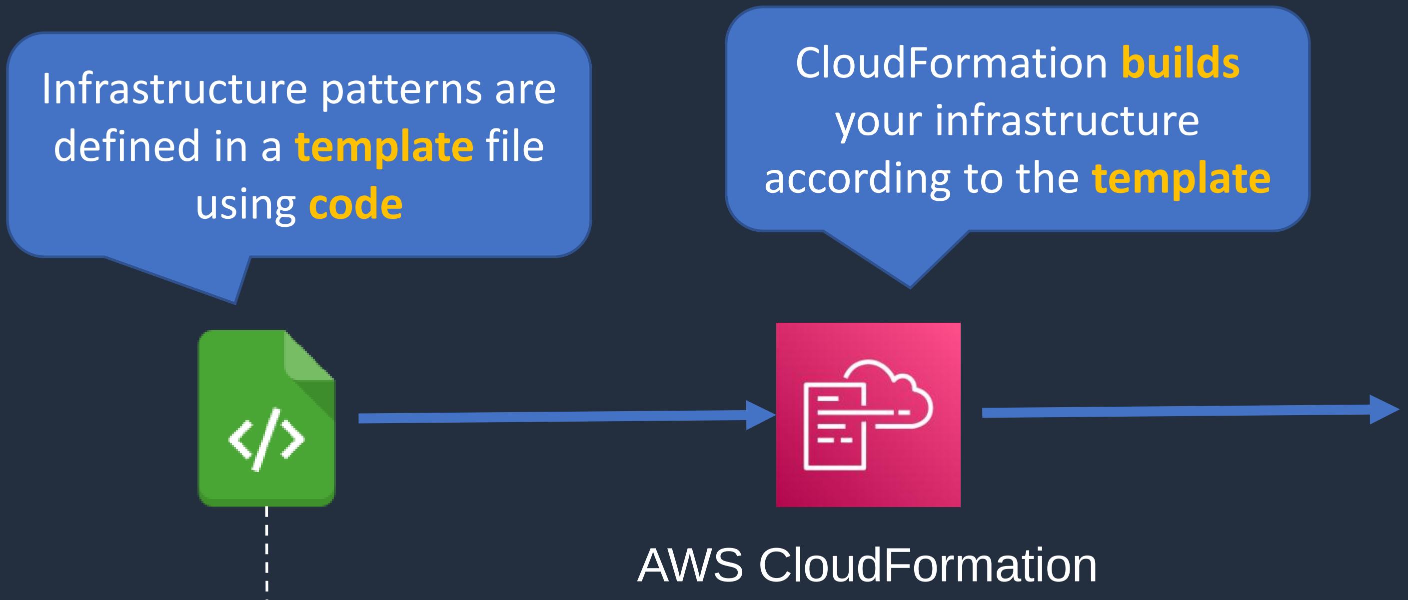


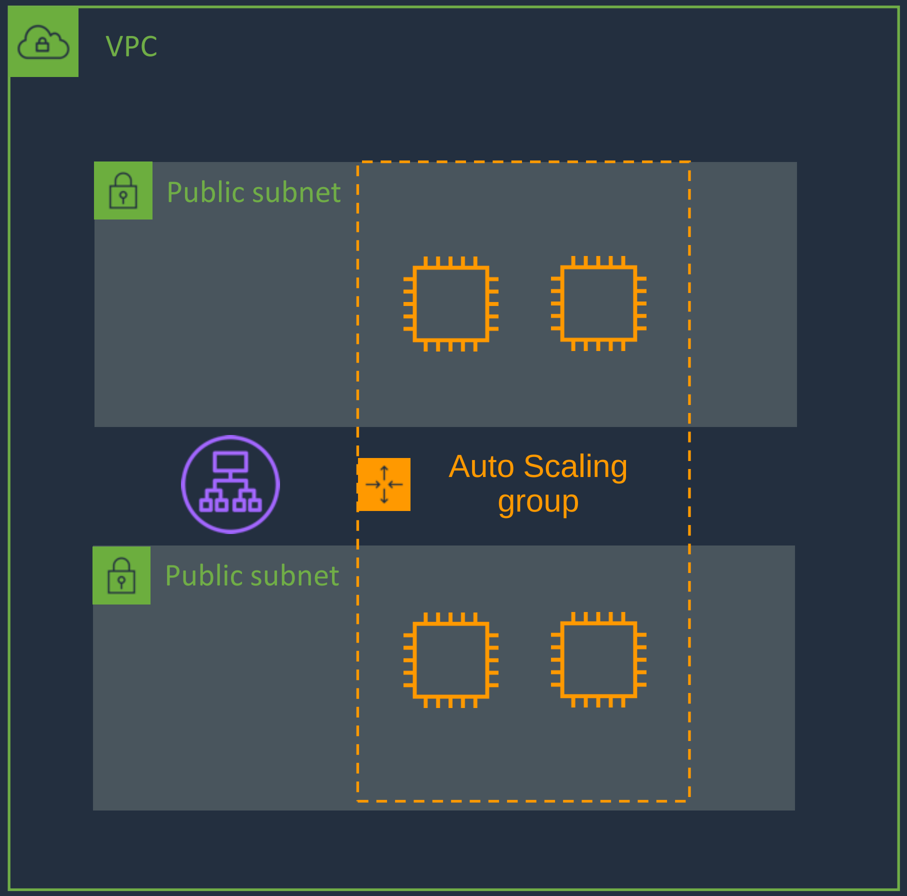
AWS Fargate Blue-Green CI/CD Pipeline



AWS CloudFormation Core Knowledge

AWS CloudFormation






```
"AWSTemplateFormatVersion" : "2010-09-09",

  "Description" : "AWS CloudFormation Sample Template WordPress_Multi_AZ: WordPress is web

  "Parameters" : {
    "VpcId" : {
      "Type" : "AWS::EC2::VPC::Id",
      "Description" : "VpcId of your existing Virtual Private Cloud (VPC)",
      "ConstraintDescription" : "must be the VPC Id of an existing Virtual Private Cloud."
    },

    "Subnets" : {
      "Type" : "List<AWS::EC2::Subnet::Id>",
      "Description" : "The list of SubnetIds in your Virtual Private Cloud (VPC)",
      "ConstraintDescription" : "must be a list of at least two existing subnets associated
    },
```

AWS CloudFormation - Benefits

- Infrastructure is provisioned consistently, with fewer mistakes (human error)
- Less time and effort than configuring resources manually
- You can use version control and peer review for your CloudFormation templates
- Free to use (you're only charged for the resources provisioned)
- Can be used to manage updates and dependencies
- Can be used to rollback and delete the entire stack as well

AWS CloudFormation

Component	Description
Templates	The JSON or YAML text file that contains the instructions for building out the AWS environment
Stacks	The entire environment described by the template and created, updated, and deleted as a single unit
StackSets	AWS CloudFormation StackSets extends the functionality of stacks by enabling you to create, update, or delete stacks across multiple accounts and regions with a single operation
Change Sets	A summary of proposed changes to your stack that will allow you to see how those changes might impact your existing resources before implementing them

AWS CloudFormation - Templates

- A template is a YAML or JSON template used to describe the end-state of the infrastructure you are either provisioning or changing
- After creating the template, you upload it to CloudFormation directly or using Amazon S3
- CloudFormation reads the template and makes the API calls on your behalf.
- The resulting resources are called a "Stack"
- Logical IDs are used to reference resources within the template
- Physical IDs identify resources outside of AWS CloudFormation templates, but only after the resources have been created

AWS CloudFormation - Stacks

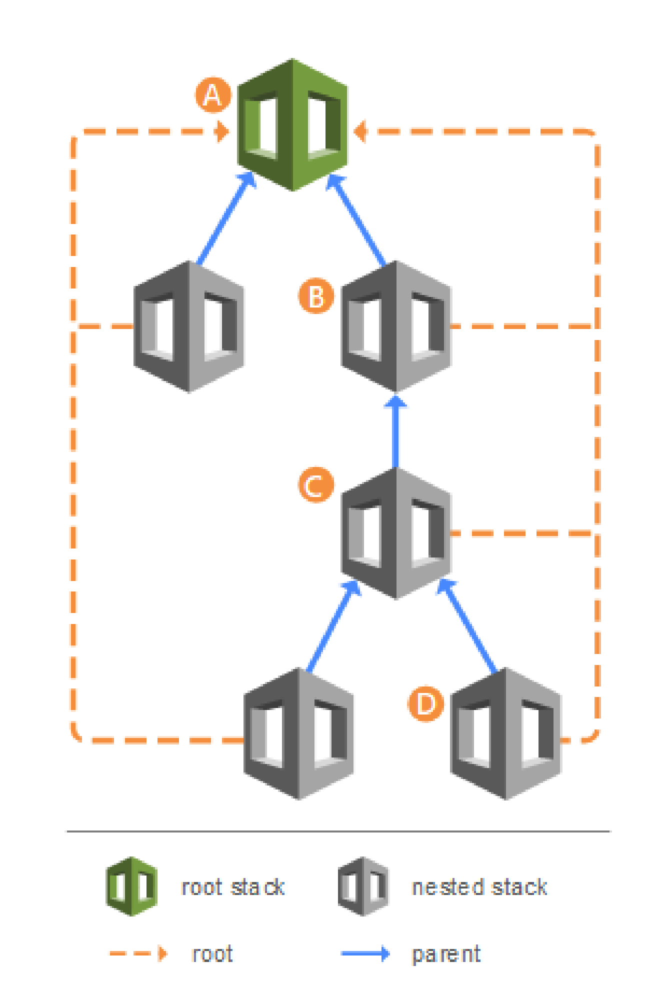
- Deployed resources based on templates
- Create, update and delete stacks using templates
- Deployed through the Management Console, CLI or APIs
- Stack creation errors:
  - Automatic rollback on error is enabled by default
  - You will be charged for resources provisioned even if there is an error

AWS CloudFormation - Stacks

- AWS CloudFormation StackSets extends the functionality of stacks by enabling you to create, update, or delete stacks across multiple accounts and regions with a single operation
- Using an administrator account, you define and manage an AWS CloudFormation template, and use the template as the basis for provisioning stacks into selected target accounts across specified regions
- An administrator account is the AWS account in which you create stack sets
- A stack set is managed by signing in to the AWS administrator account in which it was created
- A target account is the account into which you create, update, or delete one or more stacks in your stack set

AWS CloudFormation - NestedStacks

- Nested stacks allow re-use of CloudFormation code for common use cases
- For example standard configuration for a load balancer, web server, application server etc.
- Instead of copying out the code each time, create a standard template for each common use case and reference from within your CloudFormation template



AWS CloudFormation - Change Sets

- AWS CloudFormation provides two methods for updating stacks: direct update or creating and executing change sets
- When you directly update a stack, you submit changes and AWS CloudFormation immediately deploys them
- Use direct updates when you want to quickly deploy your updates
- With change sets, you can preview the changes AWS CloudFormation will make to your stack, and then decide whether to apply those changes

Create CloudFormation Stack

Create Nested Stack using the AWS CLI

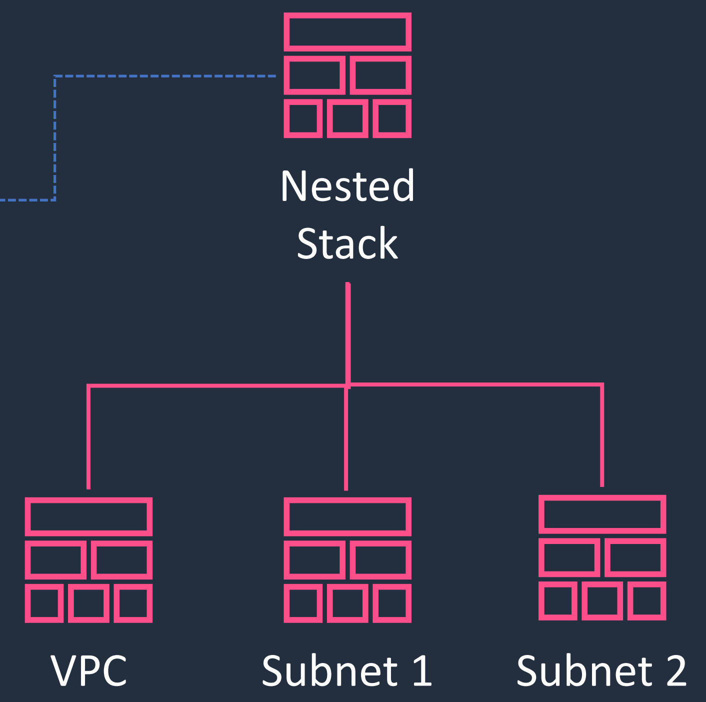
CloudFormation Nested Stacks

The AWS::CloudFormation::Stack resource nests a stack as a resource in a top-level template

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Resources:
  VPCStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-cloudformation-s3-bucket-3121s2.s3.amazonaws.com/vpc.yaml

  Subnet1Stack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-cloudformation-s3-bucket-3121s2.s3.amazonaws.com/subnet1.yaml
      Parameters:
        VpcId: !GetAtt VPCStack.Outputs.VpcId

  Subnet2Stack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-cloudformation-s3-bucket-3121s2.s3.amazonaws.com/subnet2.yaml
      Parameters:
        VpcId: !GetAtt VPCStack.Outputs.VpcId
```

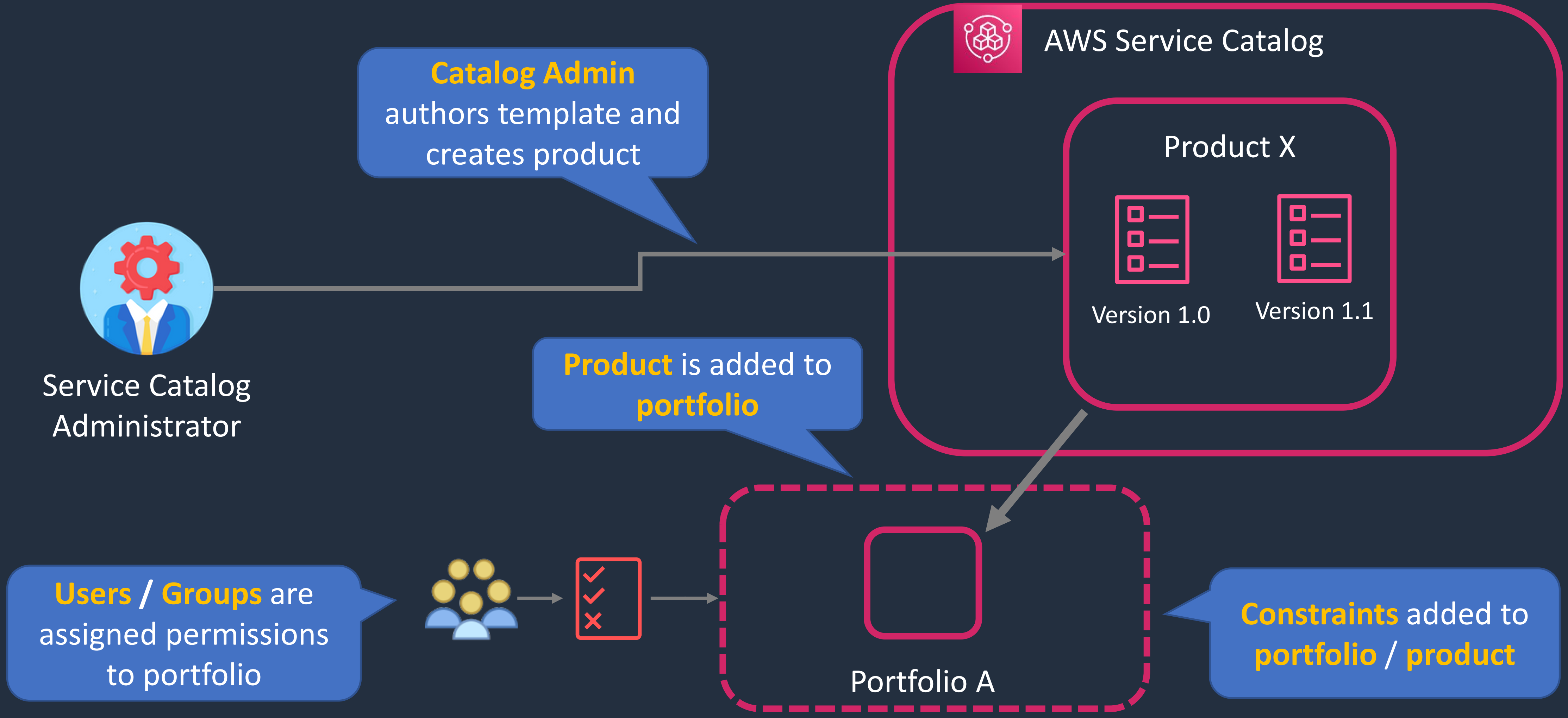


AWS Service Catalog

AWS Service Catalog

- AWS Service Catalog allows organizations to create and manage **catalogs** of **IT services** that are approved for use on AWS
- IT services can include everything from virtual machine images, servers, software, and databases to complete multi-tier application architectures
- AWS Service Catalog allows you to **centrally manage** commonly deployed IT services
- Enables users to quickly deploy only the approved IT services they need

AWS Service Catalog



AWS Service Catalog – Users

Catalog administrators

- Manage a catalog of products, organizing them into portfolios and granting access to end users

End users

- Receive AWS credentials use the AWS Management Console to launch products to which they have been granted access

AWS Service Catalog – Constraints

- Constraints control the ways that specific AWS resources can be deployed for a product.
- You can use them to apply limits to products for governance or cost control.
- **Launch constraints** - specify a role for a product in a portfolio. The role is used to provision the resources at launch, so you can restrict user permissions without impacting users' ability to provision products from the catalog
- **Notification constraints** - enable you to get notifications about stack events using an Amazon SNS topic
- **Template constraints -** restrict the configuration parameters that are available for the user when launching the product

AWS Service Catalog – Sharing

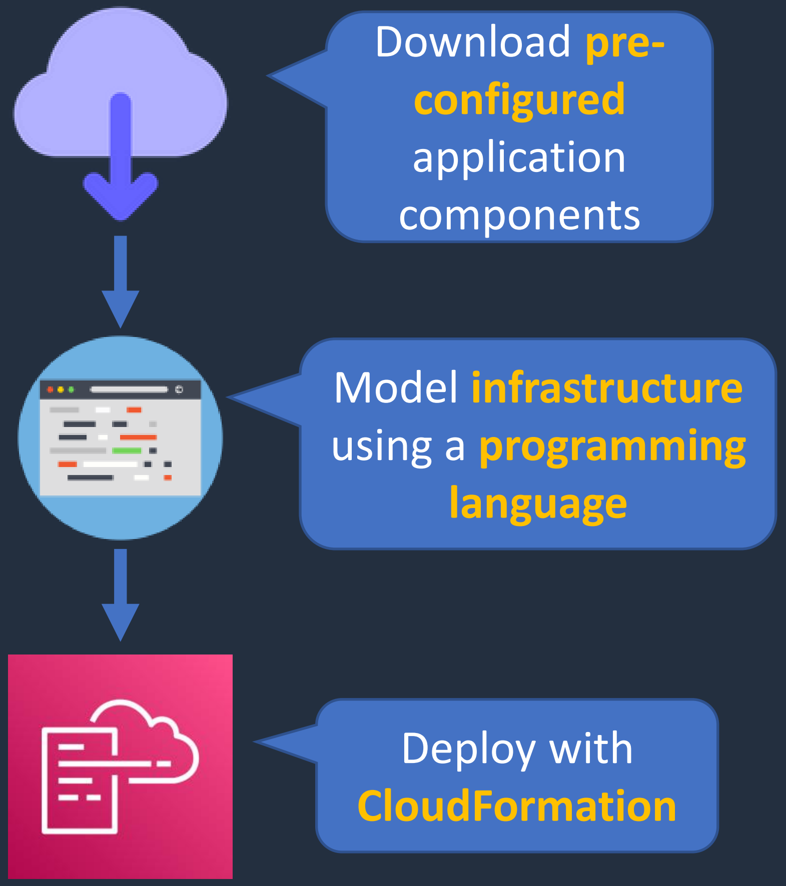
- You can share portfolios across accounts in AWS Organizations
- Either share a reference to the catalog or deploy a copy of the catalog
- With copies you must redeploy any updates
- CloudFormation StackSets can be used to deploy a catalog to multiple accounts at the same time
- Check reference link for more information on the behavior

Deploy Product using Service Catalog

AWS Cloud Development Kit

AWS Cloud Development Kit

- Open source software development framework to define your cloud application resources using **familiar programming languages**
- Preconfigures cloud resources with proven defaults using **constructs**
- Provisions your resources using **AWS CloudFormation**
- Enables you to model application infrastructure using TypeScript, Python, Java, and .NET
- Use existing IDE, testing tools, and workflow patterns

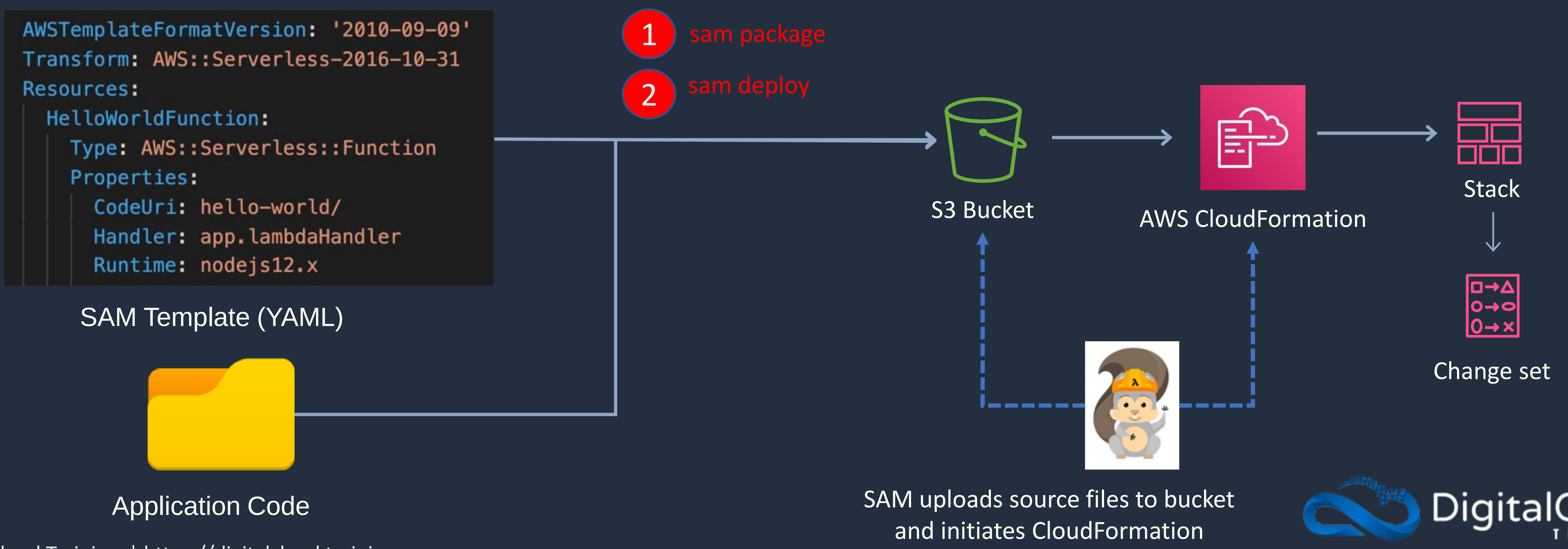


AWS Serverless Application Model (SAM)

AWS SAM

- Provides a **shorthand syntax** to express functions, APIs, databases, and event source mappings
- Can be used to create Lambda functions, API endpoints, DynamoDB tables, and other resources

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Resources:
  HelloWorldFunction:
    Type: AWS::Serverless::Function
    Properties:
      CodeUri: hello-world/
      Handler: app.lambdaHandler
      Runtime: nodejs12.x
```



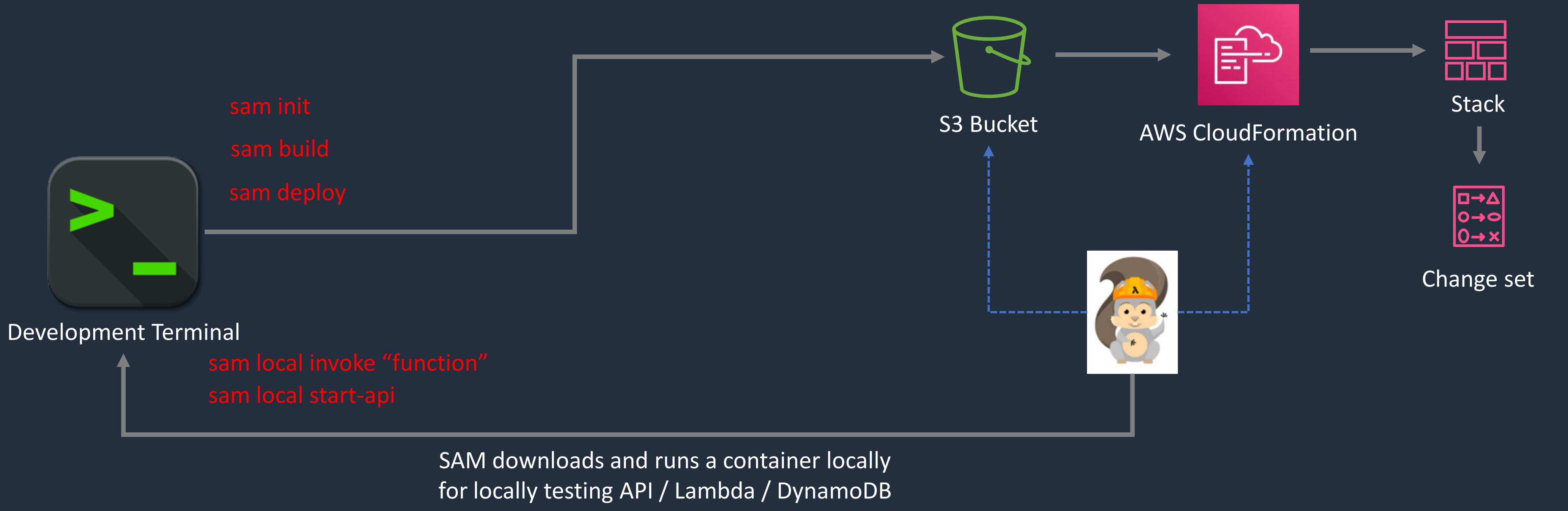
AWS SAM

- A SAM template file is a **YAML** configuration that represents the architecture of a serverless application
- You use the template to declare all of the AWS resources that comprise your serverless application in one place
- AWS SAM templates are an extension of **AWS CloudFormation** templates, so any resource that you can declare in an AWS CloudFormation template can also be declared in an AWS SAM template

AWS SAM

- The “Transform” header indicates it’s a SAM template:
  - Transform: 'AWS::Serverless-2016-10-31'
- There are several resources types:
  - AWS::Serverless::Function (AWS Lambda)
  - AWS::Serverless::Api (API Gateway)
  - AWS::Serverless::SimpleTable (DynamoDB)
  - AWS::Serverless::Application (AWS Serverless Application Repository)
  - AWS::Serverless::HttpApi (API Gateway HTTP API)
  - AWS::Serverless::LayerVersion (Lambda layers)

AWS SAM



sam init
sam build
sam deploy
S3 Bucket
AWS CloudFormation
Stack
Change set
Development Terminal
sam local invoke "function"
sam local start-api
SAM downloads and runs a container locally
for locally testing API / Lambda / DynamoDB

AWS Systems Manager

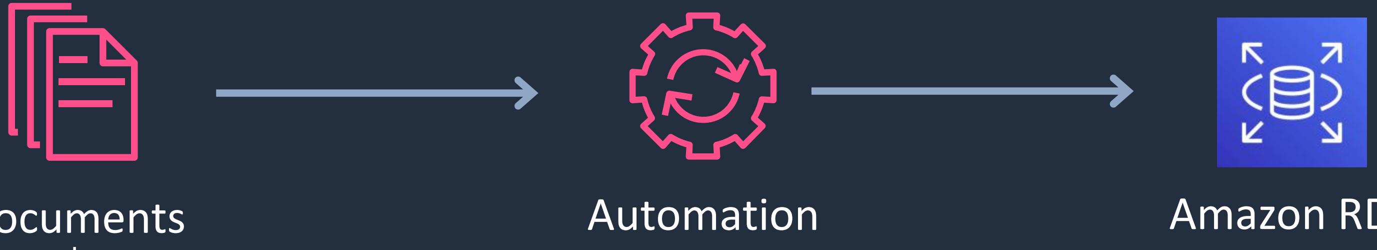
AWS Systems Manager

- Manages many AWS resources including Amazon EC2, Amazon S3, Amazon RDS etc.
- Systems Manager Components:
  - Automation
  - Run Command
  - Inventory
  - Patch Manager
  - Session Manager
  - Parameter Store

AWS Systems Manager Automation

Documents define the actions to perform (YAML or JSON)

Automates **IT operations** and **management** tasks across AWS resources




```yaml
description: Creates an RDS Snapshot for an RDS instance.
schemaVersion: '0.3'
assumeRole: "{{AutomationAssumeRole}}"
parameters:
  DBInstanceIdentifier:
    type: String
    description: (Required) The DBInstanceId ID of the RDS Instance.
  DBSnapshotIdentifier:
    type: String
    description: (Optional) The DBSnapshotIdentifier ID.
    default: ''
  InstanceTags:
    type: String
    default: ''
    description: (Optional) Tags to create for instance.
  SnapshotTags:
    type: String
    default: ''
    description: (Optional) Tags to create for snapshot
```

This automation, takes a snapshot of an RDS DB instance

AWS Systems Manager Run Command

Document types include **command**, **automation**, **package** etc.

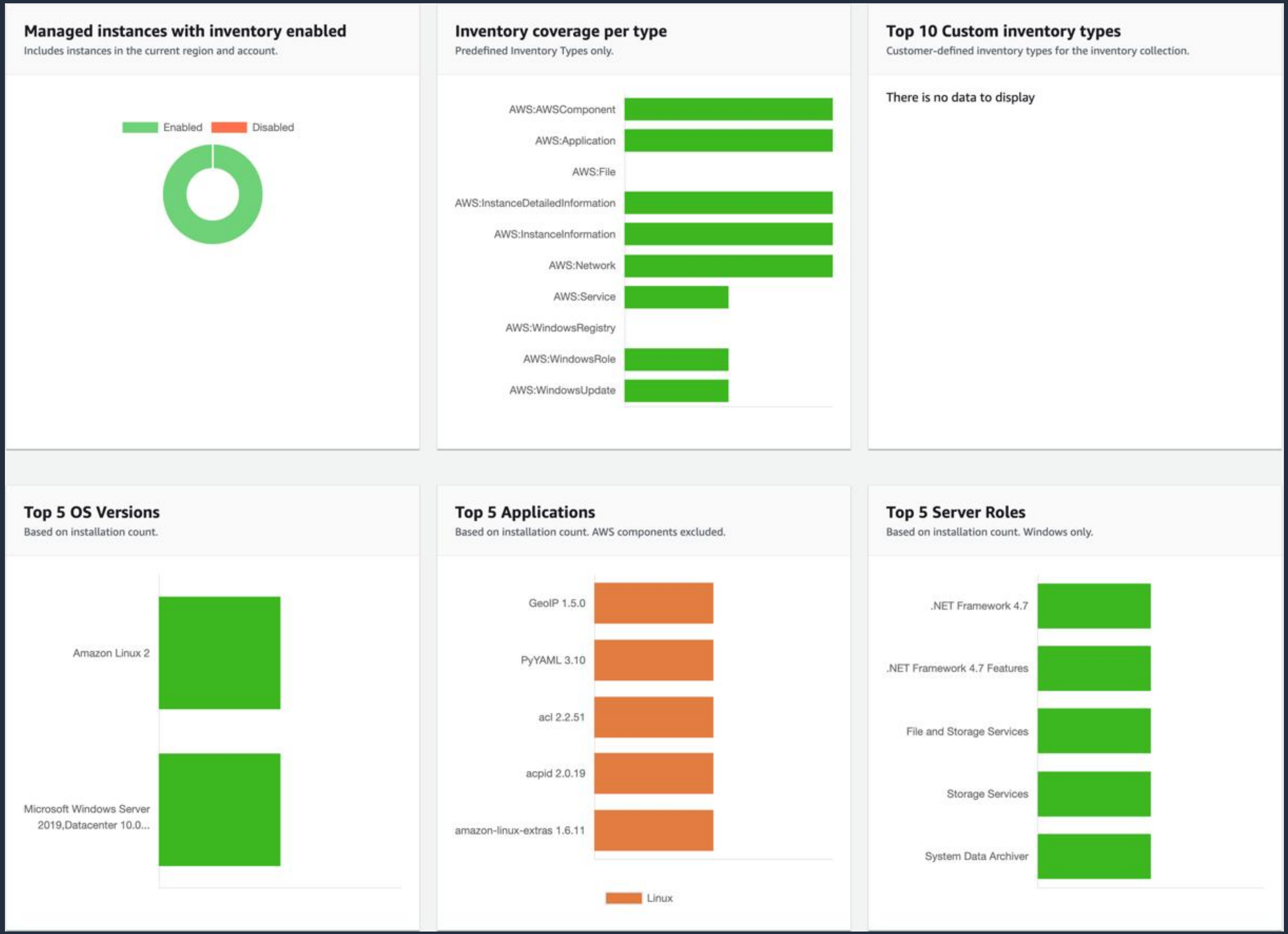
Runs commands on managed EC2 instances

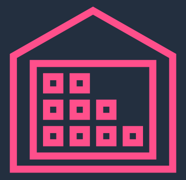

```
{
  "schemaVersion": "1.2",
  "description": "List missing Microsoft Windows updates.",
  "parameters": {
    "UpdateLevel": {
      "type": "String",
      "default": "Important",
      "description": "Important: .......",
      "allowedValues": [
        "None",
        "All",
        "Important",
        "Optional"
      ]
    },
```

This command checks for missing updates

DigitalCloud TRAINING

AWS Systems Manager Inventory





Inventory

AWS Systems Manager Patch Manager

- Helps you select and deploy operating system and software patches automatically across large groups of Amazon EC2 or on-premises instances
- Patch baselines:
  - Set rules to auto-approve select categories of patches to be installed
  - Specify a list of patches that override these rules and are automatically approved or rejected
- You can also schedule maintenance windows for your patches so that they are only applied during predefined times
- Systems Manager helps ensure that your software is up-to-date and meets your compliance policies

Patch Manager

DigitalCloud TRAINING

AWS Systems Manager Compliance

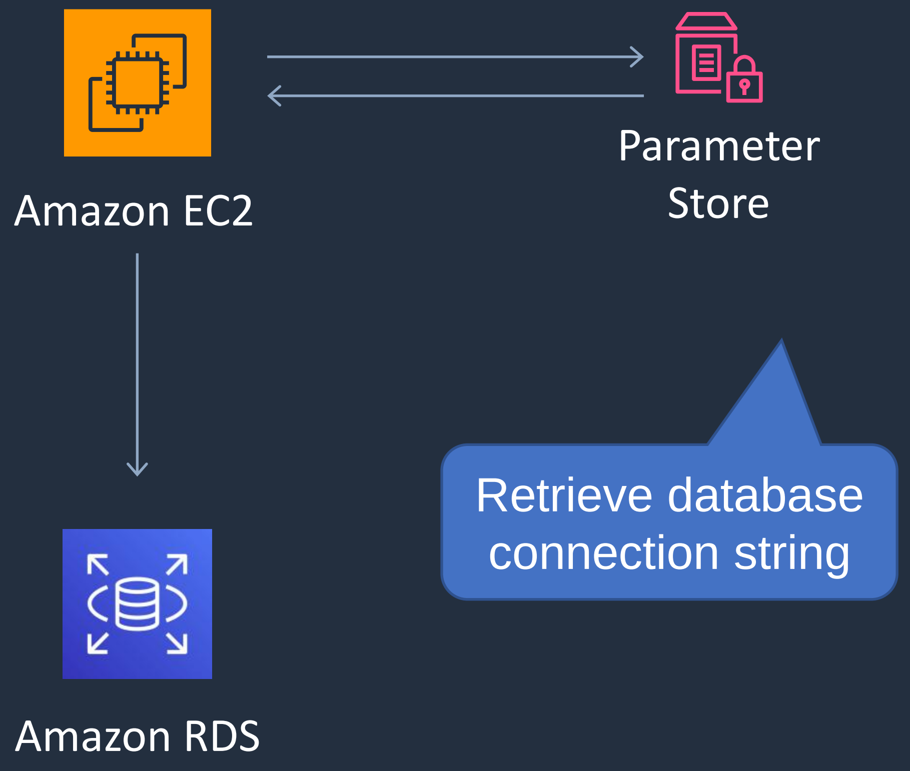
- AWS Systems Manager lets you scan your managed instances for patch compliance and configuration inconsistencies
- You can collect and aggregate data from multiple AWS accounts and Regions, and then drill down into specific resources that aren’t compliant
- By default, AWS Systems Manager displays data about patching and associations
- You can also customize the service and create your own compliance types based on your requirements (must use the AWS CLI, AWS Tools for Windows PowerShell, or the SDKs)

AWS Systems Manager Session Manager

- Secure remote management of your instances at scale without logging into your servers
- Replaces the need for bastion hosts, SSH, or remote PowerShell
- Integrates with IAM for granular permissions
- All actions taken with Systems Manager are recorded by AWS CloudTrail
- Can store session logs in an S3 and output to CloudWatch Logs
- Requires IAM permissions for EC2 instance to access SSM, S3, and CloudWatch Logs

AWS Systems Manager Parameter Store

- Parameter Store provides secure, hierarchical storage for configuration data management and secrets management
- Highly scalable, available, and durable
- Store data such as passwords, database strings, and license codes as parameter values
- Store values as plaintext (unencrypted data) or ciphertext (encrypted data)
- Reference values by using the unique name that you specified when you created the parameter
- No native rotation of keys (difference with AWS Secrets Manager which does it automatically)



Launch EC2 Managed Instances

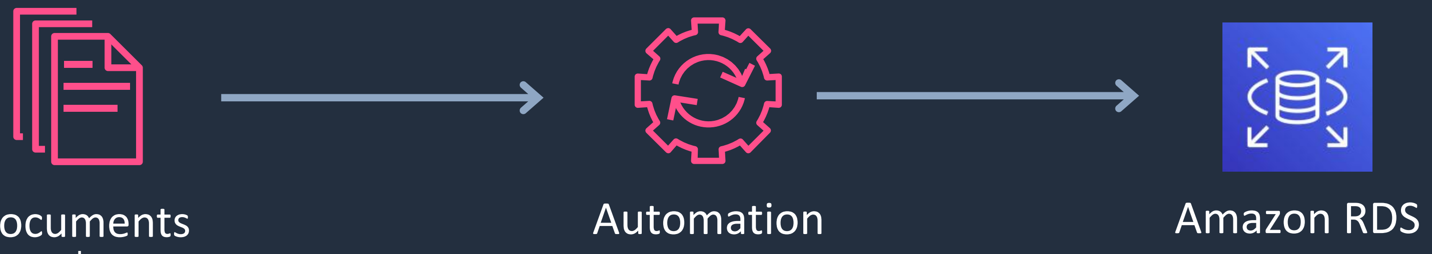
SSM Automation and Config Rules

Systems Manager Automation

AWS Systems Manager Automation

Documents define the actions to perform (YAML or JSON)

Automates **IT operations** and **management** tasks across AWS resources




```yaml
description: Creates an RDS Snapshot for an RDS instance.
schemaVersion: '0.3'
assumeRole: "{{AutomationAssumeRole}}"
parameters:
  DBInstanceIdentifier:
    type: String
    description: (Required) The DBInstanceId ID of the RDS Instance.
  DBSnapshotIdentifier:
    type: String
    description: (Optional) The DBSnapshotIdentifier ID.
    default: ''
  InstanceTags:
    type: String
    default: ''
    description: (Optional) Tags to create for instance.
  SnapshotTags:
    type: String
    default: ''
    description: (Optional) Tags to create for snapshot
```

This automation, takes a snapshot of an RDS DB instance

Systems Manager Run Command and Patch Manager

AWS Systems Manager Run Command

Document types include **command**, **automation**, **package** etc.

Runs commands on managed EC2 instances


```
{
  "schemaVersion": "1.2",
  "description": "List missing Microsoft Windows updates.",
  "parameters": {
    "UpdateLevel": {
      "type": "String",
      "default": "Important",
      "description": "Important: .......",
      "allowedValues": [
        "None",
        "All",
        "Important",
        "Optional"
      ]
    },
```

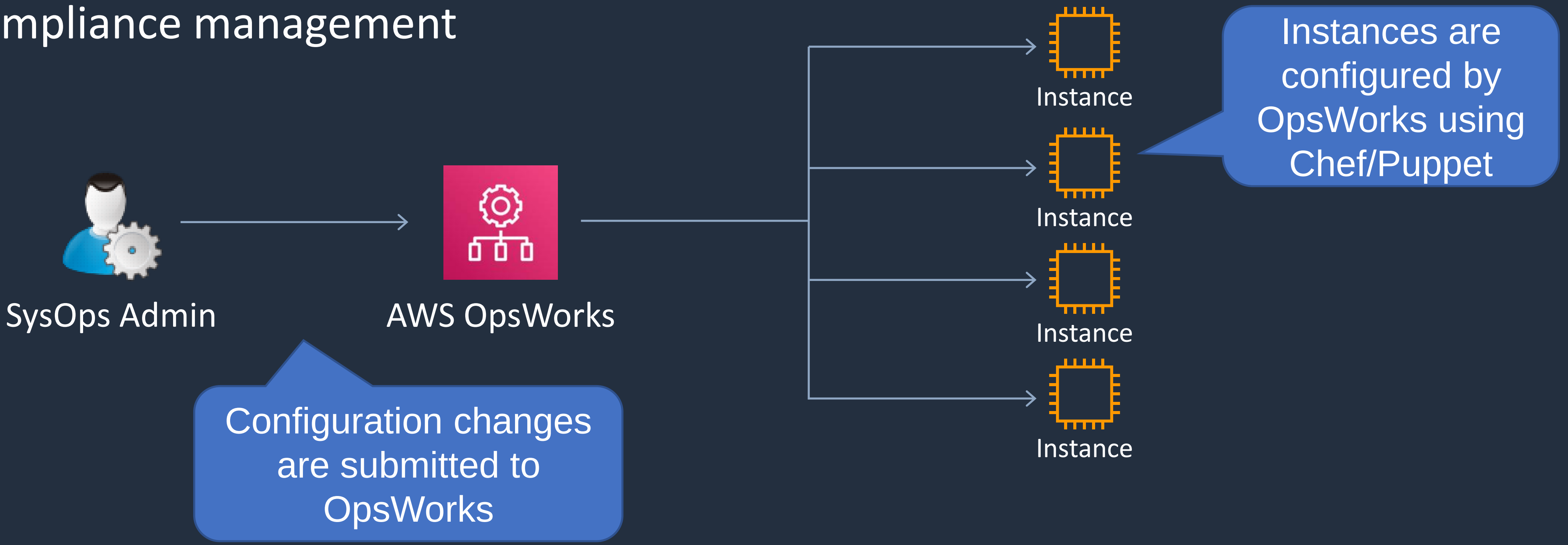
This command checks for missing updates

Systems Manager Configuration Compliance

AWS OpsWorks

AWS OpsWorks

- AWS OpsWorks is a configuration management service that provides managed instances of Chef and Puppet
- Updates include patching, updating, backup, configuration and compliance management

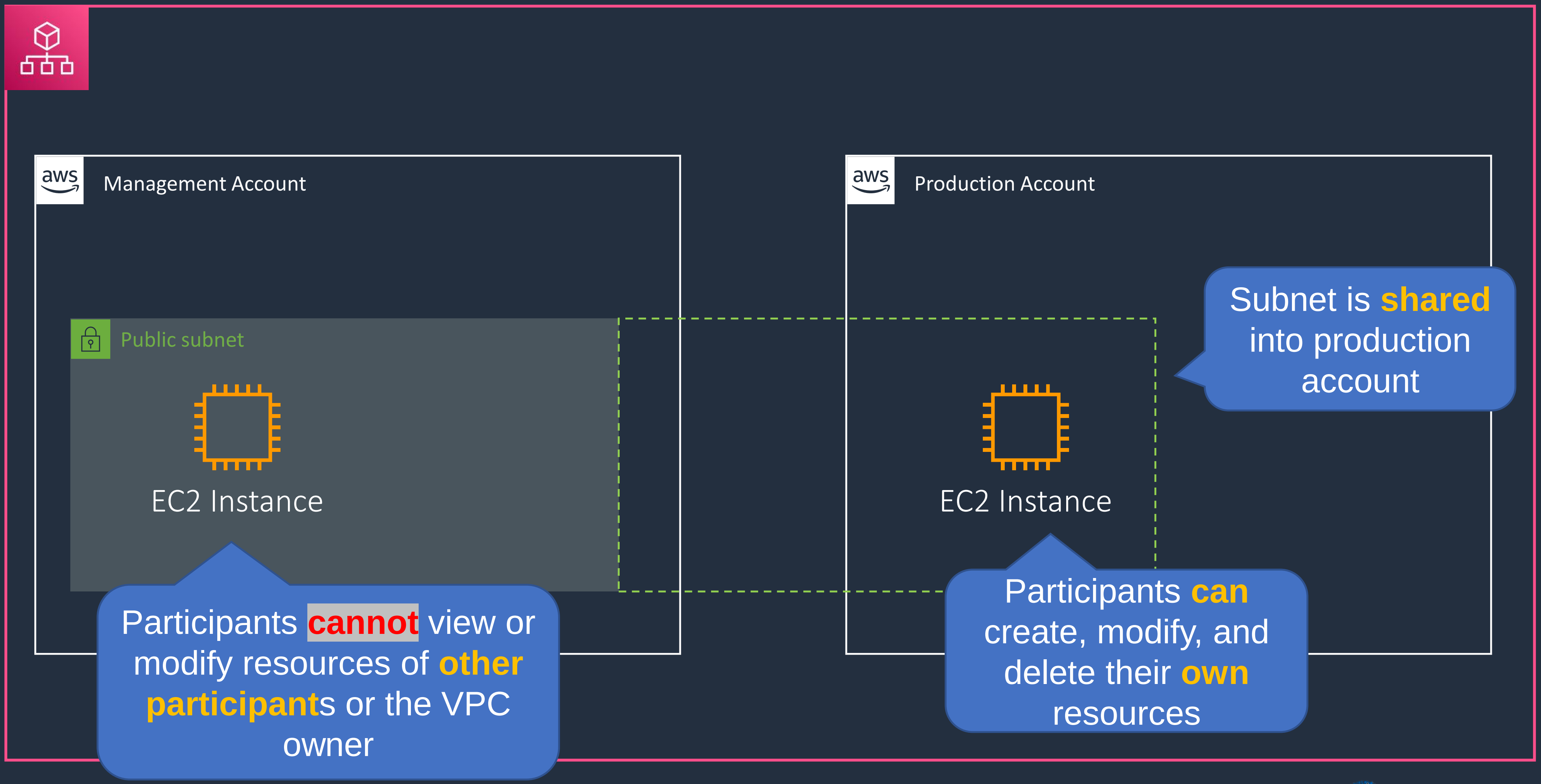


AWS Resource Access Manager (RAM)

AWS RAM

- Shares resources with accounts or organizations
- RAM can be used to share:
  - AWS Transit Gateways
  - Subnets
  - AWS License Manager configurations
  - Amazon Route 53 Resolver rules

AWS RAM



Management Account
Production Account
Public subnet
EC2 Instance
EC2 Instance
Subnet is shared into production account
Participants cannot view or modify resources of other participants or the VPC owner
Participants can create, modify, and delete their own resources

AWS Health API and Dashboards

AWS Personal Health Dashboard

- AWS Personal Health Dashboard provides alerts and remediation guidance when AWS is experiencing events that may **impact you**
- Personal Health Dashboard gives you a **personalized** view into the performance and availability of the AWS services underlying your AWS resources
- Also provides proactive notification to help you plan for scheduled activities

AWS Service Health Dashboard

Not personalized information so may not be relevant to you

No proactive notification of scheduled activities

Current Status - **Jun 7, 2020 PDT**

Amazon Web Services publishes our most up-to-the-minute information on service availability in the table below. Check back here any time to get current status information, or subscribe to an RSS feed to be notified of interruptions to each individual service. If you are experiencing a real-time, operational issue with one of our services that is not described below, please inform us by clicking on the "Contact Us" link to submit a service issue report. All dates and times are Pacific Time (PST/PDT).

North America | South America | Europe | Africa | Asia Pacific | Middle East | Contact Us

Recent Events	Details	RSS
No recent events.		

Remaining Services	Details	RSS
Alexa for Business (N. Virginia)	Service is operating normally	
Amazon API Gateway (Montreal)	Service is operating normally	
Amazon API Gateway (N. California)	Service is operating normally	
Amazon API Gateway (N. Virginia)	Service is operating normally	
Amazon API Gateway (Ohio)	Service is operating normally	
Amazon API Gateway (Oregon)	Service is operating normally	
Amazon AppStream 2.0 (N. Virginia)	Service is operating normally	
Amazon AppStream 2.0 (Oregon)	Service is operating normally	
Amazon Athena (Montreal)	Service is operating normally	
Amazon Athena (N. Virginia)	Service is operating normally	
Amazon Athena (Ohio)	Service is operating normally	
Amazon Athena (Oregon)	Service is operating normally	

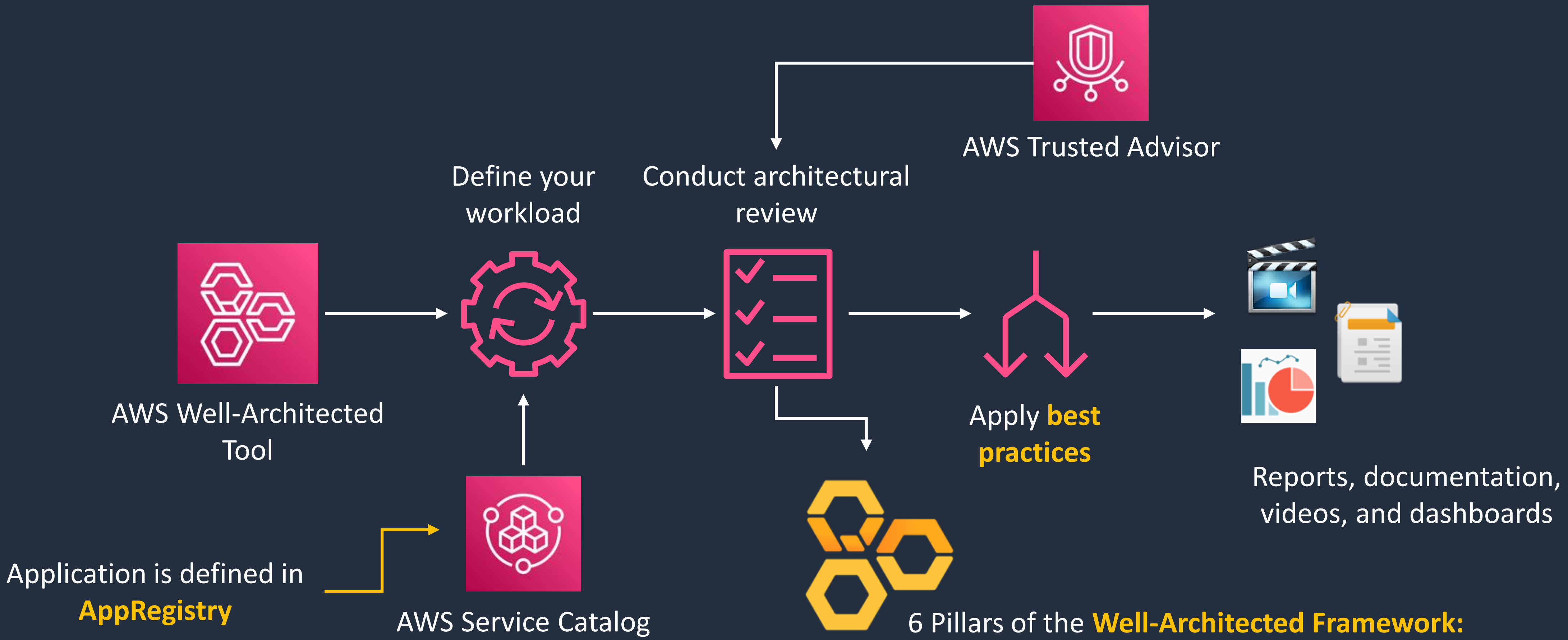
Shows current status information on service availability

DigitalCloud TRAINING

AWS Well-Architected Tool

DigitalCloud TRAINING

AWS Well-Architected Tool



6 Pillars of the **Well-Architected Framework:**

- Operational excellence
- Security
- Reliability
- Performance efficiency
- Cost optimization
- Sustainability

AWS Well-Architected Tool

- Review the state of your applications and workloads against architectural best practices
- Identify opportunities for improvement, and track progress over time
- APIs extend functionality, best practices, measurements, and learnings into your existing architecture governance processes, applications, and workflows
- Create your own custom lenses – add your own best practices pillars and questions based on what’s most important to your organization
- Integrates with AWS Organizations for sharing workloads and custom lenses with up to 300 individual IAM users or accounts
- Integrates with Trusted Advisor to aid workload reviews, providing you automated context for supported questions

AWS Well-Architected Lenses

- **AWS Well-Architected Lenses** extend the guidance offered by AWS Well-Architected to specific industry and technology domains
- Lenses provide a way for you to consistently measure your architectures against best practices and identify areas for improvement
- The AWS Well-Architected Framework Lens is automatically applied when a workload is defined
- The following lenses are provided by AWS:
  - **The Serverless Lens** focuses on designing, deploying, and architecting your serverless application workloads in the AWS Cloud
  - **The SaaS Lens** focuses on designing, deploying, and architecting your software as a service (SaaS) workloads in the AWS Cloud
  - **The FTR Lens** is designed for independent software vendors (ISVs) preparing for a Foundational Technical Review (FTR) in the AWS Partner Network (APN)

Architecture Patterns – Deployment and Management

Architecture Patterns – Deployment and Management

Requirement	Solution
Global company needs to centrally manage creation of infrastructure services to accounts in AWS Organizations	Define infrastructure in CloudFormation templates, create Service Catalog products and portfolios in central account and share using Organizations
Company is concerned about malicious attacks on RDP and SSH ports for remote access to EC2	Deploy Systems Manager agent and use Session Manager
Development team require method of deploying applications using CloudFormation. Developers typically use JavaScript and TypeScript	Define resources in JavaScript and TypeScript and use the AWS Cloud Development Kit (CDK) to create CloudFormation templates

Architecture Patterns – Deployment and Management

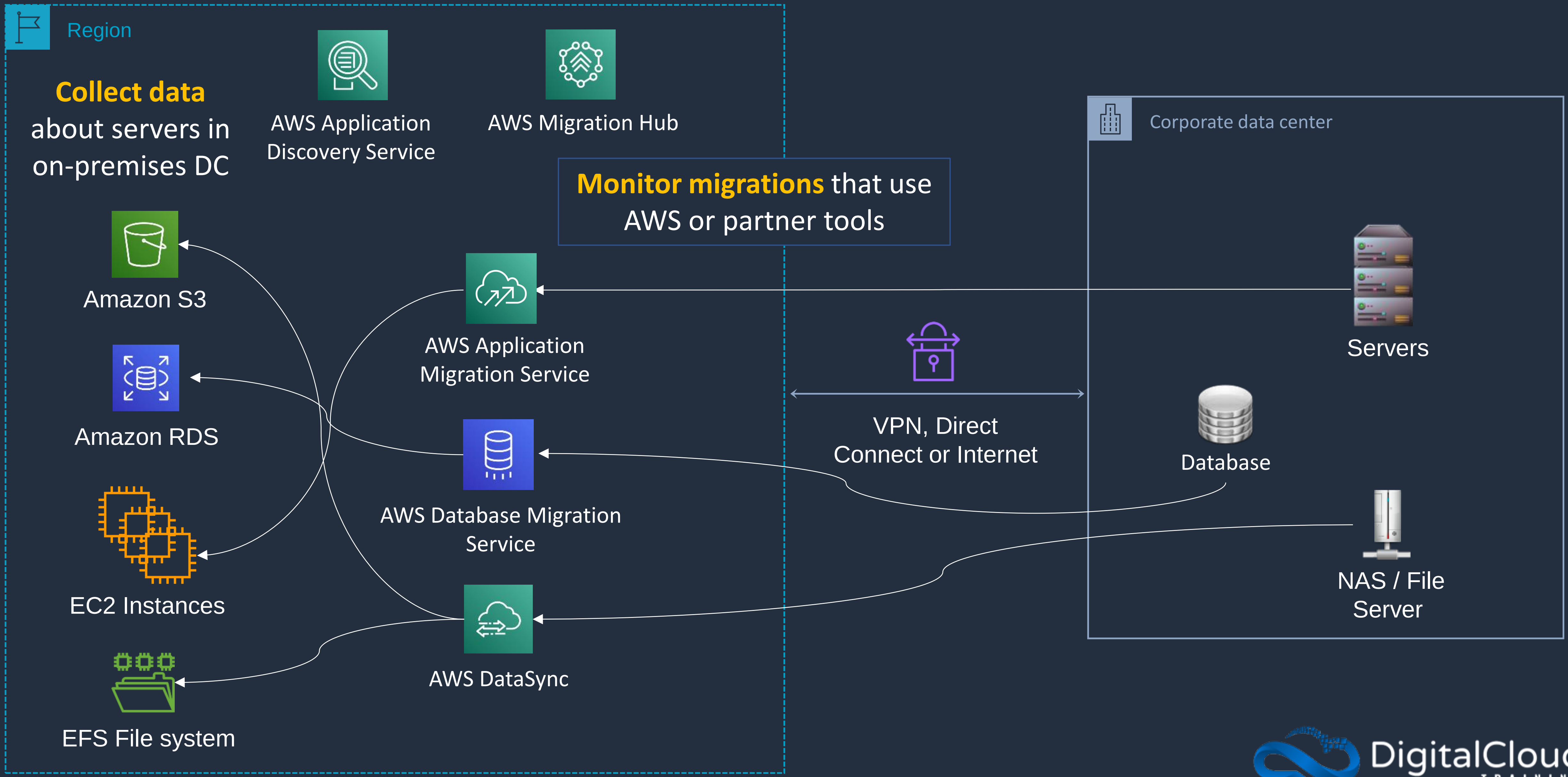
Requirement	Solution
Need to automate the process of updating an application when code is updated	Create a CodePipeline that sources code from CodeCommit and use CodeBuild and CodeDeploy
Need to safely deploy updates to EC2 through a CodePipeline. Resources defined in CF templates and app code stored in S3	Use CodeBuild to automate testing, use CF changes sets to evaluate changes and CodeDeploy to deploy using a blue/green deployment pattern
Company currently uses Chef cookbooks to manage infrastructure and is moving to the cloud. Need to minimize migration complexity	Use AWS OpsWorks for Chef Automate

SECTION 14

Migration and Transfer Services

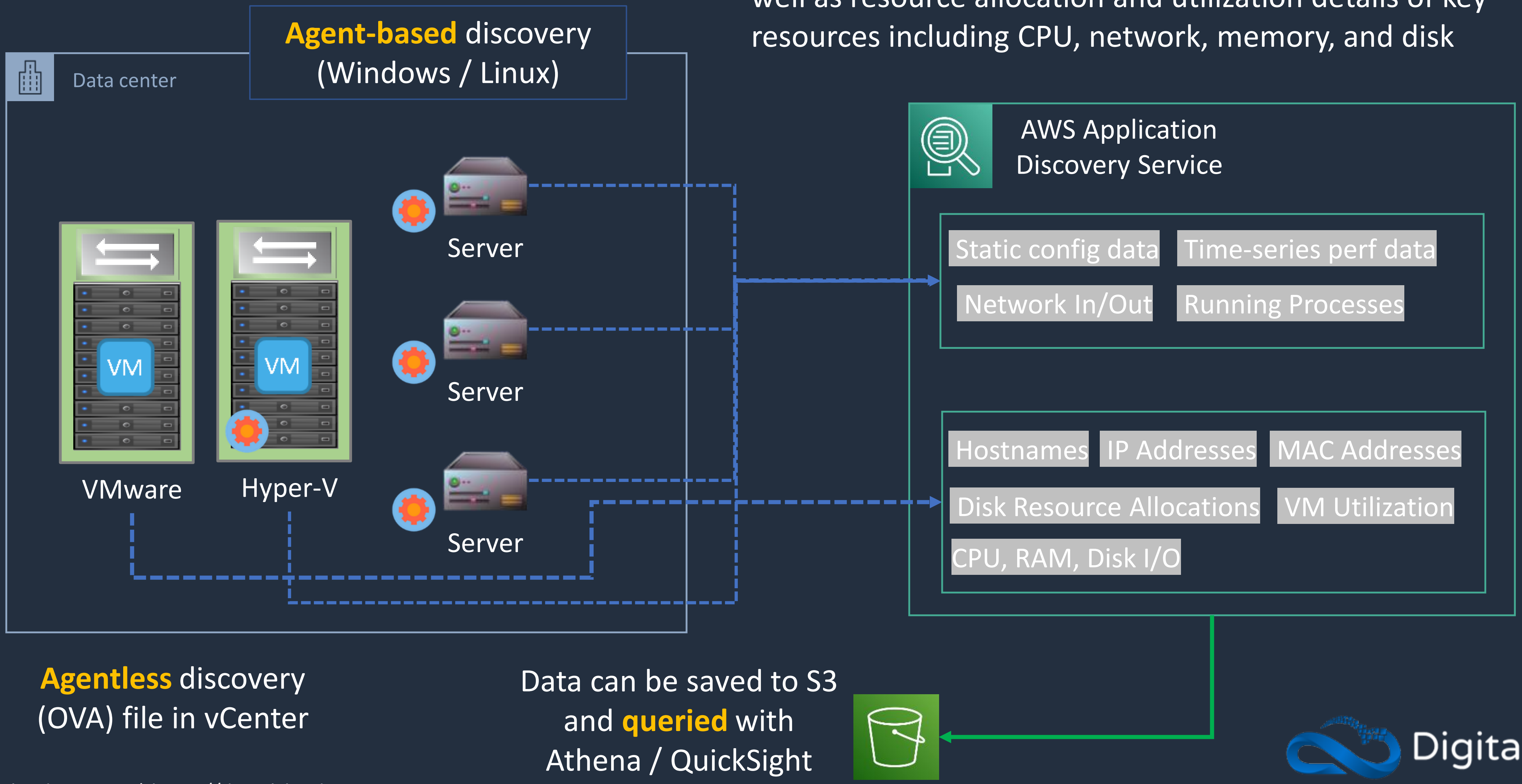
AWS Migration Tools Overview

AWS Migration Tools



AWS Application Discovery Service

Collects server hostnames, IP addresses, MAC addresses, as well as resource allocation and utilization details of key resources including CPU, network, memory, and disk



Agentless discovery
(OVA) file in vCenter

Data can be saved to S3 and **queried** with Athena / QuickSight

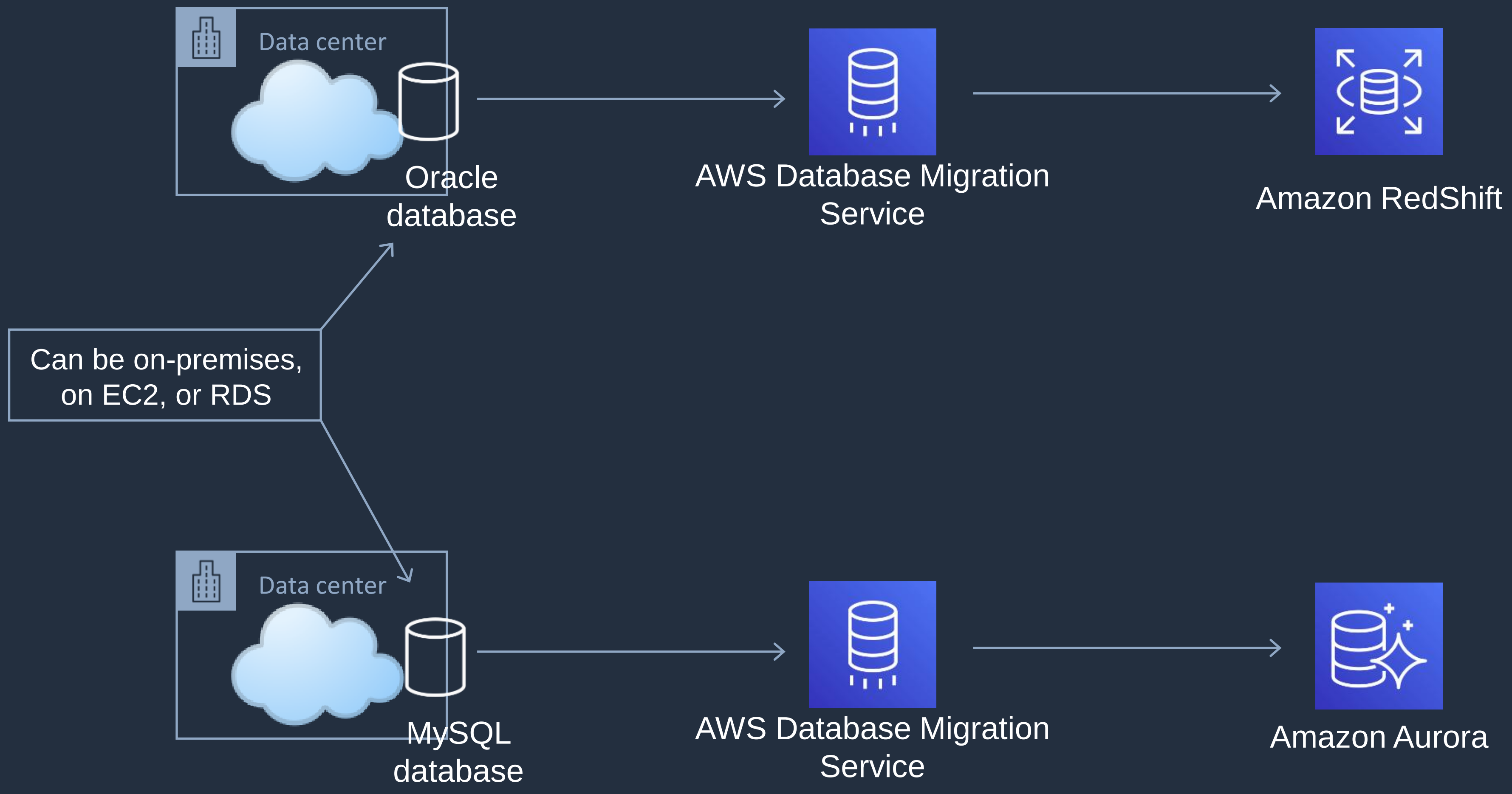
AWS Application Discovery Service

	Discovery Connector	Discovery Agent
Supported Servers	VMware	VMware, Physical
Deployment	Per vCenter	Per Server
Collected data	VM inventory, configuration, and performance history such as CPU, memory, and disk usage	System configuration System performance Running processes Network connections between systems
Supported OS	Any OS running in VMware vCenter (v5.5, v6, & v6.5	Amazon Linux, Ubuntu, RHEL, CentOS, SUSE, Windows Server

AWS Database Migration Service (DMS)

DigitalCloud TRAINING

AWS Database Migration Service (DMS)



Use the **Schema Conversion Tool** for heterogeneous migrations

Destinations include Aurora, RedShift DynamoDB, and DocumentDB

AWS DMS Use Cases

- **Cloud to Cloud** – EC2 to RDS, RDS to RDS, RDS to Aurora
- **On-Premises to Cloud**
- **Homogeneous migrations** – Oracle to Oracle, MySQL to RDS MySQL, Microsoft SQL to RDS for SQL Server
- **Heterogeneous migrations** – Oracle to Aurora, Oracle to PostgreSQL, Microsoft SQL to RDS MySQL (must convert schema first with the **Shema Conversion Tool** (SCT))

AWS DMS Use Cases

- **Development and Test** – use the cloud for dev/test workloads
- **Database consolidation** – consolidate multiple source DBs to a single target DB
- **Continuous Data Replication** – use for DR, dev/test, single source multi-target or multi-source single target

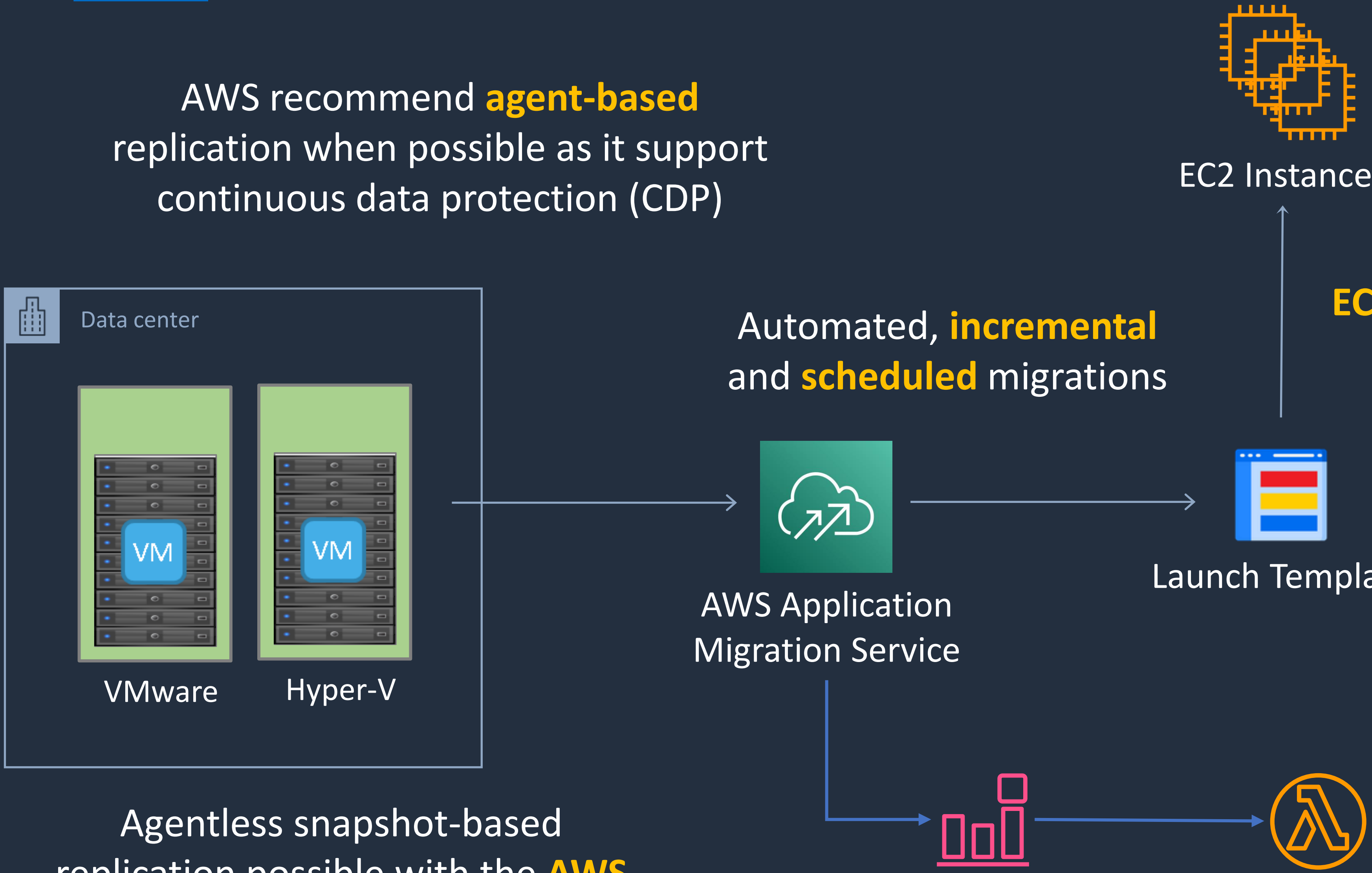
AWS Application Migration Service (MGN)

AWS Application Migration Service (MGN)

AWS recommend **agent-based** replication when possible as it support continuous data protection (CDP)

Automated, **incremental** and **scheduled** migrations

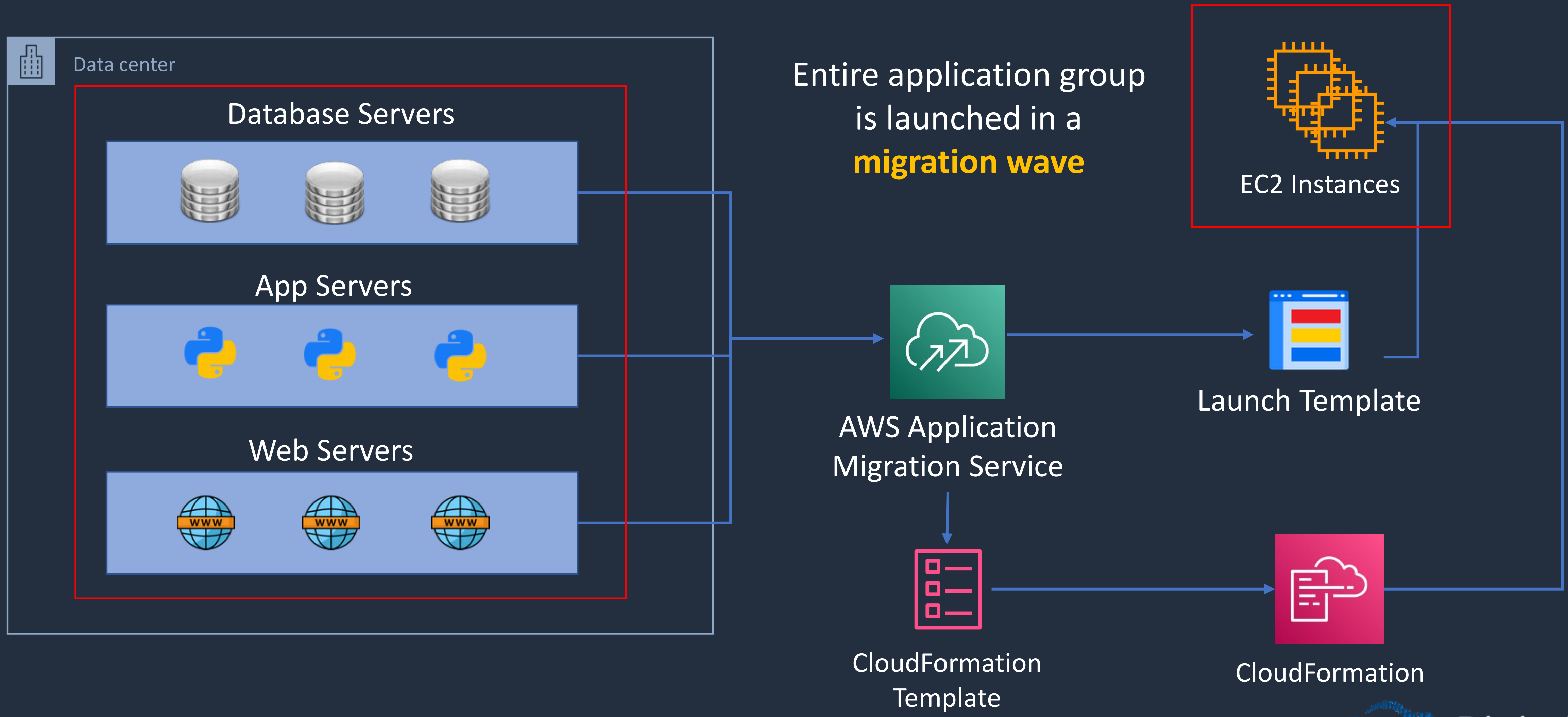
EC2 instances are launched from launch templates



CloudWatch Events and Lambda can **automate** actions

Agentless snapshot-based replication possible with the **AWS MGN vCenter Client** installed

AWS Application Migration Service (MGN)

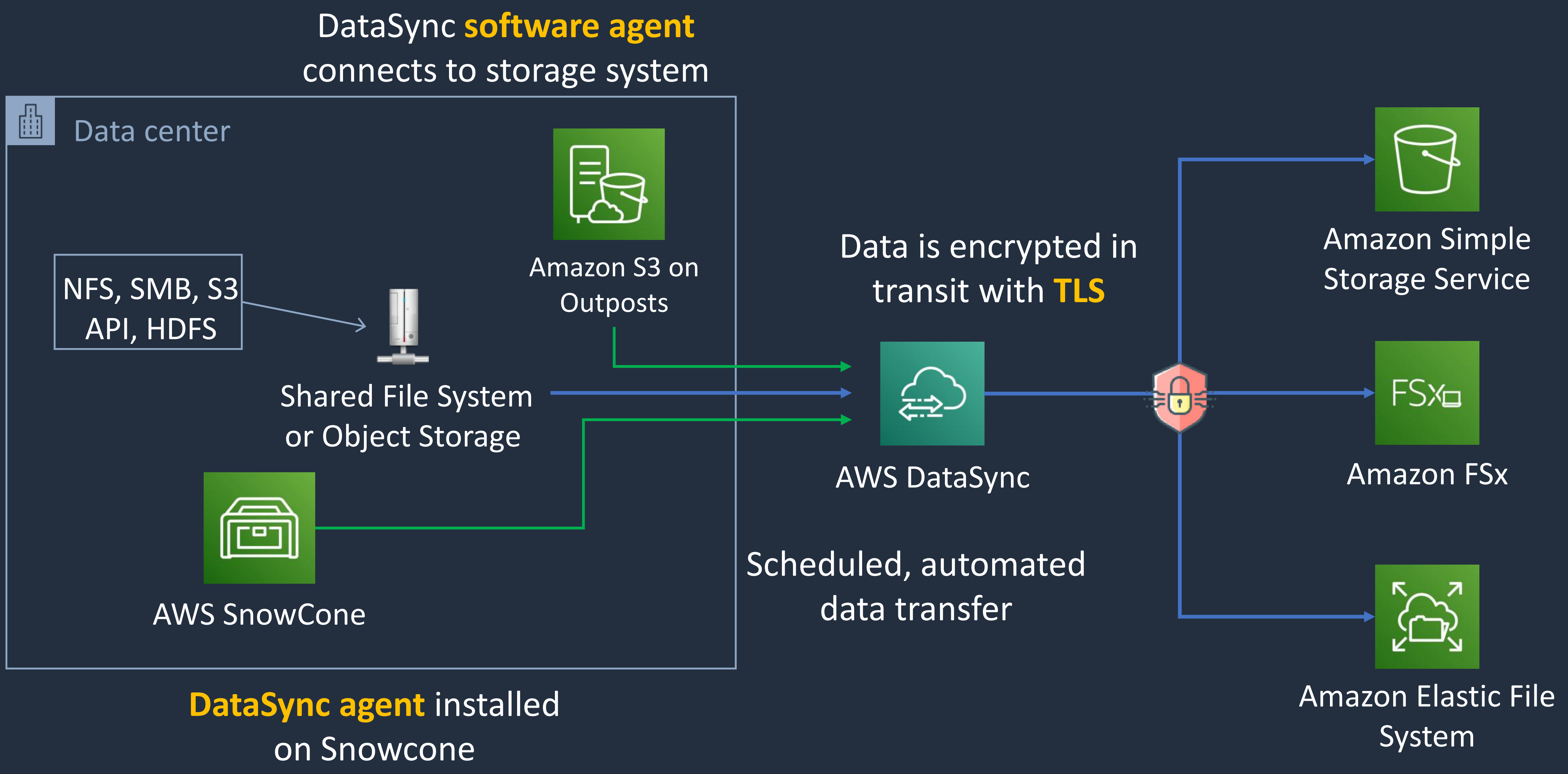


AWS Application Migration Service (MGN)

- Highly automated lift-and-shift (rehost) solution for migrating applications to AWS
- Utilizes continuous, block-level replication and enables short cutover windows measured in minutes
- Server Migration Service (SMS) utilizes incremental, snapshot-based replication and enables cutover windows measured in hours
- AWS recommend using AWS MGN instead of SMS
- Integrates with the Cloud Migration Factory for orchestrating manual processes
- Can migrate virtual and physical servers

AWS DataSync

AWS DataSync



AWS DataSync

- Secure, online service that automates and accelerates moving data between on premises and AWS Storage services
- DataSync can copy data between:
  - Network File System (NFS) shares
  - Server Message Block (SMB) shares
  - Hadoop Distributed File Systems (HDFS)
  - Self-managed object storage
  - AWS Snowcone
  - Amazon S3 buckets
  - Amazon Elastic File System (Amazon EFS) file systems
  - Amazon FSx (Windows, Lustre, OpenZFS, and NetApp ONTAP)

AWS Snow Family

AWS Snowball Family

- **AWS Snowball and Snowmobile** are used for migrating large volumes of data to AWS
- **Snowball Edge Compute Optimized**
  - Provides block and object storage and optional GPU
  - Use for data collection, machine learning and processing, and storage in environments with intermittent connectivity (edge use cases)
- **Snowball Edge Storage Optimized**
  - Provides block storage and Amazon S3-compatible object storage
  - Use for local storage and large-scale data transfer
- **Snowcone**
  - Small device used for edge computing, storage and data transfer
  - Can transfer data offline or online with AWS DataSync agent

DigitalCloud TRAINING

AWS Snowball Family

- Uses a secure storage device for physical transportation
- Snowball Client is software that is installed on a local computer and is used to identify, compress, encrypt, and transfer data
- Uses 256-bit encryption (managed with the AWS KMS) and tamper-resistant enclosures with TPM
- Snowball (80TB) (50TB) “petabyte scale”
- Snowball Edge (100TB) “petabyte scale”
- Snowmobile – “exabyte scale” with up to 100PB per Snowmobile

DigitalCloud TRAINING

AWS Snowball Family

Ways to optimize the performance of Snowball transfers:

1. Use the latest Mac or Linux Snowball client
2. Batch small files together
3. Perform multiple copy operations at one time
4. Copy from multiple workstations
5. Transfer directories, not files

DigitalCloud TRAINING

AWS Snowball Use Cases

- **Cloud data migration** – migrate data to the cloud
- **Content distribution** – send data to clients or customers
- **Tactical Edge Computing** – collect data and compute
- **Machine learning** – run ML directly on the device
- **Manufacturing** – data collection and analysis in the factory
- **Remote locations with simple data** – pre-processing, tagging, compression etc.

The 7 Rs of Migration

The 7 Rs of Migration

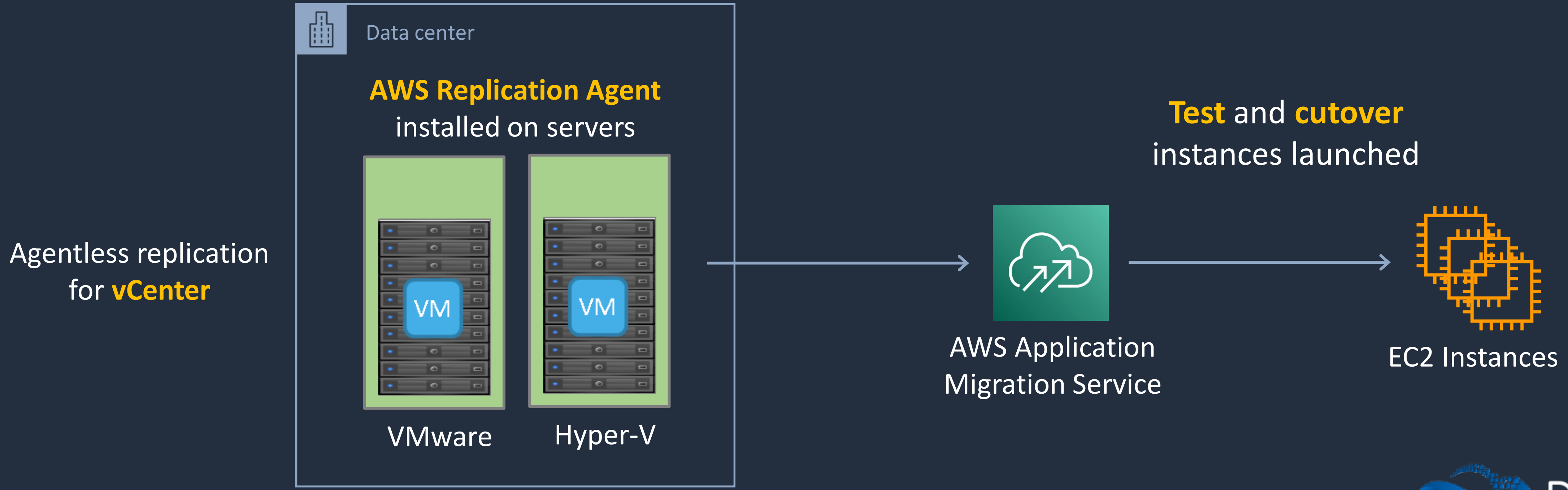
- **Refactor** – Re-architect to a cloud-native serverless architecture
- **Replatform** – Ex. database to RDS; server to Elastic Beanstalk
- **Repurchase** – Use a different solution (e.g. SaaS)
- **Rehost** – OS/App moved to another host system (lift & shift)
- **Relocate** – Move without modification (lift & Shift)
- **Retain** – Leave as is (revisit at a later date)
- **Retire** – No longer needed, get rid of it

Most Effort

Least Effort

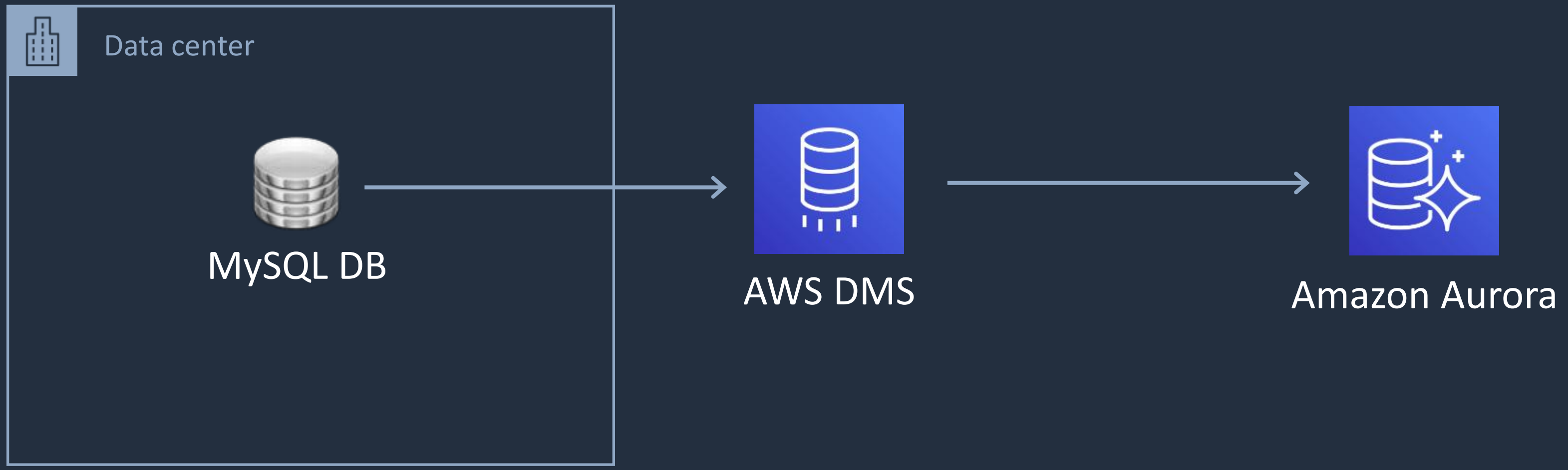
The 7 Rs of Migration

- **Rehost** – OS/App moved to another host system
  - AWS recommend the **AWS Application Migration Service** (AWS MGN) for lift & shift
  - Can also use AWS SMS and AWS VM Import / Export



The 7 Rs of Migration

- **Replatform** – Ex. database to RDS; server to Elastic Beanstalk
  - AWS Database Migration Service (AWS DMS) and Schema Conversion Tool (SCT)
  - Migrate to EB using custom AMI, or code-level migration
  - Moderate development and migration effort



The 7 Rs of Migration

- **Refactor** – Re-architect to a cloud-native serverless architecture
  - Leverage serverless services and event-driven architecture patterns
  - Migrate servers to serverless functions or containers
  - Migrate databases to managed DBs or serverless NoSQL DBs
  - Migrate file stores to object stores or elastic file systems
  - Decouple with queues, notification services, and orchestration tools
  - May involve a large amount of development and migration effort as well as expense

Architecture Patterns – Migration and Transfer

Architecture Patterns – Migration and Transfer

Requirement	Solution
Company is migrating Linux and Windows VMs in VMware to the cloud. Need to determine performance requirements for right-sizing	Install the Application Discovery Service discovery connector in VMware vCenter to gather data
Company has a mixture of VMware VMs and physical servers to migrate to AWS and dependencies exist between application components	Install the AWS MGN replication agent and create Application groups to migrate in waves
Need to migrate an Oracle data warehouse to AWS	Migrate using AWS DMS and SCT to a RedShift data warehouse

Architecture Patterns – Migration and Transfer

Requirement	Solution
Snowball Edge used to transfer millions of small files using a shell script. Transfer times are very slow	Perform multiple copy operations at one time by running each command from a separate terminal, in separate instances of the Snowball client
Need to minimize downtime for servers that must be migrated to AWS	Use AWS MGN and perform a final synchronization before cutting over in a short outage window
Need to migrate 50TB of data and company only has a 1Gbps internet link. Requirement is urgent	Use AWS Snowball to transfer the data

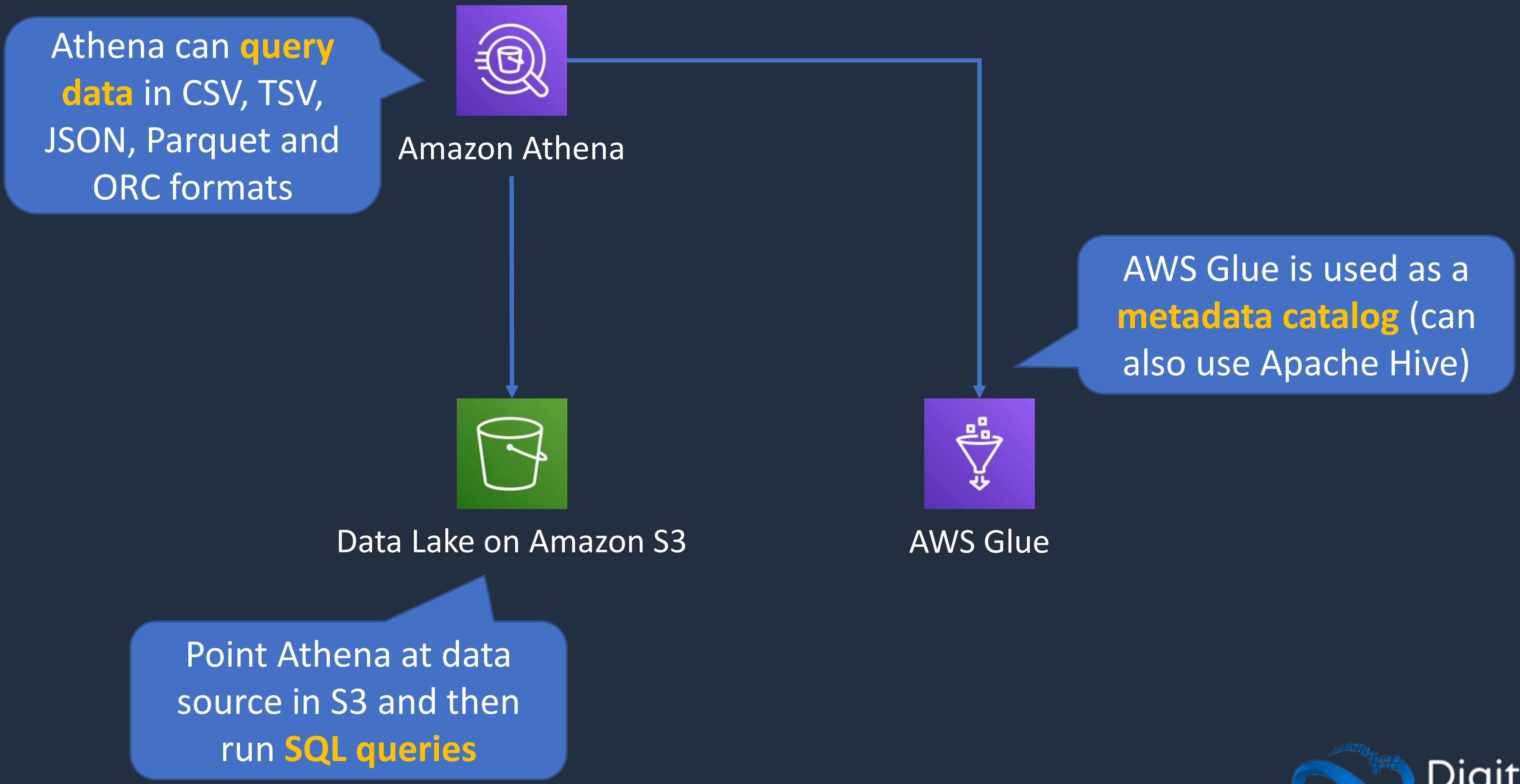
DigitalCloud TRAINING

SECTION 15

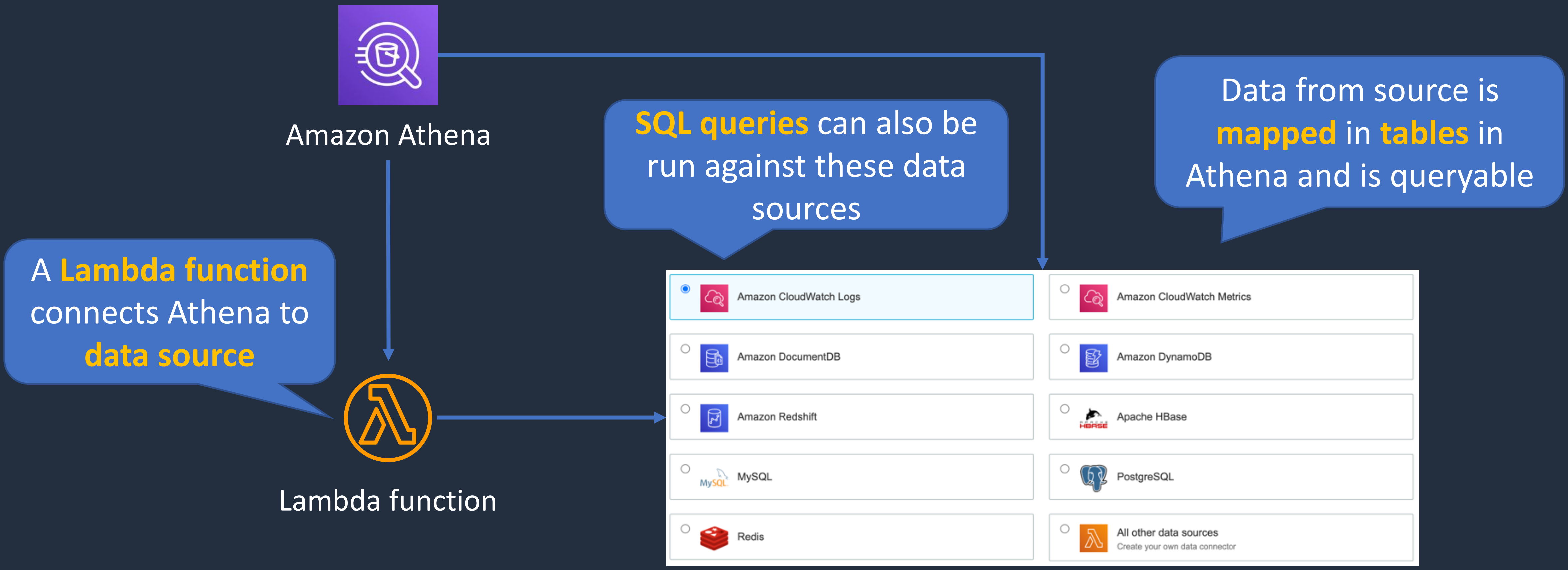
Analytics Services

Amazon Athena and AWS Glue

Amazon Athena and AWS Glue



Amazon Athena and AWS Glue



Amazon Athena

- Athena queries data in S3 using SQL
- Can be connected to other data sources with Lambda
- Data can be in CSV, TSV, JSON, Parquet and ORC formats
- Uses a managed Data Catalog (AWS Glue) to store information and schemas about the databases and tables

Optimizing Athena for Performance

- **Partition your data**
- **Bucket your data** – bucket the data within a single partition
- **Use Compression** – AWS recommend using either Apache Parquet or Apache ORC
- **Optimize file sizes**
- **Optimize columnar data store generation** – Apache Parquet and Apache ORC are popular columnar data stores
- **Optimize ORDER BY and Optimize GROUP BY**
- **Use approximate functions**
- **Only include the columns that you need**

AWS Glue

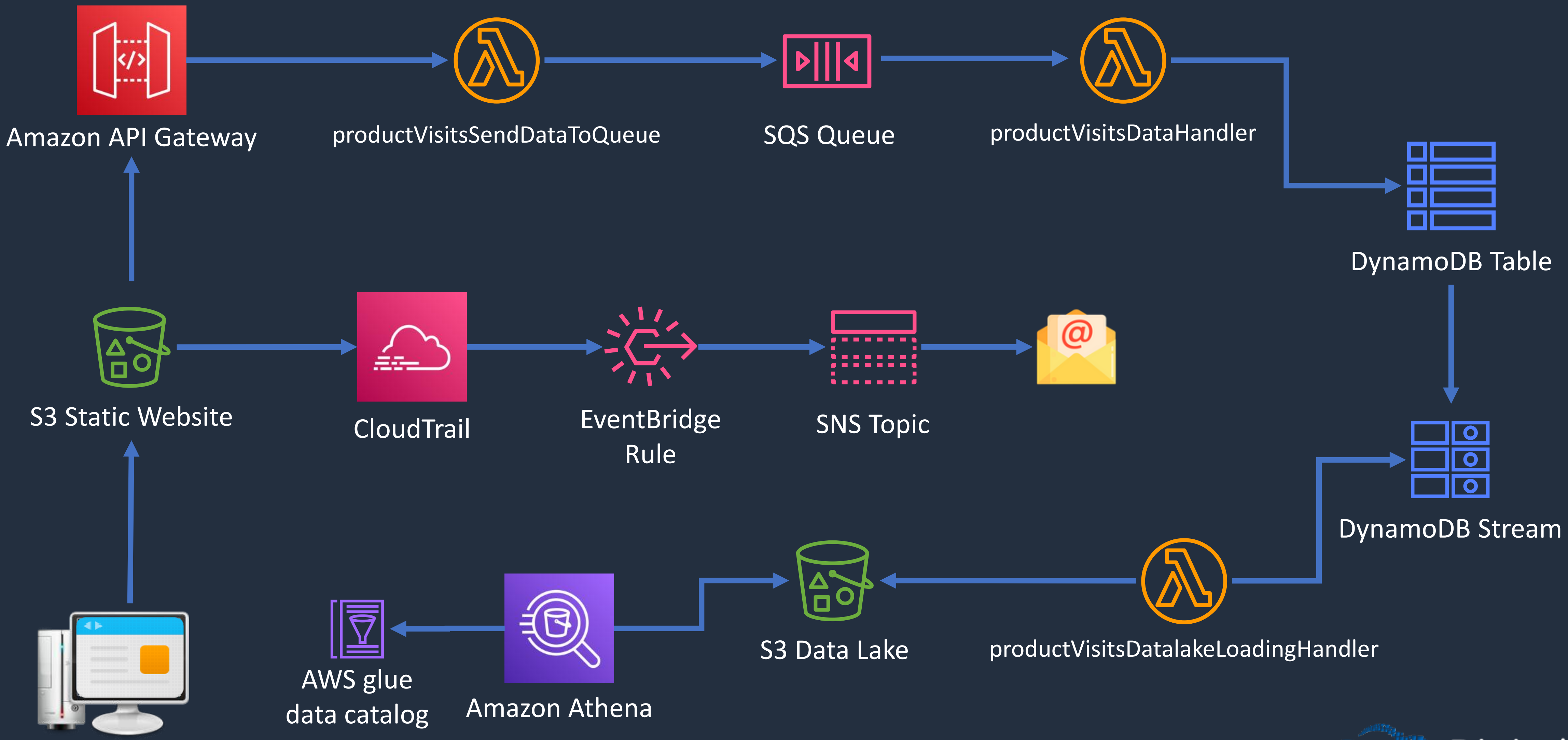
- Fully managed extract, transform and load (ETL) service
- Used for preparing data for analytics
- AWS Glue runs the ETL jobs on a fully managed, scale-out Apache Spark environment
- AWS Glue discovers data and stores the associated metadata (e.g. table definition and schema) in the AWS Glue Data Catalog
- Works with data lakes (e.g. data on S3), data warehouses (including RedShift), and data stores (including RDS or EC2 databases)

AWS Glue

- You can use a **crawler** to populate the AWS Glue Data Catalog with tables
- A crawler can crawl multiple data stores in a single run
- Upon completion, the crawler creates or updates one or more tables in your Data Catalog.
- ETL jobs that you define in AWS Glue use the Data Catalog tables as sources and targets

Build a Serverless App – Part 5

Serverless App Architecture



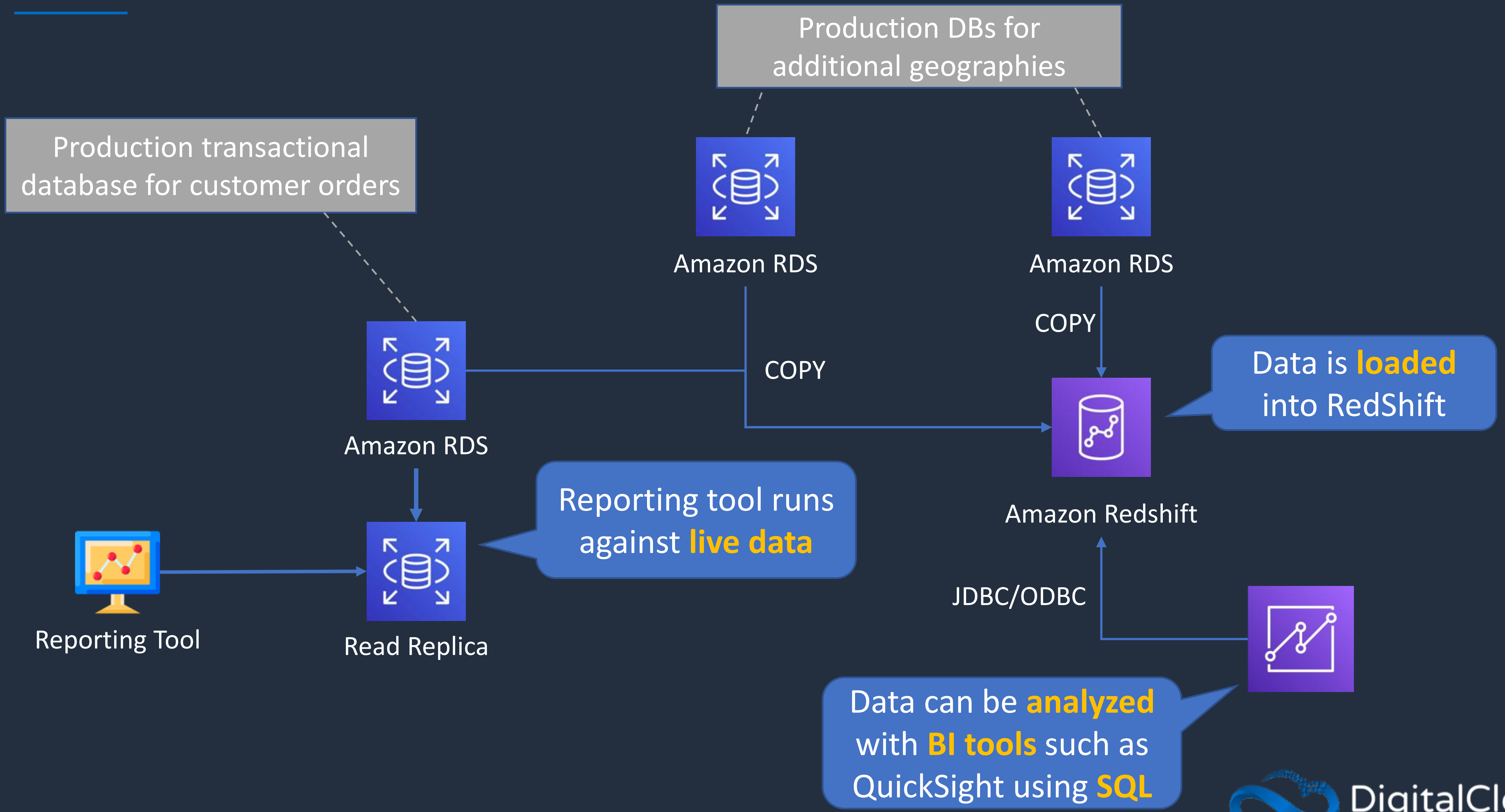
Redshift and OLAP Use Cases

OLTP vs OLAP

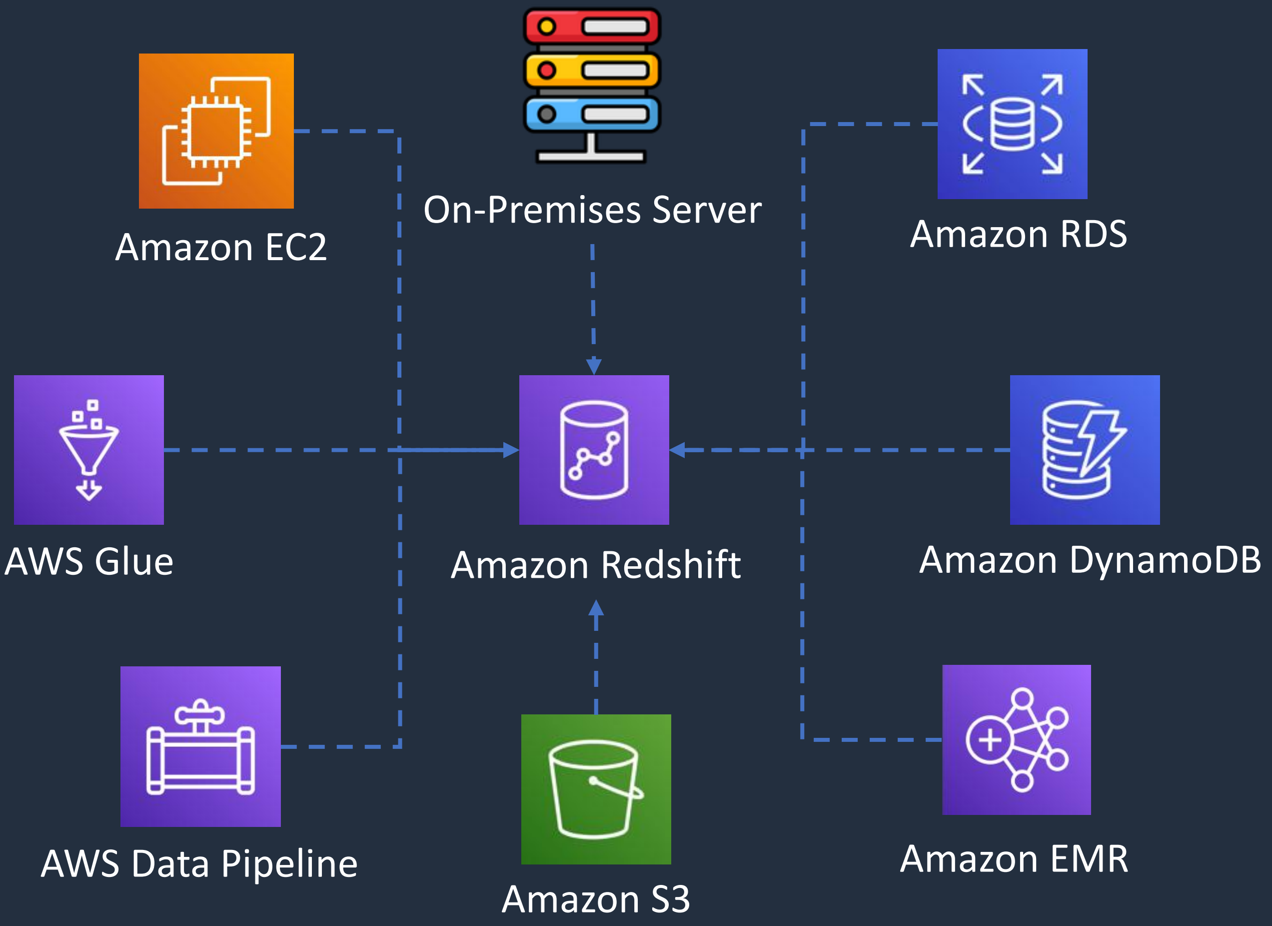
Key differences are ***use cases*** and how the database is ***optimized***

Operational / transactional	Analytical
Online Transaction Processing (OLTP)	Online Analytics Processing (OLAP) – the source data comes from OLTP DBs
Production DBs that process transactions. E.g. adding customer records, checking stock availability (INSERT, UPDATE, DELETE)	Data warehouse. Typically, separated from the customer facing DBs. Data is extracted for decision making
Short transactions and simple queries	Long transactions and complex queries
Examples: Amazon RDS, DynamoDB	Examples: Amazon RedShift, Amazon EMR

Redshift and OLAP Use Cases



Redshift Data Sources



RedShift Spectrum can run SQL queries on data directly in **S3**

RedShift Use Cases

- Perform **complex queries** on massive collections of **structured** and **semi-structured** data and get fast performance
- Frequently accessed data that needs a consistent, highly structured format
- Use **Spectrum** for direct access of **S3 objects** in a data lake
- Managed data warehouse solution with:
  - Automated provisioning, configuration and patching
  - Data durability with continuous backup to S3
  - Scales with simple API calls
  - Exabyte scale query capability

DigitalCloud TRAINING

Amazon EMR

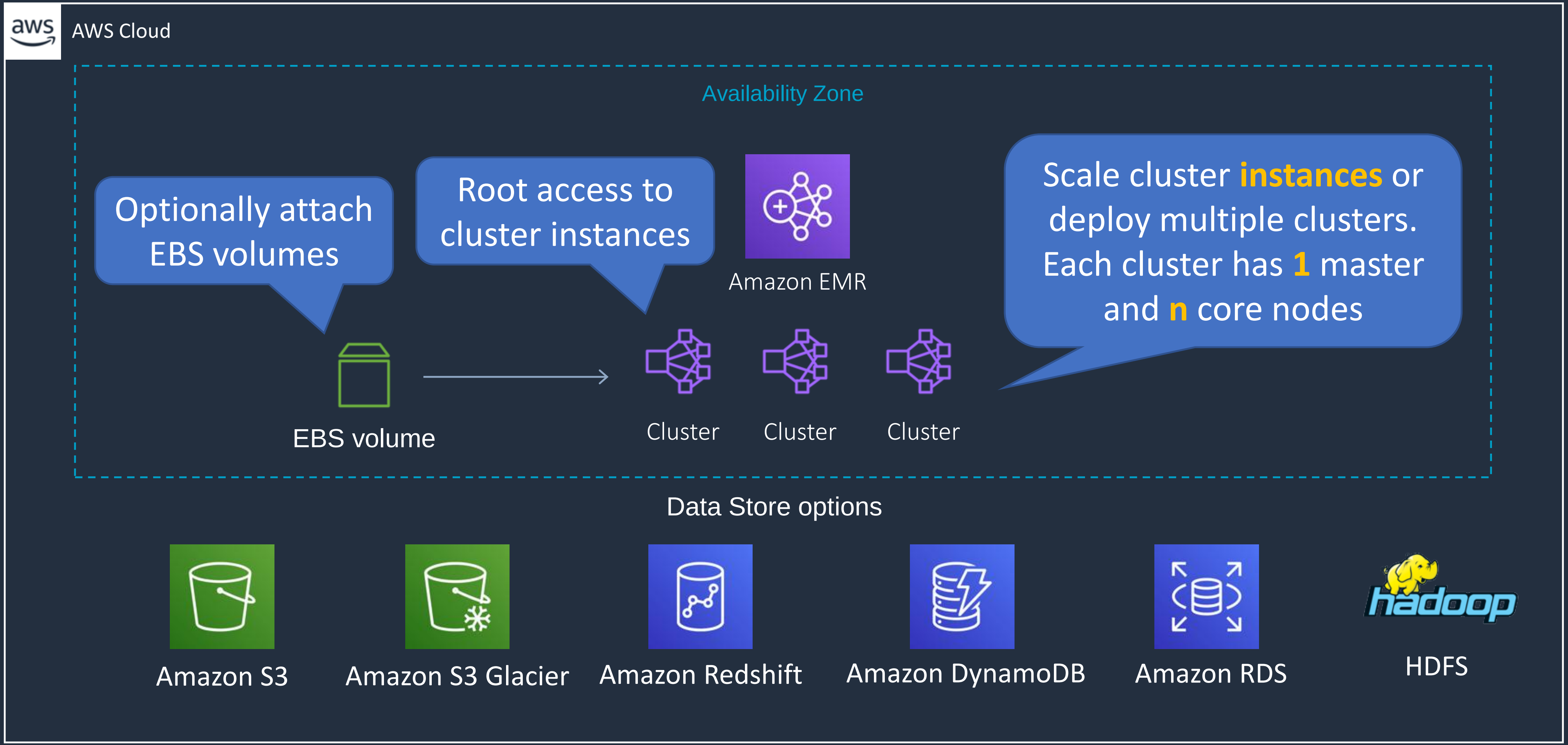
Amazon EMR

- Managed cluster platform that simplifies running big data frameworks including **Apache Hadoop** and **Apache Spark**
- Used for processing data for analytics and business intelligence
- Can also be used for transforming and moving large amounts of data
- Performs extract, transform, and load (ETL) functions

Amazon EMR Integrations

- **Amazon EC2** – the instances that comprise the nodes in the cluster
- **Amazon VPC** – configure the virtual network in which you launch your instances
- **Amazon S3** – store input and output data
- **Amazon CloudWatch** – monitor cluster performance and configure alarms
- **AWS IAM** – configure permissions
- **AWS CloudTrail** – audit requests made to the service
- **AWS Data Pipeline** – schedule and start your clusters
- **AWS Lake Formation** – discover, catalog, and secure data in an Amazon S3 data lake

Amazon EMR Architecture



Amazon EMR Use Cases

1. Data Sources
2. Amazon S3 — Data uploaded to S3 for staging
3. Amazon EMR — EMR sorts, aggregates and joins datasets
4. Amazon Redshift — Processed data loaded into RedShift
5. Reporting Tool — Reporting apps and BI tools analyze data

1. Data Sources
2. Amazon S3 — Data uploaded to S3 for staging
3. Amazon EMR — EMR transforms and cleanses the data (ETL)
4. Amazon S3 — Cleansed data loaded to S3
5. Amazon Redshift — RedShift loads data in parallel, optimizing for fast analytics
6. QuickSight — Analyze and visualize with QuickSight

Amazon Kinesis

Amazon Kinesis Services

Data Analytics uses **Apache Flink** for **processing** data streams

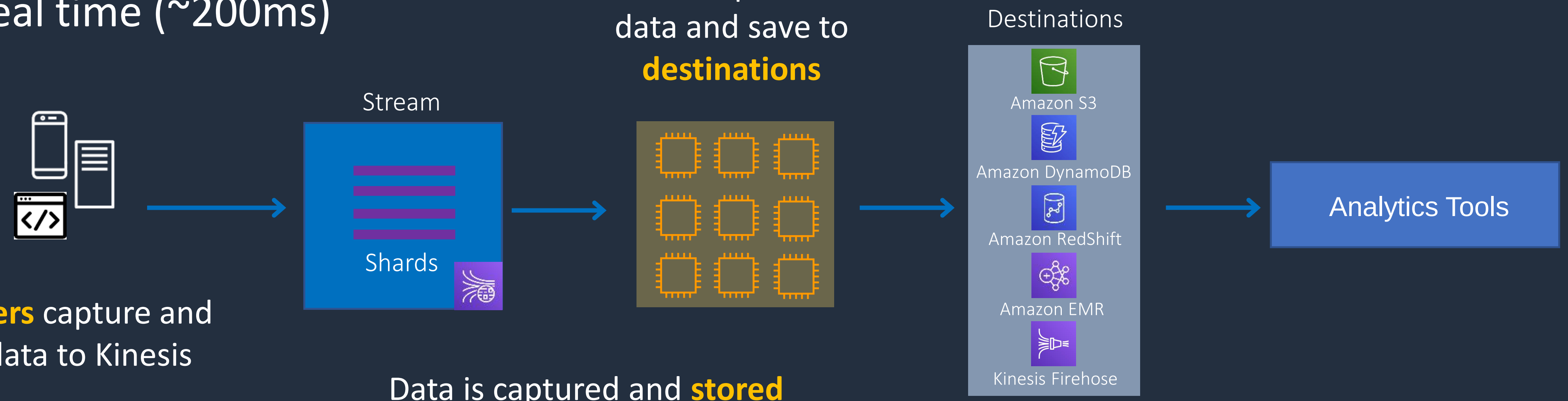
Data is **ingested** into Data Streams

Firehose **loads data** straight to **destinations**

Amazon Kinesis Data Streams

- Producers send data to Kinesis, data is stored in Shards for 24 hours (by default, up to 365 days)
- Consumers then take the data and process it - data can then be saved into another AWS service
- Real time (~200ms)

Consumers process the data and save to **destinations**



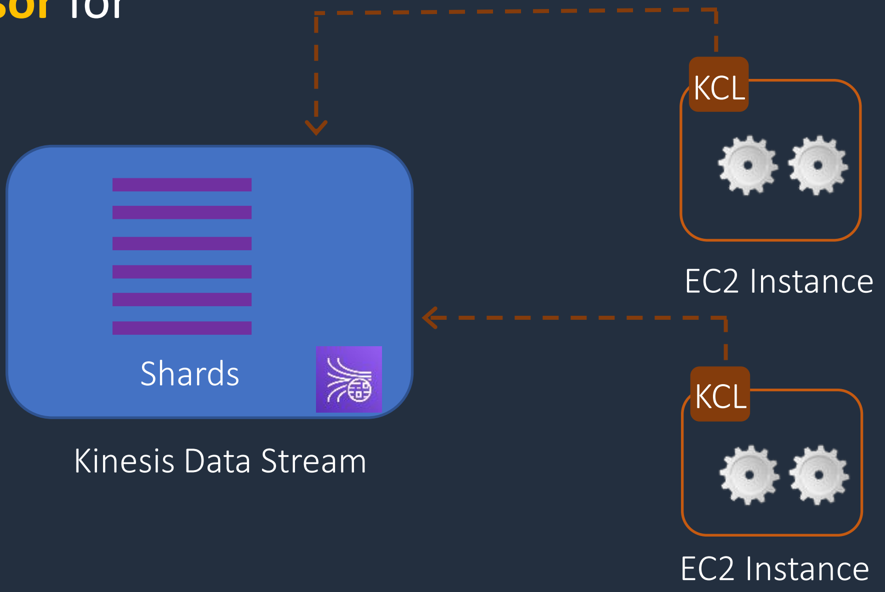
Producers capture and send data to Kinesis

Data is captured and **stored** for **processing**

Kinesis Client Library (KCL)

- The Kinesis Client Library (KCL) helps you consume and process data from a Kinesis data stream

KCL **enumerates shards** and instantiates a **record processor** for each shard it manages

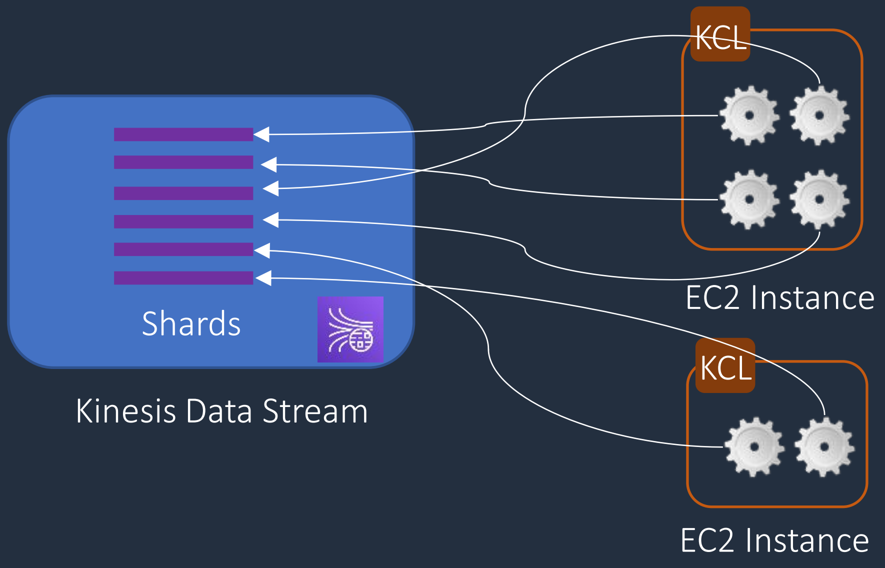


KCL Worker with multiple record processors

Kinesis Client Library (KCL)

- Each shard is processed by exactly one KCL worker and has exactly one corresponding record processor
- One worker can process any number of shards, so it's fine if the number of shards exceeds the number of instances

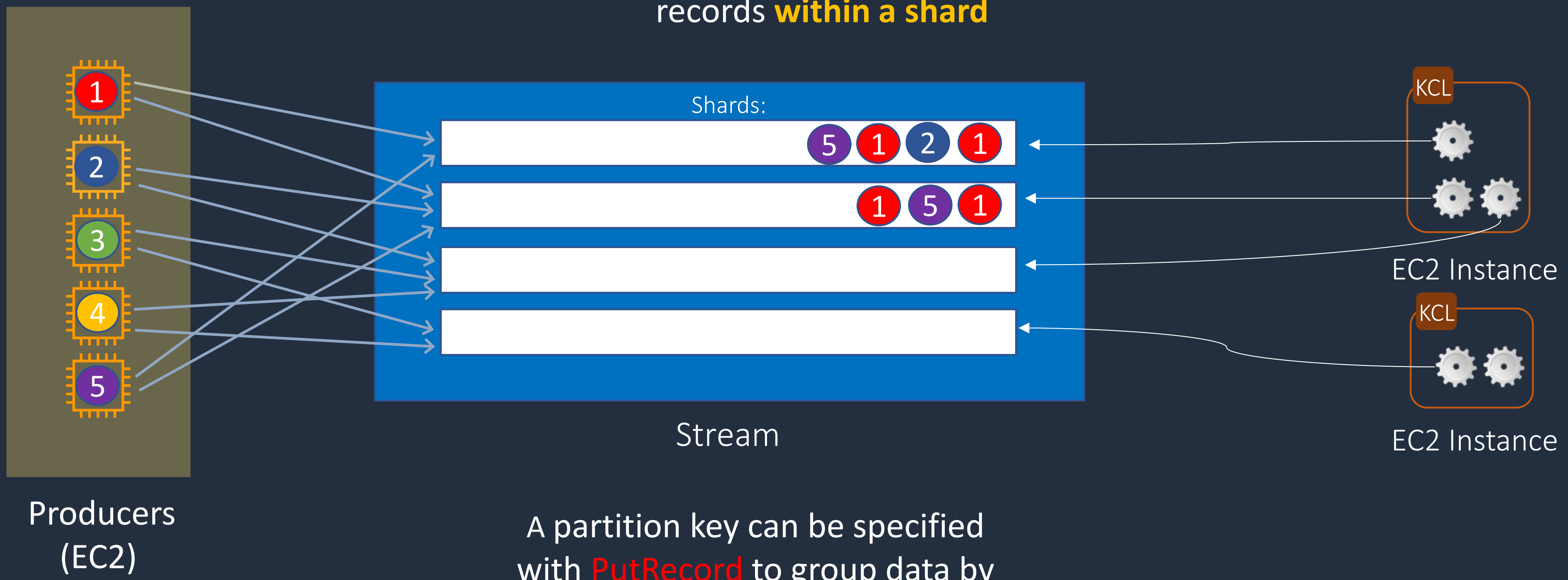
Each shard is **processed** by exactly **one** KCL worker



A **record processor** maps to exactly one **shard**

Amazon Kinesis Data Streams

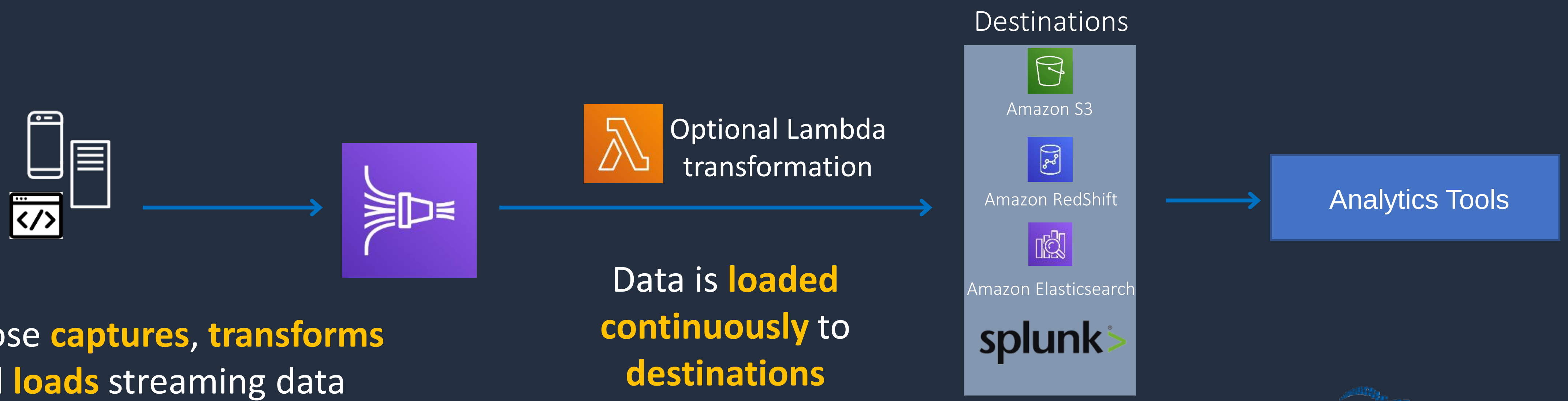
Order is maintained for records **within a shard**



A partition key can be specified with PutRecord to group data by shard

Kinesis Data Firehose

- Producers send data to Firehose
- There are no Shards, completely automated (scalability is elastic)
- Firehose data is sent to another AWS service for storing, data can be optionally processed/transformed using AWS Lambda
- Near real-time delivery (~60 seconds latency)



Firehose **captures**, **transforms** and **loads** streaming data

Data is **loaded continuously** to **destinations**

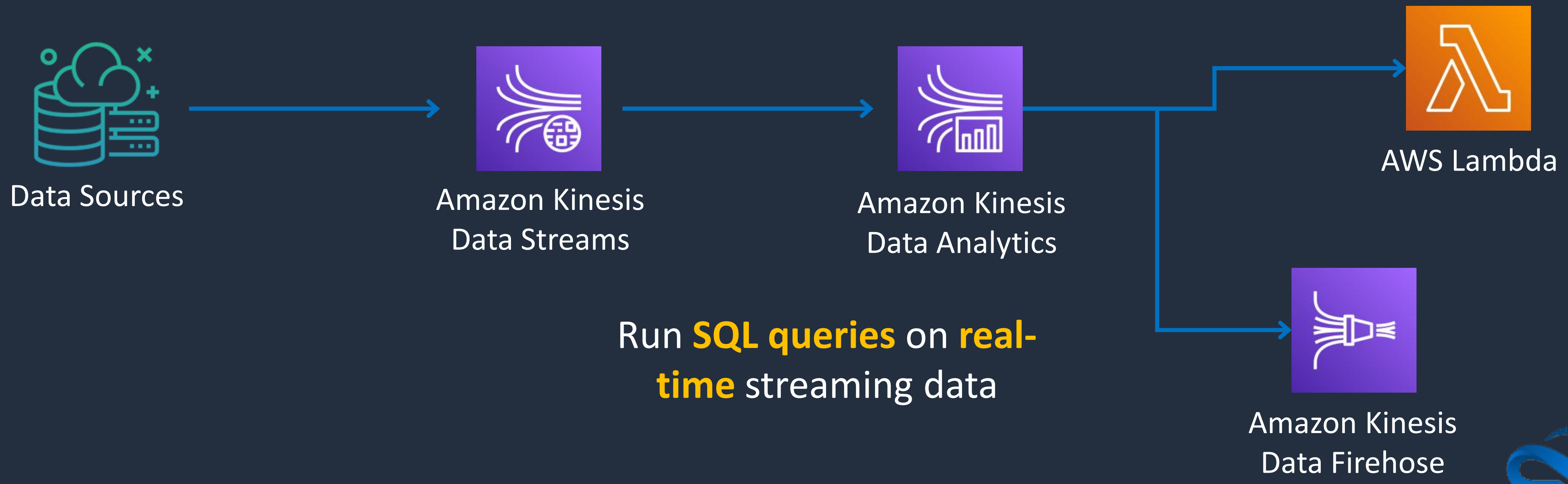
Kinesis Data Firehose

Kinesis Data Firehose destinations (changes over time):

- RedShift (via an intermediate S3 bucket)
- OpenSearch
- Amazon S3
- Splunk
- Datadog
- Honeycomb
- Coralogix
- LogicMonitor
- Logz.io
- MongoDB
- New Relic
- Sumo Logic
- HTTP Endpoint

Kinesis Data Analytics

- Provides real-time SQL processing for streaming data
- Provides analytics for data coming in from Kinesis Data Streams and Kinesis Data Firehose
- Destinations can be Kinesis Data Streams, Kinesis Data Firehose, or AWS Lambda



Run **SQL queries** on **real-time** streaming data

Other Analytics Services

Amazon Timestream

- Time series database service for IoT and operational applications
- Faster and cheaper than relational databases
- Keeps recent data in memory and moves historical data to a cost optimized storage tier based upon user defined policies
- Serverless and scales automatically

AWS Data Exchange

- AWS Data Exchange is a data marketplace with over 3,000 products from 250+ providers
- AWS Data Exchange supports Data Files, Data Tables, and Data APIs
- Consume directly into data lakes, applications, analytics, and machine learning models
- Automatically export new or updated data sets to Amazon S3
- Query data tables with AWS Data Exchange for Amazon Redshift
- Use AWS-native authentication and governance, AWS SDKs, and consistent API documentation

AWS Data Exchange - Benefits

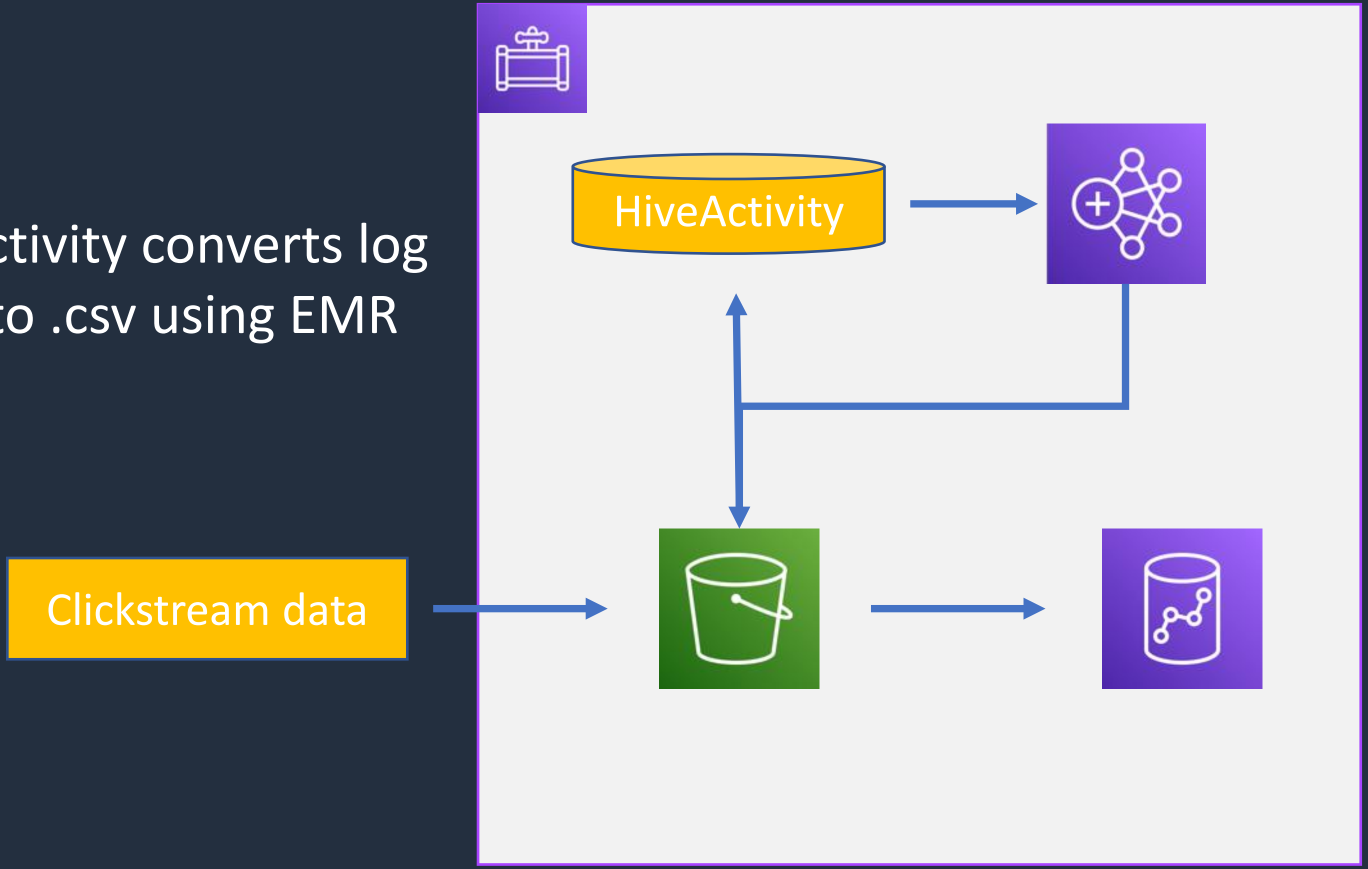
AWS Data Pipeline

- AWS Data Pipeline is a managed ETL (Extract-Transform-Load) service
- Process and move data between different AWS compute and storage services
- Data sources can also be on-premises
- Data can be processed and transformed
- Results can be loaded to services such as Amazon S3, Amazon RDS, Amazon DynamoDB, and Amazon EMR

AWS Data Pipeline – Clickstream Data Example

Pipeline

HiveActivity converts log files to .csv using EMR



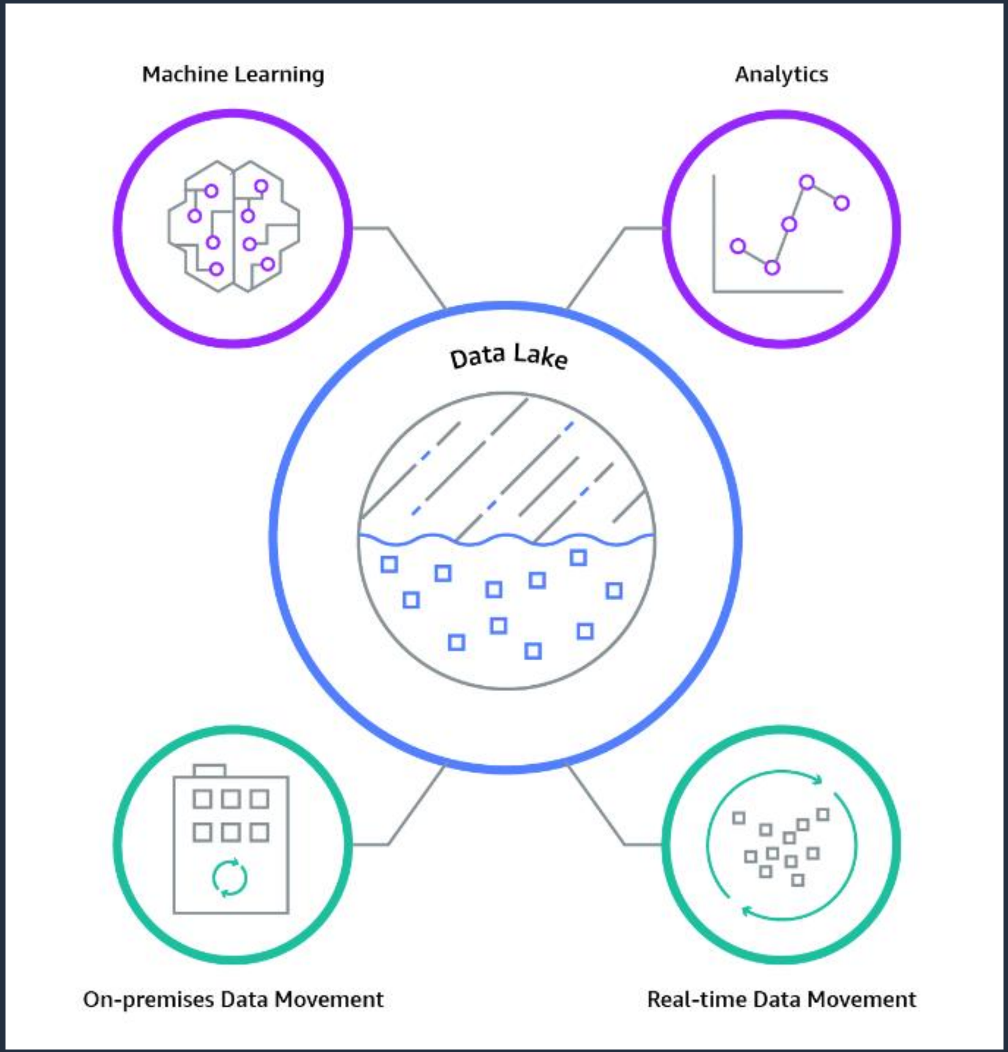
Results saved to S3

RedShiftCopyActivity loads to RedShift

What is a Data Lake?

A data lake is a centralized repository that allows you to store all your structured and unstructured data at any scale

Can create dashboards, and visualizations



You can store your data as-is, without having to first structure the data

Big data processing, real-time analytics, ML

Data Warehouse vs Data Lake

Characteristics	Data Warehouse	Data Lake
Data	Relational from transactional systems, operational databases, and line of business applications	Non-relational and relational from IoT devices, web sites, mobile apps, social media, and corporate applications
Schema	Designed prior to the DW implementation (schema-on-write)	Written at the time of analysis (schema-on-read)
Price/Performance	Fastest query results using higher cost storage	Query results getting faster using low-cost storage
Data Quality	Highly curated data that serves as the central version of the truth	Any data that may or may not be curated (ie. raw data)
Users	Business analysts	Data scientists, Data developers, and Business analysts (using curated data)
Analytics	Batch reporting, BI and visualizations	Machine Learning, Predictive analytics, data discovery and profiling

AWS Lake Formation

- AWS Lake Formation enables you to set up secure data lakes in days
- Data can be collected from databases and object storage
- It is saved to the Amazon S3 data lake
- You can then clean and classify data using ML algorithms
- Security can be applied at column, row, and cell-levels
- The data sets can then be used through services such as Amazon Redshift, Amazon Athena, Amazon EMR for Apache Spark, and Amazon QuickSight
- Lake Formation builds on the capabilities available in AWS Glue

Amazon Managed Streaming for Apache Kafka (MSK)

- Amazon MSK is used for ingesting and processing streaming data in real-time
- Build and run Apache Kafka applications
- It is a fully managed service
- Provisions, configures, and maintains Apache Kafka clusters and Apache ZooKeeper nodes
- Security levels include:
  - VPC network isolation
  - AWS IAM for control-plane API authorization
  - Encryption at rest
  - TLS encryption in-transit
  - TLS based certificate authentication
  - SASL/SCRAM authentication secured by AWS Secrets Manager

Architecture Patterns - Analytics

Architecture Patterns – Analytics

Requirement	Solution
Athena is being used to analyze a large volume of data based on data ranges. Performance must be optimized	Store data using Apache Hive partitioning with a key based on the data. Use the Apache Parquet and ORC storage formats
Lambda is processing streaming data from API Gateway and is generating a TooManyRequestsException as volume increases	Stream the data into a Kinesis Data Stream from API Gateway and process in batches
Lambda function is processsing streaming data that must be analyzed with SQL	Load data into a Kinesis Data Stream and then analyze with Kinesis Data Analytics

Architecture Patterns – Analytics

Requirement	Solution
Security logs generated by AWS WAF must be sent to a third-party auditing application	Send logs to Kinesis Data Firehose and configure the auditing application using a HTTP endpoint
Real-time streaming data must be stored for future analysis	Ingest data into a Kinesis Data Stream and then use Firehose to load to a data store for later analysis
Company runs several production databases and must run complex queries across consolidated data set for business forecasting	Load the data from the OLTP databases into a RedShift data warehouse for OLAP

SECTION 16

Monitoring, Logging, and Auditing

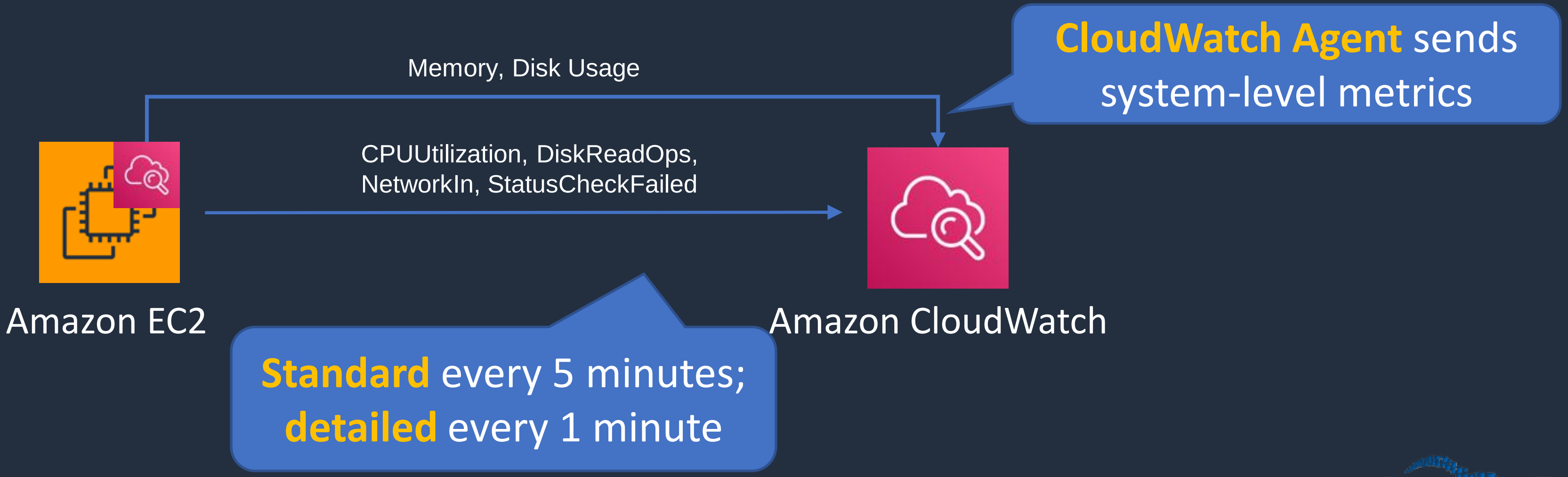
Amazon CloudWatch Features and Use Cases

Amazon CloudWatch

- **CloudWatch Metrics** – services send time-ordered data points to CloudWatch
- **CloudWatch Alarms** – monitor metrics and initiate actions
- **CloudWatch Logs** – centralized collection of system and application logs
- **CloudWatch Events** – stream of system events describing changes to AWS resources and can trigger actions

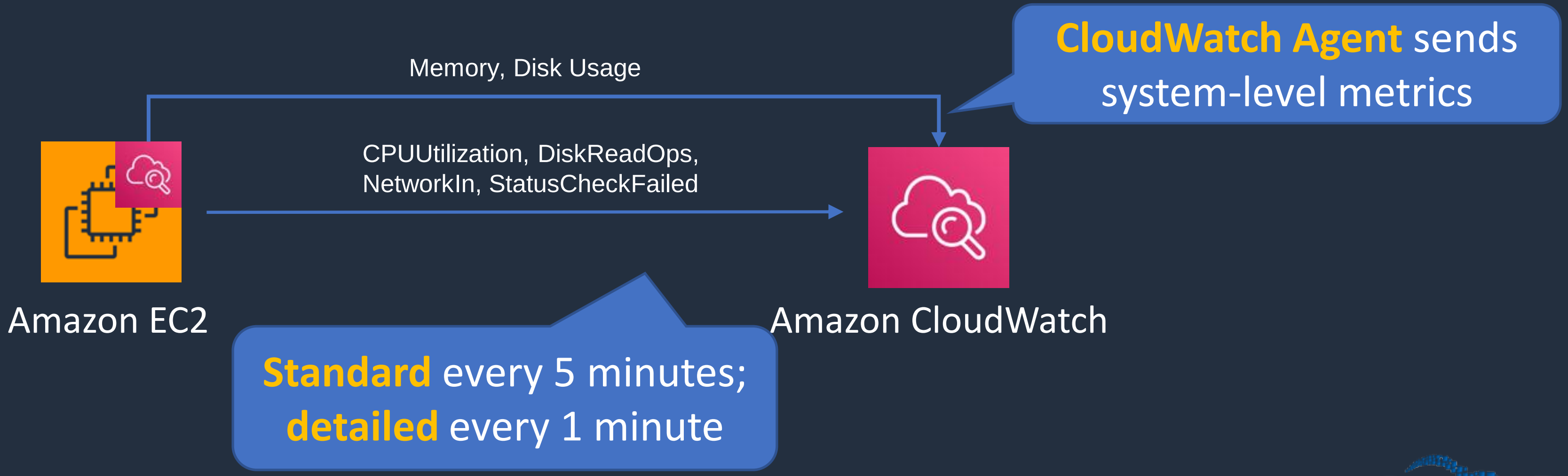
Amazon CloudWatch Metrics

- Metrics are sent to CloudWatch for many AWS services
- EC2 metrics are sent every **5 minutes** by default (free)
- Detailed EC2 monitoring sends every **1 minute** (chargeable)
- Unified CloudWatch Agent sends system-level metrics for EC2 and on-premises servers
- System-level metrics include memory and disk usage



Amazon CloudWatch Metrics

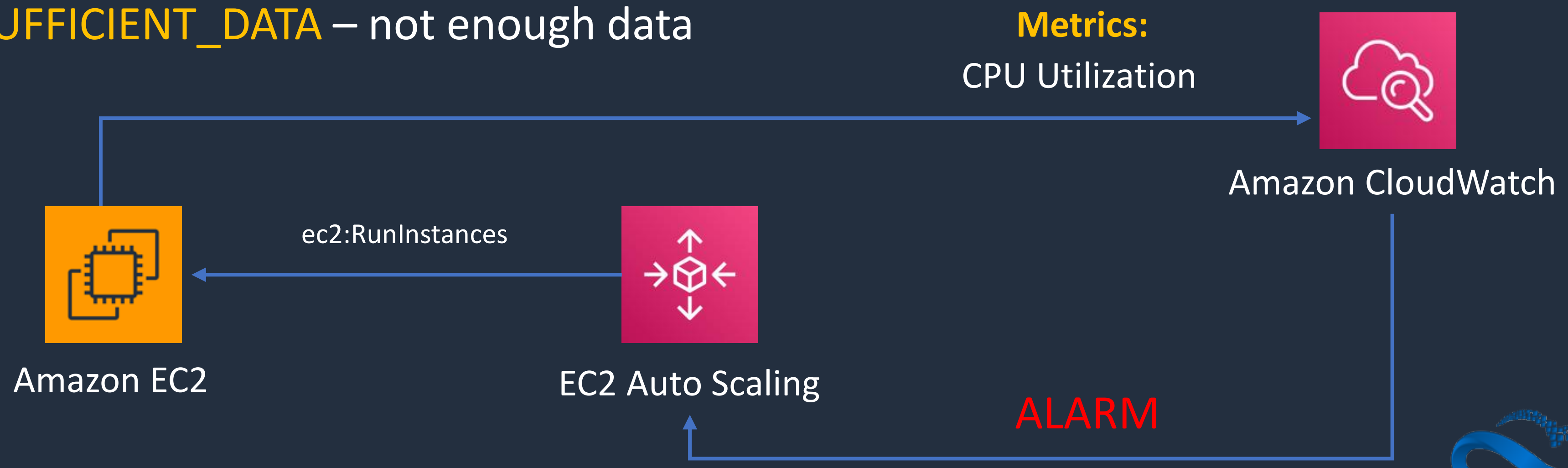
- Can publish custom metrics using CLI or API
- Custom metrics are one of the following resolutions:
  - **Standard resolution** – data having a one-minute granularity
  - **High resolution** – data at a granularity of one second
- AWS metrics are standard resolution by default



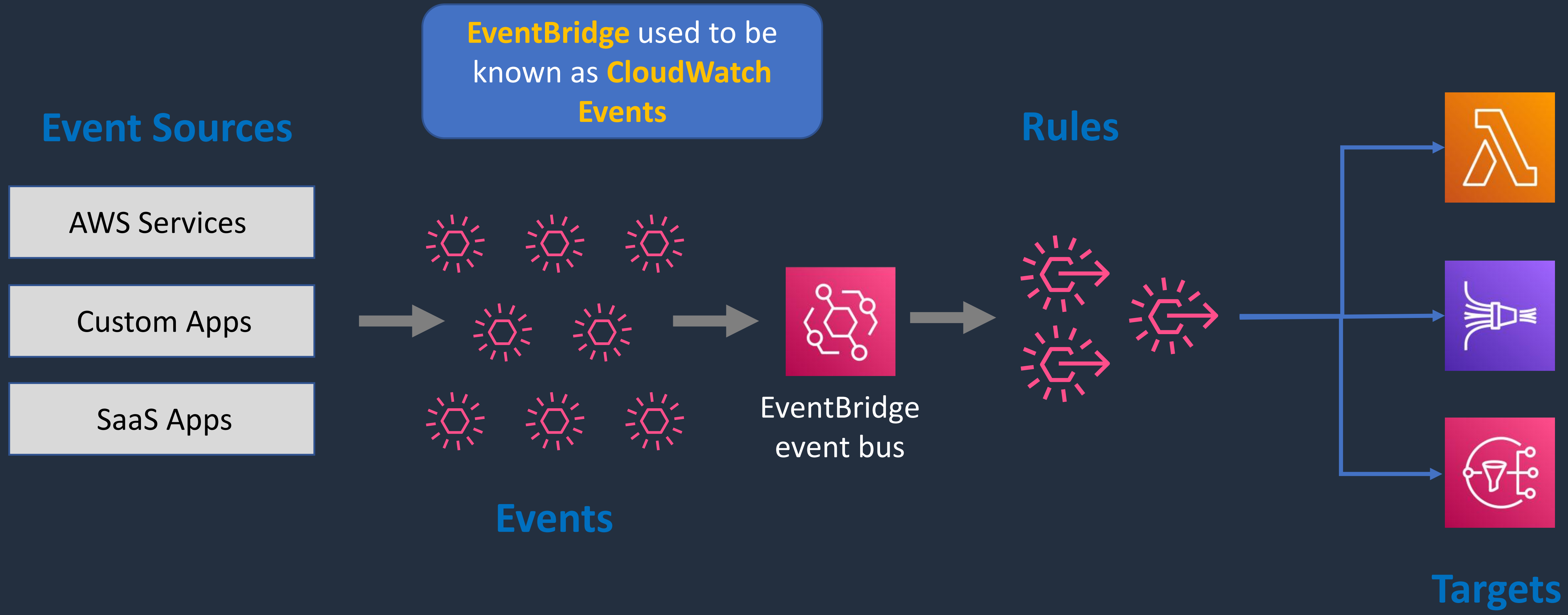
Amazon CloudWatch Alarms

Two types of alarms

- **Metric alarm** – performs one or more actions based on a single metric
- **Composite alarm** – uses a rule expression and takes into account multiple alarms
- Metric alarm states:
  - OK – Metric is within a threshold
  - ALARM – Metric is outside a threshold
  - INSUFFICIENT_DATA – not enough data

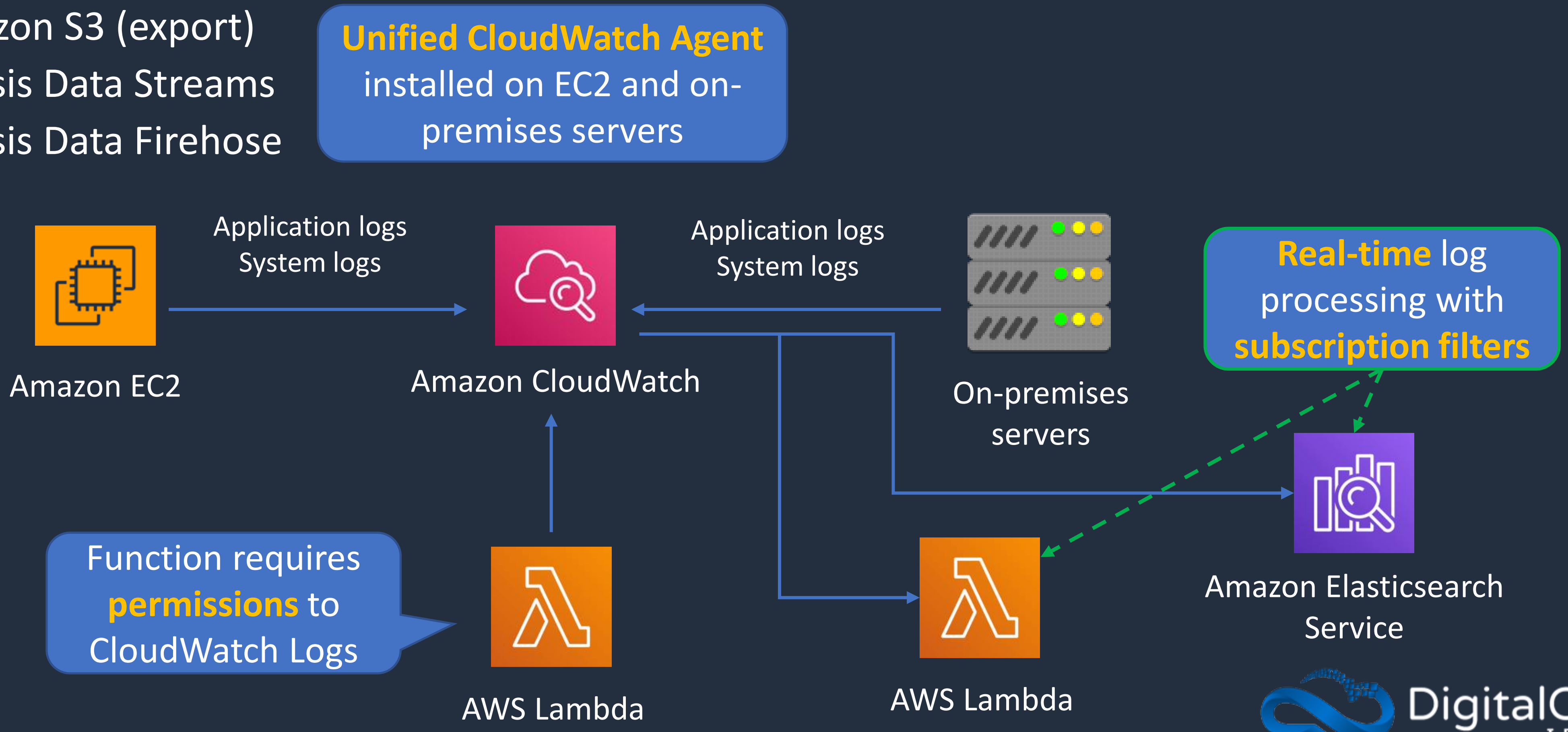


Amazon CloudWatch Events / EventBridge



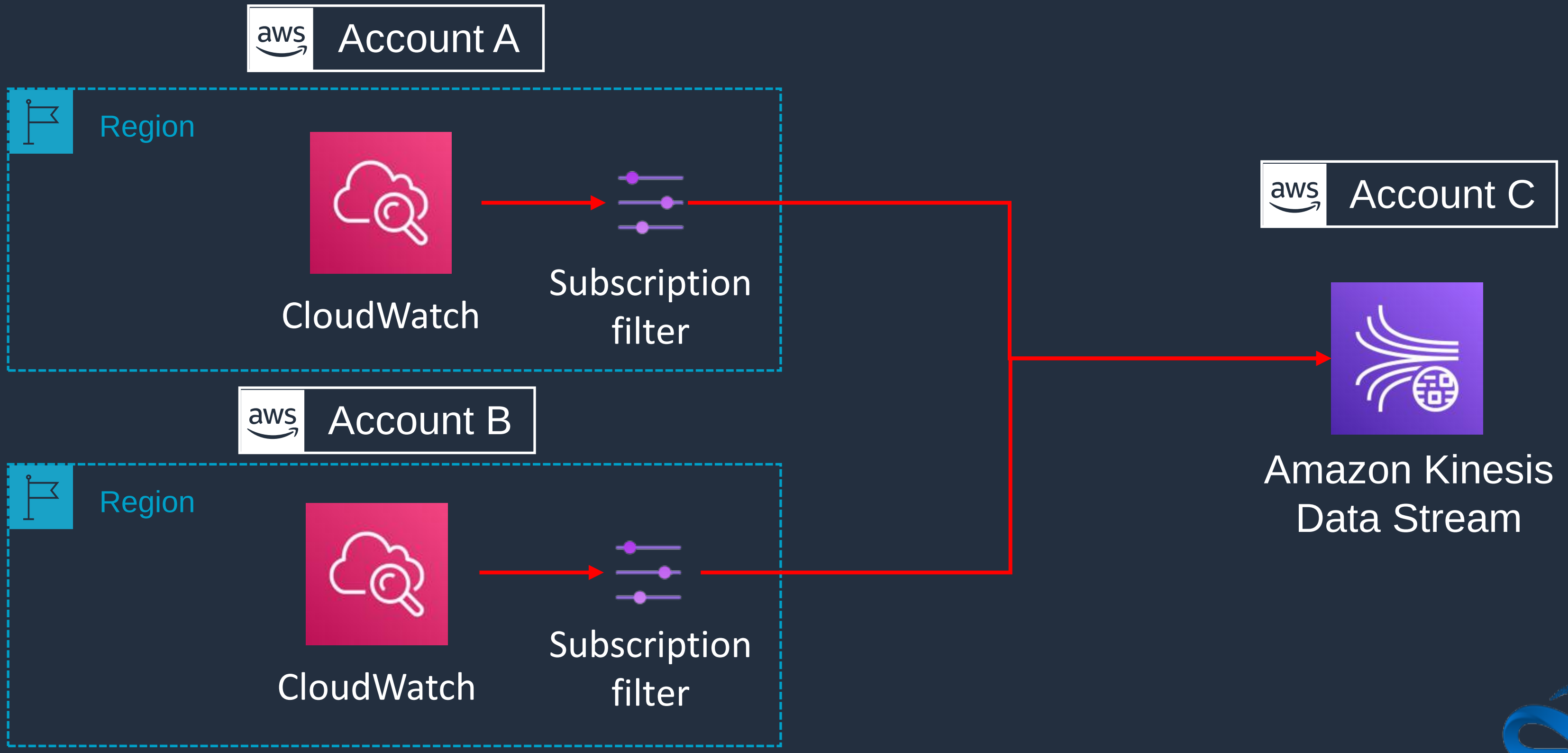
Amazon CloudWatch Logs

- Gather application and system logs in CloudWatch
- Defined expiration policies and KMS encryption
- Send to:
  - Amazon S3 (export)
  - Kinesis Data Streams
  - Kinesis Data Firehose



Cross-Account Log Data Sharing

- Share CloudWatch Logs across accounts
- Kinesis Data Streams is the only supported destination
- **Log data sender** – sends log data to the recipient
- **Log data recipient** – sends data to a Kinesis Data stream



Export CloudWatch Logs to Amazon S3

Export CloudWatch Logs to S3

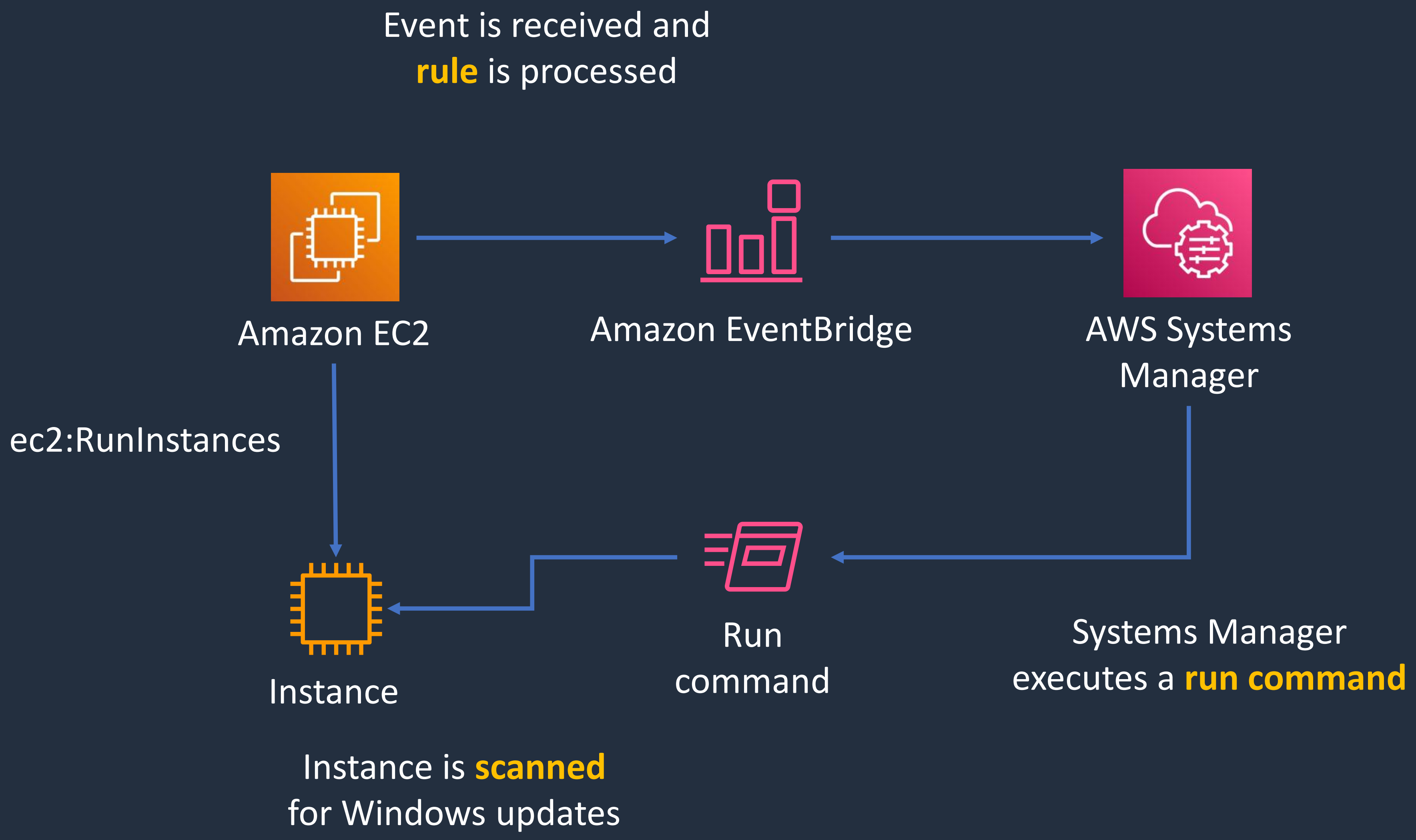
Bucket **cannot** be encrypted with **AWS KMS**

Logs are **exported** to the S3 bucket

```json
{
    "Version": "2012-10-17",
    "Statement": [
      {
          "Action": "s3:GetBucketAcl",
          "Effect": "Allow",
          "Resource": "arn:aws:s3:::mybucket",
          "Principal": { "Service": "logs.us-east-1.amazonaws.com" }
      },
      {
          "Action": "s3:PutObject" ,
          "Effect": "Allow",
          "Resource": "arn:aws:s3:::mybucket/*",
          "Condition": { "StringEquals": { "s3:x-amz-acl":
          "bucket-owner-full-control" } },
          "Principal": { "Service": "logs.us-east-1.amazonaws.com" }
      }
    ]
}
```

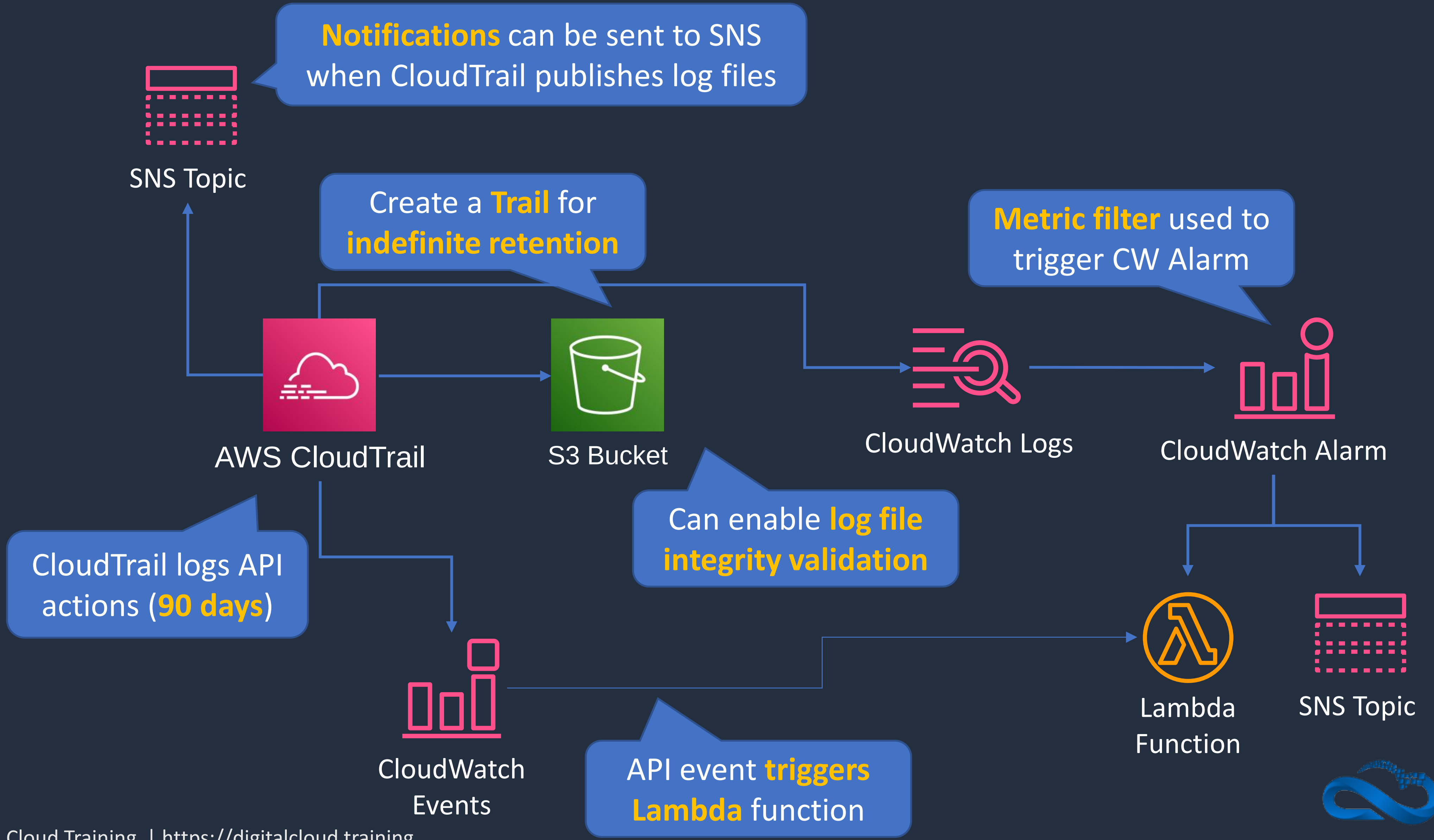
Trigger SSM on Instance Launch

Trigger SSM on Instance Launch



AWS CloudTrail Use Cases

AWS CloudTrail



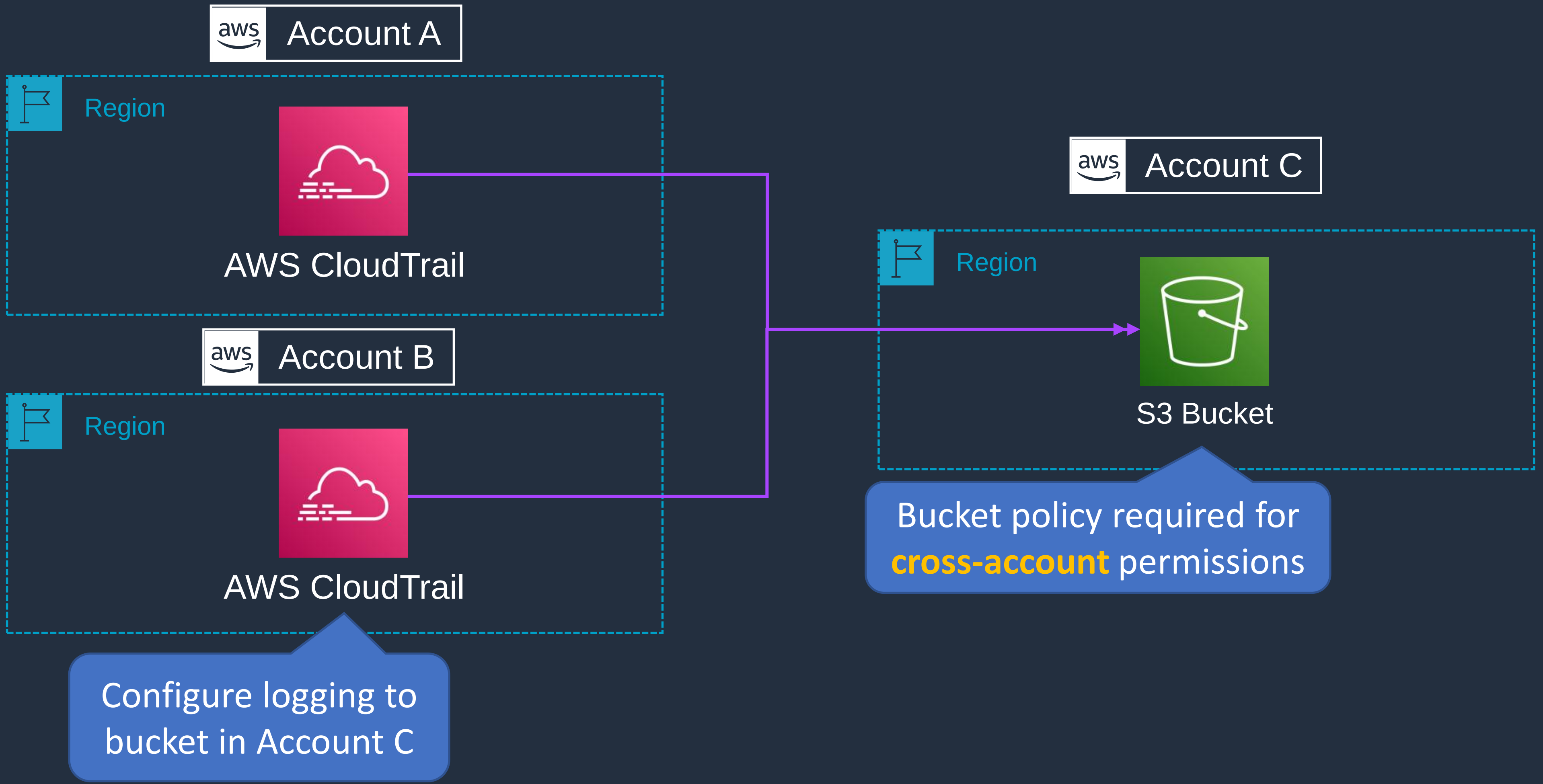
AWS CloudTrail

- CloudTrail logs **API activity** for auditing
- By default, management events are logged and retained for 90 days
- A **CloudTrail Trail** logs any events to S3 for indefinite retention
- Trail can be within Region or all Regions
- CloudWatch Events can triggered based on API calls in CloudTrail
- Events can be streamed to CloudWatch Logs

CloudTrail – Management and Data Events

- **Management events** provide information about management operations that are performed on resources in your AWS account
- **Data events** provide information about the resource operations performed on or in a resource

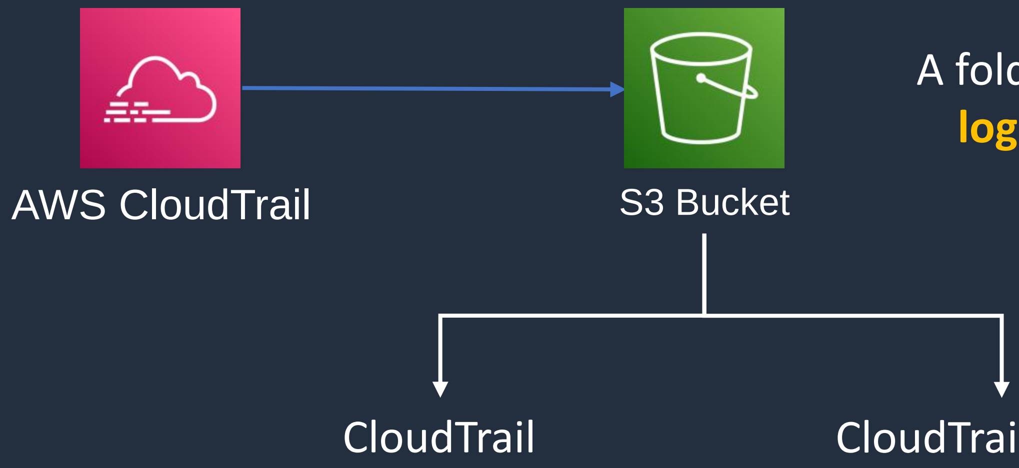
CloudTrail – Multi Account and Region



Enable CloudTrail Log File Validation

Enable CloudTrail Log File Validation

Use granular **security policies** and **MFA delete**



A folder structure separates **log files** and **digest files**

Digest files record **all modifications** to log files

Metric Analysis and Tracing (Grafana, Prometheus, X-Ray)

Tracing with AWS X-Ray

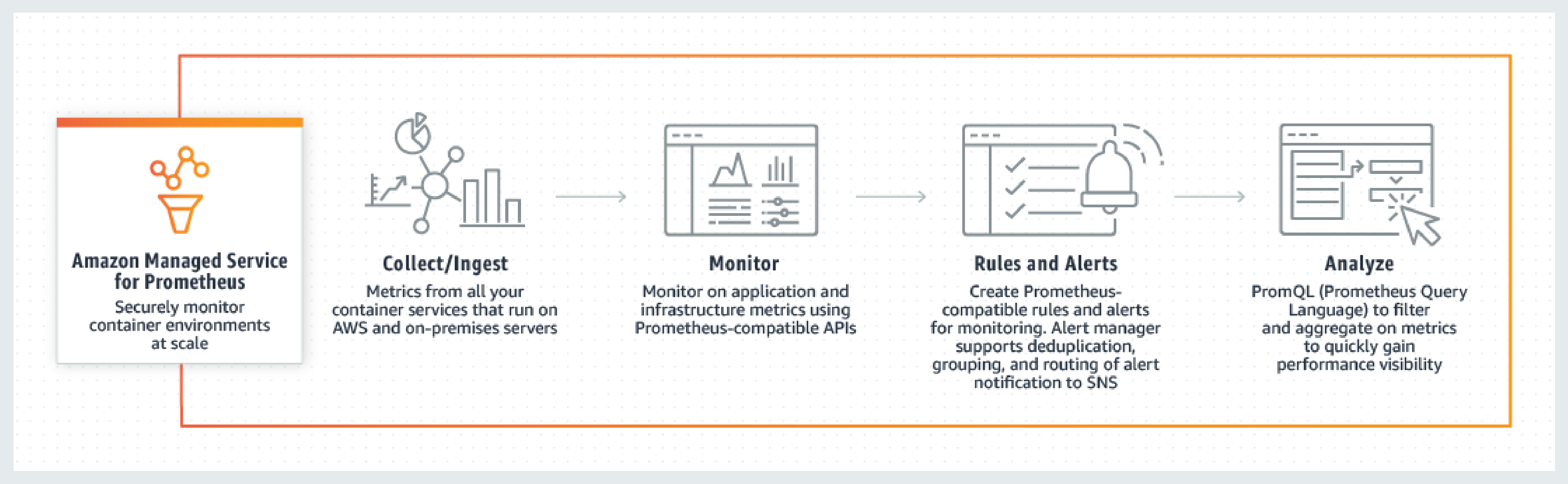
- You can use AWS X-Ray to visualize the components of your application, identify performance bottlenecks, and troubleshoot requests that resulted in an error
- AWS services send trace data to X-Ray, and X-Ray processes the data to generate a service map and searchable trace summaries

Tracing with AWS X-Ray

- X-Ray can be used with applications running on EC2, ECS, Lambda, and Elastic Beanstalk
- You must integrate the X-Ray SDK with your application and install the X-Ray agent
- The AWS X-Ray agent is a software application that gathers raw segment data and relays it to the AWS X-Ray service
- The agent works in conjunction with the AWS X-Ray SDKs so that data sent by the SDKs can reach the X-Ray service
- The X-Ray SDK captures metadata for requests made to MySQL and PostgreSQL databases and Amazon DynamoDB
- It also captures metadata for requests made to Amazon SQS and Amazon SNS

Amazon Managed Service for Prometheus

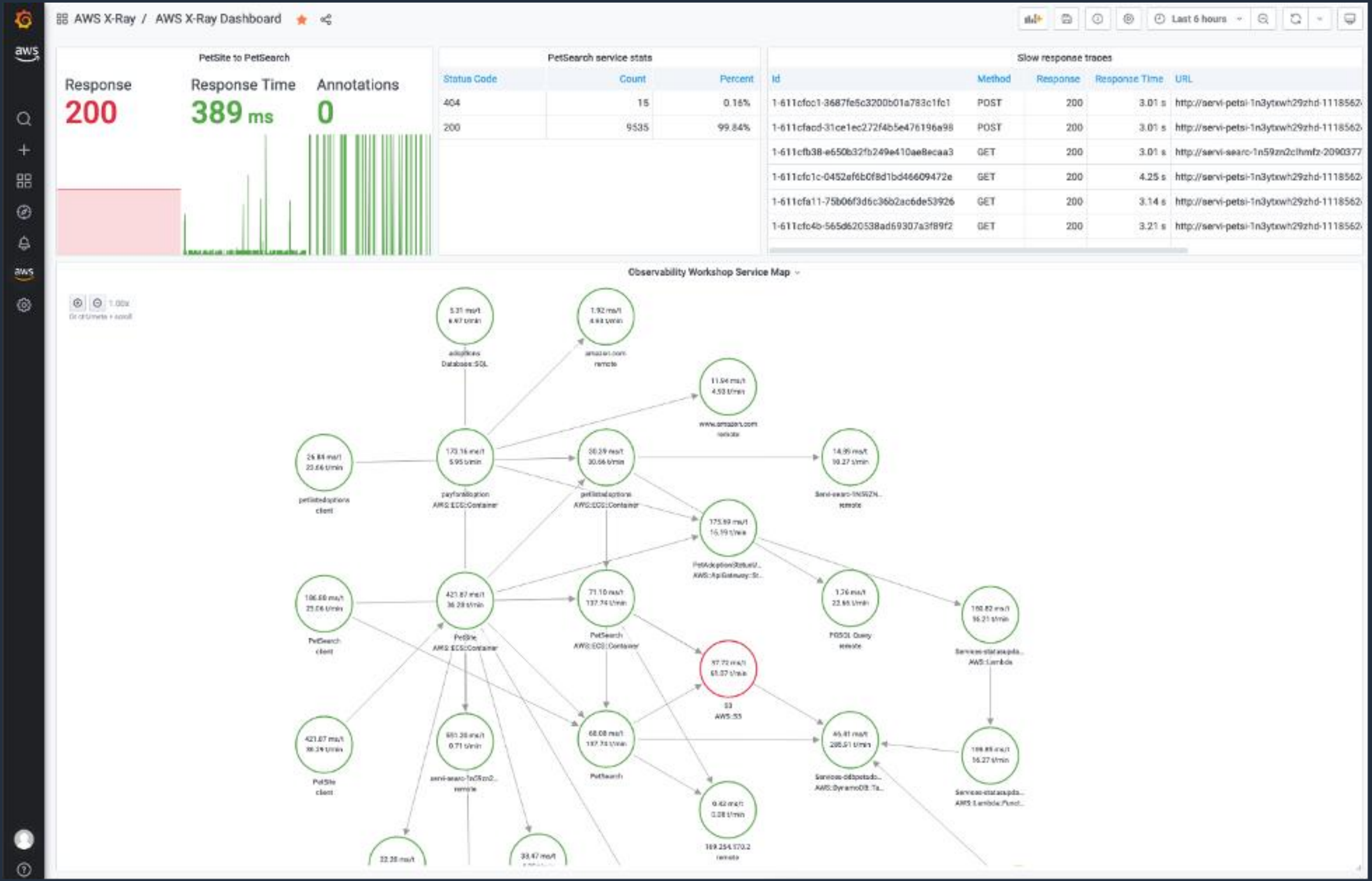
- Prometheus is an open-source monitoring system and time series database
- Use the open-source Prometheus query language (PromQL) to monitor and alert on the performance of containerized workloads
- Automatically scales the ingestion, storage, alerting, and querying of operational metrics as workloads grow or shrink
- Integrated with Amazon EKS, Amazon ECS, and AWS Distro for OpenTelemetry



Amazon Managed Grafana

- Grafana is an open-source analytics and monitoring solution for databases
- Highly scalable, highly available, and fully managed service Grafana
- Provides interactive data visualization for your monitoring and operational data
- Visualize, analyze, and alarm on your metrics, logs, and traces collected from multiple data sources
- Integrates with AWS SSO and SAML

Amazon Managed Grafana



Example of an Amazon Managed Grafana dashboard visualizing data from AWS X-Ray as a data source

Amazon Managed Grafana

Example visualizing an imported MySQL dashboard

Architecture Patterns - Monitoring, Logging and Auditing

Architecture Patterns – Monitoring, Logging and Auditing

Requirement	Solution
Need to stream logs from Amazon EC2 instances in an Auto Scaling Group	Install the unified CloudWatch Agent and collect log files in Amazon CloudWatch
Need to collect metrics from EC2 instances with a 1 second granularity	Create a custom metric with high resolution
The application logs from on-premises servers must be processed by AWS Lambda in real time	Install the unified CloudWatch Agent on the servers and use a subscription filter in CloudWatch to connect to a Lambda function

Architecture Patterns – Monitoring, Logging and Auditing

Requirement	Solution
CloudWatch Logs entries must be transformed with Lambda and then loaded into Amazon S3	Configure a Kinesis Firehose destination, transform with Lambda and then load into an S3 bucket
CloudWatch Logs entries must be analyzed and stored centrally in a security account	Use cross-account sharing and configure a Kinesis Data Stream in the security account to collect the log files then use Lambda to analyze and store
Access auditing must be enabled and records must be stored for a minimum of 5 years. Any attempts to modify the log files must be identified	Create a trail in CloudTrail that stores the data in an S3 bucket and enable log file integrity validation

Architecture Patterns – Monitoring, Logging and Auditing

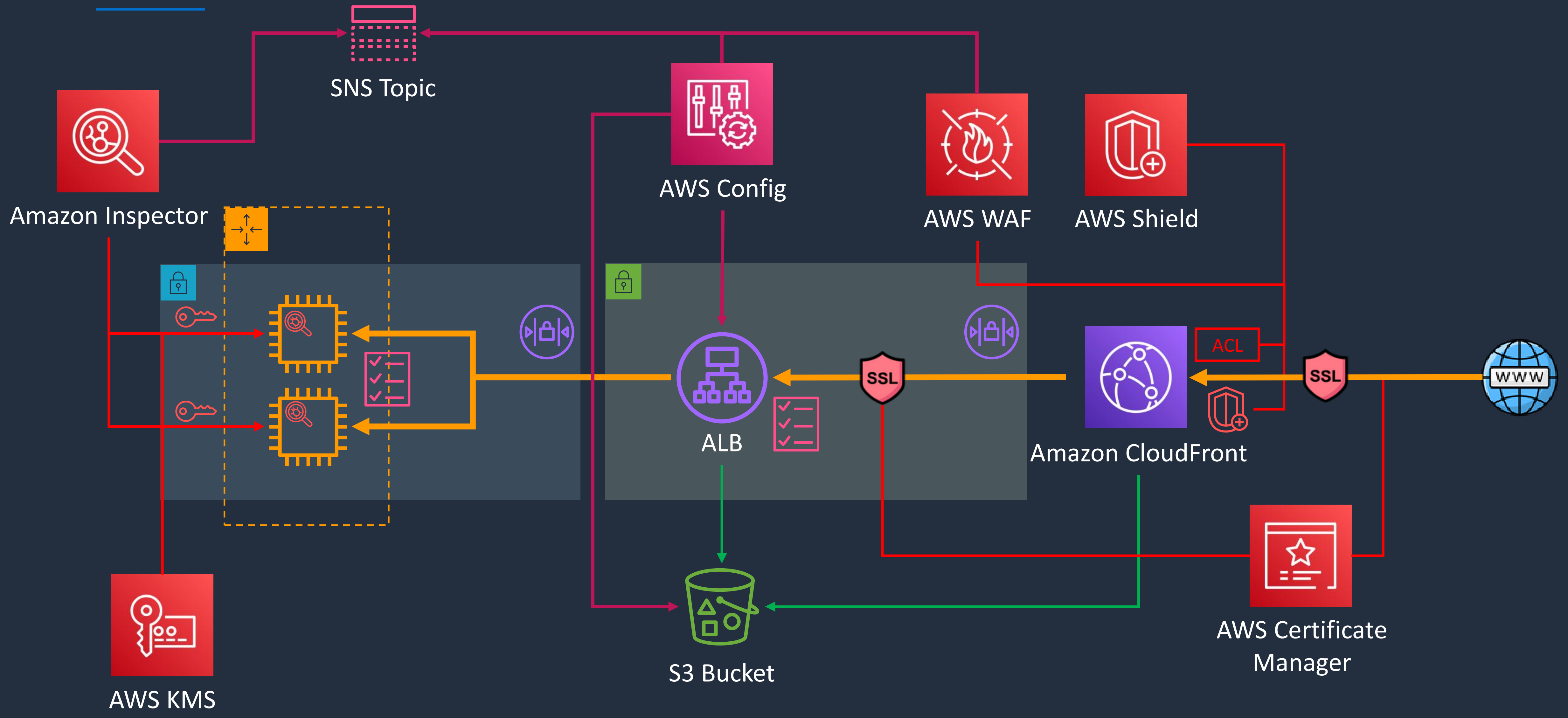
Requirement	Solution
API activity must be captured from multiple accounts and stored in a central security account	Use CloudTrail in each account to record API activity and use cross-account access to a security account to store the log files in a central S3 bucket
Need to trace and debug application with distributed components	Use AWS X-Ray to trace and debug the application

SECTION 17

Security Services

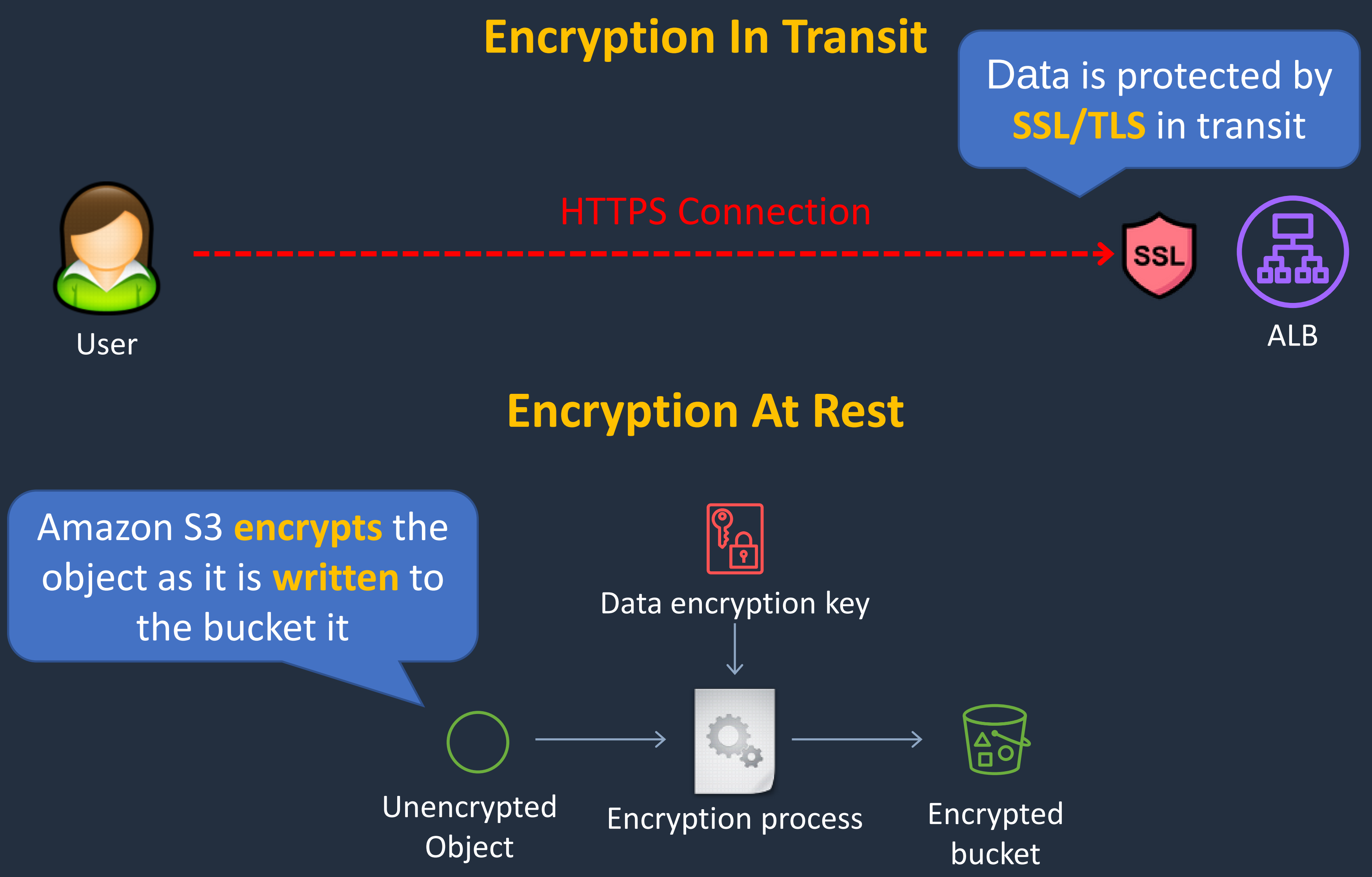
Secure Multi-Tier Architecture for HOL

Build a Secure Multi-Tier Architecture



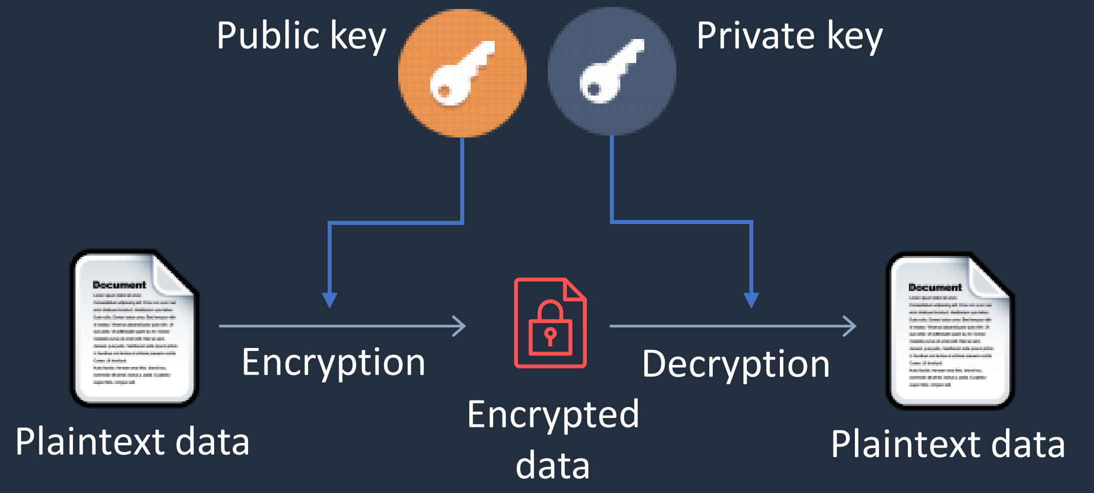
AWS Certificate Manager (ACM)

Encryption In Transit vs At Rest



Asymmetric Encryption

- Asymmetric encryption is also known as public key cryptography
- Messages encrypted with the public key can only be decrypted with the private key
- Messages encrypted with the private key can be decrypted with the public key
- Examples include SSL/TLS and SSH



AWS Certificate Manager (ACM)

- Create, store and renew SSL/TLS X.509 certificates
- Single domains, multiple domain names and wildcards
- Integrates with several AWS services including:
  - **Elastic Load Balancing**
  - **Amazon CloudFront**
  - **AWS Elastic Beanstalk**
  - **AWS Nitro Enclaves**
  - **AWS CloudFormation**

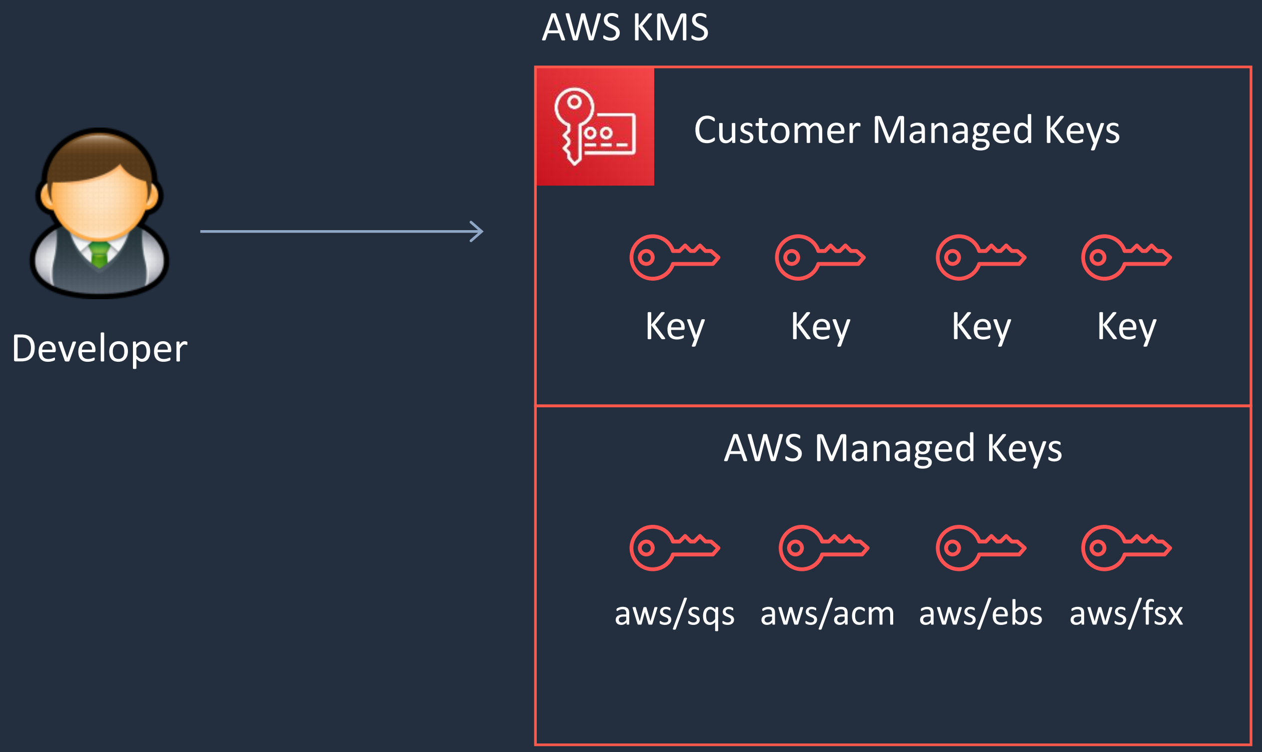
AWS Certificate Manager (ACM)

- **Public certificates** are signed by the AWS public Certificate Authority
- You can also create a Private CA with ACM
- Can then issue private certificates
- You can also import certificates from third-party issuers

AWS Key Management Service (KMS)

AWS Key Management Service (KMS)

- Create and managed **symmetric** and **asymmetric** encryption keys
- The **KMS keys** are protected by hardware security modules (HSMs)



Developer creates **customer managed** KMS keys in AWS KMS

KMS Keys

- KMS keys are the primary resources in AWS KMS
- Used to be known as “customer master keys” or CMKs
- The KMS key also contains the key material used to encrypt and decrypt data
- By default, AWS KMS creates the key material for a KMS key
- You can also import your own key material
- A KMS key can encrypt data up to 4KB in size
- A KMS key can generate, encrypt and decrypt Data Encryption Keys (DEKs)

Alternative Key Stores

External Key Store

- Keys can be stored outside of AWS to meet regulatory requirements
- You can create a KMS key in an AWS KMS external key store (XKS)
- All keys are generated and stored in an external key manager
- When using an XKS, key material never leaves your HSM

Custom Key Store

- You can create KMS keys in an AWS CloudHSM custom key store
- All keys are generated and stored in an AWS CloudHSM cluster that you own and manage
- Cryptographic operations are performed solely in the AWS CloudHSM cluster you own and manage
- Custom key stores are not available for asymmetric KMS keys

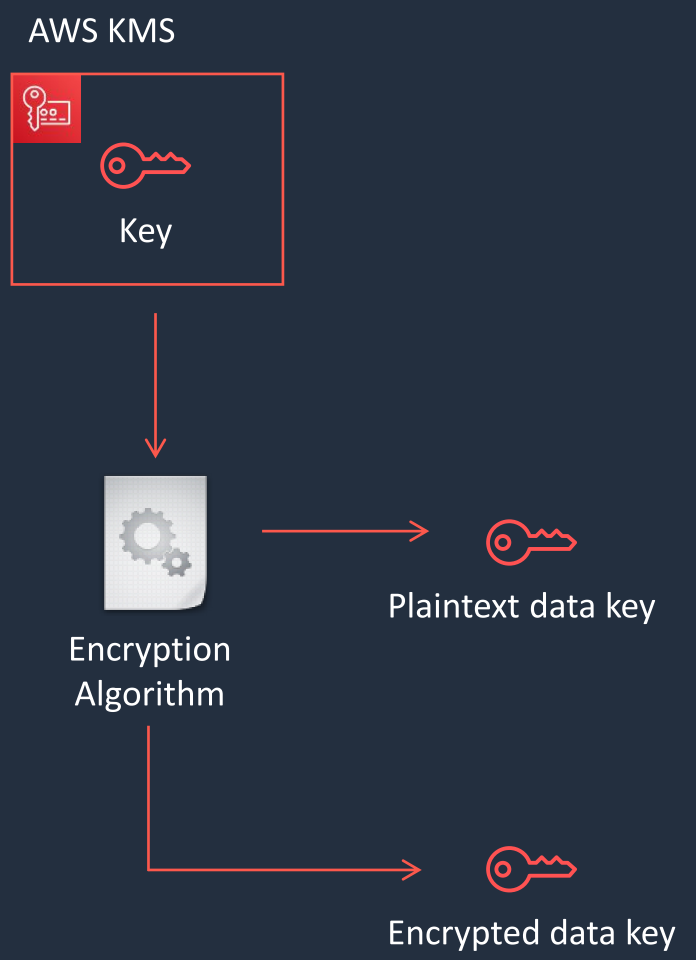
AWS Managed KMS Keys

- Created, managed, and used on your behalf by an AWS service that is integrated with AWS KMS
- You cannot manage these KMS keys, rotate them, or change their key policies
- You also cannot use AWS managed KMS keys in cryptographic operations directly; the service that creates them uses them on your behalf

Alias	Key ID
aws/sqs	025b9386-b1f8-4fa9-84e2-ac3220b1de59
aws/acm	2d604e85-c2d4-42dc-ab0b-0b356f5fe26e
aws/codecommit	41fea9df-e447-4992-8af7-6ddec6d81175
aws/elasticfilesystem	460c4f05-fe98-4a35-b940-3e1992f04314
aws/glue	617516fe-bf19-4da4-a743-2b13c41973e1
aws/lambda	7f513d01-784b-41b9-9c51-51621db7b5e1
aws/ebs	b9baa4f6-3e87-4256-af6a-d181940df286
aws/lightsail	bc7ba666-8e17-444a-800c-d3d4be303a97
aws/fsx	cebc434c-ee2b-4a61-9b5a-f63be9fdb068
aws/kinesis	d99014b5-09d4-480d-9b2a-3a7d7e3e9c5b

Data Encryption Keys

- Data keys are encryption keys that you can use to encrypt large amounts of data
- You can use AWS KMS keys to generate, encrypt, and decrypt data keys
- AWS KMS does not store, manage, or track your data keys, or perform cryptographic operations with data keys
- You must use and manage data keys outside of AWS KMS



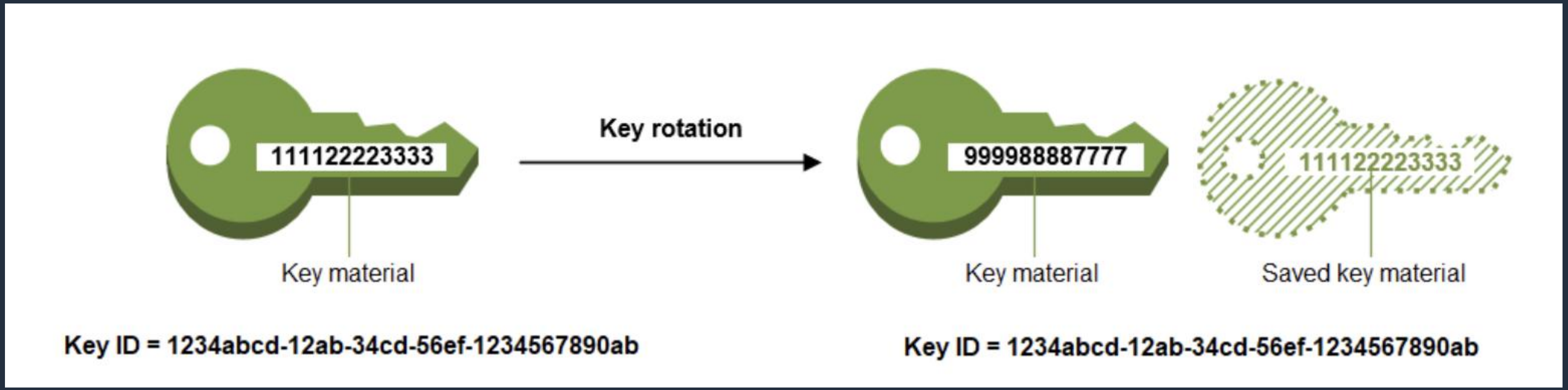
KMS Keys and Automatic Rotation

Type of KMS Key	Can view	Can manage	Used only for my AWS account	Automatic rotation
Customer managed key	Yes	Yes	Yes	Optional. Every 365 days
AWS managed key	Yes	No	Yes	Required. Every 365 days
AWS owned key	No	No	No	Varies

- You cannot enable or disable key rotation for AWS owned keys
- Automatic key rotation is supported only on symmetric encryption KMS keys with key material that AWS KMS generates (**Origin = AWS_KMS**)

KMS Keys and Automatic Rotation

- Automatic rotation generates new key material every year (*optional* for **customer managed keys**)



Rotation only changes the **key material** used for encryption, the KMS key remains the same

KMS Keys and Automatic Rotation

With automatic key rotation:

- The properties of the KMS key, including its key ID, key ARN, region, policies, and permissions, do not change when the key is rotated
- You do not need to change applications or aliases that refer to the key ID or key ARN of the KMS key
- After you enable key rotation, AWS KMS rotates the KMS key automatically every year

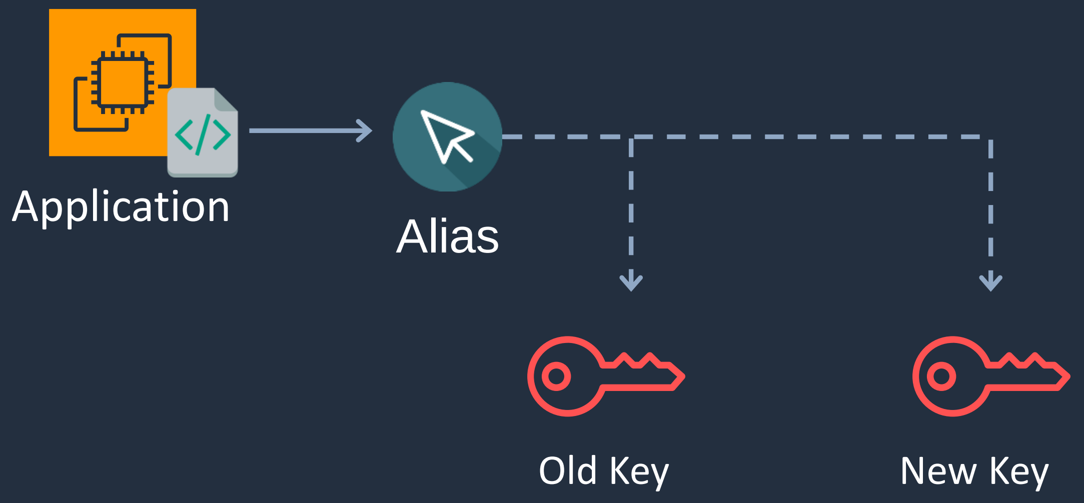
Automatic key rotation is not supported on the following types of KMS keys:

- Asymmetric KMS keys
- HMAC KMS keys
- KMS keys in custom key stores
- KMS keys with imported key material

Note: You can rotate these KMS keys **manually**

Manual Rotation

- Manual rotation is creating a new KMS key with a different key ID
- You must then update your applications with the new key ID
- You can use an **alias** to represent a KMS key so you don't need to modify your application code



The **alias** is associated with the new **KMS key**

KMS Key Policies

- Key policies define management and usage permissions for KMS keys

```
{
  "Sid": "Allow access for Key Administrators",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::111122223333:user/KMSKeyAdmin"},
  "Action": [
    "kms:Describe*",
    "kms:Put*",
    "kms:Create*",
    "kms:Update*",
    "kms:Enable*",
    "kms:Revoke*",
    "kms:List*",
    "kms:Disable*",
    "kms:Get*",
    "kms:Delete*",
    "kms:ScheduleKeyDeletion",
    "kms:CancelKeyDeletion"
  ],
  "Resource": "*"
}
```

This key policy defines the **administrative actions** that are permitted for a key administrator

KMS Key Policies

- Multiple policy statements can be combined to specify separate administrative and usage permissions

```
{
    "Sid": "Allow use of the key",
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::111122223333:role/EncryptionApp"},
    "Action": [
      "kms:DescribeKey",
      "kms:GenerateDataKey*",
      "kms:Encrypt",
      "kms:ReEncrypt*",
      "kms:Decrypt"
    ],
    "Resource": "*"
  }
```

This key policy defines the **cryptographic** actions for encrypting and decrypting data with the KMS key

KMS Key Policies

- Permissions can be specified for delegating use of the key to AWS services

```
{
  "Sid": "Allow attachment of persistent resources",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::111122223333:role/EncryptionApp"},
  "Action": [
    "kms:ListGrants",
    "kms:CreateGrant",
    "kms:RevokeGrant"
  ],
  "Resource": "*",
  "Condition": {"Bool": {"kms:GrantIsForAWSResource": true}}
}
```

Grants are useful for **temporary permissions** as they can be used without modifying key policies or IAM policies

Additional Exam Tips

- To share snapshots with another account you must specify Decrypt and CreateGrant permissions
- The kms:ViaService condition key can be used to limit key usage to specific AWS services
- For example:

```
"Condition": {
  "StringEquals": {
    "kms:ViaService": [
      "ec2.us-west-2.amazonaws.com",
      "rds.us-west-2.amazonaws.com"
```

Additional Exam Tips

- Cryptographic erasure means removing the ability to decrypt data and can be achieved when using **imported key material** and deleting that key material
- You must use the DeleteImportedKeyMaterial API to remove the key material
- An InvalidKeyId exception when using SSM Parameter Store indicates the KMS key is not enabled
- Make sure you know the differences between AWS managed and customer managed KMS keys and automatic vs manual rotation

AWS CloudHSM

AWS CloudHSM

- AWS CloudHSM is a cloud-based hardware security module (HSM)
- Generate and use your own encryption keys on the AWS Cloud
- Manage your own encryption keys using FIPS 140-2 Level 3 validated HSMs
- CloudHSM runs in your VPC

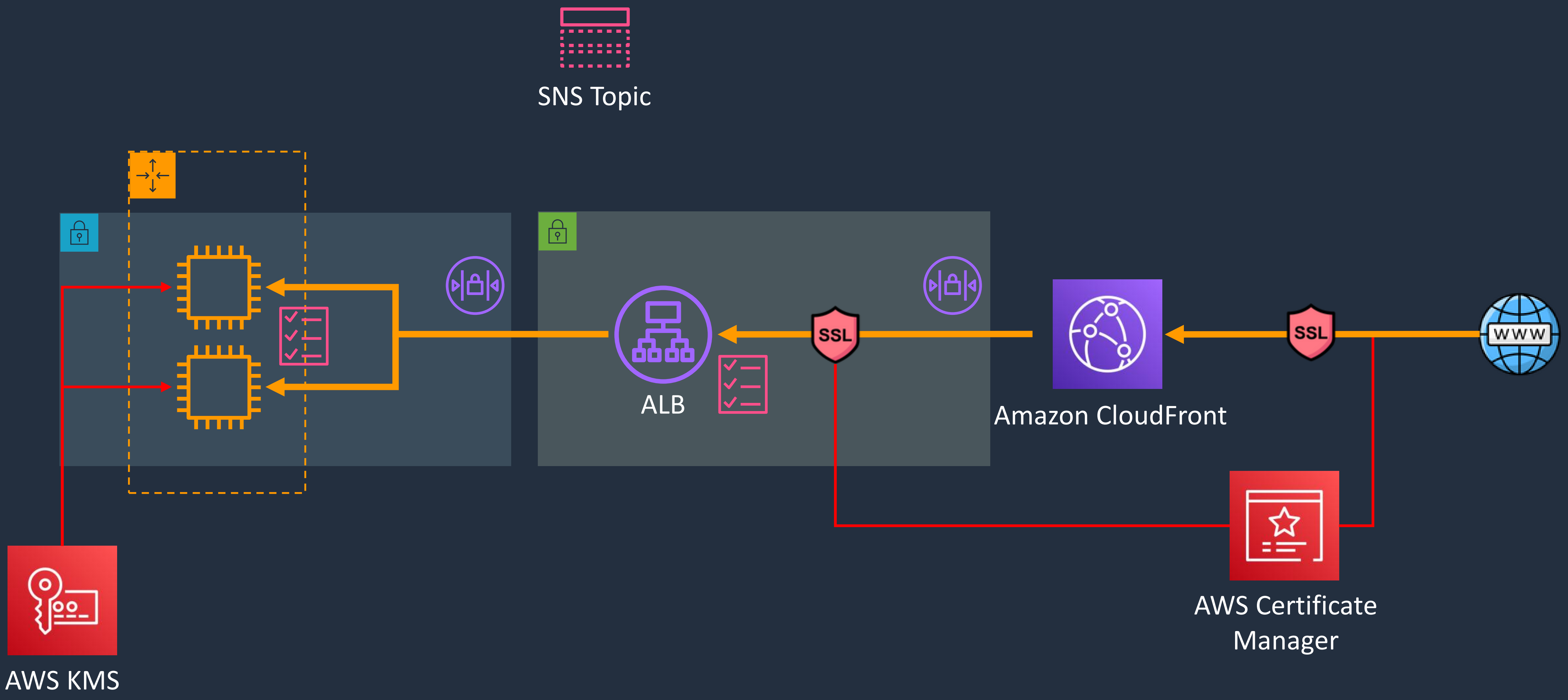
	CloudHSM	AWS KMS
Tenancy	Single-tenant HSM	Multi-tenant AWS service
Availability	Customer-managed durability and available	Highly available and durable key storage and management
Root of Trust	Customer managed root of trust	AWS managed root of trust
FIPS 140-2	Level 3	Level 2 / Level 3 in some areas
3rd Party Support	Broad 3rd Party Support	Broad AWS service support

AWS CloudHSM Benefits

- FIPS 140-2 level 3 validated HSMs
- You can configure AWS Key Management Service (KMS) to use your AWS CloudHSM cluster as a custom key store rather than the default KMS key store
- Managed service and automatically scales
- Retain control of your encryption keys - you control access (and AWS has no visibility of your encryption keys)

Build a Secure Multi-Tier Architecture - Part 1

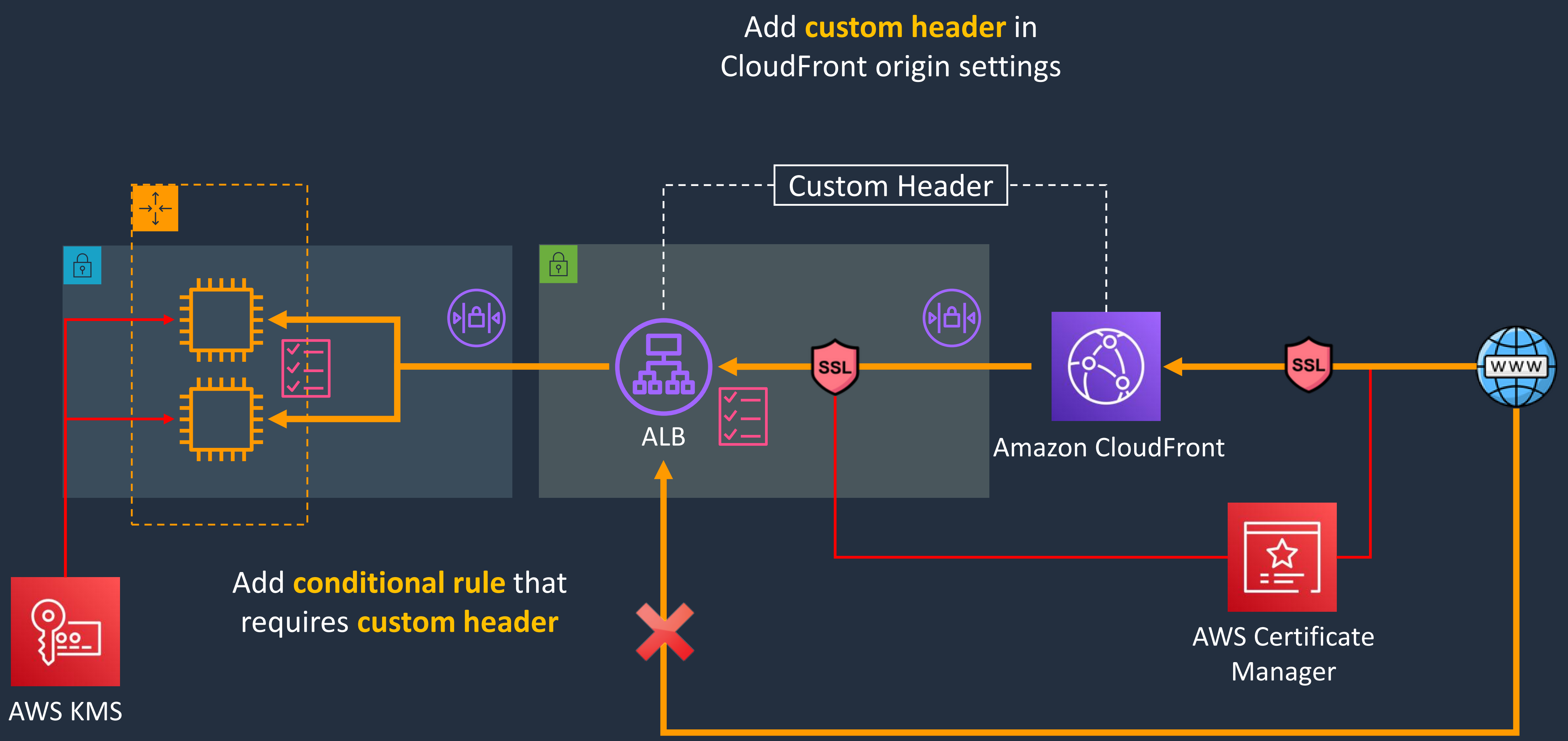
Build a Secure Multi-Tier Architecture



Build a Secure Multi-Tier Architecture - Part 2

Build a Secure Multi-Tier Architecture - Part 3

Build a Secure Multi-Tier Architecture

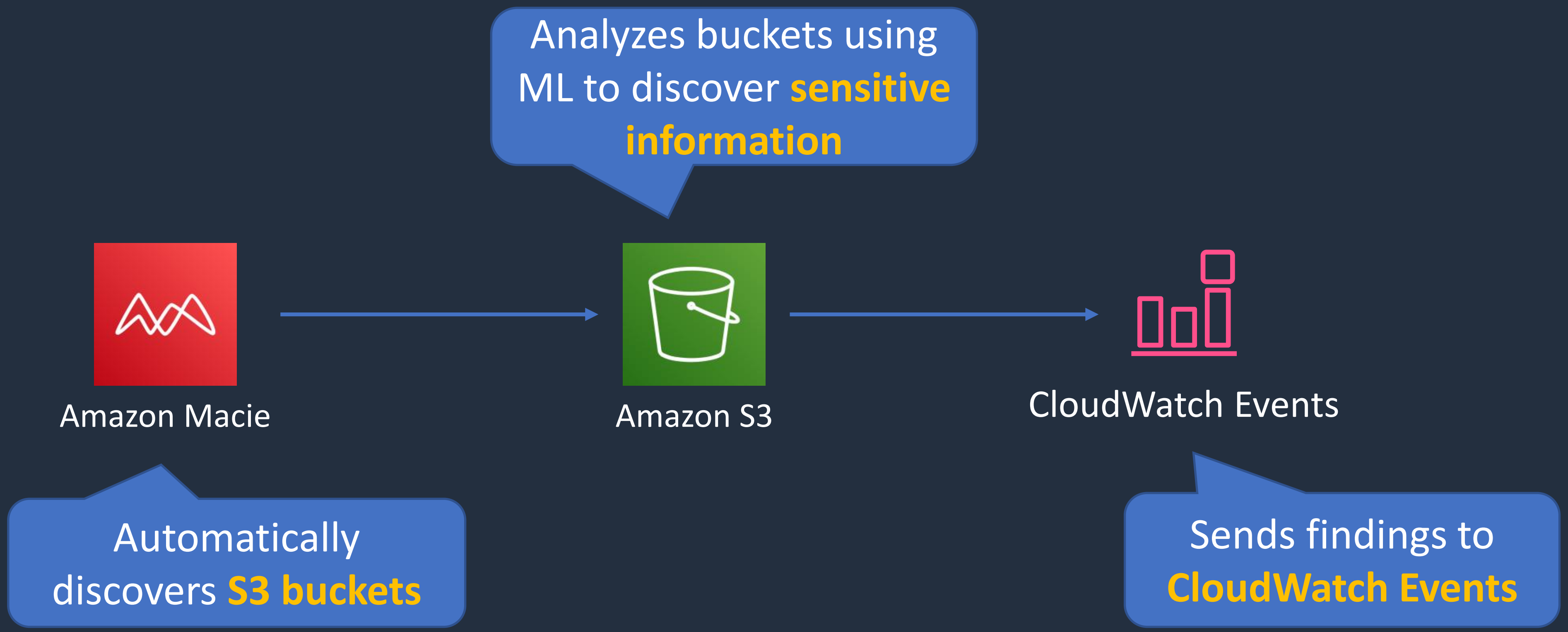


Amazon Macie

Amazon Macie

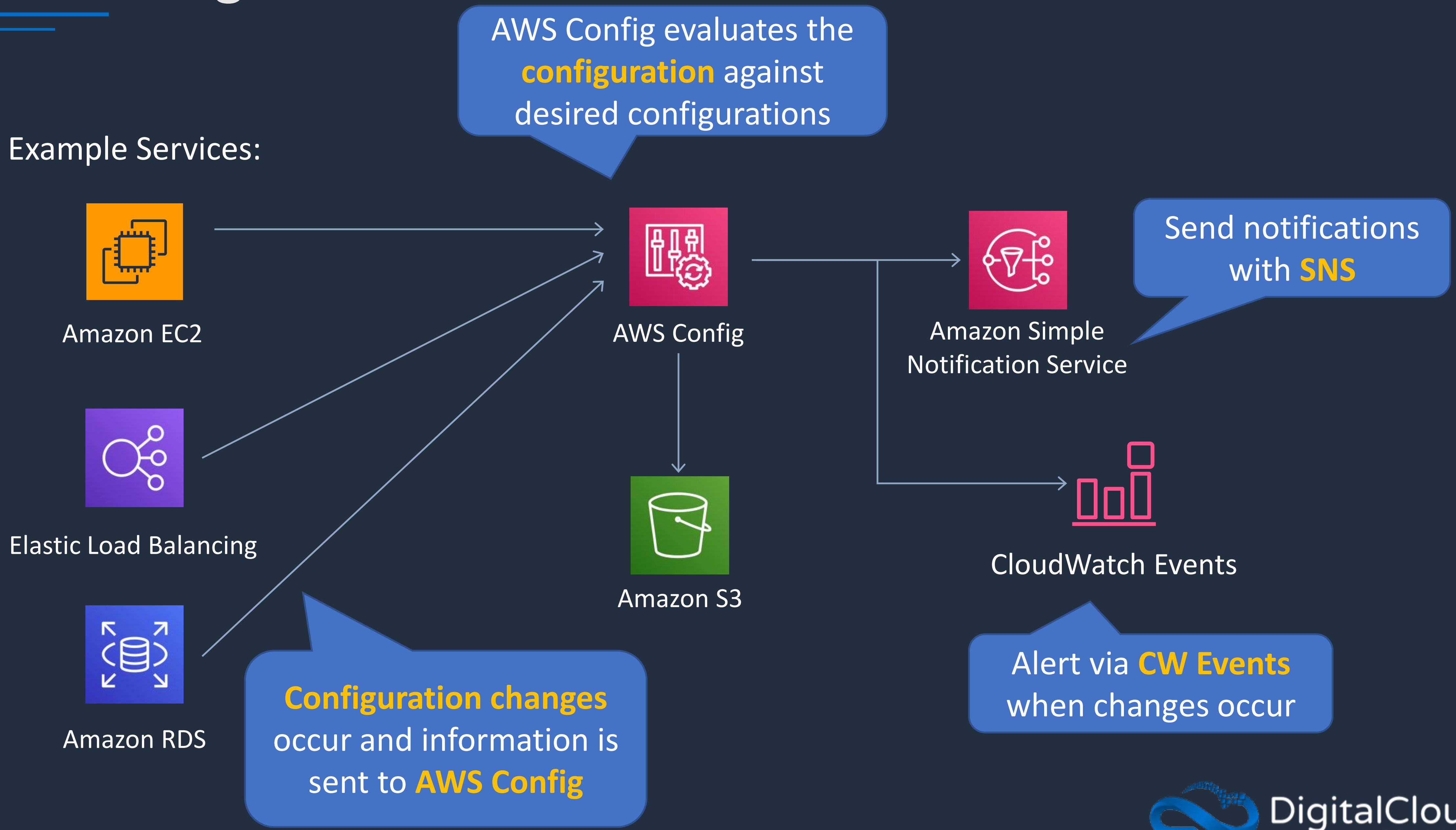
- Macie is a fully managed data security and data privacy service
- Uses machine learning and pattern matching to discover, monitor, and help you protect your sensitive data on Amazon S3
- Macie enables security compliance and preventive security as follows:
  - Identify a variety of data types, including PII, Protected Health Information (PHI), regulatory documents, API keys, and secret keys
  - Identify changes to policy and access control lists
  - Continuously monitor the security posture of Amazon S3
  - Generate security findings that you can view using the Macie console, AWS Security Hub, or Amazon EventBridge
  - Manage multiple AWS accounts using AWS Organizations

Amazon Macie



AWS Config

AWS Config



DigitalCloud TRAINING

AWS Config

Example Rule	Description
s3-bucket-server-side-encryption-enabled	Checks that your Amazon S3 bucket either has S3 default encryption enabled or that the S3 bucket policy explicitly denies put-object requests without server side encryption
restricted-ssh	Checks whether security groups that are in use disallow unrestricted incoming SSH traffic
rds-instance-public-access-check	Checks whether the Amazon Relational Database Service (RDS) instances are not publicly accessible
cloudtrail-enabled	Checks whether AWS CloudTrail is enabled in your AWS account

Amazon Inspector

Amazon Inspector

- Runs assessments that check for security exposures and vulnerabilities in EC2 instances
- Can be configured to run on a schedule
- Agent must be installed on EC2 for host assessments
- Network assessments do not require an agent

Amazon Inspector

Network Assessments

- Assessments: Network configuration analysis to check for ports reachable from outside the VPC
- If the Inspector Agent is installed on your EC2 instances, the assessment also finds processes reachable on port
- Price based on the number of instance assessments

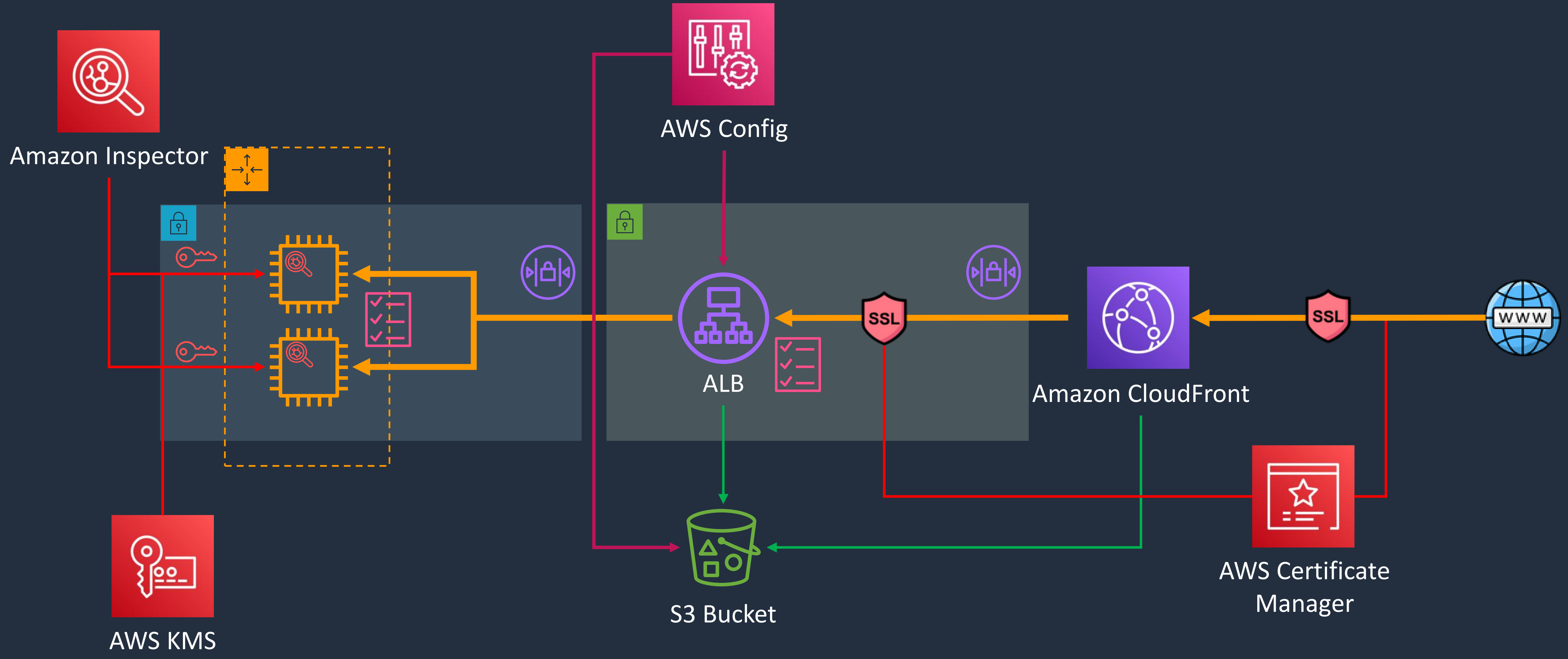
Amazon Inspector

Host Assessments

- Assessments: Vulnerable software (CVE), host hardening (CIS benchmarks), and security best practices
- Requires an agent (auto-install with SSM Run Command)
- Price based on the number of instance assessments

Build a Secure Multi-Tier Architecture - Part 4

Build a Secure Multi-Tier Architecture

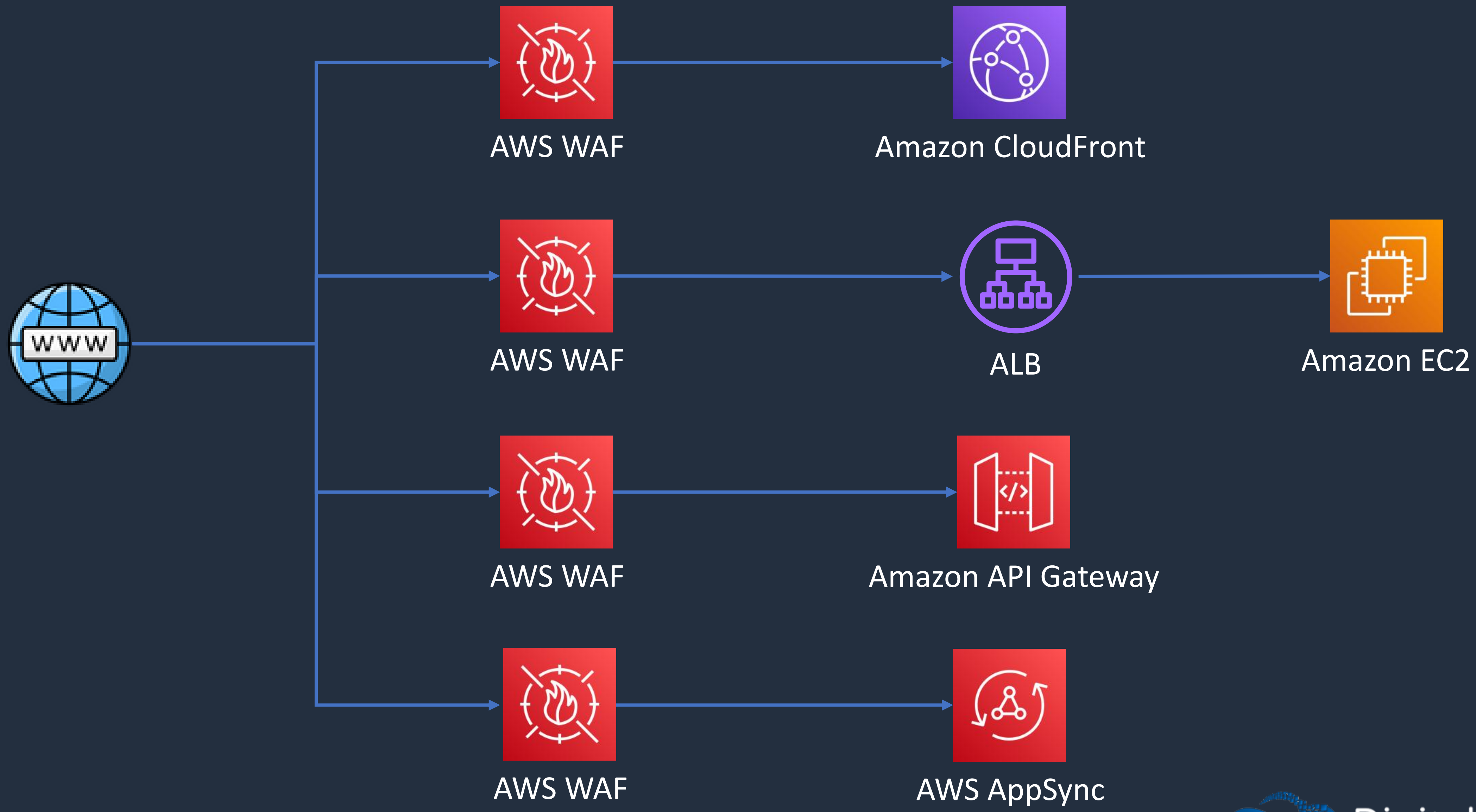


AWS Web Application Firewall (WAF)

AWS WAF

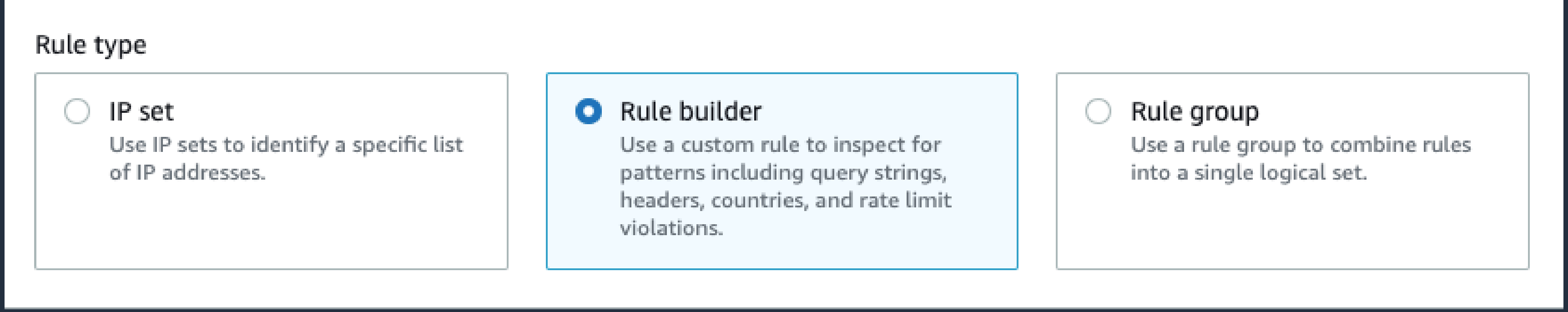
- AWS WAF is a web application firewall
- WAF lets you create rules to filter web traffic based on conditions that include IP addresses, HTTP headers and body, or custom URIs
- WAF makes it easy to create rules that block common web exploits like SQL injection and cross site scripting

AWS WAF


AWS WAF
Amazon CloudFront
AWS WAF
ALB
Amazon EC2
AWS WAF
Amazon API Gateway
AWS WAF
AWS AppSync

AWS WAF

- **Web ACLs** – You use a web access control list (ACL) to protect a set of AWS resources
- **Rules** – Each rule contains a statement that defines the inspection criteria, and an action to take if a web request meets the criteria
- **Rule groups** – You can use rules individually or in reusable rule groups



AWS WAF

- **IP Sets** - An IP set provides a collection of IP addresses and IP address ranges that you want to use together in a rule statement
- **Regex pattern set** - A regex pattern set provides a collection of regular expressions that you want to use together in a rule statement

AWS WAF

A **rule action** tells AWS WAF what to do with a web request when it ***matches*** the criteria defined in the rule:

- **Count** – AWS WAF counts the request but doesn't determine whether to allow it or block it. With this action, AWS WAF continues processing the remaining rules in the web ACL
- **Allow** – AWS WAF allows the request to be forwarded to the AWS resource for processing and response
- **Block** – AWS WAF blocks the request and the AWS resource responds with an HTTP 403 (Forbidden) status code

AWS WAF

Match statements compare the web request or its origin against conditions that you provide

Match Statement	Description
Geographic match	Inspects the request's country of origin
IP set match	Inspects the request against a set of IP addresses and address ranges
Regex pattern set	Compares regex patterns against a specified request component
Size constraint	Checks size constraints against a specified request component
SQLi attack	Inspects for malicious SQL code in a specified request component
String match	Compares a string to a specified request component
XSS scripting attack	Inspects for cross-site scripting attacks in a specified request component

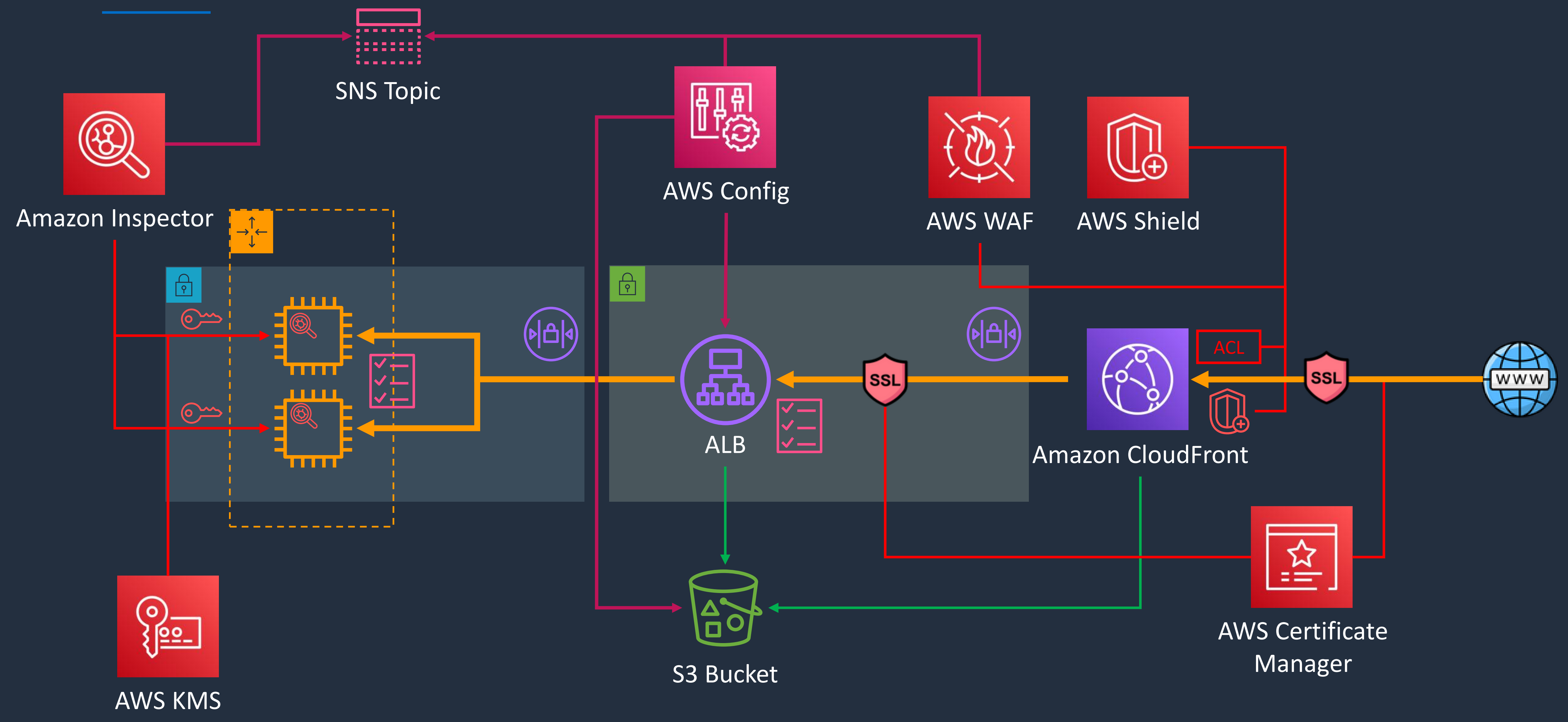
AWS Shield

AWS Shield

- AWS Shield is a managed Distributed Denial of Service (DDoS) protection service
- Safeguards web application running on AWS with always-on detection and automatic inline mitigations
- Helps to minimize application downtime and latency
- Two tiers –
  - **Standard** – no cost
  - **Advanced** - $3k USD per month and 1 year commitment
- Integrated with Amazon CloudFront (standard included by default)

Build a Secure Multi-Tier Architecture - Part 5

Build a Secure Multi-Tier Architecture



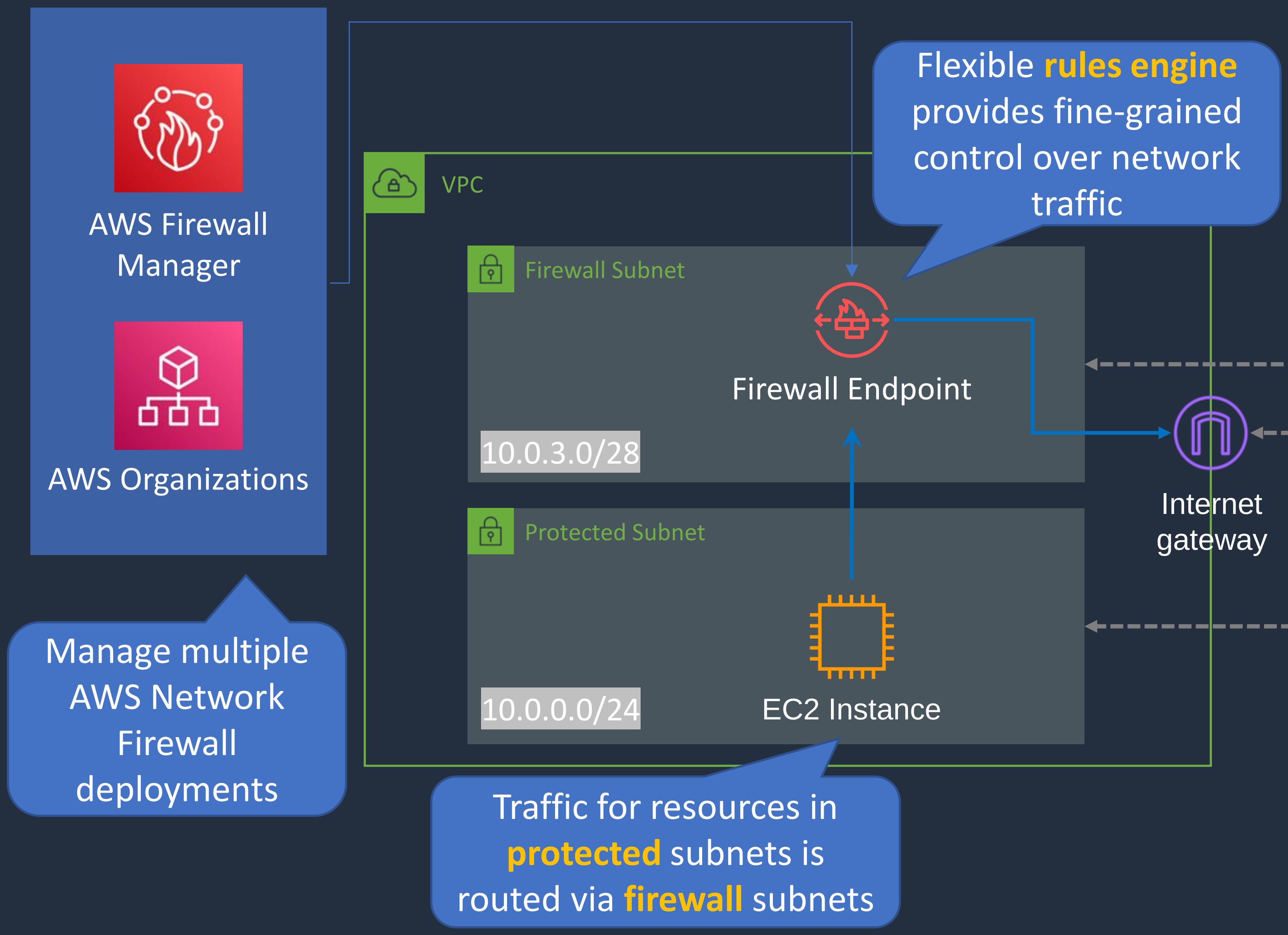
AWS GuardDuty

AWS GuardDuty

- Intelligent threat detection service
- Detects account compromise, instance compromise, malicious reconnaissance, and bucket compromise
- Continuous monitoring for events across:
  - **AWS CloudTrail Management Events**
  - **AWS CloudTrail S3 Data Events**
  - **Amazon VPC Flow Logs**
  - **DNS Logs**

Network Firewall and DNS Firewall

AWS Network Firewall



Firewall Subnet RT

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	igw-id

IGW Ingress RT

Destination	Target
10.0.0.0/16	Local
10.0.0.0/24	vpce-id-az-a

Protected Subnet RT

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	vpce-id-az-a

AWS Network Firewall

- Managed service for **VPC network protection**
- **Includes:**
  - Stateful & Stateless firewall
  - Intrusion Prevention System (IPS)
  - Web filtering
- Works with **AWS Network Firewall** manager for centrally applying policies across VPCs / accounts
- Uses a **VPC endpoint** and **Gateway Load Balancer**
- Do not deploy resources in the firewall subnet
- For HA, allocate a subnet per AZ

Route 53 Resolver DNS Firewall

- Filter and regulate outbound **DNS traffic** for **VPCs**
- Requests route through Route 53 Resolver for DNS
- Helps prevent DNS exfiltration of data
- Monitor and control the domains applications can query
- Can use AWS Firewall Manager to centrally configure and manage DNS Firewall
- Central management can span VPCs and accounts in AWS Organizations

AWS Audit Manager

AWS Audit Manager

- Continuously audit AWS usage to simplify risk assessment and compliance with regulations and industry standards
- Assesses whether your policies, procedures, and activities are operating effectively
- https://docs.aws.amazon.com/audit-manager/latest/userguide/what-is.html
- https://aws.amazon.com/blogs/security/continuous-compliance-monitoring-using-custom-audit-controls-and-frameworks-with-aws-audit-manager/

Architecture Patterns - Security

Architecture Patterns – Security

Requirement	Solution
Need to enable custom domain name and encryption in transit for an application running behind an Application Load Balancer	Use AWS Route 53 to create an Alias record to the ALB's DNS name and attach an SSL/TLS certificate issued by Amazon Certificate Manager (ACM)
Company records customer information in CSV files in an Amazon S3 bucket and must not store PII data	Use Amazon Macie to scan the S3 bucket for any PII data
For compliance reasons all S3 buckets must have encryption enabled and any non-compliant buckets must be auto remediated	Use AWS Config to check the encryption status of the buckets and use auto remediation to enable encryption as required

Architecture Patterns – Security

Requirement	Solution
EC2 instances must be checked against CIS benchmarks every 7 days	Install the Amazon Inspector agent and configure a host assessment every 7 days
Website running on EC2 instances behind and ALB must be protected against well known web exploits	Create a Web ACL in AWS WAF to protect against web exploits and attach to the ALB
Need to block access to an application running on an ALB from connections originating in a specific list of countries	Create a Web ACL in AWS WAF with a geographic match and block traffic that matches the list of countries

SECTION 18

Additional Services

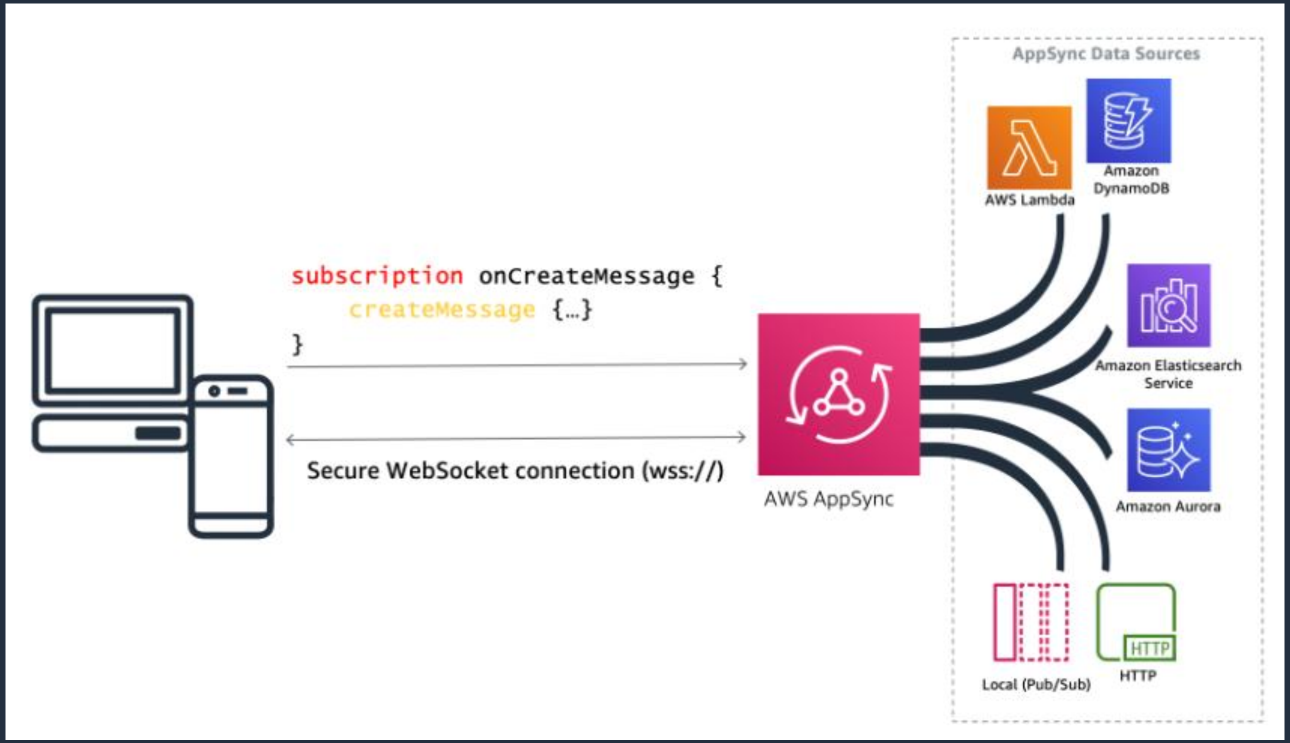
AWS Amplify and AppSync

AWS Amplify

- Tools and features for building full-stack applications on AWS
- Build web and mobile backends, and web frontend UIs
- AWS Amplify Studio is a visual interface for building web and mobile apps:
  - Use the visual interface to define a data model, user authentication, and file storage without backend expertise
  - Easily add AWS services not available within Amplify Studio using the AWS Cloud Development Kit (CDK)
  - Connect mobile and web apps using Amplify Libraries for iOS, Android, Flutter, React Native, and web (JavaScript)
- AWS Amplify Hosting is a fully managed CI/CD and hosting service for fast, secure, and reliable static and server-side rendered apps

AWS AppSync

- AWS AppSync is a fully managed service that makes it easy to develop GraphQL APIs
- Applications can securely access, manipulate, and receive real-time updates from multiple data sources such as databases or APIs

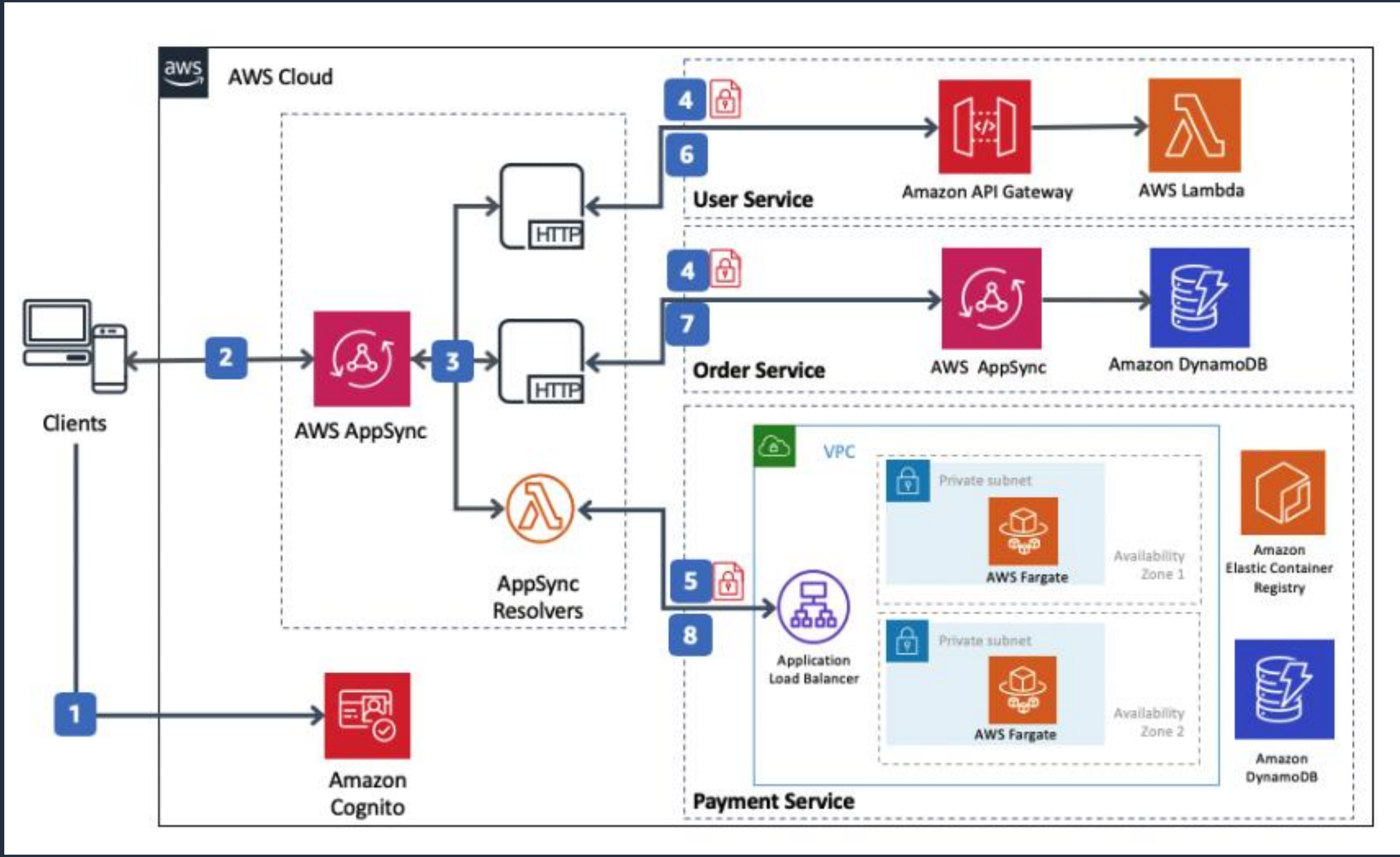


AWS AppSync

- AWS AppSync automatically scales a GraphQL API execution engine up and down to meet API request volumes
- Uses GraphQL, a data language that enables client apps to fetch, change and subscribe to data from servers
- AWS AppSync lets you specify which portions of your data should be available in a real-time manner using GraphQL Subscriptions
- AWS AppSync supports AWS Lambda, Amazon DynamoDB, and Amazon Elasticsearch
- Server-side data caching capabilities reduce the need to directly access data sources
- AppSync is fully managed and eliminates the operational overhead of managing cache clusters

AWS AppSync

Example of using **AppSync** and **Amplify** to simplify access to microservices



Amplify is used to build and host the WebStore application and create backend services

AppSync creates a unified API layer for integrating the microservices

AWS Device Farm

Internet of Things

The Internet of Things (IoT)

- Describes the network of physical objects that are embedded with sensors or software
- Each IoT device can communicate and exchange data with other devices and systems
- Use cases include:
  - Smart home automation
  - Smart healthcare
  - Manufacturing
  - Agriculture

The Internet of Things (IoT)

AWS IoT Core

- Lets you connect IoT devices to the AWS cloud without the need to provision or manage servers
- Can support billions of devices and trillions of messages

AWS IoT Device Management

- Helps you register, organize, monitor, and remotely manage IoT devices at scale

AWS IoT Device Defender

- Audit configurations, authenticate devices, detect anomalies, and receive alerts to help secure your IoT device fleet

The Internet of Things (IoT)

AWS IoT 1-Click

- Launch AWS Lambda functions from your simple devices
- Create actions in the cloud or on premises

AWS IoT Greengrass

- Open-source edge runtime and cloud service for building, deploying, and managing device software

AWS IoT Analytics

- Run analytics on IoT data and get insights to make better and more accurate decisions
- Supports up to petabytes of IoT data from millions of devices

The Internet of Things (IoT)

AWS IoT Events

- Detect and respond to events from IoT sensors and applications
- Ingest data from multiple sources to detect the state of your processes or devices, and proactively manage maintenance schedules

AWS IoT SiteWise

- Simplifies collecting, organizing, and analyzing industrial equipment data
- Organize sensor data streams from multiple production lines and facilities to drive efficiencies across locations

AWS IoT TwinMaker (formerly AWS IoT Things Graph)

- Create digital twins of real-world systems such as buildings, factories, industrial equipment, and production lines
- Quickly pinpoint and address equipment and process anomalies from the plant floor to improve worker productivity and efficiency

AWS Machine Learning Services

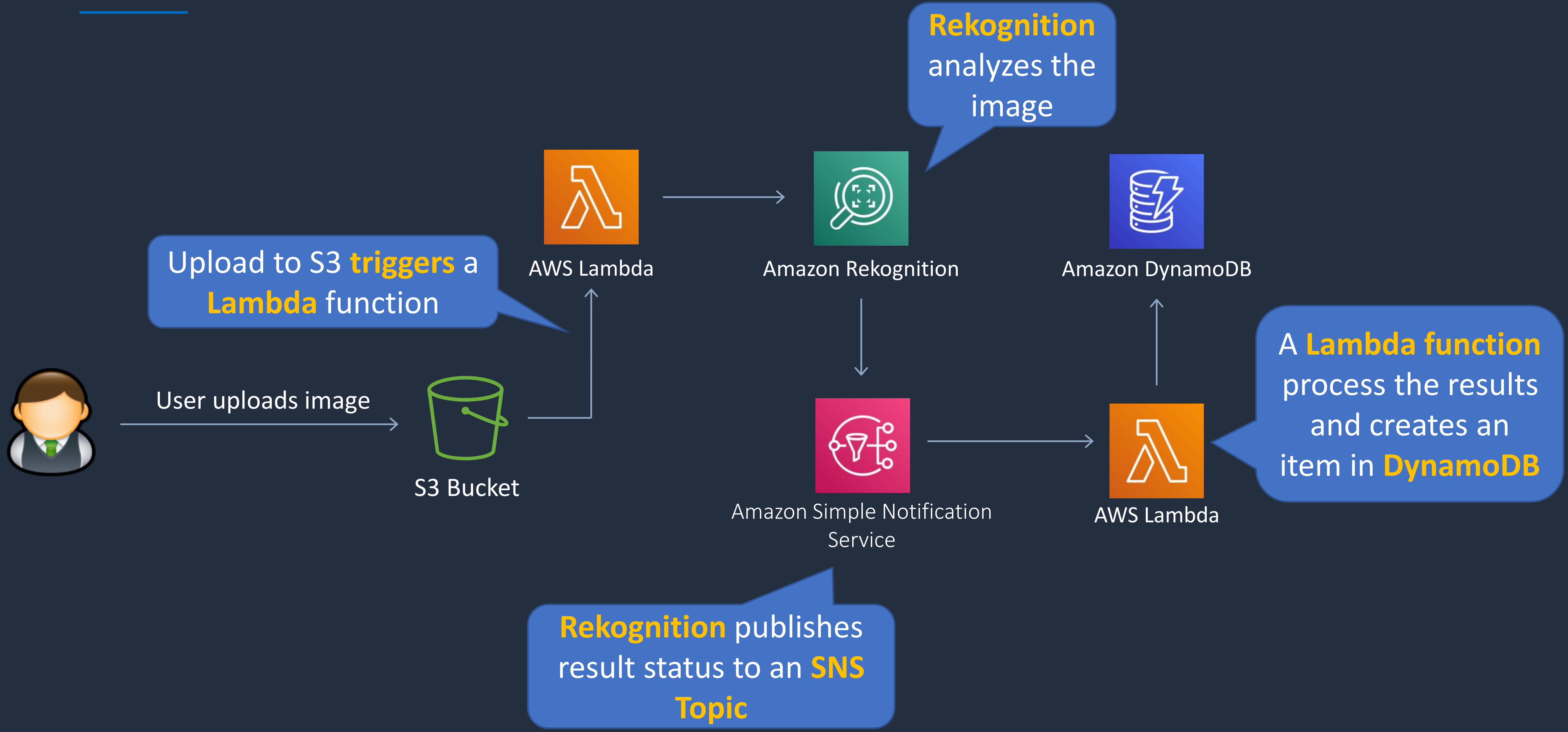
AWS Rekognition

Identify objects

Perform facial analysis

Celebrity recognition

AWS Rekognition in Event-Driven Architecture



AWS Rekognition

- Add image and video analysis to your applications
- Identify objects, people, text, scenes, and activities in images and videos
- Processes videos stored in an Amazon S3 bucket
- Publish completion status to Amazon SNS Topic

Amazon Transcribe

- Add speech to text capabilities to applications
- Recorded speech can be converted to text before it can be used in applications
- Uses a deep learning process called automatic speech recognition (ASR) to convert speech to text quickly and accurately

Amazon Translate

- Neural machine translation service that delivers fast, high-quality, and affordable language translation
- Uses deep learning models to deliver more accurate and more natural sounding translation
- Localize content such as websites and applications for your diverse users

Amazon Textract

- Automatically extract printed text, handwriting, and data from any document
- Features:
  - Optical character recognition (OCR)
  - Identifies relationships, structure, and text
  - Uses AI to extract text and structured data
  - Recognizes handwriting as well as printed text
  - Can extract from documents such as PDFs, images, forms, and tables
  - Understands context. For example know what data to extract from a receipt or invoice

Amazon SageMaker

- Helps data scientists and developers to prepare, build, train, and deploy high-quality machine learning (ML) models
- ML development activities including:
  - Data preparation
  - Feature engineering
  - Statistical bias detection
  - Auto-ML
  - Training and tuning
  - Hosting
  - Monitoring
  - Workflows

Amazon Comprehend

- Natural-language processing (NLP) service
- Uses machine learning to uncover information in unstructured data
- Can identify critical elements in data, including references to language, people, and places, and the text files can be categorized by relevant topics
- In real time, you can automatically and accurately detect customer sentiment in your content

Amazon Lex

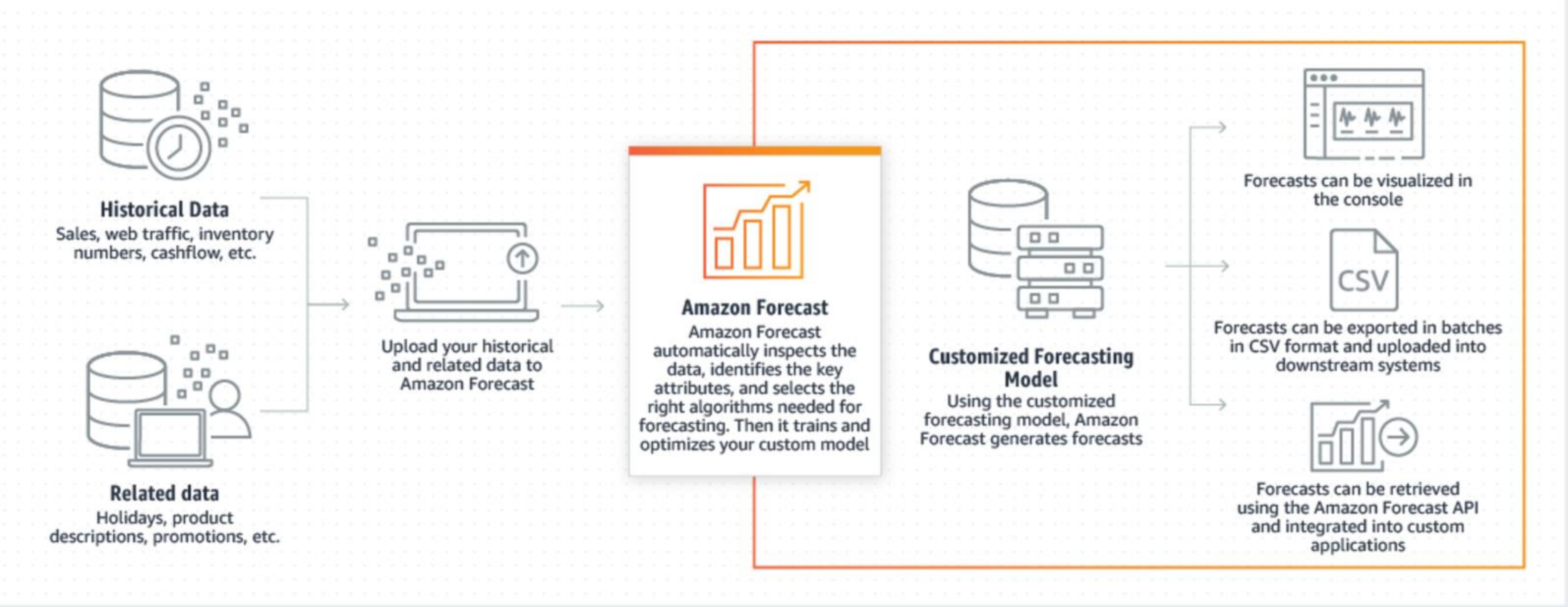
- Conversational AI for Chatbots
- Build conversational interfaces into any application using voice and text
- Build bots to increase contact center productivity, automate simple tasks, and drive operational efficiencies across the enterprise

Amazon Polly

- Turns text into lifelike speech
- Create applications that talk, and build entirely new categories of speech-enabled products
- Text-to-Speech (TTS) service uses advanced deep learning technologies to synthesize natural sounding human speech

Amazon Forecast

- Time-series forecasting service
- Uses ML and is built for business metrics analysis



Amazon DevOps Guru

- Cloud operations service for improving **application operational performance and availability**
- Detect behaviors that deviate from normal operating patterns
- Benefits:
  - Automatically detect operational issues
  - Resolve issues with ML-powered insights
  - Elastically scale operational analytics
  - Uses ML to reduce alarm noise

Transcode and Transcribe Video

Amazon Elastic Transcoder

- Transcodes video files to various formats / outputs
- AWS Elemental MediaConvert is a new service that provides more functionality and may be better for new use cases
- Some features are only available on Elastic Transcoder such as:
  - WebM (VP8/VP9) input and output
  - Animated GIF output
  - MP3, FLAC, Vorbis, and WAV audio-only output
- However, MediaConvert offers more options overall

AWS License Manager

AWS License Manager

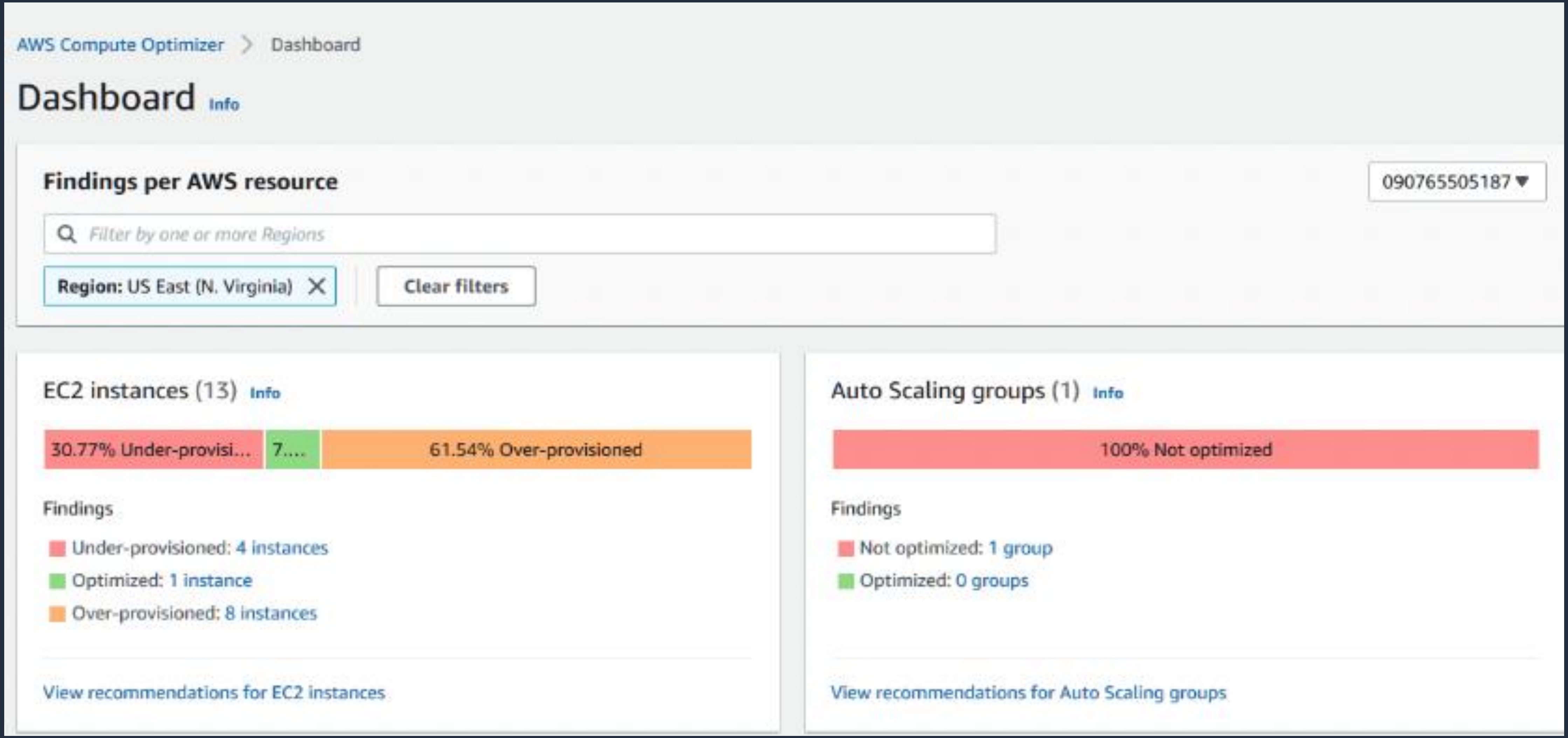
- Used to manage licenses from software vendors
- For example, manage your Microsoft, Oracle, SAP and IBM licenses
- Centralized management of software licenses for AWS and on-premises resources
- Can track license usage including when licensed based on virtual cores (vCPUs), sockets, or number of machines
- Distribute, activate, and track software licenses across accounts and throughout an organization
- Enforce limits across multiple Regions

AWS Compute Optimizer

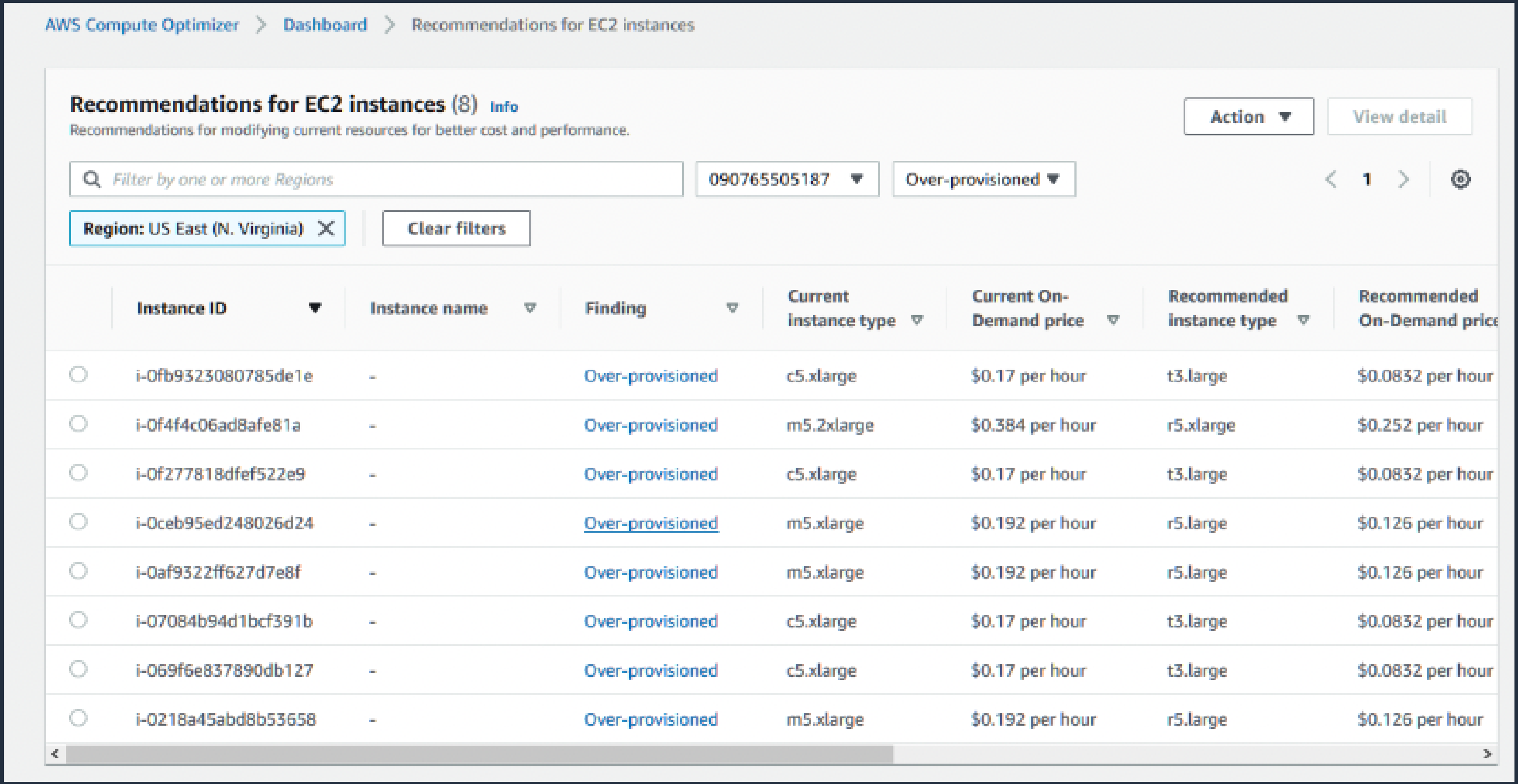
AWS Compute Optimizer

- Recommends optimal AWS resources for your workloads to reduce costs and improve performance
- Uses machine learning to analyze historical utilization metrics
- Offers optimization guidance for:
  - Amazon EC2 instances
  - Amazon EBS volumes
  - AWS Lambda functions
- Results can be viewed in the console or via the CLI

AWS Compute Optimizer



AWS Compute Optimizer



AWS Compute Optimizer > Dashboard > Recommendations for EC2 instances

Recommendations for EC2 instances (8) Info

Recommendations for modifying current resources for better cost and performance.

Action ▼ | View detail

Filter by one or more Regions | 090765505187 ▼ | Over-provisioned ▼ | < 1 >

Region: US East (N. Virginia) ✕ | Clear filters

	Instance ID	Instance name	Finding	Current instance type	Current On-Demand price	Recommended instance type	Recommended On-Demand price
○	i-0fb9323080785de1e	-	Over-provisioned	c5.xlarge	$0.17 per hour	t3.large	$0.0832 per hour
○	i-0f4f4c06ad8afe81a	-	Over-provisioned	m5.2xlarge	$0.384 per hour	r5.xlarge	$0.252 per hour
○	i-0f277818dfef522e9	-	Over-provisioned	c5.xlarge	$0.17 per hour	t3.large	$0.0832 per hour
○	i-0ceb95ed248026d24	-	Over-provisioned	m5.xlarge	$0.192 per hour	r5.large	$0.126 per hour
○	i-0af9322ff627d7e8f	-	Over-provisioned	m5.xlarge	$0.192 per hour	r5.large	$0.126 per hour
○	i-07084b94d1bcf391b	-	Over-provisioned	c5.xlarge	$0.17 per hour	t3.large	$0.0832 per hour
○	i-069f6e837890db127	-	Over-provisioned	c5.xlarge	$0.17 per hour	t3.large	$0.0832 per hour
○	i-0218a45abd8b53658	-	Over-provisioned	m5.xlarge	$0.192 per hour	r5.large	$0.126 per hour

AWS Cost Explorer

AWS Cost Allocation Tags

DEMO

AWS Cost Management Tools

AWS Cost Explorer

- The **AWS Cost Explorer** is a free tool that allows you to view charts of your costs
- You can view cost data for the past 13 months and forecast how much you are likely to spend over the next three months
- Cost Explorer can be used to discover patterns in how much you spend on AWS resources over time and to identify cost problem areas
- Cost Explorer can help you to identify service usage statistics such as:
  - Which services you use the most
  - View metrics for which AZ has the most traffic
  - Which linked account is used the most

AWS Cost & Usage Report

- Publish AWS billing reports to an Amazon S3 bucket
- Reports break down costs by:
  - Hour, day, month, product, product resource, tags
- Can update the report up to three times a day
- Create, retrieve, and delete your reports using the AWS CUR API Reference

AWS Price List API

- Query the prices of AWS services
- **Price List Service API** (AKA the Query API) – query with JSON
- **AWS Price List API** (AKA the Bulk API) – query with HTML
- Alerts via Amazon SNS when prices change